

axiomTM



The 30 Year Horizon

<i>Manuel Bronstein</i>	<i>William Burge</i>	<i>Timothy Daly</i>
<i>James Davenport</i>	<i>Michael Dewar</i>	<i>Martin Dunstan</i>
<i>Albrecht Fortenbacher</i>	<i>Patrizia Gianni</i>	<i>Johannes Grabmeier</i>
<i>Jocelyn Guidry</i>	<i>Richard Jenks</i>	<i>Larry Lambe</i>
<i>Michael Monagan</i>	<i>Scott Morrison</i>	<i>William Sit</i>
<i>Jonathan Steinbach</i>	<i>Robert Sutor</i>	<i>Barry Trager</i>
<i>Stephen Watt</i>	<i>Jim Wen</i>	<i>Clifton Williamson</i>

Volume 9: Axiom Compiler

Portions Copyright (c) 2005 Timothy Daly

The Blue Bayou image Copyright (c) 2004 Jocelyn Guidry

Portions Copyright (c) 2004 Martin Dunstan

Portions Copyright (c) 2007 Alfredo Portes

Portions Copyright (c) 2007 Arthur Ralfs

Portions Copyright (c) 2005 Timothy Daly

Portions Copyright (c) 1991-2002,
The Numerical ALgorithms Group Ltd.
All rights reserved.

This book and the Axiom software is licensed as follows:

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of The Numerical ALgorithms Group Ltd. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Inclusion of names in the list of credits is based on historical information and is as accurate as possible. Inclusion of names does not in any way imply an endorsement but represents historical influence on Axiom development.

Michael Albaugh	Cyril Alberga	Roy Adler
Christian Aistleitner	Richard Anderson	George Andrews
S.J. Atkins	Jeremy Avigad	Henry Baker
Martin Baker	Stephen Balzac	Yurij Baransky
David R. Barton	Thomas Baruchel	Gerald Baumgartner
Gilbert Baumslag	Michael Becker	Nelson H. F. Beebe
Jay Belanger	David Bindel	Fred Blair
Vladimir Bondarenko	Mark Botch	Raoul Bourquin
Alexandre Bouyer	Karen Braman	Wolfgang Brehm
Peter A. Broadbery	Martin Brock	Manuel Bronstein
Stephen Buchwald	Florian Bundschuh	Luanne Burns
William Burge	Ralph Byers	Quentin Carpent
Robert Cavines	Bruce Char	Ondrej Certik
Tzu-Yi Chen	Bobby Cheng	Cheekai Chin
David V. Chudnovsky	Gregory V. Chudnovsky	Mark Clements
James Cloos	Jia Zhao Cong	Josh Cohen
Christophe Conil	Don Coppersmith	George Corliss
Robert Corless	Gary Cornell	Meino Cramer
Jeremy Du Croz	David Cyganski	Nathaniel Daly
Timothy Daly Sr.	Timothy Daly Jr.	James H. Davenport
David Day	James Demmel	Didier Deshommes
Michael Dewar	Inderjit Dhillon	Jack Dongarra
Jean Della Dora	Gabriel Dos Reis	Claire DiCrescendo
Sam Dooley	Zlatko Drmac	Lionel Ducos
Iain Duff	Lee Duhem	Martin Dunstan
Brian Dupee	Dominique Duval	Robert Edwards
Heow Eide-Goodman	Lars Erickson	Mark Fahey
Richard Fateman	Bertfried Fauser	Stuart Feldman
John Fletcher	Brian Ford	Albrecht Fortenbacher
George Frances	Constantine Frangos	Timothy Freeman
Korrinn Fu	Marc Gaetano	Rudiger Gebauer
Van de Geijn	Kathy Gerber	Patricia Gianni
Gustavo Goertkin	Samantha Goldrich	Holger Gollan
Teresa Gomez-Diaz	Laureano Gonzalez-Vega	Stephen Gortler
Johannes Grabmeier	Matt Grayson	Klaus Ebbe Grue
James Griesmer	Vladimir Grinberg	Oswald Gschnitzer
Ming Gu	Jocelyn Guidry	Gaetan Hache
Steve Hague	Satoshi Hamaguchi	Sven Hammarling
Mike Hansen	Richard Hanson	Richard Harke
Bill Hart	Vilya Harvey	Martin Hassner
Arthur S. Hathaway	Dan Hatton	Waldek Hebisch
Karl Hegbloom	Ralf Hemmecke	Henderson
Antoine Hersen	Nicholas J. Higham	Roger House
Gernot Hueber	Pietro Iglio	Alejandro Jakubi
Richard Jenks	Bo Kagstrom	William Kahan
Kyriakos Kalorkoti	Kai Kaminski	Grant Keady
Wilfrid Kendall	Tony Kennedy	David Kincaid

Ted Kosan	Paul Kosinski	Igor Kozachenko
Fred Krogh	Klaus Kusche	Bernhard Kutzler
Tim Lahey	Larry Lambe	Kaj Laurson
Charles Lawson	George L. Legendre	Franz Lehner
Frederic Lehobey	Michel Levaud	Howard Levy
J. Lewis	Ren-Cang Li	Rudiger Loos
Craig Lucas	Michael Lucks	Richard Luczak
Camm Maguire	Francois Maltey	Osni Marques
Alasdair McAndrew	Bob McElrath	Michael McGettrick
Edi Meier	Ian Meikle	David Mentre
Victor S. Miller	Gerard Milmeister	Mohammed Mobarak
H. Michael Moeller	Michael Monagan	Marc Moreno-Maza
Scott Morrison	Joel Moses	Mark Murray
William Naylor	Patrice Naudin	C. Andrew Neff
John Nelder	Godfrey Nolan	Arthur Norman
Jinzhong Niu	Michael O'Connor	Summat Oemrawsingh
Kostas Oikonomou	Humberto Ortiz-Zuazaga	Julian A. Padget
Bill Page	David Parnas	Susan Pelzel
Michel Petitot	Didier Pinchon	Ayal Pinkus
Frederick H. Pitts	Jose Alfredo Portes	E. Quintana-Orti
Gregorio Quintana-Orti	Beresford Parlett	A. Petitot
Peter Poromaa	Claude Quitte	Arthur C. Ralfs
Norman Ramsey	Anatoly Raportirenko	Guilherme Reis
Huan Ren	Albert D. Rich	Michael Richardson
Jason Riedy	Renaud Rioboo	Jean Rivlin
Nicolas Robidoux	Simon Robinson	Raymond Rogers
Michael Rothstein	Martin Rubey	Jeff Rutter
Philip Santas	Alfred Scheerhorn	William Schelter
Gerhard Schneider	Martin Schoenert	Marshall Schor
Frithjof Schulze	Fritz Schwarz	Steven Segletes
V. Sima	Nick Simicich	William Sit
Elena Smirnova	Jacob Nyffeler Smith	Ken Stanley
Jonathan Steinbach	Fabio Stumbo	Christine Sundaresan
Robert Sutor	Moss E. Sweedler	Eugene Surowitz
Max Tegmark	T. Doug Telford	James Thatcher
Laurent Thery	Balbir Thomas	Mike Thomas
Dylan Thurston	Francoise Tisseur	Steve Toleque
Raymond Toy	Barry Trager	Themos T. Tsikas
Gregory Vanuxem	Kresimir Veselic	Christof Voemel
Bernhard Wall	Stephen Watt	Jaap Weel
Juergen Weiss	M. Weller	Mark Wegman
James Wen	Thorsten Werther	Michael Wester
R. Clint Whaley	James T. Wheeler	John M. Wiley
Berhard Will	Clifton J. Williamson	Stephen Wilson
Shmuel Winograd	Robert Wisbauer	Sandra Wityak
Waldemar Wiwianka	Knut Wolf	Yanyang Xiao
Liu Xiaojun	Clifford Yapp	David Yun
Qian Yun	Vadim Zhytnikov	Richard Zippel
Evelyn Zoernack	Bruno Zuercher	Dan Zwillinger

Contents

1	The Axiom Compiler	1
1.1	Makefile	1
2	Overview	3
2.1	Syntax by Jacob Smith	5
	Language features	8
	Semantics	8
2.2	The Input	11
2.3	The Output, the EQ.nrlib directory	15
2.4	The code.lsp and EQ.lsp files	16
2.5	The code.o file	30
2.6	The info file	30
2.7	The EQ.fn file	33
2.8	The index.kaf file	38
	The index offset byte	40
	The “loadTimeStuff”	40
	The “compilerInfo”	42
	The “constructorForm”	49
	The “constructorKind”	49
	The “constructorModemap”	49
	The “constructorCategory”	51
	The “sourceFile”	52
	The “modemaps”	52
	The “operationAlist”	54
	The “superDomain”	56
	The “signaturesAndLocals”	56
	The “attributes”	56
	The “predicates”	57
	The “abbreviation”	58
	The “parents”	58
	The “ancestors”	58
	The “documentation”	58
	The “slotInfo”	60
	The “index”	62

3	Compiler top level	63
3.1	Global Data Structures	63
3.2	Pratt Parsing	63
3.3	)compile	64
	Spad compiler	66
3.4	Operator Precedence Table Initialization	67
	LED and NUD Tables	68
3.5	Gliph Table	70
	Rename Token Table	71
	Generic function table	71
3.6	Giant steps, Baby steps	72
4	The Parser	73
4.1	EQ.spad	73
4.2	boot transformations	77
	defun string2BootTree	77
	defun new2OldLisp	78
	defun new2OldTran	78
	defun newIf2Cond	79
	defun newDef2Def	80
	defun new2OldDefForm	80
	defun newConstruct	80
4.3	preparse	81
	defvar \$index	81
	defvar \$linelist	81
	defvar \$echolinestack	81
	defvar \$preparse-last-line	82
4.4	Parsing routines	82
	defun initialize-preparse	82
	defun preparse	86
	defun Build the lines from the input for piles	87
	defun skip-ifblock	92
	defun preparseReadLine1	93
	defun expand-tabs	94
4.5	I/O Handling	95
	defun preparse-echo	95
	Parsing stack	95
	defstruct stack	95
	defun stack-load	95
	defun stack-clear	96
	defmacro stack-/empty	96
	defun stack-push	96
	defun stack-pop	97
	Parsing token	97
	defstruct token	97
	defvar prior-token	97

defvar nonblank	97
defvar current-token	98
defvar next-token	98
defvar valid-tokens	98
defun token-install	98
defun token-print	99
Parsing reduction	99
defstruct reduction	99
5 Parse Transformers	101
5.1 Direct called parse routines	101
defun parseTransform	101
defun parseTran	101
defun parseAtom	102
defun parseTranList	103
defplist parseConstruct	103
defun parseConstruct	103
5.2 Indirect called parse routines	104
defplist parseAnd	105
defun parseAnd	105
defplist parseAtSign	105
defun parseAtSign	106
defun parseType	106
defplist parseCategory	106
defun parseCategory	107
defun parseDropAssertions	107
defplist parseCoerce	107
defun parseCoerce	108
defplist parseColon	108
defun parseColon	108
defplist parseDEF	109
defun parseDEF	109
defun parseLhs	110
defun transIs	110
defun transIs1	110
defun isListConstructor	111
defplist parseDollarGreaterthan	111
defun parseDollarGreaterThan	112
defplist parseDollarGreaterEqual	112
defun parseDollarGreaterEqual	112
defun parseDollarLessEqual	113
defplist parseDollarNotEqual	113
defun parseDollarNotEqual	113
defplist parseEquivalence	114
defun parseEquivalence	114
defplist parseExit	114

defun parseExit	114
defplist parseGreaterEqual	115
defun parseGreaterEqual	115
defplist parseGreaterThan	115
defun parseGreaterThan	116
defplist parseHas	116
defun parseHas	116
defun parseHasRhs	118
defun loadIfNecessary	118
defun loadLibIfNecessary	119
defun updateCategoryFrameForConstructor	120
defun convertOpAlist2compilerInfo	120
defun updateCategoryFrameForCategory	121
defplist parseIf	121
defun parseIf	122
defun parseIf,ifTran	122
defplist parseImplies	124
defun parseImplies	124
defplist parseIn	125
defun parseIn	125
defplist parseInBy	126
defun parseInBy	126
defplist parseIs	127
defun parseIs	127
defplist parseIsnt	127
defun parseIsnt	127
defplist parseJoin	128
defun parseJoin	128
defplist parseLeave	128
defun parseLeave	129
defplist parseLessEqual	129
defun parseLessEqual	129
defplist parseLET	130
defun parseLET	130
defplist parseLETD	130
defun parseLETD	131
defplist parseMDEF	131
defun parseMDEF	131
defplist parseNot	132
defplist parseNot	132
defun parseNot	132
defplist parseNotEqual	133
defun parseNotEqual	133
defplist parseOr	133
defun parseOr	133
defplist parsePretend	134

defun parsePretend	134
defplist parseReturn	135
defun parseReturn	135
defplist parseSegment	135
defun parseSegment	135
defplist parseSeq	136
defun parseSeq	136
defplist parseVCONS	136
defun parseVCONS	137
defplist parseWhere	137
defun parseWhere	137
6 Compile Transformers	139
defun compExpression	139
6.1 Handline Category DEF forms	142
defplist compDefine plist	144
defun compDefine	144
defun compDefine1	145
defun compDefineAddSignature	147
defun compDefineFunctor	148
defun compDefineFunctor1	148
defun compDefineCapsuleFunction	155
defun compInternalFunction	159
defun compDefWhereClause	159
defun compDefineCategory	162
defun compDefineCategory1	162
defun compDefineCategory2	163
defun compDefineLisplib	167
defun compileDocumentation	169
defun compArgumentConditions	170
defun compileCases	171
defun compFunctorBody	172
defun compile	173
defvar \$NoValueMode	176
defvar \$EmptyMode	176
defun hasFullSignature	176
defun addEmptyCapsuleIfNecessary	177
defun getTargetFromRhs	177
defun giveFormalParametersValues	178
defun macroExpandInPlace	178
defun macroExpand	178
defun macroExpandList	179
defun makeCategoryPredicates	179
defun mkCategoryPackage	180
defun mkEvaluableCategoryForm	182
defun encodeFunctionName	183

defun mkRepetitionAssoc	184
defun splitEncodedFunctionName	184
defun encodeItem	185
defun getCaps	185
defun constructMacro	186
defun spadCompileOrSetq	186
defun compileConstructor	187
defun compileConstructor1	188
defun compAndDefine	189
defun putInLocalDomainReferences	189
defun NRTputInTail	189
defun NRTputInHead	190
defun getArgumentModeOrMoan	191
defun augLisplibModemapsFromCategory	191
defun mkAlistOfExplicitCategoryOps	193
defun flattenSignatureList	194
defun interactiveModemapForm	195
defun replaceVars	196
defun fixUpPredicate	196
defun orderPredicateItems	197
defun signatureTran	197
defun orderPredTran	198
defun isDomainSubst	200
defun moveORsOutside	201
defun substVars	202
defun modemapPattern	203
defun evalAndRwriteLispForm	204
defun rwriteLispForm	204
defun mkConstructor	205
defun unloadOneConstructor	205
defun lisplibDoRename	205
defun initializeLisplib	206
defun writeLib1	207
defun finalizeLisplib	207
defun getConstructorOpsAndAtts	209
defun getCategoryOpsAndAtts	210
defun getSlotFromCategoryForm	210
defun transformOperationAlist	210
defun getFunctorOpsAndAtts	212
defun getSlotFromFunctor	212
defun compMakeCategoryObject	213
defun mergeSignatureAndLocalVarAlists	213
defun lisplibWrite	214
defun isCategoryPackageName	214
defun NRTgetLookupFunction	214
defun NRTgetLocalIndex	215

	defun augmentLisplibModemapsFromFunctor	216
	defun allLASSOCs	218
	defun formal2Pattern	218
	defun mkDatabasePred	218
	defun disallowNilAttribute	219
	defun bootStrapError	219
	defun reportOnFunctorCompilation	219
	defun displayMissingFunctions	220
	defun makeFunctorArgumentParameters	221
	defun genDomainViewList0	223
	defun genDomainViewList	223
	defun genDomainView	224
	defun genDomainOps	224
	defun mkOpVec	225
	defun AssocBarGensym	226
	defun orderByDependency	227
6.2	Code optimization routines	228
	defun optimizeFunctionDef	228
	defun optimize	229
	defun optXLAMCond	230
	defun optCONDtail	231
	defvar \$BasicPredicates	231
	defun optPredicateIfTrue	231
	defun optIF2COND	232
	defun subrname	232
	Special case optimizers	233
	defplist optCall	233
	defun Optimize “call” expressions	233
	defun optPackageCall	235
	defun optCallSpecially	235
	defun optSpecialCall	236
	defun compileTimeBindingOf	237
	defun optCallEval	237
	defplist optSEQ	238
	defun optSEQ	238
	defplist optEQ	240
	defun optEQ	240
	defplist optMINUS	240
	defun optMINUS	240
	defplist optQSMINUS	241
	defun optQSMINUS	241
	defplist opt-	241
	defun opt-	242
	defplist optLESSP	242
	defun optLESSP	242
	defplist optSPADCALL	243

defun optSPADCALL	243
defplist optSuchthat	244
defun optSuchthat	244
defplist optCatch	244
defun optCatch	245
defplist optCond	246
defun optCond	246
defun EqualBarGensym	248
defplist optMkRecord	249
defun optMkRecord	249
defplist optRECORDELT	249
defun optRECORDELT	250
defplist optSETRECORDELT	250
defun optSETRECORDELT	251
defplist optRECORDCOPY	251
defun optRECORDCOPY	251
6.3 Functions to manipulate modemaps	252
defun addDomain	252
defun unknownTypeError	253
defun isFunctor	253
defun getDomainsInScope	254
defun putDomainsInScope	255
defun isSuperDomain	255
defun addNewDomain	256
defun augModemapsFromDomain	256
defun augModemapsFromDomain1	257
defun substituteCategoryArguments	257
defun addConstructorModemaps	258
defun getModemap	258
defun compApplyModemap	259
defun compMapCond	260
defun compMapCond'	261
defun compMapCond''	261
defun compMapCondFun	262
defun getUniqueSignature	263
defun getUniqueModemap	263
defun getModemapList	263
defun getModemapListFromDomain	264
defun domainMember	264
defun augModemapsFromCategory	265
defun addEltModemap	265
defun mkNewModemapList	266
defun insertModemap	267
defun mergeModemap	267
defun TruthP	268
defun evalAndSub	269

	defun getOperationAlist	269
	defvar \$FormalMapVariableList	270
	defun substNames	270
	defun augModemapsFromCategoryRep	271
6.4	Maintaining Modemaps	272
	defun addModemapKnown	272
	defun addModemap	273
	defun addModemap0	274
	defun addModemap1	274
6.5	Indirect called comp routines	275
	defplist compAdd plist	275
	defun compAdd	275
	defun compTuple2Record	277
	defplist compCapsule plist	277
	defun compCapsule	278
	defun compCapsuleInner	278
	defun processFunctor	279
	defun compCapsuleItems	279
	defun compSingleCapsuleItem	280
	defun doIt	281
	defun doItIf	285
	defun isMacro	286
	defplist compCase plist	287
	defun compCase	287
	defun compCase1	288
	defplist compCat plist	288
	defplist compCat plist	289
	defplist compCat plist	289
	defun compCat	289
	defplist compCategory plist	290
	defun compCategory	290
	defun compCategoryItem	291
	defun mkExplicitCategoryFunction	292
	defun mustInstantiate	293
	defun wrapDomainSub	294
	defplist compColon plist	294
	defun compColon	294
	defun makeCategoryForm	297
	defplist compCons plist	298
	defun compCons	298
	defun compCons1	298
	defplist compConstruct plist	299
	defun compConstruct	299
	defplist compConstructorCategory plist	300
	defplist compConstructorCategory plist	300
	defplist compConstructorCategory plist	301

defplist compConstructorCategory plist	301
defun compConstructorCategory	301
defun getAbbreviation	302
defun mkAbbrev	302
defun addSuffix	303
defun alistSize	303
defun getSignatureFromMode	303
defun getSpecialCaseAssoc	304
defun addArgumentConditions	304
defun stripOffSubdomainConditions	305
defun stripOffArgumentConditions	306
defun getSignature	306
defun checkAndDeclare	308
defun hasSigInTargetCategory	308
defun getArgumentMode	309
defplist compElt plist	310
defun compElt	310
defplist compExit plist	311
defun compExit	312
defplist compHas plist	312
defun compHas	313
defun compHasFormat	313
defun mkList	314
defplist compIf plist	314
defun compIf	315
defun compFromIf	316
defun canReturn	316
defun compBoolean	318
defun getSuccessEnvironment	318
defun getInverseEnvironment	320
defun getUnionMode	321
defun isUnionMode	321
defplist compImport plist	322
defun compImport	322
defplist compIs plist	322
defun compIs	322
defplist compJoin plist	323
defun compJoin	323
defun compForMode	325
defplist compLambda plist	325
defun compLambda	325
defplist compLeave plist	326
defun compLeave	327
defplist compMacro plist	327
defun compMacro	327
defplist compPretend plist	328

defun compPretend	328
defplist compQuote plist	329
defun compQuote	330
defplist compReduce plist	330
defun compReduce	330
defun compReduce1	330
defplist compRepeatOrCollect plist	332
defplist compRepeatOrCollect plist	332
defun compRepeatOrCollect	333
defplist compReturn plist	335
defun compReturn	335
defplist compSeq plist	336
defun compSeq	336
defun compSeq1	336
defun replaceExitEtc	337
defun convertOrCroak	338
defun compSeqItem	338
defplist compSetq plist	339
defplist compSetq plist	339
defun compSetq	339
defun compSetq1	339
defun uncons	340
defun setqMultiple	341
defun setqMultipleExplicit	343
defun setqSetelt	344
defun setqSingle	344
defun NRTassocIndex	346
defun assignError	346
defun outputComp	347
defun maxSuperType	348
defun isDomainForm	348
defun isDomainConstructorForm	348
defplist compString plist	349
defun compString	349
defplist compSubDomain plist	350
defun compSubDomain	350
defun compSubDomain1	350
defun lispize	351
defplist compSubsetCategory plist	352
defun compSubsetCategory	352
defplist compSuchthat plist	352
defun compSuchthat	353
defplist compVector plist	353
defun compVector	353
defplist compWhere plist	354
defun compWhere	354

6.6	Functions for coercion	355
	defun coerce	355
	defun coerceEasy	356
	defun coerceSubset	357
	defun coerceHard	358
	defun coerceExtraHard	359
	defun hasType	360
	defun coerceable	360
	defun coerceExit	361
	defplist compAtSign plist	361
	defun compAtSign	361
	defplist compCoerce plist	362
	defun compCoerce	362
	defun compCoerce1	363
	defun coerceByModemap	363
	defun autoCoerceByModemap	364
	defun resolve	365
	defun mkUnion	366
	defun This orders Unions	366
	defun modeEqualSubst	367
7	Post Transformers	369
7.1	Direct called postparse routines	369
	defun postTransform	369
	defun postTran	370
	defun postOp	371
	defun postAtom	371
	defun postTranList	372
	defun postScriptsForm	372
	defun postTranScripts	372
	defun postTransformCheck	373
	defun postcheck	373
	defun postError	374
	defun postForm	374
7.2	Indirect called postparse routines	375
	defplist postAdd plist	376
	defun postAdd	376
	defun postCapsule	377
	defun postBlockItemList	377
	defun postBlockItem	377
	defplist postAtSign plist	378
	defun postAtSign	378
	defun postType	379
	defplist postBigFloat plist	379
	defun postBigFloat	380
	defplist postBlock plist	380

defun postBlock	380
defplist postCategory plist	381
defun postCategory	381
defun postCollect,finish	381
defun postMakeCons	382
defplist postCollect plist	383
defun postCollect	383
defun postIteratorList	384
defplist postColon plist	384
defun postColon	385
defplist postColonColon plist	385
defun postColonColon	385
defplist postComma plist	386
defun postComma	386
defun comma2Tuple	386
defun postFlatten	386
defplist postConstruct plist	387
defun postConstruct	387
defun postTranSegment	388
defplist postDef plist	388
defun postDef	388
defun postDefArgs	390
defplist postExit plist	390
defun postExit	391
defplist postIf plist	391
defun postIf	391
defplist postIn plist	392
defun postIn	392
defun postInSeq	392
defplist postIn plist	393
defun postIn	393
defplist postJoin plist	393
defun postJoin	394
defplist postMapping plist	394
defun postMapping	394
defplist postMDef plist	395
defun postMDef	395
defplist postPretend plist	396
defun postPretend	396
defplist postQUOTE plist	396
defun postQUOTE	397
defplist postReduce plist	397
defun postReduce	397
defplist postRepeat plist	398
defun postRepeat	398
defplist postScripts plist	398

	defun postScripts	398
	defplist postSemiColon plist	399
	defun postSemiColon	399
	defun postFlattenLeft	399
	defplist postSignature plist	400
	defun postSignature	400
	defun removeSuperfluousMapping	400
	defun killColons	401
	defplist postSlash plist	401
	defun postSlash	401
	defplist postTuple plist	402
	defun postTuple	402
	defplist postTupleCollect plist	402
	defun postTupleCollect	402
	defplist postWhere plist	403
	defun postWhere	403
	defplist postWith plist	403
	defun postWith	404
7.3	Support routines	404
	defun setDefOp	404
	defun aplTran	405
	defun aplTran1	405
	defun aplTranList	407
	defun hasAplExtension	407
	defun deepestExpression	408
	defun containsBang	408
	defun getScriptName	408
	defun decodeScripts	409
8	DEF forms	411
	defvar \$defstack	411
	defvar \$is-spill	411
	defvar \$is-spill-list	411
	defvar \$v1	412
	defvar \$is-gensymlist	412
	defvar initial-gensym	412
	defvar \$is-eqlist	412
	defun hackforis	412
	defun hackforis1	413
	defun unTuple	413
8.1	The PARSE code	413
	defvar tmptok	413
	defvar tok	414
	defvar ParseMode	414
	defvar definition-name	414
	defvar lablasoc	414

defun PARSE-NewExpr	414
defun PARSE-Command	415
defun PARSE-SpecialKeyWord	415
defun PARSE-SpecialCommand	416
defun PARSE-TokenCommandTail	417
defun PARSE-TokenOption	417
defun PARSE-TokenList	418
defun PARSE-CommandTail	418
defun PARSE-PrimaryOrQM	419
defun PARSE-Option	419
defun PARSE-Statement	419
defun PARSE-InfixWith	420
defun PARSE-With	420
defun PARSE-Category	421
defun PARSE-Expression	422
defun PARSE-Import	422
defun PARSE-Expr	423
defun PARSE-LedPart	423
defun PARSE-NudPart	424
defun PARSE-Operation	424
defun PARSE-leftBindingPowerOf	425
defun PARSE-rightBindingPowerOf	425
defun PARSE-getSemanticForm	425
defun PARSE-Prefix	426
defun PARSE-Infix	426
defun PARSE-TokTail	427
defun PARSE-Qualification	427
defun PARSE-Reduction	428
defun PARSE-ReductionOp	428
defun PARSE-Form	429
defun PARSE-Application	429
defun PARSE-Label	430
defun PARSE-Selector	430
defun PARSE-PrimaryNoFloat	431
defun PARSE-Primary	431
defun PARSE-Primary1	431
defun PARSE-Float	432
defun PARSE-FloatBase	433
defun PARSE-FloatBasePart	434
defun PARSE-FloatExponent	434
defun PARSE-Enclosure	435
defun PARSE-IntegerTok	436
defun PARSE-FormalParameter	436
defun PARSE-FormalParameterTok	436
defun PARSE-Quad	436
defun PARSE-String	437

defun PARSE-VarForm	437
defun PARSE-Scripts	437
defun PARSE-ScriptItem	438
defun PARSE-Name	438
defun PARSE-Data	439
defun PARSE-Sexpr	439
defun PARSE-Sexpr1	439
defun PARSE-NBGlyphTok	441
defun PARSE-GlyphTok	441
defun PARSE-AnyId	441
defun PARSE-Sequence	442
defun PARSE-Sequence1	442
defun PARSE-OpenBracket	443
defun PARSE-OpenBrace	443
defun PARSE-IteratorTail	444
defun PARSE-Iterator	444
The PARSE implicit routines	445
defun PARSE-Suffix	445
defun PARSE-SemiColon	446
defun PARSE-Return	446
defun PARSE-Exit	447
defun PARSE-Leave	447
defun PARSE-Seg	448
defun PARSE-Conditional	448
defun PARSE-ElseClause	449
defun PARSE-Loop	449
defun PARSE-LabelExpr	450
defun PARSE-FloatTok	450
8.2 The PARSE support routines	450
String grabbing	451
defun match-string	451
defun skip-blanks	451
defun token-lookahead-type	452
defun match-advance-string	452
defun initial-substring-p	453
defun quote-if-string	453
defun escape-keywords	454
defun isTokenDelimiter	455
defun underscore	455
Token Handling	455
defun getToken	455
defun unget-tokens	456
defun match-current-token	456
defun match-token	457
defun match-next-token	457
defun current-symbol	457

defun make-symbol-of	458
defun current-token	458
defun try-get-token	458
defun next-token	459
defun advance-token	459
defvar XTokenReader	460
defun get-token	460
Character handling	460
defun current-char	460
defun next-char	460
defun char-eq	461
defun char-ne	461
Error handling	461
defvar meta-error-handler	461
defun meta-syntax-error	462
Floating Point Support	462
defun floatexpid	462
Dollar Translation	462
defun dollarTran	462
Applying metagrammatical elements of a production (e.g., Star).	463
defmacro Bang	463
defmacro must	463
defun action	464
defun optional	464
defmacro star	464
Stacking and retrieving reductions of rules.	465
defvar reduce-stack	465
defmacro reduce-stack-clear	465
defun push-reduction	465
9 Comment Recording	467
9.1 Comment Recording Layer 0 – API	468
defun recordSignatureDocumentation	468
defun recordAttributeDocumentation	468
9.2 Comment Recording Layer 1	469
defun recordDocumentation	469
9.3 Comment Recording Layer 2	469
defun collectComBlock	469
9.4 Comment Recording Layer 3	470
defun recordHeaderDocumentation	470
defun collectAndDeleteAssoc	470
10 Category handling	473
defun getConstructorExports	473

11 Building libdb.text	475
defun extendLocalLibdb	475
defun buildLibdb	476
defun buildLibdbString	478
defun dbReadLines	479
defun purgeNewConstructorLines	479
defun dbWriteLines	479
defun buildLibdbConEntry	480
defun buildLibOps	482
defun buildLibOp	482
defun buildLibAttrs	483
defun buildLibAttr	483
defun screenLocalLine	484
12 Comment Syntax Checking	485
12.1 Comment Checking Layer 0 – API	490
defun finalizeDocumentation	490
12.2 Comment Checking Layer 1	493
defun transDocList	493
12.3 Comment Checking Layer 2	494
defun transDoc	494
12.4 Comment Checking Layer 3	495
defun transformAndRecheckComments	495
12.5 Comment Checking Layer 4	496
defun checkComments	496
defun checkRewrite	497
12.6 Comment Checking Layer 5	499
defun checkArguments	499
defun checkBalance	499
12.7 Comment Checking Layer 6	500
defun checkBeginEnd	500
defun checkDecorate	502
defun checkDecorateForHt	504
defun checkDocError1	505
defun checkFixCommonProblem	505
defun checkGetLispFunctionName	506
defun checkHTargs	506
defun checkRecordHash	507
defun spadSysChoose	510
defun spadSysBranch	510
defun checkTexht	511
defun checkTransformFirsts	512
defun checkTrim	515
12.8 Comment Checking Layer 7	515
defun checkDocError	515
defun checkRemoveComments	516

defun checkSkipToken	517
defun checkSplit2Words	517
12.9 Comment Checking Layer 8	518
defun checkAddIndented	518
defun checkDocMessage	518
defun checkExtract	519
defun checkGetArgs	520
defun checkGetMargin	520
defun checkGetParse	521
defun checkGetStringBeforeRightBrace	521
defun checkLeEg	522
defun checkIndentedLines	522
defun checkSkipIdentifierToken	523
defun checkSkipOpToken	523
defun checkSplitBrace	524
defun checkTrimCommented	525
defun newString2Words	525
12.10 Comment Checking Layer 9	526
defun checkAddBackSlashes	526
defun checkAddMacros	527
defun checkAddPeriod	527
defun checkAddSpaceSegments	528
defun checkAddSpaces	528
defun checkAlphabetic	529
defun checkLeEgfun	529
defun checkIsValidType	530
defun checkLookForLeftBrace	531
defun checkLookForRightBrace	531
defun checkNumOfArgs	532
defun checkSayBracket	532
defun checkSkipBlanks	533
defun checkSplitBackslash	533
defun checkSplitOn	534
defun checkSplitPunctuation	535
defun firstNonBlankPosition	536
defun getMatchingRightPren	536
defun hasNoVowels	537
defun htcharPosition	537
defun newWordFrom	538
defun removeBackslashes	539
defun whoOwns	539

13 Utility Functions	541
defun translablel	541
defun translable1	541
defun displayPreCompilationErrors	542
defun bumperrorcount	543
defun parseTranCheckForRecord	543
defun makeSimplePredicateOrNil	544
defun parse-spadstring	544
defun parse-string	544
defun parse-identifier	545
defun parse-number	545
defun parse-keyword	546
defun parse-argument-designator	546
defun print-package	547
defun checkWarning	547
defun tuple2List	547
defmacro pop-stack-1	548
defmacro pop-stack-2	548
defmacro pop-stack-3	549
defmacro pop-stack-4	549
defmacro nth-stack	549
defun Pop-Reduction	550
defun addclose	550
defun blankp	550
defun drop	551
defun escaped	551
defvar \$comblocklist	551
defun fincomblock	551
defun indent-pos	552
defun infixtok	553
defun is-console	553
defun next-tab-loc	553
defun nonblankloc	553
defun parseprint	554
defun skip-to-endif	554
14 The Compiler	555
defvar \$newConlist	555
14.1 Compiling EQ.spad	555
14.2 The top level compiler command	558
defun compiler	560
defun compileSpad2Cmd	562
defun compileSpadLispCmd	565
compilerDoitWithScreenedLisplib	567
defun compilerDoit	568
defun /rq	569

defun /rf	569
defun /RQ,LIB	569
defun /rf-1	570
defun spad	571
defun Interpreter interface to the compiler	574
defun compTopLevel	583
defun print-defun	584
defun def-rename	585
defun compOrCroak	586
defun compOrCroak1	586
defun comp	587
defun compNoStacking	588
defun compNoStacking1	588
defun comp2	589
defun comp3	589
defun applyMapping	591
defun compApply	592
defun compTypeOf	593
defun compColonInside	594
defun compAtom	595
defun compAtomWithModemap	596
defun transImplementation	597
defun convert	597
defun primitiveType	597
defun compSymbol	598
defun compList	599
defun compForm	600
defun compForm1	600
defun compToApply	602
defun compApplication	603
defun getFormModemaps	604
defun eltModemapFilter	605
defun seteltModemapFilter	606
defun compExpressionList	607
defun compForm2	607
defun compForm3	609
defun compFocompFormWithModemap	610
defun substituteIntoFunctorModemap	612
defun compFormPartiallyBottomUp	613
defun compFormMatch	613
defun compUniquely	613
defun compArgumentsAndTryAgain	614
defun compWithMappingMode	614
defun compWithMappingModel	615
defun extractCodeAndConstructTriple	620
defun hasFormalMapVariable	621

defun argsToSig	621
defun compMakeDeclaration	622
defun modifyModeStack	622
defun Create a list of unbound symbols	623
defun compOrCroak1,compactify	624
defun Compiler/Interpreter interface	624
defun recompile-lib-file-if-necessary	625
defun spad-fixed-arg	625
defun compile-lib-file	625
defun compileFileQuietly	626
defvar \$byConstructors	626
defvar \$constructorsSeen	626
15 Level 1	629
defvar current-fragment	629
defun read-a-line	629
16 Level 0	631
16.1 Line Handling	631
Line Buffer	631
defstruct line	631
defvar current-line	632
defmacro line-clear	632
defun line-print	632
defun line-at-end-p	632
defun line-past-end-p	633
defun line-next-char	633
defun line-advance-char	633
defun line-current-segment	634
defun line-new-line	634
defun next-line	634
defun Advance-Char	635
defun storeblanks	635
defun initial-substring	635
defun get-a-line	636
17 Signatures	637
18 The Chunks	639
Bibliography	655
Index	657

New Foreword

On October 1, 2001 Axiom was withdrawn from the market and ended life as a commercial product. On September 3, 2002 Axiom was released under the Modified BSD license, including this document. On August 27, 2003 Axiom was released as free and open source software available for download from the Free Software Foundation's website, Savannah.

Work on Axiom has had the generous support of the Center for Algorithms and Interactive Scientific Computation (CAISS) at City College of New York. Special thanks go to Dr. Gilbert Baumslag for his support of the long term goal.

The online version of this documentation is roughly 1000 pages. In order to make printed versions we've broken it up into three volumes. The first volume is tutorial in nature. The second volume is for programmers. The third volume is reference material. We've also added a fourth volume for developers. All of these changes represent an experiment in print-on-demand delivery of documentation. Time will tell whether the experiment succeeded.

Axiom has been in existence for over thirty years. It is estimated to contain about three hundred man-years of research and has, as of September 3, 2003, 143 people listed in the credits. All of these people have contributed directly or indirectly to making Axiom available. Axiom is being passed to the next generation. I'm looking forward to future milestones.

With that in mind I've introduced the theme of the "30 year horizon". We must invent the tools that support the Computational Mathematician working 30 years from now. How will research be done when every bit of mathematical knowledge is online and instantly available? What happens when we scale Axiom by a factor of 100, giving us 1.1 million domains? How can we integrate theory with code? How will we integrate theorems and proofs of the mathematics with space-time complexity proofs and running code? What visualization tools are needed? How do we support the conceptual structures and semantics of mathematics in effective ways? How do we support results from the sciences? How do we teach the next generation to be effective Computational Mathematicians?

The "30 year horizon" is much nearer than it appears.

Tim Daly
CAISS, City College of New York
November 10, 2003 ((iHy))

Chapter 1

The Axiom Compiler

1.1 Makefile

a : b → This book is actually a literate program[Knut92] and contains executable source code. In particular, the Makefile for this book is part of the source of the book and is included below.

Chapter 2

Overview

The Spad language is a mathematically oriented language intended for writing computational mathematics. It derives its logical structure from abstract algebra. It features ideas that are still not available in general purpose programming languages, such as selecting overloaded procedures based on the return type as well as the types of the arguments.

The Spad language is heavily influenced by Barbara Liskov's work. It features encapsulation (aka objects), inheritance, and overloading. It has categories which are defined by the exports. Categories are parameterized functors that take arguments which define their behavior.

More details on the language and its high level concepts is available in the Programmers Guide, Volume 3.

The Spad compiler accepts the Spad language and generates a set of files used by the interpreter, detailed in Volume 5.

The compiler does not produce stand-alone executable code. It assumes that it will run inside the interpreter and that the code it generates will be loaded into the interpreter.

Some of the routines are common to both the compiler and the interpreter. Where this happens we have favored the interpreter volume (Volume 5) as the official source location. In each case we will make reference to that volume and the code in it. Thus, the compiler volume should be considered as an extension of the interpreter document.

This volume will go into painful detail of every aspect of compiling Spad code. We will start by defining the input to, and output from the compiler so we know what we are trying to achieve.

Next we will look at the top level data structures used by the compiler. Unfortunately, the compiler uses a large number of "global variables" to pass information and alter control flow. Some of these are used by many routines and some of these are very local to a small subset or a recursion. We will cover the minor ones as they arise.

Next we examine the Pratt parser idea and the Led and Nud concepts, which is used to drive the low level parsing.

Following that we journey deep into the code, trying our best not to get lost in the details. The code is introduced based on “motivation” rather than in strict execution order or related concept order. We do this to try to make the compiler a “readable novel” rather than a mud-march through the code. The goal is to keep the reader’s interest while trying to be exact. Sometimes this will require detours to discuss subtopics.

“Motivating” a piece of software is a not-very-well established form of narrative writing so we assume your forgiveness if we get it wrong. Worse yet, some of the pieces of the system are “legacy”, in that they are no longer used and should be removed. Other parts of the system may have very weak descriptions because we simply do not understand them either. Since this is a living document and the code for the system is actually the code you are reading we will expand parts as we go.

2.1 Syntax by Jacob Smith

In Smith [?] we find

<i>Module</i>	M	$::=$	Δ^+
<i>ToplevelDef</i>	Δ	$::=$	$C \mid D \mid P \mid d$
<i>CategoryDef</i>	C	$::=$	$\phi : \text{Category} == E$
<i>Exports</i>	E	$::=$	W
			$\mid$ $X W^?$
<i>Extension</i>	X	$::=$	τ
			$\mid$ $\text{Join } (\tau^+)$
<i>WithExpr</i>	W	$::=$	$\text{with } S^+$
<i>Signature</i>	S	$::=$	$x : \tau$
<i>Type</i>	τ	$::=$	Boolean
			$\mid$ Integer
			$\mid$ Float
			$\mid$ Record(S^+)
			$\mid$ Union(S^+)
			$\mid$ $\tau_1 \times \cdots \times \tau_n \rightarrow \tau_0$
			$\mid$ γ
			$\mid$ $\gamma([\tau e]^+)$
<i>DomainDef</i>	D	$::=$	$\phi : E == K$
<i>PackageDef</i>	P	$::=$	$\phi : E == K$
<i>Capsule</i>	K	$::=$	$\text{add } R^? d^+$
<i>Representation</i>	R	$::=$	$\text{Rep} := \tau$
<i>Definition</i>	d	$::=$	$[n \phi] : \tau == s$
<i>CallForm</i>	ϕ	$::=$	$x(S^+)$
<i>Statement</i>	s	$::=$	e
			$\mid$ if e then s
			$\mid$ if e then s else s
			$\mid$ i^+ repeat s
			$\mid$ $S == e$
			$\mid$ $S := e$
<i>Iterator</i>	i	$::=$	for n in e [suchthat p]? $\mid$ while p
<i>Expression</i>	e, p	$::=$	c
			$\mid$ x^τ
			$\mid$ $e(e^+)$
			$\mid$ $e.x^\tau$
			$\mid$ $e \text{ case } \tau$
			$\mid$ $e := e$
			$\mid$ $e \text{ where } d^+$
<i>Variable</i>	x^τ		
<i>Constant</i>	c^τ		
<i>Identifier</i>	x		
<i>CDPName</i>	γ		name of a Spad category, domain, or package

where $Z^?$ represents an optional Z , Z^+ a non-empty finite sequence of Z , the square brackets are used for grouping.

ToplevelDef A toplevel definition is either a Spad category definition, or a Spad domain definition, or a Spad package definition, or a delayed definition.

CategoryDef A Spad category definition specifies a class of algebras, by declaring the signatures of the required operations. A Spad category definition may extend existing Spad categories with new signatures. For example, the fragment

```
Monoid(): Category == with
  *: (%,% ) -> %
  1: %
```

declares `Monoid` as a Spad category with two signatures:

1. the symbol `*` is a binary operation on the domain belonging to this category
2. the identifier `1` denotes a constant object of a domain belonging to the category being defined.

The following category definition

```
Group(): Category == Monoid with
  inverse: % -> %
```

extends `Monoid` with the `inverse` operation, to capture the mathematical notion of group structure. One can think of Spad categories as *specifying* views on objects.

Exports Definitions for Spad categories, Spad domains, and packages specify exported operations, i.e., the “public interface” in programming languages jargon, through either a *WithExpr*, or an *Extension*, or a combination of both.

WithExpr A *WithExpr* is essentially an unnamed Spad category consisting of a list of operations with signatures (*Signature*).

Extension Definitions for Spad categories and Spad domains may extend existing Spad categories or domains. An extension may specify either a type, or multiple categories through the `Join` operator. The latter form corresponds to multiple inheritance in object-oriented programming languages.

Signature The specification of a type for an identifier can appear in a *WithExpr*, as a parameter declaration in *CallForm*, as a field of a record or union, or in a local variable definition.

Type A type is a built-in type (`Boolean`, `Integer`, `Float`), a record or union, a function type, the name of a Spad category or Spad domain, or an instantiation of a Spad category or Spad domain. All field names specified by signatures in a record must be distinct. Similarly, all field names specified in a union must be distinct and unique in the enclosing scope; this applies recursively to any other union types directly referenced in the signature list of the union.

DomainDef A Spad domain definition provides implemenations for views specified by categories. A domain definition has an interface specification part (*Exports*) stating the categories and possible additional signatures it implements, and an implementation part called *capsule*. The implementation part may define the representation of the object belonging to the domain, and provide definitions for operations declared in its *Exports*. For example, the program fragment

```
IntMonoid(): Monoid == add
  Rep == Integer
  (x:%) * (y:%) == (rep x + rep y)$Integer
  1:% == 0::Integer
```

provides an implementation of `IntMonoid` for the `Monoid` specification as follows:

- the object representation domain is `Integer`;
- “multiplication” of two objects in `IntMonoid` is the value obtained by adding their respective underlying values (returned by the `*` operator);
- the `Integer` constant 0 is the underlying value of the unit of `IntMonoid`

Note that a Spad domain almost always references the “current domain” using the symbol `%`.

PackageDef A package definition provides implementations for functions that operate on a Spad domain. Unlike a Spad domain, a package does not define a *Representation* and does not reference the symbol `%`. Like a Spad domain, it has an *Exports* part and an implementation part.

Capsule The implementation part of a Spad domain or package is its capsule. A capsule mayu specify the representation of a domain (if it si the implementation of a domain), and specifies a sequence of toplevel definitions for operations on the Spad domain objects, or the operators in a package.

sl Representation A Spad domain specifies the underlying representation of its objects by assigning a type expression to the identifier `Rep`. A *Representation* can occur only in a Spad domain definition.

Definition A (delayed) definition is the binding of an identifier or a function call expression to a Spad category, Spad domain, or an ordinary function. The body of the definition is evaluated when needed. That evaluation may happen only once for a given arugment list. Even though the evaluation is delayed, the body is still fully type checked at the definition point. The Spad language, as understood by the Spad compiler, does not allow ordinary function definitions at the toplevel. However, they are the core of the language understood by the interpreter. For uniformity, we include toplevel function definitions in the Spad subset we describe.

CallForm A call form consists of an identifier and a parenthesized sequence of signatures declaring formal parameters. A call form is needed in the definitions of a Spad category, Spad domain, and function.

Statement Statements appear in the body of function definitions. A statement is either an expression, a one or two-arm if-statement, an iteration where the body of the iteration (a statement) is controlled by a list of iterators, a local variable definition, or an assignment.

Iterator An iterator is either a sequence of items x drawn from a sequence e , possibly filtered by a predicate p , or a repeated evaluation of a predicate.

Expression An expression is either a constant, variable, function call, member selection, type-case expression, an assignment, or a qualified expression. A qualified expression is an expression that contains free variables and is immediately followed by their definitions in a where-clause. We assimilate expressions built with built-in operations – such as addition on integers, etc. – as function calls.

Variable A variable is the use of a name declared with a given type

Constant A constant is a built-in value, such as $+^{Integer \times Integer \rightarrow Integer}$, $345^{Integer}$, $true^{Boolean}$, etc.

Identifier An identifier is a finite sequence of characters. The set of identifiers in Spad is countably infinite.

Language features

The Spad programming language supports elements of dependent types, a result of the functorial nature of the data-structuring mechanisms available in Spad. That is, the Spad type system allows types to be function-like objects with arguments that depend on types and values. Dependent types enables an unusually direct style of implementation of mathematical structures.

The Spad programming language also supports general function overloading; in particular, a function can be overloaded on its argument and return types. The overload resolution algorithm exploits all context information, including arguments and target types, to select the best matching function. Implicit conversion is supported through the `coerce` operator.

Semantics

The computational rules used to evaluate Spad programs are those of *eager* semantics (and call-by-value), and the arguments of functions are passed by reference. We sketch the semantics of Spad programs in two ways: small-step operational semantics, and denotational semantics. The small-step operational semantics gives an intuitive idea of the behavior of Spad programs, whereas the denotational semantics lets us associate mathematical functions to Spad programs. The latter allows us to formally talk about the notion of a derivative of a Spad program.

Operational semantics

The Spad language is imperative in the sense that its programs operate on stores by explicit modification. Values of Spad programs can be booleans, integers, floating point numbers, ag-

gregates thereof, or function codes. We denote the collection of values by **Value**, inductively defined as:

- Location values: object locations are in **Value**
- Boolean values: **true** $\in$ **Value** and **false** $\in$ **Value**
- Integer Values: integer constants $n^{Integer}$ are in **Value**
- Float values: float constants f^{Float} are in **Value**
- Functions: If f is a defined function of type $\tau_1 \rightarrow \tau_2$
then the constant $f^{\tau_1 \rightarrow \tau_2}$ is in **Value**
- Aggregates: if $c_i^{\tau_i}$ are values of type τ_i in **Value**,
then the tuple $(c_1^{\tau_1}, \dots, c_n^{\tau_n})^{\tau_1 \times \dots \times \tau_n}$ is in **Value**.
Tuples represent record values.
Similarly, if c^{τ_1} is in **Value**, so is $c^{\tau_2 \leftarrow \tau_1}$.
It represents a value of a field of type τ_1 in a union τ_2 .

The behavior of a Spad program is a sequence of configurations $\langle p, \sigma, \Gamma \rangle$ where p denotes fragments of Spad constructs, σ the store of values, and Γ the current environment of bindings of variables to types and expressions. The notation $\Gamma, x^\tau == e$ denotes an environment obtained by extending Γ with the binding $x^\tau == e$. The $== e$ part may be missing. A store of σ is a mapping from memory locations to Spad values. We use the notation $\sigma[v/l]$ to designate an *updated function* defined by

$$\sigma[v/l] = \begin{cases} v & \text{if } x = l \\ \sigma(x) & \text{otherwise} \end{cases}$$

Each configuration is defined by structural induction on the syntax of Spad.

Denotational semantics

The basic idea of algorithmic differentiation rests on the notion that a computer program computes a function whose range has a ring structure; and the collection of such functions can be endowed with a *differential algebra* structure. The theory of *denotational semantics* is a useful tool. We see a standard denotational semantics of the Spad programming language that respects the operational semantics outlined above, i.e.

$$t \rightarrow^* v^\tau \Rightarrow [[t]] = [[v^\tau]]$$

Variable

$$\langle x^\tau, \sigma, \Gamma \rangle \rightarrow \langle \sigma(x^\tau), \sigma, \Gamma \rangle$$

Call-Arguments

$$\frac{\langle e_i, \sigma, \Gamma \rangle \rightarrow \langle e'_i, \sigma', \Gamma \rangle}{e_0(v_1, \dots, e_i, \dots, e_n), \sigma, \Gamma \rightarrow \langle e_0(v_1, \dots, e'_i, \dots, e_n), \sigma', \Gamma \rangle}$$

where the x_i are parameters of v_0

Call-Operator

$$\frac{\langle e_0, \sigma, \Gamma \rangle \rightarrow \langle e_0, \sigma', \Gamma \rangle}{\langle e_0(v_1, \dots, v_n), \sigma, \Gamma \rangle \rightarrow \langle e'_0(v_1, \dots, v_n), \sigma', \Gamma \rangle}$$

where the x_i are the parameters of v_0

Call

$$\langle v_0^{\tau_0}(v_1^{\tau_1}, \dots, v_n^{\tau_n}), \sigma, \Gamma \rangle \rightarrow \langle v_0^{\tau_0}[v_1^{\tau_1}/x_1, \dots, v_n^{\tau_n}/x_n], \sigma, \Gamma \rangle$$

where the x_i are parameters of v_0

Qualified Expression

$$\frac{\dots \frac{\langle \delta_1, \sigma, \Gamma_1 \rangle \rightarrow \langle \delta'_1, \sigma, \Gamma_2 \rangle \quad \langle \delta_2, \sigma, \Gamma_2 \rangle \rightarrow \langle \delta'_2, \sigma, \Gamma_3 \rangle}{\dots \quad \langle \delta_n, \sigma, \Gamma_n \rangle \rightarrow \langle \delta'_n, \sigma, \Gamma_{n+1} \rangle \quad \langle e, \sigma, \Gamma_{n+1} \rangle \rightarrow \langle e', \sigma', \Gamma_{n+2} \rangle}}{\langle e \text{ where } \delta_1 \dots \delta_n, \sigma, \Gamma_1 \rangle \rightarrow \langle e', \sigma', \Gamma_{n+2} \rangle}$$

Sequence-Head

$$\frac{\langle s_1, \sigma_1, \Gamma_1 \rangle \rightarrow \langle s'_1, \sigma_2, \Gamma_2 \rangle}{\langle s_1; s_2, \sigma_1, \Gamma_1 \rangle \rightarrow \langle s'_1; s_2, \sigma_2, \Gamma_2 \rangle}$$

Sequence-Tail

$$\langle v_1; s_2, \sigma, \Gamma \rangle \rightarrow \langle s_2, \sigma, \Gamma \rangle$$

If-True

$$\frac{\langle s_1, \sigma, \Gamma \rangle \rightarrow \langle s'_1, \sigma', \Gamma' \rangle}{\langle \text{if true then } s_1; s_2, \sigma, \Gamma \rangle \rightarrow \langle s'_1, \sigma', \Gamma' \rangle}$$

If-False

$$\frac{\langle s_2, \sigma, \Gamma \rangle \rightarrow \langle s'_2, \sigma', \Gamma' \rangle}{\langle \text{if false then } s_1; s_2, \sigma, \Gamma \rangle \rightarrow \langle s'_2, \sigma', \Gamma' \rangle}$$

Assignment-Left

$$\frac{\langle e_1, \sigma_0, \Gamma \rangle \rightarrow \langle e'_1, \sigma_1, \Gamma \rangle}{\langle e_1 := e_2, \sigma, \Gamma \rangle \rightarrow \langle e'_1 := e_2, \sigma_1, \Gamma \rangle}$$

Assignment-Right

$$\frac{\langle e_2, \sigma, \Gamma \rangle \rightarrow \langle e'_2, \sigma', \Gamma \rangle}{\langle l := e_2, \sigma, \Gamma \rangle \rightarrow \langle l := e'_2, \sigma', \Gamma \rangle}$$

Assignment

$$\frac{}{\langle l := v^\tau, \sigma, \Gamma \rangle \rightarrow \langle v^\tau, \sigma[v^\tau/l], \Gamma \rangle}$$

Immediate Definition

$$\frac{\langle e, \sigma, \Gamma \rangle \rightarrow \langle e', \sigma_1, \Gamma \rangle}{\langle x : \tau := e, \sigma, \Gamma \rangle \rightarrow \langle x : \tau := e', \sigma_1, \Gamma \rangle}$$

Immediate Definition

$$\langle x : \tau := v^\tau, \sigma, \Gamma \rangle \rightarrow \langle v^\tau, \sigma, \Gamma, x^\tau == v^\tau \rangle$$

2.2 The Input

```

)abbrev domain EQ Equation
--FOR THE BENEFIT OF LIBAXO GENERATION
++ Author: Stephen M. Watt, enhancements by Johannes Grabmeier
++ Date Created: April 1985
++ Date Last Updated: June 3, 1991; September 2, 1992
++ Basic Operations: =
++ Related Domains:
++ Also See:
++ AMS Classifications:
++ Keywords: equation
++ Examples:
++ References:
++ Description:
++ Equations as mathematical objects. All properties of the basis domain,
++ e.g. being an abelian group are carried over the equation domain, by
++ performing the structural operations on the left and on the
++ right hand side.
-- The interpreter translates "=" to "equation". Otherwise, it will
-- find a modemap for "=" in the domain of the arguments.

Equation(S: Type): public == private where
  Ex ==> OutputForm
  public ==> Type with
    "=": (S, S) -> $
      ++ a=b creates an equation.

```

```

equation: (S, S) -> $
  ++ equation(a,b) creates an equation.
swap: $ -> $
  ++ swap(eq) interchanges left and right hand side of equation eq.
lhs: $ -> S
  ++ lhs(eqn) returns the left hand side of equation eqn.
rhs: $ -> S
  ++ rhs(eqn) returns the right hand side of equation eqn.
map: (S -> S, $) -> $
  ++ map(f,eqn) constructs a new equation by applying f to both
  ++ sides of eqn.
if S has InnerEvalable(Symbol,S) then
  InnerEvalable(Symbol,S)
if S has SetCategory then
  SetCategory
  CoercibleTo Boolean
  if S has Evalable(S) then
    eval: ($, $) -> $
      ++ eval(eqn, x=f) replaces x by f in equation eqn.
    eval: ($, List $) -> $
      ++ eval(eqn, [x1=v1, ... xn=vn]) replaces xi by vi in equation eqn.
if S has AbelianSemiGroup then
  AbelianSemiGroup
  "+" : (S, $) -> $
    ++ x+eqn produces a new equation by adding x to both sides of
    ++ equation eqn.
  "+" : ($, S) -> $
    ++ eqn+x produces a new equation by adding x to both sides of
    ++ equation eqn.
if S has AbelianGroup then
  AbelianGroup
  leftZero : $ -> $
    ++ leftZero(eq) subtracts the left hand side.
  rightZero : $ -> $
    ++ rightZero(eq) subtracts the right hand side.
  "-" : (S, $) -> $
    ++ x-eqn produces a new equation by subtracting both sides of
    ++ equation eqn from x.
  "-" : ($, S) -> $
    ++ eqn-x produces a new equation by subtracting x from
    ++ both sides of equation eqn.
if S has SemiGroup then
  SemiGroup
  "*" : (S, $) -> $
    ++ x*eqn produces a new equation by multiplying both sides of
    ++ equation eqn by x.
  "*" : ($, S) -> $
    ++ eqn*x produces a new equation by multiplying both sides of
    ++ equation eqn by x.
if S has Monoid then

```



```

Monoid
leftOne : $ -> Union($,"failed")
  ++ leftOne(eq) divides by the left hand side, if possible.
rightOne : $ -> Union($,"failed")
  ++ rightOne(eq) divides by the right hand side, if possible.
if S has Group then
  Group
  leftOne : $ -> Union($,"failed")
    ++ leftOne(eq) divides by the left hand side.
  rightOne : $ -> Union($,"failed")
    ++ rightOne(eq) divides by the right hand side.
if S has Ring then
  Ring
  BiModule(S,S)
if S has CommutativeRing then
  Module(S)
  --Algebra(S)
if S has IntegralDomain then
  factorAndSplit : $ -> List $
    ++ factorAndSplit(eq) make the right hand side 0 and
    ++ factors the new left hand side. Each factor is equated
    ++ to 0 and put into the resulting list without repetitions.
if S has PartialDifferentialRing(Symbol) then
  PartialDifferentialRing(Symbol)
if S has Field then
  VectorSpace(S)
  "/": ($, $) -> $
    ++ e1/e2 produces a new equation by dividing the left and right
    ++ hand sides of equations e1 and e2.
  inv: $ -> $
    ++ inv(x) returns the multiplicative inverse of x.
if S has ExpressionSpace then
  subst: ($, $) -> $
    ++ subst(eq1,eq2) substitutes eq2 into both sides of eq1
    ++ the lhs of eq2 should be a kernel

private ==> add
Rep := Record(lhs: S, rhs: S)
eq1,eq2: $
s : S
if S has IntegralDomain then
  factorAndSplit eq ==
    (S has factor : S -> Factored S) =>
      eq0 := rightZero eq
      [equation(rcf.factor,0) for rcf in factors factor lhs eq0]
  [eq]
l:S = r:S == [l, r]
equation(l, r) == [l, r] -- hack! See comment above.
lhs eqn == eqn.lhs
rhs eqn == eqn.rhs

```

```

swap eqn      == [rhs eqn, lhs eqn]
map(fn, eqn)  == equation(fn(eqn.lhs), fn(eqn.rhs))

if S has InnerEvalable(Symbol,S) then
  s:Symbol
  ls:List Symbol
  x:S
  lx:List S
  eval(eqn,s,x) == eval(eqn.lhs,s,x) = eval(eqn.rhs,s,x)
  eval(eqn,ls,lx) == eval(eqn.lhs,ls,lx) = eval(eqn.rhs,ls,lx)
if S has Evalable(S) then
  eval(eqn1:$, eqn2:$):$ ==
    eval(eqn1.lhs, eqn2 pretend Equation S) =
      eval(eqn1.rhs, eqn2 pretend Equation S)
  eval(eqn1:$, leqn2:List $):$ ==
    eval(eqn1.lhs, leqn2 pretend List Equation S) =
      eval(eqn1.rhs, leqn2 pretend List Equation S)
if S has SetCategory then
  eq1 = eq2 == (eq1.lhs = eq2.lhs)@Boolean and
               (eq1.rhs = eq2.rhs)@Boolean
  coerce(eqn:$):Ex == eqn.lhs::Ex = eqn.rhs::Ex
  coerce(eqn:$):Boolean == eqn.lhs = eqn.rhs
if S has AbelianSemiGroup then
  eq1 + eq2 == eq1.lhs + eq2.lhs = eq1.rhs + eq2.rhs
  s + eq2 == [s,s] + eq2
  eq1 + s == eq1 + [s,s]
if S has AbelianGroup then
  - eq == (- lhs eq) = (-rhs eq)
  s - eq2 == [s,s] - eq2
  eq1 - s == eq1 - [s,s]
  leftZero eq == 0 = rhs eq - lhs eq
  rightZero eq == lhs eq - rhs eq = 0
  0 == equation(0$S,0$S)
  eq1 - eq2 == eq1.lhs - eq2.lhs = eq1.rhs - eq2.rhs
if S has SemiGroup then
  eq1:$ * eq2:$ == eq1.lhs * eq2.lhs = eq1.rhs * eq2.rhs
  1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs
  1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs
  eqn:$ * 1:S == eqn.lhs * 1 = eqn.rhs * 1
  -- We have to be a bit careful here: raising to a +ve integer is OK
  -- (since it's the equivalent of repeated multiplication)
  -- but other powers may cause contradictions
  -- Watch what else you add here! JHD 2/Aug 1990
if S has Monoid then
  1 == equation(1$S,1$S)
  recip eq ==
    (lh := recip lhs eq) case "failed" => "failed"
    (rh := recip rhs eq) case "failed" => "failed"
    [lh :: S, rh :: S]
  leftOne eq ==

```

```

      (re := recip lhs eq) case "failed" => "failed"
      1 = rhs eq * re
    rightOne eq ==
      (re := recip rhs eq) case "failed" => "failed"
      lhs eq * re = 1
  if S has Group then
    inv eq == [inv lhs eq, inv rhs eq]
    leftOne eq == 1 = rhs eq * inv rhs eq
    rightOne eq == lhs eq * inv rhs eq = 1
  if S has Ring then
    characteristic() == characteristic()$S
    i:Integer * eq:$ == (i::S) * eq
  if S has IntegralDomain then
    factorAndSplit eq ==
      (S has factor : S -> Factored S) =>
        eq0 := rightZero eq
        [equation(rcf.factor,0) for rcf in factors factor lhs eq0]
      (S has Polynomial Integer) =>
        eq0 := rightZero eq
        MF ==> MultivariateFactorize(Symbol, IndexedExponents Symbol, _
          Integer, Polynomial Integer)
        p : Polynomial Integer := (lhs eq0) pretend Polynomial Integer
        [equation((rcf.factor) pretend S,0) for rcf in factors factor(p)$MF]
      [eq]
  if S has PartialDifferentialRing(Symbol) then
    differentiate(eq:$, sym:Symbol):$ ==
      [differentiate(lhs eq, sym), differentiate(rhs eq, sym)]
  if S has Field then
    dimension() == 2 :: CardinalNumber
    eq1:$ / eq2:$ == eq1.lhs / eq2.lhs = eq1.rhs / eq2.rhs
    inv eq == [inv lhs eq, inv rhs eq]
  if S has ExpressionSpace then
    subst(eq1,eq2) ==
      eq3 := eq2 pretend Equation S
      [subst(lhs eq1,eq3),subst(rhs eq1,eq3)]

```

2.3 The Output, the EQ.nrlib directory

The Spad compiler generates several files in a directory named after the input abbreviation. The input file contains an abbreviation line:

```
)abbrev domain EQ Equation
```

for each category, domain, or package. The abbreviation line has 3 parts.

- one of “category”, “domain”, or “package”

- the abbreviation for this domain (8 Uppercase Characters maximum)
- the name of this domain

Since the abbreviation for the Equation domain is EQ, the compiler will put all of its output into a subdirectory called “EQ.nrlib”. The “nrlib” is a port of a very old VMLisp file format, simulated with directories.

For the EQ input file, the compiler will create the following output files, each of which we will explain in detail below.

```
/research/test/int/algebra/EQ.nrlib:
used 216 available 4992900
drwxr-xr-x    2 root root   4096 2010-12-09 11:20 .
drwxr-xr-x 1259 root root  73728 2010-12-09 11:43 ..
-rw-r--r--    1 root root  19228 2010-12-09 11:20 code.lsp
-rw-r--r--    1 root root  34074 2010-12-09 11:20 code.o
-rw-r--r--    1 root root  13543 2010-12-09 11:20 EQ.fn
-rw-r--r--    1 root root  19228 2010-12-09 11:20 EQ.lsp
-rw-r--r--    1 root root  36148 2010-12-09 11:20 index.kaf
-rw-r--r--    1 root root   6236 2010-12-09 11:20 info
```

2.4 The code.lsp and EQ.lsp files

```
(/VERSIONCHECK 2)
```

```
(DEFUN |EQ;factorAndSplit;$L;1| (|eq| $)
  (PROG (|eq0| #:G1403 |rcf| #:G1404)
    (RETURN
      (SEQ (COND
        ((|HasSignature| (QREFELT $ 6)
          (LIST '|factor|
            (LIST (LIST '|Factored|
              (|devaluate| (QREFELT $ 6)))
              (|devaluate| (QREFELT $ 6)))))
          (SEQ (LETT |eq0| (SPADCALL |eq| (QREFELT $ 8))
            |EQ;factorAndSplit;$L;1|)
            (EXIT (PROGN
              (LETT #:G1403 NIL |EQ;factorAndSplit;$L;1|)
              (SEQ (LETT |rcf| NIL
                |EQ;factorAndSplit;$L;1|)
                (LETT #:G1404
                  (SPADCALL
                    (SPADCALL
                      (SPADCALL |eq0| (QREFELT $ 9))
                      (QREFELT $ 11))
                      (QREFELT $ 15))
                    |EQ;factorAndSplit;$L;1|)
```

```

G190
(COND
  ((OR (ATOM #:G1404)
    (PROGN
      (LETT |rcf| (CAR #:G1404)
        |EQ;factorAndSplit;$L;1|)
        NIL))
    (GO G191)))
(SEQ (EXIT
  (LETT #:G1403
    (CONS
      (SPADCALL (QCAR |rcf|)
        (|spadConstant| $ 16)
        (QREFELT $ 17))
        #:G1403)
      |EQ;factorAndSplit;$L;1|)))
  (LETT #:G1404 (CDR #:G1404)
    |EQ;factorAndSplit;$L;1|)
  (GO G190) G191
  (EXIT (NREVERSEO #:G1403))))))
('T (LIST |eq|))))))

(PUT (QUOTE |EQ;=;2S$;2|) (QUOTE |SPADreplace|) (QUOTE CONS))

(DEFUN |EQ;=;2S$;2| (|l| |r| $) (CONS |l| |r|))

(PUT (QUOTE |EQ;equation;2S$;3|) (QUOTE |SPADreplace|) (QUOTE CONS))

(DEFUN |EQ;equation;2S$;3| (|l| |r| $) (CONS |l| |r|))

(PUT (QUOTE |EQ;lhs;$S;4|) (QUOTE |SPADreplace|) (QUOTE QCAR))

(DEFUN |EQ;lhs;$S;4| (|eqn| $) (QCAR |eqn|))

(PUT (QUOTE |EQ;rhs;$S;5|) (QUOTE |SPADreplace|) (QUOTE QCDR))

(DEFUN |EQ;rhs;$S;5| (|eqn| $) (QCDR |eqn|))

(DEFUN |EQ;swap;2$;6| (|eqn| $) (CONS (SPADCALL |eqn| (QREFELT $ 21))
  (SPADCALL |eqn| (QREFELT $ 9))))

(DEFUN |EQ;map;M2$;7| (|fn| |eqn| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn|) |fn|)
    (SPADCALL (QCDR |eqn|) |fn|)
    (QREFELT $ 17)))

(DEFUN |EQ;eval;$SS$;8| (|eqn| |s| |x| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn|) |s| |x| (QREFELT $ 26))

```

```

(SPADCALL (QCDR |eqn|) |s| |x| (QREFELT $ 26))
(QREFELT $ 20)))

(DEFUN |EQ;eval;$LL$;9| (|eqn| |ls| |lx| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn|) |ls| |lx| (QREFELT $ 30))
    (SPADCALL (QCDR |eqn|) |ls| |lx| (QREFELT $ 30))
    (QREFELT $ 20)))

(DEFUN |EQ;eval;3$;10| (|eqn1| |eqn2| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn1|) |eqn2| (QREFELT $ 33))
    (SPADCALL (QCDR |eqn1|) |eqn2| (QREFELT $ 33))
    (QREFELT $ 20)))

(DEFUN |EQ;eval;$L$;11| (|eqn1| |eqn2| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn1|) |eqn2| (QREFELT $ 36))
    (SPADCALL (QCDR |eqn1|) |eqn2| (QREFELT $ 36))
    (QREFELT $ 20)))

(DEFUN |EQ;=;2$B;12| (|eq1| |eq2| $)
  (COND
    ((SPADCALL (QCAR |eq1|) (QCAR |eq2|) (QREFELT $ 39))
     (SPADCALL (QCDR |eq1|) (QCDR |eq2|) (QREFELT $ 39)))
    ((QUOTE T) (QUOTE NIL))))

(DEFUN |EQ;coerce;$Of;13| (|eqn| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn|) (QREFELT $ 42))
    (SPADCALL (QCDR |eqn|) (QREFELT $ 42))
    (QREFELT $ 43)))

(DEFUN |EQ;coerce;$B;14| (|eqn| $)
  (SPADCALL (QCAR |eqn|) (QCDR |eqn|) (QREFELT $ 39)))

(DEFUN |EQ;+;3$;15| (|eq1| |eq2| $)
  (SPADCALL
    (SPADCALL (QCAR |eq1|) (QCAR |eq2|) (QREFELT $ 46))
    (SPADCALL (QCDR |eq1|) (QCDR |eq2|) (QREFELT $ 46))
    (QREFELT $ 20)))

(DEFUN |EQ;+;S2$;16| (|s| |eq2| $)
  (SPADCALL (CONS |s| |s|) |eq2| (QREFELT $ 47)))

(DEFUN |EQ;+;$S$;17| (|eq1| |s| $)
  (SPADCALL |eq1| (CONS |s| |s|) (QREFELT $ 47)))

(DEFUN |EQ;-;2$;18| (|eq| $)
  (SPADCALL

```

```

(SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QREFELT $ 50))
(SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QREFELT $ 50))
(QREFELT $ 20)))

(DEFUN |EQ;-;S2$;19| (|s| |eq2| $)
  (SPADCALL (CONS |s| |s|) |eq2| (QREFELT $ 52)))

(DEFUN |EQ;-;$S$;20| (|eq1| |s| $)
  (SPADCALL |eq1| (CONS |s| |s|) (QREFELT $ 52)))

(DEFUN |EQ;leftZero;2$;21| (|eq| $)
  (SPADCALL
    (|spadConstant| $ 16)
    (SPADCALL
      (SPADCALL |eq| (QREFELT $ 21))
      (SPADCALL |eq| (QREFELT $ 9))
      (QREFELT $ 56))
    (QREFELT $ 20)))

(DEFUN |EQ;rightZero;2$;22| (|eq| $)
  (SPADCALL
    (SPADCALL
      (SPADCALL |eq| (QREFELT $ 9))
      (SPADCALL |eq| (QREFELT $ 21))
      (QREFELT $ 56))
    (|spadConstant| $ 16)
    (QREFELT $ 20)))

(DEFUN |EQ;Zero;$;23| ($)
  (SPADCALL (|spadConstant| $ 16) (|spadConstant| $ 16) (QREFELT $ 17)))

(DEFUN |EQ;-;3$;24| (|eq1| |eq2| $)
  (SPADCALL
    (SPADCALL (QCAR |eq1|) (QCAR |eq2|) (QREFELT $ 56))
    (SPADCALL (QCDR |eq1|) (QCDR |eq2|) (QREFELT $ 56))
    (QREFELT $ 20)))

(DEFUN |EQ;*;3$;25| (|eq1| |eq2| $)
  (SPADCALL
    (SPADCALL (QCAR |eq1|) (QCAR |eq2|) (QREFELT $ 58))
    (SPADCALL (QCDR |eq1|) (QCDR |eq2|) (QREFELT $ 58))
    (QREFELT $ 20)))

(DEFUN |EQ;*;S2$;26| (|l| |eqn| $)
  (SPADCALL
    (SPADCALL |l| (QCAR |eqn|) (QREFELT $ 58))
    (SPADCALL |l| (QCDR |eqn|) (QREFELT $ 58))
    (QREFELT $ 20)))

(DEFUN |EQ;*;S2$;27| (|l| |eqn| $)

```

```

(SPADCALL
  (SPADCALL |l| (QCAR |eqn|) (QREFELT $ 58))
  (SPADCALL |l| (QCDR |eqn|) (QREFELT $ 58))
  (QREFELT $ 20)))

(DEFUN |EQ;*;$$;28| (|eqn| |l| $)
  (SPADCALL
    (SPADCALL (QCAR |eqn|) |l| (QREFELT $ 58))
    (SPADCALL (QCDR |eqn|) |l| (QREFELT $ 58))
    (QREFELT $ 20)))

(DEFUN |EQ;One;$;29| ($)
  (SPADCALL (|spadConstant| $ 62) (|spadConstant| $ 62) (QREFELT $ 17)))

(DEFUN |EQ;recip;$U;30| (|eq| $)
  (PROG (|lh| |rh|)
    (RETURN
      (SEQ
        (LETT |lh|
          (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QREFELT $ 65))
          |EQ;recip;$U;30|)
        (EXIT
          (COND
            ((QEQCAR |lh| 1) (CONS 1 "failed"))
            ('T
              (SEQ
                (LETT |rh|
                  (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QREFELT $ 65))
                  |EQ;recip;$U;30|)
                (EXIT
                  (COND
                    ((QEQCAR |rh| 1) (CONS 1 "failed"))
                    ('T
                      (CONS 0
                        (CONS (QCDR |lh|) (QCDR |rh|))))))))))))))

(DEFUN |EQ;leftOne;$U;31| (|eq| $)
  (PROG (|re|)
    (RETURN
      (SEQ
        (LETT |re|
          (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QREFELT $ 65))
          |EQ;leftOne;$U;31|)
        (EXIT
          (COND
            ((QEQCAR |re| 1) (CONS 1 "failed"))
            ('T
              (CONS 0
                (SPADCALL
                  (|spadConstant| $ 62)

```



```

      (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QCDR |re|) (QREFELT $ 58))
      (QREFELT $ 20)))))))))

(DEFUN |EQ;rightOne;$U;32| (|eq| $)
  (PROG (|re|)
    (RETURN
      (SEQ
        (LETT |re|
          (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QREFELT $ 65))
          |EQ;rightOne;$U;32|)
        (EXIT
          (COND
            ((QEQCAR |re| 1) (CONS 1 "failed"))
            ('T
              (CONS 0
                (SPADCALL
                  (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QCDR |re|) (QREFELT $ 58))
                  (|spadConstant| $ 62)
                  (QREFELT $ 20)))))))))

(DEFUN |EQ;inv;2$;33| (|eq| $)
  (CONS (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QREFELT $ 69))
    (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QREFELT $ 69)))

(DEFUN |EQ;leftOne;$U;34| (|eq| $)
  (CONS 0
    (SPADCALL (|spadConstant| $ 62)
      (SPADCALL (SPADCALL |eq| (QREFELT $ 21))
        (SPADCALL (SPADCALL |eq| (QREFELT $ 21))
          (QREFELT $ 69))
          (QREFELT $ 58))
        (QREFELT $ 20))))

(DEFUN |EQ;rightOne;$U;35| (|eq| $)
  (CONS 0
    (SPADCALL
      (SPADCALL (SPADCALL |eq| (QREFELT $ 9))
        (SPADCALL (SPADCALL |eq| (QREFELT $ 21))
          (QREFELT $ 69))
          (QREFELT $ 58))
      (|spadConstant| $ 62) (QREFELT $ 20)))

(DEFUN |EQ;characteristic;Nni;36| ($) (SPADCALL (QREFELT $ 72)))

(DEFUN |EQ;*;I2$;37| (|i| |eq| $)
  (SPADCALL (SPADCALL |i| (QREFELT $ 75)) |eq| (QREFELT $ 60)))

```

```

(DEFUN |EQ;factorAndSplit;$L;38| (|eq| $)
  (PROG (#:G1488 #:G1489 |eq0| |p| #:G1490 |rcf| #:G1491)
    (RETURN
      (SEQ (COND
        ((|HasSignature| (QREFELT $ 6)
          (LIST '|factor|
            (LIST (LIST '|Factored|
              (|devaluate| (QREFELT $ 6)))
              (|devaluate| (QREFELT $ 6))))))
          (SEQ (LETT |eq0| (SPADCALL |eq| (QREFELT $ 8))
            |EQ;factorAndSplit;$L;38|)
            (EXIT (PROGN
              (LETT #:G1488 NIL |EQ;factorAndSplit;$L;38|)
              (SEQ (LETT |rcf| NIL
                |EQ;factorAndSplit;$L;38|)
                (LETT #:G1489
                  (SPADCALL
                    (SPADCALL
                      (SPADCALL |eq0| (QREFELT $ 9))
                      (QREFELT $ 11))
                      (QREFELT $ 15))
                    |EQ;factorAndSplit;$L;38|)
                  G190
                  (COND
                    ((OR (ATOM #:G1489)
                      (PROGN
                        (LETT |rcf| (CAR #:G1489)
                          |EQ;factorAndSplit;$L;38|)
                          NIL))
                      (GO G191))))
                    (SEQ (EXIT
                      (LETT #:G1488
                        (CONS
                          (SPADCALL (QCAR |rcf|)
                            (|spadConstant| $ 16)
                            (QREFELT $ 17))
                          #:G1488)
                          |EQ;factorAndSplit;$L;38|)))
                      (LETT #:G1489 (CDR #:G1489)
                        |EQ;factorAndSplit;$L;38|)
                      (GO G190) G191
                      (EXIT (NREVERSEO #:G1488))))))
                    ((EQUAL (QREFELT $ 6) (|Polynomial| (|Integer|)))
                    (SEQ (LETT |eq0| (SPADCALL |eq| (QREFELT $ 8))
                      |EQ;factorAndSplit;$L;38|)
                      (LETT |p| (SPADCALL |eq0| (QREFELT $ 9))
                        |EQ;factorAndSplit;$L;38|)
                      (EXIT (PROGN
                        (LETT #:G1490 NIL |EQ;factorAndSplit;$L;38|)
                        (SEQ (LETT |rcf| NIL

```

```

      |EQ;factorAndSplit;$L;38|)
(LETT #:G1491
  (SPADCALL
    (SPADCALL |p| (QREFELT $ 80))
    (QREFELT $ 83))
  |EQ;factorAndSplit;$L;38|)
G190
(COND
  ((OR (ATOM #:G1491)
    (PROGN
      (LETT |rcf| (CAR #:G1491)
        |EQ;factorAndSplit;$L;38|)
      NIL))
    (GO G191)))
  (SEQ (EXIT
    (LETT #:G1490
      (CONS
        (SPADCALL (QCAR |rcf|)
          (|spadConstant| $ 16)
          (QREFELT $ 17))
        #:G1490)
        |EQ;factorAndSplit;$L;38|)))
    (LETT #:G1491 (CDR #:G1491)
      |EQ;factorAndSplit;$L;38|)
    (GO G190) G191
    (EXIT (NREVERSEO #:G1490))))))
('T (LIST |eq|))))))

(DEFUN |EQ;differentiate;$S$;39| (|eq| |sym| $)
  (CONS (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) |sym| (QREFELT $ 84))
    (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) |sym| (QREFELT $ 84))))

(DEFUN |EQ;dimension;Cn;40| ($) (SPADCALL 2 (QREFELT $ 87)))

(DEFUN |EQ;/;3$;41| (|eq1| |eq2| $)
  (SPADCALL (SPADCALL (QCAR |eq1|) (QCAR |eq2|) (QREFELT $ 89))
    (SPADCALL (QCDR |eq1|) (QCDR |eq2|) (QREFELT $ 89))
    (QREFELT $ 20)))

(DEFUN |EQ;inv;2$;42| (|eq| $)
  (CONS (SPADCALL (SPADCALL |eq| (QREFELT $ 9)) (QREFELT $ 69))
    (SPADCALL (SPADCALL |eq| (QREFELT $ 21)) (QREFELT $ 69))))

(DEFUN |EQ;subst;3$;43| (|eq1| |eq2| $)
  (PROG (|eq3|)
    (RETURN
      (SEQ (LETT |eq3| |eq2| |EQ;subst;3$;43|)
        (EXIT (CONS (SPADCALL (SPADCALL |eq1| (QREFELT $ 9)) |eq3|
          (QREFELT $ 92))

```

```

(SPADCALL (SPADCALL |eq1| (QREFELT $ 21)) |eq3|
(QREFELT $ 92)))))))))

(DEFUN |Equation| (#:G1503)
  (PROG ()
    (RETURN
      (PROG (#:G1504)
        (RETURN
          (COND
            ((LETT #:G1504
              (|lassocShiftWithFunction|
                (LIST (|devaluate| #:G1503))
                (HGET |$ConstructorCache| '|Equation|)
                '|domainEqualList|)
              |Equation|)
              (|CDRwithIncrement| #:G1504))
            ('T
              (UNWIND-PROTECT
                (PROG1 (|Equation;| #:G1503)
                  (LETT #:G1504 T |Equation|))
                (COND
                  ((NOT #:G1504) (HREM |$ConstructorCache| '|Equation|))))))))))

(DEFUN |Equation;| (|#1|)
  (PROG (DV$1 |dv$| $ #:G1502 #:G1501 #:G1500 #:G1499 #:G1498 |pv$|)
    (RETURN
      (PROGN
        (LETT DV$1 (|devaluate| |#1|) |Equation|)
        (LETT |dv$| (LIST '|Equation| DV$1) |Equation|)
        (LETT $ (make-array 98) |Equation|)
        (QSETREFV $ 0 |dv$|)
        (QSETREFV $ 3
          (LETT |pv$|
            (|buildPredVector| 0 0
              (LIST (|HasCategory| |#1| '|(Field|)')
                (|HasCategory| |#1| '|(SetCategory|)')
                (|HasCategory| |#1| '|(Ring|)')
                (|HasCategory| |#1|
                  '|(PartialDifferentialRing| (|Symbol|))))
              (OR (|HasCategory| |#1|
                '|(PartialDifferentialRing|
                  (|Symbol|))
                (|HasCategory| |#1| '|(Ring|)')
                (|HasCategory| |#1| '|(Group|)')
                (|HasCategory| |#1|
                  (LIST '|InnerEvalable| '|(Symbol|)
                    (|devaluate| |#1|))))
                (AND (|HasCategory| |#1|
                  (LIST '|Evalable|
                    (|devaluate| |#1|))))

```

```

(|HasCategory| |#1| '(|SetCategory|)))
(|HasCategory| |#1| '(|IntegralDomain|))
(|HasCategory| |#1| '(|ExpressionSpace|))
(OR (|HasCategory| |#1| '(|Field|))
    (|HasCategory| |#1| '(|Group|)))
(OR (|HasCategory| |#1| '(|Group|))
    (|HasCategory| |#1| '(|Ring|)))
(LETT #:G1502
    (|HasCategory| |#1|
        '(|CommutativeRing|))
    |Equation|)
(OR #:G1502 (|HasCategory| |#1| '(|Field|))
    (|HasCategory| |#1| '(|Ring|)))
(OR #:G1502
    (|HasCategory| |#1| '(|Field|)))
(LETT #:G1501
    (|HasCategory| |#1| '(|Monoid|))
    |Equation|)
(OR (|HasCategory| |#1| '(|Group|))
    #:G1501)
(LETT #:G1500
    (|HasCategory| |#1| '(|SemiGroup|))
    |Equation|)
(OR (|HasCategory| |#1| '(|Group|)) #:G1501
    #:G1500)
(LETT #:G1499
    (|HasCategory| |#1|
        '(|AbelianGroup|))
    |Equation|)
(OR (|HasCategory| |#1|
    '(|PartialDifferentialRing|
        (|Symbol|)))
    #:G1499 #:G1502
    (|HasCategory| |#1| '(|Field|))
    (|HasCategory| |#1| '(|Ring|)))
(OR #:G1499 #:G1501)
(LETT #:G1498
    (|HasCategory| |#1|
        '(|AbelianSemiGroup|))
    |Equation|)
(OR (|HasCategory| |#1|
    '(|PartialDifferentialRing|
        (|Symbol|)))
    #:G1499 #:G1498 #:G1502
    (|HasCategory| |#1| '(|Field|))
    (|HasCategory| |#1| '(|Ring|)))
(OR (|HasCategory| |#1|
    '(|PartialDifferentialRing|
        (|Symbol|)))
    #:G1499 #:G1498 #:G1502

```

```

(|HasCategory| |#1| '(|Field|))
(|HasCategory| |#1| '(|Group|)) #:G1501
(|HasCategory| |#1| '(|Ring|)) #:G1500
(|HasCategory| |#1| '(|SetCategory|))))))
|Equation|))
(|haddProp| |$ConstructorCache| '|Equation| (LIST DV$1)
(CONS 1 $))
(|stuffDomainSlots| $)
(QSETREFV $ 6 |#1|)
(QSETREFV $ 7 (|Record| (|:| |lhs| |#1|) (|:| |rhs| |#1|)))
(COND
  ((|testBitVector| |pv$| 9)
   (QSETREFV $ 19
    (CONS (|dispatchFunction| |EQ;factorAndSplit;$L;1|) $))))
(COND
  ((|testBitVector| |pv$| 7)
   (PROGN
    (QSETREFV $ 27
     (CONS (|dispatchFunction| |EQ;eval;$SS$;8|) $))
    (QSETREFV $ 31
     (CONS (|dispatchFunction| |EQ;eval;$LL$;9|) $))))))
(COND
  ((|HasCategory| |#1| (LIST '|Evalable| (|devaluate| |#1|)))
   (PROGN
    (QSETREFV $ 34
     (CONS (|dispatchFunction| |EQ;eval;3$;10|) $))
    (QSETREFV $ 37
     (CONS (|dispatchFunction| |EQ;eval;$L$;11|) $))))))
(COND
  ((|testBitVector| |pv$| 2)
   (PROGN
    (QSETREFV $ 40
     (CONS (|dispatchFunction| |EQ;=;2$B;12|) $))
    (QSETREFV $ 44
     (CONS (|dispatchFunction| |EQ;coerce;$Of;13|) $))
    (QSETREFV $ 45
     (CONS (|dispatchFunction| |EQ;coerce;$B;14|) $))))))
(COND
  ((|testBitVector| |pv$| 23)
   (PROGN
    (QSETREFV $ 47 (CONS (|dispatchFunction| |EQ;+;3$;15|) $))
    (QSETREFV $ 48
     (CONS (|dispatchFunction| |EQ;+;S2$;16|) $))
    (QSETREFV $ 49
     (CONS (|dispatchFunction| |EQ;+;$S$;17|) $))))))
(COND
  ((|testBitVector| |pv$| 20)
   (PROGN
    (QSETREFV $ 51 (CONS (|dispatchFunction| |EQ;-;2$;18|) $))
    (QSETREFV $ 53

```

```

(CONS (|dispatchFunction| |EQ;-;S2$;19|) $))
(QSETREFV $ 54
(CONS (|dispatchFunction| |EQ;-;S$;20|) $))
(QSETREFV $ 57
(CONS (|dispatchFunction| |EQ;leftZero;2$;21|) $))
(QSETREFV $ 8
(CONS (|dispatchFunction| |EQ;rightZero;2$;22|) $))
(QSETREFV $ 55
(CONS IDENTITY
(FUNCALL (|dispatchFunction| |EQ;Zero;$;23|) $)))
(QSETREFV $ 52 (CONS (|dispatchFunction| |EQ;-;3$;24|) $))))))
(COND
((|testBitVector| |pv$| 18)
(PROGN
(QSETREFV $ 59 (CONS (|dispatchFunction| |EQ;*;3$;25|) $))
(QSETREFV $ 60
(CONS (|dispatchFunction| |EQ;*;S2$;26|) $))
(QSETREFV $ 60
(CONS (|dispatchFunction| |EQ;*;S2$;27|) $))
(QSETREFV $ 61
(CONS (|dispatchFunction| |EQ;*;S$;28|) $))))))
(COND
((|testBitVector| |pv$| 16)
(PROGN
(QSETREFV $ 63
(CONS IDENTITY
(FUNCALL (|dispatchFunction| |EQ;One;$;29|) $)))
(QSETREFV $ 66
(CONS (|dispatchFunction| |EQ;recip;$U;30|) $))
(QSETREFV $ 67
(CONS (|dispatchFunction| |EQ;leftOne;$U;31|) $))
(QSETREFV $ 68
(CONS (|dispatchFunction| |EQ;rightOne;$U;32|) $))))))
(COND
((|testBitVector| |pv$| 6)
(PROGN
(QSETREFV $ 70
(CONS (|dispatchFunction| |EQ;inv;2$;33|) $))
(QSETREFV $ 67
(CONS (|dispatchFunction| |EQ;leftOne;$U;34|) $))
(QSETREFV $ 68
(CONS (|dispatchFunction| |EQ;rightOne;$U;35|) $))))))
(COND
((|testBitVector| |pv$| 3)
(PROGN
(QSETREFV $ 73
(CONS (|dispatchFunction| |EQ;characteristic;Nni;36|)
$))
(QSETREFV $ 76
(CONS (|dispatchFunction| |EQ;*;I2$;37|) $))))))

```

```

(COND
  ((|testBitVector| |pv$| 9)
   (QSETREFV $ 19
    (CONS (|dispatchFunction| |EQ;factorAndSplit;$L;38|) $))))
(COND
  ((|testBitVector| |pv$| 4)
   (QSETREFV $ 85
    (CONS (|dispatchFunction| |EQ;differentiate;$S$;39|) $))))
(COND
  ((|testBitVector| |pv$| 1)
   (PROGN
    (QSETREFV $ 88
     (CONS (|dispatchFunction| |EQ;dimension;Cn;40|) $))
    (QSETREFV $ 90 (CONS (|dispatchFunction| |EQ;/;3$;41|) $))
    (QSETREFV $ 70
     (CONS (|dispatchFunction| |EQ;inv;2$;42|) $))))))
(COND
  ((|testBitVector| |pv$| 10)
   (QSETREFV $ 93
    (CONS (|dispatchFunction| |EQ;subst;3$;43|) $))))
$))))

(setf (get 'Equation| 'infovec|)
  (LIST '#(NIL NIL NIL NIL NIL (|local| |#1|) 'Rep|
    (0 . |rightZero|) |EQ;lhs;$S;4| (|Factored| $)
    (5 . |factor|)
    (|Record| (|:| |factor| 6) (|:| |exponent| 74))
    (|List| 12) (|Factored| 6) (10 . |factors|) (15 . |Zero|)
    |EQ;equation;2S$;3| (|List| $) (19 . |factorAndSplit|)
    |EQ;=;2S$;2| |EQ;rhs;$S;5| |EQ;swap;2$;6| (|Mapping| 6 6)
    |EQ;map;M2$;7| (|Symbol|) (24 . |eval|) (31 . |eval|)
    (|List| 25) (|List| 6) (38 . |eval|) (45 . |eval|)
    (|Equation| 6) (52 . |eval|) (58 . |eval|) (|List| 32)
    (64 . |eval|) (70 . |eval|) (|Boolean|) (76 . =) (82 . =)
    (|OutputForm|) (88 . |coerce|) (93 . =) (99 . |coerce|)
    (104 . |coerce|) (109 . +) (115 . +) (121 . +) (127 . +)
    (133 . -) (138 . -) (143 . -) (149 . -) (155 . -)
    (161 . |Zero|) (165 . -) (171 . |leftZero|) (176 . *)
    (182 . *) (188 . *) (194 . *) (200 . |One|) (204 . |One|)
    (|Union| $ "failed") (208 . |recip|) (213 . |recip|)
    (218 . |leftOne|) (223 . |rightOne|) (228 . |inv|)
    (233 . |inv|) (|NonNegativeInteger|)
    (238 . |characteristic|) (242 . |characteristic|)
    (|Integer|) (246 . |coerce|) (251 . *) (|Factored| 78)
    (|Polynomial| 74)
    (|MultivariateFactorize| 25 (|IndexedExponents| 25) 74 78)
    (257 . |factor|)
    (|Record| (|:| |factor| 78) (|:| |exponent| 74))
    (|List| 81) (262 . |factors|) (267 . |differentiate|)
    (273 . |differentiate|) (|CardinalNumber|)

```



```

(279 . |coerce|) (284 . |dimension|) (288 . /) (294 . /)
(|Equation| $) (300 . |subst|) (306 . |subst|)
(|PositiveInteger|) (|List| 71) (|SingleInteger|)
(|String|))
'#(= 312 |zero?| 318 |swap| 323 |subtractIfCan| 328 |subst|
334 |sample| 340 |rightZero| 344 |rightOne| 349 |rhs| 354
|recip| 359 |one?| 364 |map| 369 |lhs| 375 |leftZero| 380
|leftOne| 385 |latex| 390 |inv| 395 |hash| 400
|factorAndSplit| 405 |eval| 410 |equation| 436 |dimension|
442 |differentiate| 446 |conjugate| 472 |commutator| 478
|coerce| 484 |characteristic| 499 ^ 503 |Zero| 521 |One|
525 D 529 = 555 / 567 - 579 + 602 ** 620 * 638)
'((|unitsKnown| . 12) (|rightUnitary| . 3)
(|leftUnitary| . 3))
(CONS (|makeByteWordVec2| 25
      '(1 15 4 14 5 14 3 5 3 21 21 6 21 17 24 19 25 0 2
        25 2 7))
      (CONS '#(|VectorSpace&| |Module&|
                |PartialDifferentialRing&| NIL |Ring&| NIL NIL
                NIL NIL |AbelianGroup&| NIL |Group&|
                |AbelianMonoid&| |Monoid&| |AbelianSemiGroup&|
                |SemiGroup&| |SetCategory&| NIL NIL
                |BasicType&| NIL |InnerEvalable&|)
            (CONS '#(|VectorSpace| 6) (|Module| 6)
                  (|PartialDifferentialRing| 25)
                  (|BiModule| 6 6) (|Ring|)
                  (|LeftModule| 6) (|RightModule| 6)
                  (|Rng|) (|LeftModule| $$)
                  (|AbelianGroup|)
                  (|CancellationAbelianMonoid|) (|Group|)
                  (|AbelianMonoid|) (|Monoid|)
                  (|AbelianSemiGroup|) (|SemiGroup|)
                  (|SetCategory|) (|Type|)
                  (|CoercibleTo| 41) (|BasicType|)
                  (|CoercibleTo| 38)
                  (|InnerEvalable| 25 6))
            (|makeByteWordVec2| 97
              '(1 0 0 0 8 1 6 10 0 11 1 14 13 0 15 0
                6 0 16 1 0 18 0 19 3 6 0 0 25 6 26 3
                0 0 0 25 6 27 3 6 0 0 28 29 30 3 0 0
                0 28 29 31 2 6 0 0 32 33 2 0 0 0 0 34
                2 6 0 0 35 36 2 0 0 0 18 37 2 6 38 0
                0 39 2 0 38 0 0 40 1 6 41 0 42 2 41 0
                0 0 43 1 0 41 0 44 1 0 38 0 45 2 6 0
                0 0 46 2 0 0 0 0 47 2 0 0 6 0 48 2 0
                0 0 6 49 1 6 0 0 50 1 0 0 0 51 2 0 0
                0 0 52 2 0 0 6 0 53 2 0 0 0 6 54 0 0
                0 55 2 6 0 0 0 56 1 0 0 0 57 2 6 0 0
                0 58 2 0 0 0 0 59 2 0 0 6 0 60 2 0 0
                0 6 61 0 6 0 62 0 0 0 63 1 6 64 0 65

```

```

1 0 64 0 66 1 0 64 0 67 1 0 64 0 68 1
6 0 0 69 1 0 0 0 70 0 6 71 72 0 0 71
73 1 6 0 74 75 2 0 0 74 0 76 1 79 77
78 80 1 77 82 0 83 2 6 0 0 25 84 2 0
0 0 25 85 1 86 0 71 87 0 0 86 88 2 6
0 0 0 89 2 0 0 0 0 90 2 6 0 0 91 92 2
0 0 0 0 93 2 2 38 0 0 1 1 20 38 0 1 1
0 0 0 22 2 20 64 0 0 1 2 10 0 0 0 93
0 22 0 1 1 20 0 0 8 1 16 64 0 68 1 0
6 0 21 1 16 64 0 66 1 16 38 0 1 2 0 0
23 0 24 1 0 6 0 9 1 20 0 0 57 1 16 64
0 67 1 2 97 0 1 1 11 0 0 70 1 2 96 0
1 1 9 18 0 19 2 8 0 0 0 34 2 8 0 0 18
37 3 7 0 0 25 6 27 3 7 0 0 28 29 31 2
0 0 6 6 17 0 1 86 88 2 4 0 0 28 1 2 4
0 0 25 85 3 4 0 0 28 95 1 3 4 0 0 25
71 1 2 6 0 0 0 1 2 6 0 0 0 1 1 3 0 74
1 1 2 41 0 44 1 2 38 0 45 0 3 71 73 2
6 0 0 74 1 2 16 0 0 71 1 2 18 0 0 94
1 0 20 0 55 0 16 0 63 2 4 0 0 28 1 2
4 0 0 25 1 3 4 0 0 28 95 1 3 4 0 0 25
71 1 2 2 38 0 0 40 2 0 0 6 6 20 2 11
0 0 0 90 2 1 0 0 6 1 1 20 0 0 51 2 20
0 0 0 52 2 20 0 6 0 53 2 20 0 0 6 54
2 23 0 0 0 47 2 23 0 6 0 48 2 23 0 0
6 49 2 6 0 0 74 1 2 16 0 0 71 1 2 18
0 0 94 1 2 20 0 71 0 1 2 20 0 74 0 76
2 23 0 94 0 1 2 18 0 0 0 59 2 18 0 0
6 61 2 18 0 6 0 60))))))

```

```

'|lookupComplete|))

```

2.5 The code.o file

The Spad compiler translates the Spad language into Common Lisp. It eventually invokes the Common Lisp “compile-file” command to output files in binary. Depending on the lisp system this filename can vary (e.g “code.fasl”). The details of how these are used depends on the Common Lisp in use.

By default, Axiom uses Gnu Common Lisp (GCL), which generates “.o” files.

2.6 The info file

```

(((* (($ $ $) (|arguments| (|eq2| . $) (|eq1| . $)) (S (* S S S))
($ (= $ S S)))
(($ $ $ S) (|arguments| (|l| . S) (|eqn| . $)) (S (* S S S))
($ (= $ S S)))

```

```

(($ #0=(|Integer|) $) (|arguments| (|l| . #0#) (|eq| . $))
  (S (|coerce| S (|Integer|))) ($ (* $ S $)))
(($ S $) (|arguments| (|l| . S) (|eqn| . $)) (S (* S S S))
  ($ (= $ S S)))
(+ (($ $ $) (|arguments| (|eq2| . $) (|eq1| . $)) (S (+ S S S))
  ($ (= $ S S)))
(($ $ $) (|arguments| (|s| . S) (|eq1| . $)) ($ (+ $ $ $)))
(($ S $) (|arguments| (|s| . S) (|eq2| . $)) ($ (+ $ $ $)))
(- (($ $ $) (|arguments| (|eq2| . $) (|eq1| . $)) (S (- S S S))
  ($ (= $ S S)))
(($ $ $) (|arguments| (|s| . S) (|eq1| . $)) ($ (- $ $ $)))
(($ $) (|arguments| (|eq| . $)) (S (- S S))
  ($ (|rhs| S $) (|lhs| S $) (= $ S S)))
(($ S $) (|arguments| (|s| . S) (|eq2| . $)) ($ (- $ $ $)))
(/ (($ $ $) (|arguments| (|eq2| . $) (|eq1| . $)) (S (/ S S S))
  ($ (= $ S S)))
(= (($ S S) (|arguments| (|r| . S) (|l| . S)))
  (((|Boolean|) $ $) ((|Boolean|) (|false| (|Boolean|)))
    (|locals| (#:G1393 |Boolean|))
    (|arguments| (|eq2| . $) (|eq1| . $)) (S (= (|Boolean|) S S))))
(|One| (($) (S (|One| S)) ($ (|equation| $ S S))))
(|Zero| (($) (S (|Zero| S)) ($ (|equation| $ S S))))
(|characteristic|
  (((|NonNegativeInteger|)
    (S (|characteristic| (|NonNegativeInteger|)))))
(|coerce|
  (((|Boolean|) $) (|arguments| (|eqn| . $))
    (S (= (|Boolean|) S S)))
  (((|OutputForm|) $)
    ((|OutputForm|) (= (|OutputForm|) (|OutputForm|) (|OutputForm|)))
    (|arguments| (|eqn| . $)) (S (|coerce| (|OutputForm|) S))))
(|constructor|
  (NIL (|locals|
    (|Rep| |Join| (|SetCategory|)
      (CATEGORY |domain|
        (SIGNATURE |construct|
          ((|Record| (|:| |lhs| S) (|:| |rhs| S)) S)
          S))
        (SIGNATURE |coerce|
          ((|OutputForm|)
            (|Record| (|:| |lhs| S) (|:| |rhs| S))))
          (SIGNATURE |elt|
            (S (|Record| (|:| |lhs| S) (|:| |rhs| S))
              "lhs"))
          (SIGNATURE |elt|
            (S (|Record| (|:| |lhs| S) (|:| |rhs| S))
              "rhs"))
          (SIGNATURE |setelt|
            (S (|Record| (|:| |lhs| S) (|:| |rhs| S))
              "lhs" S))

```

```

(SIGNATURE |setelt|
  (S (|Record| (|:| |lhs| S) (|:| |rhs| S))
    "rhs" S))
(SIGNATURE |copy|
  ((|Record| (|:| |lhs| S) (|:| |rhs| S))
    (|Record| (|:| |lhs| S) (|:| |rhs| S))))))
(|differentiate|
  (($ $ #1=(|Symbol|)) (|arguments| (|sym| . #1#) (|eq| . $))
    (S (|differentiate| S S (|Symbol|)) ($ (|rhs| S $) (|lhs| S $))))
(|dimension|
  ((#2=(|CardinalNumber|))
    (#2# (|coerce| (|CardinalNumber|) (|NonNegativeInteger|))))
(|equation| (($ S S) (|arguments| (|r| . S) (|l| . S))))
(|eval| (($ $ $) (|arguments| (|eqn2| . $) (|eqn1| . $))
  (S (|eval| S S (|Equation| S)) ($ (= $ S S)))
  (($ $ #3=(|List| $))
    (|arguments| (|eqn2| . #3#) (|eqn1| . $))
    (S (|eval| S S (|List| (|Equation| S)))) ($ (= $ S S)))
  (($ $ #4=(|List| #5=(|Symbol|)) #6=(|List| S))
    (|arguments| (|lx| . #6#) (|ls| . #4#) (|eqn| . $))
    (S (|eval| S S (|List| (|Symbol|)) (|List| S)))
    ($ (= $ S S)))
  (($ $ #5# S) (|arguments| (|x| . S) (|s| . #5#) (|eqn| . $))
    (S (|eval| S S (|Symbol| S)) ($ (= $ S S))))
(|factorAndSplit|
  (((|List| $) $)
    ((|MultivariateFactorize| (|Symbol|)
      (|IndexedExponents| (|Symbol|)) (|Integer|)
      (|Polynomial| (|Integer|)))
      (|factor| (|Factored| (|Polynomial| (|Integer|))
        (|Polynomial| (|Integer|))))
      (|Factored| S)
      (|factors|
        (|List| (|Record| (|:| |factor| S)
          (|:| |exponent| (|Integer|))))
        (|Factored| S)))
      (|Factored| (|Polynomial| (|Integer|))
        (|factors|
          (|List| (|Record| (|:| |factor| (|Polynomial| (|Integer|))
            (|:| |exponent| (|Integer|))))
            (|Factored| (|Polynomial| (|Integer|))))
          (|locals| (|p| |Polynomial| (|Integer|)) (|eq0| . $))
          (|arguments| (|eq| . $))
          (S (|factor| (|Factored| S) S) (|Zero| S))
          ($ (|rightZero| $ $) (|lhs| S $) (|equation| $ S S))))
      (|inv| (($ $) (|arguments| (|eq| . $)) (S (|inv| S S))
        ($ (|rhs| S $) (|lhs| S $))))
      (|leftOne|
        (((|Union| $ "failed") $) (|locals| (|re| |Union| S "failed"))
          (|arguments| (|eq| . $))

```

```

(S (|recip| (|Union| S "failed") S) (|inv| S S) (|One| S)
  (* S S S))
($ (|rhs| S $) (|lhs| S $) (|One| $) (= $ S S)))
(|leftZero|
  (($ $) (|arguments| (|eq| . $)) (S (|Zero| S) (- S S S))
    ($ (|rhs| S $) (|lhs| S $) (|Zero| $) (= $ S S)))
  (|lhs| ((S $) (|arguments| (|eqn| . $))))
  (|map| (($ #7=(|Mapping| S S) $)
    (|arguments| (|fn| . #7#) (|eqn| . $)) ($ (|equation| $ S S))))
  (|recip| (((|Union| $ "failed") $)
    (|locals| (|rh| |Union| S "failed")
      (|lh| |Union| S "failed"))
    (|arguments| (|eq| . $))
    (S (|recip| (|Union| S "failed") S))
    ($ (|rhs| S $) (|lhs| S $))))
  (|rhs| ((S $) (|arguments| (|eqn| . $))))
  (|rightOne|
    (((|Union| $ "failed") $) (|locals| (|re| |Union| S "failed"))
      (|arguments| (|eq| . $))
      (S (|recip| (|Union| S "failed") S) (|inv| S S) (|One| S)
        (* S S S))
      ($ (|rhs| S $) (|lhs| S $) (= $ S S))))
  (|rightZero|
    (($ $) (|arguments| (|eq| . $)) (S (|Zero| S) (- S S S))
      ($ (|rhs| S $) (|lhs| S $) (= $ S S)))
    (|subst| (($ $ $) (|locals| (|eq3| |Equation| S))
      (|arguments| (|eq2| . $) (|eq1| . $))
      (S (|subst| S S (|Equation| S)))
      ($ (|rhs| S $) (|lhs| S $))))
    (|swap| (($ $) (|arguments| (|eqn| . $)) ($ (|rhs| S $) (|lhs| S $)))))

```

2.7 The EQ.fn file

```

(in-package 'compiler)(init-fn)
(ADD-FN-DATA '(
#S(FN NAME BOOT::|EQ;*;S2$;26| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;rightOne;$U;32| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (BOOT::|spadConstant| VMLISP:QCDR CONS VMLISP:QCAR EQL
    BOOT::|EQQCAR COND VMLISP:EXIT CDR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL BOOT::|LETT VMLISP:SEQ RETURN)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QCDR VMLISP:QCAR BOOT::|EQQCAR COND

```

```

        VMLISP:EXIT VMLISP:QREFELT BOOT:SPADCALL BOOT::LETT
        VMLISP:SEQ RETURN))
#S(FN NAME BOOT::|EQ;lhs;$S;4| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CAR VMLISP:QCAR) RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT
  NIL MACROS (VMLISP:QCAR))
#S(FN NAME BOOT::|EQ;+;3$;15| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;dimension;Cn;40| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;rightZero;2$;22| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (BOOT::|spadConstant| CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;coerce;$0f;13| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;One;$;29| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (CDR BOOT::|spadConstant| CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;inv;2$;42| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL CONS)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;-;$S$;20| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CONS CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;=;2$B;12| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL COND)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL COND))
#S(FN NAME BOOT::|EQ;/;3$;41| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS

```

```

(VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;recip;$U;30| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR LIST* CONS VMLISP:QCAR EQL BOOT::QEQCAR COND
    VMLISP:EXIT CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL
    BOOT::LETT VMLISP:SEQ RETURN)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR BOOT::QEQCAR COND VMLISP:EXIT
    VMLISP:QREFELT BOOT:SPADCALL BOOT::LETT VMLISP:SEQ RETURN))
#S(FN NAME BOOT::|EQ;-;3$;24| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;eval;$L$;11| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;leftZero;2$;21| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (CDR BOOT::|spadConstant| CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;*;S2$;27| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;*;I2$;37| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL) RETURN-TYPE
  NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;eval;3$;10| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;eval;$SS$;8| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;factorAndSplit;$L;38| DEF DEFUN VALUE-TYPE T
  FUN-VALUES NIL CALLEES

```

```

(BOOT:|Integer| BOOT:|Polynomial| EQUAL BOOT:NREVERSEO
  BOOT::|spadConstant| VMLISP:QCAR CONS ATOM VMLISP:EXIT CDR
  CAR BOOT:SPADCALL BOOT::LETT BOOT::|devaluate| LIST SVREF
  VMLISP:QREFELT BOOT::|HasSignature| COND VMLISP:SEQ RETURN)
RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
(BOOT::|spadConstant| VMLISP:QCAR VMLISP:EXIT BOOT:SPADCALL
  BOOT::LETT VMLISP:QREFELT COND VMLISP:SEQ RETURN))
#S(FN NAME BOOT::|EQ;differentiate;$S$;39| DEF DEFUN VALUE-TYPE T
  FUN-VALUES NIL CALLEES
  (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL CONS) RETURN-TYPE NIL
  ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;eval;$LL$;9| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;leftOne;$U;34| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (CDR BOOT::|spadConstant| CAR SVREF VMLISP:QREFELT BOOT:SPADCALL
    CONS)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;map;M2$;7| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
    BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;-;$S$;19| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CONS CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;equation;2S$;3| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES (CONS) RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL
  MACROS NIL)
#S(FN NAME BOOT::|EQ;+;$S$;17| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CONS CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;factorAndSplit;$L;1| DEF DEFUN VALUE-TYPE T
  FUN-VALUES NIL CALLEES
  (BOOT:NREVERSEO BOOT::|spadConstant| VMLISP:QCAR CONS ATOM
    VMLISP:EXIT CDR CAR BOOT:SPADCALL BOOT::LETT
    BOOT::|devaluate| LIST SVREF VMLISP:QREFELT
    BOOT::|HasSignature| COND VMLISP:SEQ RETURN)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QCAR VMLISP:EXIT BOOT:SPADCALL
    BOOT::LETT VMLISP:QREFELT COND VMLISP:SEQ RETURN))

```



```

#S(FN NAME BOOT::|EQ;*;3$;25| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;Zero;$;23| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (CDR BOOT::|spadConstant| CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T) NO-EMIT NIL MACROS
  (BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;characteristic;Nni;36| DEF DEFUN VALUE-TYPE T
  FUN-VALUES NIL CALLEES
  (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL) RETURN-TYPE NIL
  ARG-TYPES (T) NO-EMIT NIL MACROS (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;leftOne;$U;31| DEF DEFUN VALUE-TYPE T FUN-VALUES
  NIL CALLEES
  (VMLISP:QCDR BOOT::|spadConstant| CONS VMLISP:QCAR EQL
   BOOT::|EQQCAR COND VMLISP:EXIT CDR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL BOOT::LETT VMLISP:SEQ RETURN)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR BOOT::|spadConstant| VMLISP:QCAR BOOT::|EQQCAR COND
   VMLISP:EXIT VMLISP:QREFELT BOOT:SPADCALL BOOT::LETT
   VMLISP:SEQ RETURN))
#S(FN NAME BOOT::|EQ;swap;2$;6| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL CONS)
  RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;-;2$;18| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL) RETURN-TYPE
  NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;subst;3$;43| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL CONS VMLISP:EXIT
   BOOT::LETT VMLISP:SEQ RETURN)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QREFELT BOOT:SPADCALL VMLISP:EXIT BOOT::LETT VMLISP:SEQ
   RETURN))
#S(FN NAME BOOT::|EQ;=;2S$;2| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CONS) RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL
  MACROS NIL)
#S(FN NAME BOOT::|EQ;+;$S$;28| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES
  (VMLISP:QCDR CDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
   BOOT:SPADCALL)
  RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
  (VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;+;S2$;16| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
  CALLEES (CDR CONS CAR SVREF VMLISP:QREFELT BOOT:SPADCALL)

```

```

RETURN-TYPE NIL ARG-TYPES (T T T) NO-EMIT NIL MACROS
(VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|Equation;| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
CALLEES
(BOOT::|EQ;One;$;29| BOOT::|EQ;Zero;$;23|
  BOOT::|dispatchFunction| BOOT::|testBitVector| COND
  BOOT::|Record0| BOOT::|Record| BOOT::|stuffDomainSlots| CONS
  BOOT::|haddProp| BOOT::|HasCategory| BOOT::|buildPredVector|
  SYSTEM:SVSET SETF VMLISP:QSETREFV LIST
  BOOT::|devaluate| BOOT::LETT RETURN)
RETURN-TYPE NIL ARG-TYPES (T) NO-EMIT NIL MACROS
(BOOT::|dispatchFunction| COND BOOT::|Record| SETF
  VMLISP:QSETREFV BOOT::LETT RETURN))
#S(FN NAME BOOT::|EQ;coerce;$B;14| DEF DEFUN VALUE-TYPE T FUN-VALUES
NIL CALLEES
(CDR VMLISP:QCDR VMLISP:QCAR CAR SVREF VMLISP:QREFELT
  BOOT:SPADCALL)
RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
(VMLISP:QCDR VMLISP:QCAR VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;rhs;$S;5| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
CALLEES (CDR VMLISP:QCDR) RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT
NIL MACROS (VMLISP:QCDR))
#S(FN NAME OTHER-FORM DEF NIL VALUE-TYPE NIL FUN-VALUES NIL CALLEES NIL
RETURN-TYPE NIL ARG-TYPES NIL NO-EMIT NIL MACROS NIL)
#S(FN NAME BOOT::|EQ;inv;2$;33| DEF DEFUN VALUE-TYPE T FUN-VALUES NIL
CALLEES (CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL CONS)
RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
(VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|EQ;rightOne;$U;35| DEF DEFUN VALUE-TYPE T FUN-VALUES
NIL CALLEES
(BOOT::|spadConstant| CDR CAR SVREF VMLISP:QREFELT BOOT:SPADCALL
  CONS)
RETURN-TYPE NIL ARG-TYPES (T T) NO-EMIT NIL MACROS
(BOOT::|spadConstant| VMLISP:QREFELT BOOT:SPADCALL))
#S(FN NAME BOOT::|Equation| DEF DEFUN VALUE-TYPE T FUN-VALUES
(SINGLE-VALUE) CALLEES
(REMHASH VMLISP:HREM BOOT::|Equation;| PROG1
  BOOT::|CDRwithIncrement| GETHASH VMLISP:HGET
  BOOT::|devaluate| LIST BOOT::|lassocShiftWithFunction|
  BOOT::LETT COND RETURN)
RETURN-TYPE NIL ARG-TYPES (T) NO-EMIT NIL MACROS
(VMLISP:HREM PROG1 VMLISP:HGET BOOT::LETT COND RETURN))

```

2.8 The index.kaf file

Each constructor (e.g. EQ) had one library directory (e.g. EQ.nrlib). This directory contained a random access file called the index.kaf file. These files contain runtime information such as the operationAlist and the ConstructorModemap. At system build time we merge all

of these `.nrllib/index.kaf` files into one database, `INTERP.daase`. Requests to get information from this database are cached so that multiple references do not cause additional disk i/o.

Before getting into the contents, we need to understand the format of an `index.kaf` file. The `kaf` file is a random access file, originally used as a database. In the current system we make a pass to combine these files at build time to construct the various `daase` files.

This is just a file of lisp objects, one after another, in `(read)` format.

A `kaf` file starts with an integer, in this case, 35695. This integer gives the byte offset to the index. Due to the way the file is constructed, the index is at the end of the file. To read a `kaf` file, first read the integer, then seek to that location in the file, and do a `(read)`. This will return the index, in this case:

```
((("slot1Info" 0 32444)
  ("documentation" 0 29640)
  ("ancestors" 0 28691)
  ("parents" 0 28077)
  ("abbreviation" 0 28074)
  ("predicates" 0 25442)
  ("attributes" 0 25304)
  ("signaturesAndLocals" 0 23933)
  ("superDomain" 0 NIL)
  ("operationAlist" 0 20053)
  ("modemaps" 0 17216)
  ("sourceFile" 0 17179)
  ("constructorCategory" 0 15220)
  ("constructorModemap" 0 13215)
  ("constructorKind" 0 13206)
  ("constructorForm" 0 13191)
  ("compilerInfo" 0 4433)
  ("loadTimeStuff" 0 20))
```

This is a list of triples. The first item in each triple is a string that is used as a lookup key (e.g. `"operationAlist"`). The second element is no longer used. The third element is the byte offset from the beginning of the file.

So to read the `"operationAlist"` from this file you would:

1. open the `index.kaf` file
2. `(read)` the integer
3. `(seek)` to the integer offset from the beginning of the file
4. `(read)` the index of triples
5. find the keyword (e.g. `"operationAlist"`) triple
6. select the third element, an integer
7. `(seek)` to the integer offset from the beginning of the file

8. (read) the “operationAlist”

Note that the information below has been reformatted to fit this document. In order to save space the index.kaf file does not use prettyprint since it is normally only read by machine.

The index offset byte

35695

The “loadTimeStuff”

```
(setf (get 'Equation| 'infovec|)
      (LIST '#(NIL NIL NIL NIL NIL NIL (|local| |#1|) 'Rep|
              (0 . |rightZero|) |EQ;lhs;$S;4| (|Factored| $)
              (5 . |factor|)
              (|Record| (|:| |factor| 6) (|:| |exponent| 74))
              (|List| 12) (|Factored| 6) (10 . |factors|) (15 . |Zero|)
              |EQ;equation;2S$;3| (|List| $) (19 . |factorAndSplit|)
              |EQ;=;2S$;2| |EQ;rhs;$S;5| |EQ;swap;2$;6| (|Mapping| 6 6)
              |EQ;map;M2$;7| (|Symbol|) (24 . |eval|) (31 . |eval|)
              (|List| 25) (|List| 6) (38 . |eval|) (45 . |eval|)
              (|Equation| 6) (52 . |eval|) (58 . |eval|) (|List| 32)
              (64 . |eval|) (70 . |eval|) (|Boolean|) (76 . =) (82 . =)
              (|OutputForm|) (88 . |coerce|) (93 . =) (99 . |coerce|)
              (104 . |coerce|) (109 . +) (115 . +) (121 . +) (127 . +)
              (133 . -) (138 . -) (143 . -) (149 . -) (155 . -)
              (161 . |Zero|) (165 . -) (171 . |leftZero|) (176 . *)
              (182 . *) (188 . *) (194 . *) (200 . |One|) (204 . |One|)
              (|Union| $ "failed") (208 . |recip|) (213 . |recip|)
              (218 . |leftOne|) (223 . |rightOne|) (228 . |inv|)
              (233 . |inv|) (|NonNegativeInteger|)
              (238 . |characteristic|) (242 . |characteristic|)
              (|Integer|) (246 . |coerce|) (251 . *) (|Factored| 78)
              (|Polynomial| 74)
              (|MultivariateFactorize| 25 (|IndexedExponents| 25) 74 78)
              (257 . |factor|)
              (|Record| (|:| |factor| 78) (|:| |exponent| 74))
              (|List| 81) (262 . |factors|) (267 . |differentiate|)
              (273 . |differentiate|) (|CardinalNumber|)
              (279 . |coerce|) (284 . |dimension|) (288 . /) (294 . /)
              (|Equation| $) (300 . |subst|) (306 . |subst|)
              (|PositiveInteger|) (|List| 71) (|SingleInteger|)
              (|String|))
      '#(~= 312 |zero?| 318 |swap| 323 |subtractIfCan| 328 |subst|
          334 |sample| 340 |rightZero| 344 |rightOne| 349 |rhs| 354
          |recip| 359 |one?| 364 |map| 369 |lhs| 375 |leftZero| 380
          |leftOne| 385 |latex| 390 |inv| 395 |hash| 400
          |factorAndSplit| 405 |eval| 410 |equation| 436 |dimension|
```

```

442 |differentiate| 446 |conjugate| 472 |commutator| 478
|coerce| 484 |characteristic| 499 ^ 503 |Zero| 521 |One|
525 D 529 = 555 / 567 - 579 + 602 ** 620 * 638)
'((|unitsKnown| . 12) (|rightUnitary| . 3)
(|leftUnitary| . 3))
(CONS (|makeByteWordVec2| 25
      '(1 15 4 14 5 14 3 5 3 21 21 6 21 17 24 19 25 0 2
        25 2 7))
      (CONS '#(|VectorSpace&| |Module&|
                |PartialDifferentialRing&| NIL |Ring&| NIL NIL
                NIL NIL |AbelianGroup&| NIL |Group&|
                |AbelianMonoid&| |Monoid&| |AbelianSemiGroup&|
                |SemiGroup&| |SetCategory&| NIL NIL
                |BasicType&| NIL |InnerEvalable&|)
            (CONS '#(|VectorSpace| 6) (|Module| 6)
                  (|PartialDifferentialRing| 25)
                  (|BiModule| 6 6) (|Ring|)
                  (|LeftModule| 6) (|RightModule| 6)
                  (|Rng|) (|LeftModule| $$)
                  (|AbelianGroup|)
                  (|CancellationAbelianMonoid|) (|Group|)
                  (|AbelianMonoid|) (|Monoid|)
                  (|AbelianSemiGroup|) (|SemiGroup|)
                  (|SetCategory|) (|Type|)
                  (|CoercibleTo| 41) (|BasicType|)
                  (|CoercibleTo| 38)
                  (|InnerEvalable| 25 6))
            (|makeByteWordVec2| 97
              '(1 0 0 0 8 1 6 10 0 11 1 14 13 0 15 0
                6 0 16 1 0 18 0 19 3 6 0 0 25 6 26 3
                0 0 0 25 6 27 3 6 0 0 28 29 30 3 0 0
                0 28 29 31 2 6 0 0 32 33 2 0 0 0 0 34
                2 6 0 0 35 36 2 0 0 0 18 37 2 6 38 0
                0 39 2 0 38 0 0 40 1 6 41 0 42 2 41 0
                0 0 43 1 0 41 0 44 1 0 38 0 45 2 6 0
                0 0 46 2 0 0 0 0 47 2 0 0 6 0 48 2 0
                0 0 6 49 1 6 0 0 50 1 0 0 0 51 2 0 0
                0 0 52 2 0 0 6 0 53 2 0 0 0 6 54 0 0
                0 55 2 6 0 0 0 56 1 0 0 0 57 2 6 0 0
                0 58 2 0 0 0 0 59 2 0 0 6 0 60 2 0 0
                0 6 61 0 6 0 62 0 0 0 63 1 6 64 0 65
                1 0 64 0 66 1 0 64 0 67 1 0 64 0 68 1
                6 0 0 69 1 0 0 0 70 0 6 71 72 0 0 71
                73 1 6 0 74 75 2 0 0 74 0 76 1 79 77
                78 80 1 77 82 0 83 2 6 0 0 25 84 2 0
                0 0 25 85 1 86 0 71 87 0 0 86 88 2 6
                0 0 0 89 2 0 0 0 0 90 2 6 0 0 91 92 2
                0 0 0 0 93 2 2 38 0 0 1 1 20 38 0 1 1
                0 0 0 22 2 20 64 0 0 1 2 10 0 0 0 93
                0 22 0 1 1 20 0 0 8 1 16 64 0 68 1 0

```

```

6 0 21 1 16 64 0 66 1 16 38 0 1 2 0 0
23 0 24 1 0 6 0 9 1 20 0 0 57 1 16 64
0 67 1 2 97 0 1 1 11 0 0 70 1 2 96 0
1 1 9 18 0 19 2 8 0 0 0 34 2 8 0 0 18
37 3 7 0 0 25 6 27 3 7 0 0 28 29 31 2
0 0 6 6 17 0 1 86 88 2 4 0 0 28 1 2 4
0 0 25 85 3 4 0 0 28 95 1 3 4 0 0 25
71 1 2 6 0 0 0 1 2 6 0 0 0 1 1 3 0 74
1 1 2 41 0 44 1 2 38 0 45 0 3 71 73 2
6 0 0 74 1 2 16 0 0 71 1 2 18 0 0 94
1 0 20 0 55 0 16 0 63 2 4 0 0 28 1 2
4 0 0 25 1 3 4 0 0 28 95 1 3 4 0 0 25
71 1 2 2 38 0 0 40 2 0 0 6 6 20 2 11
0 0 0 90 2 1 0 0 6 1 1 20 0 0 51 2 20
0 0 0 52 2 20 0 6 0 53 2 20 0 0 6 54
2 23 0 0 0 47 2 23 0 6 0 48 2 23 0 0
6 49 2 6 0 0 74 1 2 16 0 0 71 1 2 18
0 0 94 1 2 20 0 71 0 1 2 20 0 74 0 76
2 23 0 94 0 1 2 18 0 0 0 59 2 18 0 0
6 61 2 18 0 6 0 60))))))
'|lookupComplete|))

```

The “compilerInfo”

```

(SETQ |$CategoryFrame|
  (|put| '|Equation| '|isFunction|
    '(((|eval| ($ $ (|List| (|Symbol|)) (|List| |#1|)))
      (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|))
      (ELT $ 31))
      (|eval| ($ $ (|Symbol|) |#1|))
      (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|))
      (ELT $ 27))
      (|= ((|Boolean|) $ $)) (|has| |#1| (|SetCategory|))
      (ELT $ NIL))
      (|= ((|Boolean|) $ $)) (|has| |#1| (|SetCategory|))
      (ELT $ 40))
      (|coerce| ((|OutputForm|) $))
      (|has| |#1| (|SetCategory|)) (ELT $ 44))
      (|hash| ((|SingleInteger|) $))
      (|has| |#1| (|SetCategory|)) (ELT $ NIL))
      (|latex| ((|String|) $)) (|has| |#1| (|SetCategory|))
      (ELT $ NIL))
      (|coerce| ((|Boolean|) $)) (|has| |#1| (|SetCategory|))
      (ELT $ 45))
      (+ ($ $ $)) (|has| |#1| (|AbelianSemiGroup|))
      (ELT $ 47))
      (* ($ (|PositiveInteger|) $))
      (|has| |#1| (|AbelianSemiGroup|)) (ELT $ NIL))
      (|Zero| ($)) (|has| |#1| (|AbelianGroup|))

```

```

(CONST $ 55))
(|sample| ($))
(OR (|has| |#1| (|AbelianGroup|))
    (|has| |#1| (|Monoid|)))
(CONST $ NIL))
(|zero?| ((|Boolean|) $)) (|has| |#1| (|AbelianGroup|))
(ELT $ NIL))
(* ($ (|NonNegativeInteger|) $))
(|has| |#1| (|AbelianGroup|)) (ELT $ NIL))
(|subtractIfCan| ((|Union| $ "failed") $ $))
(|has| |#1| (|AbelianGroup|)) (ELT $ NIL))
(= ($ $)) (|has| |#1| (|AbelianGroup|)) (ELT $ 51))
(< ($ $ $)) (|has| |#1| (|AbelianGroup|)) (ELT $ 52))
(* ($ (|Integer|) $)) (|has| |#1| (|AbelianGroup|))
(ELT $ 76))
(* ($ $ $)) (|has| |#1| (|SemiGroup|)) (ELT $ 59))
(** ($ $ (|PositiveInteger|))
(|has| |#1| (|SemiGroup|)) (ELT $ NIL))
(^ ($ $ (|PositiveInteger|))
(|has| |#1| (|SemiGroup|)) (ELT $ NIL))
(|One| ($)) (|has| |#1| (|Monoid|)) (CONST $ 63))
(|one?| ((|Boolean|) $)) (|has| |#1| (|Monoid|))
(ELT $ NIL))
(** ($ $ (|NonNegativeInteger|))
(|has| |#1| (|Monoid|)) (ELT $ NIL))
(^ ($ $ (|NonNegativeInteger|))
(|has| |#1| (|Monoid|)) (ELT $ NIL))
(|recip| ((|Union| $ "failed") $))
(|has| |#1| (|Monoid|)) (ELT $ 66))
(|inv| ($ $))
(OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|))
(ELT $ 70))
(/ ($ $ $))
(OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|))
(ELT $ 90))
(** ($ $ (|Integer|)) (|has| |#1| (|Group|))
(ELT $ NIL))
(^ ($ $ (|Integer|)) (|has| |#1| (|Group|))
(ELT $ NIL))
(|conjugate| ($ $ $)) (|has| |#1| (|Group|))
(ELT $ NIL))
(|commutator| ($ $ $)) (|has| |#1| (|Group|))
(ELT $ NIL))
(|characteristic| ((|NonNegativeInteger|))
(|has| |#1| (|Ring|)) (ELT $ 73))
(|coerce| ($ (|Integer|)) (|has| |#1| (|Ring|))
(ELT $ NIL))
(* ($ |#1| $)) (|has| |#1| (|SemiGroup|)) (ELT $ 60))
(* ($ $ |#1|)) (|has| |#1| (|SemiGroup|)) (ELT $ 61))
(|differentiate| ($ $ (|Symbol|)))

```

```

(|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
(ELT $ 85))
(|differentiate| ($ $ (|List| (|Symbol|))))
(|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
(ELT $ NIL))
(|differentiate|
  ($ $ (|Symbol|) (|NonNegativeInteger|)))
(|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
(ELT $ NIL))
(|differentiate|
  ($ $ (|List| (|Symbol|))
    (|List| (|NonNegativeInteger|))))
(|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
(ELT $ NIL))
((D ($ $ (|Symbol|)))
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ELT $ NIL))
((D ($ $ (|List| (|Symbol|))))
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ELT $ NIL))
((D ($ $ (|Symbol|) (|NonNegativeInteger|)))
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ELT $ NIL))
((D ($ $ (|List| (|Symbol|))
  (|List| (|NonNegativeInteger|))))
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ELT $ NIL))
((/ ($ $ |#1|)) (|has| |#1| (|Field|)) (ELT $ NIL))
(|dimension| ((|CardinalNumber|)))
(|has| |#1| (|Field|)) (ELT $ 88))
(|subst| ($ $ $)) (|has| |#1| (|ExpressionSpace|))
(ELT $ 93))
(|factorAndSplit| ((|List| $) $))
(|has| |#1| (|IntegralDomain|)) (ELT $ 19))
(|rightOne| ((|Union| $ "failed") $))
(|has| |#1| (|Monoid|)) (ELT $ 68))
(|leftOne| ((|Union| $ "failed") $))
(|has| |#1| (|Monoid|)) (ELT $ 67))
((- ($ $ |#1|)) (|has| |#1| (|AbelianGroup|))
  (ELT $ 54))
((- ($ |#1| $)) (|has| |#1| (|AbelianGroup|))
  (ELT $ 53))
(|rightZero| ($ $)) (|has| |#1| (|AbelianGroup|))
  (ELT $ 8))
(|leftZero| ($ $)) (|has| |#1| (|AbelianGroup|))
  (ELT $ 57))
( (+ ($ $ |#1|)) (|has| |#1| (|AbelianSemiGroup|))
  (ELT $ 49))
( (+ ($ |#1| $)) (|has| |#1| (|AbelianSemiGroup|))
  (ELT $ 48))

```



```

((|eval| ($ $ (|List| $)))
  (AND (|has| |#1| (|Evalable| |#1|))
        (|has| |#1| (|SetCategory|)))
  (ELT $ 37))
((|eval| ($ $ $))
  (AND (|has| |#1| (|Evalable| |#1|))
        (|has| |#1| (|SetCategory|)))
  (ELT $ 34))
((|map| ($ (|Mapping| |#1| |#1|) $)) T (ELT $ 24))
((|rhs| (|#1| $)) T (ELT $ 21))
((|lhs| (|#1| $)) T (ELT $ 9))
((|swap| ($ $)) T (ELT $ 22))
((|equation| ($ |#1| |#1|)) T (ELT $ 17))
((= ($ |#1| |#1|)) T (ELT $ 20))
(|addModemap| 'Equation| 'Equation| |#1|)
  '(|Join| (|Type|)
    (CATEGORY |domain|
      (SIGNATURE = ($ |#1| |#1|))
      (SIGNATURE |equation| ($ |#1| |#1|))
      (SIGNATURE |swap| ($ $))
      (SIGNATURE |lhs| (|#1| $))
      (SIGNATURE |rhs| (|#1| $))
      (SIGNATURE |map|
        ($ (|Mapping| |#1| |#1|) $))
      (IF (|has| |#1|
        (|InnerEvalable| (|Symbol|) |#1|))
        (ATTRIBUTE
          (|InnerEvalable| (|Symbol|) |#1|))
          |noBranch|)
      (IF (|has| |#1| (|SetCategory|))
        (PROGN
          (ATTRIBUTE (|SetCategory|))
          (ATTRIBUTE
            (|CoercibleTo| (|Boolean|)))
          (IF (|has| |#1| (|Evalable| |#1|))
            (PROGN
              (SIGNATURE |eval| ($ $ $))
              (SIGNATURE |eval|
                ($ $ (|List| $))))
              |noBranch|))
          |noBranch|)
      (IF (|has| |#1| (|AbelianSemiGroup|))
        (PROGN
          (ATTRIBUTE (|AbelianSemiGroup|))
          (SIGNATURE + ($ |#1| $))
          (SIGNATURE + ($ $ |#1|)))
          |noBranch|)
      (IF (|has| |#1| (|AbelianGroup|))
        (PROGN
          (ATTRIBUTE (|AbelianGroup|))

```

```

(SIGNATURE |leftZero| ($ $))
(SIGNATURE |rightZero| ($ $))
(SIGNATURE - ($ |#1| $))
(SIGNATURE - ($ $ |#1|))
|noBranch|
(IF (|has| |#1| (|SemiGroup|))
  (PROGN
    (ATTRIBUTE (|SemiGroup|))
    (SIGNATURE * ($ |#1| $))
    (SIGNATURE * ($ $ |#1|))
  )
  |noBranch|
)
(IF (|has| |#1| (|Monoid|))
  (PROGN
    (ATTRIBUTE (|Monoid|))
    (SIGNATURE |leftOne|
      ((|Union| $ "failed") $))
    (SIGNATURE |rightOne|
      ((|Union| $ "failed") $))
    )
  |noBranch|
)
(IF (|has| |#1| (|Group|))
  (PROGN
    (ATTRIBUTE (|Group|))
    (SIGNATURE |leftOne|
      ((|Union| $ "failed") $))
    (SIGNATURE |rightOne|
      ((|Union| $ "failed") $))
    )
  |noBranch|
)
(IF (|has| |#1| (|Ring|))
  (PROGN
    (ATTRIBUTE (|Ring|))
    (ATTRIBUTE (|BiModule| |#1| |#1|))
  )
  |noBranch|
)
(IF (|has| |#1| (|CommutativeRing|))
  (ATTRIBUTE (|Module| |#1|))
  |noBranch|
)
(IF (|has| |#1| (|IntegralDomain|))
  (SIGNATURE |factorAndSplit|
    ((|List| $) $))
  |noBranch|
)
(IF (|has| |#1|
  (|PartialDifferentialRing|
    (|Symbol|)))
  (ATTRIBUTE
    (|PartialDifferentialRing|
      (|Symbol|)))
  |noBranch|
)
(IF (|has| |#1| (|Field|))
  (PROGN
    (ATTRIBUTE (|VectorSpace| |#1|))
    (SIGNATURE / ($ $ $))
  )
)

```

```

(SIGNATURE |inv| ($ $)))
|noBranch|)
(IF (|has| |#1| (|ExpressionSpace|))
(SIGNATURE |subst| ($ $ $))
|noBranch|)))
(|Type|))
T '|Equation|
(|put| '|Equation| '|mode|
'|Mapping|
(|Join| (|Type|)
(CATEGORY |domain|
(SIGNATURE = ($ |#1| |#1|))
(SIGNATURE |equation|
($ |#1| |#1|))
(SIGNATURE |swap| ($ $))
(SIGNATURE |lhs| (|#1| $))
(SIGNATURE |rhs| (|#1| $))
(SIGNATURE |map|
($ (|Mapping| |#1| |#1|) $))
(IF
(|has| |#1|
(|InnerEvalable| (|Symbol|
|#1|))
(ATTRIBUTE
(|InnerEvalable| (|Symbol|
|#1|))
|noBranch|)
(IF (|has| |#1| (|SetCategory|))
(PROGN
(ATTRIBUTE (|SetCategory|))
(ATTRIBUTE
(|CoercibleTo| (|Boolean|)))
(IF
(|has| |#1|
(|Evalable| |#1|))
(PROGN
(SIGNATURE |eval| ($ $ $))
(SIGNATURE |eval|
($ $ (|List| $))))
|noBranch|))
|noBranch|)
(IF
(|has| |#1|
(|AbelianSemiGroup|))
(PROGN
(ATTRIBUTE
(|AbelianSemiGroup|))
(SIGNATURE + ($ |#1| $))
(SIGNATURE + ($ $ |#1|))
|noBranch|)

```

```

(IF (|has| |#1| (|AbelianGroup|))
  (PROGN
    (ATTRIBUTE (|AbelianGroup|))
    (SIGNATURE |leftZero| ($ $))
    (SIGNATURE |rightZero| ($ $))
    (SIGNATURE - ($ |#1| $))
    (SIGNATURE - ($ $ |#1|)))
  |noBranch|)
(IF (|has| |#1| (|SemiGroup|))
  (PROGN
    (ATTRIBUTE (|SemiGroup|))
    (SIGNATURE * ($ |#1| $))
    (SIGNATURE * ($ $ |#1|)))
  |noBranch|)
(IF (|has| |#1| (|Monoid|))
  (PROGN
    (ATTRIBUTE (|Monoid|))
    (SIGNATURE |leftOne|
      ((|Union| $ "failed") $))
    (SIGNATURE |rightOne|
      ((|Union| $ "failed") $)))
  |noBranch|)
(IF (|has| |#1| (|Group|))
  (PROGN
    (ATTRIBUTE (|Group|))
    (SIGNATURE |leftOne|
      ((|Union| $ "failed") $))
    (SIGNATURE |rightOne|
      ((|Union| $ "failed") $)))
  |noBranch|)
(IF (|has| |#1| (|Ring|))
  (PROGN
    (ATTRIBUTE (|Ring|))
    (ATTRIBUTE
      (|BiModule| |#1| |#1|)))
  |noBranch|)
(IF
  (|has| |#1| (|CommutativeRing|))
  (ATTRIBUTE (|Module| |#1|))
  |noBranch|)
(IF
  (|has| |#1| (|IntegralDomain|))
  (SIGNATURE |factorAndSplit|
    ((|List| $) $))
  |noBranch|)
(IF
  (|has| |#1|
    (|PartialDifferentialRing|
      (|Symbol|)))
  (ATTRIBUTE

```

```

(|PartialDifferentialRing|
  (|Symbol|)))
|noBranch|)
(IF (|has| |#1| (|Field|))
  (PROGN
    (ATTRIBUTE
      (|VectorSpace| |#1|))
    (SIGNATURE / ($ $ $))
    (SIGNATURE |inv| ($ $)))
    |noBranch|)
  (IF
    (|has| |#1| (|ExpressionSpace|))
    (SIGNATURE |subst| ($ $ $))
    |noBranch|)))
(|Type|))
|$CategoryFrame|))))

```

The “constructorForm”

```
(|Equation| S)
```

The “constructorKind”

```
|domain|
```

The “constructorModemap”

```

(((|Equation| |#1|)
  (|Join| (|Type|)
    (CATEGORY |domain| (SIGNATURE = ($ |#1| |#1|))
      (SIGNATURE |equation| ($ |#1| |#1|))
      (SIGNATURE |swap| ($ $)) (SIGNATURE |lhs| (|#1| $))
      (SIGNATURE |rhs| (|#1| $))
      (SIGNATURE |map| ($ (|Mapping| |#1| |#1|) $))
      (IF (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|))
        (ATTRIBUTE (|InnerEvalable| (|Symbol|) |#1|))
        |noBranch|)
      (IF (|has| |#1| (|SetCategory|))
        (PROGN
          (ATTRIBUTE (|SetCategory|))
          (ATTRIBUTE (|CoercibleTo| (|Boolean|)))
          (IF (|has| |#1| (|Evalable| |#1|))
            (PROGN
              (SIGNATURE |eval| ($ $ $))
              (SIGNATURE |eval| ($ $ (|List| $))))
            |noBranch|))
          |noBranch|)
        |noBranch|)
    |noBranch|)
  |noBranch|)

```

```

(IF (|has| |#1| (|AbelianSemiGroup|))
  (PROGN
    (ATTRIBUTE (|AbelianSemiGroup|))
    (SIGNATURE + ($ |#1| $))
    (SIGNATURE + ($ $ |#1|)))
  |noBranch|)
(IF (|has| |#1| (|AbelianGroup|))
  (PROGN
    (ATTRIBUTE (|AbelianGroup|))
    (SIGNATURE |leftZero| ($ $))
    (SIGNATURE |rightZero| ($ $))
    (SIGNATURE - ($ |#1| $))
    (SIGNATURE - ($ $ |#1|)))
  |noBranch|)
(IF (|has| |#1| (|SemiGroup|))
  (PROGN
    (ATTRIBUTE (|SemiGroup|))
    (SIGNATURE * ($ |#1| $))
    (SIGNATURE * ($ $ |#1|)))
  |noBranch|)
(IF (|has| |#1| (|Monoid|))
  (PROGN
    (ATTRIBUTE (|Monoid|))
    (SIGNATURE |leftOne| ((|Union| $ "failed") $))
    (SIGNATURE |rightOne| ((|Union| $ "failed") $)))
  |noBranch|)
(IF (|has| |#1| (|Group|))
  (PROGN
    (ATTRIBUTE (|Group|))
    (SIGNATURE |leftOne| ((|Union| $ "failed") $))
    (SIGNATURE |rightOne| ((|Union| $ "failed") $)))
  |noBranch|)
(IF (|has| |#1| (|Ring|))
  (PROGN
    (ATTRIBUTE (|Ring|))
    (ATTRIBUTE (|BiModule| |#1| |#1|)))
  |noBranch|)
(IF (|has| |#1| (|CommutativeRing|))
  (ATTRIBUTE (|Module| |#1|)) |noBranch|)
(IF (|has| |#1| (|IntegralDomain|))
  (SIGNATURE |factorAndSplit| ((|List| $) $))
  |noBranch|)
(IF (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ATTRIBUTE (|PartialDifferentialRing| (|Symbol|)))
  |noBranch|)
(IF (|has| |#1| (|Field|))
  (PROGN
    (ATTRIBUTE (|VectorSpace| |#1|))
    (SIGNATURE / ($ $ $))
    (SIGNATURE |inv| ($ $)))

```

```

      |noBranch|)
    (IF (|has| |#1| (|ExpressionSpace|))
      (SIGNATURE |subst| ($ $ $)) |noBranch|)))
  (|Type|))
(T |Equation|))

```

The “constructorCategory”

```

(|Join| (|Type|)
  (CATEGORY |domain| (SIGNATURE = ($ |#1| |#1|))
    (SIGNATURE |equation| ($ |#1| |#1|))
    (SIGNATURE |swap| ($ $)) (SIGNATURE |lhs| (|#1| $))
    (SIGNATURE |rhs| (|#1| $))
    (SIGNATURE |map| ($ (|Mapping| |#1| |#1|) $))
    (IF (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|))
      (ATTRIBUTE (|InnerEvalable| (|Symbol|) |#1|))
      |noBranch|)
    (IF (|has| |#1| (|SetCategory|))
      (PROGN
        (ATTRIBUTE (|SetCategory|))
        (ATTRIBUTE (|CoercibleTo| (|Boolean|)))
        (IF (|has| |#1| (|Evalable| |#1|))
          (PROGN
            (SIGNATURE |eval| ($ $ $))
            (SIGNATURE |eval| ($ $ (|List| $))))
          |noBranch|))
        |noBranch|)
    (IF (|has| |#1| (|AbelianSemiGroup|))
      (PROGN
        (ATTRIBUTE (|AbelianSemiGroup|))
        (SIGNATURE + ($ |#1| $))
        (SIGNATURE + ($ $ |#1|)))
        |noBranch|)
    (IF (|has| |#1| (|AbelianGroup|))
      (PROGN
        (ATTRIBUTE (|AbelianGroup|))
        (SIGNATURE |leftZero| ($ $))
        (SIGNATURE |rightZero| ($ $))
        (SIGNATURE - ($ |#1| $))
        (SIGNATURE - ($ $ |#1|)))
        |noBranch|)
    (IF (|has| |#1| (|SemiGroup|))
      (PROGN
        (ATTRIBUTE (|SemiGroup|))
        (SIGNATURE * ($ |#1| $))
        (SIGNATURE * ($ $ |#1|)))
        |noBranch|)
    (IF (|has| |#1| (|Monoid|))
      (PROGN

```

```

      (ATTRIBUTE (|Monoid|))
      (SIGNATURE |leftOne| ((|Union| $ "failed") $))
      (SIGNATURE |rightOne| ((|Union| $ "failed") $)))
|noBranch|)
(IF (|has| |#1| (|Group|))
  (PROGN
    (ATTRIBUTE (|Group|))
    (SIGNATURE |leftOne| ((|Union| $ "failed") $))
    (SIGNATURE |rightOne| ((|Union| $ "failed") $)))
|noBranch|)
(IF (|has| |#1| (|Ring|))
  (PROGN
    (ATTRIBUTE (|Ring|))
    (ATTRIBUTE (|BiModule| |#1| |#1|)))
|noBranch|)
(IF (|has| |#1| (|CommutativeRing|))
  (ATTRIBUTE (|Module| |#1|)) |noBranch|)
(IF (|has| |#1| (|IntegralDomain|))
  (SIGNATURE |factorAndSplit| ((|List| $) $)) |noBranch|)
(IF (|has| |#1| (|PartialDifferentialRing| (|Symbol|)))
  (ATTRIBUTE (|PartialDifferentialRing| (|Symbol|)))
|noBranch|)
(IF (|has| |#1| (|Field|))
  (PROGN
    (ATTRIBUTE (|VectorSpace| |#1|))
    (SIGNATURE / ($ $ $))
    (SIGNATURE |inv| ($ $)))
|noBranch|)
(IF (|has| |#1| (|ExpressionSpace|))
  (SIGNATURE |subst| ($ $ $)) |noBranch|)))

```

The “sourceFile”

```
"/research/test/int/algebra/EQ.spad"
```

The “modemaps”

```

((= (*1 *1 *2 *2)
  (AND (|isDomain| *1 (|Equation| *2)) (|ofCategory| *2 (|Type|))))
(|equation| (*1 *1 *2 *2)
  (AND (|isDomain| *1 (|Equation| *2)) (|ofCategory| *2 (|Type|))))
(|swap| (*1 *1 *1)
  (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Type|))))
(|lhs| (*1 *2 *1)
  (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Type|))))
(|rhs| (*1 *2 *1)

```



```

      (AND (|isDomain| *1 (|Equation| *2))
            (|ofCategory| *2 (|Type|)))
(|map| (*1 *1 *2 *1)
      (AND (|isDomain| *2 (|Mapping| *3 *3))
            (|ofCategory| *3 (|Type|))
            (|isDomain| *1 (|Equation| *3))))
(|eval| (*1 *1 *1 *1)
      (AND (|ofCategory| *2 (|Evalable| *2))
            (|ofCategory| *2 (|SetCategory|))
            (|ofCategory| *2 (|Type|))
            (|isDomain| *1 (|Equation| *2))))
(|eval| (*1 *1 *1 *2)
      (AND (|isDomain| *2 (|List| (|Equation| *3)))
            (|ofCategory| *3 (|Evalable| *3))
            (|ofCategory| *3 (|SetCategory|))
            (|ofCategory| *3 (|Type|))
            (|isDomain| *1 (|Equation| *3))))
(+ (*1 *1 *2 *1)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianSemiGroup|))
        (|ofCategory| *2 (|Type|))))
(+ (*1 *1 *1 *2)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianSemiGroup|))
        (|ofCategory| *2 (|Type|))))
(|leftZero| (*1 *1 *1)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianGroup|))
        (|ofCategory| *2 (|Type|))))
(|rightZero| (*1 *1 *1)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianGroup|))
        (|ofCategory| *2 (|Type|))))
(- (*1 *1 *2 *1)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianGroup|)) (|ofCategory| *2 (|Type|))))
(- (*1 *1 *1 *2)
  (AND (|isDomain| *1 (|Equation| *2))
        (|ofCategory| *2 (|AbelianGroup|)) (|ofCategory| *2 (|Type|))))
(|leftOne| (*1 *1 *1)
  (|partial| AND (|isDomain| *1 (|Equation| *2))
                 (|ofCategory| *2 (|Monoid|)) (|ofCategory| *2 (|Type|))))
(|rightOne| (*1 *1 *1)
  (|partial| AND (|isDomain| *1 (|Equation| *2))
                 (|ofCategory| *2 (|Monoid|)) (|ofCategory| *2 (|Type|))))
(|factorAndSplit| (*1 *2 *1)
  (AND (|isDomain| *2 (|List| (|Equation| *3)))
        (|isDomain| *1 (|Equation| *3))
        (|ofCategory| *3 (|IntegralDomain|))
        (|ofCategory| *3 (|Type|))))

```

```

(|subst| (*1 *1 *1 *1)
  (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|ExpressionSpace|))
    (|ofCategory| *2 (|Type|))))
(* (*1 *1 *1 *2)
  (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|SemiGroup|)) (|ofCategory| *2 (|Type|))))
(* (*1 *1 *2 *1)
  (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|SemiGroup|)) (|ofCategory| *2 (|Type|))))
(/ (*1 *1 *1 *1)
  (OR (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Field|)) (|ofCategory| *2 (|Type|)))
    (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Group|)) (|ofCategory| *2 (|Type|)))))
(|inv| (*1 *1 *1)
  (OR (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Field|))
    (|ofCategory| *2 (|Type|)))
    (AND (|isDomain| *1 (|Equation| *2))
    (|ofCategory| *2 (|Group|))
    (|ofCategory| *2 (|Type|)))))

```

The “operationAlist”

```

((~= (((|Boolean|) $ $) NIL (|has| |#1| (|SetCategory|))))
(|zero?| (((|Boolean|) $) NIL (|has| |#1| (|AbelianGroup|))))
(|swap| (($ $) 22))
(|subtractIfCan|
  (((|Union| $ "failed") $ $) NIL (|has| |#1| (|AbelianGroup|))))
(|subst| (($ $ $) 93 (|has| |#1| (|ExpressionSpace|))))
(|sample|
  (($) NIL
    (OR (|has| |#1| (|AbelianGroup|)) (|has| |#1| (|Monoid|))) CONST))
(|rightZero| (($ $) 8 (|has| |#1| (|AbelianGroup|))))
(|rightOne| (((|Union| $ "failed") $) 68 (|has| |#1| (|Monoid|))))
(|rhs| ((|#1| $) 21))
(|recip| (((|Union| $ "failed") $) 66 (|has| |#1| (|Monoid|))))
(|one?| (((|Boolean|) $) NIL (|has| |#1| (|Monoid|))))
(|map| (($ (|Mapping| |#1| |#1|) $) 24) (|lhs| ((|#1| $) 9))
(|leftZero| (($ $) 57 (|has| |#1| (|AbelianGroup|))))
(|leftOne| (((|Union| $ "failed") $) 67 (|has| |#1| (|Monoid|))))
(|latex| (((|String|) $) NIL (|has| |#1| (|SetCategory|))))
(|inv| (($ $) 70 (OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|)))))
(|hash| (((|SingleInteger|) $) NIL (|has| |#1| (|SetCategory|))))
(|factorAndSplit| (((|List| $) $) 19 (|has| |#1| (|IntegralDomain|))))
(|eval| (($ $ $) 34
  (AND (|has| |#1| (|Evalable| |#1|))
    (|has| |#1| (|SetCategory|))))

```

```

(($ $ (|List| $)) 37
  (AND (|has| |#1| (|Evalable| |#1|))
        (|has| |#1| (|SetCategory|))))
(($ $ (|Symbol|) |#1|) 27
  (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|)))
(($ $ (|List| (|Symbol|)) (|List| |#1|)) 31
  (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|)))
(|equation| (($ |#1| |#1|) 17))
(|dimension| (((|CardinalNumber|)) 88 (|has| |#1| (|Field|))))
(|differentiate|
  (($ $ (|List| (|Symbol|)) (|List| (|NonNegativeInteger|))) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|Symbol|) (|NonNegativeInteger|)) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|List| (|Symbol|))) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|Symbol|)) 85
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
(|conjugate| (($ $ $) NIL (|has| |#1| (|Group|))))
(|commutator| (($ $ $) NIL (|has| |#1| (|Group|))))
(|coerce| (($ (|Integer|)) NIL (|has| |#1| (|Ring|))))
  (((|Boolean|) $) 45 (|has| |#1| (|SetCategory|)))
  (((|OutputForm|) $) 44 (|has| |#1| (|SetCategory|)))
(|characteristic| (((|NonNegativeInteger|)) 73 (|has| |#1| (|Ring|))))
(^ (($ $ (|Integer|)) NIL (|has| |#1| (|Group|)))
  (($ $ (|NonNegativeInteger|)) NIL (|has| |#1| (|Monoid|)))
  (($ $ (|PositiveInteger|)) NIL (|has| |#1| (|SemiGroup|))))
(|Zero| (($) 55 (|has| |#1| (|AbelianGroup|)) CONST))
(|One| (($) 63 (|has| |#1| (|Monoid|)) CONST))
(D (($ $ (|List| (|Symbol|)) (|List| (|NonNegativeInteger|))) NIL
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|Symbol|) (|NonNegativeInteger|)) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|List| (|Symbol|))) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  (($ $ (|Symbol|)) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
(= (($ |#1| |#1|) 20)
  (((|Boolean|) $ $) 40 (|has| |#1| (|SetCategory|))))
(/ (($ $ |#1|) NIL (|has| |#1| (|Field|)))
  (($ $ $) 90 (OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|)))))
(- (($ |#1| $) 53 (|has| |#1| (|AbelianGroup|)))
  (($ $ |#1|) 54 (|has| |#1| (|AbelianGroup|)))
  (($ $ $) 52 (|has| |#1| (|AbelianGroup|)))
  (($ $) 51 (|has| |#1| (|AbelianGroup|)))
(+) (($ |#1| $) 48 (|has| |#1| (|AbelianSemiGroup|)))
  (($ $ |#1|) 49 (|has| |#1| (|AbelianSemiGroup|)))
  (($ $ $) 47 (|has| |#1| (|AbelianSemiGroup|)))
(** (($ $ (|Integer|)) NIL (|has| |#1| (|Group|)))
  (($ $ (|NonNegativeInteger|)) NIL (|has| |#1| (|Monoid|))))

```

```

(($ $ (|PositiveInteger|)) NIL (|has| |#1| (|SemiGroup|)))
(* (($ $ |#1|) 61 (|has| |#1| (|SemiGroup|)))
  (($ |#1| $) 60 (|has| |#1| (|SemiGroup|)))
  (($ $ $) 59 (|has| |#1| (|SemiGroup|)))
  (($ (|Integer|) $) 76 (|has| |#1| (|AbelianGroup|)))
  (($ (|NonNegativeInteger|) $) NIL (|has| |#1| (|AbelianGroup|)))
  (($ (|PositiveInteger|) $) NIL (|has| |#1| (|AbelianSemiGroup|))))

```

The “superDomain”

The “signaturesAndLocals”

```

(|EQ;subst;3$;43| ($ $ $)) (|EQ;inv;2$;42| ($ $))
(|EQ;/;3$;41| ($ $ $)) (|EQ;dimension;Cn;40| ((|CardinalNumber|)))
(|EQ;differentiate;$S$;39| ($ $ (|Symbol|)))
(|EQ;factorAndSplit;$L;38| ((|List| $) $))
(|EQ;*;I2$;37| ($ (|Integer|) $))
(|EQ;characteristic;Nni;36| ((|NonNegativeInteger|)))
(|EQ;rightOne;$U;35| ((|Union| $ "failed") $))
(|EQ;leftOne;$U;34| ((|Union| $ "failed") $)) (|EQ;inv;2$;33| ($ $))
(|EQ;rightOne;$U;32| ((|Union| $ "failed") $))
(|EQ;leftOne;$U;31| ((|Union| $ "failed") $))
(|EQ;recip;$U;30| ((|Union| $ "failed") $)) (|EQ;One;$;29| ($))
(|EQ;*;$S$;28| ($ $ S)) (|EQ;*;S2$;27| ($ S $))
(|EQ;*;S2$;26| ($ S $)) (|EQ;*;3$;25| ($ $ $)) (|EQ;-;3$;24| ($ $ $))
(|EQ;Zero;$;23| ($)) (|EQ;rightZero;2$;22| ($ $))
(|EQ;leftZero;2$;21| ($ $)) (|EQ;-;$S$;20| ($ $ S))
(|EQ;-;S2$;19| ($ S $)) (|EQ;-;2$;18| ($ $)) (|EQ;+;$S$;17| ($ $ S))
(|EQ;+;S2$;16| ($ S $)) (|EQ;+;3$;15| ($ $ $))
(|EQ;coerce;$B;14| ((|Boolean|) $))
(|EQ;coerce;$Of;13| ((|OutputForm|) $))
(|EQ;=;2$B;12| ((|Boolean|) $ $)) (|EQ;eval;$L$;11| ($ $ (|List| $)))
(|EQ;eval;3$;10| ($ $ $))
(|EQ;eval;$LL$;9| ($ $ (|List| (|Symbol|)) (|List| S)))
(|EQ;eval;$SS$;8| ($ $ (|Symbol|) S))
(|EQ;map;M2$;7| ($ (|Mapping| S S) $)) (|EQ;swap;2$;6| ($ $))
(|EQ;rhs;$S;5| (S $)) (|EQ;lhs;$S;4| (S $))
(|EQ;equation;2S$;3| ($ S S)) (|EQ;=;2S$;2| ($ S S))
(|EQ;factorAndSplit;$L;1| ((|List| $) $))

```

The “attributes”

```

((|unitsKnown| OR (|has| |#1| (|Ring|)) (|has| |#1| (|Group|)))
  (|rightUnitary| |has| |#1| (|Ring|))
  (|leftUnitary| |has| |#1| (|Ring|)))

```

The “predicates”

```

((|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|SetCategory|))
(|HasCategory| |#1| '(|Ring|))
(|HasCategory| |#1| (LIST '(|PartialDifferentialRing| '(|Symbol|)))
(OR (|HasCategory| |#1| (LIST '(|PartialDifferentialRing| '(|Symbol|)))
(|HasCategory| |#1| '(|Ring|)))
(|HasCategory| |#1| '(|Group|))
(|HasCategory| |#1|
(LIST '(|InnerEvalable| '(|Symbol|) (|devaluate| |#1|)))
(AND (|HasCategory| |#1| (LIST '(|Evalable| (|devaluate| |#1|)))
(|HasCategory| |#1| '(|SetCategory|)))
(|HasCategory| |#1| '(|IntegralDomain|))
(|HasCategory| |#1| '(|ExpressionSpace|))
(OR (|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|Group|)))
(OR (|HasCategory| |#1| '(|Group|)) (|HasCategory| |#1| '(|Ring|)))
(|HasCategory| |#1| '(|CommutativeRing|))
(OR (|HasCategory| |#1| '(|CommutativeRing|))
(|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|Ring|)))
(OR (|HasCategory| |#1| '(|CommutativeRing|))
(|HasCategory| |#1| '(|Field|)))
(|HasCategory| |#1| '(|Monoid|))
(OR (|HasCategory| |#1| '(|Group|)) (|HasCategory| |#1| '(|Monoid|)))
(|HasCategory| |#1| '(|SemiGroup|))
(OR (|HasCategory| |#1| '(|Group|)) (|HasCategory| |#1| '(|Monoid|))
(|HasCategory| |#1| '(|SemiGroup|)))
(|HasCategory| |#1| '(|AbelianGroup|))
(OR (|HasCategory| |#1| (LIST '(|PartialDifferentialRing| '(|Symbol|)))
(|HasCategory| |#1| '(|AbelianGroup|))
(|HasCategory| |#1| '(|CommutativeRing|))
(|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|Ring|)))
(OR (|HasCategory| |#1| '(|AbelianGroup|))
(|HasCategory| |#1| '(|Monoid|)))
(|HasCategory| |#1| '(|AbelianSemiGroup|))
(OR (|HasCategory| |#1| (LIST '(|PartialDifferentialRing| '(|Symbol|)))
(|HasCategory| |#1| '(|AbelianGroup|))
(|HasCategory| |#1| '(|AbelianSemiGroup|))
(|HasCategory| |#1| '(|CommutativeRing|))
(|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|Ring|)))
(OR (|HasCategory| |#1| (LIST '(|PartialDifferentialRing| '(|Symbol|)))
(|HasCategory| |#1| '(|AbelianGroup|))
(|HasCategory| |#1| '(|AbelianSemiGroup|))
(|HasCategory| |#1| '(|CommutativeRing|))
(|HasCategory| |#1| '(|Field|)) (|HasCategory| |#1| '(|Group|))
(|HasCategory| |#1| '(|Monoid|)) (|HasCategory| |#1| '(|Ring|))
(|HasCategory| |#1| '(|SemiGroup|))
(|HasCategory| |#1| '(|SetCategory|))))

```

The “abbreviation”

EQ

The “parents”

```
(((|Type|) . T)
  ((|InnerEvalable| (|Symbol|) S) |has| S
   (|InnerEvalable| (|Symbol|) S))
  ((|CoercibleTo| (|Boolean|)) |has| S (|SetCategory|))
  ((|SetCategory|) |has| S (|SetCategory|))
  ((|AbelianSemiGroup|) |has| S (|AbelianSemiGroup|))
  ((|AbelianGroup|) |has| S (|AbelianGroup|))
  ((|SemiGroup|) |has| S (|SemiGroup|)) ((|Monoid|) |has| S (|Monoid|))
  ((|Group|) |has| S (|Group|)) ((|BiModule| S S) |has| S (|Ring|))
  ((|Ring|) |has| S (|Ring|)) ((|Module| S) |has| S (|CommutativeRing|))
  ((|PartialDifferentialRing| (|Symbol|)) |has| S
   (|PartialDifferentialRing| (|Symbol|)))
  ((|VectorSpace| S) |has| S (|Field|)))
```

The “ancestors”

```
(((|AbelianGroup|) |has| S (|AbelianGroup|))
  ((|AbelianMonoid|) |has| S (|AbelianGroup|))
  ((|AbelianSemiGroup|) |has| S (|AbelianSemiGroup|))
  ((|BasicType|) |has| S (|SetCategory|))
  ((|BiModule| S S) |has| S (|Ring|))
  ((|CancellationAbelianMonoid|) |has| S (|AbelianGroup|))
  ((|CoercibleTo| (|OutputForm|)) |has| S (|SetCategory|))
  ((|CoercibleTo| (|Boolean|)) |has| S (|SetCategory|))
  ((|Group|) |has| S (|Group|))
  ((|InnerEvalable| (|Symbol|) S) |has| S
   (|InnerEvalable| (|Symbol|) S))
  ((|LeftModule| $) |has| S (|Ring|))
  ((|LeftModule| S) |has| S (|Ring|))
  ((|Module| S) |has| S (|CommutativeRing|))
  ((|Monoid|) |has| S (|Monoid|))
  ((|PartialDifferentialRing| (|Symbol|)) |has| S
   (|PartialDifferentialRing| (|Symbol|)))
  ((|RightModule| S) |has| S (|Ring|)) ((|Ring|) |has| S (|Ring|))
  ((|Rng|) |has| S (|Ring|)) ((|SemiGroup|) |has| S (|SemiGroup|))
  ((|SetCategory|) |has| S (|SetCategory|)) ((|Type|) . T)
  ((|VectorSpace| S) |has| S (|Field|)))
```

The “documentation”

```
((|constructor|
```

```

(NIL "Equations as mathematical objects. All properties of the basis
    domain,{ } \spadignore{e.g.} being an abelian group are carried
    over the equation domain,{ } by performing the structural operations
    on the left and on the right hand side.")
(|subst| (($ $ $)
    "\spad{subst(eq1,{ }eq2)} substitutes \spad{eq2} into both sides
    of \spad{eq1} the \spad{lhs} of \spad{eq2} should be a kernel"))
(|inv| (($ $)
    "\spad{inv(x)} returns the multiplicative inverse of \spad{x}."))
(/ (($ $ $)
    "\spad{e1/e2} produces a new equation by dividing the left and right
    hand sides of equations \spad{e1} and \spad{e2}."))
(|factorAndSplit|
    (((|List| $) $)
    "\spad{factorAndSplit(eq)} make the right hand side 0 and factors the
    new left hand side. Each factor is equated to 0 and put into the
    resulting list without repetitions."))
(|rightOne|
    (((|Union| $ "failed") $)
    "\spad{rightOne(eq)} divides by the right hand side.")
    (((|Union| $ "failed") $)
    "\spad{rightOne(eq)} divides by the right hand side,{ } if possible."))
(|leftOne|
    (((|Union| $ "failed") $)
    "\spad{leftOne(eq)} divides by the left hand side.")
    (((|Union| $ "failed") $)
    "\spad{leftOne(eq)} divides by the left hand side,{ } if possible."))
(* (($ $ |#1|)
    "\spad{eqn*x} produces a new equation by multiplying both sides of
    equation eqn by \spad{x}."))
(($ |#1| $)
    "\spad{x*eqn} produces a new equation by multiplying both sides of
    equation eqn by \spad{x}."))
(- (($ $ |#1|)
    "\spad{eqn-x} produces a new equation by subtracting \spad{x} from
    both sides of equation eqn.")
    (($ |#1| $)
    "\spad{x-eqn} produces a new equation by subtracting both sides of
    equation eqn from \spad{x}."))
(|rightZero|
    (($ $) "\spad{rightZero(eq)} subtracts the right hand side."))
(|leftZero|
    (($ $) "\spad{leftZero(eq)} subtracts the left hand side."))
(+ (($ $ |#1|)
    "\spad{eqn+x} produces a new equation by adding \spad{x} to both
    sides of equation eqn.")
    (($ |#1| $)
    "\spad{x+eqn} produces a new equation by adding \spad{x} to both
    sides of equation eqn."))
(|eval| (($ $ (|List| $))

```

```

"\spad{eval(eqn,{ [x1=v1,{ ... xn=vn]})} replaces \spad{xi}
by \spad{vi} in equation \spad{eqn}."
(($ $ $)
"\spad{eval(eqn,{ x=f})} replaces \spad{x} by \spad{f} in
equation \spad{eqn}."))
(|map| (($ (|Mapping| |#1| |#1|) $)
"\spad{map(f,{eqn})} constructs a new equation by applying
\spad{f} to both sides of \spad{eqn}."))
(|rhs| ((|#1| $)
"\spad{rhs(eqn)} returns the right hand side of equation
\spad{eqn}."))
(|lhs| ((|#1| $)
"\spad{lhs(eqn)} returns the left hand side of equation
\spad{eqn}."))
(|swap| (($ $)
"\spad{swap(eq)} interchanges left and right hand side of
equation \spad{eq}."))
(|equation|
(($ |#1| |#1|) "\spad{equation(a,{b})} creates an equation."))
(= (($ |#1| |#1|) "\spad{a=b} creates an equation."))

```

The “slotInfo”

```

(|Equation|
(NIL (~= ((38 0 0) NIL (|has| |#1| (|SetCategory|))))
(|zero?| ((38 0) NIL (|has| |#1| (|AbelianGroup|))))
(|swap| ((0 0) 22))
(|subtractIfCan| ((64 0 0) NIL (|has| |#1| (|AbelianGroup|))))
(|subst| ((0 0 0) 93 (|has| |#1| (|ExpressionSpace|))))
(|sample|
((0) NIL
(OR (|has| |#1| (|AbelianGroup|))
(|has| |#1| (|Monoid|)))
CONST))
(|rightZero| ((0 0) 8 (|has| |#1| (|AbelianGroup|))))
(|rightOne| ((64 0) 68 (|has| |#1| (|Monoid|))))
(|rhs| ((6 0) 21))
(|recip| ((64 0) 66 (|has| |#1| (|Monoid|))))
(|one?| ((38 0) NIL (|has| |#1| (|Monoid|))))
(|map| ((0 23 0) 24)) (|lhs| ((6 0) 9))
(|leftZero| ((0 0) 57 (|has| |#1| (|AbelianGroup|))))
(|leftOne| ((64 0) 67 (|has| |#1| (|Monoid|))))
(|latex| ((97 0) NIL (|has| |#1| (|SetCategory|))))
(|inv| ((0 0) 70
(OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|)))))
(|hash| ((96 0) NIL (|has| |#1| (|SetCategory|))))
(|factorAndSplit| ((18 0) 19 (|has| |#1| (|IntegralDomain|))))
(|eval| ((0 0 28 29) 31
(|has| |#1| (|InnerEvalable| (|Symbol|) |#1|))))

```



```

((0 0 25 6) 27
  (|has| |#1| (|InnerEvalable| (|Symbol|) |#1|)))
((0 0 18) 37
  (AND (|has| |#1| (|Evalable| |#1|))
        (|has| |#1| (|SetCategory|))))
((0 0 0) 34
  (AND (|has| |#1| (|Evalable| |#1|))
        (|has| |#1| (|SetCategory|))))
(|equation| ((0 6 6) 17))
(|dimension| ((86) 88 (|has| |#1| (|Field|))))
(|differentiate|
  ((0 0 25 71) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 28 95) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 25) 85
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 28) NIL
    (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
(|conjugate| ((0 0 0) NIL (|has| |#1| (|Group|))))
(|commutator| ((0 0 0) NIL (|has| |#1| (|Group|))))
(|coerce| ((38 0) 45 (|has| |#1| (|SetCategory|)))
  ((41 0) 44 (|has| |#1| (|SetCategory|)))
  ((0 74) NIL (|has| |#1| (|Ring|))))
(|characteristic| ((71) 73 (|has| |#1| (|Ring|))))
(^ ((0 0 94) NIL (|has| |#1| (|SemiGroup|)))
  ((0 0 71) NIL (|has| |#1| (|Monoid|)))
  ((0 0 74) NIL (|has| |#1| (|Group|))))
(|Zero| ((0) 55 (|has| |#1| (|AbelianGroup|)) CONST))
(|One| ((0) 63 (|has| |#1| (|Monoid|)) CONST))
(D ((0 0 25 71) NIL
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 28 95) NIL
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 25) NIL
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
  ((0 0 28) NIL
  (|has| |#1| (|PartialDifferentialRing| (|Symbol|))))
(= ((0 6 6) 20) ((38 0 0) 40 (|has| |#1| (|SetCategory|))))
(/ ((0 0 6) NIL (|has| |#1| (|Field|)))
  ((0 0 0) 90
  (OR (|has| |#1| (|Field|)) (|has| |#1| (|Group|))))
(- ((0 0 6) 54 (|has| |#1| (|AbelianGroup|)))
  ((0 6 0) 53 (|has| |#1| (|AbelianGroup|)))
  ((0 0 0) 52 (|has| |#1| (|AbelianGroup|)))
  ((0 0) 51 (|has| |#1| (|AbelianGroup|))))
(+ ((0 0 6) 49 (|has| |#1| (|AbelianSemiGroup|)))
  ((0 6 0) 48 (|has| |#1| (|AbelianSemiGroup|)))
  ((0 0 0) 47 (|has| |#1| (|AbelianSemiGroup|))))
(** ((0 0 94) NIL (|has| |#1| (|SemiGroup|)))

```

```

((0 0 71) NIL (|has| |#1| (|Monoid|)))
((0 0 74) NIL (|has| |#1| (|Group|)))
(* ((0 6 0) 60 (|has| |#1| (|SemiGroup|)))
   ((0 0 6) 61 (|has| |#1| (|SemiGroup|)))
   ((0 0 0) 59 (|has| |#1| (|SemiGroup|)))
   ((0 94 0) NIL (|has| |#1| (|AbelianSemiGroup|)))
   ((0 74 0) 76 (|has| |#1| (|AbelianGroup|)))
   ((0 71 0) NIL (|has| |#1| (|AbelianGroup|))))))

```

The “index”

```

(("slot1Info" 0 32444) ("documentation" 0 29640) ("ancestors" 0 28691)
 ("parents" 0 28077) ("abbreviation" 0 28074) ("predicates" 0 25442)
 ("attributes" 0 25304) ("signaturesAndLocals" 0 23933)
 ("superDomain" 0 NIL) ("operationAlist" 0 20053) ("modemaps" 0 17216)
 ("sourceFile" 0 17179) ("constructorCategory" 0 15220)
 ("constructorModemap" 0 13215) ("constructorKind" 0 13206)
 ("constructorForm" 0 13191) ("compilerInfo" 0 4433)
 ("loadTimeStuff" 0 20))

```

Chapter 3

Compiler top level

3.1 Global Data Structures

3.2 Pratt Parsing

Parsing involves understanding the association of symbols and operators. Vaughn Pratt [Prat73] poses the question “Given a substring AEB where A takes a right argument, B a left, and E is an expression, does E associate with A or B?”.

Floyd [Floy63] associates a precedence with operators, storing them in a table, called “binding powers”. The expression E would associate with the argument position having the highest binding power. This leads to a large set of numbers, one for every situation.

Pratt assigns data types to “classes” and then creates a total order on the classes. He lists, in ascending order, Outcomes, Booleans, Graphs (trees, lists, etc), Strings, Algebraics (e.g. Integer, complex numbers, polynomials, real arrays) and references (e.g. the left hand side of assignments). Thus, Strings $\leq$ References. The key restriction is “that the class of the type at any argument that might participate in an association problem not be less than the class of the data type of the result of the function taking that argument”.

For a less-than comparison (“ $<$ ”) the argument types are Algebraics but the result type is Boolean. Since Algebraics are greater than Boolean we can associate the Algebraics together and apply them as arguments to the Boolean.

In more detail, there an “association” is a function of 4 types:

- a_A – The data type of the right argument
- r_A – The return type of the right argument
- a_B – The data type of the left argument
- r_B – The return type of the left argument

Note that the return types might depend on the type of the expression E . If all 4 are of the same class then the association is to the left.

Using these ideas and given the restriction above, Pratt proves that every association problem has at most one solution consistent with the data types of the associated operators.

Pratt proves that there exists an assignment of integers to the argument positions of each token in the language such that the correct association, if any, is always in the direction of the argument position with the larger number, with ties being broken to the left.

To construct the proper numbers, first assign even integers to the data type classes. Then to each argument position assign an integer lying strictly (where possible) between the integers corresponding to the classes of the argument and result types.

For tokens like “and”, “or”, +, *, and the Booleans and Algebras can be subdivided into pseudo-classes so that

terms < factors < primaries

Then + is defined over terms, * over factors, and over primaries with coercions allowed from primaries to factors to terms. To be consistent with Algol, the primaries should be a right associative class (e.g. xyz)

3.3)compile

This is the implementation of the)compile command.

You use this command to invoke the new Axiom library compiler or the old Axiom system compiler. The)compile system command is actually a combination of Axiom processing and a call to the Aldor compiler. It is performing double-duty, acting as a front-end to both the Aldor compiler and the old Axiom system compiler. (The old Axiom system compiler was written in Lisp and was an integral part of the Axiom environment. The Aldor compiler is written in C and executed by the operating system when called from within Axiom.)

User Level Required: compiler

Command Syntax:

```

)compile

)compile fileName

)compile fileName.spad

)compile directory/fileName.spad

)compile fileName )old

)compile fileName )translate

)compile fileName )quiet

)compile fileName )noquiet

```

```

)compile fileName )moreargs
)compile fileName )onlyargs
)compile fileName )break
)compile fileName )nobreak
)compile fileName )library
)compile fileName )nolibrary
)compile fileName )vartrace
)compile fileName )constructor nameOrAbbrev

```

Command Description:

The first thing `)compile` does is look for a source code filename among its arguments. Thus

```

)compile mycode.spad
)compile /u/jones/mycode.spad
)compile mycode

```

all invoke `)compiler` on the file `/u/jones/mycode.spad` if the current Axiom working directory is `/u/jones`. (Recall that you can set the working directory via the `)cd` command. If you don't set it explicitly, it is the directory from which you started Axiom.)

If you omit the file extension, the command looks to see if you have specified the `)new` or `)old` option. If you have given one of these options, the corresponding compiler is used.

The command first looks in the standard system directories for files with extension `.as`, `.ao` and `.al` and then files with extension `.spad`. The first file found has the appropriate compiler invoked on it. If the command cannot find a matching file, an error message is displayed and the command terminates.

The first thing `)compile` does is look for a source code filename among its arguments. Thus

```

)compile mycode
)co mycode
)co mycode.spad

```

all invoke `)compiler` on the file `/u/jones/mycode.spad` if the current Axiom working directory is `/u/jones`. Recall that you can set the working directory via the `)cd` command. If you don't set it explicitly, it is the directory from which you started Axiom.

This is frequently all you need to compile your file.

This simple command:

1. Invokes the Spad compiler and produces Lisp output.
2. Calls the Lisp compiler if the compilation was successful.

3. Uses the `)library` command to tell Axiom about the contents of your compiled file and arrange to have those contents loaded on demand.

Should you not want the `)library` command automatically invoked, call `)compile` with the `)nolibrary` option. For example,

```
)compile mycode )nolibrary
```

By default, the `)library` system command *exposes* all domains and categories it processes. This means that the Axiom interpreter will consider those domains and categories when it is trying to resolve a reference to a function. Sometimes domains and categories should not be exposed. For example, a domain may just be used privately by another domain and may not be meant for top-level use. The `)library` command should still be used, though, so that the code will be loaded on demand. In this case, you should use the `)nolibrary` option on `)compile` and the `)noexpose` option in the `)library` command. For example,

```
)compile mycode )nolibrary
)library mycode )noexpose
```

Once you have established your own collection of compiled code, you may find it handy to use the `)dir` option on the `)library` command. This causes `)library` to process all compiled code in the specified directory. For example,

```
)library )dir /u/jones/quantum
```

You must give an explicit directory after `)dir`, even if you want all compiled code in the current working directory processed, e.g.

```
)library )dir .
```

Spad compiler

This command compiles files with file extension `.spad` with the Spad system compiler.

The `)translate` option is used to invoke a special version of the old system compiler that will translate a `.spad` file to a `.as` file. That is, the `.spad` file will be parsed and analyzed and a file using the new syntax will be created.

By default, the `.as` file is created in the same directory as the `.spad` file. If that directory is not writable, the current directory is used. If the current directory is not writable, an error message is given and the command terminates. Note that `)translate` implies the `)old` option so the file extension can safely be omitted. If `)translate` is given, all other options are ignored. Please be aware that the translation is not necessarily one hundred percent complete or correct. You should attempt to compile the output with the Aldor compiler and make any necessary corrections.

You can compile category, domain, and package constructors contained in files with file extension `.spad`. You can compile individual constructors or every constructor in a file.

The full filename is remembered between invocations of this command and `)edit` commands. The sequence of commands

```
)compile matrix.spad
)edit
)compile
```

will call the compiler, edit, and then call the compiler again on the file **matrix.spad**. If you do not specify a *directory*, the working current directory is searched for the file. If the file is not found, the standard system directories are searched.

If you do not give any options, all constructors within a file are compiled. Each constructor should have an `)abbreviation` command in the file in which it is defined. We suggest that you place the `)abbreviation` commands at the top of the file in the order in which the constructors are defined.

The `)library` option causes directories containing the compiled code for each constructor to be created in the working current directory. The name of such a directory consists of the constructor abbreviation and the **.nrlib** file extension. For example, the directory containing the compiled code for the **MATRIX** constructor is called **MATRIX.nrlib**. The `)nolibrary` option says that such files should not be created. The default is `)library`.

The `)vartrace` option causes the compiler to generate extra code for the constructor to support conditional tracing of variable assignments. Without this option, this code is suppressed and one cannot use the `)vars` option for the trace command.

The `)constructor` option is used to specify a particular constructor to compile. All other constructors in the file are ignored. The constructor name or abbreviation follows `)constructor`. Thus either

```
)compile matrix.spad )constructor RectangularMatrix
```

or

```
)compile matrix.spad )constructor RMATRIX
```

compiles the **RectangularMatrix** constructor defined in **matrix.spad**.

The `)break` and `)nobreak` options determine what the spad compiler does when it encounters an error. `)break` is the default and it indicates that processing should stop at the first error. The value of the `)set break` variable then controls what happens.

3.4 Operator Precedence Table Initialization

```
; PURPOSE: This file sets up properties which are used by the Boot lexical
; analyzer for bottom-up recognition of operators. Also certain
; other character-class definitions are included, as well as
; table accessing functions.
;
```

```
; ORGANIZATION: Each section is organized in terms of Creation and Access code.
;
;           1. Led and Nud Tables
;           2. GLIPH Table
;           3. RENAMETOK Table
;           4. GENERIC Table
;           5. Character syntax class predicates
```

LED and NUD Tables

```
; **** 1. LED and NUD Tables

; ** TABLE PURPOSE

; Led and Nud have to do with operators. An operator with a Led property takes
; an operand on its left (infix/suffix operator).

; An operator with a Nud takes no operand on its left (prefix/nilfix).
; Some have both (e.g. - ). This terminology is from the Pratt parser.
; The translator for Scratchpad II is a modification of the Pratt parser which
; branches to special handlers when it is most convenient and practical to
; do so (Pratt's scheme cannot handle local contexts very easily).

; Both LEDs and NUDs have right and left binding powers. This is meaningful
; for prefix and infix operators. These powers are stored as the values of
; the LED and NUD properties of an atom, if the atom has such a property.
; The format is:

;      <Operator Left-Binding-Power Right-Binding-Power <Special-Handler>>

; where the Special-Handler is the name of a function to be evaluated when that
; keyword is encountered.

; The default values of Left and Right Binding-Power are NIL. NIL is a
; legitimate value signifying no precedence. If the Special-Handler is NIL,
; this is just an ordinary operator (as opposed to a surfix operator like
; if-then-else).
;
; The Nud value gives the precedence when the operator is a prefix op.
; The Led value gives the precedence when the operator is an infix op.
; Each op has 2 priorities, left and right.
; If the right priority of the first is greater than or equal to the
; left priority of the second then collect the second operator into
; the right argument of the first operator.
```



```

; ** TABLE CREATION

(defun makenewop (x y) (makeop x y '|PARSE-NewKEY|))

(defun makeop (x y keyname)
  (if (or (not (cdr x)) (numberp (second x)))
      (setq x (cons (first x) x)))
  (if (and (alpha-char-p (elt (princ-to-string (first x)) 0))
          (not (member (first x) (eval keyname))))
      (set keyname (cons (first x) (eval keyname))))
  (put (first x) y x)
  (second x))

(setq |PARSE-NewKEY| nil) ;;list of keywords

(mapcar #'(LAMBDA(J) (MAKENEWOP J '|Led|))
  '((* 800 801) (|rem| 800 801) (|mod| 800 801)
    (|quo| 800 801) (|div| 800 801)
    (/ 800 801) (** 900 901) (^ 900 901)
    (|exquo| 800 801) (+ 700 701)
    (\- 700 701) (\-> 1001 1002) (\<\- 1001 1002)
    (\: 996 997) (\:\: 996 997)
    (\@ 996 997) (|pretend| 995 996)
    (\.) (\! \! 1002 1001)
    (\, 110 111)
    (\; 81 82 (|PARSE-SemiColon|))
    (\< 400 400) (\> 400 400)
    (\<\< 400 400) (\>\> 400 400)
    (\<= 400 400) (\>= 400 400)
    (= 400 400) (~= 400 400)
    (\~= 400 400)
    (|in| 400 400) (|case| 400 400)
    (|add| 400 120) (|with| 2000 400 (|PARSE-InfixWith|))
    (|has| 400 400)
    (|where| 121 104) ; must be 121 for SPAD, 126 for boot--> nboot
    (|when| 112 190)
    (|otherwise| 119 190 (|PARSE-Suffix|))
    (|is| 400 400) (|isnt| 400 400)
    (|and| 250 251) (|or| 200 201)
    (/ \ 250 251) (\ \ / 200 201)
    (\.\. SEGMENT 401 699 (|PARSE-Seg|))
    (= \> 123 103)
    (+- \> 995 112)
    (== DEF 122 121)
    (== \> MDEF 122 121)
    (\| 108 111) ;was 190 190
    (\:- LETD 125 124) (\:= LET 125 124)))

(mapcar #'(LAMBDA(J) (MAKENEWOP J '|Nud|))
  '((|for| 130 350 (|PARSE-Loop|))

```

```

(|while| 130 190 (|PARSE-Loop|))
(|until| 130 190 (|PARSE-Loop|))
(|repeat| 130 190 (|PARSE-Loop|))
(|import| 120 0 (|PARSE-Import|) )
(|unless|)
(|add| 900 120)
(|with| 1000 300 (|PARSE-With|))
(|has| 400 400)
(|- 701 700) ; right-prec. wants to be -1 + left-prec
;;
(|+ 701 700)
(|# 999 998)
(|! 1002 1001)
(|' 999 999 (|PARSE-Data|))
(|<< 122 120 (|PARSE-LabelExpr|))
(|>>)
(|^ 260 259 NIL)
(|-> 1001 1002)
(|: 194 195)
(|not| 260 259 NIL)
(|~ 260 259 nil)
(|= 400 700)
(|return| 202 201 (|PARSE-Return|))
(|leave| 202 201 (|PARSE-Leave|))
(|exit| 202 201 (|PARSE-Exit|))
(|from|)
(|iterate|)
(|yield|)
(|if| 130 0 (|PARSE-Conditional|)) ; was 130
(| 0 190)
(|suchthat|)
(|then| 0 114)
(|else| 0 114)))

```

3.5 Glyph Table

Gliph is a symbol clump. The gliph property of a symbol gives the tree describing the tokens which begin with that symbol. The token reader uses the gliph property to determine the longest token. Thus `:=` is read as one token not as `:` followed by `=`.

— GLIPHTable —

```

(mapcar #'(lambda (x) (put (car x) 'gliph (cdr x)))
  '(
    ( \ | (\))

```

```

( * (*) )
( \ ( (<) (\|) )
( + (- (>)) )
( - (>) )
( < (=) (<) )
;; ( / (\) ) breaks */xxx
( \ \ (/) )
( > (=) (>) (\))
( = (= (>)) (>) )
( \. (\.) )
( ^ (=) )
( \~ (=) )
( \: (=) (-) (\:)))

```

Rename Token Table

RENAMETOK defines alternate token strings which can be used for different keyboards which define equivalent tokens.

— **RENAMETOKTable** —

```

(mapcar #'(lambda (x) (put (car x) 'renametok (cadr x)) (makenewop x nil))
  '( (\(\| \[) ; (| |) means []
    (\| \])
    (\(< \{) ; (< >) means {}
    (>\ \})))

```

Generic function table

GENERIC operators be suffixed by \$ qualifications in SPAD code. \$ is then followed by a domain label, such as I for Integer, which signifies which domain the operator refers to. For example **+\$Integer** is + for Integers.

— **GENERICTable** —

```

(mapcar #'(lambda (x) (put x 'generic 'true))
  '(- = * |rem| |mod| |quo| |div| / ** |exquo| + - < > <= >= ^= ))

```

3.6 Giant steps, Baby steps

We will walk through the compiler with the EQ.spad example using a Giant-steps, Baby-steps approach. That is, we will show the large scale (Giant) transformations at each stage of compilation and discuss the details (Baby) in subsequent chapters.

Chapter 4

The Parser

4.1 EQ.spad

We will explain the compilation function using the file `EQ.spad`. We trace the execution of the various functions to understand the actual call parameters and results returned. The `EQ.spad` file is:

```
)abbrev domain EQ Equation
--FOR THE BENEFIT OF LIBAXO GENERATION
++ Author: Stephen M. Watt, enhancements by Johannes Grabmeier
++ Date Created: April 1985
++ Date Last Updated: June 3, 1991; September 2, 1992
++ Basic Operations: =
++ Related Domains:
++ Also See:
++ AMS Classifications:
++ Keywords: equation
++ Examples:
++ References:
++ Description:
++ Equations as mathematical objects. All properties of the basis domain,
++ e.g. being an abelian group are carried over the equation domain, by
++ performing the structural operations on the left and on the
++ right hand side.
-- The interpreter translates "=" to "equation". Otherwise, it will
-- find a modemap for "=" in the domain of the arguments.

Equation(S: Type): public == private where
  Ex ==> OutputForm
  public ==> Type with
    "=": (S, S) -> $
    ++ a=b creates an equation.
```

```

equation: (S, S) -> $
  ++ equation(a,b) creates an equation.
swap: $ -> $
  ++ swap(eq) interchanges left and right hand side of equation eq.
lhs: $ -> S
  ++ lhs(eqn) returns the left hand side of equation eqn.
rhs: $ -> S
  ++ rhs(eqn) returns the right hand side of equation eqn.
map: (S -> S, $) -> $
  ++ map(f,eqn) constructs a new equation by applying f to both
  ++ sides of eqn.
if S has InnerEvalable(Symbol,S) then
  InnerEvalable(Symbol,S)
if S has SetCategory then
  SetCategory
  CoercibleTo Boolean
  if S has Evalable(S) then
    eval: ($, $) -> $
      ++ eval(eqn, x=f) replaces x by f in equation eqn.
    eval: ($, List $) -> $
      ++ eval(eqn, [x1=v1, ... xn=vn]) replaces xi by vi in equation eqn.
if S has AbelianSemiGroup then
  AbelianSemiGroup
  "+": (S, $) -> $
    ++ x+eqn produces a new equation by adding x to both sides of
    ++ equation eqn.
  "+": ($, S) -> $
    ++ eqn+x produces a new equation by adding x to both sides of
    ++ equation eqn.
if S has AbelianGroup then
  AbelianGroup
  leftZero : $ -> $
    ++ leftZero(eq) subtracts the left hand side.
  rightZero : $ -> $
    ++ rightZero(eq) subtracts the right hand side.
  "-": (S, $) -> $
    ++ x-eqn produces a new equation by subtracting both sides of
    ++ equation eqn from x.
  "-": ($, S) -> $
    ++ eqn-x produces a new equation by subtracting x from both sides of
    ++ equation eqn.
if S has SemiGroup then
  SemiGroup
  "*": (S, $) -> $
    ++ x*eqn produces a new equation by multiplying both sides of
    ++ equation eqn by x.
  "*": ($, S) -> $
    ++ eqn*x produces a new equation by multiplying both sides of
    ++ equation eqn by x.
if S has Monoid then

```

```

Monoid
leftOne : $ -> Union($,"failed")
  ++ leftOne(eq) divides by the left hand side, if possible.
rightOne : $ -> Union($,"failed")
  ++ rightOne(eq) divides by the right hand side, if possible.
if S has Group then
  Group
  leftOne : $ -> Union($,"failed")
    ++ leftOne(eq) divides by the left hand side.
  rightOne : $ -> Union($,"failed")
    ++ rightOne(eq) divides by the right hand side.
if S has Ring then
  Ring
  BiModule(S,S)
if S has CommutativeRing then
  Module(S)
  --Algebra(S)
if S has IntegralDomain then
  factorAndSplit : $ -> List $
    ++ factorAndSplit(eq) make the right hand side 0 and
    ++ factors the new left hand side. Each factor is equated
    ++ to 0 and put into the resulting list without repetitions.
if S has PartialDifferentialRing(Symbol) then
  PartialDifferentialRing(Symbol)
if S has Field then
  VectorSpace(S)
  "/": ($, $) -> $
    ++ e1/e2 produces a new equation by dividing the left and right
    ++ hand sides of equations e1 and e2.
  inv: $ -> $
    ++ inv(x) returns the multiplicative inverse of x.
if S has ExpressionSpace then
  subst: ($, $) -> $
    ++ subst(eq1,eq2) substitutes eq2 into both sides of eq1
    ++ the lhs of eq2 should be a kernel

private ==> add
Rep := Record(lhs: S, rhs: S)
eq1,eq2: $
s : S
if S has IntegralDomain then
  factorAndSplit eq ==
    (S has factor : S -> Factored S) =>
      eq0 := rightZero eq
      [equation(rcf.factor,0) for rcf in factors factor lhs eq0]
  [eq]
  l:S = r:S == [l, r]
  equation(l, r) == [l, r] -- hack! See comment above.
  lhs eqn == eqn.lhs
  rhs eqn == eqn.rhs

```

```

swap eqn      == [rhs eqn, lhs eqn]
map(fn, eqn)  == equation(fn(eqn.lhs), fn(eqn.rhs))

if S has InnerEvalable(Symbol,S) then
  s:Symbol
  ls:List Symbol
  x:S
  lx:List S
  eval(eqn,s,x) == eval(eqn.lhs,s,x) = eval(eqn.rhs,s,x)
  eval(eqn,ls,lx) == eval(eqn.lhs,ls,lx) = eval(eqn.rhs,ls,lx)
if S has Evalable(S) then
  eval(eqn1:$, eqn2:$):$ ==
    eval(eqn1.lhs, eqn2 pretend Equation S) =
      eval(eqn1.rhs, eqn2 pretend Equation S)
  eval(eqn1:$, leqn2:List $):$ ==
    eval(eqn1.lhs, leqn2 pretend List Equation S) =
      eval(eqn1.rhs, leqn2 pretend List Equation S)
if S has SetCategory then
  eq1 = eq2 == (eq1.lhs = eq2.lhs)@Boolean and
               (eq1.rhs = eq2.rhs)@Boolean
  coerce(eqn:$):Ex == eqn.lhs::Ex = eqn.rhs::Ex
  coerce(eqn:$):Boolean == eqn.lhs = eqn.rhs
if S has AbelianSemiGroup then
  eq1 + eq2 == eq1.lhs + eq2.lhs = eq1.rhs + eq2.rhs
  s + eq2 == [s,s] + eq2
  eq1 + s == eq1 + [s,s]
if S has AbelianGroup then
  - eq == (- lhs eq) = (-rhs eq)
  s - eq2 == [s,s] - eq2
  eq1 - s == eq1 - [s,s]
  leftZero eq == 0 = rhs eq - lhs eq
  rightZero eq == lhs eq - rhs eq = 0
  0 == equation(0$S,0$S)
  eq1 - eq2 == eq1.lhs - eq2.lhs = eq1.rhs - eq2.rhs
if S has SemiGroup then
  eq1:$ * eq2:$ == eq1.lhs * eq2.lhs = eq1.rhs * eq2.rhs
  1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs
  1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs
  eqn:$ * 1:S == eqn.lhs * 1 = eqn.rhs * 1
  -- We have to be a bit careful here: raising to a +ve integer is OK
  -- (since it's the equivalent of repeated multiplication)
  -- but other powers may cause contradictions
  -- Watch what else you add here! JHD 2/Aug 1990
if S has Monoid then
  1 == equation(1$S,1$S)
  recip eq ==
    (lh := recip lhs eq) case "failed" => "failed"
    (rh := recip rhs eq) case "failed" => "failed"
    [lh :: S, rh :: S]
  leftOne eq ==

```



```

      (re := recip lhs eq) case "failed" => "failed"
      1 = rhs eq * re
    rightOne eq ==
      (re := recip rhs eq) case "failed" => "failed"
      lhs eq * re = 1
  if S has Group then
    inv eq == [inv lhs eq, inv rhs eq]
    leftOne eq == 1 = rhs eq * inv rhs eq
    rightOne eq == lhs eq * inv rhs eq = 1
  if S has Ring then
    characteristic() == characteristic()$S
    i:Integer * eq:$ == (i::S) * eq
  if S has IntegralDomain then
    factorAndSplit eq ==
      (S has factor : S -> Factored S) =>
        eq0 := rightZero eq
        [equation(rcf.factor,0) for rcf in factors factor lhs eq0]
      (S has Polynomial Integer) =>
        eq0 := rightZero eq
        MF ==> MultivariateFactorize(Symbol, IndexedExponents Symbol, _
          Integer, Polynomial Integer)
        p : Polynomial Integer := (lhs eq0) pretend Polynomial Integer
        [equation((rcf.factor) pretend S,0) for rcf in factors factor(p)$MF]
      [eq]
  if S has PartialDifferentialRing(Symbol) then
    differentiate(eq:$, sym:Symbol):$ ==
      [differentiate(lhs eq, sym), differentiate(rhs eq, sym)]
  if S has Field then
    dimension() == 2 :: CardinalNumber
    eq1:$ / eq2:$ == eq1.lhs / eq2.lhs = eq1.rhs / eq2.rhs
    inv eq == [inv lhs eq, inv rhs eq]
  if S has ExpressionSpace then
    subst(eq1,eq2) ==
      eq3 := eq2 pretend Equation S
      [subst(lhs eq1,eq3),subst(rhs eq1,eq3)]

```

4.2 boot transformations

defun string2BootTree

```

[new2OldLisp p78]
[def-rename p585]
[boot-line-stack p??]
[xtokenreader p??]
[line-handler p??]
[$boot p??]

```

[[\\$spad p571](#)]

— defun string2BootTree —

```
(defun string2BootTree (s)
  (init-boot/spad-reader)
  (let* ((boot-line-stack (list (cons 1 s)))
        ($boot t)
        ($spad nil)
        (xtokenreader 'get-boot-token)
        (line-handler 'next-boot-line)
        (parseout (progn (|PARSE-Expression|) (pop-stack-1))))
    (declare (special boot-line-stack $boot $spad xtokenreader line-handler))
    (def-rename (new2OldLisp parseout))))
```

—————

defun new2OldLisp

[[new2OldTran p78](#)]
 [[postTransform p369](#)]

— defun new2OldLisp —

```
(defun new2OldLisp (x)
  (new2OldTran (postTransform x)))
```

—————

defun new2OldTran

[[dcq p??](#)]
 [[new2OldTran p78](#)]
 [[newDef2Def p80](#)]
 [[newIf2Cond p79](#)]
 [[newConstruct p80](#)]
 [[\\$new2OldRenameAssoc p??](#)]

— defun new2OldTran —

```
(defun new2OldTran (x)
  (prog (tmp1 tmp2 tmp3 tmp4 tmp5 tmp6 a b c d)
    (declare (special |$new2OldRenameAssoc|))
    (return
```

```

(prog nil
  (if (atom x)
    (return (let ((y (assoc x |$new2OldRenameAssoc|)))
              (if y (cdr y) x))))
  (if (and (dcq (tmp1 a b . tmp2) x)
    (null tmp2)
    (eq tmp1 '|where|)
    (dcq (tmp3 . tmp4) b)
    (dcq ((tmp5 d . tmp6) . c) (reverse tmp4))
    (null tmp6)
    (eq tmp5 '|exit|)
    (eq tmp3 'seq)
    (or (setq c (nreverse c)) t))
    (return
      '(|where| ,(new2OldTran a) ,@(new2OldTran c)
        ,(new2OldTran d))))
  (return
    (case (car x)
      (quote x)
      (def (newDef2Def x))
      (if (newIf2Cond x))
      ; construct == #'list (see patches.lisp) TPD 12/2011
      (|construct| (newConstruct (new2OldTran (cdr x))))
      (t '(',(new2OldTran (car x)) . ,(new2OldTran (cdr x))))))))

```

defun newIf2Cond

[letError p??]
[new2OldTran p78]

— defun newIf2Cond —

```

(defun newIf2Cond (cond-expr)
  (if (not (and (= (length cond-expr) 4) (eq (car cond-expr) 'if)))
    (letError "(IF,a,b,c)" cond-expr)
    (let ((a (second cond-expr))
          (b (third cond-expr))
          (c (fourth cond-expr)))
      (setq a (new2OldTran a) b (new2OldTran b) c (new2OldTran c))
      (if (eq c '|noBranch|)
        '(if ,a ,b)
        '(if ,a ,b ,c))))

```

```
(defun letError (form val) (—systemError— (format nil " S is not matched by structure S
```

defun newDef2Def

```
[letError p??]  
[new2OldDefForm p80]  
[new2OldTran p78]
```

— defun newDef2Def —

```
(defun newDef2Def (def-expr)  
  (if (not (and (= (length def-expr) 5) (eq (car def-expr) 'def)))  
      (letError "(DEF,form,a,b,c)" def-expr)  
      (let ((form (second def-expr))  
            (a (third def-expr))  
            (b (fourth def-expr))  
            (c (fifth def-expr)))  
        '(def ,(new2OldDefForm form) ,(new2OldTran a)  
              ,(new2OldTran b) ,(new2OldTran c)))))
```

defun new2OldDefForm

```
[new2OldTran p78]  
[new2OldDefForm p80]
```

— defun new2OldDefForm —

```
(defun new2OldDefForm (x)  
  (cond  
    ((atom x) (new2OldTran x))  
    ((and (listp x) (listp (car x)) (eq (caar x) '|is|) (= (length (car x)) 3))  
      (let ((a (second (car x))) (b (third (car x))) (y (cdr x)))  
        (new2OldDefForm '((setq ,a ,b) ,@y))))  
    ((cons (new2OldTran (car x)) (new2OldDefForm (cdr x))))))
```

defun newConstruct

— defun newConstruct —

```
(defun newConstruct (z)
  (if (atom z)
      z
      '(cons ,(car z) ,(newConstruct (cdr z)))))
```

4.3 preparse

The first large transformation of this input occurs in the function `preparse`. The `preparse` function reads the source file and breaks the input into a list of pairs. The first part of the pair is the line number of the input file and the second part of the pair is the actual source text as a string.

One feature that is added is semicolons at the end of the strings where the “pile” structure of the code has been converted to a semicolon delimited form.

defvar \$index

— initvars —

```
(defvar $index 0 "File line number of most recently read line")
```

defvar \$linelist

— initvars —

```
(defvar $linelist nil "Stack of preparsed lines")
```

defvar \$echolinestack

— initvars —

```
(defvar $echolinestack nil "Stack of lines to list")
```

```
defvar $preparse-last-line
```

```
— initvars —
```

```
(defvar $preparse-last-line nil "Most recently read line")
```

4.4 Parsing routines

The **initialize-preparse** expects to be called before the **preparse** function. It initializes the state, in particular, it reads a single line from the input stream and stores it in **\$preparse-last-line**. The caller gives a stream and the **\$preparse-last-line** variable is initialized as:

```
2> (INITIALIZE-PREPARSE #<input stream "/tmp/EQ.spad">)
<2 (INITIALIZE-PREPARSE ")abbrev domain EQ Equation")
```

```
defun initialize-preparse
```

```
[get-a-line p636]
[$index p81]
[$linelist p81]
[$echolinestack p81]
[$preparse-last-line p82]
```

```
— defun initialize-preparse —
```

```
(defun initialize-preparse (strm)
  (setq $index 0)
  (setq $linelist nil)
  (setq $echolinestack nil)
  (setq $preparse-last-line (get-a-line strm)))
```

The **preparse** function returns a list of pairs of the form: ((linenumber . linestring) (linenumber . linestring)) For instance, for the file **EQ.spad**, we get:

```

2> (PREPARSE #<input stream "/tmp/EQ.spad">)
3> (PREPARSE1 ("abbrev domain EQ Equation"))
4> (|doSystemCommand| "abbrev domain EQ Equation")
<4 (|doSystemCommand| NIL)
<3 (PREPARSE1 ( ...[snip]... )
<2 (PREPARSE (
(19 . "Equation(S: Type): public == private where")
(20 . " (Ex ==> OutputForm;")
(21 . "   public ==> Type with")
(22 . "     (\ "=": (S, S) -> $;")
(24 . "       equation: (S, S) -> $;")
(26 . "         swap: $ -> $;")
(28 . "           lhs: $ -> S;")
(30 . "             rhs: $ -> S;")
(32 . "               map: (S -> S, $) -> $;")
(35 . "                 if S has InnerEvalable(Symbol,S) then")
(36 . "                   InnerEvalable(Symbol,S);")
(37 . "                 if S has SetCategory then")
(38 . "                   (SetCategory;")
(39 . "                     CoercibleTo Boolean;")
(40 . "                     if S has Evalable(S) then")
(41 . "                       (eval: ($, $) -> $;")
(43 . "                         eval: ($, List $) -> $));")
(45 . "                 if S has AbelianSemiGroup then")
(46 . "                   (AbelianSemiGroup;")
(47 . "                     \"+\": (S, $) -> $;")
(50 . "                     \"+\": ($, S) -> $);")
(53 . "                 if S has AbelianGroup then")
(54 . "                   (AbelianGroup;")
(55 . "                     leftZero : $ -> $;")
(57 . "                     rightZero : $ -> $;")
(59 . "                     \"-\": (S, $) -> $;")
(62 . "                     \"-\": ($, S) -> $);")
(65 . "                 if S has SemiGroup then")
(66 . "                   (SemiGroup;")
(67 . "                     \ "* \": (S, $) -> $;")
(70 . "                     \ "* \": ($, S) -> $);")
(73 . "                 if S has Monoid then")
(74 . "                   (Monoid;")
(75 . "                     leftOne : $ -> Union($,\"failed\");")
(77 . "                     rightOne : $ -> Union($,\"failed\");")
(79 . "                 if S has Group then")
(80 . "                   (Group;")
(81 . "                     leftOne : $ -> Union($,\"failed\");")
(83 . "                     rightOne : $ -> Union($,\"failed\");")
(85 . "                 if S has Ring then")
(86 . "                   (Ring;")
(87 . "                     BiModule(S,S));")
(88 . "                 if S has CommutativeRing then")
(89 . "                   Module(S);")

```

```

(91 . "    if S has IntegralDomain then")
(92 . "        factorAndSplit : $ -> List $;"")
(96 . "    if S has PartialDifferentialRing(Symbol) then")
(97 . "        PartialDifferentialRing(Symbol);"")
(98 . "    if S has Field then")
(99 . "        (VectorSpace(S);"")
(100 . "        \"/\": ($, $) -> $;"")
(103 . "        inv: $ -> $);"")
(105 . "    if S has ExpressionSpace then")
(106 . "        subst: ($, $) -> $);"")
(109 . "private ==> add")
(110 . "    (Rep := Record(lhs: S, rhs: S);"")
(111 . "        eq1,eq2: $;"")
(112 . "        s : S;"")
(113 . "    if S has IntegralDomain then")
(114 . "        factorAndSplit eq ==")
(115 . "            ((S has factor : S -> Factored S) =>")
(116 . "                (eq0 := rightZero eq;"")
(117 . "                    [equation(rcf.factor,0)
                        for rcf in factors factor lhs eq0]);")
(118 . "                    [eq]);")
(119 . "    l:S = r:S      == [l, r];")
(120 . "    equation(l, r) == [l, r];")
(121 . "    lhs eqn        == eqn.lhs;"")
(122 . "    rhs eqn        == eqn.rhs;"")
(123 . "    swap eqn       == [rhs eqn, lhs eqn];")
(124 . "    map(fn, eqn)    == equation(fn(eqn.lhs), fn(eqn.rhs));")
(125 . "    if S has InnerEvalable(Symbol,S) then")
(126 . "        (s:Symbol;"")
(127 . "        ls:List Symbol;"")
(128 . "        x:S;"")
(129 . "        lx:List S;"")
(130 . "        eval(eqn,s,x) == eval(eqn.lhs,s,x) = eval(eqn.rhs,s,x);"")
(131 . "        eval(eqn,ls,lx) == eval(eqn.lhs,ls,lx) =
                                eval(eqn.rhs,ls,lx));")
(132 . "    if S has Evalable(S) then")
(133 . "        (eval(eqn1:$, eqn2:$):$ ==")
(134 . "            eval(eqn1.lhs, eqn2 pretend Equation S) =")
(135 . "                eval(eqn1.rhs, eqn2 pretend Equation S);")
(136 . "            eval(eqn1:$, leqn2:List $):$ ==")
(137 . "                eval(eqn1.lhs, leqn2 pretend List Equation S) =")
(138 . "                    eval(eqn1.rhs, leqn2 pretend List Equation S));")
(139 . "    if S has SetCategory then")
(140 . "        (eq1 = eq2 == (eq1.lhs = eq2.lhs)@Boolean and")
(141 . "            (eq1.rhs = eq2.rhs)@Boolean;"")
(142 . "        coerce(eqn:$):Ex == eqn.lhs::Ex = eqn.rhs::Ex;"")
(143 . "        coerce(eqn:$):Boolean == eqn.lhs = eqn.rhs);"")
(144 . "    if S has AbelianSemiGroup then")
(145 . "        (eq1 + eq2 == eq1.lhs + eq2.lhs = eq1.rhs + eq2.rhs;"")
(146 . "            s + eq2 == [s,s] + eq2);"")

```



```

(147 . "      eq1 + s == eq1 + [s,s]);")
(148 . "  if S has AbelianGroup then")
(149 . "    (- eq == (- lhs eq) = (-rhs eq));")
(150 . "      s - eq2 == [s,s] - eq2;")
(151 . "      eq1 - s == eq1 - [s,s];")
(152 . "      leftZero eq == 0 = rhs eq - lhs eq;")
(153 . "      rightZero eq == lhs eq - rhs eq = 0;")
(154 . "      0 == equation(0$S,0$S);")
(155 . "      eq1 - eq2 == eq1.lhs - eq2.lhs = eq1.rhs - eq2.rhs);")
(156 . "  if S has SemiGroup then")
(157 . "    (eq1:$ * eq2:$ == eq1.lhs * eq2.lhs = eq1.rhs * eq2.rhs;")
(158 . "      1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs;")
(159 . "      1:S * eqn:$ == 1 * eqn.lhs = 1 * eqn.rhs;")
(160 . "      eqn:$ * 1:S == eqn.lhs * 1 = eqn.rhs * 1);")
(165 . "  if S has Monoid then")
(166 . "    (1 == equation(1$S,1$S);")
(167 . "      recip eq ==")
(168 . "        ((lh := recip lhs eq) case \"failed\" => \"failed\");")
(169 . "        (rh := recip rhs eq) case \"failed\" => \"failed\");")
(170 . "        [lh :: S, rh :: S]);")
(171 . "      leftOne eq ==")
(172 . "        ((re := recip lhs eq) case \"failed\" => \"failed\");")
(173 . "        1 = rhs eq * re);")
(174 . "      rightOne eq ==")
(175 . "        ((re := recip rhs eq) case \"failed\" => \"failed\");")
(176 . "        lhs eq * re = 1));")
(177 . "  if S has Group then")
(178 . "    (inv eq == [inv lhs eq, inv rhs eq];")
(179 . "      leftOne eq == 1 = rhs eq * inv rhs eq;")
(180 . "      rightOne eq == lhs eq * inv rhs eq = 1);")
(181 . "  if S has Ring then")
(182 . "    (characteristic() == characteristic())$S;")
(183 . "    i:Integer * eq:$ == (i::S) * eq;")
(184 . "  if S has IntegralDomain then")
(185 . "    factorAndSplit eq ==")
(186 . "      ((S has factor : S -> Factored S) =>")
(187 . "        (eq0 := rightZero eq;")
(188 . "          [equation(rcf.factor,0)
            for rcf in factors factor lhs eq0]));")
(189 . "      (S has Polynomial Integer) =>")
(190 . "        (eq0 := rightZero eq;")
(191 . "          MF ==> MultivariateFactorize(Symbol,
            IndexedExponents Symbol,
            Integer, Polynomial Integer);")
(193 . "          p : Polynomial Integer :=
            (lhs eq0) pretend Polynomial Integer;")
(194 . "          [equation((rcf.factor) pretend S,0)
            for rcf in factors factor(p)$MF]);")
(195 . "          [eq]);")
(196 . "  if S has PartialDifferentialRing(Symbol) then")

```

```

(197 . "      differentiate(eq:$, sym:Symbol):$ ==")
(198 . "      [differentiate(lhs eq, sym), differentiate(rhs eq, sym)];")
(199 . "      if S has Field then")
(200 . "      (dimension() == 2 :: CardinalNumber;")
(201 . "      eq1:$ / eq2:$ == eq1.lhs / eq2.lhs = eq1.rhs / eq2.rhs;")
(202 . "      inv eq == [inv lhs eq, inv rhs eq]);")
(203 . "      if S has ExpressionSpace then")
(204 . "      subst(eq1,eq2) ==")
(205 . "      (eq3 := eq2 pretend Equation S;")
(206 . "      [subst(lhs eq1,eq3),subst(rhs eq1,eq3)])))))

```

defun preparse

```

[preparse p86]
[preparse1 p87]
[parseprint p554]
[ifcar p??]
[$comblocklist p551]
[$skipme p??]
[$preparse-last-line p82]
[$index p81]
[$docList p??]
[$preparseReportIfTrue p??]
[$headerDocumentation p??]
[$maxSignatureLineNumber p??]
[$constructorLineNumber p??]

```

— defun preparse —

```

(defun preparse (strm &aux (stack ()))
  (declare (special $comblocklist $skipme $preparse-last-line $index |$docList|
                    $preparseReportIfTrue |$headerDocumentation|
                    |$maxSignatureLineNumber| |$constructorLineNumber|))
  (setq $comblocklist nil)
  (setq $skipme nil)
  (when $preparse-last-line
    (if (consp $preparse-last-line)
        (setq stack $preparse-last-line)
        (push $preparse-last-line stack))
    (setq $index (- $index (length stack))))
  (let ((u (preparse1 stack)))
    (if $skipme
        (preparse strm)
        (progn
         (when $preparseReportIfTrue (parseprint u))
         (setq |$headerDocumentation| nil)
         (setq |$docList| nil)
         (setq |$maxSignatureLineNumber| 0)

```

```
(setq |$constructorLineNumber| (ifcar (ifcar u)))
u)))
```

defun Build the lines from the input for piles

The READLOOP calls `preparseReadLine` which returns a pair of the form

```
(number . string)
```

```
[preparseReadLine p??]
[preparse-echo p95]
[fincomblock p551]
[parsepiles p??]
[preparse1 doSystemCommand (vol5)]
[escaped p551]
[indent-pos p552]
[make-full-cvec p??]
[maxindex p??]
[preparse1 strposl (vol5)]
[is-console p553]
[spad-reader p??]
[$echolinestack p81]
[$byConstructors p626]
[$skipme p??]
[$constructorsSeen p626]
[$preparse-last-line p82]
[$preparse-last-line p82]
[$index p81]
[$index p81]
[$linelist p81]
[$in-stream p??]
```

preparse1 : (List String) → (List (Cons NNI String)) where the input **List String** is the **abbrev** line:

```
(")abbrev domain EQ Equation")
\end{verbatim}
and the output is a {\bf List (Cons NNI String)} containing
a line number and the String from that line:
\begin{verbatim}
... (13 . " (Ex ==> OutputForm;")...)
\end{verbatim}
\begin{chunk}{defun preparse1}
```

```

(defun preparsel (linelist)
  (labels (
    (isSystemCommand (line)
      (and (> (length line) 0) (eq (char line 0) #\ ) )))
    (executeSystemCommand (line)
      (catch 'spad_reader (|doSystemCommand| (subseq line 1))))
  )
  (prog (($linelist linelist) $echolinestack num line i l psloc
    instring pcount comsym strsym oparsym cparsym n ncomsym tmp1
    (sloc -1) continue (parenlev 0) ncomblock lines locs nums functor)
    (declare (special $linelist $echolinestack |$byConstructors| $skipme
      |$constructorsSeen| $preparsel-last-line $index in-stream))
  READLOOP
    (setq tmp1 (preparselReadLine linelist))
    (setq num (car tmp1))
    (setq line (cdr tmp1))
    (unless (stringp line)
      (preparsel-echo linelist)
      (cond
        ((null lines) (return nil))
        (ncomblock (fincomblock nil nums locs ncomblock nil)))
      (return
        (pair (nreverse nums) (parsepiles (nreverse locs) (nreverse lines)))))
    (when (and (null lines) (isSystemCommand line))
      (preparsel-echo linelist)
      (setq $preparsel-last-line nil) ;don't reread this line
      (executeSystemCommand line)
      (go READLOOP))
    (setq l (length line))
    ; if we get a null line, read the next line
    (when (eq l 0) (go READLOOP))
    ; otherwise we have to parse this line
    (setq psloc sloc)
    (setq i 0)
    (setq instring nil)
    (setq pcount 0)
  STRLOOP ;; handle things that need ignoring, quoting, or grouping
    ; are we in a comment, quoting, or grouping situation?
    (setq strsym (or (position #\" line :start i ) 1))
    (setq comsym (or (search "--" line :start2 i ) 1))
    (setq ncomsym (or (search "++" line :start2 i ) 1))
    (setq oparsym (or (position #\" line :start i ) 1))
    (setq cparsym (or (position #\\ line :start i ) 1))
    (setq n (min strsym comsym ncomsym oparsym cparsym))
    (cond
      ; nope, we found no comment, quoting, or grouping
      ((= n 1) (go NOCOMS))
      ((escaped line n))
      ; scan until we hit the end of the string
      ((= n strsym) (setq instring (not instring)))

```

```

; we are in a string, just continue looping
(instring)
;; handle -- comments by ignoring them
((= n comsym)
 (setq line (subseq line 0 n))
 (go NOCOMS)) ; discard trailing comment
;; handle ++ comments by chunking them together
((= n ncomsym)
 (setq sloc (indent-pos line))
 (cond
  ((= sloc n)
   (when (and ncomblock (not (= n (car ncomblock))))
     (fincomblock num nums locs ncomblock linelist)
     (setq ncomblock nil))
   (setq ncomblock (cons n (cons line (ifcdr ncomblock))))
   (setq line ""))
  (t
   (push (strconc (make-full-cvec n " ") (substring line n ())) $linelist)
   (setq $index (1- $index))
   (setq line (subseq line 0 n))))
 (go NOCOMS))
; know how deep we are into parens
((= n oparsym) (setq pcount (1+ pcount)))
((= n cparsym) (setq pcount (1- pcount)))
(setq i (1+ n))
(go STRLOOP)
NOCOMS
; remember the indentation level
(setq sloc (indent-pos line))
(setq line (string-right-trim " " line))
(when (null sloc)
 (setq sloc psloc)
 (go READLOOP))
; handle line that ends in a continuation character
(cond
 ((eq (elt line (maxindex line)) #\_)
  (setq continue t)
  (setq line (subseq line (maxindex line))))
 ((setq continue nil)))
; test for skipping constructors
(when (and (null lines) (= sloc 0))
 (if (and |$byConstructors|
        (null (search "==" line))
        (not
         (member
          (setq functor
            (intern (substring line 0 (strpos1 ":(=" line 0 nil))))
          |$byConstructors|)))
      (setq $skipme 't)
      (progn

```

```

        (push functor |$constructorsSeen|)
        (setq $skipme nil))))
; is this thing followed by ++ comments?
(when (and lines (eql sloc 0))
  (when (and ncomblock (not (zerop (car ncomblock))))
    (fincomblock num nums locs ncomblock linelist))
  (when (not (is-console in-stream))
    (setq $preparse-last-line (nreverse $echolinestack)))
  (return
    (pair (nreverse nums) (parsepiles (nreverse locs) (nreverse lines))))))
(when (> parenlev 0)
  (push nil locs)
  (setq sloc psloc)
  (go REREAD))
(when ncomblock
  (fincomblock num nums locs ncomblock linelist)
  (setq ncomblock ()))
(push sloc locs)
REREAD
  (preparse-echo linelist)
  (push line lines)
  (push num nums)
  (setq parenlev (+ parenlev pcount))
  (when (and (is-console in-stream) (not continue))
    (setq $preparse-last-line nil)
    (return
      (pair (nreverse nums) (parsepiles (nreverse locs) (nreverse lines))))))
  (go READLOOP)))

\end{chunk}

\defun{parsepiles}{parsepiles}
Add parens and semis to lines to aid parsing.
\calls{parsepiles}{add-parens-and-semis-to-line}
\begin{chunk}{\defun parsepiles}
(defun parsepiles (locs lines)
  (mapl #'add-parens-and-semis-to-line
    (nconc lines '(" ")) (nconc locs '(nil)))
  lines)

\end{chunk}

\defun{add-parens-and-semis-to-line}{add-parens-and-semis-to-line}
The line to be worked on is (CAR SLINES). It's indentation is (CAR SLOCS).
There is a notion of current indentation. Then:
\begin{itemize}
\item Add open paren to beginning of following line if following line's
indentation is greater than current, and add close paren to end of
last succeeding line with following line's indentation.
\item Add semicolon to end of line if following line's indentation is the same.

```

```

\item If the entire line consists of the single keyword then or else,
leave it alone."
\end{itemize}
\calls{add-parens-and-semis-to-line}{infixtok}
\calls{add-parens-and-semis-to-line}{drop}
\calls{add-parens-and-semis-to-line}{addclose}
\calls{add-parens-and-semis-to-line}{nonblankloc}
\begin{chunk}{defun add-parens-and-semis-to-line}
(defun add-parens-and-semis-to-line (slines slocs)
  (let ((start-column (car slocs)))
    (when (and start-column (> start-column 0))
      (let ((count 0) (i 0))
        (seq
         (mapl #'(lambda (next-lines nlocs)
                   (let ((next-line (car next-lines)) (next-column (car nlocs)))
                     (incf i)
                     (when next-column
                      (setq next-column (abs next-column))
                      (when (< next-column start-column) (exit nil))
                      (cond
                       ((and (eq next-column start-column)
                            (rplaca nlocs (- (car nlocs))))
                        (not (infixtok next-line)))
                       (setq next-lines (drop (1- i) slines))
                       (rplaca next-lines (addclose (car next-lines) #\;))
                       (setq count (1+ count)))))))
                    (cdr slines) (cdr slocs)))
        (when (> count 0)
          (setf (char (car slines) (1- (nonblankloc (car slines)))) #\ ( )
                (setq slines (drop (1- i) slines))
                (rplaca slines (addclose (car slines) #\ ) ))))))))
\end{chunk}

\defun{preparseReadLine}{preparseReadLine}
\calls{preparseReadLine}{preparseReadLine1}
\calls{preparseReadLine}{initial-substring}
\calls{preparseReadLine}{string2BootTree}
\calls{preparseReadLine}{storeblanks}
\calls{preparseReadLine}{skip-to-endif}
\calls{preparseReadLine}{preparseReadLine}
\refsdollar{preparseReadLine}{*eof*}
\sig{preparseReadLine1}{nil}{(Cons NNI String)}
where the result is a pair with the next input line number (\$index)
and the input string.
\begin{verbatim}
(0 . ")abbrev domain EQ Equation"

```

— defun preparseReadLine —

```

(defun preparsedReadLine (x)
  (let (line ind tmp1)
    (declare (special *eof*))
    (setq tmp1 (preparsedReadLine1))
    (setq ind (car tmp1))
    (setq line (cdr tmp1))
    (cond
      ((not (stringp line)) (cons ind line))
      ((zerop (size line)) (cons ind line))
      ((char= (elt line 0) #\ )
       (cond
         ((initial-substring ")if" line)
          (if (eval (string2BootTree (storeblanks line 3)))
              (preparsedReadLine x)
              (skip-ifblock x)))
         ((initial-substring ")elseif" line) (skip-to-endif x))
         ((initial-substring ")else" line) (skip-to-endif x))
         ((initial-substring ")endif" line) (preparsedReadLine x))
         ((initial-substring ")fin" line)
          (setq *eof* t)
          (cons ind nil))))))
    (cons ind line)))

```

defun skip-ifblock

[\[preparsedReadLine1 p93\]](#)
[\[skip-ifblock p92\]](#)
[\[initial-substring p635\]](#)
[\[string2BootTree p77\]](#)
[\[storeblanks p635\]](#)

— defun skip-ifblock —

```

(defun skip-ifblock (x)
  (let (line ind tmp1)
    (setq tmp1 (preparsedReadLine1))
    (setq ind (car tmp1))
    (setq line (cdr tmp1))
    (cond
      ((not (stringp line))
       (cons ind line))
      ((zerop (size line))
       (skip-ifblock x))
      ((char= (elt line 0) #\ )
       (cond

```



```

((initial-substring ")if" line)
  (cond
    ((eval (string2BootTree (storeblanks line 3)))
      (preparseReadLine X))
    (t (skip-ifblock x))))
((initial-substring ")elseif" line)
  (cond
    ((eval (string2BootTree (storeblanks line 7)))
      (preparseReadLine X))
    (t (skip-ifblock x))))
((initial-substring ")else" line)
  (preparseReadLine x))
((initial-substring ")endif" line)
  (preparseReadLine x))
((initial-substring ")fin" line)
  (cons ind nil))))
(t (skip-ifblock x))))

```

defun preparseReadLine1

```

[get-a-line p636]
[expand-tabs p94]
[maxindex p??]
[strconc p??]
[preparseReadLine1 p93]
[$linelist p81]
[$linelist p81]
[$preparse-last-line p82]
[$index p81]
[$index p81]
[$EchoLineStack p??]

```

preparseReadLine1 : **nil** → (**Cons NNI String**) where the result is a pair with the next input line number (**\$index**) and the input string.

```
(0 . ")abbrev domain EQ Equation")
```

— **defun preparseReadLine1** —

```

(defun preparseReadLine1 ()
  (labels (
    (accumulateLinesWithTrailingEscape (line)
      (let (ind)

```

```

(declare (special $preparse-last-line))
(if (and (> (setq ind (maxindex line)) -1) (char= (elt line ind) #\_))
    (setq $preparse-last-line
        (strconc (substring line 0 ind) (cdr (preparseReadLine1))))
    line)))
(let (line)
(declare (special $linelist $preparse-last-line $index $EchoLineStack))
(setq line
    (if $linelist
        (pop $linelist)
        (expand-tabs (get-a-line in-stream))))
(setq $preparse-last-line line)
(if (stringp line)
    (progn
        (incf $index) ;; $index is the current line number
        (setq line (string-right-trim " " line))
        (push (copy-seq line) $EchoLineStack)
        (cons $index (accumulateLinesWithTrailingEscape line)))
    (cons $index line))))

```

defun expand-tabs

[nonblankloc p553]

[indent-pos p552]

— defun expand-tabs —

```

(defun expand-tabs (str)
  (if (and (stringp str) (> (length str) 0))
      (let ((bpos (nonblankloc str))
            (tpos (indent-pos str)))
        (setq str
            (if (eql bpos tpos)
                str
                (concatenate 'string (make-string tpos :initial-element #\space)
                             (subseq str bpos))))
        ;; remove dos CR
        (let ((lpos (maxindex str)))
          (if (eq (char str lpos) #\Return)
              (subseq str 0 lpos)
              str)))
      str))

```

4.5 I/O Handling

defun preparse-echo

```
[Echo-Meta p??]
[ $EchoLineStack p??]
```

— defun preparse-echo —

```
(defun preparse-echo (linelist)
  (declare (special $EchoLineStack Echo-Meta) (ignore linelist))
  (if Echo-Meta
    (dolist (x (reverse $EchoLineStack))
      (format out-stream "~&;~A~%" x))
    (setq $EchoLineStack ()))
```

—————

Parsing stack

defstruct stack

— initvars —

```
(defstruct stack      "A stack"
  (store nil)        ; contents of the stack
  (size 0)           ; number of elements in Store
  (top nil)          ; first element of Store
  (updated nil)      ; whether something has been pushed on the stack
                    ; since this flag was last set to NIL
)
```

—————

defun stack-load

```
[ $stack p95]
```

— defun stack-load —

```
(defun stack-load (list stack)
  (setf (stack-store stack) list)
  (setf (stack-size stack) (length list))
```

```
(setf (stack-top stack) (car list)))
```

defun stack-clear

[[\\$stack p95](#)]

— defun stack-clear —

```
(defun stack-clear (stack)
  (setf (stack-store stack) nil)
  (setf (stack-size stack) 0)
  (setf (stack-top stack) nil)
  (setf (stack-updated stack) nil))
```

defmacro stack-/-empty

[[\\$stack p95](#)]

— defmacro stack-/-empty —

```
(defmacro stack-/-empty (stack) '(> (stack-size ,stack) 0))
```

defun stack-push

[[\\$stack p95](#)]

— defun stack-push —

```
(defun stack-push (x stack)
  (push x (stack-store stack))
  (setf (stack-top stack) x)
  (setf (stack-updated stack) t)
  (incf (stack-size stack))
  x)
```

defun stack-pop

[\$stack p95]

— defun stack-pop —

```
(defun stack-pop (stack)
  (let ((y (pop (stack-store stack))))
    (decf (stack-size stack))
    (setf (stack-top stack)
          (if (stack-/empty stack) (car (stack-store stack)))
          y)))
```

—————

Parsing token**defstruct token**

A token is a Symbol with a Type. The type is either NUMBER, IDENTIFIER or SPECIAL-CHAR. NonBlank is true if the token is not preceded by a blank.

— initvars —

```
(defstruct token
  (symbol nil)
  (type nil)
  (nonblank t))
```

—————

defvar prior-token

[\$token p97]

— initvars —

```
(defvar prior-token (make-token) "What did I see last")
```

—————

defvar nonblank

— initvars —

```
(defvar nonblank t "Is there no blank in front of the current token.")
```

defvar current-token

Token at head of input stream. [Token p97]

— **initvars** —

```
(defvar current-token (make-token))
```

defvar next-token

[Token p97]

— **initvars** —

```
(defvar next-token (make-token) "Next token in input stream.")
```

defvar valid-tokens

[Token p97]

— **initvars** —

```
(defvar valid-tokens 0 "Number of tokens in buffer (0, 1 or 2)")
```

defun token-install

[Token p97]

— **defun token-install** —

```
(defun token-install (symbol type token &optional (nonblank t))
  (setf (token-symbol token) symbol)
  (setf (token-type token) type)
  (setf (token-nonblank token) nonblank)
  token)
```

defun token-print

[`$token` p97]

— defun token-print —

```
(defun token-print (token)
  (format out-stream "(token (symbol ~S) (type ~S))~%"
    (token-symbol token) (token-type token)))
```

Parsing reduction

defstruct reduction

A reduction of a rule is any S-Expression the rule chooses to stack.

— **initvars** —

```
(defstruct (reduction (:type list))
  (rule nil)           ; Name of rule
  (value nil))
```

Chapter 5

Parse Transformers

5.1 Direct called parse routines

defun parseTransform

[parseTran p101]
[\$defOp p??]

— defun parseTransform —

```
(defun |parseTransform| (x)
  (let (|$defOp|)
    (declare (special |$defOp|))
    (setq |$defOp| nil)
    (setq x (subst '$ '% x :test #'equal)) ; for new compiler compatibility
    (|parseTran| x)))
```

—————

defun parseTran

[parseAtom p102]
[parseConstruct p103]
[parseTran p101]
[parseTranList p103]
[getl p??]
[\$op p??]

— defun parseTran —

```

(defun |parseTran| (x)
  (labels (
    (g (op)
      (let (tmp1 tmp2 x)
        (seq
          (if (and (consp op) (eq (qfirst op) '|elt|)
              (progn
                (setq tmp1 (qrest op))
                (and (consp tmp1)
                  (progn
                    (setq op (qfirst tmp1))
                    (setq tmp2 (qrest tmp1))
                    (and (consp tmp2)
                      (eq (qrest tmp2) nil)
                      (progn (setq x (qfirst tmp2)) t))))))
              (exit (g x)))
          (exit op))))))
  (let (|$op| argl u r fn)
    (declare (special |$op|))
    (setq |$op| nil)
    (if (atom x)
        (|parseAtom| x)
        (progn
          (setq |$op| (car x))
          (setq argl (cdr x))
          (setq u (g |$op|))
          (cond
            ((eq u '|construct|)
             (setq r (|parseConstruct| argl))
             (if (and (consp |$op|) (eq (qfirst |$op|) '|elt|))
                 (cons (|parseTran| |$op|) (cdr r))
                 r))
            ((and (atom u) (setq fn (getl u '|parseTran|)))
             (funcall fn argl))
            (t (cons (|parseTran| |$op|) (|parseTranList| argl)))))))

```

defun parseAtom

[parseLeave p129]
 [\$NoValue p??]

— defun parseAtom —

```

(defun |parseAtom| (x)
  (declare (special |$NoValue|))

```

```
(if (eq x '|break|)
    (|parseLeave| (list '|$NoValue|'))
    x))
```

defun parseTranList

```
[parseTran p101]
[parseTranList p103]
```

— defun parseTranList —

```
(defun |parseTranList| (x)
  (if (atom x)
      (|parseTran| x)
      (cons (|parseTran| (car x)) (|parseTranList| (cdr x)))))
```

deflist parseConstruct

— postvars —

```
(eval-when (eval load)
  (setf (get '|construct| '|parseTran|) '|parseConstruct|))
```

defun parseConstruct

```
[parseTranList p103]
[$insideConstructIfTrue p??]
```

— defun parseConstruct —

```
(defun |parseConstruct| (u)
  (let (|$insideConstructIfTrue| x)
    (declare (special |$insideConstructIfTrue|))
    (setq |$insideConstructIfTrue| t)
    (setq x (|parseTranList| u))
    (cons '|construct| x)))
```

5.2 Indirect called parse routines

In the `parseTran` function there is the code:

```
((and (atom u) (setq fn (getl u '|parseTran|)))
  (funcall fn argl))
```

The functions in this section are called through the symbol-plist of the symbol being parsed. The original list read:

<code>and</code>	<code>parseAnd</code>
<code>@</code>	<code>parseAtSign</code>
<code>CATEGORY</code>	<code>parseCategory</code>
<code>::</code>	<code>parseCoerce</code>
<code>\:</code>	<code>parseColon</code>
<code>construct</code>	<code>parseConstruct</code>
<code>DEF</code>	<code>parseDEF</code>
<code>\$<=</code>	<code>parseDollarLessEqual</code>
<code>\$></code>	<code>parseDollarGreaterThan</code>
<code>\$>=</code>	<code>parseDollarGreaterEqual</code>
<code>\$^=</code>	<code>parseDollarNotEqual</code>
<code>eqv</code>	<code>parseEquivalence</code>
<code>exit</code>	<code>parseExit</code>
<code>></code>	<code>parseGreaterThan</code>
<code>>=</code>	<code>parseGreaterEqual</code>
<code>has</code>	<code>parseHas</code>
<code>IF</code>	<code>parseIf</code>
<code>implies</code>	<code>parseImplies</code>
<code>IN</code>	<code>parseIn</code>
<code>INBY</code>	<code>parseInBy</code>
<code>is</code>	<code>parseIs</code>
<code>isnt</code>	<code>parseIsnt</code>
<code>Join</code>	<code>parseJoin</code>
<code>leave</code>	<code>parseLeave</code>
<code>;;control-H</code>	<code>parseLeftArrow</code>
<code><=</code>	<code>parseLessEqual</code>
<code>LET</code>	<code>parseLET</code>
<code>LETD</code>	<code>parseLETD</code>
<code>MDEF</code>	<code>parseMDEF</code>
<code>^</code>	<code>parseNot</code>
<code>not</code>	<code>parseNot</code>
<code>^=</code>	<code>parseNotEqual</code>
<code>or</code>	<code>parseOr</code>
<code>pretend</code>	<code>parsePretend</code>
<code>return</code>	<code>parseReturn</code>
<code>SEGMENT</code>	<code>parseSegment</code>

```

SEQ          parseSeq
;;control-V  parseUpArrow
VCONS       parseVCONS
where        parseWhere

```

defplist parseAnd

— postvars —

```

(eval-when (eval load)
  (setf (get '|and| '|parseTran|) '|parseAnd|))

```

—————

defun parseAnd

```

[parseTran p101]
[parseAnd p105]
[parseTranList p103]
[parseIf p122]
[$InteractiveMode p??]

```

— defun parseAnd —

```

(defun |parseAnd| (arg)
  (cond
    (|$InteractiveMode| (cons '|and| (|parseTranList| arg)))
    ((null arg) '|true|)
    ((null (cdr arg)) (car arg))
    (t
     (|parseIf|
      (list (|parseTran| (car arg)) (|parseAnd| (CDR arg)) '|false| )))))

```

—————

defplist parseAtSign

— postvars —

```

(eval-when (eval load)
  (setf (get '|@ '|parseTran|) '|parseAtSign|))

```

defun parseAtSign

[parseTran p101]
 [parseType p106]
 [\$InteractiveMode p??]

— defun parseAtSign —

```
(defun |parseAtSign| (arg)
  (declare (special |$InteractiveMode|))
  (if |$InteractiveMode|
    (list '@ (|parseTran| (first arg)) (|parseTran| (|parseType| (second arg))))
    (list '@ (|parseTran| (first arg)) (|parseTran| (second arg)))))
```

defun parseType

[parseTran p101]

— defun parseType —

```
(defun |parseType| (x)
  (declare (special |$EmptyMode| |$quadSymbol|))
  (setq x (subst |$EmptyMode| |$quadSymbol| x :test #'equal))
  (if (and (consp x) (eq (qfirst x) '|typeOf|)
        (consp (qrest x)) (eq (qcddr x) nil))
      (list '|typeOf| (|parseTran| (qsecond x)))
      x))
```

defplist parseCategory

— postvars —

```
(eval-when (eval load)
  (setf (get 'category '|parseTran|) '|parseCategory|))
```

defun parseCategory

[parseTranList p103]
 [parseDropAssertions p107]
 [contained p??]

— defun parseCategory —

```
(defun |parseCategory| (arg)
  (let (z key)
    (setq z (|parseTranList| (|parseDropAssertions| arg)))
    (setq key (if (contained '$ z) '|domain| '|package|))
    (cons 'category (cons key z))))
```

—————

defun parseDropAssertions

[parseDropAssertions p107]

— defun parseDropAssertions —

```
(defun |parseDropAssertions| (x)
  (cond
    ((not (consp x)) x)
    ((and (consp (qfirst x)) (eq (qcaar x) 'if)
          (consp (qcadar x))
          (eq (qcadar x) '|asserted|))
     (|parseDropAssertions| (qrest x)))
    (t (cons (qfirst x) (|parseDropAssertions| (qrest x))))))
```

—————

defplist parseCoerce

— postvars —

```
(eval-when (eval load)
  (setf (get '|::| '|parseTran|) '|parseCoerce|))
```

—————

defun parseCoerce

```
[parseType p106]
[parseTran p101]
[$InteractiveMode p??]
```

— defun parseCoerce —

```
(defun |parseCoerce| (arg)
  (if |$InteractiveMode|
    (list '|::|
      (|parseTran| (first arg)) (|parseTran| (|parseType| (second arg))))
    (list '|::| (|parseTran| (first arg)) (|parseTran| (second arg)))))
```

defplist parseColon

— postvars —

```
(eval-when (eval load)
  (setf (get '|::| '|parseTran|) '|parseColon|))
```

defun parseColon

```
[parseTran p101]
[parseType p106]
[$InteractiveMode p??]
[$insideConstructIfTrue p??]
```

— defun parseColon —

```
(defun |parseColon| (arg)
  (declare (special |$insideConstructIfTrue|))
  (cond
    ((and (consp arg) (eq (qrest arg) nil))
     (list '|::| (|parseTran| (first arg))))
    ((and (consp arg) (consp (qrest arg)) (eq (qcddr arg) nil))
     (if |$InteractiveMode|
       (if |$insideConstructIfTrue|
         (list 'tag (|parseTran| (first arg))
```



```

      (|parseTran| (second arg)))
    (list '|:| (|parseTran| (first arg))
      (|parseTran| (|parseType| (second arg)))))
  (list '|:| (|parseTran| (first arg))
    (|parseTran| (second arg)))))

```

defplist parseDEF

— postvars —

```

(eval-when (eval load)
  (setf (get 'def '|parseTran|) '|parseDEF|))

```

defun parseDEF

```

[setDefOp p404]
[parseLhs p110]
[parseTranList p103]
[parseTranCheckForRecord p543]
[opFf p??]
[$lhs p??]

```

— defun parseDEF —

```

(defun |parseDEF| (arg)
  (let (|$lhs| tList specialList body)
    (declare (special |$lhs|))
    (setq |$lhs| (first arg))
    (setq tList (second arg))
    (setq specialList (third arg))
    (setq body (fourth arg))
    (setDefOp |$lhs|)
    (list 'def (|parseLhs| |$lhs|)
      (|parseTranList| tList)
      (|parseTranList| specialList)
      (|parseTranCheckForRecord| body (|opOf| |$lhs|)))))

```

defun parseLhs

[parseTran p101]
[transIs p110]

— defun parseLhs —

```
(defun |parseLhs| (x)
  (let (result)
    (cond
      ((atom x) (|parseTran| x))
      ((atom (car x))
       (cons (|parseTran| (car x))
              (dolist (y (cdr x) (nreverse result))
                (push (|transIs| (|parseTran| y)) result))))
      (t (|parseTran| x)))))
```

—————

defun transIs

[isListConstructor p111]
[transIs1 p110]

— defun transIs —

```
(defun |transIs| (u)
  (if (|isListConstructor| u)
      (cons '|construct| (|transIs1| u))
      u))
```

—————

defun transIs1

[nreverse0 p??]
[transIs p110]
[transIs1 p110]

— defun transIs1 —

```
(defun |transIs1| (u)
  (let (x h v tmp3)
    (cond
```

```

((and (consp u) (eq (qfirst u) '|construct|))
  (dolist (x (qrest u) (nreverse0 tmp3))
    (push (|transIs| x) tmp3)))
((and (consp u) (eq (qfirst u) '|append|) (consp (qrest u))
  (consp (qcddr u)) (eq (qcdddr u) nil))
  (setq x (qsecond u))
  (setq h (list '|:| (|transIs| x)))
  (setq v (|transIs1| (qthird u)))
  (cond
    ((and (consp v) (eq (qfirst v) '|:|)
      (consp (qrest v)) (eq (qcddr v) nil))
      (list h (qsecond v)))
    ((eq v '|nil|) (car (cdr h)))
    ((atom v) (list h (list '|:| v)))
    (t (cons h v))))
((and (consp u) (eq (qfirst u) '|cons|) (consp (qrest u))
  (consp (qcddr u)) (eq (qcdddr u) nil))
  (setq h (|transIs| (qsecond u)))
  (setq v (|transIs1| (qthird u)))
  (cond
    ((and (consp v) (eq (qfirst v) '|:|) (consp (qrest v))
      (eq (qcddr v) nil))
      (cons h (list (qsecond v))))
    ((eq v '|nil|) (cons h nil))
    ((atom v) (list h (list '|:| v)))
    (t (cons h v))))
(t u))))

```

defun isListConstructor

[member p??]

— defun isListConstructor —

```

(defun |isListConstructor| (u)
  (and (consp u) (|member| (qfirst u) '(|construct| |append| |cons|))))

```

defplist parseDollarGreaterthan

— postvars —

```
(eval-when (eval load)
  (setf (get '$>' |parseTran|) '|parseDollarGreaterThan|))
```

defun parseDollarGreaterThan

```
[parseTran p101]
[$op p??]
```

— defun parseDollarGreaterThan —

```
(defun |parseDollarGreaterThan| (arg)
  (declare (special |$op|))
  (list (subst '$<' '$>' |$op| :test #'equal)
        (|parseTran| (second arg))
        (|parseTran| (first arg))))
```

deflist parseDollarGreaterEqual

— postvars —

```
(eval-when (eval load)
  (setf (get '$>=' |parseTran|) '|parseDollarGreaterEqual|))
```

defun parseDollarGreaterEqual

```
[parseTran p101]
[$op p??]
```

— defun parseDollarGreaterEqual —

```
(defun |parseDollarGreaterEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '$<' '$>=' |$op| :test #'equal) arg)))))
```

— postvars —

```
(eval-when (eval load)
  (setf (get '|$<=|' |parseTran|) '|parseDollarLessEqual|))
```

defun parseDollarLessEqual

```
[parseTran p101]
[$op p??]
```

— defun parseDollarLessEqual —

```
(defun |parseDollarLessEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '$> '$<= |$op| :test #'equal) arg)))))
```

deflist parseDollarNotEqual

— postvars —

```
(eval-when (eval load)
  (setf (get '|$^=|' |parseTran|) '|parseDollarNotEqual|))
```

defun parseDollarNotEqual

```
[parseTran p101]
[$op p??]
```

— defun parseDollarNotEqual —

```
(defun |parseDollarNotEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '$= '$^= |$op| :test #'equal) arg)))))
```

defplist parseEquivalence

— postvars —

```
(eval-when (eval load)
  (setf (get 'eqv| 'parseTran|) '|parseEquivalence|))
```

defun parseEquivalence

[parseIf p122]

— defun parseEquivalence —

```
(defun |parseEquivalence| (arg)
  (|parseIf|
   (list (first arg) (second arg)
         (|parseIf| (cons (second arg) '(|false| |true|))))))
```

defplist parseExit

— postvars —

```
(eval-when (eval load)
  (setf (get 'exit| 'parseTran|) '|parseExit|))
```

defun parseExit

```
[parseTran p101]
[moan p??]
```

— defun parseExit —

```
(defun |parseExit| (arg)
  (let (a b)
    (setq a (|parseTran| (car arg)))
    (setq b (|parseTran| (cdr arg)))
    (if b
      (cond
        ((null (integerp a))
         (moan "first arg " a " for exit must be integer")
         (list '|exit| 1 a ))
        (t
         (cons '|exit| (cons a b))))
      (list '|exit| 1 a )))))
```

defpllist parseGreaterEqual

— postvars —

```
(eval-when (eval load)
  (setf (get '|>=| '|parseTran|) '|parseGreaterEqual|))
```

defun parseGreaterEqual

```
[parseTran p101]
[$op p??]
```

— defun parseGreaterEqual —

```
(defun |parseGreaterEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '< '>= |$op| :test #'equal) arg)))))
```

defpllist parseGreaterThan

— postvars —

```
(eval-when (eval load)
  (setf (get '|>' '|parseTran|) '|parseGreaterThan|))
```

defun parseGreaterThan

```
[parseTran p101]
[$op p??]
```

— defun parseGreaterThan —

```
(defun |parseGreaterThan| (arg)
  (declare (special |$op|))
  (list (subst '<' '>' |$op| :test #'equal)
        (|parseTran| (second arg)) (|parseTran| (first arg))))
```

defplist parseHas

— postvars —

```
(eval-when (eval load)
  (setf (get '|has|' '|parseTran|) '|parseHas|))
```

defun parseHas

```
[unabbrevAndLoad p??]
[getdatabase p??]
[opOf p??]
[makeNonAtomic p??]
[parseHasRhs p118]
[member p??]
[parseType p106]
[nreverse0 p??]
[$InteractiveMode p??]
[$CategoryFrame p??]
```

— defun parseHas —


```

(defun |parseHas| (arg)
  (labels (
    (fn (arg)
      (let (tmp4 tmp6 map op kk)
        (declare (special |$InteractiveMode|))
        (when |$InteractiveMode| (setq arg (|unabbrevAndLoad| arg)))
        (cond
          ((and (consp arg) (eq (qfirst arg) '|:|) (consp (qrest arg))
            (consp (qcddr arg)) (eq (qcdddr arg) nil)
            (consp (qthird arg))
            (eq (qcaaddr arg) '|Mapping|))
           (setq map (rest (third arg)))
           (setq op (second arg))
           (setq op (if (stringp op) (intern op) op))
           (list (list 'signature op map)))
          ((and (consp arg) (eq (qfirst arg) '|Join|))
           (dolist (z (rest arg) tmp4)
             (setq tmp4 (append tmp4 (fn z))))))
          ((and (consp arg) (eq (qfirst arg) 'category))
           (dolist (z (rest arg) tmp6)
             (setq tmp6 (append tmp6 (fn z))))))
          (t
           (setq kk (getdatabase (|opOf| arg) 'constructorkind))
           (cond
            ((or (eq kk '|domain|) (eq kk '|category|))
             (list (|makeNonAtomic| arg)))
            ((and (consp arg) (eq (qfirst arg) 'attribute))
             (list arg))
            ((and (consp arg) (eq (qfirst arg) 'signature))
             (list arg))
            (|$InteractiveMode|
             (|parseHasRhs| arg))
            (t
             (list (list 'attribute arg)))))))
    (let (tmp1 tmp2 tmp3 x)
      (declare (special |$InteractiveMode| |$CategoryFrame|))
      (setq x (first arg))
      (setq tmp1 (|get| x '|value| |$CategoryFrame|))
      (when |$InteractiveMode|
        (setq x
          (if (and (consp tmp1) (consp (qrest tmp1)) (consp (qcddr tmp1))
            (eq (qcdddr tmp1) nil)
            (|member| (second tmp1)
              '((|Mode|) (|Domain|) (|SubDomain| (|Domain|)))))
            (first tmp1)
            (|parseType| x))))
      (setq tmp2
        (dolist (u (fn (second arg)) (nreverse0 tmp3))
          (push (list '|has| x u ) tmp3)))
      (if (and (consp tmp2) (eq (qrest tmp2) nil))

```

```
(qfirst tmp2)
(cons '|and| tmp2))))))
```

defun parseHasRhs

```
[get p??]
[member p??]
[abbreviation? p??]
[loadIfNecessary p118]
[unabbrevAndLoad p??]
[$CategoryFrame p??]
```

— defun parseHasRhs —

```
(defun |parseHasRhs| (u)
  (let (tmp1 y)
    (declare (special |$CategoryFrame|))
    (setq tmp1 (|get| u '|value| |$CategoryFrame|))
    (cond
      ((and (consp tmp1) (consp (qrest tmp1))
            (consp (qcddr tmp1)) (eq (qcdddr tmp1) nil)
            (|member| (second tmp1)
                      '((|Mode|) (|Domain|) (|SubDomain| (|Domain|)))))
        (second tmp1))
      ((setq y (|abbreviation?| u))
        (if (|loadIfNecessary| y)
            (list (|unabbrevAndLoad| y))
            (list (list 'attribute u))))
      (t (list (list 'attribute u))))))
```

defun loadIfNecessary

```
[loadLibIfNecessary p119]
```

— defun loadIfNecessary —

```
(defun |loadIfNecessary| (u)
  (|loadLibIfNecessary| u t))
```

defun loadLibIfNecessary

```
[loadLibIfNecessary p119]
[canFuncall? p??]
[macrop p??]
[getl p??]
[loadLib p??]
[lassoc p??]
[getProplist p??]
[getdatabase p??]
[updateCategoryFrameForCategory p121]
[updateCategoryFrameForConstructor p120]
[throwKeyedMsg p??]
[$CategoryFrame p??]
[$InteractiveMode p??]
```

— defun loadLibIfNecessary —

```
(defun |loadLibIfNecessary| (u mustExist)
  (let (value y)
    (declare (special |$CategoryFrame| |$InteractiveMode|))
    (cond
      ((eq u '|$EmptyMode|) u)
      ((null (atom u)) (|loadLibIfNecessary| (car u) mustExist))
      (t
       (setq value
        (cond
          ((or (canFuncall? u) (|macrop| u)) u)
          ((getl u 'loaded) u)
          ((|loadLib| u) u)))
        (cond
          ((and (null |$InteractiveMode|)
                (or (null (setq y (|getProplist| u |$CategoryFrame|)))
                    (and (null (lassoc '|isFunctor| y))
                        (null (lassoc '|isCategory| y))))))
           (if (setq y (getdatabase u 'constructorkind))
               (if (eq y '|category|)
                   (|updateCategoryFrameForCategory| u)
                   (|updateCategoryFrameForConstructor| u))
               (|throwKeyedMsg| " %1p is not a known type." (list u))))
          (t value))))))
```

—————

defun updateCategoryFrameForConstructor

```
[getdatabase p??]
[put p??]
[convertOpAlist2compilerInfo p120]
[addModemap p273]
[$CategoryFrame p??]
[$CategoryFrame p??]
```

— defun updateCategoryFrameForConstructor —

```
(defun |updateCategoryFrameForConstructor| (constructor)
  (let (opAlist tmp1 dc sig pred impl)
    (declare (special |$CategoryFrame|))
    (setq opalist (getdatabase constructor 'operationalist))
    (setq tmp1 (getdatabase constructor 'constructormodemap))
    (setq dc (caar tmp1))
    (setq sig (cdar tmp1))
    (setq pred (caadr tmp1))
    (setq impl (cadadr tmp1))
    (setq |$CategoryFrame|
      (|put| constructor '|isFunctor|
        (|convertOpAlist2compilerInfo| opalist)
        (|addModemap| constructor dc sig pred impl
          (|put| constructor '|mode| (cons '|Mapping| sig) |$CategoryFrame|))))))
```

defun convertOpAlist2compilerInfo

— defun convertOpAlist2compilerInfo —

```
(defun |convertOpAlist2compilerInfo| (opalist)
  (labels (
    (formatSig (op arg2)
      (let (typelist slot stuff pred impl)
        (setq typelist (car arg2))
        (setq slot (cadr arg2))
        (setq stuff (cddr arg2))
        (setq pred (if stuff (car stuff) t))
        (setq impl (if (cdr stuff) (cadr stuff) 'elt))
        (list (list op typelist) pred (list impl '$ slot)))))
    (let (data result)
      (setq data
        (loop for item in opalist
          collect
```

```

(loop for sig in (rest item)
  collect (formatSig (car item) sig))))
(dolist (term data result)
  (setq result (append result term))))))

```

defun updateCategoryFrameForCategory

```

[getdatabase p??]
[put p??]
[addModemap p273]
[$CategoryFrame p??]
[$CategoryFrame p??]

```

— defun updateCategoryFrameForCategory —

```

(defun |updateCategoryFrameForCategory| (category)
  (let (tmp1 dc sig pred impl)
    (declare (special |$CategoryFrame|))
    (setq tmp1 (getdatabase category 'constructormodemap))
    (setq dc (caar tmp1))
    (setq sig (cdar tmp1))
    (setq pred (caadr tmp1))
    (setq impl (cadadr tmp1))
    (setq |$CategoryFrame|
      (|put| category '|isCategory| t
        (|addModemap| category dc sig pred impl |$CategoryFrame|))))))

```

defplist parseIf

— postvars —

```

(eval-when (eval load)
  (setf (get 'if '|parseTran|) '|parseIf|))

```

defun parseIf

[parseIf,ifTran p122]
 [parseTran p101]

— defun parseIf —

```
(defun |parseIf| (arg)
  (if (null (and (consp arg) (consp (qrest arg))
                (consp (qcddr arg)) (eq (qcdddr arg) nil)))
      arg
      (|parseIf,ifTran|
       (|parseTran| (first arg))
       (|parseTran| (second arg))
       (|parseTran| (third arg)))))
```

—————

defun parseIf,ifTran

[parseIf,ifTran p122]
 [incExitLevel p??]
 [makeSimplePredicateOrNil p544]
 [incExitLevel p??]
 [parseTran p101]
 [\$InteractiveMode p??]

— defun parseIf,ifTran —

```
(defun |parseIf,ifTran| (pred a b)
  (let (pp z ap bp tmp1 tmp2 tmp3 tmp4 tmp5 tmp6 val s)
    (declare (special |$InteractiveMode|))
    (cond
      ((and (null |$InteractiveMode|) (eq pred '|true|))
       a)
      ((and (null |$InteractiveMode|) (eq pred '|false|))
       b)
      ((and (consp pred) (eq (qfirst pred) '|not|)
            (consp (qrest pred)) (eq (qcddr pred) nil))
       (|parseIf,ifTran| (second pred) b a))
      ((and (consp pred) (eq (qfirst pred) '|if|)
            (progn
              (setq tmp1 (qrest pred))
              (and (consp tmp1)
                   (progn
                     (setq pp (qfirst tmp1))
                     (setq tmp2 (qrest tmp1))
```

```

      (and (consp tmp2)
        (progn
          (setq ap (qfirst tmp2))
          (setq tmp3 (qrest tmp2))
          (and (consp tmp3)
            (eq (qrest tmp3) nil)
            (progn (setq bp (qfirst tmp3)) t))))))
    (|parseIf,ifTran| pp
      (|parseIf,ifTran| ap (copy a) (copy b))
      (|parseIf,ifTran| bp a b)))
  ((and (consp pred) (eq (qfirst pred) 'seq)
    (consp (qrest pred)) (progn (setq tmp2 (reverse (qrest pred))) t)
    (and (consp tmp2)
      (consp (qfirst tmp2))
      (eq (qcaar tmp2) '|exit|)
      (progn
        (setq tmp4 (qcddr tmp2))
        (and (consp tmp4)
          (equal (qfirst tmp4) 1)
          (progn
            (setq tmp5 (qrest tmp4))
            (and (consp tmp5)
              (eq (qrest tmp5) nil)
              (progn (setq pp (qfirst tmp5)) t))))
        (progn (setq z (qrest tmp2)) t))
      (progn (setq z (nreverse z)) t))
    (cons 'seq
      (append z
        (list
          (list '|exit| 1 (|parseIf,ifTran| pp
            (|incExitLevel| a)
            (|incExitLevel| b))))))
    ((and (consp a) (eq (qfirst a) 'if) (consp (qrest a))
      (equal (qsecond a) pred) (consp (qcddr a))
      (consp (qcdddr a))
      (eq (qcdddr a) nil))
      (list 'if pred (third a) b))
    ((and (consp b) (eq (qfirst b) 'if)
      (consp (qrest b)) (equal (qsecond b) pred)
      (consp (qcddr b))
      (consp (qcdddr b))
      (eq (qcdddr b) nil))
      (list 'if pred a (fourth b)))
    ((progn
      (setq tmp1 (|makeSimplePredicateOrNil| pred))
      (and (consp tmp1) (eq (qfirst tmp1) 'seq)
        (progn
          (setq tmp2 (qrest tmp1))
          (and (consp tmp2)
            (progn (setq tmp3 (reverse tmp2)) t))

```

```

      (and (consp tmp3)
        (progn
          (setq tmp4 (qfirst tmp3))
          (and (consp tmp4) (eq (qfirst tmp4) '|exit|)
            (progn
              (setq tmp5 (qrest tmp4))
              (and (consp tmp5) (equal (qfirst tmp5) 1)
                (progn
                  (setq tmp6 (qrest tmp5))
                  (and (consp tmp6) (eq (qrest tmp6) nil)
                    (progn (setq val (qfirst tmp6)) t)))))))
              (progn (setq s (qrest tmp3)) t))))))
      (setq s (nreverse s))
      (|parseTran|
        (cons 'seq
          (append s
            (list (list '|exit| 1 (|incExitLevel| (list 'if val a b)))))))
      (t
        (list 'if pred a b )))))

```

defplist parseImplies

— postvars —

```

(eval-when (eval load)
  (setf (get '|implies| '|parseTran|) '|parseImplies|))

```

defun parseImplies

[parseIf [p122](#)]

— defun parseImplies —

```

(defun |parseImplies| (arg)
  (|parseIf| (list (first arg) (second arg) '|true|)))

```

defplist parseIn

— postvars —

```
(eval-when (eval load)
  (setf (get 'in '|parseTran|) '|parseIn|))
```

—————

defun parseIn

[parseTran p101]

[postError p374]

— defun parseIn —

```
(defun |parseIn| (arg)
  (let (i n)
    (setq i (|parseTran| (first arg)))
    (setq n (|parseTran| (second arg)))
    (cond
      ((and (consp n) (eq (qfirst n) 'segment)
            (consp (qrest n)) (eq (qcddr n) nil))
       (list 'step i (second n) 1))
      ((and (consp n) (eq (qfirst n) '|reverse|)
            (consp (qrest n)) (eq (qcddr n) nil)
            (consp (qsecond n)) (eq (qcaadr n) 'segment)
            (consp (qcdadr n))
            (eq (qcddadr n) nil))
       (postError (list " You cannot reverse an infinite sequence." )))
      ((and (consp n) (eq (qfirst n) 'segment)
            (consp (qrest n)) (consp (qcddr n))
            (eq (qcdddr n) nil))
       (if (third n)
           (list 'step i (second n) 1 (third n))
           (list 'step i (second n) 1)))
      ((and (consp n) (eq (qfirst n) '|reverse|)
            (consp (qrest n)) (eq (qcddr n) nil)
            (consp (qsecond n)) (eq (qcaadr n) 'segment)
            (consp (qcdadr n))
            (consp (qcddadr n))
            (eq (qrest (qcddadr n)) nil))
       (if (third (second n))
           (list 'step i (third (second n)) -1 (second (second n)))
           (postError (list " You cannot reverse an infinite sequence." ))))
      ((and (consp n) (eq (qfirst n) '|tails|)
```

```

      (consp (qrest n)) (eq (qcddr n) nil))
    (list 'on i (second n)))
  (t
    (list 'in i n))))

```

defplist parseInBy

— postvars —

```

(eval-when (eval load)
  (setf (get 'inby '|parseTran|) '|parseInBy|))

```

defun parseInBy

```

[postError p374]
[parseTran p101]
[bright p??]
[parseIn p125]

```

— defun parseInBy —

```

(defun |parseInBy| (arg)
  (let (i n inc u)
    (setq i (first arg))
    (setq n (second arg))
    (setq inc (third arg))
    (setq u (|parseIn| (list i n)))
    (cond
      ((null (and (consp u) (eq (qfirst u) 'step)
                  (consp (qrest u))
                  (consp (qcddr u))
                  (consp (qcdddr u)))))
       (postError
        (cons '| You cannot use|
              (append (|bright| "by")
                      (list "except for an explicitly indexed sequence."))))))
    (t
     (setq inc (|parseTran| inc))
     (cons 'step
           (cons (second u)

```

```
(cons (third u)
      (cons (|parseTran| inc) (cddddr u)))))))))
```

defplist parseIs

— postvars —

```
(eval-when (eval load)
  (setf (get '|is| '|parseTran|) '|parseIs|))
```

defun parseIs

```
[parseTran p101]
[transIs p110]
```

— defun parseIs —

```
(defun |parseIs| (arg)
  (list '|is| (|parseTran| (first arg)) (|transIs| (|parseTran| (second arg)))))
```

defplist parseIsnt

— postvars —

```
(eval-when (eval load)
  (setf (get '|isnt| '|parseTran|) '|parseIsnt|))
```

defun parseIsnt

```
[parseTran p101]
[transIs p110]
```

— defun parseIsnt —

```
(defun |parseIsnt| (arg)
  (list '|isnt|
        (|parseTran| (first arg))
        (|transIs| (|parseTran| (second arg)))))
```

—————

defplist parseJoin

— postvars —

```
(eval-when (eval load)
  (setf (get '|Join| '|parseTran|) '|parseJoin|))
```

—————

defun parseJoin

[parseTranList [p103](#)]

— defun parseJoin —

```
(defun |parseJoin| (thejoin)
  (labels (
    (fn (arg)
      (cond
        ((null arg)
         nil)
        ((and (consp arg) (consp (qfirst arg)) (eq (qcaar arg) '|Join|))
         (append (cdar arg) (fn (rest arg))))
        (t
         (cons (first arg) (fn (rest arg))))))
    )
  (cons '|Join| (fn (|parseTranList| thejoin)))))
```

—————

defplist parseLeave

— postvars —

```
(eval-when (eval load)
  (setf (get '|leave| '|parseTran|) '|parseLeave|))
```

defun parseLeave

[parseTran p101]

— defun parseLeave —

```
(defun |parseLeave| (arg)
  (let (a b)
    (setq a (|parseTran| (car arg)))
    (setq b (|parseTran| (cdr arg)))
    (cond
     (b
      (cond
       ((null (integerp a))
        (moan "first arg " a " for 'leave' must be integer")
        (list '|leave| 1 a))
       (t (cons '|leave| (cons a b))))))
    (t (list '|leave| 1 a)))))
```

defplis parseLessEqual

— postvars —

```
(eval-when (eval load)
  (setf (get '|<=| '|parseTran|) '|parseLessEqual|))
```

defun parseLessEqual

[parseTran p101]

[\$op p??]

— defun parseLessEqual —

```
(defun |parseLessEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '>' '<=' |$op| :test #'equal) arg))))
```

defplist parseLET

— postvars —

```
(eval-when (eval load)
  (setf (get 'let '|parseTran|) '|parseLET|))
```

defun parseLET

```
[parseTran p101]
[parseTranCheckForRecord p543]
[opOf p??]
[transIs p110]
```

— defun parseLET —

```
(defun |parseLET| (arg)
  (let (p)
    (setq p
      (list 'let (|parseTran| (first arg))
        (|parseTranCheckForRecord| (second arg) (|opOf| (first arg)))))
    (if (eq (|opOf| (first arg)) '|cons|)
      (list 'let (|transIs| (second p)) (third p))
      p)))
```

defplist parseLETD

— postvars —

```
(eval-when (eval load)
  (setf (get 'letd '|parseTran|) '|parseLETD|))
```

defun parseLETD

[parseTran p101]
 [parseType p106]

— **defun parseLETD** —

```
(defun |parseLETD| (arg)
  (list 'letd
        (|parseTran| (first arg))
        (|parseTran| (|parseType| (second arg)))))
```

defplist parseMDEF

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'mdef '|parseTran|) '|parseMDEF|))
```

defun parseMDEF

[parseTran p101]
 [parseTranList p103]
 [parseTranCheckForRecord p543]
 [opOf p??]
 [\$lhs p??]

— **defun parseMDEF** —

```
(defun |parseMDEF| (arg)
  (let (|$lhs|)
    (declare (special |$lhs|))
    (setq |$lhs| (first arg))
    (list 'mdef
          (|parseTran| |$lhs|)
          (|parseTranList| (second arg)))
```

```
(|parseTranList| (third arg))
(|parseTranCheckForRecord| (fourth arg) (|opOf| |$lhs|))))))
```

defplist parseNot

— postvars —

```
(eval-when (eval load)
  (setf (get 'not| 'parseTran|) 'parseNot|))
```

defplist parseNot

— postvars —

```
(eval-when (eval load)
  (setf (get '|^| 'parseTran|) 'parseNot|))
```

defun parseNot

```
[parseTran p101]
[$InteractiveMode p??]
```

— defun parseNot —

```
(defun |parseNot| (arg)
  (declare (special |$InteractiveMode|))
  (if |$InteractiveMode|
    (list 'not| (|parseTran| (car arg)))
    (|parseTran| (cons 'if (cons (car arg) '(|false| |true|))))))
```

defplist parseNotEqual

— postvars —

```
(eval-when (eval load)
  (setf (get '|^=' '|parseTran|) '|parseNotEqual|))
```

—————

defun parseNotEqual

```
[parseTran p101]
[$op p??]
```

— defun parseNotEqual —

```
(defun |parseNotEqual| (arg)
  (declare (special |$op|))
  (|parseTran| (list '|not| (cons (subst '= '^= |$op| :test #'equal) arg))))
```

—————

defplist parseOr

— postvars —

```
(eval-when (eval load)
  (setf (get '|or=' '|parseTran|) '|parseOr|))
```

—————

defun parseOr

```
[parseTran p101]
[parseTranList p103]
[parseIf p122]
[parseOr p133]
```

— defun parseOr —

```
(defun |parseOr| (arg)
  (let (x)
    (setq x (|parseTran| (car arg)))
    (cond
      (|$InteractiveMode| (cons '|or| (|parseTranList| arg)))
      ((null arg) '|false|)
      ((null (cdr arg)) (car arg))
      ((and (consp x) (eq (qfirst x) '|not|)
            (consp (qrest x)) (eq (qcddr x) nil))
       (|parseIf| (list (second x) (|parseOr| (cdr arg)) '|true|)))
      (t
       (|parseIf| (list x '|true| (|parseOr| (cdr arg))))))))
```

defplist parsePretend

— postvars —

```
(eval-when (eval load)
  (setf (get '|pretend| '|parseTran|) '|parsePretend|))
```

defun parsePretend

[[parseTran p101](#)]
 [[parseType p106](#)]

— defun parsePretend —

```
(defun |parsePretend| (arg)
  (if |$InteractiveMode|
    (list '|pretend|
          (|parseTran| (first arg))
          (|parseTran| (|parseType| (second arg))))
    (list '|pretend|
          (|parseTran| (first arg))
          (|parseTran| (second arg)))))
```

defplist parseReturn

— postvars —

```
(eval-when (eval load)
  (setf (get '|return| '|parseTran|) '|parseReturn|))
```

—————

defun parseReturn

```
[parseTran p101]
[moan p??]
```

— defun parseReturn —

```
(defun |parseReturn| (arg)
  (let (a b)
    (setq a (|parseTran| (car arg)))
    (setq b (|parseTran| (cdr arg)))
    (cond
     (b
      (unless (eql a 1) (moan "multiple-level 'return' not allowed"))
      (cons '|return| (cons 1 b)))
     (t (list '|return| 1 a)))))
```

—————

defplist parseSegment

— postvars —

```
(eval-when (eval load)
  (setf (get '|segment| '|parseTran|) '|parseSegment|))
```

—————

defun parseSegment

```
[parseTran p101]
```

— defun parseSegment —

```
(defun |parseSegment| (arg)
  (if (and (consp arg) (consp (qrest arg)) (eq (qcddr arg) nil))
      (if (second arg)
          (list 'segment (|parseTran| (first arg)) (|parseTran| (second arg)))
          (list 'segment (|parseTran| (first arg))))
      (cons 'segment arg)))
```

defplist parseSeq

— postvars —

```
(eval-when (eval load)
  (setf (get 'seq '|parseTran|) '|parseSeq|))
```

defun parseSeq

```
[postError p374]
[transSeq p??]
[mapInto p??]
[last p??]
```

— defun parseSeq —

```
(defun |parseSeq| (arg)
  (let (tmp1)
    (when (consp arg) (setq tmp1 (reverse arg)))
    (if (null (and (consp arg) (consp tmp1)
                  (consp (qfirst tmp1)) (eq (qcaar tmp1) '|exit|)))
        (postError (list " Invalid ending to block: " (|last| arg)))
        (|transSeq| (|mapInto| arg '|parseTran|)))))
```

defplist parseVCONS

— postvars —

```
(eval-when (eval load)
  (setf (get 'vcons '|parseTran|) '|parseVCONS|))
```

defun parseVCONS

```
[parseTranList p103]
```

— defun parseVCONS —

```
(defun |parseVCONS| (arg)
  (cons 'vector (|parseTranList| arg)))
```

defplst parseWhere

— postvars —

```
(eval-when (eval load)
  (setf (get '|where| '|parseTran|) '|parseWhere|))
```

defun parseWhere

```
[mapInto p??]
```

— defun parseWhere —

```
(defun |parseWhere| (arg)
  (cons '|where| (|mapInto| arg '|parseTran|)))
```

Chapter 6

Compile Transformers

With some specific exceptions most compile transformers are invoked through the property list item “**special**”. When a specific keyword is encountered in a list form the **compExpression** function looks up the keyword on the property list and funcalls the handler function, passing the form, the mode, and the environment.

If a handler for the keyword is not found then the **compForm** function is called to attempt to compile the form.

defun compExpression

```
[getl p??]  
[compForm p600]  
[$insideExpressionIfTrue p??]
```

— defun compExpression —

```
(defun |compExpression| (form mode env)  
  (let (|$insideExpressionIfTrue| fn)  
    (declare (special |$insideExpressionIfTrue|))  
    (setq |$insideExpressionIfTrue| t)  
    (if (and (atom (car form)) (setq fn (getl (car form) 'special)))  
        (funcall fn form mode env)  
        (|compForm| form mode env))))
```

The functions in this section are called through the symbol-plist of the symbol being parsed. In general, each of these functions takes 3 arguments

1. the **form** which is specific to the function

2. the **mode** a ---Join--- , which is a set of categories and domains
3. the **env** which is a list of functions and their modemaps

and the functions return modified versions of the three arguments suitable for further pro-

	DEF	(p144)	compDefine
	add	(p275)	compAdd
	@	(p361)	compAtSign
	CAPSULE	(p278)	compCapsule
	case	(p287)	compCase
	Mapping	(p289)	compCat
	Record	(p289)	compCat
	Union	(p289)	compCat
	CATEGORY	(p290)	compCategory
	::	(p362)	compCoerce
	:	(p294)	compColon
	CONS	(p298)	compCons
	construct	(p299)	compConstruct
	ListCategory	(p301)	compConstructorCategory
	RecordCategory	(p301)	compConstructorCategory
	UnionCategory	(p301)	compConstructorCategory
	VectorCategory	(p301)	compConstructorCategory
	elt	(p310)	compElt
	exit	(p312)	compExit
	has	(p313)	compHas(pred mode \$e)
cessing.	IF	(p315)	compIf
	import	(p322)	compImport
	is	(p322)	compIs
	Join	(p323)	compJoin
	+->	(p325)	compLambda
	leave	(p327)	compLeave
	MDEF	(p327)	compMacro
	pretend	(p328)	compPretend
	QUOTE	(p330)	compQuote
	REDUCE	(p330)	compReduce
	COLLECT	(p333)	compRepeatOrCollect
	REPEAT	(p333)	compRepeatOrCollect
	return	(p335)	compReturn
	SEQ	(p336)	compSeq
	LET	(p339)	compSetq
	SETQ	(p339)	compSetq
	String	(p349)	compString
	SubDomain	(p350)	compSubDomain
	SubsetCategory	(p352)	compSubsetCategory
		(p353)	compSuchthat
	VECTOR	(p353)	compVector
	where	(p354)	compWhere

6.1 Handline Category DEF forms

This is the graph of the functions used for `compDefine`. The syntax is a graphviz dot file. To generate this graph as a JPEG file, type:

```
tangle v9compDefine.dot bookvol9.pamphlet >v9compdefine.dot
dot -Tjpg v9compdefine.dot >v9compdefine.jpg
```

— v9compDefine.dot —

```
digraph pic {
    fontsize=10;
    bgcolor="#ECEA81";
    node [shape=box, color=white, style=filled];

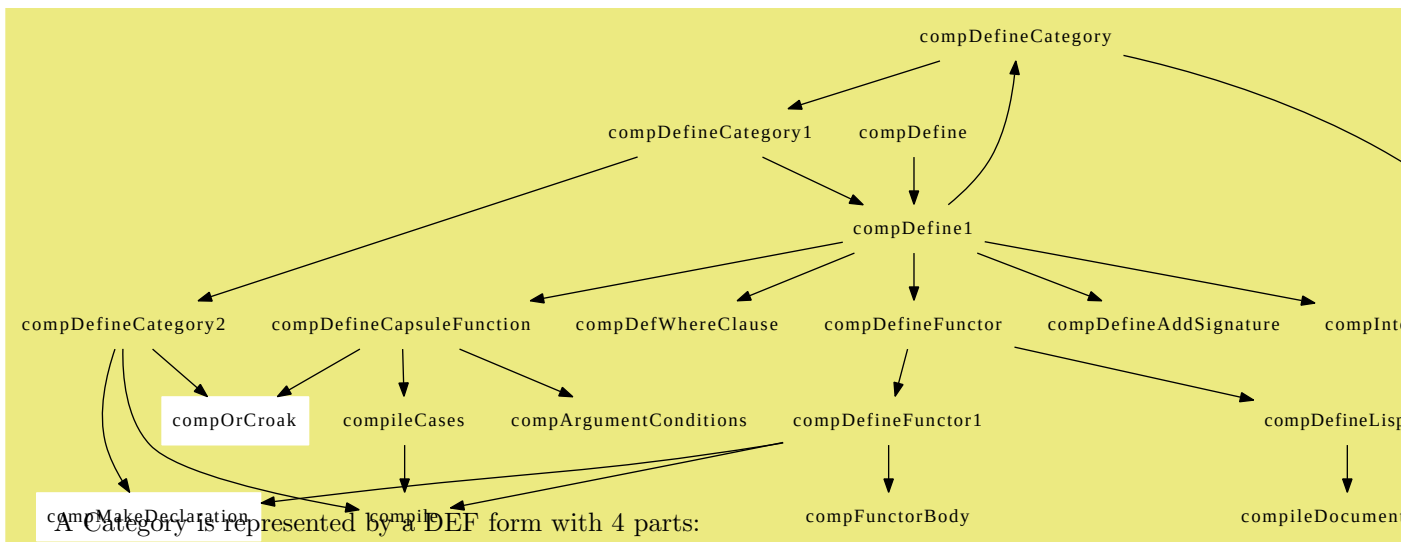
    "compArgumentConditions"      [color="#ECEA81"]
    "compDefWhereClause"          [color="#ECEA81"]
    "compDefine"                  [color="#ECEA81"]
    "compDefine1"                 [color="#ECEA81"]
    "compDefineAddSignature"       [color="#ECEA81"]
    "compDefineCapsuleFunction"    [color="#ECEA81"]
    "compDefineCategory"          [color="#ECEA81"]
    "compDefineCategory1"         [color="#ECEA81"]
    "compDefineCategory2"         [color="#ECEA81"]
    "compDefineFunctor"           [color="#ECEA81"]
    "compDefineFunctor1"          [color="#ECEA81"]
    "compDefineLisplib"           [color="#ECEA81"]
    "compInternalFunction"        [color="#ECEA81"]
    "compMakeDeclaration"         [color="#FFFFFF"]
    "compFunctorBody"             [color="#ECEA81"]
    "compOrCroak"                 [color="#FFFFFF"]
    "compile"                     [color="#ECEA81"]
    "compileCases"                [color="#ECEA81"]
    "compileDocumentation"         [color="#ECEA81"]

    "compDefine" -> "compDefine1"
    "compDefine1" -> "compDefineCapsuleFunction"
    "compDefine1" -> "compDefWhereClause"
    "compDefine1" -> "compDefineAddSignature"
    "compDefine1" -> "compDefineCategory"
    "compDefine1" -> "compDefineFunctor"
    "compDefine1" -> "compInternalFunction"
    "compDefineCapsuleFunction" -> "compArgumentConditions"
    "compDefineCapsuleFunction" -> "compOrCroak"
    "compDefineCapsuleFunction" -> "compileCases"
    "compDefineCategory" -> "compDefineCategory1"
    "compDefineCategory" -> "compDefineLisplib"
    "compDefineCategory1" -> "compDefine1"
```

```

"compDefineCategory1" -> "compDefineCategory2"
"compDefineCategory2" -> "compMakeDeclaration"
"compDefineCategory2" -> "compOrCroak"
"compDefineCategory2" -> "compile"
"compDefineFunctor" -> "compDefineFunctor1"
"compDefineFunctor" -> "compDefineLisplib"
"compDefineFunctor1" -> "compMakeDeclaration"
"compDefineFunctor1" -> "compFunctorBody"
"compDefineFunctor1" -> "compile"
"compDefineLisplib" -> "compileDocumentation"
"compileCases" -> "compile"
}

```



- a name
- a distnature
- an SC
- a body

For example, the BasicType category is written as

```

BasicType(): Category == with
  "=: (%,% ) -> Boolean  ++ x=y tests if x and y are equal.
  "~=: (%,% ) -> Boolean  ++ x~y tests if x and y are not equal.

```

```
add
  _~_(x:%,y:%) : Boolean == not(x=y)
```

Which compiles to the DEF form:

```
(DEF
  (|BasicType|)
  ((|Category|))
  (NIL)
  (|add|
    (CATEGORY |domain|
      (SIGNATURE = ((|Boolean|) $ $))
      (SIGNATURE ~ = ((|Boolean|) $ $)))
    (CAPSULE
      (DEF
        (~ = |x| |y|)
        ((|Boolean|) $ $)
        (NIL NIL NIL)
        (IF (= |x| |y|) |false| |true|))))))
```

defplist compDefine plist

We set up the `compDefine` function to handle the DEF keyword by setting the `special` keyword on the DEF symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'def 'special) '|compDefine|))
```

—————

defun compDefine

The `compDefine` function expects three arguments:

1. the **form** which is an def specifying the domain to define.
2. the **mode** a —Join—, which is a set of categories and domains
3. the **env** which is a list of functions and their modemaps

```
[compDefine1 p145]
[$tripleCache p??]
[$tripleHits p??]
[$macroIfTrue p??]
[$packagesUsed p??]
```

— **defun compDefine** —

```
(defun |compDefine| (form mode env)
  (let (|$tripleCache| |$tripleHits| |$macroIfTrue| |$packagesUsed|)
    (declare (special |$tripleCache| |$tripleHits| |$macroIfTrue|
                      |$packagesUsed|))
    (setq |$tripleCache| nil)
    (setq |$tripleHits| 0)
    (setq |$macroIfTrue| nil)
    (setq |$packagesUsed| nil)
    (|compDefine1| form mode env)))
```

defun compDefine1

```
[macroExpand p178]
[isMacro p286]
[getSignatureFromMode p303]
[compDefine1 p145]
[compInternalFunction p159]
[compDefineAddSignature p147]
[compDefWhereClause p159]
[compDefineCategory p162]
[isDomainForm p348]
[getTargetFromRhs p177]
[giveFormalParametersValues p178]
[addEmptyCapsuleIfNecessary p177]
[compDefineFunctor p148]
[stackAndThrow p??]
[strconc p??]
[getAbbreviation p302]
[length p??]
[compDefineCapsuleFunction p155]
[$insideExpressionIfTrue p??]
[$formalArgList p??]
[$form p??]
[$op p??]
[$prefix p??]
[$insideFunctorIfTrue p??]
[$Category p??]
[$insideCategoryIfTrue p??]
[$insideCapsuleFunctionIfTrue p??]
[$ConstructorNames p??]
```

[\[NoValueMode p176\]](#)
[\[EmptyMode p176\]](#)
[\[insideWhereIfTrue p??\]](#)
[\[insideExpressionIfTrue p??\]](#)

— defun compDefine1 —

```

(defun |compDefine1| (form mode env)
  (let (|$insideExpressionIfTrue| lhs specialCases sig signature rhs newPrefix
        (tmp1 t))
    (declare (special |$insideExpressionIfTrue| |$formalArgList| |$form|
                      |$op| |$prefix| |$insideFunctorIfTrue| |$Category|
                      |$insideCategoryIfTrue| |$insideCapsuleFunctionIfTrue|
                      |$ConstructorNames| |$NoValueMode| |$EmptyMode|
                      |$insideWhereIfTrue| |$insideExpressionIfTrue|))
    (setq |$insideExpressionIfTrue| nil)
    (setq form (|macroExpand| form env))
    (setq lhs (second form))
    (setq signature (third form))
    (setq specialCases (fourth form))
    (setq rhs (fifth form))
    (cond
     ((and |$insideWhereIfTrue|
          (|isMacro| form env)
          (or (equal mode |$EmptyMode|) (equal mode |$NoValueMode|)))
      (list lhs mode (|put| (car lhs) '|macro| rhs env)))
     ((and (null (car signature)) (consp rhs)
          (null (member (qfirst rhs) |$ConstructorNames|))
          (setq sig (|getSignatureFromMode| lhs env)))
      (|compDefine1|
       (list 'def lhs (cons (car sig) (cdr signature)) specialCases rhs)
       mode env))
     (|$insideCapsuleFunctionIfTrue| (|compInternalFunction| form mode env))
     (t
      (when (equal (car signature) |$Category|) (setq |$insideCategoryIfTrue| t))
      (setq env (|compDefineAddSignature| lhs signature env))
      (cond
       ((null (dolist (x (rest signature) tmp1) (setq tmp1 (and tmp1 (null x)))))
        (|compDefWhereClause| form mode env))
       ((equal (car signature) |$Category|)
        (|compDefineCategory| form mode env nil |$formalArgList|))
       ((and (|isDomainForm| rhs env) (null |$insideFunctorIfTrue|))
        (when (null (car signature))
         (setq signature
          (cons (|getTargetFromRhs| lhs rhs
                  (|giveFormalParametersValues| (cdr lhs) env))
                (cdr signature))))
        (setq rhs (|addEmptyCapsuleIfNecessary| (car signature) rhs))
        (|compDefineFunctor|

```

```

      (list 'def lhs signature specialCases rhs)
      mode env NIL |$formalArgList|))
((null |$form|)
 (|stackAndThrow| (list "bad == form " form)))
(t
 (setq newPrefix
  (if |$prefix|
    (intern (strconc (|encodeItem| |$prefix|) "," (|encodeItem| |$op|)))
    (|getAbbreviation| |$op| (|#| (cdr |$form|)))))
 (|compDefineCapsuleFunction|
  form mode env newPrefix |$formalArgList|))))))

```

defun compDefineAddSignature

[\[hasFullSignature p176\]](#)
[\[assoc p??\]](#)
[\[lassoc p??\]](#)
[\[getProplist p??\]](#)
[\[comp p587\]](#)
[\[\\$EmptyMode p176\]](#)

— defun compDefineAddSignature —

```

(defun |compDefineAddSignature| (form signature env)
  (let (sig declForm)
    (declare (special |$EmptyMode|))
    (if
      (and (setq sig (|hasFullSignature| (rest form) signature env))
        (null (|assoc| (cons '$ sig)
          (|lassoc| '|modemap| (|getProplist| (car form) env))))))
      (progn
        (setq declForm
          (list '|:|
            (cons (car form)
              (loop for x in (rest form)
                for m in (rest sig)
                collect (list '|:| x m)))
            (car signature)))
          (third (|comp| declForm |$EmptyMode| env)))
        env)))

```

defun compDefineFunctor

```
[compDefineLisplib p167]
[compDefineFunctor1 p148]
[$domainShell p??]
[$profileCompiler p??]
[$lisplib p??]
[$profileAlist p??]
```

— defun compDefineFunctor —

```
(defun |compDefineFunctor| (df mode env prefix fal)
  (let (|$domainShell| |$profileCompiler| |$profileAlist|)
    (declare (special |$domainShell| |$profileCompiler| $lisplib |$profileAlist|))
    (setq |$domainShell| nil)
    (setq |$profileCompiler| t)
    (setq |$profileAlist| nil)
    (if $lisplib
      (|compDefineLisplib| df mode env prefix fal '|compDefineFunctor1|)
      (|compDefineFunctor1| df mode env prefix fal))))
```

defun compDefineFunctor1

```
[isCategoryPackageName p214]
[getArgumentModeOrMoan p191]
[getModemap p258]
[giveFormalParametersValues p178]
[compMakeCategoryObject p213]
[sayBrightly p??]
[pp p??]
[strconc p??]
[pname p??]
[disallowNilAttribute p219]
[remdup p??]
[NRTgenInitialAttributeAlist p??]
[NRTgetLocalIndex p215]
[compMakeDeclaration p622]
[augModemapsFromCategoryRep p271]
[augModemapsFromCategory p265]
[sublis p??]
[maxindex p??]
[makeFunctorArgumentParameters p221]
[compFunctorBody p172]
```


[reportOnFunctorCompilation p219]
 [compile p173]
 [augmentLisplibModemapsFromFunctor p216]
 [reportOnFunctorCompilation p219]
 [getParentsFor p??]
 [computeAncestorsOf p??]
 [constructor? p??]
 [NRTmakeSlot1Info p??]
 [isCategoryPackageName p214]
 [lisplibWrite p214]
 [mkq p??]
 [getdatabase p??]
 [NRTgetLookupFunction p214]
 [simpBool p??]
 [removeZeroOne p??]
 [evalAndRwriteLispForm p204]
 [\$lisplib p??]
 [\$top-level p??]
 [\$bootStrapMode p??]
 [\$CategoryFrame p??]
 [\$CheckVectorList p??]
 [\$FormalMapVariableList p270]
 [\$LocalDomainAlist p??]
 [\$NRTaddForm p??]
 [\$NRTaddList p??]
 [\$NRTattributeAlist p??]
 [\$NRTbase p??]
 [\$NRTdeltaLength p??]
 [\$NRTdeltaListComp p??]
 [\$NRTdeltaList p??]
 [\$NRTdomainFormList p??]
 [\$NRTloadTimeAlist p??]
 [\$NRTslot1Info p??]
 [\$NRTslot1PredicateList p??]
 [\$Representation p??]
 [\$addForm p??]
 [\$attributesName p??]
 [\$byteAddress p??]
 [\$byteVec p??]
 [\$compileOnlyCertainItems p??]
 [\$condAlist p??]
 [\$domainShell p??]
 [\$form p??]
 [\$functionLocations p??]
 [\$functionStats p??]
 [\$functorForm p??]

```

[$functorLocalParameters p??]
[$functorStats p??]
[$functorSpecialCases p??]
[$functorTarget p??]
[$functorsUsed p??]
[$genFVar p??]
[$genSDVar p??]
[$getDomainCode p??]
[$goGetList p??]
[$insideCategoryPackageIfTrue p??]
[$insideFunctorIfTrue p??]
[$isOpPackageName p??]
[$libFile p??]
[$lisplibAbbreviation p??]
[$lisplibAncestors p??]
[$lisplibCategoriesExtended p??]
[$lisplibCategory p??]
[$lisplibForm p??]
[$lisplibKind p??]
[$lisplibMissingFunctions p??]
[$lisplibModemap p??]
[$lisplibOperationAlist p??]
[$lisplibParents p??]
[$lisplibSlot1 p??]
[$lookupFunction p??]
[$myFunctorBody p??]
[$mutableDomain p??]
[$mutableDomains p??]
[$op p??]
[$pairlis p??]
[$QuickCode p??]
[$setelt p??]
[$signature p??]
[$template p??]
[$uncondAlist p??]
[$viewNames p??]
[$lisplibFunctionLocations p??]

```

— **defun compDefineFunctor1** —

```

(defun |compDefineFunctor1| (df mode |$e| |$prefix| |$formalArgList|)
  (declare (special |$e| |$prefix| |$formalArgList|))
  (labels (
    (FindRep (cb)
      (loop while cb do
        (when (atom cb) (return nil))

```

```

    (when (and (consp cb) (consp (qfirst cb)) (eq (qcaar cb) 'let)
              (consp (qcdr cb)) (eq (qcadar cb) 'Rep|)
              (consp (qcddar cb)))
      (return (caddar cb)))
    (pop cb))))
(let (|$addForm| |$viewNames| |$functionStats| |$functorStats|
      |$form| |$op| |$signature| |$functorTarget|
      |$Representation| |$LocalDomainAlist| |$functorForm|
      |$functorLocalParameters| |$CheckVectorList|
      |$getDomainCode| |$insideFunctorIfTrue| |$functorsUsed|
      |$setelt| $TOP_LEVEL |$genFVar| |$genSDVar|
      |$mutableDomain| |$attributesName| |$goGetList|
      |$condAlist| |$uncondAlist| |$NRTslot1PredicateList|
      |$NRTattributeAlist| |$NRTslot1Info| |$NRTbase|
      |$NRTaddForm| |$NRTdeltaList| |$NRTdeltaListComp|
      |$NRTaddList| |$NRTdeltaLength| |$NRTloadTimeAlist|
      |$NRTdomainFormList| |$template| |$functionLocations|
      |$isOpPackageName| |$lookupFunction| |$byteAddress|
      |$byteVec| form signature body originale argl signaturep target ds
      attributeList parSignature parForm
      argPars opp rettype tt bodyp lamOrSlam fun
      operationAlist modemap libFn tmp1)
(declare (special $lisplib $top_level |$bootStrapMode| |$CategoryFrame|
  |$CheckVectorList| |$FormalMapVariableList| | | | |
  |$LocalDomainAlist| |$NRTaddForm| |$NRTaddList|
  |$NRTattributeAlist| |$NRTbase| |$NRTdeltaLength|
  |$NRTdeltaListComp| |$NRTdeltaList| |$NRTdomainFormList|
  |$NRTloadTimeAlist| |$NRTslot1Info| |$NRTslot1PredicateList|
  |$Representation| |$addForm| |$attributesName|
  |$byteAddress| |$byteVec| |$compileOnlyCertainItems|
  |$condAlist| |$domainShell| |$form| |$functionLocations|
  |$functionStats| |$functorForm| |$functorLocalParameters|
  |$functorStats| |$functorSpecialCases| |$functorTarget|
  |$functorsUsed| |$genFVar| |$genSDVar| |$getDomainCode|
  |$goGetList| |$insideCategoryPackageIfTrue|
  |$insideFunctorIfTrue| |$isOpPackageName| |$libFile|
  |$lisplibAbbreviation| |$lisplibAncestors|
  |$lisplibCategoriesExtended| |$lisplibCategory|
  |$lisplibForm| |$lisplibKind| |$lisplibMissingFunctions|
  |$lisplibModemap| |$lisplibOperationAlist| |$lisplibParents|
  |$lisplibSlot1| |$lookupFunction| |$myFunctorBody|
  |$mutableDomain| |$mutableDomains| |$op| |$pairlis|
  |$QuickCode| |$setelt| |$signature| |$template|
  |$uncondAlist| |$viewNames| |$lisplibFunctionLocations|))
(setq form (second df))
(setq signature (third df))
(setq |$functorSpecialCases| (fourth df))
(setq body (fifth df))
(setq |$addForm| nil)
(setq |$viewNames| nil)

```

```

(setq |$functionStats| (list 0 0))
(setq |$functorStats| (list 0 0))
(setq |$form| nil)
(setq |$op| nil)
(setq |$signature| nil)
(setq |$functorTarget| nil)
(setq |$Representation| nil)
(setq |$LocalDomainAlist| nil)
(setq |$functorForm| nil)
(setq |$functorLocalParameters| nil)
(setq |$myFunctorBody| body)
(setq |$CheckVectorList| nil)
(setq |$getDomainCode| nil)
(setq |$insideFunctorIfTrue| t)
(setq |$functorsUsed| nil)
(setq |$setelt| (if |$QuickCode| 'qsetrefv 'setelt))
(setq $top_level nil)
(setq |$genFVar| 0)
(setq |$genSDVar| 0)
(setq originale |$e|)
(setq |$op| (first form))
(setq argl (rest form))
(setq |$formalArgList| (append argl |$formalArgList|))
(setq |$pairlis|
  (loop for a in argl for v in |$FormalMapVariableList|
    collect (cons a v)))
(setq |$mutableDomain|
  (OR (|isCategoryPackageName| |$op|)
    (COND
      ((boundp '|$mutableDomains|)
       (member |$op| |$mutableDomains|))
      ('T NIL))))

(setq signaturep
  (cons (car signature)
    (loop for a in argl collect (|getArgumentModeOrMoan| a form |$e|))))
(setq |$form| (cons |$op| argl))
(setq |$functorForm| |$form|)
(unless (car signaturep)
  (setq signaturep (cdar (|getModemap| |$form| |$e|))))
(setq target (first signaturep))
(setq |$functorTarget| target)
(setq |$e| (|giveFormalParametersValues| argl |$e|))
(setq tmp1 (|compMakeCategoryObject| target |$e|))
(if tmp1
  (progn
    (setq ds (first tmp1))
    (setq |$e| (third tmp1))
    (setq |$domainShell| (copy-seq ds))
    (setq |$attributesName| (intern (strconc (pname |$op|) ";attributes"))))
  (setq attributeList (|disallowNilAttribute| (elt ds 2)))

```

```

(setq |$goGetList| nil)
(setq |$condAlist| nil)
(setq |$uncondAlist| nil)
(setq |$NRTslot1PredicateList|
  (remdup (loop for x in attributeList collect (second x))))
(setq |$NRTattributeAlist| (|NRTgenInitialAttributeAlist| attributeList))
(setq |$NRTslot1Info| nil)
(setq |$NRTbase| 6)
(setq |$NRTaddForm| nil)
(setq |$NRTdeltaList| nil)
(setq |$NRTdeltaListComp| nil)
(setq |$NRTaddList| nil)
(setq |$NRTdeltaLength| 0)
(setq |$NRTloadTimeAlist| nil)
(setq |$NRTdomainFormList| nil)
(setq |$template| nil)
(setq |$functionLocations| nil)
(loop for x in argl do (|NRTgetLocalIndex| x))
(setq |$e|
  (third (|compMakeDeclaration| (list '||:| '$ target) mode |$e|)))
(unless |$insideCategoryPackageIfTrue|
  (if
    (and (consp body) (eq (qfirst body) '|add|)
      (consp (qrest body))
      (consp (qsecond body))
      (consp (qcddr body))
      (eq (qcdddr body) nil)
      (consp (qthird body))
      (eq (qcaaddr body) '|capsule|)
      (member (qcaadr body) '(|List| |Vector|))
      (equal (FindRep (qcdaddr body)) (second body)))
    (setq |$e| (|augModemapsFromCategoryRep| '$
      (second body) (cdaddr body) target |$e|))
    (setq |$e| (|augModemapsFromCategory| '$ '$ target |$e|))))
(setq |$signature| signaturep)
(setq operationAlist (sublis |$pairlis| (elt |$domainShell| 1)))
(setq parSignature (sublis |$pairlis| signaturep))
(setq parForm (sublis |$pairlis| form))
(setq argPars (|makeFunctorArgumentParameters| argl
  (cdr signaturep) (car signaturep)))
(setq |$functorLocalParameters| argl)
(setq opp |$op|)
(setq rettype (CAR signaturep))
(setq tt (|compFunctorBody| body rettype |$e| parForm))
(cond
  (|$compileOnlyCertainItems|
    (|reportOnFunctorCompilation|
      (list nil (cons '|Mapping| signaturep) originale)))
  (t
    (setq bodyp (first tt))

```

```

(setq lamOrSlam (if |$mutableDomain| 'lam 'spadslam))
(setq fun
  (|compile| (sublis |$pairlis| (list opp (list lamOrSlam arg1 body))))))
(setq operationAlist (sublis |$pairlis| |$lisplibOperationAlist|))
(cond
  ($lisplib
    (|augmentLisplibModemapsFromFunctor| parForm
      operationAlist parSignature)))
(|reportOnFunctorCompilation|)
(cond
  ($lisplib
    (setq modemap (list (cons parForm parSignature) (list t opp)))
    (setq |$lisplibModemap| modemap)
    (setq |$lisplibCategory| (cadar modemap))
    (setq |$lisplibParents|
      (|getParentsFor| |$op| |$FormalMapVariableList| |$lisplibCategory|))
    (setq |$lisplibAncestors| (|computeAncestorsOf| |$form| NIL))
    (setq |$lisplibAbbreviation| (|constructor?| |$op|)))
    (setq |$insideFunctorIfTrue| NIL)
  (cond
    ($lisplib
      (setq |$lisplibKind|
        (if (and (consp |$functorTarget|)
          (eq (qfirst |$functorTarget|) 'category)
          (consp (qrest |$functorTarget|))
          (not (eq (qsecond |$functorTarget|) 'domain))))
        '|package|
        '|domain|))
      (setq |$lisplibForm| form)
      (cond
        ((null |$bootStrapMode|)
          (setq |$NRTslot1Info| (|NRTmakeSlot1Info|))
          (setq |$isOpPackageName| (|isCategoryPackageName| |$op|))
          (when |$isOpPackageName|
            (|lisplibWrite| "slot1DataBase"
              (list '|updateSlot1DataBase| (mkq |$NRTslot1Info|))
              |$libFile|))
            (setq |$lisplibFunctionLocations|
              (sublis |$pairlis| |$functionLocations|))
            (setq |$lisplibCategoriesExtended|
              (sublis |$pairlis| |$lisplibCategoriesExtended|))
            (setq libFn (getdatabase opp 'abbreviation))
            (setq |$lookupFunction|
              (|NRTgetLookupFunction| |$functorForm|
                (cadar |$lisplibModemap|) |$NRTaddForm|))
            (setq |$byteAddress| 0)
            (setq |$byteVec| NIL)
            (setq |$NRTslot1PredicateList|
              (loop for x in |$NRTslot1PredicateList|
                collect (|simpBool| x)))

```

```

(|rewriteLispForm| 'loadTimeStuff|
  '(setf (get ,(mkq |$op|) '|infovec|) ,(|getInfovecCode|))))
(setq |$lisplibSlot1| |$NRTslot1Info|)
(setq |$lisplibOperationAlist| operationAlist)
(setq |$lisplibMissingFunctions| |$CheckVectorList|))
(|lisplibWrite| "compilerInfo"
  (|removeZeroOne|
    (list 'setq '|$CategoryFrame|
      (list '|put| (list 'quote opp) ''|isFunctor|
        (list 'quote operationAlist)
        (list '|addModemap|
          (list 'quote opp)
          (list 'quote parForm)
          (list 'quote parSignature)
          t
          (list 'quote opp)
          (list '|put| (list 'quote opp) ''|mode|
            (list 'quote (cons '|Mapping| parSignature))
            '|$CategoryFrame|))))
        |$libFile|)
      (unless arg1
        (|evalAndRwriteLispForm| 'niladic
          '(setf (get ',opp 'niladic) t)))
      (list fun (cons '|Mapping| signaturep) originale))))
  (progn
    (|sayBrightly| "    cannot produce category object:")
    (|pp| target)
    nil))))

```

defun compDefineCapsuleFunction

```

[length p??]
[get p??]
[profileRecord p??]
[compArgumentConditions p170]
[addDomain p252]
[giveFormalParametersValues p178]
[getSignature p306]
[put p??]
[getArgumentModeOrMoan p191]
[checkAndDeclare p308]
[hasSigInTargetCategory p308]
[stripOffSubdomainConditions p305]
[stripOffArgumentConditions p306]
[resolve p365]

```

```

[member p??]
[getmode p??]
[formatUnabbreviated p??]
[sayBrightly p??]
[compOrCroak p586]
[NRTassignCapsuleFunctionSlot p??]
[mkq p??]
[replaceExitEtc p337]
[addArgumentConditions p304]
[compileCases p171]
[addStats p??]
[$semanticErrorStack p??]
[$DomainsInScope p??]
[$op p??]
[$formalArgList p??]
[$signatureOfForm p??]
[$functionLocations p??]
[$profileCompiler p??]
[$compileOnlyCertainItems p??]
[$returnMode p??]
[$functorStats p??]
[$functionStats p??]
[$form p??]
[$functionStats p??]
[$argumentConditionList p??]
[$finalEnv p??]
[$initCapsuleErrorCount p??]
[$insideCapsuleFunctionIfTrue p??]
[$CapsuleModemapFrame p??]
[$CapsuleDomainsInScope p??]
[$insideExpressionIfTrue p??]
[$returnMode p??]
[$op p??]
[$formalArgList p??]
[$signatureOfForm p??]
[$functionLocations p??]

```

— **defun compDefineCapsuleFunction** —

```

(defun |compDefineCapsuleFunction| (df m oldE |$prefix| |$formalArgList|)
  ; df is ['DEF,form,signature,specialCases,body]
  (declare (special |$prefix| |$formalArgList|))
  (let (|$form| |$op| |$functionStats| |$argumentConditionList| |$finalEnv|
        |$initCapsuleErrorCount| |$insideCapsuleFunctionIfTrue|
        |$CapsuleModemapFrame| |$CapsuleDomainsInScope|
        |$insideExpressionIfTrue| form signature body tmp1 lineNumber

```



```

    specialCases arg1 identSig argModeList signaturep e rettype tmp2
    localOrExported formattedSig tt catchTag bodyp finalBody fun val)
(declare (special |$form| |$op| |$functionStats| |$functorStats|
    |$argumentConditionList| |$finalEnv| |$returnMode|
    |$initCapsuleErrorCount| |$newCompCompare| |$NoValueMode|
    |$insideCapsuleFunctionIfTrue|
    |$CapsuleModemapFrame| |$CapsuleDomainsInScope| | |
    |$insideExpressionIfTrue| |$compileOnlyCertainItems|
    |$profileCompiler| |$functionLocations| |$finalEnv|
    |$signatureOfForm| |$semanticErrorStack|))
(setq form (second df))
(setq signature (third df))
(setq specialCases (fourth df))
(setq body (fifth df))
(setq tmp1 specialCases)
(setq lineNumber (first tmp1))
(setq specialCases (rest tmp1))
(setq e oldE)
;-1. bind global variables
(setq |$form| nil)
(setq |$op| nil)
(setq |$functionStats| (list 0 0))
(setq |$argumentConditionList| nil)
(setq |$finalEnv| nil)
; used by ReplaceExitEtc to get a common environment
(setq |$initCapsuleErrorCount| (|#| |$semanticErrorStack|))
(setq |$insideCapsuleFunctionIfTrue| t)
(setq |$CapsuleModemapFrame| e)
(setq |$CapsuleDomainsInScope| (|get| '|$DomainsInScope| 'special e))
(setq |$insideExpressionIfTrue| t)
(setq |$returnMode| m)
(setq |$op| (first form))
(setq arg1 (rest form))
(setq |$form| (cons |$op| arg1))
(setq arg1 (|stripOffArgumentConditions| arg1))
(setq |$formalArgList| (append arg1 |$formalArgList|))
; let target and local signatures help determine modes of arguments
(setq argModeList
  (cond
    ((setq identSig (|hasSigInTargetCategory| arg1 form (car signature) e))
      (setq e (|checkAndDeclare| arg1 form identSig e))
      (cdr identSig))
    (t
      (loop for a in arg1
        collect (|getArgumentModeOrMoan| a form e))))))
(setq argModeList (|stripOffSubdomainConditions| argModeList arg1))
(setq signaturep (cons (car signature) argModeList))
(unless identSig
  (setq oldE (|put| |$op| '|mode| (cons '|Mapping| signaturep) oldE)))
; obtain target type if not given

```

```

(cond
  ((null (car signaturep))
    (setq signaturep
      (cond
        (identSig identSig)
        (t (|getSignature| |$op| (cdr signaturep) e))))))
  (when signaturep
    (setq e (|giveFormalParametersValues| argl e))
    (setq |$signatureOfForm| signaturep)
    (setq |$functionLocations|
      (cons (cons (list |$op| |$signatureOfForm|) lineNumber)
        |$functionLocations|))
    (setq e (|addDomain| (car signaturep) e))
    (setq e (|compArgumentConditions| e))
    (when |$profileCompiler|
      (loop for x in argl for y in signaturep
        do (|profileRecord| 'arguments| x y)))
    ; 4. introduce needed domains into extendedEnv
    (loop for domain in signaturep
      do (setq e (|addDomain| domain e)))
    ; 6. compile body in environment with extended environment
    (setq rettype (|resolve| (car signaturep) |$returnMode|))
    (setq localOrExported
      (if (and (null (|member| |$op| |$formalArgList|))
        (eq (first tmp2) '|Mapping|))
        '|local|
        '|exported|))
    ; 6a skip if compiling only certain items but not this one
    ; could be moved closer to the top
    (setq formattedSig (|formatUnabbreviated| (cons '|Mapping| signaturep)))
    (cond
      ((and |$compileOnlyCertainItems|
        (null (|member| |$op| |$compileOnlyCertainItems|)))
        (|sayBrightly|
          (cons " skipping " (cons localOrExported (|bright| |$op|))))
        (list nil (cons '|Mapping| signaturep) oldE))
      (t
        (|sayBrightly|
          (cons " compiling " (cons localOrExported (append (|bright| |$op|)
            (cons ": " formattedSig)))))
        (setq tt (catch '|compCapsuleBody| (|compOrCroak| body rettype e)))
        (|NRTassignCapsuleFunctionSlot| |$op| signaturep)
        ; A THROW to the above CATCH occurs if too many semantic errors occur
        ; see stackSemanticError
        (setq catchTag (mkq (gensym)))
        (setq fun
          (progn
            (setq bodyp
              (|replaceExitEtc| (car tt) catchTag '|TAGGEDreturn| |$returnMode|))
            (setq bodyp (|addArgumentConditions| bodyp |$op|)))

```

```

      (setq finalBody (list 'catch catchTag bodyp))
      (|compileCases|
        (list |$op| (list 'lam (append argl (list '$)) finalBody))
        oldE)))
      (setq |$functorStats| (|addStats| |$functorStats| |$functionStats|))
; 7. give operator a 'value property
      (setq val (list fun signaturep e))
      (list fun (list '|Mapping| signaturep) oldE))))))

```

defun compInternalFunction

```

[identp p??]
[stackAndThrow p??]

```

— defun compInternalFunction —

```

(defun |compInternalFunction| (df m env)
  (let (form signature specialCases body op argl nbody nf ress)
    (setq form (second df))
    (setq signature (third df))
    (setq specialCases (fourth df))
    (setq body (fifth df))
    (setq op (first form))
    (setq argl (rest form))
    (cond
      ((null (identp op))
        (|stackAndThrow| (list '|Bad name for internal function:| op)))
      ((eql (|#| argl) 0)
        (|stackAndThrow|
          (list '|Argumentless internal functions unsupported:| op )))
      (t
        (setq nbody (list '+-> argl body))
        (setq nf (list 'let (list '|:| op (cons '|Mapping| signature)) nbody))
        (setq ress (|comp| nf m env) ress))))))

```

defun compDefWhereClause

```

[getmode p??]
[userError p??]
[concat p??]
[lassoc p??]

```

```

[pairList p??]
[union p??]
[listOfIdentifiersIn p??]
[delete p??]
[orderByDependency p227]
[assocleft p??]
[assocright p??]
[comp p587]
[$sigAlist p??]
[$predAlist p??]

```

— defun compDefWhereClause —

```

(defun |compDefWhereClause| (arg mode env)
  (labels (
    (transformType (x)
      (declare (special |$sigAlist|))
      (cond
        ((atom x) x)
        ((and (consp x) (eq (qfirst x) '|:|) (consp (qrest x))
          (consp (qcddr x)) (eq (qcdddr x) nil))
         (setq |$sigAlist|
           (cons (cons (second x) (transformType (third x)))
             |$sigAlist|))
         x)
        ((and (consp x) (eq (qfirst x) '|Record|)) x)
        (t
         (cons (first x)
           (loop for y in (rest x)
             collect (transformType y))))))
    (removeSuchthat (x)
      (declare (special |$predAlist|))
      (if (and (consp x) (eq (qfirst x) '|\\|') (consp (qrest x))
        (consp (qcddr x)) (eq (qcdddr x) nil))
        (progn
          (setq |$predAlist| (cons (cons (second x) (third x)) |$predAlist|))
          (second x))
        x))
    (fetchType (a x env form)
      (if x
        x
        (or (|getmode| a env)
          (|userError| (|concat|
            "There is no mode for argument" a "of function" (first form))))))
    (addSuchthat (x y)
      (let (p)
        (declare (special |$predAlist|))
        (if (setq p (lassoc x |$predAlist|)) (list '|\\| y p) y)))
  )

```

```

(let (|$sigAlist| |$predAlist| form signature specialCases body sigList
    argList argSigAlist argDepAlist varList whereList formxx signaturex
    deform formx)
  (declare (special |$sigAlist| |$predAlist|))
; form is lhs (f a1 ... an) of definition; body is rhs;
; signature is (t0 t1 ... tn) where t0= target type, ti=type of ai, i > 0;
; specialCases is (NIL l1 ... ln) where li is list of special cases
; which can be given for each ti
;
; removes declarative and assignment information from form and
; signature, placing it in list L, replacing form by ("where",form',:L),
; signature by a list of NILs (signifying declarations are in e)
  (setq form (second arg))
  (setq signature (third arg))
  (setq specialCases (fourth arg))
  (setq body (fifth arg))
  (setq |$sigAlist| nil)
  (setq |$predAlist| nil)
; 1. create sigList= list of all signatures which have embedded
;   declarations moved into global variable $sigAlist
  (setq sigList
    (loop for a in (rest form) for x in (rest signature)
      collect (transformType (fetchType a x env form))))
; 2. replace each argument of the form (| x p) by x, recording
;   the given predicate in global variable $predAlist
  (setq argList
    (loop for a in (rest form)
      collect (removeSuchthat a)))
  (setq argSigAlist (append |$sigAlist| (pairList argList sigList)))
  (setq argDepAlist
    (loop for pear in argSigAlist
      collect
        (cons (car pear)
              (|union| (|listOfIdentifiersIn| (cdr pear))
                       (|delete| (car pear)
                                (|listOfIdentifiersIn| (lassoc (car pear) |$predAlist|)))))))
; 3. obtain a list of parameter identifiers (x1 .. xn) ordered so that
;   the type of xi is independent of xj if i < j
  (setq varList
    (|orderByDependency| (assocleft argDepAlist) (assocright argDepAlist)))
; 4. construct a WhereList which declares and/or defines the xi's in
;   the order constructed in step 3
  (setq whereList
    (loop for x in varList
      collect (addSuchthat x (list '|:| x (lassoc x argSigAlist)))))
  (setq formxx (cons (car form) argList))
  (setq signaturex
    (cons (car signature)
          (loop for x in (rest signature) collect nil)))
  (setq deform (list 'def formxx signaturex specialCases body))

```

```

    (setq formx (cons '|where| (cons deform whereList)))
; 5. compile new ('DEF,("where",form',:WhereList),..) where
;    all argument parameters of form' are bound/declared in WhereList
    (|comp| formx mode env)))

```

defun compDefineCategory

```

[compDefineLisplib p167]
[compDefineCategory1 p162]
[$domainShell p??]
[$lisplibCategory p??]
[$lisplib p??]
[$insideFunctorIfTrue p??]

```

— defun compDefineCategory —

```

(defun |compDefineCategory| (df mode env prefix fal)
  (let (|$domainShell| |$lisplibCategory|)
    (declare (special |$domainShell| |$lisplibCategory| $lisplib
                      |$insideFunctorIfTrue|))
    (setq |$domainShell| nil) ; holds the category of the object being compiled
    (setq |$lisplibCategory| nil)
    (if (and (null |$insideFunctorIfTrue|) $lisplib)
        (|compDefineLisplib| df mode env prefix fal '|compDefineCategory1|)
        (|compDefineCategory1| df mode env prefix fal))))

```

defun compDefineCategory1

```

[compDefineCategory2 p163]
[makeCategoryPredicates p179]
[compDefine1 p145]
[mkCategoryPackage p180]
[$insideCategoryPackageIfTrue p??]
[$EmptyMode p176]
[$categoryPredicateList p??]
[$lisplibCategory p??]
[$bootstrapMode p??]

```

— defun compDefineCategory1 —

```

(defun |compDefineCategory1| (df mode env prefix fal)
  (let (|$insideCategoryPackageIfTrue| |$categoryPredicateList| form
        sig sc cat body categoryCapsule d tmp1 tmp3)
    (declare (special |$insideCategoryPackageIfTrue| |$EmptyMode|
                      |$categoryPredicateList| |$lisplibCategory|
                      |$bootStrapMode|))
    ;; a category is a DEF form with 4 parts:
    ;; ((DEF (|BasicType|) ((|Category|)) (NIL)
    ;;      (|add| (CATEGORY |domain| (SIGNATURE = ((|Boolean|) $ $))
    ;;              (SIGNATURE ~= ((|Boolean|) $ $)))
    ;;      (CAPSULE (DEF (~= |x| |y|) ((|Boolean|) $ $) (NIL NIL NIL)
    ;;                (IF (= |x| |y|) |false| |true|)))))
    (setq form (second df))
    (setq sig (third df))
    (setq sc (fourth df))
    (setq body (fifth df))
    (setq categoryCapsule
      (when (and (consp body) (eq (qfirst body) '|add|)
                (consp (qrest body)) (consp (qcddr body))
                (eq (qcdddr body) nil))
        (setq tmp1 (third body))
        (setq body (second body))
        tmp1))
    (setq tmp3 (|compDefineCategory2| form sig sc body mode env prefix fal))
    (setq d (first tmp3))
    (setq mode (second tmp3))
    (setq env (third tmp3))
    (when (and categoryCapsule (null |$bootStrapMode|))
      (setq |$insideCategoryPackageIfTrue| t)
      (setq |$categoryPredicateList|
        (|makeCategoryPredicates| form |$lisplibCategory|))
      (setq env (third
        (|compDefine1|
          (|mkCategoryPackage| form cat categoryCapsule) |$EmptyMode| env))))
    (list d mode env)))

```

defun compDefineCategory2

```

[addBinding p??]
[getArgumentModeOrMoan p191]
[giveFormalParametersValues p178]
[take p??]
[sublis p??]
[compMakeDeclaration p622]
[opOf p??]

```

```

[optFunctorBody p??]
[compOrCroak p586]
[mkConstructor p205]
[compile p173]
[lisplibWrite p214]
[removeZeroOne p??]
[mkq p??]
[evalAndRwriteLispForm p204]
[eval p??]
[getParentsFor p??]
[computeAncestorsOf p??]
[constructor? p??]
[augLisplibModemapsFromCategory p191]
[$prefix p??]
[$formalArgList p??]
[$definition p??]
[$form p??]
[$op p??]
[$extraParms p??]
[$lisplibCategory p??]
[$FormalMapVariableList p270]
[$libFile p??]
[$TriangleVariableList p??]
[$lisplib p??]
[$formalArgList p??]
[$insideCategoryIfTrue p??]
[$top-level p??]
[$definition p??]
[$form p??]
[$op p??]
[$extraParms p??]
[$functionStats p??]
[$functorStats p??]
[$frontier p??]
[$getDomainCode p??]
[$addForm p??]
[$lisplibAbbreviation p??]
[$functorForm p??]
[$lisplibAncestors p??]
[$lisplibCategory p??]
[$lisplibParents p??]
[$lisplibModemap p??]
[$lisplibKind p??]
[$lisplibForm p??]
[$domainShell p??]

```


— defun compDefineCategory2 —

```

(defun |compDefineCategory2|
  (form signature specialCases body mode env |$prefix| |$formalArgList|)
  (declare (special |$prefix| |$formalArgList|) (ignore specialCases))
  (let (|$insideCategoryIfTrue| $TOP_LEVEL |$definition| |$form| |$op|
        |$extraParms| |$functionStats| |$functorStats| |$frontier|
        |$getDomainCode| |$addForm| argl sargl aList signaturep opp formp
        formalBody formals actuals g fun pairlis parSignature parForm modemap)
    (declare (special |$insideCategoryIfTrue| $top_level |$definition|
                    |$form| |$op| |$extraParms| |$functionStats|
                    |$functorStats| |$frontier| |$getDomainCode|
                    |$addForm| |$lisplibAbbreviation| |$functorForm|
                    |$lisplibAncestors| |$lisplibCategory|
                    |$FormalMapVariableList| |$lisplibParents|
                    |$lisplibModemap| |$lisplibKind| |$lisplibForm|
                    $lisplib |$domainShell| |$libFile|
                    |$TriangleVariableList|))
      ; 1. bind global variables
      (setq |$insideCategoryIfTrue| t)
      (setq $top_level nil)
      (setq |$definition| nil)
      (setq |$form| nil)
      (setq |$op| nil)
      (setq |$extraParms| nil)
      ; 1.1 augment e to add declaration $: <form>
      (setq |$definition| form)
      (setq |$op| (car |$definition|))
      (setq argl (cdr |$definition|))
      (setq env (|addBinding| '$ (list (cons '|mode| |$definition|)) env))
      ; 2. obtain signature
      (setq signaturep
        (cons (car signature)
              (loop for a in argl
                    collect (|getArgumentModeOrMoan| a |$definition| env))))
      (setq env (|giveFormalParametersValues| argl env))
      ; 3. replace arguments by $1,..., substitute into body,
      ;    and introduce declarations into environment
      (setq sargl (take (|#| argl) |$TriangleVariableList|))
      (setq |$form| (cons |$op| sargl))
      (setq |$functorForm| |$form|)
      (setq |$formalArgList| (append sargl |$formalArgList|))
      (setq aList (loop for a in argl for sa in sargl collect (cons a sa)))
      (setq formalBody (sublis aList body))
      (setq signaturep (sublis aList signaturep))
      ; Begin lines for category default definitions
      (setq |$functionStats| (list 0 0))
      (setq |$functorStats| (list 0 0))
      (setq |$frontier| 0)
      (setq |$getDomainCode| nil)

```

```

(setq |$addForm| nil)
(loop for x in sargl for r in (rest signaturep)
  do (setq env (third (|compMakeDeclaration| (list '|:| x r) mode env))))
; 4. compile body in environment of %type declarations for arguments
(setq opp |$op|)
(when (and (not (eq (|opOf| formalBody) '|Join|))
  (not (eq (|opOf| formalBody) '|mkCategory|))))
  (setq formalBody (list '|Join| formalBody)))
(setq body
  (|optFunctorBody| (car (|compOrCroak| formalBody (car signaturep) env))))
(when |$extraParms|
  (setq actuals nil)
  (setq formals nil)
  (loop for u in |$extraParms| do
    (setq formals (cons (car u) formals))
    (setq actuals (cons (mkq (cdr u)) actuals)))
  (setq body
    (list '|sublisV| (list 'pair (list 'quote formals) (cons 'list actuals))
      body)))
; always subst for args after extraparms
(when argl
  (setq body
    (list '|sublisV|
      (list 'pair
        (list 'quote sargl)
        (cons 'list (loop for u in sargl collect (list '|devalueate| u))))
      body)))
(setq body
  (list 'prog1 (list 'let (setq g (gensym)) body)
    (list 'setelt g 0 (|mkConstructor| |$form|))))
(setq fun (|compile| (list opp (list 'lam sargl body))))
; 5. give operator a 'modemap property
(setq pairlis
  (loop for a in argl for v in |$FormalMapVariableList|
    collect (cons a v)))
(setq parSignature (sublis pairlis signaturep))
(setq parForm (sublis pairlis form))
(|lisplibWrite| "compilerInfo"
  (|removeZeroOne|
    (list 'setq '|$CategoryFrame|
      (list '|put| (list 'quote opp) '|isCategory| t
        (list '|addModemap| (mkq opp) (mkq parForm)
          (mkq parSignature) t (mkq fun) '|$CategoryFrame|))))
  |$libFile|)
(unless sargl
  (|evalAndRwriteLispForm| 'niladic
    '(setf (get ',opp 'niladic) t)))
;; 6 put modemaps into InteractiveModemapFrame
(setq |$domainShell| (|eval| (cons opp (mapcar 'mkq sargl))))
(setq |$lisplibCategory| formalBody)

```

```

(when $lisplib
  (setq |$lisplibForm| form)
  (setq |$lisplibKind| '|category|)
  (setq modemap (list (cons parForm parSignature) (list t opp)))
  (setq |$lisplibModemap| modemap)
  (setq |$lisplibParents|
    (|getParentsFor| |$op| |$FormalMapVariableList| |$lisplibCategory|))
  (setq |$lisplibAncestors| (|computeAncestorsOf| |$form| nil))
  (setq |$lisplibAbbreviation| (|constructor?| |$op|))
  (setq formp (cons opp sargl))
  (|augLisplibModemapsFromCategory| formp formalBody signaturep))
(list fun '(|Category|) env)))

```

defun compDefineLisplib

```

[sayMSG p??]
[fillerSpaces p??]
[getConstructorAbbreviation p??]
[compileDocumentation p169]
[bright p??]
[finalizeLisplib p207]
[rshut p??]
[lisplibDoRename p205]
[filep p??]
[rpackfile p??]
[unloadOneConstructor p205]
[localdatabase p??]
[getdatabase p??]
[updateCategoryFrameForCategory p121]
[updateCategoryFrameForConstructor p120]
[$compileDocumentation p169]
[$filep p??]
[$spadLibFT p??]
[$algebraOutputStream p??]
[$newConlist p555]
[$lisplibKind p??]
[$lisplib p??]
[$op p??]
[$lisplibParents p??]
[$lisplibPredicates p??]
[$lisplibCategoriesExtended p??]
[$lisplibForm p??]
[$lisplibKind p??]

```

```

[$lisplibAbbreviation p??]
[$lisplibAncestors p??]
[$lisplibModemap p??]
[$lisplibModemapAlist p??]
[$lisplibSlot1 p??]
[$lisplibOperationAlist p??]
[$lisplibSuperDomain p??]
[$libFile p??]
[$lisplibVariableAlist p??]
[$lisplibCategory p??]
[$newConlist p555]

```

— defun compDefineLisplib —

```

(defun |compDefineLisplib| (df m env prefix fal fn)
  (let ($LISPLIB |$op| |$lisplibAttributes| |$lisplibPredicates|
        |$lisplibCategoriesExtended| |$lisplibForm| |$lisplibKind|
        |$lisplibAbbreviation| |$lisplibParents| |$lisplibAncestors|
        |$lisplibModemap| |$lisplibModemapAlist| |$lisplibSlot1|
        |$lisplibOperationAlist| |$lisplibSuperDomain| |$libFile|
        |$lisplibVariableAlist| |$lisplibCategory| op libname res ok filearg)
    (declare (special $lisplib |$op| |$lisplibAttributes| |$newConlist|
                      |$lisplibPredicates| |$lisplibCategoriesExtended| | |
                      |$lisplibForm| |$lisplibKind| |$algebraOutputStream|
                      |$lisplibAbbreviation| |$lisplibParents| |$spadLibFT|
                      |$lisplibAncestors| |$lisplibModemap| $filep
                      |$lisplibModemapAlist| |$lisplibSlot1|
                      |$lisplibOperationAlist| |$lisplibSuperDomain|
                      |$libFile| |$lisplibVariableAlist|
                      |$lisplibCategory| |$compileDocumentation|))
      (when (eq (car df) 'def) (car df))
      (setq op (caadr df))
      (|sayMSG| (|fillerSpaces| 72 "-"))
      (setq $lisplib t)
      (setq |$op| op)
      (setq |$lisplibAttributes| nil)
      (setq |$lisplibPredicates| nil)
      (setq |$lisplibCategoriesExtended| nil)
      (setq |$lisplibForm| nil)
      (setq |$lisplibKind| nil)
      (setq |$lisplibAbbreviation| nil)
      (setq |$lisplibParents| nil)
      (setq |$lisplibAncestors| nil)
      (setq |$lisplibModemap| nil)
      (setq |$lisplibModemapAlist| nil)
      (setq |$lisplibSlot1| nil)
      (setq |$lisplibOperationAlist| nil)
      (setq |$lisplibSuperDomain| nil)
      (setq |$libFile| nil)

```

```

(setq |$lisplibVariableAlist| nil)
(setq |$lisplibCategory| nil)
(setq libname (|getConstructorAbbreviation| op))
(cond
  ((and (boundp '|$compileDocumentation|) |$compileDocumentation|)
    (|compileDocumentation| libname))
  (t
    (|sayMSG| (cons "    initializing " (cons |$spadLibFT|
      (append (|bright| libname) (cons "for" (|bright| op))))))
    (|initializeLisplib| libname)
    (|sayMSG|
      (cons "    compiling into " (cons |$spadLibFT| (|bright| libname))))
    (setq ok nil)
    (unwind-protect
      (progn
        (setq res (funcall fn df m env prefix fal))
        (|sayMSG| (cons "    finalizing " (cons |$spadLibFT| (|bright| libname))))
        (|finalizeLisplib| libname)
        (setq ok t))
      (rshut |$libFile|))
    (when ok (|lisplibDoRename| libname))
    (setq filearg ($filep libname |$spadLibFT| 'a))
    (rpackfile filearg)
    (fresh-line |$algebraOutputStream|)
    (|sayMSG| (|fillerSpaces| 72 "-"))
    (|unloadOneConstructor| op libname)
    (localdatabase (list (getdatabase op 'abbreviation)) nil)
    (setq |$newConlist| (cons op |$newConlist|))
    (when (eq |$lisplibKind| '|category|)
      (|updateCategoryFrameForCategory| op)
      (|updateCategoryFrameForConstructor| op))
    res))))

```

defun compileDocumentation

```

[makeInputFilename p??]
[rdefiostream p??]
[lisplibWrite p214]
[finalizeDocumentation p490]
[rshut p??]
[rpackfile p??]
[replaceFile p??]
[$fcopy p??]
[$spadLibFT p??]
[$EmptyMode p176]

```

`[ $\$e$  p??]`

— **defun compileDocumentation** —

```
(defun |compileDocumentation| (libName)
  (let (filename stream)
    (declare (special | $\$e$ | |$EmptyMode| |$spadLibFT| $fcopy))
    (setq filename (makeInputFilename libName |$spadLibFT|))
    ($fcopy filename (cons libname (list 'doclb)))
    (setq stream
      (rdefiostream (cons (list 'file libName 'doclb) (list (cons 'mode 'o)))))
    (|lisplibWrite| "documentation" (|finalizeDocumentation|) stream)
    (rshut stream)
    (rpackfile (list libName 'doclb))
    (replaceFile (list libName |$spadLibFT|) (list libName 'doclb))
    (list 'dummy| |$EmptyMode| | $\$e$ |)))
```

defun compArgumentConditions

`[compOrCroak p586]`
`[ $\$$ Boolean p??]`
`[ $\$$ argumentConditionList p??]`
`[ $\$$ argumentConditionList p??]`

— **defun compArgumentConditions** —

```
(defun |compArgumentConditions| (env)
  (let (n a x tmp1)
    (declare (special |$Boolean| |$argumentConditionList|))
    (setq |$argumentConditionList|
      (loop for item in |$argumentConditionList|
        do
          (setq n (first item))
          (setq a (second item))
          (setq x (third item))
          (setq y (subst a '|#1| x :test #'equal))
          (setq tmp1 (|compOrCroak| y |$Boolean| env))
          (setq env (third tmp1))
        collect
        (list n x (first tmp1))))
    env))
```

defun compileCases

```
[eval p??]
[compile p173]
[getSpecialCaseAssoc p304]
[get p??]
[assocleft p??]
[outerProduct p??]
[assocright p??]
[mkpf p??]
[$getDomainCode p??]
[$insideFunctorIfTrue p??]
[$specialCaseKeyList p??]
```

— defun compileCases —

```
(defun |compileCases| (x |$e|)
  (declare (special |$e|))
  (labels (
    (isEltArgumentIn (Rlist x)
      (cond
        ((atom x) nil)
        ((and (consp x) (eq (qfirst x) 'elt) (consp (qrest x))
          (consp (qcddr x)) (eq (qcdddr x) nil))
          (or (member (second x) Rlist)
            (isEltArgumentIn Rlist (cdr x))))
        ((and (consp x) (eq (qfirst x) 'qrefelt) (consp (qrest x))
          (consp (qcddr x)) (eq (qcdddr x) nil))
          (or (member (second x) Rlist)
            (isEltArgumentIn Rlist (cdr x))))
        (t
          (or (isEltArgumentIn Rlist (car x))
            (isEltArgumentIn Rlist (CDR x))))))
    (FindNamesFor (r rp)
      (let (v u)
        (declare (special |$getDomainCode|))
        (cons r
          (loop for item in |$getDomainCode|
            do
              (setq v (second item))
              (setq u (third item))
              when (and (equal (second u) r) (|eval| (subst rp r u :test #'equal)))
                collect v))))))
  (let (|$specialCaseKeyList| specialCaseAssoc listOfDomains listOfAllCases cl)
    (declare (special |$specialCaseKeyList| |$true| |$insideFunctorIfTrue|))
    (setq |$specialCaseKeyList| nil)
    (cond
      ((null (eq |$insideFunctorIfTrue| t)) (|compile| x))
      (t
```

```

(setq specialCaseAssoc
  (loop for y in (|getSpecialCaseAssoc|)
    when (and (null (|get| (first y) '|specialCase| |$e|))
      (isEltArgumentIn (FindNamesFor (first y) (second y)) x))
    collect y))
(cond
  ((null specialCaseAssoc) (|compile| x))
  (t
    (setq listOfDomains (assocleft specialCaseAssoc))
    (setq listOfAllCases (|outerProduct| (assocright specialCaseAssoc)))
    (setq cl
      (loop for z in listOfAllCases
        collect
          (progn
            (setq |$specialCaseKeyList|
              (loop for d in listOfDomains for c in z
                collect (cons d c)))
            (cons
              (mkpf
                (loop for d in listOfDomains for c in z
                  collect (list 'equal d c))
                'and)
              (list (|compile| (copy x)))))))
    (setq |$specialCaseKeyList| nil)
    (cons 'cond (append cl (list (list |$true| (|compile| x))))))))))

```

defun compFunctorBody

```

[bootstrapError p219]
[compOrCroak p586]
[/editfile p??]
[$NRTaddForm p??]
[$functorForm p??]
[$bootstrapMode p??]

```

— defun compFunctorBody —

```

(defun |compFunctorBody| (form mode env parForm)
  (declare (ignore parForm))
  (let (tt)
    (declare (special |$NRTaddForm| |$functorForm| |$bootstrapMode| /editfile))
    (if |$bootstrapMode|
      (list (|bootstrapError| |$functorForm| /editfile) mode env)
      (progn
        (setq tt (|compOrCroak| form mode env))

```



```

(if (and (consp form) (member (qfirst form) '(|add| capsule)))
  tt
  (progn
    (setq |$NRTaddForm|
      (if (and (consp form) (eq (qfirst form) '|SubDomain|)
              (consp (qrest form)) (consp (qcddr form))
              (eq (qcdddr form) nil))
          (qsecond form)
          form))
    tt))))))

```

defun compile

```

[member p??]
[getmode p??]
[get p??]
[modeEqual p366]
[userError p??]
[encodeItem p185]
[strconc p??]
[encodeFunctionName p183]
[splitEncodedFunctionName p184]
[sayBrightly p??]
[optimizeFunctionDef p228]
[putInLocalDomainReferences p189]
[constructMacro p186]
[spadCompileOrSetq p186]
[elapsedTime p??]
[addStats p??]
[printStats p??]
[$functionStats p??]
[$macroIfTrue p??]
[$doNotCompileJustPrint p??]
[$insideCapsuleFunctionIfTrue p??]
[$saveableItems p??]
[$lisplibItemsAlreadyThere p??]
[$splitUpItemsAlreadyThere p??]
[$lisplib p??]
[$compileOnlyCertainItems p??]
[$functorForm p??]
[$signatureOfForm p??]
[$suffix p??]
[$prefix p??]

```

```

[$signatureOfForm p??]
[$e p??]
[$functionStats p??]
[$savableItems p??]
[$suffix p??]

```

— defun compile —

```

(defun |compile| (u)
  (labels (
    (isLocalFunction (op)
      (let (tmp1)
        (declare (special |$e| |$formalArgList|))
        (and (null (|member| op |$formalArgList|))
          (progn
            (setq tmp1 (|getmode| op |$e|))
            (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|)))))))
    (let (op lamExpr DC sig sel opexport opmodes opp parts s tt unew
      optimizedBody stuffToCompile result functionStats)
      (declare (special |$functionStats| |$macroIfTrue| |$doNotCompileJustPrint|
        |$insideCapsuleFunctionIfTrue| |$saveableItems| |$e|
        |$lisplibItemsAlreadyThere| |$splitUpItemsAlreadyThere|
        |$compileOnlyCertainItems| $LISPLIB |$suffix|
        |$signatureOfForm| |$functorForm| |$prefix|
        |$savableItems|))
        (setq op (first u))
        (setq lamExpr (second u))
        (when |$suffix|
          (setq |$suffix| (1+ |$suffix|))
          (setq opp
            (progn
              (setq opexport nil)
              (setq opmodes
                (loop for item in (|get| op '|modemap| |$e|)
                  do
                    (setq dc (caar item))
                    (setq sig (cdar item))
                    (setq sel (cadadr item))
                    when (and (eq dc '$)
                      (setq opexport t)
                      (let ((result t))
                        (loop for x in sig for y in |$signatureOfForm|
                          do (setq result (|modeEqual| x y)))
                        result)))
                    collect sel)))
              (cond
                ((isLocalFunction op)
                  (when opexport
                    (|userError| (list op " is local and exported")))))

```

```

    (intern (strconc (|encodeItem| |$prefix|) ";" (|encodeItem| op))))
  (t
    (|encodeFunctionName| op |$functorForm| |$signatureOfForm|
      '|;| |$suffix|))))))
  (setq u (list opp lamExpr)))
(when (and $lisplib |$compileOnlyCertainItems|)
  (setq parts (|splitEncodedFunctionName| (elt u 0) '|;|))
  (cond
    ((eq parts '|inner|)
      (setq |$savableItems| (cons (elt u 0) |$savableItems|)))
    (t
      (setq unew nil)
      (loop for item in |$splitUpItemsAlreadyThere|
        do
          (setq s (first item))
          (setq tt (second item))
          (when
            (and (equal (elt parts 0) (elt s 0))
                  (equal (elt parts 1) (elt s 1))
                  (equal (elt parts 2) (elt s 2)))
            (setq unew tt)))
      (cond
        ((null unew)
          (|sayBrightly| (list " Error: Item did not previously exist"))
          (|sayBrightly| (cons " Item not saved: " (|bright| (elt u 0))))
          (|sayBrightly|
            (list " What's there is: " |$lisplibItemsAlreadyThere|))
          nil)
        (t
          (|sayBrightly| (list " Renaming " (elt u 0) " as " unew))
          (setq u (cons unew (cdr u)))
          (setq |$savableItems| (cons unew |$saveableItems|))))))
  (setq optimizedBody (|optimizeFunctionDef| u))
  (setq stuffToCompile
    (if |$insideCapsuleFunctionIfTrue|
      (|putInLocalDomainReferences| optimizedBody)
      optimizedBody))
  (cond
    ((eq |$doNotCompileJustPrint| t)
      (prettyprint stuffToCompile)
      opp)
    (|$macroIfTrue| (|constructMacro| stuffToCompile))
    (t
      (setq result (|spadCompileOrSetq| stuffToCompile))
      (setq functionStats (list 0 (|elapsedTime|)))
      (setq |$functionStats| (|addStats| |$functionStats| functionStats))
      (|printStats| functionStats)
      result))))))

```

defvar \$NoValueMode

— initvars —

```
(defvar |$NoValueMode| ' |NoValueMode|)
```

defvar \$EmptyMode

`$EmptyMode` is a constant whose value is `$EmptyMode`. It is used by `isPartialMode` to decide if a modemap is partially constructed. If the `$EmptyMode` constant occurs anywhere in the modemap structure at any depth then the modemap is still incomplete. To find this constant the `isPartialMode` function calls `CONTAINED $EmptyMode Y` which will walk the structure `Y` looking for this constant.

— initvars —

```
(defvar |$EmptyMode| ' |EmptyMode|)
```

defun hasFullSignature

TPDHERE: test with BASTYPE [get p??]

— defun hasFullSignature —

```
(defun |hasFullSignature| (arg1 signature env)
  (let (target ml u)
    (setq target (first signature))
    (setq ml (rest signature))
    (when target
      (setq u
        (loop for x in arg1 for m in ml
              collect (or m (|get| x ' |mode| env) (return 'failed))))
      (unless (eq u 'failed) (cons target u))))))
```

defun addEmptyCapsuleIfNecessary

[[\\$SpecialDomainNames](#) [p??](#)]

— defun addEmptyCapsuleIfNecessary —

```
(defun |addEmptyCapsuleIfNecessary| (target rhs)
  (declare (special |$SpecialDomainNames|) (ignore target))
  (if (member (ifcar rhs) |$SpecialDomainNames|)
      rhs
      (list '|add| rhs (list 'capsule))))
```

—————

defun getTargetFromRhs

[[stackSemanticError](#) [p??](#)]

[[getTargetFromRhs](#) [p177](#)]

[[compOrCroak](#) [p586](#)]

— defun getTargetFromRhs —

```
(defun |getTargetFromRhs| (lhs rhs env)
  (declare (special |$EmptyMode|))
  (cond
    ((and (consp rhs) (eq (qfirst rhs) 'capsule))
     (|stackSemanticError|
      (list "target category of " lhs
            " cannot be determined from definition")
      nil))
    ((and (consp rhs) (eq (qfirst rhs) '|SubDomain|) (consp (qrest rhs)))
     (|getTargetFromRhs| lhs (second rhs) env))
    ((and (consp rhs) (eq (qfirst rhs) '|add|)
          (consp (qrest rhs)) (consp (qcddr rhs))
          (eq (qcdddr rhs) nil)
          (consp (qthird rhs))
          (eq (qcaaddr rhs) 'capsule))
     (|getTargetFromRhs| lhs (second rhs) env))
    ((and (consp rhs) (eq (qfirst rhs) '|Record|))
     (cons '|RecordCategory| (rest rhs)))
    ((and (consp rhs) (eq (qfirst rhs) '|Union|))
     (cons '|UnionCategory| (rest rhs)))
    ((and (consp rhs) (eq (qfirst rhs) '|List|))
     (cons '|ListCategory| (rest rhs)))
    ((and (consp rhs) (eq (qfirst rhs) '|Vector|))
     (cons '|VectorCategory| (rest rhs)))
    (t
```

```
(second (|compOrCroak| rhs |$EmptyMode| env))))))
```

defun giveFormalParametersValues

```
[put p??]  
[get p??]
```

— defun giveFormalParametersValues —

```
(defun |giveFormalParametersValues| (arg1 env)  
  (dolist (x arg1)  
    (setq env  
      (|put| x '|value|  
        (list (|genSomeVariable|) (|get| x '|mode| env) nil) env)))  
  env)
```

defun macroExpandInPlace

```
[macroExpand p178]
```

— defun macroExpandInPlace —

```
(defun |macroExpandInPlace| (form env)  
  (let (y)  
    (setq y (|macroExpand| form env))  
    (if (or (atom form) (atom y))  
        y  
        (progn  
          (rplaca form (car y))  
          (rplacd form (cdr y))  
          form  
        ))))
```

defun macroExpand

```
[macroExpand p178]  
[macroExpandList p179]
```

— defun macroExpand —

```
(defun |macroExpand| (form env)
  (let (u)
    (cond
      ((atom form)
       (if (setq u (|get| form '|macro| env))
           (|macroExpand| u env)
           form))
      ((and (consp form) (eq (qfirst form) 'def)
            (consp (qrest form))
            (consp (qcddr form))
            (consp (qcdddr form))
            (consp (qcdddr form))
            (eq (qrest (qcdddr form)) nil))
       (list 'def (|macroExpand| (second form) env)
              (|macroExpandList| (third form) env)
              (|macroExpandList| (fourth form) env)
              (|macroExpand| (fifth form) env)))
      (t (|macroExpandList| form env))))
```

defun macroExpandList

[macroExpand p178]
[getdatabase p??]

— defun macroExpandList —

```
(defun |macroExpandList| (lst env)
  (let (tmp)
    (if (and (consp lst) (eq (qrest lst) nil)
            (identp (qfirst lst)) (getdatabase (qfirst lst) 'niladic)
            (setq tmp (|get| (qfirst lst) '|macro| env)))
        (|macroExpand| tmp env)
        (loop for x in lst collect (|macroExpand| x env))))
```

defun makeCategoryPredicates

[\$FormalMapVariableList p270]
[\$TriangleVariableList p??]

```
[$mvl p??]
[$tv1 p??]
```

— **defun makeCategoryPredicates** —

```
(defun |makeCategoryPredicates| (form u)
  (labels (
    (fn (u pl)
      (declare (special |$tv1| |$mvl|))
      (cond
        ((and (consp u) (eq (qfirst u) '|Join|) (consp (qrest u)))
          (fn (car (reverse (qrest u))) pl))
        ((and (consp u) (eq (qfirst u) '|has|))
          (|insert| (eqsubstlist |$mvl| |$tv1| u) pl))
        ((and (consp u) (member (qfirst u) '(signature attribute))) pl)
        ((atom u) pl)
        (t (fn1 u pl))))
    (fn1 (u pl)
      (dolist (x u) (setq pl (fn x pl)))
      pl))
  (declare (special |$FormalMapVariableList| |$mvl| |$tv1|
    |$TriangleVariableList|))
  (setq |$tv1| (take (|#| (cdr form)) |$TriangleVariableList|))
  (setq |$mvl| (take (|#| (cdr form)) (cdr |$FormalMapVariableList|)))
  (fn u nil)))
```

defun mkCategoryPackage

```
[strconc p??]
[pname p??]
[getdatabase p??]
[abbreviationsSpad2Cmd p??]
[JoinInner p??]
[assoc p??]
[sublislis p??]
[$options p??]
[$categoryPredicateList p??]
[$e p??]
[$FormalMapVariableList p270]
```

— **defun mkCategoryPackage** —

```
(defun |mkCategoryPackage| (form cat def)
  (labels (
```



```

(fn (x oplist)
  (cond
    ((atom x) oplist)
    ((and (consp x) (eq (qfirst x) 'def) (consp (qrest x)))
      (cons (second x) oplist))
    (t
      (fn (cdr x) (fn (car x) oplist))))))
(gn (cat)
  (cond
    ((and (consp cat) (eq (qfirst cat) 'category)) (cddr cat))
    ((and (consp cat) (eq (qfirst cat) '|Join|)) (gn (|last| (qrest cat))))
    (t nil)))
(let (|$options| op argl packageName packageAbb nameForDollar packageArgl
      capsuleDefAlist explicitCatPart catvec fullCatOpList op1 sig
      catOpList packageCategory nils packageSig)
  (declare (special |$options| |$categoryPredicateList| |$e|
                    |$FormalMapVariableList|))
  (setq op (car form))
  (setq argl (cdr form))
  (setq packageName (intern (strconc (pname op) "&")))
  (setq packageAbb (intern (strconc (getdatabase op 'abbreviation) "-")))
  (setq |$options| nil)
  (|abbreviationsSpad2Cmd| (list '|domain| packageAbb packageName))
  (setq nameForDollar (car (setdifference '(s a b c d e f g h i) argl)))
  (setq packageArgl (cons nameForDollar argl))
  (setq capsuleDefAlist (fn def nil))
  (setq explicitCatPart (gn cat))
  (setq catvec (|eval| (|mkEvaluableCategoryForm| form)))
  (setq fullCatOpList (elt (|JoinInner| (list catvec) |$e|) 1))
  (setq catOpList
    (loop for x in fullCatOpList do
      (setq op1 (caar x))
      (setq sig (cadar x))
      when (|assoc| op1 capsuleDefAlist)
        collect (list 'signature op1 sig)))
  (when catOpList
    (setq packageCategory
      (cons 'category
        (cons '|domain| (sublislis argl |$FormalMapVariableList| catOpList))))
    (setq nils (loop for x in argl collect nil))
    (setq packageSig (cons packageCategory (cons form nils)))
    (setq |$categoryPredicateList|
      (subst nameForDollar '$ |$categoryPredicateList| :test #'equal))
    (subst nameForDollar '$
      (list 'def (cons packageName packageArgl)
        packageSig (cons nil nils) def) :test #'equal))))

```

defun mkEvalableCategoryForm

```
[mkEvalableCategoryForm p182]
[compOrCroak p586]
[getdatabase p??]
[get p??]
[mkq p??]
[$Category p??]
[$e p??]
[$EmptyMode p176]
[$CategoryFrame p??]
[$Category p??]
[$CategoryNames p??]
[$e p??]
```

— defun mkEvalableCategoryForm —

```
(defun |mkEvalableCategoryForm| (c)
  (let (op argl tmp1 x m)
    (declare (special |$Category| |$e| |$EmptyMode| |$CategoryFrame|
                      |$CategoryNames|))
    (if (consp c)
        (progn
          (setq op (qfirst c))
          (setq argl (qrest c))
          (cond
            ((eq op '|Join|)
             (cons '|Join|
                   (loop for x in argl
                        collect (|mkEvalableCategoryForm| x))))
            ((eq op '|DomainSubstitutionMacro|)
             (|mkEvalableCategoryForm| (cadr argl)))
            ((eq op '|mkCategory|) c)
            ((member op |$CategoryNames|)
             (setq tmp1 (|compOrCroak| c |$EmptyMode| |$e|))
             (setq x (car tmp1))
             (setq m (cadr tmp1))
             (setq |$e| (caddr tmp1))
             (when (equal m |$Category|) x))
            ((or (eq (getdatabase op 'constructorkind) '|category|)
                  (|get| op '|isCategory| |$CategoryFrame|))
             (cons op
                   (loop for x in argl
                        collect (mkq x))))
            (t
             (setq tmp1 (|compOrCroak| c |$EmptyMode| |$e|))
             (setq x (car tmp1))
             (setq m (cadr tmp1))
             (setq |$e| (caddr tmp1))
```

```
(when (equal m |$Category|) x)))
(mkq c)))
```

defun encodeFunctionName

Code for encoding function names inside package or domain [mkRepetitionAssoc p184]

```
[encodeItem p185]
[internl p??]
[getAbbreviation p302]
[length p??]
[$lisplib p??]
[$lisplibSignatureAlist p??]
[$lisplibSignatureAlist p??]
```

— defun encodeFunctionName —

```
(defun |encodeFunctionName| (fun package signature sep count)
  (let (packageName arglist signaturep reducedSig n x encodedSig encodedName)
    (declare (special |$lisplibSignatureAlist| $lisplib))
    (setq packageName (car package))
    (setq arglist (cdr package))
    (setq signaturep (subst '$ package signature :test #'equal))
    (setq reducedSig
      (|mkRepetitionAssoc| (append (cdr signaturep) (list (car signaturep)))))
    (setq encodedSig
      (let ((result ""))
        (loop for item in reducedSig
          do
            (setq n (car item))
            (setq x (cdr item))
            (setq result
              (strconc result
                (if (eql n 1)
                  (|encodeItem| x)
                  (strconc (princ-to-string n) (|encodeItem| x))))))
        result))
    (setq encodedName
      (internl (|getAbbreviation| packageName (|#| arglist))
        '|;| (|encodeItem| fun) '|;| encodedSig sep (princ-to-string count)))
    (when $lisplib
      (setq |$lisplibSignatureAlist|
        (cons (cons encodedName signaturep) |$lisplibSignatureAlist|)))
    encodedName))
```

defun mkRepititionAssoc

[mkRepfun p??]

— defun mkRepititionAssoc —

```
(defun |mkRepititionAssoc| (z)
  (labels (
    (mkRepfun (z n)
      (cond
        ((null z) nil)
        ((and (consp z) (eq (qrest z) nil) (list (cons n (qfirst z)))))
        ((and (consp z) (consp (qrest z)) (equal (qsecond z) (qfirst z)))
          (mkRepfun (cdr z) (1+ n)))
        (t (cons (cons n (car z)) (mkRepfun (cdr z) 1))))))
    (mkRepfun z 1)))
```

defun splitEncodedFunctionName

[strpos p??]

— defun splitEncodedFunctionName —

```
(defun |splitEncodedFunctionName| (encodedName sep)
  (let (sep0 p1 p2 p3 s1 s2 s3 s4)
    ; sep0 is the separator used in "encodeFunctionName".
    (setq sep0 ";")
    (unless (stringp encodedName) (setq encodedName (princ-to-string encodedName)))
    (cond
      ((null (setq p1 (strpos sep0 encodedName 0 "*"))) nil)
      ; This is picked up in compile for inner functions in partial compilation
      ((null (setq p2 (strpos sep0 encodedName (1+ p1) "*"))) '|inner|)
      ((null (setq p3 (strpos sep encodedName (1+ p2) "*"))) nil)
      (t
       (setq s1 (substring encodedName 0 p1))
       (setq s2 (substring encodedName (1+ p1) (- p2 p1 1)))
       (setq s3 (substring encodedName (1+ p2) (- p3 p2 1)))
       (setq s4 (substring encodedName (1+ p3) nil))
       (list s1 s2 s3 s4)))))
```

defun encodeItem

```
[getCaps p185]
[identp p??]
[pname p??]
```

— defun encodeItem —

```
(defun |encodeItem| (x)
  (cond
    ((consp x) (|getCaps| (qfirst x)))
    ((identp x) (pname x))
    (t (princ-to-string x))))
```

defun getCaps

```
[maxindex p??]
[downcase p??]
[strconc p??]
```

— defun getCaps —

```
(defun |getCaps| (x)
  (let (s c clist tmp1)
    (setq s (princ-to-string x))
    (setq clist
      (loop for i from 0 to (maxindex s)
            when (upper-case-p (setq c (elt s i)))
              collect c))
    (cond
      ((null clist) "_")
      (t
       (setq tmp1
         (cons (first clist) (loop for u in (rest clist) collect (downcase u))))
       (let ((result ""))
         (loop for u in tmp1
               do (setq result (strconc result u)))
         result))))))
```

defun constructMacro

```
constructMacro (form is [nam,[lam,vl,body]]) [stackSemanticError p??]
[identp p??]
```

— defun constructMacro —

```
(defun |constructMacro| (form)
  (let (vl body)
    (setq vl (cadadr form))
    (setq body (car (cddadr form)))
    (cond
      ((null (let ((result t))
                (loop for x in vl
                      do (setq result (and result (atom x))))
               result))
        (|stackSemanticError| (list '|illegal parameters for macro: | vl) nil))
      (t
       (list 'xlam (loop for x in vl when (identp x) collect x) body)))))
```

—————

defun spadCompileOrSetq

```
[contained p??]
[sayBrightly p??]
[bright p??]
[LAM,EVALANDFILEACTQ p??]
[mkq p??]
[comp p587]
[compileConstructor p187]
[$insideCapsuleFunctionIfTrue p??]
```

— defun spadCompileOrSetq —

```
(defun |spadCompileOrSetq| (form)
  (let (nam lam vl body namp tmp1 e vlp macform)
    (declare (special |$insideCapsuleFunctionIfTrue|))
    (setq nam (car form))
    (setq lam (caadr form))
    (setq vl (cadadr form))
    (setq body (car (cddadr form)))
    (cond
      ((and (consp vl) (progn (setq tmp1 (reverse vl)) t)
            (consp tmp1)
            (progn
```

```

      (setq e (qfirst tmp1))
      (setq vlp (qrest tmp1))
      t)
    (progn (setq vlp (nreverse vlp)) t)
    (consp body)
    (progn (setq namp (qfirst body)) t)
    (equal (qrest body) vlp))
  (|LAM,EVALANDFILEACTQ|
   (list 'put (mkq nam) (mkq '|SPADreplace|) (mkq namp)))
  (|sayBrightly|
   (cons "      " (append (|bright| nam)
                          (cons "is replaced by" (|bright| namp))))))
  ((and (or (atom body)
            (let ((result t))
              (loop for x in body
                    do (setq result (and result (atom x))))
              result)))
        (consp vl)
        (progn (setq tmp1 (reverse vl)) t)
        (consp tmp1)
        (progn
         (setq e (qfirst tmp1))
         (setq vlp (qrest tmp1))
         t)
        (progn (setq vlp (nreverse vlp)) t)
        (null (contained e body)))
    (setq macform (list 'xlam vlp body))
    (|LAM,EVALANDFILEACTQ|
     (list 'put (mkq nam) (mkq '|SPADreplace|) (mkq macform)))
    (|sayBrightly| (cons "      " (append (|bright| nam)
                                           (cons "is replaced by" (|bright| body))))))
  (t nil))
(if |$insideCapsuleFunctionIfTrue|
  (car (comp (list form)))
  (|compileConstructor| form)))

```

defun compileConstructor

[compileConstructor1 p188]
 [clearClams p??]

— defun compileConstructor —

```

(defun |compileConstructor| (form)
  (let (u)

```

```
(setq u (|compileConstructor1| form))
(|clearClams|)
u))
```

defun compileConstructor1

```
[getdatabase p??]
[compAndDefine p189]
[comp p587]
[clearConstructorCache p??]
[$mutableDomain p??]
[$ConstructorCache p??]
[$clamList p??]
[$clamList p??]
```

— defun compileConstructor1 —

```
(defun |compileConstructor1| (form)
  (let (|$clamList| fn key vl body1 lambdaOrSlam compForm u)
    (declare (special |$clamList| |$ConstructorCache| |$mutableDomain|))
    (setq fn (car form))
    (setq key (caadr form))
    (setq vl (cadadr form))
    (setq body1 (cddadr form))
    (setq |$clamList| nil)
    (setq lambdaOrSlam
      (cond
        ((eq (getdatabase fn 'constructorkind) '|category|) 'spadslam)
        (|$mutableDomain| 'lambda)
        (t
         (setq |$clamList|
              (cons (list fn '|$ConstructorCache| '|domainEqualList| '|count|)
                    |$clamList|))
          'lambda)))
    (setq compForm (list (list fn (cons lambdaOrSlam (cons vl body1)))))
    (if (eq (getdatabase fn 'constructorkind) '|category|)
        (setq u (|compAndDefine| compForm))
        (setq u (comp compForm)))
    (|clearConstructorCache| fn)
    (car u)))
```

defun compAndDefine

This function is used but never defined. We define a dummy function here. All references to it should be removed. **TPDHERE: This function is used but never defined. Remove it.**

— defun compAndDefine —

```
(defun compAndDefine (arg)
  (declare (ignore arg))
  nil)
```

—————

defun putInLocalDomainReferences

```
[NRTputInTail p189]
[$QuickCode p??]
[$elt p310]
```

— defun putInLocalDomainReferences —

```
(defun |putInLocalDomainReferences| (def)
  (let (|$elt| opName lam var1 body)
    (declare (special |$elt| |$QuickCode|))
    (setq opName (car def))
    (setq lam (caadr def))
    (setq var1 (cadadr def))
    (setq body (car (cddadr def)))
    (setq |$elt| (if |$QuickCode| 'qrefelt 'elt))
    (|NRTputInTail| (cddadr def))
    def))
```

—————

defun NRTputInTail

```
[lassoc p??]
[NRTassocIndex p346]
[rplaca p??]
[NRTputInHead p190]
[$elt p310]
[$devaluateList p??]
```

— defun NRTputInTail —

```

(defun |NRTputInTail| (x)
  (let (u k)
    (declare (special |$elt| |$devalueList|))
    (maplist #'(lambda (y)
      (cond
        ((atom (setq u (car y)))
         (cond
           ((or (eq u '$) (lassoc u |$devalueList|))
            nil)
           ((setq k (|NRTassocIndex| u))
            (cond
              ; u atomic means that the slot will always contain a vector
              ((atom u) (rplaca y (list |$elt| '$ k)))
              ; this reference must check that slot is a vector
              (t (rplaca y (list 'spadcheckelt '$ k))))
            (t nil))))
        (t (|NRTputInHead| u))))
    x)
  x))

```

defun NRTputInHead

[\[NRTputInTail p189\]](#)
[\[NRTassocIndex p346\]](#)
[\[NRTputInHead p190\]](#)
[\[lastnode p??\]](#)
[\[keyedSystemError p??\]](#)
[\[\\$elt p310\]](#)

— defun NRTputInHead —

```

(defun |NRTputInHead| (bod)
  (let (fn clauses dom tmp2 ind k)
    (declare (special |$elt|))
    (cond
      ((atom bod) bod)
      ((and (consp bod) (eq (qcar bod) 'spadcall) (consp (qcdr bod))
        (progn (setq tmp2 (reverse (qcdr bod))) t) (consp tmp2))
       (setq fn (qcar tmp2))
       (|NRTputInTail| (cdr bod))
       (cond
         ((and (consp fn) (consp (qcdr fn)) (consp (qcdr (qcdr fn)))
          (eq (qcdddr fn) nil) (null (eq (qsecond fn) '$))
          (member (qcar fn) '(elt qrefelt const)))
          (when (setq k (|NRTassocIndex| (qsecond fn)))

```

```

      (rplaca (lastnode bod) (list |$elt| '$ k)))
    (t (|NRTputInHead| fn) bod)))
  ((and (consp bod) (eq (qcar bod) 'cond))
    (setq clauses (qcdr bod))
    (loop for cc in clauses do (|NRTputInTail| cc))
    bod)
  ((and (consp bod) (eq (qcar bod) 'quote)) bod)
  ((and (consp bod) (eq (qcar bod) 'closedfn)) bod)
  ((and (consp bod) (eq (qcar bod) 'spadconst) (consp (qcdr bod))
    (consp (qcddr bod)) (eq (qcdddr bod) nil))
    (setq dom (qsecond bod))
    (setq ind (qthird bod))
    (rplaca bod |$elt|)
    (cond
      ((eq dom '$) nil)
      ((setq k (|NRTassocIndex| dom))
        (rplaca (lastnode bod) (list |$elt| '$ k))
        bod)
      (t
        (|keyedSystemError|
          "Unexpected error or improper call to system function %1: %2"
          (list "NRTputInHead" "unexpected SPADCONST form")))))
  (t
    (|NRTputInHead| (car bod))
    (|NRTputInTail| (cdr bod) bod)))

```

defun getArgumentModeOrMoan

[getArgumentMode p309]
 [stackSemanticError p??]

— defun getArgumentModeOrMoan —

```

(defun |getArgumentModeOrMoan| (x form env)
  (or (|getArgumentMode| x env)
    (|stackSemanticError|
      (list '|argument| x '| of | form '| is not declared|) nil)))

```

defun augLisplibModemapsFromCategory

[sublis p??]
 [mkAlistOfExplicitCategoryOps p193]

```

[isCategoryForm p??]
[lassoc p??]
[member p??]
[mkpf p??]
[interactiveModemapForm p195]
[$lisplibModemapAlist p??]
[$EmptyEnvironment p??]
[$domainShell p??]
[$PatternVariableList p??]
[$lisplibModemapAlist p??]

```

— defun augLisplibModemapsFromCategory —

```

(defun |augLisplibModemapsFromCategory| (form body signature)
  (let (argl sl opAlist nonCategorySigAlist domainList catPredList op sig
        pred sel predp modemap)
    (declare (special |$lisplibModemapAlist| |$EmptyEnvironment|
                      |$domainShell| |$PatternVariableList|))
    (setq op (car form))
    (setq argl (cdr form))
    (setq sl
      (cons (cons '$ '*1)
            (loop for a in argl for p in (rest |$PatternVariableList|)
                  collect (cons a p))))
    (setq form (sublis sl form))
    (setq body (sublis sl body))
    (setq signature (sublis sl signature))
    (when (setq opAlist (sublis sl (elt |$domainShell| 1)))
      (setq nonCategorySigAlist
        (|mkAlistOfExplicitCategoryOps| (subst '*1 '$ body :test #'equal)))
      (setq domainList
        (loop for a in (rest form) for m in (rest signature)
              when (|isCategoryForm| m |$EmptyEnvironment|)
                collect (list a m)))
      (setq catPredList
        (loop for u in (cons (list '*1 form) domainList)
              collect (cons '|ofCategory| u)))
      (loop for entry in opAlist
            when (|member| (cadar entry) (lassoc (caar entry) nonCategorySigAlist))
              do
                (setq op (caar entry))
                (setq sig (cadar entry))
                (setq pred (cadr entry))
                (setq sel (caddr entry))
                (setq predp (mkpf (cons pred catPredList) 'and))
                (setq modemap (list (cons '*1 sig) (list predp sel)))
                (setq |$lisplibModemapAlist|
                  (cons (cons op (|interactiveModemapForm| modemap))
                        |$lisplibModemapAlist|))))))

```

defun mkAlistOfExplicitCategoryOps

```

[keyedSystemError p??]
[union p??]
[mkAlistOfExplicitCategoryOps p193]
[flattenSignatureList p194]
[nreverse0 p??]
[remdup p??]
[assocleft p??]
[isCategoryForm p??]
[$e p??]

```

— defun mkAlistOfExplicitCategoryOps —

```

(defun |mkAlistOfExplicitCategoryOps| (target)
  (labels (
    (atomizeOp (op)
      (cond
        ((atom op) op)
        ((and (consp op) (eq (qrest op) nil)) (qfirst op))
        (t (|keyedSystemError|
             "Unexpected error or improper call to system function %1: %2"
             (list "mkAlistOfExplicitCategoryOps" "bad signature")))))
    (fn (op u)
      (if (and (consp u) (consp (qfirst u)))
        (if (equal (qcaar u) op)
          (cons (qcdar u) (fn op (qrest u)))
          (fn op (qrest u)))))
    (let (z tmp1 op sig u opList)
      (declare (special |$e|))
      (when (and (consp target) (eq (qfirst target) '|add|) (consp (qrest target)))
        (setq target (second target)))
      (cond
        ((and (consp target) (eq (qfirst target) '|Join|))
         (setq z (qrest target))
         (PROG (tmp1)
          (RETURN
            (DO ((G167566 z (CDR G167566)) (cat nil))
              ((OR (ATOM G167566) (PROGN (setq cat (CAR G167566)) nil))
               tmp1)
              (setq tmp1 (|union| tmp1 (|mkAlistOfExplicitCategoryOps| cat)))))))
        ((and (consp target) (eq (qfirst target) 'category)
          (progn
            (setq tmp1 (qrest target))

```

```

      (and (consp tmp1)
            (progn (setq z (qrest tmp1)) t))))
    (setq z (|flattenSignatureList| (cons 'progn z)))
    (setq u
      (prog (G167577)
        (return
          (do ((G167583 z (cdr G167583)) (x nil))
              ((or (atom G167583)) (nreverse0 G167577))
            (setq x (car G167583))
            (cond
              ((and (consp x) (eq (qfirst x) 'signature) (consp (qrest x))
                    (consp (qcddr x)))
               (setq op (qsecond x))
               (setq sig (qthird x))
               (setq G167577 (cons (cons (atomizeOp op) sig) G167577)))))))
      (setq opList (remdup (assocleft u)))
      (prog (G167593)
        (return
          (do ((G167598 opList (cdr G167598)) (x nil))
              ((or (atom G167598)) (nreverse0 G167593))
            (setq x (car G167598))
            (setq G167593 (cons (cons x (fn x u)) G167593))))))
    ((|isCategoryForm| target |$e|) nil)
    (t
      (|keyedSystemError|
        "Unexpected error or improper call to system function %1: %2"
        (list "mkAlistOfExplicitCategoryOps" "bad signature"))))

```

defun flattenSignatureList

[flattenSignatureList p194]

— defun flattenSignatureList —

```

(defun |flattenSignatureList| (x)
  (let (zz)
    (cond
      ((atom x) nil)
      ((and (consp x) (eq (qfirst x) 'signature)) (list x))
      ((and (consp x) (eq (qfirst x) 'if) (consp (qrest x))
            (consp (qcddr x)) (consp (qcdddr x))
            (eq (qcdddr x) nil))
       (append (|flattenSignatureList| (third x))
                (|flattenSignatureList| (fourth x))))
      ((and (consp x) (eq (qfirst x) 'progn))
       (loop for x in (qrest x)

```

```

do
  (if (and (consp x) (eq (qfirst x) 'signature))
      (setq zz (cons x zz))
      (setq zz (append (|flattenSignatureList| x) zz)))
  zz)
(t nil)))

```

defun interactiveModemapForm

Create modemap form for use by the interpreter. This function replaces all specific domains mentioned in the modemap with pattern variables, and predicates [replaceVars p196]

[modemapPattern p203]

[substVars p202]

[fixUpPredicate p196]

[\$PatternVariableList p??]

[\$FormalMapVariableList p270]

— defun interactiveModemapForm —

```

(defun |interactiveModemapForm| (mm)
  (labels (
    (fn (x)
      (if (and (consp x) (consp (qrest x))
              (consp (qcddr x)) (eq (qcddr x) nil)
              (not (eq (qfirst x) '|isFreeFunction|))
              (atom (qthird x)))
          (list (first x) (second x) (list (third x)))
          x)))
    (let (pattern dc sig mmpat patternAlist partial patvars
          domainPredicateList tmp1 pred dependList cond)
      (declare (special |$PatternVariableList| |$FormalMapVariableList|))
      (setq mm
        (|replaceVars| (copy mm) |$PatternVariableList| |$FormalMapVariableList|))
      (setq pattern (car mm))
      (setq dc (caar mm))
      (setq sig (cdar mm))
      (setq pred (cadr mm))
      (setq pred
        (prog ()
          (return
            (do ((x pred (cdr x)) (result nil))
              ((atom x) (nreverse0 result))
              (setq result (cons (fn (car x)) result))))))
      (setq tmp1 (|modemapPattern| pattern sig))
      (setq mmpat (car tmp1))

```

```

(setq patternAlist (cadr tmp1))
(setq partial (caddr tmp1))
(setq patvars (caddr tmp1))
(setq tmp1 (|substVars| pred patternAlist patvars))
(setq pred (car tmp1))
(setq domainPredicateList (cadr tmp1))
(setq tmp1 (|fixUpPredicate| pred domainPredicateList partial (cdr mmpat)))
(setq pred (car tmp1))
(setq dependList (cdr tmp1))
(setq cond (car pred))
(list mmpat cond)))

```

defun replaceVars

Replace every identifier in oldvars with the corresponding identifier in newvars in the expression x

— defun replaceVars —

```

(defun |replaceVars| (x oldvars newvars)
  (loop for old in oldvars for new in newvars
    do (setq x (subst new old x :test #'equal)))
  x)

```

defun fixUpPredicate

[length p??]
 [orderPredicateItems p[197](#)]
 [moveORsOutside p[201](#)]

— defun fixUpPredicate —

```

(defun |fixUpPredicate| (predClause domainPreds partial sig)
  (let (predicate fn skip predicates tmp1 dependList pred)
    (setq predicate (car predClause))
    (setq fn (cadr predClause))
    (setq skip (caddr predClause))
    (cond
      ((eq (car predicate) 'and)
       (setq predicates (append domainPreds (cdr predicate))))
      ((not (equal predicate (mkq t)))
       (setq predicates (cons predicate domainPreds)))
    )
  )

```



```

(t
  (setq predicates (or domainPreds (list predicate))))
(cond
  ((> (|#| predicates) 1)
    (setq pred (cons 'and predicates))
    (setq tmp1 (|orderPredicateItems| pred sig skip))
    (setq pred (car tmp1))
    (setq dependList (cdr tmp1))
    tmp1)
  (t
    (setq pred (|orderPredicateItems| (car predicates) sig skip))
    (setq dependList
      (when (and (consp pred) (eq (qfirst pred) 'isDomain|)
        (consp (qrest pred)) (consp (qcddr pred))
        (eq (qcddr pred) nil)
        (consp (qthird pred))
        (eq (qcdaddr pred) nil))
      (list (second pred)))))
    (setq pred (|moveORsOutside| pred))
    (when partial (setq pred (cons '|partial| pred)))
    (cons (cons pred (cons fn skip)) dependList)))

```

defun orderPredicateItems

[signatureTran p197]
 [orderPredTran p198]

— defun orderPredicateItems —

```

(defun |orderPredicateItems| (pred1 sig skip)
  (let (pred)
    (setq pred (|signatureTran| pred1))
    (if (and (consp pred) (eq (qfirst pred) 'and))
      (|orderPredTran| (qrest pred) sig skip
        pred)))

```

defun signatureTran

[signatureTran p197]
 [isCategoryForm p??]
 [\$e p??]

— defun signatureTran —

```
(defun |signatureTran| (pred)
  (declare (special |$e|))
  (cond
    ((atom pred) pred)
    ((and (consp pred) (eq (qfirst pred) '|has|) (CONSP (qrest pred))
      (consp (qcddr pred))
      (eq (qcdddr pred) nil)
      (|isCategoryForm| (third pred) |$e|))
      (list '|ofCategory| (second pred) (third pred))))
  (t
    (loop for p in pred
      collect (|signatureTran| p)))))
```

—————

defun orderPredTran

```
[member p??]
[delete p??]
[unionq p??]
[listOfPatternIds p??]
[intersectionq p??]
[setdifference p??]
[insertWOC p??]
[isDomainSubst p200]
```

— defun orderPredTran —

```
(defun |orderPredTran| (oldList sig skip)
  (let (lastDependList somethingDone lastPreds indepv1 depv1 dependList
        noldList x ids fullDependList newList answer)
    ; --(1) make two kinds of predicates appear last:
    ; ----- (op *target ..) when *target does not appear later in sig
    ; ----- (isDomain *1 ..)
    (SEQ
      (loop for pred in oldList
        do (cond
          ((or (and (consp pred) (consp (qrest pred))
            (consp (qcddr pred))
            (eq (qcdddr pred) nil)
            (member (qfirst pred) '|isDomain| |ofCategory|))
            (equal (qsecond pred) (car sig))
            (null (|member| (qsecond pred) (cdr sig))))
            (and (null skip) (consp pred) (eq (qfirst pred) '|isDomain|)
```

```

      (consp (qrest pred)) (consp (qcddr pred))
      (eq (qcddr pred) nil)
      (equal (qsecond pred) '*1)))
    (setq oldList (|delete| pred oldList))
    (setq lastPreds (cons pred lastPreds))))
; --(2a) lastDependList=list of all variables that lastPred forms depend upon
(setq lastDependList
  (let (result)
    (loop for x in lastPreds
      do (setq result (unionq result (|listOfPatternIds| x))))
    result))
; --(2b) dependList=list of all variables that isDom/ofCat forms depend upon
(setq dependList
  (let (result)
    (loop for x in oldList
      do (when
          (and (consp x)
               (or (eq (qfirst x) '|isDomain|) (eq (qfirst x) '|ofCategory|))
               (consp (qrest x)) (consp (qcddr x))
               (eq (qcddr x) nil))
            (setq result (unionq result (|listOfPatternIds| (third x))))))
    result))
; --(3a) newList= list of ofCat/isDom entries that don't depend on
(loop for x in oldList
  do
    (cond
      ((and (consp x)
            (or (eq (qfirst x) '|ofCategory|) (eq (qfirst x) '|isDomain|))
            (consp (qrest x)) (consp (qcddr x))
            (eq (qcddr x) nil))
          (setq indepvl (|listOfPatternIds| (second x)))
          (setq depvl (|listOfPatternIds| (third x)))
          (t
           (setq indepvl (|listOfPatternIds| x))
           (setq depvl nil)))
      (when
        (and (null (intersectionq indepvl dependList))
              (intersectionq indepvl lastDependList))
          (setq somethingDone t)
          (setq lastPreds (append lastPreds (list x)))
          (setq oldList (|delete| x oldList))))))
; --(3b) newList= list of ofCat/isDom entries that don't depend on
(loop while oldList do
  (loop for x in oldList do
    (cond
      ((and (consp x)
            (or (eq (qfirst x) '|ofCategory|) (eq (qfirst x) '|isDomain|))
            (consp (qrest x))
            (consp (qcddr x)) (eq (qcddr x) nil))
          (setq indepvl (|listOfPatternIds| (second x))

```

```

      (setq depv1 (|listOfPatternIds| (third x)))
    (t
      (setq indepv1 (|listOfPatternIds| x))
      (setq depv1 nil)))
    (when (null (intersectionq indepv1 dependList))
      (setq dependList (SETDIFFERENCE dependList depv1))
      (setq newList (APPEND newList (list x))))))
; --(4) noldList= what is left over
(cond
  ((equal (setq noldList (setdifference oldList newList)) oldList)
    (setq newList (APPEND newList oldList))
    (return nil))
  (t
    (setq oldList noldList)))
(loop for pred in newList do
  (when
    (and (consp pred)
      (or (eq (qfirst pred) '|isDomain|) (eq (qfirst x) '|ofCategory|))
      (consp (qrest pred))
      (consp (qcddr pred))
      (eq (qcdddr pred) nil))
    (setq ids (|listOfPatternIds| (third pred)))
    (when
      (let (result)
        (loop for id in ids do
          (setq result (and result (|member| id fullDependList))))
        result)
      (setq fullDependList (|insertWOC| (second pred) fullDependList)))
    (setq fullDependList (unionq fullDependList ids)))
  (setq newList (append newList lastPreds))
  (setq newList (|isDomainSubst| newList))
  (setq answer
    (cons (cons 'and newList) (intersectionq fullDependList sig))))))

```

defun isDomainSubst

— defun isDomainSubst —

```

(defun |isDomainSubst| (u)
  (labels (
    (findSub (x alist)
      (cond
        ((null alist) nil)
        ((and (consp alist) (consp (qfirst alist))
          (eq (qcaar alist) '|isDomain|)

```

```

      (consp (qcdar alist))
      (consp (qcddar alist))
      (eq (qcdddar alist) nil)
      (equal x (cadar alist)))
      (caddar alist))
    (t (findSub x (cdr alist))))))
(fn (x alist)
  (let (s)
    (declare (special |$PatternVariableList|))
    (if (atom x)
      (if
        (and (identp x)
              (member x |$PatternVariableList|)
              (setq s (findSub x alist)))
          s
          x)
        (cons (car x)
              (loop for y in (cdr x)
                    collect (fn y alist)))))))
  (let (head tail nhead)
    (if (consp u)
      (progn
        (setq head (qfirst u))
        (setq tail (qrest u))
        (setq nhead
          (cond
            ((and (consp head) (eq (qfirst head) '|isDomain|)
                  (consp (qrest head)) (consp (qcddr head))
                  (eq (qcdddr head) nil))
              (list '|isDomain| (second head)
                    (fn (third head) tail)))
            (t head)))
          (cons nhead (|isDomainSubst| (cdr u))))
      u))))

```

defun moveORsOutside

[moveORsOutside p201]

— defun moveORsOutside —

```

(defun |moveORsOutside| (p)
  (let (q x)
    (cond
      ((and (consp p) (eq (qfirst p) 'and))
        (setq q

```

```

(prog (G167169)
  (return
    (do ((G167174 (cdr p) (cdr G167174)) (|r| nil))
      ((or (atom G167174)) (nreverse0 G167169))
      (setq |r| (CAR G167174))
      (setq G167169 (cons (|moveORsOutside| |r|) G167169))))))
(cond
  ((setq x
    (let (tmp1)
      (loop for r in q
        when (and (consp r) (eq (qfirst r) 'or))
        do (setq tmp1 (or tmp1 r)))
      tmp1))
    (|moveORsOutside|
      (cons 'or
        (let (tmp1)
          (loop for tt in (cdr x)
            do (setq tmp1 (cons (cons 'and (subst tt x q :test #'equal)) tmp1)))
          (nreverse0 tmp1)))))
    (t (cons 'and q))))
(t p))))

(defun |moveORsOutside| (p)
  ; (let (q s x tmp1)
  ; (cond
  ; ((and (consp p) (eq (qfirst p) 'and))
  ;   (setq q (loop for r in (qrest p) collect (|moveORsOutside| r)))
  ;   (setq tmp1
  ;     (loop for r in q
  ;       when (and (consp r) (eq (qrest r) 'or))
  ;       collect r))
  ;   (setq x (mapcar #'(lambda (a b) (or a b)) tmp1))
  ;   (if x
  ;     (|moveORsOutside|
  ;       (cons 'or
  ;         (loop for tt in (cdr x)
  ;           collect (cons 'and (subst tt x q :test #'equal)))))
  ;     (cons 'and q)))
  ;   ('t p))))

```

defun substVars

Make pattern variable substitutions. [nsubst p??]

[contained p??]

[\$FormalMapVariableList p270]

— defun substVars —

```
(defun |substVars| (pred patternAlist patternVarList)
  (let (patVar value everything replacementVar domainPredicates)
    (declare (special |$FormalMapVariableList|))
    (setq domainPredicates NIL)
    (maplist
      #'(lambda (x)
        (setq patVar (caar x))
        (setq value (cdar x))
        (setq pred (subst patVar value pred :test #'equal))
        (setq patternAlist (|subst| patVar value patternAlist))
        (setq domainPredicates
          (subst patVar value domainPredicates :test #'equal))
        (unless (member value |$FormalMapVariableList|)
          (setq domainPredicates
            (cons (list '|isDomain| patVar value) domainPredicates))))
      patternAlist)
    (setq everything (list pred patternAlist domainPredicates))
    (dolist (var |$FormalMapVariableList|)
      (cond
        ((contained var everything)
         (setq replacementVar (car patternVarList))
         (setq patternVarList (cdr patternVarList))
         (setq pred (subst replacementVar var pred :test #'equal))
         (setq domainPredicates
           (subst replacementVar var domainPredicates :test #'equal))))
      (list pred domainPredicates)))
```

—————

defun modemapPattern

```
[rassoc p??]
|$PatternVariableList p??]
```

— defun modemapPattern —

```
(defun |modemapPattern| (mmPattern sig)
  (let (partial patvar patvars mmpat patternAlist)
    (declare (special |$PatternVariableList|))
    (setq patternAlist nil)
    (setq mmpat nil)
    (setq patvars |$PatternVariableList|)
    (setq partial nil)
    (maplist
```

```

#' (lambda (xTails)
  (let ((x (car xTails)))
    (when (and (consp x) (eq (qfirst x) '|Union|)
              (consp (qrest x)) (consp (qcddr x))
              (eq (qcdddr x) nil)
              (equal (third x) "failed")
              (equal xTails sig))
      (setq x (second x))
      (setq partial t))
    (setq patvar (|rassoc| x patternAlist))
    (cond
     ((null (null patvar))
      (setq mmpat (cons patvar mmpat)))
     (t
      (setq patvar (car patvars))
      (setq patvars (cdr patvars))
      (setq mmpat (cons patvar mmpat))
      (setq patternAlist (cons (cons patvar x) patternAlist))))))
  mmPattern)
(list (nreverse mmpat) patternAlist partial patvars)))

```

defun evalAndRwriteLispForm

```

[eval p??]
[rwriteLispForm p204]

```

— defun evalAndRwriteLispForm —

```

(defun |evalAndRwriteLispForm| (key form)
  (|eval| form)
  (|rwriteLispForm| key form))

```

defun rwriteLispForm

```

[$libFile p??]
[$lisplib p??]

```

— defun rwriteLispForm —

```

(defun |rwriteLispForm| (key form)
  (declare (special |$libFile| $lisplib))

```



```
(when $lisplib
  (|rwrite| key form |$libFile|)
  (|LAM,FILEACTQ| key form)))
```

defun mkConstructor

[mkConstructor p205]

— defun mkConstructor —

```
(defun |mkConstructor| (form)
  (cond
    ((atom form) (list '|devaluate| form))
    ((null (rest form)) (list 'quote (list (first form))))
    (t
     (cons 'list
           (cons (mkq (first form))
                 (loop for x in (rest form) collect (|mkConstructor| x)))))))
```

defun unloadOneConstructor

[remprop p??]
[mkAutoLoad p??]

— defun unloadOneConstructor —

```
(defun |unloadOneConstructor| (cnam fn)
  (remprop cnam 'loaded)
  (setf (symbol-function cnam) (|mkAutoLoad| fn cnam)))
```

defun lisplibDoRename

[replaceFile p??]
[\$spadLibFT p??]

— defun lisplibDoRename —

```
(defun |lisplibDoRename| (libName)
  (declare (special |$spadLibFT|))
  (replaceFile (list libName |$spadLibFT| 'a) (list libName 'errorlib 'a)))
```

defun initializeLisplib

```
[erase p??]
[writeLib1 p207]
[addoptions p??]
[pathnameTypeId p??]
[LAM,FILEACTQ p??]
[$erase p??]
[$libFile p??]
[$libFile p??]
[$lisplibForm p??]
[$lisplibModemap p??]
[$lisplibKind p??]
[$lisplibModemapAlist p??]
[$lisplibAbbreviation p??]
[$lisplibAncestors p??]
[$lisplibOpAlist p??]
[$lisplibOperationAlist p??]
[$lisplibSuperDomain p??]
[$lisplibVariableAlist p??]
[$lisplibSignatureAlist p??]
[/editfile p??]
[/major-version p??]
[errors p??]
```

— defun initializeLisplib —

```
(defun |initializeLisplib| (libName)
  (declare (special $erase |$libFile| |$lisplibForm|
    |$lisplibModemap| |$lisplibKind| |$lisplibModemapAlist|
    |$lisplibAbbreviation| |$lisplibAncestors|
    |$lisplibOpAlist| |$lisplibOperationAlist|
    |$lisplibSuperDomain| |$lisplibVariableAlist| errors
    |$lisplibSignatureAlist| /editfile /major-version errors))
  ($erase libName 'errorlib 'a)
  (setq errors 0)
  (setq |$libFile| (|writeLib1| libname 'errorlib 'a))
  (addoptions 'file |$libFile|)
  (setq |$lisplibForm| nil)
  (setq |$lisplibModemap| nil)
```

```

(setq |$lisplibKind| nil)
(setq |$lisplibModemapAlist| nil)
(setq |$lisplibAbbreviation| nil)
(setq |$lisplibAncestors| nil)
(setq |$lisplibOpAlist| nil)
(setq |$lisplibOperationAlist| nil)
(setq |$lisplibSuperDomain| nil)
(setq |$lisplibVariableAlist| nil)
(setq |$lisplibSignatureAlist| nil)
(when (eq (|pathnameTypeId| /editfile) 'spad)
  (|LAM,FILEACTQ| 'version (list '/versioncheck /major-version))))

```

defun writeLib1

[rdefiostream p??]

— defun writeLib1 —

```

(defun |writeLib1| (fn ft fm)
  (rdefiostream (cons (list 'file fn ft fm) (list '(mode . output))))))

```

defun finalizeLisplib

[lisplibWrite p214]
 [removeZeroOne p??]
 [namestring p??]
 [getConstructorOpsAndAtts p209]
 [NRTgenInitialAttributeAlist p??]
 [mergeSignatureAndLocalVarAlists p213]
 [finalizeDocumentation p490]
 [profileWrite p??]
 [sayMSG p??]
 [\$lisplibForm p??]
 [\$libFile p??]
 [\$lisplibKind p??]
 [\$lisplibModemap p??]
 [\$lisplibCategory p??]
 [\$/editfile p??]
 [\$lisplibModemapAlist p??]
 [\$lisplibForm p??]

```

[$lisplibModemap p??]
[$FormalMapVariableList p270]
[$lisplibSuperDomain p??]
[$lisplibSignatureAlist p??]
[$lisplibVariableAlist p??]
[$lisplibAttributes p??]
[$lisplibPredicates p??]
[$lisplibAbbreviation p??]
[$lisplibParents p??]
[$lisplibAncestors p??]
[$lisplibSlot1 p??]
[$profileCompiler p??]
[$spadLibFT p??]
[$lisplibCategory p??]
[$pairlis p??]
[$NRTslot1PredicateList p??]

```

— **defun finalizeLisplib** —

```

(defun |finalizeLisplib| (libName)
  (let (|$pairlis| |$NRTslot1PredicateList| kind opsAndAtts)
    (declare (special |$pairlis| |$NRTslot1PredicateList| |$spadLibFT|
                |$lisplibForm| |$profileCompiler| |$libFile|
                |$lisplibSlot1| |$lisplibAncestors| |$lisplibParents|
                |$lisplibAbbreviation| |$lisplibPredicates|
                |$lisplibAttributes| |$lisplibVariableAlist|
                |$lisplibSignatureAlist| |$lisplibSuperDomain|
                |$FormalMapVariableList| |$lisplibModemap|
                |$lisplibModemapAlist| /editfile |$lisplibCategory|
                |$lisplibKind| errors))
      (|lisplibWrite| "constructorForm"
        (|removeZeroOne| |$lisplibForm|) |$libFile|)
      (|lisplibWrite| "constructorKind"
        (setq kind (|removeZeroOne| |$lisplibKind|)) |$libFile|)
      (|lisplibWrite| "constructorModemap"
        (|removeZeroOne| |$lisplibModemap|) |$libFile|)
      (setq |$lisplibCategory| (or |$lisplibCategory| (cadar |$lisplibModemap|)))
      (|lisplibWrite| "constructorCategory" |$lisplibCategory| |$libFile|)
      (|lisplibWrite| "sourceFile" (|namestring| /editfile) |$libFile|)
      (|lisplibWrite| "modemaps"
        (|removeZeroOne| |$lisplibModemapAlist|) |$libFile|)
      (setq opsAndAtts
        (|getConstructorOpsAndAtts| |$lisplibForm| kind |$lisplibModemap|))
      (|lisplibWrite| "operationAlist"
        (|removeZeroOne| (car opsAndAtts)) |$libFile|)
      (when (eq kind '|category|)
        (setq |$pairlis|
          (loop for a in (rest |$lisplibForm|)

```

```

        for v in |$FormalMapVariableList|
        collect (cons a v)))
    (setq |$NRTslot1PredicateList| nil)
    (|NRTgenInitialAttributeAlist| (cdr opsAndAtts)))
(|lisplibWrite| "superDomain"
  (|removeZeroOne| |$lisplibSuperDomain|) |$libFile|)
(|lisplibWrite| "signaturesAndLocals"
  (|removeZeroOne|
    (|mergeSignatureAndLocalVarAlists| |$lisplibSignatureAlist|
                                          |$lisplibVariableAlist|))
    |$libFile|)
(|lisplibWrite| "attributes"
  (|removeZeroOne| |$lisplibAttributes|) |$libFile|)
(|lisplibWrite| "predicates"
  (|removeZeroOne| |$lisplibPredicates|) |$libFile|)
(|lisplibWrite| "abbreviation" |$lisplibAbbreviation| |$libFile|)
(|lisplibWrite| "parents" (|removeZeroOne| |$lisplibParents|) |$libFile|)
(|lisplibWrite| "ancestors" (|removeZeroOne| |$lisplibAncestors|) |$libFile|)
(|lisplibWrite| "documentation" (|finalizeDocumentation|) |$libFile|)
(|lisplibWrite| "slot1Info" (|removeZeroOne| |$lisplibSlot1|) |$libFile|)
(when |$profileCompiler| (|profileWrite|))
(when (and |$lisplibForm| (null (cdr |$lisplibForm|)))
  (setf (get (car |$lisplibForm|) 'niladic) t))
(unless (eql errors 0)
  (|sayMSG| (list "    Errors in processing " kind " " libName ":")
    (|sayMSG| (list "      not replacing " |$spadLibFT| " for" libName)))))

```

defun getConstructorOpsAndAtts

[getCategoryOpsAndAtts p210]
 [getFunctorOpsAndAtts p212]

— defun getConstructorOpsAndAtts —

```

(defun |getConstructorOpsAndAtts| (form kind modemap)
  (if (eq kind '|category|)
    (|getCategoryOpsAndAtts| form)
    (|getFunctorOpsAndAtts| form modemap)))

```

defun getCategoryOpsAndAtts

```
[transformOperationAlist p210]
[getSlotFromCategoryForm p210]
[getSlotFromCategoryForm p210]
```

— defun getCategoryOpsAndAtts —

```
(defun |getCategoryOpsAndAtts| (catForm)
  (cons (|transformOperationAlist| (|getSlotFromCategoryForm| catForm 1))
        (|getSlotFromCategoryForm| catForm 2)))
```

—————

defun getSlotFromCategoryForm

```
[eval p??]
[take p??]
[systemErrorHere p??]
[$FormalMapVariableList p270]
```

— defun getSlotFromCategoryForm —

```
(defun |getSlotFromCategoryForm| (opargs index)
  (let (op arg1 u)
    (declare (special |$FormalMapVariableList|))
    (setq op (first opargs))
    (setq arg1 (rest opargs))
    (setq u
      (|eval| (cons op (mapcar 'mkq (take (|#| arg1) |$FormalMapVariableList|)))))
    (if (null (vecp u))
        (|systemErrorHere| "getSlotFromCategoryForm")
        (elt u index))))
```

—————

defun transformOperationAlist

This transforms the operationAlist which is written out onto LISPLIBs. The original form of this list is a list of items of the form:

```
((<op> <signature>) (<condition> (ELT $ n)))
```

The new form is an op-Alist which has entries

```
(<op> . signature-Alist)
```

where signature-Alist has entries

```
(<signature> . item)
```

where item has form

```
(<slotNumber> <condition> <kind>)
```

```
where <kind> =
  NIL => function
  CONST => constant ... and others
```

```
[member p??]
[keyedSystemError p??]
[assoc p??]
[lassq p??]
[insertAlist p??]
[$functionLocations p??]
```

— defun transformOperationAlist —

```
(defun |transformOperationAlist| (operationAlist)
  (let (op sig condition implementation eltEtc impOp kind u n signatureItem
        itemList newAlist)
    (declare (special |$functionLocations|))
    (setq newAlist nil)
    (dolist (item operationAlist)
      (setq op (caar item))
      (setq sig (cadar item))
      (setq condition (cadr item))
      (setq implementation (caddr item))
      (setq kind
        (cond
          ((and (consp implementation) (consp (qrest implementation))
                (consp (qcddr implementation))
                (eq (qcdddr implementation) nil)
                (progn (setq n (qthird implementation)) t)
                (|member| (setq eltEtc (qfirst implementation)) '(const elt)))
            eltEtc)
          ((consp implementation)
            (setq impOp (qfirst implementation))
            (cond
              ((eq impOp 'xlam) implementation)
              ((|member| impOp '(const |Subsumed|)) impOp)
              (t (|keyedSystemError| "Unexpected type of entry in domain: %1s"
                (list impOp)))))))
      (push item newAlist)))
```

```

    ((eq implementation '|mkRecord|) '|mkRecord|)
    (t (|keyedSystemError| "Unexpected type of entry in domain: %1s"
        (list implementation))))
    (when (setq u (|assoc| (list op sig) |$functionLocations|))
      (setq n (cons n (cdr u))))
    (setq signatureItem
      (if (eq kind 'elt)
        (if (eq condition t)
          (list sig n)
          (list sig n condition))
        (list sig n condition kind)))
    (setq itemList (cons signatureItem (lassq op newAlist)))
    (setq newAlist (|insertAlist| op itemList newAlist))
    newAlist))

```

defun getFunctorOpsAndAtts

[transformOperationAlist p210]
 [getSlotFromFunctor p212]

— defun getFunctorOpsAndAtts —

```

(defun |getFunctorOpsAndAtts| (form modemap)
  (cons (|transformOperationAlist| (|getSlotFromFunctor| form 1 modemap))
        (|getSlotFromFunctor| form 2 modemap)))

```

defun getSlotFromFunctor

[compMakeCategoryObject p213]
 [systemErrorHere p??]
 [\$e p??]
 [\$lisplibOperationAlist p??]

— defun getSlotFromFunctor —

```

(defun |getSlotFromFunctor| (arg1 slot arg2)
  (declare (ignore arg1))
  (let (tt)
    (declare (special |$e| |$lisplibOperationAlist|))
    (cond
      ((eq slot 1) |$lisplibOperationAlist|)

```



```
(t
  (setq tt (or (|compMakeCategoryObject| (cadar arg2) |$e|)
               (|systemErrorHere| "getSlotFromFunctor")))
  (elt (car tt) slot))))
```

defun compMakeCategoryObject

```
[isCategoryForm p??]
[mkEvaluableCategoryForm p182]
[$e p??]
[$Category p??]
```

— defun compMakeCategoryObject —

```
(defun |compMakeCategoryObject| (c |$e|)
  (declare (special |$e|))
  (let (u)
    (declare (special |$Category|))
    (cond
      ((null (|isCategoryForm| c |$e|)) nil)
      ((setq u (|mkEvaluableCategoryForm| c)) (list (|eval| u) |$Category| |$e|))
      (t nil))))
```

defun mergeSignatureAndLocalVarAlists

```
[lassoc p??]
```

— defun mergeSignatureAndLocalVarAlists —

```
(defun |mergeSignatureAndLocalVarAlists| (signatureAlist localVarAlist)
  (loop for item in signatureAlist
    collect
      (cons (first item)
            (cons (rest item)
                  (lassoc (first item) localVarAlist)))))
```

defun lisplibWrite

```
[rwrite128 p??]
[$lisplib p??]
```

— defun lisplibWrite —

```
(defun |lisplibWrite| (prop val filename)
  (declare (special $lisplib))
  (when $lisplib (|rwrite| prop val filename)))
```

—————

defun isCategoryPackageName

```
[pname p??]
[maxindex p??]
[char p??]
```

— defun isCategoryPackageName —

```
(defun |isCategoryPackageName| (nam)
  (let (p)
    (setq p (pname (|op0f| nam)))
    (equal (elt p (maxindex p)) #\&)))
```

—————

defun NRTgetLookupFunction

Compute the lookup function (complete or incomplete) [sublis p??]

```
[NRTextendsCategory1 p??]
[getExportCategory p??]
[sayBrightly p??]
[sayBrightlyNT p??]
[bright p??]
[form2String p??]
[$why p??]
[$why p??]
[$pairlis p??]
```

— defun NRTgetLookupFunction —

```
(defun |NRTgetLookupFunction| (domform exCategory addForm)
```

```

(let (|$why| extends u msg v)
(declare (special |$why| |$pairlis|))
(setq domform (sublis |$pairlis| domform))
(setq addForm (sublis |$pairlis| addForm))
(setq |$why| nil)
(cond
  ((atom addForm) '|lookupComplete|)
  (t
   (setq extends
    (|NRTextendsCategory1| domform exCategory (|getExportCategory| addForm)))
   (cond
    ((null extends)
     (setq u (car |$why|))
     (setq msg (cadr |$why|))
     (setq v (caddr |$why|))
     (|sayBrightly|
      "-----non extending category-----")
     (|sayBrightlyNT|
      (cons ".."
       (append (|bright| (|form2String| domform)) (list '|of cat |))))
     (print u)
     (|sayBrightlyNT| (|bright| msg))
     (if v (print (car v)) (terpri))))
    (if extends
     '|lookupIncomplete|
     '|lookupComplete|))))))

```

defun NRTgetLocalIndex

[\[NRTassocIndex p346\]](#)
[\[NRTaddInner p??\]](#)
[\[compOrCroak p586\]](#)
[\[rplaca p??\]](#)
[\[\\$NRTaddForm p??\]](#)
[\[\\$formalArgList p??\]](#)
[\[\\$NRTdeltaList p??\]](#)
[\[\\$NRTdeltaListComp p??\]](#)
[\[\\$NRTdeltaLength p??\]](#)
[\[\\$NRTbase p??\]](#)
[\[\\$EmptyMode p176\]](#)
[\[\\$e p??\]](#)

— defun NRTgetLocalIndex —

```
(defun |NRTgetLocalIndex| (item)
```

```

(let (k value saveNRTdeltaListComp saveIndex compEntry)
  (declare (special |$e| |$EmptyModel| |$NRTdeltaLength| |$NRTbase|
                    |$NRTdeltaListComp| |$NRTdeltaList| |$formalArgList|
                    |$NRTaddForm|))
  (cond
    ((setq k (|NRTassocIndex| item)) k)
    ((equal item |$NRTaddForm|) 5)
    ((eq item '$) 0)
    ((eq item '$$) 2)
    (t
     (when (member item |$formalArgList|) (setq value item))
     (cond
       ((and (atom item) (null (member item '($ $$))) (null value))
        (setq |$NRTdeltaList|
              (cons (cons '|domain| (cons (|NRTaddInner| item) value))
                    |$NRTdeltaList|))
        (setq |$NRTdeltaListComp| (cons item |$NRTdeltaListComp|))
        (setq |$NRTdeltaLength| (1+ |$NRTdeltaLength|))
        (1- (+ |$NRTbase| |$NRTdeltaLength|)))
       (t
        (setq |$NRTdeltaList|
              (cons (cons '|domain| (cons (|NRTaddInner| item) value))
                    |$NRTdeltaList|))
        (setq saveNRTdeltaListComp
              (setq |$NRTdeltaListComp| (cons nil |$NRTdeltaListComp|)))
        (setq saveIndex (+ |$NRTbase| |$NRTdeltaLength|))
        (setq |$NRTdeltaLength| (1+ |$NRTdeltaLength|))
        (setq compEntry (car (|compOrCroak| item |$EmptyModel| |$e|)))
        (rplaca saveNRTdeltaListComp compEntry)
        saveIndex))))))

```

defun augmentLisplibModemapsFromFunctor

```

[formal2Pattern p218]
[mkAlistOfExplicitCategoryOps p193]
[allLASSOCs p218]
[member p??]
[mkDatabasePred p218]
[mkpf p??]
[listOfPatternIds p??]
[interactiveModemapForm p195]
[$lisplibModemapAlist p??]
[$PatternVariableList p??]
[$e p??]
[$lisplibModemapAlist p??]

```

[*\$e p??*]

— `defun augmentLisplibModemapsFromFunctor` —

```
(defun |augmentLisplibModemapsFromFunctor| (form opAlist signature)
  (let (argl nonCategorySigAlist op pred sel predList sig predp z skip modemap)
    (declare (special |$lisplibModemapAlist| |$PatternVariableList| |$e|))
    (setq form (|formal2Pattern| form))
    (setq argl (cdr form))
    (setq opAlist (|formal2Pattern| opAlist))
    (setq signature (|formal2Pattern| signature))
    ; We are going to be EVALing categories containing these pattern variables
    (loop for u in form for v in signature
      do (when (member u |$PatternVariableList|)
          (setq |$e| (|put| u 'mode| v |$e|))))
    (when
      (setq nonCategorySigAlist (|mkAlistOfExplicitCategoryOps| (CAR signature)))
      (loop for entry in opAlist
        do
          (setq op (caar entry))
          (setq sig (cadar entry))
          (setq pred (cadr entry))
          (setq sel (caddr entry))
          (when
            (let (result)
              (loop for catSig in (|allLASSOCs| op nonCategorySigAlist)
                do (setq result (or result (|member| sig catSig))))
              result)
            (setq skip (when (and argl (contained '$ (cdr sig))) 'skip))
            (setq sel (subst form '$ sel :test #'equal))
            (setq predList
              (loop for a in argl for m in (rest signature)
                when (|member| a |$PatternVariableList|)
                collect (list a m)))
            (setq sig (subst form '$ sig :test #'equal))
            (setq predp
              (mkpf
                (cons pred (loop for y in predList collect (|mkDatabasePred| y)))
                'and))
            (setq z (|listOfPatternIds| predList))
            (when (some #'(lambda (u) (null (member u z))) argl)
              (|sayMSG| (list "cannot handle modemap for " op "by pattern match"))
              (setq skip 'skip))
            (setq modemap (list (cons form sig) (cons predp (cons sel skip))))
            (setq |$lisplibModemapAlist|
              (cons
                (cons op (|interactiveModemapForm| modemap))
                |$lisplibModemapAlist|))))))
```

defun allLASSOCs

— defun allLASSOCs —

```
(defun |allLASSOCs| (op alist)
  (loop for value in alist
        when (equal (car value) op)
        collect value))
```

defun formal2Pattern

```
[sublis p??]
[pairList p??]
[$PatternVariableList p??]
```

— defun formal2Pattern —

```
(defun |formal2Pattern| (x)
  (declare (special |$PatternVariableList|))
  (sublis (pairList |$FormalMapVariableList| (cdr |$PatternVariableList|)) x))
```

defun mkDatabasePred

```
[isCategoryForm p??]
[$e p??]
```

— defun mkDatabasePred —

```
(defun |mkDatabasePred| (arg)
  (let (a z)
    (declare (special |$e|))
    (setq a (car arg))
    (setq z (cadr arg))
    (if (|isCategoryForm| z |$e|)
        (list 'ofCategory| a z)
        (list 'ofType| a z))))
```

defun disallowNilAttribute

— defun disallowNilAttribute —

```
(defun |disallowNilAttribute| (x)
  (loop for y in x when (and (car y) (not (eq (car y) '|nil|)))
    collect y))
```

defun bootStrapError

```
[mkq p??]
[namestring p??]
[mkDomainConstructor p??]
```

— defun bootStrapError —

```
(defun |bootStrapError| (functorForm sourceFile)
  (list 'cond
    (list '|$bootStrapMode|
      (list 'vector (|mkDomainConstructor| functorForm) nil nil nil nil nil))
    (list ''t
      (list '|systemError|
        (list 'list (MKQ (CAR functorForm)) "from"
          (mkq (|namestring| sourceFile)) "needs to be compiled")))))
```

defun reportOnFunctorCompilation

```
[displayMissingFunctions p220]
[sayBrightly p??]
[displaySemanticErrors p??]
[displayWarnings p??]
[addStats p??]
[normalizeStatAndStringify p??]
[$op p??]
[$functorStats p??]
[$functionStats p??]
```

```
[$warningStack p??]
[$semanticErrorStack p??]
```

— **defun reportOnFunctorCompilation** —

```
(defun |reportOnFunctorCompilation| ()
  (declare (special |$op| |$functorStats| |$functionStats|
                    |$warningStack| |$semanticErrorStack|))
  (|displayMissingFunctions|)
  (when |$semanticErrorStack| (|sayBrightly| " "))
  (|displaySemanticErrors|)
  (when |$warningStack| (|sayBrightly| " "))
  (|displayWarnings|)
  (setq |$functorStats| (|addStats| |$functorStats| |$functionStats|))
  (|sayBrightly|
   (cons '|%l|
         (append (|bright| " Cumulative Statistics for Constructor")
                  (list |$op|))))
  (|sayBrightly|
   (cons " Time:"
         (append (|bright| (|normalizeStatAndStringify| (second |$functorStats|)))
                  (list "seconds"))))
  (|sayBrightly| " ")
  '|done|)
```

defun displayMissingFunctions

```
[member p??]
[getmode p??]
[sayBrightly p??]
[bright p??]
[formatUnabbreviatedSig p??]
[$env p??]
[$formalArgList p??]
[$CheckVectorList p??]
```

— **defun displayMissingFunctions** —

```
(defun |displayMissingFunctions| ()
  (let (i loc exp)
    (declare (special |$env| |$formalArgList| |$CheckVectorList|))
    (unless |$CheckVectorList|
      (setq loc nil)
      (setq exp nil)
      (loop for cvl in |$CheckVectorList| do
```



```

(unless (cdr cvl)
  (if (and (null (|member| (caar cvl) |$formalArgList|))
    (consp (|getmode| (caar cvl) |$env|))
    (eq (qfirst (|getmode| (caar cvl) |$env|)) '|Mapping|))
    (push (list (caar cvl) (cadar cvl)) loc)
    (push (list (caar cvl) (cadar cvl)) exp))))
(when loc
  (|sayBrightly| (cons '|%l| (|bright| " Missing Local Functions:"))))
(setq i 0)
(loop for item in loc do
  (|sayBrightly|
    (cons " [" (cons (incf i) (cons "]"
      (append (|bright| (first item))
        (cons '|: | (|formatUnabbreviatedSig| (second item))))))))))
(when exp
  (|sayBrightly| (cons '|%l| (|bright| " Missing Exported Functions:"))))
(setq i 0)
(loop for item in exp do
  (|sayBrightly|
    (cons " [" (cons (incf i) (cons "]"
      (append (|bright| (first item))
        (cons '|: | (|formatUnabbreviatedSig| (second item))))))))))

```

—————

defun makeFunctorArgumentParameters

```

[assq p??]
[isCategoryForm p??]
[genDomainViewList0 p223]
[union p??]
[$ConditionalOperators p??]
[$alternateViewList p??]
[$forceAdd p??]

```

— defun makeFunctorArgumentParameters —

```

(defun |makeFunctorArgumentParameters| (argl sigl target)
  (labels (
    (augmentSig (s ss)
      (let (u)
        (declare (special |$ConditionalOperators|))
        (if ss
          (progn
            (loop for u in ss do (push (rest u) |$ConditionalOperators|))
            (if (and (consp s) (eq (qfirst s) '|Join|))
              (progn

```

```

(if (setq u (assq 'category ss))
  (subst (append u ss) u s :test #'equal)
  (cons '|Join|
    (append (rest s) (list (cons 'category (cons '|package| ss))))))
(list '|Join| s (cons 'category (cons '|package| ss))))
s)))
(fn (a s)
  (declare (special |$CategoryFrame|))
  (if (|isCategoryForm| s |$CategoryFrame|)
    (if (and (consp s) (eq (qfirst s) '|Join|))
      (|genDomainViewList0| a (rest s))
      (list (|genDomainView| a s '|getDomainView|)))
    (list a)))
(findExtras (a target)
  (cond
    ((and (consp target) (eq (qfirst target) '|Join|))
      (reduce #'|union|
        (loop for x in (qrest target)
          collect (findExtras a x))))
    ((and (consp target) (eq (qfirst target) 'category))
      (reduce #'|union|
        (loop for x in (qcddr target)
          collect (findExtras1 a x))))))
(findExtras1 (a x)
  (cond
    ((and (consp x) (or (eq (qfirst x) 'and)) (eq (qfirst x) 'or))
      (reduce #'|union|
        (loop for y in (rest x) collect (findExtras1 a y))))
    ((and (consp x) (eq (qfirst x) 'if)
      (consp (qrest x)) (consp (qcddr x))
      (consp (qcdddr x))
      (eq (qcddddd x) nil))
      (|union| (findExtrasP a (second x))
        (|union|
          (findExtras1 a (third x))
          (findExtras1 a (fourth x))))))
    (t (findExtrasP a x))))
(findExtrasP (a x)
  (cond
    ((and (consp x) (or (eq (qfirst x) 'and)) (eq (qfirst x) 'or))
      (reduce #'|union|
        (loop for y in (rest x) collect (findExtrasP a y))))
    ((and (consp x) (eq (qfirst x) '|has|)
      (consp (qrest x)) (consp (qcddr x))
      (consp (qcdddr x))
      (eq (qcddddd x) nil))
      (|union| (findExtrasP a (second x))
        (|union|
          (findExtras1 a (third x))
          (findExtras1 a (fourth x))))))
    (t (and (consp x) (eq (qfirst x) '|has|)
      (findExtrasP a (second x))
      (findExtrasP a (third x))
      (findExtrasP a (fourth x))))))

```

```

      (cons (qrest x) (equal (qsecond x) a))
      (cons (qcddr x)
        (eq (qcdddr x) nil)
        (cons (qthird x)
          (eq (qcaaddr x) 'signature)))
      (list (third x))))
)
(let (|$alternateViewList| |$forceAdd| |$ConditionalOperators|)
  (declare (special |$alternateViewList| |$forceAdd| |$ConditionalOperators|))
  (setq |$alternateViewList| nil)
  (setq |$forceAdd| t)
  (setq |$ConditionalOperators| nil)
  (mapcar #'reduce
    (loop for a in argl for s in sigl do
      (fn a (augmentSig s (findExtras a target))))))
)

```

defun genDomainViewList0

[getDomainViewList p??]

— defun genDomainViewList0 —

```

(defun |genDomainViewList0| (id catlist)
  (|genDomainViewList| id catlist t))

```

defun genDomainViewList

[isCategoryForm p??]
 [genDomainView p224]
 [genDomainViewList p223]
 [\$EmptyEnvironment p??]

— defun genDomainViewList —

```

(defun |genDomainViewList| (id catlist firsttime)
  (declare (special |$EmptyEnvironment|) (ignore firsttime))
  (cond
    ((null catlist) nil)
    ((and (cons catlist) (eq (qrest catlist) nil)
      (null (|isCategoryForm| (first catlist) |$EmptyEnvironment|))))
  )

```

```

      nil)
    (t
      (cons
        (|genDomainView| id (first catlist) '|getDomainView|)
        (|genDomainViewList| id (rest catlist) nil))))))

```

defun genDomainView

```

[genDomainOps p224]
[augModemapsFromCategory p265]
[mkDomainConstructor p??]
[member p??]
[$e p??]
[$getDomainCode p??]

```

— defun genDomainView —

```

(defun |genDomainView| (name c viewSelector)
  (let (code cd)
    (declare (special |$getDomainCode| |$e|))
    (cond
      ((and (consp c) (eq (qfirst c) 'category) (consp (qrest c)))
        (|genDomainOps| name name c))
      (t
        (setq code
          (if (and (consp c) (eq (qfirst c) '|SubsetCategory|)
            (consp (qrest c)) (consp (qcddr c))
            (eq (qcdddr c) nil))
            (second c)
            c))
          (setq |$e| (|augModemapsFromCategory| name nil c |$e|))
          (setq cd
            (list 'let name (list viewSelector name (|mkDomainConstructor| code))))
          (unless (|member| cd |$getDomainCode|)
            (setq |$getDomainCode| (cons cd |$getDomainCode|)))
          name))))

```

defun genDomainOps

```

[getOperationAlist p269]
[substNames p270]

```

```
[mkq p??]
[mkDomainConstructor p??]
[addModemap p273]
[$e p??]
[$ConditionalOperators p??]
[$getDomainCode p??]
```

— defun genDomainOps —

```
(defun |genDomainOps| (viewName dom cat)
  (let (siglist oplist cd i)
    (declare (special |$e| |$ConditionalOperators| |$getDomainCode|))
    (setq oplist (|getOperationAlist| dom dom cat))
    (setq siglist (loop for lst in oplist collect (first lst)))
    (setq oplist (|substNames| dom viewName dom oplist))
    (setq cd
      (list 'let viewName
        (list 'mkOpVec| dom
          (cons 'list
            (loop for opsig in siglist
              collect
                (list 'list (mkq (first opsig))
                  (cons 'list
                    (loop for mode in (rest opsig)
                      collect (|mkDomainConstructor| mode))))))))))
    (setq |$getDomainCode| (cons cd |$getDomainCode|))
    (setq i 0)
    (loop for item in oplist do
      (if (|member| (first item) |$ConditionalOperators|)
        (setq |$e| (|addModemap| (caar item) dom (cadar item) nil
          (list 'elt viewName (incf i)) |$e|))
        (setq |$e| (|addModemap| (caar item) dom (cadar item) (second item)
          (list 'elt viewName (incf i)) |$e|))))
    viewName))
```

defun mkOpVec

```
[getPrincipalView p??]
[getOperationAlistFromLisplib p??]
[opOf p??]
[length p??]
[assq p??]
[assoc p??]
[sublis p??]
[AssocBarGensym p226]
```

[[\\$FormalMapVariableList](#) p270]

[Undef p??]

— defun mkOpVec —

```
(defun |mkOpVec| (dom siglist)
  (let (substargs oplist ops u noplist i tmp1)
    (declare (special |$FormalMapVariableList| |Undef|))
    (setq dom (|getPrincipalView| dom))
    (setq substargs
      (cons (cons '$ (elt dom 0))
            (loop for a in |$FormalMapVariableList| for x in (rest (elt dom 0))
                  collect (cons a x))))
    (setq oplist (|getOperationAlistFromLisplib| (|opOf| (elt dom 0))))
    (setq ops (make-array (|#| siglist)))
    (setq i -1)
    (loop for opSig in siglist do
      (incf i)
      (setq u (assq (first opSig) oplist))
      (setq tmp1 (|assoc| (second opSig) u))
      (cond
        ((and (consp tmp1) (consp (qrest tmp1))
              (consp (qcddr tmp1)) (consp (qcdddr tmp1))
              (eq (qcdddr tmp1) nil)
              (eq (qfourth tmp1) 'elt))
         (setelt ops i (elt dom (second tmp1))))
        (t
         (setq noplist (sublis substargs u))
         (setq tmp1
           (|AssocBarGensym|
            (subst (elt dom 0) '$ (second opSig) :test #'equal) noplist))
         (cond
           ((and (consp tmp1) (consp (qrest tmp1)) (consp (qcddr tmp1))
                 (consp (qcdddr tmp1))
                 (eq (qcdddr tmp1) nil)
                 (eq (qfourth tmp1) 'elt))
            (setelt ops i (elt dom (second tmp1))))
           (t
            (setelt ops i (cons |Undef| (cons (list (elt dom 0) i) opSig)))))))
    ops))
```

—————→

defun AssocBarGensym

[[EqualBarGensym](#) p248]

— defun AssocBarGensym —

```
(defun |AssocBarGensym| (key z)
  (loop for x in z
    do (when (and (consp x) (|EqualBarGensym| key (car x))) (return x))))
```

defun orderByDependency

```
[say p??]
[userError p??]
[intersection p??]
[member p??]
[remdup p??]
```

— defun orderByDependency —

```
(defun |orderByDependency| (vl dl)
  (let (selfDependents fatalError newl orderedVarList vlp dlp)
    (setq selfDependents
      (loop for v in vl for d in dl
        when (member v d)
        collect v))
    (loop for v in vl for d in dl
      when (member v d)
      do (say v "depends on itself")
        (setq fatalError t))
    (cond
      (fatalError (|userError| "Parameter specification error"))
      (t
        (loop until (null vl) do
          (setq newl
            (loop for v in vl for d in dl
              when (null (|intersection| d vl))
              collect v))
          (if (null newl)
            (setq vl nil) ; force loop exit
            (progn
              (setq orderedVarList (append newl orderedVarList))
              (setq vlp (setdifference vl newl))
              (setq dlp
                (loop for x in vl for d in dl
                  when (|member| x vlp)
                  collect (setdifference d newl)))
              (setq vl vlp)
              (setq dl dlp))))
          (when (and newl orderedVarList) (remdup (nreverse orderedVarList)))))))
```

6.2 Code optimization routines

defun optimizeFunctionDef

```
[rplac p??]
[sayBrightlyI p??]
[optimize p229]
[pp p??]
[bright p??]
[$reportOptimization p??]
```

— defun optimizeFunctionDef —

```
(defun |optimizeFunctionDef| (def)
  (labels (
    (fn (x g)
      (cond
        ((and (consp x) (eq (qfirst x) 'throw) (consp (qrest x))
          (equal (qsecond x) g))
         (|rplac| (car x) 'return)
         (|rplac| (cdr x)
          (replaceThrowByReturn (qcddr x) g)))
        ((atom x) nil)
        (t
         (replaceThrowByReturn (car x) g)
         (replaceThrowByReturn (cdr x) g))))
    (replaceThrowByReturn (x g)
      (fn x g)
        x)
    (removeTopLevelCatch (body)
      (if (and (consp body) (eq (qfirst body) 'catch) (consp (qrest body))
        (consp (qcddr body)) (eq (qcdddr body) nil))
        (removeTopLevelCatch
          (replaceThrowByReturn
            (qthird body) (qsecond body)))
        body)))
  (let (defp name slamOrLam args body bodyp)
    (declare (special |$reportOptimization|)
      (when |$reportOptimization|
        (|sayBrightlyI| (|bright| "Original LISP code:"))
        (|pp| def))
      (setq defp (|optimize| (copy def)))
      (when |$reportOptimization|
        (|sayBrightlyI| (|bright| "Optimized LISP code:"))
        (|pp| defp))
```



```
(|sayBrightlyI| (|bright| "Final LISP code:"))
(setq name (car defp))
(setq slamOrLam (caadr defp))
(setq args (cadadr defp))
(setq body (car (cddadr defp)))
(setq bodyp (removeTopLevelCatch body))
(list name (list slamOrLam args bodyp))))
```

defun optimize

```
[optimize p229]
[say p??]
[prettyprint p??]
[rplac p??]
[optIF2COND p232]
[getl p??]
[subrname p232]
```

— defun optimize —

```
(defun |optimize| (x)
  (labels (
    (opt (x)
      (let (argl body a y op)
        (cond
          ((atom x) nil)
          ((eq (setq y (car x)) 'quote) nil)
          ((eq y 'closedfn) nil)
          ((and (consp y) (consp (qfirst y)) (eq (qcaar y) 'xlam)
                (consp (qcadar y)) (consp (qcddar y))
                (eq (qcdddar y) nil))
           (setq argl (qcadar y))
           (setq body (qcaddar y))
           (setq a (qrest y))
           (|optimize| (cdr x))
           (cond
            ((eq argl '|ignore|) (rplac (car x) body))
            (t
             (when (null (<= (length argl) (length a)))
               (say "length mismatch in XLAM expression")
               (prettyprint y))
             (rplac (car x)
                    (|optimize|
                     (|optXLAMCond|
                      (sublis (pairList argl a) body))))))))
```

```

((atom y)
  (|optimize| (cdr x))
  (cond
    ((eq y '|true|) (rplac (car x) ''T))
    ((eq y '|false|) (rplac (car x) nil))))
((eq (car y) 'if)
  (rplac (car x) (|optIF2COND| y))
  (setq y (car x))
  (when (setq op (get1 (|subname| (car y)) 'optimize))
    (|optimize| (cdr x))
    (rplac (car x) (funcall op (|optimize| (car x)))))
  (setq op (get1 (|subname| (car y)) 'optimize))
  (|optimize| (cdr x))
  (rplac (car x) (funcall op (|optimize| (car x)))))
(t
  (rplac (car x) (|optimize| (car x)))
  (|optimize| (cdr x)))))
(opt x)
x))

```

defun optXLAMCond

[\[optCONDtail p231\]](#)
[\[optPredicateIfTrue p231\]](#)
[\[optXLAMCond p230\]](#)
[\[rplac p??\]](#)

— defun optXLAMCond —

```

(defun |optXLAMCond| (x)
  (cond
    ((and (consp x) (eq (qfirst x) 'cond) (consp (qrest x))
      (consp (qsecond x)) (consp (qcdadr x))
      (eq (qcddadr x) nil))
      (if (|optPredicateIfTrue| (qcaadr x))
        (qcadadr x)
        (cons 'cond (cons (qsecond x) (|optCONDtail| (qcddr x)))))
    ((atom x) x)
    (t
      (rplac (car x) (|optXLAMCond| (car x)))
      (rplac (cdr x) (|optXLAMCond| (cdr x)))
      x)))

```

defun optCONDtail

[optCONDtail p231]
 [\$true p??]

— defun optCONDtail —

```
(defun |optCONDtail| (z)
  (declare (special |$true|))
  (when z
    (cond
      ((|optPredicateIfTrue| (caar z)) (list (list |$true| (cadar z))))
      ((null (cdr z)) (list (car z) (list |$true| (list '|CondError|))))
      (t (cons (car z) (|optCONDtail| (cdr z)))))))
```

—————

defvar \$BasicPredicates

If these predicates are found in an expression the code optimizer routine `optPredicateIfTrue` then `optXLAM` will replace the call with the argument. This is used for predicates that test the type of their argument so that, for instance, a call to `integerp` on an integer will be replaced by that integer if it is true. This represents a simple kind of compile-time type evaluation.

— initvars —

```
(defvar |$BasicPredicates| ' (integerp stringp floatp))
```

—————

defun optPredicateIfTrue

[\$BasicPredicates p231]

— defun optPredicateIfTrue —

```
(defun |optPredicateIfTrue| (p)
  (declare (special |$BasicPredicates|))
  (cond
    ((and (consp p) (eq (qfirst p) 'quote)) T)
    ((and (consp p) (consp (qrest p)) (eq (qcddr p) nil)
      (member (qfirst p) |$BasicPredicates|) (funcall (qfirst p) (qsecond p)))
      t)
    (t nil)))
```

defun optIF2COND

[optIF2COND p232]
 [\$true p??]

— defun optIF2COND —

```
(defun |optIF2COND| (arg)
  (let (a b c)
    (declare (special |$true|))
    (setq a (cadr arg))
    (setq b (caddr arg))
    (setq c (cadddr arg))
    (cond
      ((eq b '|noBranch|) (list 'cond (list (list 'null a ) c)))
      ((eq c '|noBranch|) (list 'cond (list a b)))
      ((and (consp c) (eq (qfirst c) 'if))
       (cons 'cond (cons (list a b) (cdr (|optIF2COND| c)))))
      ((and (consp c) (eq (qfirst c) 'cond))
       (cons 'cond (cons (list a b) (qrest c))))
      (t
       (list 'cond (list a b) (list |$true| c))))))
```

defun subrname

[identp p??]
 [compiled-function-p p??]
 [mbpip p??]
 [bpiname p??]

— defun subrname —

```
(defun |subrname| (u)
  (cond
    ((identp u) u)
    ((or (compiled-function-p u) (mbpip u)) (bpiname u))
    (t nil)))
```

Special case optimizers

Optimization functions are called through the OPTIMIZE property on the symbol property list. The current list is:

call	optCall
seq	optSEQ
eq	optEQ
minus	optMINUS
qsminus	optQSMINUS
-	opt-
lessp	optLESSP
spadcall	optSPADCALL
	optSuchthat
catch	optCatch
cond	optCond
mkRecord	optMkRecord
recordelt	optRECORDELT
setrecordelt	optSETRECORDELT
recordcopy	optRECORDCOPY

Be aware that there are case-sensitivity issues. When found in the s-expression, each symbol in the left column will call a custom optimization routine in the right column. The optimization routines are below. Note that each routine has a special chunk in postvars using eval-when to set the property list at load time.

These optimizations are done destructively. That is, they modify the function in-place using rplac.

Not all of the optimization routines are called through the property list. Some are called only from other optimization routines, e.g. optPackageCall.

defplist optCall

— postvars —

```
(eval-when (eval load)
  (setf (get '|call| 'optimize) '|optCall|))
```

—————

defun Optimize “call” expressions

```
[optimize p229]
[rplac p??]
[optPackageCall p235]
```

```
[optCallSpecially p235]
[systemErrorHere p??]
[$QuickCode p??]
[$bootStrapMode p??]
```

— defun optCall —

```
(defun |optCall| (x)
  (let (u tmp1 fn a name q r n w)
    (declare (special |$QuickCode| |$bootStrapMode|))
    (setq u (cdr x))
    (setq x (|optimize| (list u)))
    (cond
      ((atom (car x)) (car x))
      (t
       (setq tmp1 (car x))
       (setq fn (car tmp1))
       (setq a (cdr tmp1))
       (cond
         ((atom fn) (rplac (cdr x) a) (rplac (car x) fn))
         ((and (consp fn) (eq (qfirst fn) 'pac)) (|optPackageCall| x fn a))
         ((and (consp fn) (eq (qfirst fn) '|applyFun|)
              (consp (qrest fn)) (eq (qcddr fn) nil))
          (setq name (qsecond fn))
          (rplac (car x) 'spadcall)
          (rplac (cdr x) (append a (cons name nil))))
         x)
        ((and (consp fn) (consp (qrest fn)) (consp (qcddr fn))
              (eq (qcddr fn) nil)
              (member (qfirst fn) '(elt qrefelt const)))
         (setq q (qfirst fn))
         (setq r (qsecond fn))
         (setq n (qthird fn))
         (cond
           ((and (null |$bootStrapMode|) (setq w (|optCallSpecially| q x n r)))
            w)
           ((eq q 'const)
            (list '|spadConstant| r n))
           (t
            (rplac (car x) 'spadcall)
            (when |$QuickCode| (rplaca fn 'qrefelt))
            (rplac (cdr x) (append a (list fn)))
            x)))
        (t (|systemErrorHere| "optCall"))))))))
```

—

defun optPackageCall

```
[rplaca p??]
[rplacd p??]
```

— defun optPackageCall —

```
(defun |optPackageCall| (x arg2 arglist)
  (let (packageVariableOrForm functionName)
    (setq packageVariableOrForm (second arg2))
    (setq functionName (third arg2))
    (rplaca x functionName)
    (rplacd x (append arglist (list packageVariableOrForm)))
    x))
```

—————

defun optCallSpecially

```
[lassoc p??]
[get p??]
[opOf p??]
[optSpecialCall p236]
[$specialCaseKeyList p??]
[$getDomainCode p??]
[$optimizableConstructorNames p??]
[$e p??]
```

— defun optCallSpecially —

```
(defun |optCallSpecially| (q x n r)
  (declare (ignore q))
  (labels (
    (lookup (a z)
      (let (zp)
        (when z
          (setq zp (car z))
          (setq z (cdr x))
          (if (and (consp zp) (eq (qfirst zp) 'let) (consp (qrest zp))
              (equal (qsecond zp) a) (consp (qcddr zp)))
              (qthird zp)
              (lookup a z))))))
    (let (tmp1 op y prop yy)
      (declare (special |$specialCaseKeyList| |$getDomainCode| |$e|
                        |$optimizableConstructorNames|))
      (cond
```

```

(setq y (lassoc r |$specialCaseKeyList|))
(|optSpecialCall| x y n))
(member (ifcar r) |$optimizableConstructorNames|)
(|optSpecialCall| x r n))
((and (setq y (|get| r '|value| |$e|))
      (member (|opOf| (car y)) |$optimizableConstructorNames|))
  (|optSpecialCall| x (car y) n))
((and (setq y (lookup r |$getDomainCode|))
      (progn
        (setq tmp1 y)
        (setq op (first tmp1))
        (setq y (second tmp1))
        (setq prop (third tmp1))
        tmp1)
      (setq yy (lassoc y |$specialCaseKeyList|)))
  (|optSpecialCall| x (list op yy prop) n))
(t nil))))

```

defun optSpecialCall

```

[optCallEval p237]
[function p??]
[keyedSystemError p??]
[mkq p??]
[getl p??]
[compileTimeBindingOf p237]
[rplac p??]
[optimize p229]
[rplacw p??]
[rplaca p??]
[$QuickCode p??]
[$Undef p??]

```

— defun optSpecialCall —

```

(defun |optSpecialCall| (x y n)
  (let (yval args tmp1 fn a)
    (declare (special |$QuickCode| |Undef|))
    (setq yval (|optCallEval| y))
    (cond
      ((eq (caaar x) 'const)
       (cond
         ((equal (ifcar (elt yval n)) #'|Undef|)
          (|keyedSystemError|
            "Unexpected error or improper call to system function %1: %2"

```



```

      (list "optSpecialCall" "invalid constant"))))
    (t (mkq (elt yval n))))))
  ((setq fn (get1 (|compileTimeBindingOf| (car (elt yval n))) '|SPADreplace|))
   (|rplac| (cdr x) (cdar x))
   (|rplac| (car x) fn)
   (when (and (consp fn) (eq (qfirst fn) 'xlam))
    (setq x (car (|optimize| (list x)))))
   (if (and (consp x) (eq (qfirst x) 'equal) (progn (setq args (qrest x)) t))
    (rplacw x (def-equal args))
    x))
  (t
   (setq tmp1 (car x))
   (setq fn (car tmp1))
   (setq a (cdr tmp1))
   (rplac (car x) 'spadcall)
   (when |$QuickCode| (rplaca fn 'qrefelt))
   (rplac (cdr x) (append a (list fn)))
   x))))

```

defun compileTimeBindingOf

```

[bpiname p??]
[keyedSystemError p??]
[moan p??]

```

— defun compileTimeBindingOf —

```

(defun |compileTimeBindingOf| (u)
  (let (name)
    (cond
      ((null (setq name (bpiname u)))
       (|keyedSystemError| "Irregular slot entry: %1s" (list u)))
      ((eq name '|Undef|)
       (moan "optimiser found unknown function"))
      (t name))))

```

defun optCallEval

```

[List p??]
[Integer p??]
[Vector p??]

```

```
[PrimitiveArray p??]
[FactoredForm p??]
[Matrix p??]
[eval p??]
```

— defun optCallEval —

```
(defun |optCallEval| (u)
  (cond
    ((and (consp u) (eq (qfirst u) '|List|))
     (|List| (|Integer|)))
    ((and (consp u) (eq (qfirst u) '|Vector|))
     (|Vector| (|Integer|)))
    ((and (consp u) (eq (qfirst u) '|PrimitiveArray|))
     (|PrimitiveArray| (|Integer|)))
    ((and (consp u) (eq (qfirst u) '|FactoredForm|))
     (|FactoredForm| (|Integer|)))
    ((and (consp u) (eq (qfirst u) '|Matrix|))
     (|Matrix| (|Integer|)))
    (t
     (|eval| u))))
```

—————

defplist optSEQ

— postvars —

```
(eval-when (eval load)
  (setf (get 'seq 'optimize) '|optSEQ|))
```

—————

defun optSEQ

— defun optSEQ —

```
(defun |optSEQ| (arg)
  (labels (
    (tryToRemoveSEQ (z)
      (if (and (consp z) (eq (qfirst z) 'seq) (consp (qrest z))
              (eq (qcddr z) nil) (consp (qsecond z))
              (consp (qcdadr z))
```

```

        (eq (qcddadr z) nil)
        (member (qcaadr z) '(exit return throw)))
    (qcadadr z)
    z))
(SEQToCOND (z)
  (let (transform before aft)
    (setq transform
      (loop for x in z
        while
          (and (consp x) (eq (qfirst x) 'cond) (consp (qrest x))
            (eq (qcddr x) nil) (consp (qsecond x))
            (consp (qcddadr x))
            (eq (qcddadr x) nil)
            (consp (qcadadr x))
            (eq (qfirst (qcadadr x)) 'exit)
            (consp (qrest (qcadadr x)))
            (eq (qcddr (qcadadr x)) nil))
        collect
          (list (qcaadr x)
            (qsecond (qcadadr x)))))
    (setq before (take (|#| transform) z))
    (setq aft (|after| z before))
    (cond
      ((null before) (cons 'seq aft))
      ((null aft)
        (cons 'cond (append transform (list '(t (|conderr|))))))
      (t
        (cons 'cond (append transform
          (list (list 't (|optSEQ| (cons 'seq aft))))))))))
(getRidOfTemps (z)
  (let (g x r)
    (cond
      ((null z) nil)
      ((and (consp z) (consp (qfirst z)) (eq (qcaar z) 'let)
        (consp (qcddr z)) (consp (qcddadr z))
        (gensymp (qcddadr z))
        (> 2 (|numOfOccurencesOf| (qcddadr z) (qrest z))))
        (setq g (qcddadr z))
        (setq x (qcddadr z))
        (setq r (qrest z))
        (getRidOfTemps (subst x g r :test #'equal)))
      ((eq (car z) '(/throwAway|)
        (getRidOfTemps (cdr z)))
      (t
        (cons (car z) (getRidOfTemps (cdr z))))))
  (tryToRemoveSEQ (SEQToCOND (getRidOfTemps (cdr arg))))))

```

defplist optEQ

— postvars —

```
(eval-when (eval load)
  (setf (get 'eq 'optimize) '|optEQ|))
```

—————

defun optEQ

— defun optEQ —

```
(defun |optEQ| (u)
  (let (z r)
    (cond
      ((and (consp u) (eq (qfirst u) 'eq) (consp (qrest u))
            (consp (qcddr u)) (eq (qcdddr u) nil))
       (setq z (qsecond u))
       (setq r (qthird u))
       ; That undoes some weird work in Boolean to do with the definition of true
       (if (and (numberp z) (numberp r))
           (list 'quote (eq z r))
           u))
      (t u))))
```

—————

defplist optMINUS

— postvars —

```
(eval-when (eval load)
  (setf (get 'minus 'optimize) '|optMINUS|))
```

—————

defun optMINUS

— defun optMINUS —

```
(defun |optMINUS| (u)
  (let (v)
    (cond
      ((and (consp u) (eq (qfirst u) 'minus) (consp (qrest u))
            (eq (qcddr u) nil))
       (setq v (qsecond u))
       (cond ((numberp v) (- v)) (t u)))
      (t u))))
```

defplist optQSMINUS

— postvars —

```
(eval-when (eval load)
  (setf (get 'qsminus 'optimize) '|optQSMINUS|))
```

defun optQSMINUS

— defun optQSMINUS —

```
(defun |optQSMINUS| (u)
  (let (v)
    (cond
      ((and (consp u) (eq (qfirst u) 'qsminus) (consp (qrest u))
            (eq (qcddr u) nil))
       (setq v (qsecond u))
       (cond ((numberp v) (- v)) (t u)))
      (t u))))
```

defplist opt-

— postvars —

```
(eval-when (eval load)
```

```
(setf (get '- 'optimize) '|opt-|))
```

defun opt-

— defun opt- —

```
(defun |opt-| (u)
  (let (v)
    (cond
      ((and (consp u) (eq (qfirst u) '-') (consp (qrest u))
            (eq (qcddr u) NIL))
       (setq v (qsecond u))
       (cond ((numberp v) (- v)) (t u)))
      (t u))))
```

defplist optLESSP

— postvars —

```
(eval-when (eval load)
  (setf (get 'lessp 'optimize) '|optLESSP|))
```

defun optLESSP

— defun optLESSP —

```
(defun |optLESSP| (u)
  (let (a b)
    (cond
      ((and (consp u) (eq (qfirst u) 'lessp) (consp (qrest u))
            (consp (qcddr u))
            (eq (qcdddr u) nil))
       (setq a (qsecond u))
       (setq b (qthird u)))
```

— postvars —

— defun optSPADCALL —

```
(defun |optSPADCALL| (form)
  (let (fun arg1 tmp1 dom slot)
    (declare (special |$InteractiveMode|))
    (setq arg1 (cdr form))
    (cond
      ; last arg is function/env, but may be a form
      ((null |$InteractiveMode|) form)
      ((and (consp arg1)
             (progn (setq tmp1 (reverse arg1)) t)
             (consp tmp1))
       (setq fun (qfirst tmp1))
       (setq arg1 (qrest tmp1))
       (setq arg1 (nreverse arg1))
       (cond
         ((and (consp fun)
                (eq (qfirst fun) 'elt)
                (progn
                  (and (consp (qrest fun))
                      (progn
                        (setq dom (qsecond fun))
                        (and (consp (qcddr fun))
```

```

                                (eq (qcdddr fun) nil)
                                (progn
                                  (setq slot (qthird fun))
                                  t))))))
  (|optCall| (cons '|call| (cons (list 'elt dom slot) arg1))))
  (t form)))
  (t form)))

```

defplist optSuchthat

— postvars —

```

(eval-when (eval load)
  (setf (get '|\\| 'optimize) '|optSuchthat|))

```

defun optSuchthat

— defun optSuchthat —

```

(defun |optSuchthat| (arg)
  (cons 'suchthat (cdr arg)))

```

defplist optCatch

— postvars —

```

(eval-when (eval load)
  (setf (get 'catch 'optimize) '|optCatch|))

```

defun optCatch

[rplac p??]
 [optimize p229]
 [\$InteractiveMode p??]

— defun optCatch —

```
(defun |optCatch| (x)
  (labels (
    (changeThrowToExit (s g)
      (cond
        ((or (atom s) (member (car s) '(quote seq repeat collect))) nil)
        ((and (consp s) (eq (qfirst s) 'throw) (consp (qrest s))
          (equal (qsecond s) g))
          (|rplac| (car s) 'exit)
          (|rplac| (cdr s) (qcddr s)))
        (t
          (changeThrowToExit (car s) g)
          (changeThrowToExit (cdr s) g))))
    (hasNoThrows (a g)
      (cond
        ((and (consp a) (eq (qfirst a) 'throw) (consp (qrest a))
          (equal (qsecond a) g))
          nil)
        ((atom a) t)
        (t
          (and (hasNoThrows (car a) g)
            (hasNoThrows (cdr a) g))))))
    (changeThrowToGo (s g)
      (let (u)
        (cond
          ((or (atom s) (eq (car s) 'quote)) nil)
          ((and (consp s) (eq (qfirst s) 'throw) (consp (qrest s))
            (equal (qsecond s) g) (consp (qcddr s))
            (eq (qcdddr s) nil))
            (setq u (qthird s))
            (changeThrowToGo u g)
            (|rplac| (car s) 'progn)
            (|rplac| (cdr s) (list (list 'let (cadr g) u) (list 'go (cadr g)))))
          (t
            (changeThrowToGo (car s) g)
            (changeThrowToGo (cdr s) g))))))
    (let (g tmp2 u s tmp6 a)
      (declare (special |$InteractiveMode|))
      (setq g (cadr x))
      (setq a (caddr x))
      (cond
        (|$InteractiveMode| x)
```

```

((atom a) a)
(t
 (cond
  ((and (consp a) (eq (qfirst a) 'seq) (consp (qrest a))
    (progn (setq tmp2 (reverse (qrest a))) t)
    (consp tmp2) (consp (qfirst tmp2)) (eq (qcaar tmp2) 'throw)
    (consp (qcddar tmp2))
    (equal (qcadar tmp2) g)
    (consp (qcdddar tmp2))
    (eq (qcdddar tmp2) nil))
  (setq u (qcaddar tmp2))
  (setq s (qrest tmp2))
  (setq s (nreverse s))
  (changeThrowToExit s g)
  (|rplac| (cdr a) (append s (list (list 'exit u)))))
  (setq tmp6 (|optimize| x))
  (setq a (caddr tmp6))))
(cond
 ((hasNoThrows a g)
  (|rplac| (car x) (car a))
  (|rplac| (cdr x) (cdr a)))
 (t
  (changeThrowToGo a g)
  (|rplac| (car x) 'seq)
  (|rplac| (cdr x)
    (list (list 'exit a) (cadr g) (list 'exit (cadr g)))))
  x))))

```

defplist optCond

— postvars —

```

(eval-when (eval load)
  (setf (get 'cond 'optimize) '|optCond|))

```

defun optCond

```

[rplacd p??]
[TruthP p268]
[EqualBarGensym p248]

```

[rplac p??]

— defun optCond —

```
(defun |optCond| (x)
  (let (z p1 p2 c3 c1 c2 a result)
    (setq z (cdr x))
    (when
      (and (consp z) (consp (qrest z)) (eq (qcddr z) nil)
           (consp (qsecond z)) (consp (qcdadr z))
           (eq (qrest (qcdadr z)) nil)
           (|TruthP| (qcaadr z))
           (consp (qcadadr z))
           (eq (qfirst (qcadadr z)) 'cond))
      (rplacd (cdr x) (qrest (qcadadr z))))
    (cond
      ((and (consp z) (consp (qfirst z)) (consp (qrest z)) (consp (qsecond z)))
        (setq p1 (qcaar z))
        (setq c1 (qcdar z))
        (setq p2 (qcaadr z))
        (setq c2 (qcdadr z))
        (when
          (or (and (consp p1) (eq (qfirst p1) 'null) (consp (qrest p1))
                  (eq (qcddr p1) nil)
                  (equal (qsecond p1) p2))
              (and (consp p2) (eq (qfirst p2) 'null) (consp (qrest p2))
                  (eq (qcddr p2) nil)
                  (equal (qsecond p2) p1)))
            (setq z (list (cons p1 c1) (cons 't c2)))
            (rplacd x z))
          (when
            (and (consp c1) (eq (qrest c1) nil) (equal (qfirst c1) 'nil)
                 (equal p2 't) (equal (car c2) 't))
              (if (and (consp p1) (eq (qfirst p1) 'null) (consp (qrest p1))
                      (eq (qcddr p1) nil))
                  (setq result (qsecond p1))
                  (setq result (list 'null p1))))))
      (if result
          result
          (cond
            ((and (consp z) (consp (qfirst z)) (consp (qrest z)) (consp (qsecond z))
                  (consp (qcddr z)) (eq (qcddr z) nil)
                  (consp (qthird z))
                  (|TruthP| (qcaaddr z)))
              (setq p1 (qcaar z))
              (setq c1 (qcdar z))
              (setq p2 (qcaadr z))
              (setq c2 (qcdadr z))
              (setq c3 (qcdaddr z))
              (cond
```

```

(|EqualBarGensym| c1 c3)
(list 'cond
  (cons (list 'or p1 (list 'null p2)) c1) (cons (list 'quote t) c2)))
(|EqualBarGensym| c1 c2)
(list 'cond (cons (list 'or p1 p2) c1) (cons (list 'quote t) c3)))
(t x)))
(t
  (do ((y z (cdr y)))
    ((atom y) nil)
    (do ()
      ((null (and (consp y) (consp (qfirst y)) (consp (qcadr y))
        (eq (qcddar y) nil) (consp (qrest y))
        (consp (qsecond y)) (consp (qcdadr y))
        (eq (qcddadr y) nil)
        (|EqualBarGensym| (qcadar y)
          (qcadadr y)))))
      nil)
      (setq a (list 'or (qcaar y) (qcaadr y)))
      (rplac (car (car y)) a)
      (rplac (cdr y) (qcddr y))))
  x))))

```

defun EqualBarGensym

```

[gensymp p??]
[$GensymAssoc p??]
[$GensymAssoc p??]

```

— defun EqualBarGensym —

```

(defun |EqualBarGensym| (x y)
  (labels (
    (fn (x y)
      (let (z)
        (declare (special |$GensymAssoc|))
        (cond
          ((equal x y) t)
          ((and (gensymp x) (gensymp y))
            (if (setq z (|assoc| x |$GensymAssoc|))
              (if (equal y (cdr z)) t nil)
              (progn
                (setq |$GensymAssoc| (cons (cons x y) |$GensymAssoc|))
                t)))
          ((null x) (and (consp y) (eq (qrest y) nil) (gensymp (qfirst y))))
          ((null y) (and (consp x) (eq (qrest x) nil) (gensymp (qfirst x))))

```

```

((or (atom x) (atom y)) nil)
(t
  (and (fn (car x) (car y))
        (fn (cdr x) (cdr y))))))
(let (|$GensymAssoc|)
  (declare (special |$GensymAssoc|))
  (setq |$GensymAssoc| NIL)
  (fn x y)))

```

defplist optMkRecord

— postvars —

```

(eval-when (eval load)
  (setf (get 'mkRecord 'optimize) 'optMkRecord))

```

defun optMkRecord

[length p??]

— defun optMkRecord —

```

(defun |optMkRecord| (arg)
  (let (u)
    (setq u (cdr arg))
    (cond
      ((and (consp u) (eq (qrest u) nil)) (list 'list (qfirst u)))
      ((eq1 (|#| u) 2) (cons 'cons u))
      (t (cons 'vector u)))))

```

defplist optRECORDELT

— postvars —

```
(eval-when (eval load)
  (setf (get 'recordelt 'optimize) '|optRECORDELT|))
```

defun optRECORDELT

```
[keyedSystemError p??]
```

— defun optRECORDELT —

```
(defun |optRECORDELT| (arg)
  (let (name ind len)
    (setq name (cadr arg))
    (setq ind (caddr arg))
    (setq len (cadddr arg))
    (cond
      ((eql len 1)
       (cond
         ((eql ind 0) (list 'qcar name))
         (t (|keyedSystemError| "Bad index in record optimization: %1"
                               (list ind)))))
      ((eql len 2)
       (cond
         ((eql ind 0) (list 'qcar name))
         ((eql ind 1) (list 'qcdr name))
         (t (|keyedSystemError| "Bad index in record optimization: %1"
                               (list ind)))))
      (t (list 'qvelt name ind)))))
```

defpllist optSETRECORDELT

— postvars —

```
(eval-when (eval load)
  (setf (get 'setrecordelt 'optimize) '|optSETRECORDELT|))
```

defun optSETRECORDELT

[keyedSystemError p??]

— defun optSETRECORDELT —

```

(defun |optSETRECORDELT| (arg)
  (let (name ind len expr)
    (setq name (cadr arg))
    (setq ind (caddr arg))
    (setq len (caddr arg))
    (setq expr (car (caddr arg)))
    (cond
      ((eql len 1)
       (if (eql ind 0)
           (list 'progn (list 'rplaca name expr) (list 'qcar name))
           (|keyedSystemError| "Bad index in record optimization: %1" (list ind))))
      ((eql len 2)
       (cond
         ((eql ind 0)
          (list 'progn (list 'rplaca name expr) (list 'qcar name)))
         ((eql ind 1)
          (list 'progn (list 'rplacd name expr) (list 'qcdr name)))
         (t (|keyedSystemError| "Bad index in record optimization: %1"
                                (list ind)))))
      (t
       (list 'qsetvelt name ind expr)))))

```

—

defplist optRECORDCOPY

— postvars —

```

(eval-when (eval load)
  (setf (get 'recordcopy 'optimize) '|optRECORDCOPY|))

```

—

defun optRECORDCOPY

— defun optRECORDCOPY —

```
(defun |optRECORDCOPY| (arg)
  (let (name len)
    (setq name (cadr arg))
    (setq len (caddr arg))
    (cond
      ((eql len 1) (list 'list (list 'car name)))
      ((eql len 2) (list 'cons (list 'car name) (list 'cdr name)))
      (t (list 'replace (list 'make-array len) name)))))
```

6.3 Functions to manipulate modemap

defun addDomain

```
[identp p??]
[qslessp p??]
[getDomainsInScope p254]
[domainMember p264]
[isLiteral p??]
[addNewDomain p256]
[getmode p??]
[isCategoryForm p??]
[isFunctor p253]
[constructor? p??]
[member p??]
[unknownTypeError p253]
```

— defun addDomain —

```
(defun |addDomain| (domain env)
  (let (s name tmp1)
    (cond
      ((atom domain)
        (cond
          ((eq domain '|$EmptyMode|) env)
          ((eq domain '|$NoValueMode|) env)
          ((or (null (identp domain))
              (and (qslessp 2 (|#| (setq s (princ-to-string domain))))
                   (eq #\# (elt s 0))
                   (eq #\# (elt s 1))))
            env)
          ((member domain (|getDomainsInScope| env)) env)
          ((|isLiteral| domain env) env)
          (t (|addNewDomain| domain env))))
      ((eq (setq name (car domain)) '|Category|) env)
```



```

(|domainMember| domain (|getDomainsInScope| env)) env)
((and (progn
  (setq tmp1 (|getmode| name env))
  (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|)
    (consp (qrest tmp1))))
  (|isCategoryForm| (second tmp1) env))
  (|addNewDomain| domain env))
((or (|isFunction| name) (|constructor?| name))
  (|addNewDomain| domain env))
(t
  (when (and (null (|isCategoryForm| domain env))
    (null (|member| name '(|Mapping| category))))
    (|unknownTypeError| name))
  env))))

```

defun unknownTypeError

[stackSemanticError p??]

— defun unknownTypeError —

```

(defun |unknownTypeError| (name)
  (let (op)
    (setq name
      (if (and (consp name) (setq op (qfirst name)))
        op
        name))
    (|stackSemanticError| (list name '|is not a known type|) nil)))

```

defun isFunctor

[opOf p??]
 [identp p??]
 [getdatabase p??]
 [get p??]
 [constructor? p??]
 [updateCategoryFrameForCategory p121]
 [updateCategoryFrameForConstructor p120]
 [\$CategoryFrame p??]
 [\$InteractiveMode p??]

— defun isFunctor —

```
(defun |isFunctor| (x)
  (let (op u prop)
    (declare (special |$CategoryFrame| |$InteractiveMode|))
    (setq op (|opOf| x))
    (cond
      ((null (identp op)) nil)
      (|$InteractiveMode|
       (if (member op '(|Union| |SubDomain| |Mapping| |Record|))
           t
           (member (getdatabase op 'constructorkind) '(|domain| |package|))))
      ((setq u
        (or (|get| op 'isFunctor| |$CategoryFrame|)
            (member op '(|SubDomain| |Union| |Record|))))
         u)
      ((|constructor?| op)
       (cond
         ((setq prop (|get| op 'isFunctor| |$CategoryFrame|)) prop)
         (t
          (if (eq (getdatabase op 'constructorkind) '|category|)
              (|updateCategoryFrameForCategory| op)
              (|updateCategoryFrameForConstructor| op))
          (|get| op 'isFunctor| |$CategoryFrame|)))
       (t nil))))
```

—

defun getDomainsInScope

The way XLAMs work:

```
((XLAM ($1 $2 $3) (SETELT $1 0 $3)) X "c" V) ==> (SETELT X 0 V)
```

```
[get p??]
|$CapsuleDomainsInScope p??]
|$insideCapsuleFunctionIfTrue p??]
```

— defun getDomainsInScope —

```
(defun |getDomainsInScope| (env)
  (declare (special |$CapsuleDomainsInScope| |$insideCapsuleFunctionIfTrue|))
  (if |$insideCapsuleFunctionIfTrue|
      |$CapsuleDomainsInScope|
      (|get| '|$DomainsInScope| 'special env)))
```

defun putDomainsInScope

```
[getDomainsInScope p254]
[put p??]
[delete p??]
[say p??]
[member p??]
[$CapsuleDomainsInScope p??]
[$insideCapsuleFunctionIfTrue p??]
```

— defun putDomainsInScope —

```
(defun |putDomainsInScope| (x env)
  (let (z newValue)
    (declare (special |$CapsuleDomainsInScope| |$insideCapsuleFunctionIfTrue|))
    (setq z (|getDomainsInScope| env))
    (when (|member| x z) (say "***** Domain: " x " already in scope"))
    (setq newValue (cons x (|delete| x z)))
    (if |$insideCapsuleFunctionIfTrue|
        (progn
          (setq |$CapsuleDomainsInScope| newValue)
          env)
        (|put| '|$DomainsInScope| 'special newValue env))))
```

defun isSuperDomain

```
[isSubset p??]
[lassoc p??]
[opOf p??]
[get p??]
```

— defun isSuperDomain —

```
(defun |isSuperDomain| (domainForm domainFormp env)
  (cond
    ((|isSubset| domainFormp domainForm env) t)
    ((and (eq domainForm '|Rep|) (eq domainFormp '$)) t)
    (t (lassoc (|opOf| domainFormp) (|get| domainForm '|SubDomain| env))))))
```

defun addNewDomain

[augModemapsFromDomain p256]

— defun addNewDomain —

```
(defun |addNewDomain| (domain env)
  (|augModemapsFromDomain| domain domain env))
```

—————

defun augModemapsFromDomain

```
[member p??]
[getDomainsInScope p254]
[getdatabase p??]
[opOf p??]
[addNewDomain p256]
[listOrVectorElementNode p??]
[stripUnionTags p??]
[augModemapsFromDomain1 p257]
[$Category p??]
[$DummyFunctorNames p??]
```

— defun augModemapsFromDomain —

```
(defun |augModemapsFromDomain| (name functorForm env)
  (let (curDomainsInScope u innerDom)
    (declare (special |$Category| |$DummyFunctorNames|))
    (cond
      ((|member| (or (ifcar name) name) |$DummyFunctorNames|)
       env)
      ((or (equal name |$Category|) (|isCategoryForm| name env))
       env)
      ((|member| name (setq curDomainsInScope (|getDomainsInScope| env)))
       env)
      (t
       (when (setq u (getdatabase (|opOf| functorForm) 'superdomain))
         (setq env (|addNewDomain| (car u) env)))
       (when (setq innerDom (|listOrVectorElementNode| name))
         (setq env (|addDomain| innerDom env)))
       (when (and (consp name) (eq (qfirst name) '|Union|))
         (dolist (d (|stripUnionTags| (qrest name)))
           (setq env (|addDomain| d env))))
       (|augModemapsFromDomain1| name functorForm env))))))
```

defun augModemapsFromDomain1

```
[getl p??]
[addConstructorModemaps p258]
[getmode p??]
[augModemapsFromCategory p265]
[getmodeOrMapping p??]
[substituteCategoryArguments p257]
[stackMessage p??]
```

— defun augModemapsFromDomain1 —

```
(defun |augModemapsFromDomain1| (name functorForm env)
  (let (mappingForm categoryForm functArgTypes catform)
    (cond
      ((getl (ifcar functorForm) '|makeFunctionList|)
        (|addConstructorModemaps| name functorForm env))
      ((and (atom functorForm) (setq catform (|getmode| functorForm env))
        (|augModemapsFromCategory| name functorForm catform env))
        (|augModemapsFromCategory| name functorForm catform env))
      ((setq mappingForm (|getmodeOrMapping| (ifcar functorForm) env))
        (when (eq (car mappingForm) '|Mapping|) (car mappingForm))
        (setq categoryForm (cadr mappingForm))
        (setq functArgTypes (cddr mappingForm))
        (setq catform
          (|substituteCategoryArguments| (cdr functorForm) categoryForm))
        (|augModemapsFromCategory| name functorForm catform env))
      (t
        (|stackMessage| (list functorForm '| is an unknown mode|))
        env))))
```

defun substituteCategoryArguments

```
[internal p??]
[sublis p??]
```

— defun substituteCategoryArguments —

```
(defun |substituteCategoryArguments| (argl catform)
  (let (arglAssoc (i 0))
    (setq argl (subst '$$ '$ argl :test #'equal))
    (setq arglAssoc
      (loop for a in argl
```

```
collect (cons (internl '|#| (princ-to-string (incf i))) a)))
(sublis arglAssoc catform)))
```

defun addConstructorModemaps

```
[putDomainsInScope p255]
[getl p??]
[addModemap p273]
[$InteractiveMode p??]
```

— defun addConstructorModemaps —

```
(defun |addConstructorModemaps| (name form env)
  (let (|$InteractiveMode| functorName fn tmp1 funList op sig nsig opcode)
    (declare (special |$InteractiveMode|))
    (setq functorName (car form))
    (setq |$InteractiveMode| nil)
    (setq env (|putDomainsInScope| name env))
    (setq fn (getl functorName '|makeFunctionList|))
    (setq tmp1 (funcall fn name form env))
    (setq funList (car tmp1))
    (setq env (cadr tmp1))
    (dolist (item funList)
      (setq op (first item))
      (setq sig (second item))
      (setq opcode (third item))
      (when (and (consp opcode) (consp (qrest opcode))
                  (consp (qcddr opcode))
                  (eq (qcddr opcode) nil)
                  (eq (qfirst opcode) 'elt))
        (setq nsig (subst '$$$ name sig :test #'equal))
        (setq nsig
          (subst '$ '$$$ (subst '$$ '$ nsig :test #'equal) :test #'equal))
        (setq opcode (list (first opcode) (second opcode) nsig)))
      (setq env (|addModemap| op name sig t opcode env)))
    env))
```

defun getModemap

```
[get p??]
[compApplyModemap p259]
```

```
[sublis p??]
```

— defun getModemap —

```
(defun |getModemap| (x env)
  (let (u)
    (dolist (modemap (|get| (first x) '|modemap| env))
      (when (setq u (|compApplyModemap| x modemap env nil))
        (return (sublis (third u) modemap))))))
```

defun compApplyModemap

```
[length p??]
[pmatchWithSl p??]
[sublis p??]
[comp p587]
[coerce p355]
[compMapCond p260]
[member p??]
[genDeltaEntry p??]
[$e p??]
[$bindings p??]
[$e p??]
[$bindings p??]
```

— defun compApplyModemap —

```
(defun |compApplyModemap| (form modemap |$e| sl)
  (declare (special |$e|))
  (let (op argl mc mr margl fnsel g mp lt ltp temp1 f)
    (declare (special |$bindings| |$e|))
    ; -- $e      is the current environment
    ; -- sl      substitution list, nil means bottom-up, otherwise top-down
    ; -- 0.      fail immediately if #argl=#margl
    (setq op (car form))
    (setq argl (cdr form))
    (setq mc (caar modemap))
    (setq mr (cadar modemap))
    (setq margl (cddar modemap))
    (setq fnsel (cdr modemap))
    (when (= (|#| argl) (|#| margl))
      ; 1. use modemap to evaluate arguments, returning failed if not possible
      (setq lt
        (prog (t0)
```

```

(return
  (do ((t1 arg1 (cdr t1)) (y NIL) (t2 marg1 (cdr t2)) (m nil))
      ((or (atom t1) (atom t2)) (nreverse0 t0))
      (setq y (car t1))
      (setq m (car t2))
      (setq t0
        (cons
          (progn
            (setq sl (|pmatchWithSl| mp m sl))
            (setq g (sublis sl m))
            (setq temp1 (or (|comp| y g |$e|) (return '|failed|)))
            (setq mp (cadr temp1))
            (setq |$e| (caddr temp1))
            temp1)
          t0))))))
; 2. coerce each argument to final domain, returning failed
;   if not possible
(unless (eq lt '|failed|)
  (setq ltp
    (loop for y in lt for d in (sublis sl marg1)
      collect (or (|coerce| y d) (return '|failed|))))
  (unless (eq ltp '|failed|)
    ; 3. obtain domain-specific function, if possible, and return
    ; $bindings is bound by compMapCond
    (setq temp1 (|compMapCond| op mc sl fnsl))
    (when temp1
      ; can no longer trust what the modemap says for a reference into
      ; an exterior domain (it is calculating the displacement based on view
      ; information which is no longer valid; thus ignore this index and
      ; store the signature instead.)
      (setq f (car temp1))
      (setq |$bindings| (cadr temp1))
      (if (and (consp f) (consp (qcdr f)) (consp (qcddr f)) ; f is [op1,.]
              (eq (qcddr f) nil)
              (|member| (qcar f) '(elt const |Subsumed|))))
          (list (|genDeltaEntry| (cons op modemap)) ltp |$bindings|)
          (list f ltp |$bindings|))))))

```

defun compMapCond

[compMapCond' p261]
 [\$bindings p??]

— defun compMapCond —

```
(defun |compMapCond| (op mc |$bindings| fnsl)
```



```
(declare (special |$bindings|))
(let (t0)
  (do ((t1 nil t0) (t2 fnsel (cdr t2)) (u nil))
      ((or t1 (atom t2) (progn (setq u (car t2)) nil)) t0)
      (setq t0 (or t0 (|compMapCond'| u op mc |$bindings|))))))
```

defun compMapCond'

```
[compMapCond" p261]
[compMapConfFun p??]
[stackMessage p??]
```

— defun compMapCond' —

```
(defun |compMapCond'| (t0 op dc bindings)
  (let ((cexpr (car t0)) (fnexpr (cadr t0)))
    (if (|compMapCond''| cexpr dc)
        (|compMapCondFun| fnexpr op dc bindings)
        (|stackMessage| ("not known that" ,dc "has" ,cexpr)))))
```

defun compMapCond"

```
[compMapCond" p261]
[knownInfo p??]
[get p??]
[stackMessage p??]
[$Information p??]
[$e p??]
```

— defun compMapCond" —

```
(defun |compMapCond''| (cexpr dc)
  (let (l u tmp1 tmp2)
    (declare (special |$Information| |$e|))
    (cond
      ((eq cexpr t) t)
      ((and (consp cexpr)
            (eq (qcar cexpr) 'and)
            (progn (setq l (qcdr cexpr)) t))
       (prog (t0)
```

```

(setq t0 t)
(return
  (do ((t1 nil (null t0)) (t2 1 (cdr t2)) (u nil))
      ((or t1 (atom t2) (progn (setq u (car t2)) nil)) t0)
      (setq t0 (and t0 (|compMapCond''| u dc))))))
((and (consp cexpr)
      (eq (qcar cexpr) 'or)
      (progn (setq l (qcdr cexpr)) t))
 (prog (t3)
   (setq t3 nil)
   (return
     (do ((t4 nil t3) (t5 1 (cdr t5)) (u nil))
         ((or t4 (atom t5) (progn (setq u (car t5)) nil)) t3)
         (setq t3 (or t3 (|compMapCond''| u dc))))))
  ((and (consp cexpr)
        (eq (qcar cexpr) '|not|)
        (progn
          (setq tmp1 (qcdr cexpr))
          (and (consp tmp1)
                (eq (qcdr tmp1) nil)
                (progn (setq u (qcar tmp1)) t))))
    (null (|compMapCond''| u dc)))
  ((and (consp cexpr)
        (eq (qcar cexpr) '|has|)
        (progn
          (setq tmp1 (qcdr cexpr))
          (and (consp tmp1)
                (progn
                  (setq tmp2 (qcdr tmp1))
                  (and (consp tmp2)
                        (eq (qcdr tmp2) nil))))))
    (cond
      ((|knownInfo| cexpr) t)
      (t nil)))
  ((|member|
    (cons 'attribute (cons dc (cons cexpr nil)))
    (|get| '|$Information| 'special |$e|))
   t)
  (t
   (|stackMessage| '("not known that" ,dc "has" ,cexpr)
   nil))))

```

defun compMapCondFun

— defun compMapCondFun —

```
(defun |compMapCondFun| (fnexpr op dc bindings)
  (declare (ignore op) (ignore dc))
  (cons fnexpr (cons bindings nil)))
```

defun getUniqueSignature

[getUniqueModemap p263]

— defun getUniqueSignature —

```
(defun |getUniqueSignature| (form env)
  (cdar (|getUniqueModemap| (first form) (|#| (rest form)) env)))
```

defun getUniqueModemap

[getModemapList p263]

[qslessp p??]

[stackWarning p??]

— defun getUniqueModemap —

```
(defun |getUniqueModemap| (op numOfArgs env)
  (let (mml)
    (cond
      ((eq 1 (|#| (setq mml (|getModemapList| op numOfArgs env))))
       (car mml))
      ((qslessp 1 (|#| mml))
       (|stackWarning|
        (list numOfArgs " argument form of: " op " has more than one modemap"))
       (car mml))
      (t nil))))
```

defun getModemapList

[getModemapListFromDomain p264]

[nreverse0 p??]

[get p??]

— defun getModemapList —

```
(defun |getModemapList| (op numOfArgs env)
  (let (result)
    (cond
      ((and (consp op) (eq (qfirst op) '|elt|) (consp (qrest op))
        (consp (qcddr op)) (eq (qcdddr op) nil))
        (|getModemapListFromDomain| (third op) numOfArgs (second op) env))
      (t
        (dolist (term (|get| op '|modemap| env) (nreverse0 result))
          (when (eql numOfArgs (|#| (cddar term))) (push term result)))))))
```

—————

defun getModemapListFromDomain

[get p??]

— defun getModemapListFromDomain —

```
(defun |getModemapListFromDomain| (op numOfArgs d env)
  (loop for term in (|get| op '|modemap| env)
    when (and (equal (caar term) d) (eql (|#| (cddar term)) numOfArgs))
    collect term))
```

—————

defun domainMember

[modeEqual p366]

— defun domainMember —

```
(defun |domainMember| (dom domList)
  (let (result)
    (dolist (d domList result)
      (setq result (or result (|modeEqual| dom d)))))
```

—————

defun augModemapsFromCategory

```
[evalAndSub p269]
[compilerMessage p??]
[putDomainsInScope p255]
[addModemapKnown p272]
[$base p??]
```

— **defun augModemapsFromCategory** —

```
(defun |augModemapsFromCategory| (domainName functorform categoryForm env)
  (let (tmp1 op sig cond fnsel)
    (declare (special |$base|))
    (setq tmp1 (|evalAndSub| domainName domainName functorform categoryForm env))
    (|compilerMessage| (list '|Adding| domainName '| modemaps|))
    (setq env (|putDomainsInScope| domainName (second tmp1)))
    (setq |$base| 4)
    (dolist (u (first tmp1))
      (setq op (caar u))
      (setq sig (cadar u))
      (setq cond (cadr u))
      (setq fnsel (caddr u))
      (setq env (|addModemapKnown| op domainName sig cond fnsel env)))
    env))
```

defun addEltModemap

This is a hack to change selectors from strings to identifiers; and to add flag identifiers as literals in the environment [makeLiteral p??]

```
[addModemap1 p274]
[systemErrorHere p??]
[$insideCapsuleFunctionIfTrue p??]
[$e p??]
```

— **defun addEltModemap** —

```
(defun |addEltModemap| (op mc sig pred fn env)
  (let (tmp1 v sel lt id)
    (declare (special |$e| |$insideCapsuleFunctionIfTrue|))
    (cond
      ((and (eq op '|elt|) (consp sig))
       (setq tmp1 (reverse sig))
       (setq sel (qfirst tmp1))
       (setq lt (nreverse (qrest tmp1)))
```

```

(cond
  ((stringp sel)
   (setq id (intern sel))
   (if |$insideCapsuleFunctionIfTrue|
       (setq |$e| (|makeLiteral| id |$e|))
       (setq env (|makeLiteral| id env)))
   (|addModemap1| op mc (append lt (list id)) pred fn env))
  (t (|addModemap1| op mc sig pred fn env))))
((and (eq op '|setelt|) (consp sig))
 (setq tmp1 (reverse sig))
 (setq v (qfirst tmp1))
 (setq sel (qsecond tmp1))
 (setq lt (nreverse (qcddr tmp1)))
 (cond
  ((stringp sel) (setq id (intern sel))
   (if |$insideCapsuleFunctionIfTrue|
       (setq |$e| (|makeLiteral| id |$e|))
       (setq env (|makeLiteral| id env)))
   (|addModemap1| op mc (append lt (list id v)) pred fn env))
  (t (|addModemap1| op mc sig pred fn env))))
(t (|systemErrorHere| "addEltModemap")))))

```

defun mkNewModemapList

```

[member p??]
[assoc p??]
[mergeModemap p267]
[nreverse0 p??]
[insertModemap p267]
[$InteractiveMode p??]
[$forceAdd p??]

```

— defun mkNewModemapList —

```

(defun |mkNewModemapList| (mc sig pred fn curModemapList env filenameOrNil)
  (let (map entry oldMap opred result)
    (declare (special |$InteractiveMode| |$forceAdd|))
    (setq entry
      (cons (setq map (cons mc sig)) (cons (list pred fn) filenameOrNil)))
    (cond
     ((|member| entry curModemapList) curModemapList)
     ((and (setq oldMap (|assoc| map curModemapList))
           (consp oldMap) (consp (qrest oldMap))
           (consp (qsecond oldMap))
           (consp (qcddr oldMap))

```

```

      (eq (qcddadr oldMap) nil)
      (equal (qcadadr oldMap) fn))
    (setq opred (qcaadr oldMap))
    (cond
     (|$forceAdd| (|mergeModemap| entry curModemapList env))
     ((eq opred t) curModemapList)
     (t
      (when (and (not (eq pred t)) (not (equal pred opred)))
        (setq pred (list 'or pred opred)))
      (dolist (x curModemapList (nreverse0 result))
        (push
         (if (equal x oldMap)
              (cons map (cons (list pred fn) filenameOrNil))
              x)
          result))))))
    (|$InteractiveMode|
     (|insertModemap| entry curModemapList))
    (t
     (|mergeModemap| entry curModemapList env))))

```

defun insertModemap

— defun insertModemap —

```

(defun |insertModemap| (new mmList)
  (if (null mmList) (list new) (cons new mmList)))

```

defun mergeModemap

[isSuperDomain p255]
 [TruthP p268]
 [\$forceAdd p??]

— defun mergeModemap —

```

(defun |mergeModemap| (entry modemapList env)
  (let (mc sig pred mcp sigp predp newmm mm)
    (declare (special |$forceAdd|))
    ; break out the condition, signature, and predicate fields of the new entry
    (setq mc (caar entry))

```

```

(setq sig (cdar entry))
(setq pred (caadr entry))
(seq
  ; walk across the successive tails of the modemap list
  (do ((mmtail modemapList (cdr mmtail)))
      ((atom mmtail) nil)
      (setq mcp (caaar mmtail))
      (setq sigp (cdaar mmtail))
      (setq predp (caadar mmtail))
      (cond
        ((or (equal mc mcp) (|isSuperDomain| mcp mc env))
         ; if this is a duplicate condition
         (exit
          (progn
            (setq newmm nil)
            (setq mm modemapList)
            ; copy the unique modemap terms
            (loop while (not (eq mm mmtail)) do
              (setq newmm (cons (car mm) newmm))
              (setq mm (cdr mm)))
            ; if the conditions and signatures are equal
            (when (and (equal mc mcp) (equal sig sigp))
              ; we only need one of these unless the conditions are hairy
              (cond
                ((and (null |$forceAdd|) (|TruthP| predp))
                 ; the new predicate buys us nothing
                 (setq entry nil)
                 (return modemapList))
                ((|TruthP| pred)
                 ; the thing we matched against is useless, by comparison
                 (setq mmtail (cdr mmtail)))))
              (setq modemapList (nconc (nreverse newmm) (cons entry mmtail)))
              (setq entry nil)
              (return modemapList))))))
  ; if the entry is still defined, add it to the modemap
  (if entry
    (append modemapList (list entry)
      modemapList))))

```

defun TruthP

— defun TruthP —

```

(defun |TruthP| (x)
  (cond

```



```

((null x) nil)
((eq x t) t)
((and (consp x) (eq (qfirst x) 'quote)) t)
(t nil)))

```

defun evalAndSub

```

[isCategory p??]
[substNames p270]
[contained p??]
[put p??]
[get p??]
[getOperationAlist p269]
[$lhsOfColon p??]

```

— defun evalAndSub —

```

(defun |evalAndSub| (domainName viewName functorForm form |$e|)
  (declare (special |$e|))
  (let (|$lhsOfColon| opAlist substAlist)
    (declare (special |$lhsOfColon|))
    (setq |$lhsOfColon| domainName)
    (cond
      ((|isCategory| form)
       (list (|substNames| domainName viewName functorForm (elt form 1)) |$e|))
      (t
       (when (contained '$$ form)
         (setq |$e| (|put| '$$ '|model| (|get| '$ '|model| |$e|) |$e|)))
       (setq opAlist (|getOperationAlist| domainName functorForm form))
       (setq substAlist (|substNames| domainName viewName functorForm opAlist))
       (list substAlist |$e|))))))

```

defun getOperationAlist

```

[getdatabase p??]
[isFunctor p253]
[systemError p??]
[compMakeCategoryObject p213]
[stackMessage p??]
[$e p??]

```

```
[$domainShell p??]
[$insideFunctorIfTrue p??]
[$functorForm p??]
```

— defun getOperationAlist —

```
(defun |getOperationAlist| (name functorForm form)
  (let (u tt)
    (declare (special |$e| |$domainShell| |$insideFunctorIfTrue| |$functorForm|))
    (when (and (atom name) (getdatabase name 'niladic))
      (setq functorform (list functorForm)))
    (cond
      ((and (setq u (|isFunction| functorForm))
        (null (and |$insideFunctorIfTrue|
          (equal (first functorForm) (first |$functorForm|)))))
        u)
      ((and |$insideFunctorIfTrue| (eq name '$))
        (if |$domainShell|
          (elt |$domainShell| 1)
          (|systemError| "$ has no shell now")))
      ((setq tt (|compMakeCategoryObject| form |$e|))
        (setq |$e| (third tt))
        (elt (first tt) 1))
      (t
        (|stackMessage| (list 'not a category form: | form|))))))
```

—————

defvar \$FormalMapVariableList

— initvars —

```
(defvar |$FormalMapVariableList|
  '(\#1 \#2 \#3 \#4 \#5 \#6 \#7 \#8 \#9 \#10 \#11 \#12 \#13 \#14 \#15))
```

—————

defun substNames

```
[isCategoryPackageName p214]
[eqsubstlist p??]
[nreverse0 p??]
[$FormalMapVariableList p270]
```

— defun substNames —

```
(defun |substNames| (domainName viewName functorForm opalist)
  (let (nameForDollar sel pos modemapform tmp0 tmp1)
    (declare (special |$FormalMapVariableList|))
    (setq functorForm (subst '$$ '$ functorForm))
    (setq nameForDollar
      (if (|isCategoryPackageName| functorForm)
          (second functorForm)
          domainName))
    ; following calls to SUBSTQ must copy to save RPLAC's in
    ; putInLocalDomainReferences
    (dolist (term
              (eqsubstlist (ifcdr functorForm) |$FormalMapVariableList| opalist)
              (nreverse0 tmp0)))
      (setq tmp1 (reverse term))
      (setq sel (caar tmp1))
      (setq pos (caddr tmp1))
      (setq modemapform (nreverse (cdr tmp1)))
      (push
        (append
          (subst '$ '$$ (subst nameForDollar '$ modemapform))
          (list
            (list sel viewName (if (eq domainName '$) pos (cadar modemapform))))
          tmp0))))
```

—

defun augModemapsFromCategoryRep

```
[evalAndSub p269]
[isCategory p??]
[compilerMessage p??]
[putDomainsInScope p255]
[assoc p??]
[addModemap p273]
[$base p??]
```

— defun augModemapsFromCategoryRep —

```
(defun |augModemapsFromCategoryRep|
  (domainName repDefn functorBody categoryForm env)
  (labels (
    (redefinedList (op z)
      (let (result)
        (dolist (u z result)
```

```

      (setq result (or result (redefined op u))))))
(redefined (opname u)
  (let (op z result)
    (when (consp u)
      (setq op (qfirst u))
      (setq z (qrest u))
      (cond
        ((eq op 'def) (equal opname (caar z)))
        ((member op '(progn seq)) (redefinedList opname z))
        ((eq op 'cond)
          (dolist (v z result)
            (setq result (or result (redefinedList opname (cdr v))))))))))
(let (fnAlist tmp1 repFnAlist catform lhs op sig cond fnsel u)
  (declare (special |$base|))
  (setq tmp1 (|evalAndSub| domainName domainName domainName categoryForm env))
  (setq fnAlist (car tmp1))
  (setq env (cadr tmp1))
  (setq tmp1 (|evalAndSub| '|Rep| '|Rep| repDefn (|getmode| repDefn env) env))
  (setq repFnAlist (car tmp1))
  (setq env (cadr tmp1))
  (setq catform
    (if (|isCategory| categoryForm) (elt categoryForm 0) categoryForm))
  (|compilerMessage| (list '|Adding| domainName '| modemap|))
  (setq env (|putDomainsInScope| domainName env))
  (setq |$base| 4)
  (dolist (term fnAlist)
    (setq lhs (car term))
    (setq op (caar term))
    (setq sig (cadar term))
    (setq cond (cadr term))
    (setq fnsel (caddr term))
    (setq u (|assoc| (subst '|Rep| domainName lhs :test #'equal) repFnAlist))
    (if (and u (null (redefinedList op functorBody)))
      (setq env (|addModemap| op domainName sig cond (caddr u) env))
      (setq env (|addModemap| op domainName sig cond fnsel env))))
  env)))

```

6.4 Maintaining Modemaps

defun addModemapKnown

```

[addModemap0 p274]
[$e p??]
[$insideCapsuleFunctionIfTrue p??]

```

```
[$CapsuleModemapFrame p??]
```

— **defun addModemapKnown** —

```
(defun |addModemapKnown| (op mc sig pred fn |$e|)
  (declare (special |$e| |$CapsuleModemapFrame| |$insideCapsuleFunctionIfTrue|))
  (if (eq |$insideCapsuleFunctionIfTrue| t)
      (progn
        (setq |$CapsuleModemapFrame|
          (|addModemap0| op mc sig pred fn |$CapsuleModemapFrame|)
          |$e|)
        (|addModemap0| op mc sig pred fn |$e|)))
```

defun addModemap

```
[addModemap0 p274]
[knownInfo p??]
[$e p??]
[$InteractiveMode p??]
[$insideCapsuleFunctionIfTrue p??]
[$CapsuleModemapFrame p??]
[$CapsuleModemapFrame p??]
```

— **defun addModemap** —

```
(defun |addModemap| (op mc sig pred fn |$e|)
  (declare (special |$e| |$CapsuleModemapFrame| |$InteractiveMode|
    |$insideCapsuleFunctionIfTrue|))
  (cond
    (|$InteractiveMode| |$e|)
    (t
      (when (|knownInfo| pred) (setq pred t))
      (cond
        ((eq |$insideCapsuleFunctionIfTrue| t)
          (setq |$CapsuleModemapFrame|
            (|addModemap0| op mc sig pred fn |$CapsuleModemapFrame|)
            |$e|)
          (t
            (|addModemap0| op mc sig pred fn |$e|))))))
```

defun addModemap0

```
[addEltModemap p265]
[addModemap1 p274]
[$functorForm p??]
```

— defun addModemap0 —

```
(defun |addModemap0| (op mc sig pred fn env)
  (declare (special |$functorForm|))
  (cond
    ((and (consp |$functorForm|)
          (eq (qfirst |$functorForm|) '|CategoryDefaults|)
          (eq mc '$))
     env)
    ((or (eq op '|elt|) (eq op '|setelt|))
     (|addEltModemap| op mc sig pred fn env))
    (t (|addModemap1| op mc sig pred fn env))))
```

defun addModemap1

```
[getProplist p??]
[mkNewModemapList p266]
[lassoc p??]
[augProplist p??]
[unErrorRef p??]
[addBinding p??]
```

— defun addModemap1 —

```
(defun |addModemap1| (op mc sig pred fn env)
  (let (currentProplist newModemapList newProplist newProplistp)
    (when (eq mc '|Rep|) (setq sig (subst '$ '|Rep| sig :test #'equal)))
    (setq currentProplist (or (|getProplist| op env) nil))
    (setq newModemapList
      (|mkNewModemapList| mc sig pred fn
        (lassoc '|modemap| currentProplist) env nil))
    (setq newProplist (|augProplist| currentProplist '|modemap| newModemapList))
    (setq newProplistp (|augProplist| newProplist 'fluid t))
    (|unErrorRef| op)
    (|addBinding| op newProplistp env)))
```

6.5 Indirect called comp routines

In the **compExpression** function there is the code:

```
(if (and (atom (car x)) (setq fn (get1 (car x) 'special)))
    (funcall fn x m e)
    (|compForm| x m e)))
```

defplist compAdd plist

We set up the **compAdd** function to handle the **add** keyword by setting the **special** keyword on the **add** symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'add| 'special) 'compAdd))
```

—————

defun compAdd

The **compAdd** function expects three arguments:

1. the **form** which is an —add— specifying the domain to extend and a set of functions to be added
2. the **mode** a —Join—, which is a set of categories and domains
3. the **env** which is a list of functions and their modemaps

The bulk of the work is performed by a call to **compOrCroak** which compiles the functions in the **add** form capsule.

The **compAdd** function returns a triple, the result of a call to **compCapsule**.

1. the **compiled capsule** which is a progn form which returns the domain
2. the **mode** from the input argument
3. the **env** prepended with the signatures of the functions in the body of the **add**.

```
[comp p587]
[compSubDomain1 p350]
[nreverse0 p??]
[NRTgetLocalIndex p215]
[compTuple2Record p277]
```

```

[compOrCroak p586]
[compCapsule p278]
[/editfile p??]
[$addForm p??]
[$addFormLhs p??]
[$EmptyMode p176]
[$NRTaddForm p??]
[$packagesUsed p??]
[$functorForm p??]
[$bootStrapMode p??]

```

— defun compAdd —

```

(defun compAdd (form mode env)
  (let (|$addForm| |$addFormLhs| code domainForm predicate tmp3 tmp4)
    (declare (special |$addForm| |$addFormLhs| |$EmptyMode| |$NRTaddForm|
                      |$packagesUsed| |$functorForm| |$bootStrapMode| /editfile))
    (setq |$addForm| (second form))
    (cond
      ((eq |$bootStrapMode| t)
       (cond
         ((and (consp |$addForm|) (eq (qfirst |$addForm|) '|@Tuple|))
          (setq code nil))
         (t
          (setq tmp3 (|comp| |$addForm| mode env))
          (setq code (first tmp3))
          (setq mode (second tmp3))
          (setq env (third tmp3)) tmp3))
        (list
         (list 'cond
              (list '|$bootStrapMode| code)
              (list 't
                   (list '|systemError|
                        (list 'list (mkq (car |$functorForm|)) "from"
                                     (mkq (|namestring| /editfile))
                                     "needs to be compiled"))))
              mode env))
          (t
           (setq |$addFormLhs| |$addForm|)
           (cond
             ((and (consp |$addForm|) (eq (qfirst |$addForm|) '|SubDomain|)
                  (consp (qrest |$addForm|)) (consp (qcddr |$addForm|))
                  (eq (qcdddr |$addForm|) nil))
              (setq domainForm (second |$addForm|))
              (setq predicate (third |$addForm|))
              (setq |$packagesUsed| (cons domainForm |$packagesUsed|))
              (setq |$NRTaddForm| domainForm)
              (|NRTgetLocalIndex| domainForm)
              ; need to generate slot for add form since all $ go-get

```



```

; slots will need to access it
(setq tmp3 (|compSubDomain1| domainForm predicate mode env))
(setq |$addForm| (first tmp3))
(setq env (third tmp3)) tmp3)
(t
 (setq |$packagesUsed|
  (if (and (consp |$addForm|) (eq (qfirst |$addForm|) '@Tuple|))
    (append (qrest |$addForm|) |$packagesUsed|)
    (cons |$addForm| |$packagesUsed|)))
 (setq |$NRTaddForm| |$addForm|)
 (setq tmp3
  (cond
   ((and (consp |$addForm|) (eq (qfirst |$addForm|) '@Tuple|))
    (setq |$NRTaddForm|
     (cons '@Tuple|
      (dolist (x (cdr |$addForm|) (nreverse0 tmp4))
        (push (|NRTgetLocalIndex| x) tmp4))))
    (|compOrCroak| (|compTuple2Record| |$addForm|) |$EmptyMode| env))
   (t
    (|compOrCroak| |$addForm| |$EmptyMode| env))))
 (setq |$addForm| (first tmp3))
 (setq env (third tmp3))
 tmp3))
(|compCapsule| (third form) mode env))))

```

defun compTuple2Record

— defun compTuple2Record —

```

(defun |compTuple2Record| (u)
  (let ((i 0))
    (cons '|Record|
      (loop for x in (rest u)
        collect (list '|:| (incf i) x)))))

```

defplist compCapsule plist

We set up the `compCapsule` function to handle the `capsule` keyword by setting the `special` keyword on the `capsule` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'capsule 'special) '|compCapsule|))
```

defun compCapsule

```
[bootStrapError p219]
[compCapsuleInner p278]
[addDomain p252]
[editfile p??]
[$insideExpressionIfTrue p??]
[$functorForm p??]
[$bootStrapMode p??]
```

— defun compCapsule —

```
(defun |compCapsule| (form mode env)
  (let (|$insideExpressionIfTrue| itemList)
    (declare (special |$insideExpressionIfTrue| |$functorForm| /editfile
                     |$bootStrapMode|))
    (setq itemList (cdr form))
    (cond
      ((eq |$bootStrapMode| t)
       (list (|bootStrapError| |$functorForm| /editfile) mode env))
      (t
       (setq |$insideExpressionIfTrue| nil)
       (|compCapsuleInner| itemList mode (|addDomain| '$ env))))))
```

defun compCapsuleInner

```
[addInformation p??]
[compCapsuleItems p279]
[processFunctor p279]
[mkpf p??]
[$getDomainCode p??]
[$signature p??]
[$form p??]
[$addForm p??]
[$insideCategoryPackageIfTrue p??]
[$insideCategoryIfTrue p??]
[$functorLocalParameters p??]
```

— defun compCapsuleInner —

```
(defun |compCapsuleInner| (form mode env)
  (let (localParList data code)
    (declare (special |$getDomainCode| |$signature| |$form| |$addForm|
                      |$insideCategoryPackageIfTrue| |$insideCategoryIfTrue|
                      |$functorLocalParameters|))
    (setq env (|addInformation| mode env))
    (setq data (cons 'progn form))
    (setq env (|compCapsuleItems| form nil env))
    (setq localParList |$functorLocalParameters|)
    (when |$addForm| (setq data (list 'add |$addForm| data)))
    (setq code
      (if (and |$insideCategoryIfTrue| (null |$insideCategoryPackageIfTrue|))
          data
          (|processFunctor| |$form| |$signature| data localParList env)))
    (cons (mkpf (append |$getDomainCode| (list code)) 'progn) (list mode env))))
```

—————

defun processFunctor

```
[error p??]
[buildFunctor p??]
```

— defun processFunctor —

```
(defun |processFunctor| (form signature data localParList e)
  (cond
    ((and (consp form) (eq (qrest form) nil)
      (eq (qfirst form) '|CategoryDefaults|))
      (|error| '|CategoryDefaults is a reserved name|))
    (t (|buildFunctor| form signature data localParList e))))
```

—————

defun compCapsuleItems

The variable data appears to be unbound at runtime. Optimized code won't check for this but interpreted code fails. We should PROVE that data is unbound at runtime but have not done so yet. Rather than remove the code entirely (since there MIGHT be a path where it is used) we check for the runtime bound case and assign `$myFunctorBody` if data has a value.

The `compCapsuleInner` function in this file LOOKS like it sets data and expects code to manipulate the assigned data structure. Since we can't be sure we take the least disruptive course of action.

```
[compSingleCapsuleItem p280]
[$top-level p??]
[$myFunctorBody p??]
[$signatureOfForm p??]
[$suffix p??]
[$e p??]
[$pred p??]
[$e p??]
```

— **defun compCapsuleItems** —

```
(defun |compCapsuleItems| (itemlist |$predl| |$e|)
  (declare (special |$predl| |$e|))
  (let ($top_level |$myFunctorBody| |$signatureOfForm| |$suffix|)
    (declare (special $top_level |$myFunctorBody| |$signatureOfForm| |$suffix|))
    (setq $top_level nil)
    (setq |$myFunctorBody| nil)
    (when (boundp '|data|) (setq |$myFunctorBody| |data|))
    (setq |$signatureOfForm| nil)
    (setq |$suffix| 0)
    (loop for item in itemlist do
      (setq |$e| (|compSingleCapsuleItem| item |$predl| |$e|))
      |$e|))
```

—————

defun compSingleCapsuleItem

```
[doit p??]
[$pred p??]
[$e p??]
[macroExpandInPlace p178]
```

— **defun compSingleCapsuleItem** —

```
(defun |compSingleCapsuleItem| (item |$predl| |$e|)
  (declare (special |$predl| |$e|))
  (|doIt| (|macroExpandInPlace| item |$e|) |$predl|)
  |$e|)
```

—————

defun doIt

```

[lastnode p??]
[compSingleCapsuleItem p280]
[isDomainForm p348]
[stackWarning p??]
[doIt p281]
[compOrCroak p586]
[stackSemanticError p??]
[bright p??]
[member p??]
[—isFunction p??]
[insert p??]
[opOf p??]
[get p??]
[NRTgetLocalIndex p215]
[sublis p??]
[compOrCroak p586]
[sayBrightly p??]
[formatUnabbreviated p??]
[doItIf p285]
[isMacro p286]
[put p??]
[cannotDo p??]
[$predl p??]
[$e p??]
[$EmptyMode p176]
[$NonMentionableDomainNames p??]
[$functorLocalParameters p??]
[$functorsUsed p??]
[$packagesUsed p??]
[$NRTopt p??]
[$Representation p??]
[$LocalDomainAlist p??]
[$QuickCode p??]
[$signatureOfForm p??]
[$genno p??]
[$e p??]
[$functorLocalParameters p??]
[$functorsUsed p??]
[$packagesUsed p??]
[$Representation p??]
[$LocalDomainAlist p??]

```

— defun doIt —

```

(defun |doIt| (item |$predl|)
  (declare (special |$predl|))
  (prog ($genno x rhs lhsp lhs rhsp rhsCode z tmp1 tmp2 tmp6 op body tt
        functionPart u code)
    (declare (special $genno |$e| |$EmptyMode| |$signatureOfForm|
                      |$QuickCode| |$LocalDomainAlist| |$Representation|
                      |$NRTopt| |$packagesUsed| |$functorsUsed|
                      |$functorLocalParameters| |$NonMentionableDomainNames|))
    (setq $genno 0)
    (cond
     ((and (consp item) (eq (qfirst item) 'seq) (consp (qrest item)))
      (progn (setq tmp6 (reverse (qrest item))) t)
      (consp tmp6) (consp (qfirst tmp6))
      (eq (qcaar tmp6) '|exit|)
      (consp (qcдар tmp6))
      (equal (qcadar tmp6) 1)
      (consp (qcddar tmp6))
      (eq (qcdddar tmp6) nil))
      (setq x (qcaddar tmp6))
      (setq z (qrest tmp6))
      (setq z (nreverse z))
      (rplaca item 'progn)
      (rplaca (lastnode item) x)
      (loop for it1 in (rest item)
        do (setq |$e| (|compSingleCapsuleItem| it1 |$predl| |$e|))))
    ((|isDomainForm| item |$e|)
     (setq u (list '|import| (cons (car item) (cdr item))))
     (|stackWarning| (list '|Use: import | (cons (car item) (cdr item))))
     (rplaca item (car u))
     (rplacd item (cdr u))
     (|doIt| item |$predl|))
    ((and (consp item) (eq (qfirst item) 'let) (consp (qrest item)))
     (consp (qcddr item)))
     (setq lhs (qsecond item))
     (setq rhs (qthird item))
     (cond
      ((null (progn
               (setq tmp2 (|compOrCroak| item |$EmptyMode| |$e|))
               (and (consp tmp2)
                    (progn
                     (setq code (qfirst tmp2))
                     (and (consp (qrest tmp2))
                          (progn
                           (and (consp (qcddr tmp2))
                                (eq (qcdddar tmp2) nil)
                                (PROGN
                                 (setq |$e| (qthird tmp2))
                                 t))))))))
       (|stackSemanticError|
        (cons '|cannot compile assigned value to| (|bright| lhs))

```

```

    nil))
  ((null (and (consp code) (eq (qfirst code) 'let)
    (progn
      (and (consp (qrest code))
        (progn
          (setq lhsp (qsecond code))
          (and (consp (qcddr code))))))
      (atom (qsecond code))))
    (cond
      ((and (consp code) (eq (qfirst code) 'progn))
        (|stackSemanticError|
          (list '|multiple assignment | item '| not allowed|)
          nil))
      (t
        (rplaca item (car code))
        (rplacd item (cdr code))))))
  (t
    (setq lhs lhsp)
    (cond
      ((and (null (|member| (ifcar rhs) |$NonMentionableDomainNames|))
        (null (member lhs |$functorLocalParameters|)))
        (setq |$functorLocalParameters|
          (append |$functorLocalParameters| (list lhs))))
      (cond
        ((and (consp code) (eq (qfirst code) 'let)
          (progn
            (setq tmp2 (qrest code))
            (and (consp tmp2)
              (progn
                (setq tmp6 (qrest tmp2))
                (and (consp tmp6)
                  (progn
                    (setq rhsp (qfirst tmp6))
                    t))))))
            (|isDomainForm| rhsp |$e|))
          (cond
            ((|isFunctor| rhsp)
              (setq |$functorsUsed| (|insert| (|opOf| rhsp) |$functorsUsed|))
              (setq |$packagesUsed| (|insert| (list (|opOf| rhsp))
                |$packagesUsed|))))
            (cond
              ((eq lhs '|Rep|)
                (setq |$Representation| (elt (|get| '|Rep| '|value| |$e|) 0))
                (cond
                  ((eq |$NRTopt| t)
                    (|NRTgetLocalIndex| |$Representation|))
                  (t nil))))
              (setq |$LocalDomainAlist|
                (cons (cons lhs
                  (sublis |$LocalDomainAlist| (elt (|get| lhs '|value| |$e|) 0)))

```

```

    |$LocalDomainAlist|))))
(cond
  ((and (consp code) (eq (qfirst code) 'let))
    (rplaca item (if |$QuickCode| 'qsetrefv 'setelt))
    (setq rhsCode rhsp)
    (rplacd item (list '$ (|NRTgetLocalIndex| lhs) rhsCode)))
  (t
    (rplaca item (car code))
    (rplacd item (cdr code))))))
((and (consp item) (eq (qfirst item) '|:|') (consp (qrest item))
  (consp (qcddr item)) (eq (qcdddr item) nil))
  (setq tmp1 (|compOrCroak| item |$EmptyMode| |$e|))
  (setq |$e| (caddr tmp1))
  tmp1)
((and (consp item) (eq (qfirst item) '|import|')
  (loop for dom in (qrest item)
    do (|sayBrightly| (cons "    importing " (|formatUnabbreviated| dom))))
  (setq tmp1 (|compOrCroak| item |$EmptyMode| |$e|))
  (setq |$e| (caddr tmp1))
  (rplaca item 'progn)
  (rplacd item nil))
((and (consp item) (eq (qfirst item) 'if))
  (|doItIf| item |$predl| |$e|))
((and (consp item) (eq (qfirst item) '|where|') (consp (qrest item)))
  (|compOrCroak| item |$EmptyMode| |$e|))
((and (consp item) (eq (qfirst item) 'mdef))
  (setq tmp1 (|compOrCroak| item |$EmptyMode| |$e|))
  (setq |$e| (caddr tmp1)) tmp1)
((and (consp item) (eq (qfirst item) 'def) (consp (qrest item))
  (consp (qsecond item)))
  (setq op (qcaadr item))
  (cond
    ((setq body (|isMacro| item |$e|))
      (setq |$e| (|put| op '|macro| body |$e|)))
    (t
      (setq tt (|compOrCroak| item |$EmptyMode| |$e|))
      (setq |$e| (caddr tt))
      (rplaca item '|CodeDefine|)
      (rplacd (cadr item) (list |$signatureOfForm|))
      (setq functionPart (list '|dispatchFunction| (car tt)))
      (rplaca (cddr item) functionPart)
      (rplacd (cddr item) nil))))
  ((setq u (|compOrCroak| item |$EmptyMode| |$e|))
    (setq code (car u))
    (setq |$e| (caddr u))
    (rplaca item (car code))
    (rplacd item (cdr code)))
  (t (|cannotDo|))))))

```


defun doItIf

```
[comp p587]
[userError p??]
[compSingleCapsuleItem p280]
[getSuccessEnvironment p318]
[localExtras p??]
[rplaca p??]
[rplacd p??]
[$e p??]
[$functorLocalParameters p??]
[$pred1 p??]
[$e p??]
[$functorLocalParameters p??]
[$getDomainCode p??]
[$Boolean p??]
```

— defun doItIf —

```
(defun |doItIf| (item |$pred1| |$e|)
  (declare (special |$pred1| |$e|))
  (labels (
    (localExtras (oldFLP)
      (let (oldFLPp flp1 gv ans nils n)
        (declare (special |$functorLocalParameters| |$getDomainCode|))
        (unless (eq oldFLP |$functorLocalParameters|)
          (setq flp1 |$functorLocalParameters|)
          (setq oldFLPp oldFLP)
          (setq n 0)
          (loop while oldFLPp
            do
              (setq oldFLPp (cdr oldFLPp))
              (setq n (1+ n)))
          (setq nils (setq ans nil))
          (loop for u in flp1
            do
              (if (or (atom u)
                (let (result)
                  (loop for v in |$getDomainCode|
                    do
                      (setq result (or result
                        (and (consp v) (consp (qrest v))
                          (equal (qsecond v) u))))
                  result)))
              ))
          ; Now we have to add code to compile all the elements of
          ; functorLocalParameters that were added during the conditional compilation
```

```

      (setq nils (cons u nils))
      (progn
        (setq gv (gensym))
        (setq ans (cons (list 'let gv u) ans))
        (setq nils (CONS gv nils))))
      (setq n (1+ n)))
    (setq |$functorLocalParameters| (append oldFLP (nreverse nils)))
    (nreverse ans))))))
(let (p x y olde tmp1 pp xp oldFLP yp)
(declare (special |$functorLocalParameters| |$Boolean|))
  (setq p (second item))
  (setq x (third item))
  (setq y (fourth item))
  (setq olde |$e|)
  (setq tmp1
    (or (|comp| p |$Boolean| |$e|)
        (|userError| (list "not a Boolean:" p))))
  (setq pp (first tmp1))
  (setq |$e| (third tmp1))
  (setq oldFLP |$functorLocalParameters|)
  (unless (eq x '|noBranch|)
    (|compSingleCapsuleItem| x |$predl| (|getSuccessEnvironment| p |$e|))
    (setq xp (localExtras oldFLP)))
  (setq oldFLP |$functorLocalParameters|)
  (unless (eq y '|noBranch|)
    (|compSingleCapsuleItem| y |$predl| (|getInverseEnvironment| p olde))
    (setq yp (localExtras oldFLP)))
  (rplaca item 'cond)
  (rplacd item (list (cons pp (cons x xp)) (cons ''t (cons y yp))))))

```

—————→

defun isMacro

[get p??]

— defun isMacro —

```

(defun |isMacro| (x env)
  (let (op args signature body)
    (when
      (and (consp x) (eq (qfirst x) 'def) (consp (qrest x))
           (consp (qsecond x)) (consp (qcddr x))
           (consp (qcdddr x))
           (consp (qcddddd x))
           (eq (qrest (qcddddd x)) nil))
      (setq op (qcaadr x))
      (setq args (qcdadr x))

```

```
(setq signature (qthird x))
(setq body (qfirst (qcdddr x)))
(when
  (and (null (|get| op '|modemap| env))
        (null args)
        (null (|get| op '|mode| env))
        (consp signature)
        (eq (qrest signature) nil)
        (null (qfirst signature)))
  body)))
```

defplist compCase plist

We set up the `compCase` function to handle the `case` keyword by setting the `special` keyword on the `case` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|case| 'special) '|compCase|))
```

defun compCase

Will the jerk who commented out these two functions please NOT do so again. These functions ARE needed, and `case` can NOT be done by `modemap` alone. The reason is that A `case` B requires to take A evaluated, but B unevaluated. Therefore a special function is required. You may have thought that you had tested this on “failed” etc., but “failed” evaluates to it’s own mode. Try it on `x case $` next time.

An angry JHD - August 15th., 1984 [addDomain p252]

[compCase1 p288]

[coerce p355]

— **defun compCase** —

```
(defun |compCase| (form mode env)
  (let (mp td)
    (setq mp (third form))
    (setq env (|addDomain| mp env))
    (when (setq td (|compCase1| (second form) mp env)) (|coerce| td mode))))
```

defun compCase1

[comp p587]
 [getModemapList p263]
 [nreverse0 p??]
 [modeEqual p366]
 [\$Boolean p??]
 [\$EmptyMode p176]

— defun compCase1 —

```
(defun |compCase1| (form mode env)
  (let (xp mp ep map tmp3 tmp5 tmp6 u fn)
    (declare (special |$Boolean| |$EmptyMode|))
    (when (setq tmp3 (|comp| form |$EmptyMode| env))
      (setq xp (first tmp3))
      (setq mp (second tmp3))
      (setq ep (third tmp3))
      (when
        (setq u
          (dolist (modemap (|getModemapList| 'case 2 ep) (nreverse0 tmp5))
            (setq map (first modemap))
            (when
              (and (consp map) (consp (qrest map)) (consp (qcddr map))
                (consp (qcdddr map))
                (eq (qcdddr map) nil)
                (|modeEqual| (fourth map) mode)
                (|modeEqual| (third map) mp))
              (push (second modemap) tmp5))))
          (when
            (setq fn
              (dolist (onepair u tmp6)
                (when (first onepair) (setq tmp6 (or tmp6 (second onepair))))))
              (list (list 'call fn xp) |$Boolean| ep))))))
```

—————

defplist compCat plist

We set up the `compCat` function to handle the `Record` keyword by setting the `special` keyword on the `Record` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'Record 'special) 'compCat))
```

—————

defplist compCat plist

We set up the `compCat` function to handle the `Mapping` keyword by setting the `special` keyword on the `Mapping` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|Mapping| 'special) '|compCat|))
```

—————

defplist compCat plist

We set up the `compCat` function to handle the `Union` keyword by setting the `special` keyword on the `Union` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|Union| 'special) '|compCat|))
```

—————

defun compCat

[getl p??]

— **defun compCat** —

```
(defun |compCat| (form mode env)
  (declare (ignore mode))
  (let (functorName fn tmp1 tmp2 funList op sig catForm)
    (setq functorName (first form))
    (when (setq fn (getl functorName '|makeFunctionList|))
      (setq tmp1 (funcall fn form form env))
      (setq funList (first tmp1))
      (setq env (second tmp1))
      (setq catForm
        (list '|Join| '(|SetCategory|)
          (cons 'category
            (cons '|domain|
              (dolist (item funList (nreverse0 tmp2))
                (setq op (first item))
                (setq sig (second item))
                (unless (eq op '=) (push (list 'signature op sig) tmp2)))))))
      (list form catForm env))))
```

defplist compCategory plist

We set up the `compCategory` function to handle the `category` keyword by setting the `special` keyword on the `category` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'category 'special) '|compCategory|))
```

defun compCategory

```
[resolve p365]
[compCategoryItem p291]
[mkExplicitCategoryFunction p292]
[systemErrorHere p??]
[$sigList p??]
[$atList p??]
[$top-level p??]
[$sigList p??]
[$atList p??]
```

— defun compCategory —

```
(defun |compCategory| (form mode env)
  (let ($top_level |$sigList| |$atList| domainOrPackage z rep)
    (declare (special $top_level |$sigList| |$atList|))
    (setq $top_level t)
    (cond
      ((and
        (equal (setq mode (|resolve| mode (list '|Category|)))
              (list '|Category|))
        (consp form)
        (eq (qfirst form) 'category)
        (consp (qrest form)))
        (setq domainOrPackage (second form))
        (setq z (qcddr form))
        (setq |$sigList| nil)
        (setq |$atList| nil)
        (dolist (x z) (|compCategoryItem| x nil))
        (setq rep
          (|mkExplicitCategoryFunction| domainOrPackage |$sigList| |$atList|))
        (list rep mode env))
```

```
(t
  (|systemErrorHere| "compCategory")))))
```

defun compCategoryItem

```
[compCategoryItem p291]
[mkpf p??]
[$sigList p??]
[$atList p??]
```

— defun compCategoryItem —

```
(defun |compCategoryItem| (x predl)
  (let (p e a b c predlp pred y z op sig)
    (declare (special |$sigList| |$atList|))
    (cond
      ((null x) nil)
      ; 1. if x is a conditional expression, recurse; otherwise, form the predicate
      ((and (consp x) (eq (qfirst x) 'cond)
        (consp (qrest x)) (eq (qcddr x) nil)
        (consp (qsecond x))
        (consp (qcadadr x))
        (eq (qcddadr x) nil))
        (setq p (qcaadr x))
        (setq e (qcadadr x))
        (setq predlp (cons p predl))
        (cond
          ((and (consp e) (eq (qfirst e) 'progn))
            (setq z (qrest e))
            (dolist (y z) (|compCategoryItem| y predlp)))
          (t (|compCategoryItem| e predlp))))
      ((and (consp x) (eq (qfirst x) 'if) (consp (qrest x))
        (consp (qcddr x)) (consp (qcdddr x))
        (eq (qcddddr x) nil))
        (setq a (qsecond x))
        (setq b (qthird x))
        (setq c (qfourth x))
        (setq predlp (cons a predl))
        (unless (eq b '|noBranch|)
          (cond
            ((and (consp b) (eq (qfirst b) 'progn))
              (setq z (qrest b))
              (dolist (y z) (|compCategoryItem| y predlp)))
            (t (|compCategoryItem| b predlp))))
        (cond
```

```

((eq c '|noBranch|) nil)
(t
 (setq predlp (cons (list '|not| a) predl))
 (cond
  ((and (consp c) (eq (qfirst c) 'progn))
   (setq z (qrest c))
   (dolist (y z) (|compCategoryItem| y predlp)))
  (t (|compCategoryItem| c predlp))))))
(t
 (setq pred (if predl (mkpf predl 'and) t))
 (cond
  ; 2. if attribute, push it and return
  ((and (consp x) (eq (qfirst x) 'attribute)
        (consp (qrest x)) (eq (qcddr x) nil))
   (setq y (qsecond x))
   (push (mkq (list y pred)) |$atList|))
  ; 3. it may be a list, with PROG as the CAR, and some information as the CDR
  ((and (consp x) (eq (qfirst x) 'progn))
   (setq z (qrest x))
   (dolist (u z) (|compCategoryItem| u predl)))
  (t
   ; 4. otherwise, x gives a signature for a single operator name or a list of
   ; names; if a list of names, recurse
   (cond ((eq (car x) 'signature) (car x)))
   (setq op (cadr x))
   (setq sig (cddr x))
   (cond
    ((null (atom op))
     (dolist (y op)
      (|compCategoryItem| (cons 'signature (cons y sig)) predl)))
    (t
     ; 5. branch on a single type or a signature %with source and target
     (push (mkq (list (cdr x) pred)) |$sigList|)))))))))

```

defun mkExplicitCategoryFunction

```

[mkq p??]
[union p??]
[mustInstantiate p293]
[remdup p??]
[identp p??]
[wrapDomainSub p294]

```

— defun mkExplicitCategoryFunction —


```

(defun |mkExplicitCategoryFunction| (domainOrPackage sigList atList)
  (let (body sig parameters)
    (setq body
      (list '|mkCategory| (mkq domainOrPackage)
        (cons 'list (reverse sigList))
        (cons 'list (reverse atList))
        (mkq
          (let (result)
            (loop for item in sigList
              do
                (setq sig (car (cdaadr item)))
                (setq result
                  (|union| result
                    (loop for d in sig
                      when (|mustInstantiate| d)
                        collect d))))
            result)))
          nil)))
    (setq parameters
      (remdup
        (let (result)
          (loop for item in sigList
            do
              (setq sig (car (cdaadr item)))
              (setq result
                (append result
                  (loop for x in sig
                    when (and (identp x) (not (eq x '$)))
                      collect x))))
            result)))
        (|wrapDomainSub| parameters body)))

```

defun mustInstantiate

```

[getl p??]
[$DummyFunctorNames p??]

```

— defun mustInstantiate —

```

(defun |mustInstantiate| (d)
  (declare (special |$DummyFunctorNames|))
  (and (consp d)
    (null (or (member (qfirst d) |$DummyFunctorNames|)
      (getl (qfirst d) '|makeFunctionList|))))))

```

defun wrapDomainSub

— defun wrapDomainSub —

```
(defun |wrapDomainSub| (parameters x)
  (list '|DomainSubstitutionMacro| parameters x))
```

defplist compColon plist

We set up the `compColon` function to handle the `:` keyword by setting the `special` keyword on the `:` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|:| 'special) '|compColon|))
```

defun compColon

```
[compColonInside p594]
[assoc p??]
[getDomainsInScope p254]
[isDomainForm p348]
[compColon member (vol5)]
[addDomain p252]
[isCategoryForm p??]
[unknownTypeError p253]
[compColon p294]
[eqsubstlist p??]
[take p??]
[length p??]
[nreverse0 p??]
[getmode p??]
[systemErrorHere p??]
[put p??]
[makeCategoryForm p297]
[genSomeVariable p??]
```

```

[$lhsOfColon p??]
[$noEnv p??]
[$insideFunctorIfTrue p??]
[$bootStrapMode p??]
[$FormalMapVariableList p270]
[$insideCategoryIfTrue p??]
[$insideExpressionIfTrue p??]

```

— defun compColon —

```

(defun |compColon| (form mode env)
  (let (|$lhsOfColon| argf argt tprime mprime r td op argl newTarget a
        signature tmp2 catform tmp3 g2 g5)
    (declare (special |$lhsOfColon| |$noEnv| |$insideFunctorIfTrue|
                      |$bootStrapMode| |$FormalMapVariableList|
                      |$insideCategoryIfTrue| |$insideExpressionIfTrue|))
    (setq argf (second form))
    (setq argt (third form))
    (if |$insideExpressionIfTrue|
        (|compColonInside| argf mode env argt)
        (progn
          (setq |$lhsOfColon| argf)
          (setq argt
            (cond
              ((and (atom argt)
                    (setq tprime (|assoc| argt (|getDomainsInScope| env))))
               tprime)
              ((and (|isDomainForm| argt env) (null |$insideCategoryIfTrue|))
               (unless (|member| argt (|getDomainsInScope| env))
                 (setq env (|addDomain| argt env)))
               argt)
              ((or (|isDomainForm| argt env) (|isCategoryForm| argt env))
               argt)
              ((and (consp argt) (eq (qfirst argt) '|Mapping|)
                    (progn
                     (setq tmp2 (qrest argt))
                     (and (consp tmp2)
                         (progn
                          (setq mprime (qfirst tmp2))
                          (setq r (qrest tmp2))
                          t))))
               argt)
              (t
               (|unknownTypeError| argt)
               argt)))
          (cond
            ((eq (car argf) '|listof|)
             (dolist (x (cdr argf) td)
              (setq td (|compColon| (list '|:| x argt) mode env))

```

```

    (setq env (third td))))
(t
 (setq env
  (cond
   ((and (consp argf)
        (progn
         (setq op (qfirst argf))
         (setq argl (qrest argf))
         t)
        (null (and (consp argt) (eq (qfirst argt) '|Mapping|)))))
   (setq newTarget
    (eqsubstlist (take (|#| argl) |$FormalMapVariableList|)
     (dolist (x argl (nreverse0 g2))
      (setq g2
       (cons
        (cond
         ((and (consp x) (eq (qfirst x) '|:|)
          (progn
           (setq tmp2 (qrest x))
           (and (consp tmp2)
                (progn
                 (setq a (qfirst tmp2))
                 (setq tmp3 (qrest tmp2))
                 (and (consp tmp3)
                      (eq (qrest tmp3) nil)
                      (progn
                       (setq mode (qfirst tmp3))
                       t))))))
          a)
         (t x))
        g2)))
    argt))
 (setq signature
  (cons '|Mapping|
   (cons newTarget
    (dolist (x argl (nreverse0 g5))
     (setq g5
      (cons
       (cond
        ((and (consp x) (eq (qfirst x) '|:|)
         (progn
          (setq tmp2 (qrest x))
          (and (consp tmp2)
               (progn
                (setq a (qfirst tmp2))
                (setq tmp3 (qrest tmp2))
                (and (consp tmp3)
                     (eq (qrest tmp3) nil)
                     (progn
                      (setq mode (qfirst tmp3))
                      t))))))
         (t x))
       g5)))
    signature)))

```

```

t))))))
mode)
(t
  (or (|getmode| x env)
      (|systemErrorHere| "compColonOld"))))
g5))))))
(|put| op '|mode| signature env))
(t (|put| argf '|mode| argt env)))
(cond
  ((and (null |$bootStrapMode|) |$insideFunctorIfTrue|
        (progn
          (setq tmp2 (|makeCategoryForm| argt env))
          (and (consp tmp2)
                (progn
                  (setq catform (qfirst tmp2))
                  (setq tmp3 (qrest tmp2))
                  (and (consp tmp3)
                        (eq (qrest tmp3) nil)
                        (progn
                          (setq env (qfirst tmp3))
                          t)))))))
   (setq env
    (|put| argf '|value| (list (|genSomeVariable|) argt |$noEnv|)
      env)))
  (list '|/throwAway| (|getmode| argf env) env ))))))))

```

defun makeCategoryForm

[isCategoryForm p??]
 [compOrCroak p586]
 [\$EmptyMode p176]

— defun makeCategoryForm —

```

(defun |makeCategoryForm| (c env)
  (let (tmp1)
    (declare (special |$EmptyMode|))
    (when (|isCategoryForm| c env)
      (setq tmp1 (|compOrCroak| c |$EmptyMode| env))
      (list (first tmp1) (third tmp1)))))

```

defplist compCons plist

We set up the `compCons` function to handle the `cons` keyword by setting the `special` keyword on the `cons` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'cons 'special) '|compCons|))
```

—————

defun compCons

[[compCons1](#) p298]
[[compForm](#) p600]

— **defun compCons** —

```
(defun |compCons| (form mode env)
  (or (|compCons1| form mode env) (|compForm| form mode env)))
```

—————

defun compCons1

[[comp](#) p587]
[[convert](#) p597]
[[\\$EmptyMode](#) p176]

— **defun compCons1** —

```
(defun |compCons1| (arg mode env)
  (let (mx y my yt mp mr ytp tmp1 x td)
    (declare (special |$EmptyMode|))
    (setq x (second arg))
    (setq y (third arg))
    (when (setq tmp1 (|comp| x |$EmptyMode| env))
      (setq x (first tmp1))
      (setq mx (second tmp1))
      (setq env (third tmp1))
      (cond
        ((null y)
         (|convert| (list (list 'list x) (list '|List| mx) env ) mode))
        (t
```

```

(when (setq yt (|comp| y |$EmptyMode| env))
  (setq y (first yt))
  (setq my (second yt))
  (setq env (third yt))
  (setq td
    (cond
      ((and (consp my) (eq (qfirst my) '|List|) (consp (qrest my)))
        (setq mp (second my))
        (when (setq mr (list '|List| (|resolve| mp mx)))
          (when (setq ytp (|convert| yt mr))
            (when (setq tmp1 (|convert| (list x mx (third ytp)) (second mr)))
              (setq x (first tmp1))
              (setq env (third tmp1))
              (cond
                ((and (consp (car ytp)) (eq (qfirst (car ytp)) 'list))
                  (list (cons 'list (cons x (cdr (car ytp)))) mr env))
                (t
                  (list (list 'cons x (car ytp)) mr env)))))))
      (t
        (list (list 'cons x y) (list '|Pair| mx my) env ))))
    (|convert| td mode))))))

```

defplist compConstruct plist

We set up the `compConstruct` function to handle the `construct` keyword by setting the `special` keyword on the `construct` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get '|construct| 'special) '|compConstruct|))

```

defun compConstruct

```

[modeIsAggregateOf p??]
[compList p599]
[convert p597]
[compForm p600]
[compVector p353]
[getDomainsInScope p254]

```

— defun compConstruct —

```

(defun |compConstruct| (form mode env)
  (let (z y td tp)
    (setq z (cdr form))
    (cond
      ((setq y (|modeIsAggregateOf| '|List| mode env))
       (if (setq td (|compList| z (list '|List| (cadr y)) env))
           (|convert| td mode)
           (|compForm| form mode env)))
      ((setq y (|modeIsAggregateOf| '|Vector| mode env))
       (if (setq td (|compVector| z (list '|Vector| (cadr y)) env))
           (|convert| td mode)
           (|compForm| form mode env)))
      ((setq td (|compForm| form mode env)) td)
      (t
       (dolist (d (|getDomainsInScope| env))
         (cond
           ((and (setq y (|modeIsAggregateOf| '|List| d env))
                  (setq td (|compList| z (list '|List| (cadr y)) env))
                  (setq tp (|convert| td mode)))
            (return tp))
           ((and (setq y (|modeIsAggregateOf| '|Vector| d env))
                  (setq td (|compVector| z (list '|Vector| (cadr y)) env))
                  (setq tp (|convert| td mode)))
            (return tp)))))))

```

defplist compConstructorCategory plist

We set up the `compConstructorCategory` function to handle the `ListCategory` keyword by setting the `special` keyword on the `ListCategory` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get '|ListCategory| 'special) '|compConstructorCategory|))

```

defplist compConstructorCategory plist

We set up the `compConstructorCategory` function to handle the `RecordCategory` keyword by setting the `special` keyword on the `RecordCategory` symbol property list.

— postvars —

```

(eval-when (eval load)

```



```
(setf (get '|RecordCategory| 'special) '|compConstructorCategory|))
```

deflist compConstructorCategory plist

We set up the `compConstructorCategory` function to handle the `UnionCategory` keyword by setting the `special` keyword on the `UnionCategory` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|UnionCategory| 'special) '|compConstructorCategory|))
```

deflist compConstructorCategory plist

We set up the `compConstructorCategory` function to handle the `VectorCategory` keyword by setting the `special` keyword on the `VectorCategory` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|VectorCategory| 'special) '|compConstructorCategory|))
```

defun compConstructorCategory

[[resolve p365](#)]
 [\$Category p??]

— defun compConstructorCategory —

```
(defun |compConstructorCategory| (form mode env)
  (declare (special |$Category|))
  (list form (|resolve| |$Category| mode) env))
```

defun getAbbreviation

```
[constructor? p??]
[assq p??]
[mkAbbrev p302]
[rplac p??]
[$abbreviationTable p??]
[$abbreviationTable p??]
```

— defun getAbbreviation —

```
(defun |getAbbreviation| (name c)
  (let (cname x n upc newAbbreviation)
    (declare (special |$abbreviationTable|))
    (setq cname (|constructor?| name))
    (cond
      ((setq x (assq cname |$abbreviationTable|))
       (cond
         ((setq n (assq name (cdr x)))
          (cond
            ((setq upc (assq c (cdr n)))
             (cdr upc))
            (t
             (setq newAbbreviation (|mkAbbrev| x cname))
             (rplac (cdr n) (cons (cons c newAbbreviation) (cdr n)))
             newAbbreviation)))
          (t
           (setq newAbbreviation (|mkAbbrev| x x))
           (rplac (cdr x)
                  (cons (cons name (list (cons c newAbbreviation))) (cdr x)))
           newAbbreviation)))
      (t
       (setq |$abbreviationTable|
              (cons (list cname (list name (cons c cname))) |$abbreviationTable|)
                    cname))))))
```

— —

defun mkAbbrev

```
[addSuffix p303]
[alistSize p303]
```

— defun mkAbbrev —

```
(defun |mkAbbrev| (x z)
  (|addSuffix| (|alistSize| (cdr x)) z))
```

defun addSuffix

— defun addSuffix —

```

(defun |addSuffix| (n u)
  (let (s)
    (if (alpha-char-p (elt (setq s (princ-to-string u)) (maxindex s)))
        (intern (strconc s (princ-to-string n)))
        (intern1 (strconc s (princ-to-string '|;|) (princ-to-string n)))))

```

defun alistSize

— defun alistSize —

```

(defun |alistSize| (c)
  (labels (
    (count (x level)
      (cond
        ((eq1 level 2) (|#| x))
        ((null x) 0)
        (+ (count (cdar x) (1+ level))
           (count (cdr x) level)))))
    (count c 1)))

```

defun getSignatureFromMode

```

[getmode p??]
[opOf p??]
[length p??]
[stackAndThrow p??]
[eqsubstlist p??]
[take p??]
[$FormalMapVariableList p270]

```

— defun getSignatureFromMode —

```
(defun |getSignatureFromMode| (form env)
  (let (tmp1 signature)
    (declare (special |$FormalMapVariableList|))
    (setq tmp1 (|getmode| (|opOf| form) env))
    (when (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|))
      (setq signature (qrest tmp1))
      (if (not (eql (|#| form) (|#| signature)))
          (|stackAndThrow| (list '|Wrong number of arguments: | form))
          (eqsubstlist (cdr form)
                        (take (|#| (cdr form)) |$FormalMapVariableList|
                              signature))))))
```

—————

defun getSpecialCaseAssoc

```
[|$functorForm p??|
 |$functorSpecialCases p??|
```

— defun getSpecialCaseAssoc —

```
(defun |getSpecialCaseAssoc| ()
  (declare (special |$functorSpecialCases| |$functorForm|))
  (loop for r in (rest |$functorForm|)
        for z in (rest |$functorSpecialCases|)
        when z
        collect (cons r z)))
```

—————

defun addArgumentConditions

```
[mkq p??]
[systemErrorHere p??]
[$true p??]
[$functionName p??]
[$body p??]
[$argumentConditionList p??]
[$argumentConditionList p??]
```

— defun addArgumentConditions —

```

(defun |addArgumentConditions| (|$body| |$functionName|)
  (declare (special |$body| |$functionName| |$argumentConditionList| |$true|))
  (labels (
    (fn (clist)
      (let (n untypedCondition typedCondition)
        (cond
          ((and (consp clist) (consp (qfirst clist)) (consp (qcdar clist))
              (consp (qcddar clist))
              (eq (qcdddar clist) nil))
            (setq n (qcaar clist))
            (setq untypedCondition (qcadar clist))
            (setq typedCondition (qcaddar clist))
            (list 'cond
              (list typedCondition (fn (cdr clist)))
              (list |$true|
                (list '|argumentDataError| n
                  (mkq untypedCondition) (mkq |$functionName|))))))
          ((null clist) |$body|)
          (t (|systemErrorHere| "addArgumentConditions")))))
    (if |$argumentConditionList|
      (fn |$argumentConditionList|
        |$body|)))

```

defun stripOffSubdomainConditions

```

[assoc p??]
[mkpf p??]
|$argumentConditionList p??]
|$argumentConditionList p??]

```

— defun stripOffSubdomainConditions —

```

(defun |stripOffSubdomainConditions| (margl argl)
  (let (pair (i 0))
    (declare (special |$argumentConditionList|))
    (loop for x in margl for arg in argl
      do (incf i)
      collect
      (cond
        ((and (consp x) (eq (qfirst x) '|SubDomain|) (consp (qrest x))
            (consp (qcddr x)) (eq (qcdddr x) nil))
          (cond
            ((setq pair (|assoc| i |$argumentConditionList|))
              (rplac (cadr pair) (mkpf (list (third x) (cadr pair)) 'and))

```

```

      (second x))
    (t
      (setq |$argumentConditionList|
        (cons (list i arg (third x)) |$argumentConditionList|))
      (second x))))
    (t x))))

```

defun stripOffArgumentConditions

```

[$argumentConditionList p??]
[$argumentConditionList p??]

```

— defun stripOffArgumentConditions —

```

(defun |stripOffArgumentConditions| (arg1)
  (let (condition (i 0))
    (declare (special |$argumentConditionList|))
    (loop for x in arg1
      do (incf i)
      collect
      (cond
        ((and (consp x) (eq (qfirst x) '|\\|') (consp (qrest x))
          (consp (qcddr x)) (eq (qcdddr x) nil))
          (setq condition (subst '|#1| (second x) (third x) :test #'equal))
          (setq |$argumentConditionList|
            (cons (list i (second x) condition) |$argumentConditionList|))
          (second x))
        (t x))))))

```

defun getSignature

Try to return a signature. If there isn't one, complain and return nil. If there are more than one then remove any that are subsumed. If there is still more than one complain else return the only signature. [get p??]

```

[length p??]
[remdup p??]
[knownInfo p??]
[getmode p??]
[say p??]
[printSignature p??]

```

```
[SourceLevelSubsume p??]
[stackSemanticError p??]
[$e p??]
```

— defun getSignature —

```
(defun |getSignature| (op argModeList |$e|)
  (declare (special |$e|))
  (let (mmList pred u tmp1 dc sig sigl)
    (setq mmList (|get| op '|modemap| |$e|))
    (cond
      ((eq 1
        (|#| (setq sigl (remdup
          (loop for item in mmList
            do
              (setq dc (caar item))
              (setq sig (cdar item))
              (setq pred (caadr item))
              when (and (eq dc '$) (equal (cdr sig) argModeList) (|knownInfo| pred))
                collect sig))))))
        (car sigl))
      ((null sigl)
        (cond
          ((progn
              (setq tmp1 (setq u (|getmode| op |$e|)))
              (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|)))
            (qrest tmp1))
          (t
            (say "***** USER ERROR *****")
            (say "available signatures for " op ": ")
            (cond
              ((null mmList) (say "    NONE"))
              (t
                (loop for item in mmList
                  do (|printSignature| '|      | op (cdar item)))
                (|printSignature| '|NEED | op (cons '? argModeList))))
            nil)))
      (t
        ; Before we complain about duplicate signatures, we should
        ; check that we do not have for example, a partial - as
        ; well as a total one. SourceLevelSubsume should do this
        (loop for u in sigl do
          (loop for v in sigl
            when (null (equal u v))
            do (when (|SourceLevelSubsume| u v) (setq sigl (|delete| v sigl)))))
        (cond
          ((eq 1 (|#| sigl)) (car sigl))
          (t
            (|stackSemanticError|
```

```
(list '|duplicate signatures for | op '|: | argModeList) nil))))))
```

defun checkAndDeclare

```
[getArgumentMode p309]
[modeEqual p366]
[put p??]
[sayBrightly p??]
[bright p??]
```

— defun checkAndDeclare —

```
(defun |checkAndDeclare| (argl form sig env)
  (let (m1 stack)
    (loop for a in argl for m in (rest sig)
      do
        (if (setq m1 (|getArgumentMode| a env))
          (if (null (|modeEqual| m1 m))
            (setq stack
              (cons '| | (append (|bright| a)
                (cons "must have type "
                  (cons m
                    (cons " not "
                      (cons m1
                        (cons '|%1| stack))))))))
            (setq env (|put| a '|mode| m env)))
          (when stack
            (|sayBrightly|
              (cons " Parameters of "
                (append (|bright| (car form))
                  (cons " are of wrong type:"
                    (cons '|%1| stack))))))
            env)))
```

defun hasSigInTargetCategory

```
[getArgumentMode p309]
[remdup p??]
[length p??]
[getSignatureFromMode p303]
[stackWarning p??]
```



```
[compareMode2Arg p??]
[bright p??]
[$domainShell p??]
```

— **defun hasSigInTargetCategory** —

```
(defun |hasSigInTargetCategory| (argl form opsig env)
  (labels (
    (fn (opName sig opsig mList form)
      (declare (special |$op|))
      (and
        (and
          (and (equal opName |$op|) (equal (|#| sig) (|#| form)))
          (or (null opsig) (equal opsig (car sig))))
        (let ((result t))
          (loop for x in mList for y in (rest sig)
            do (setq result (and result (or (null x) (|modeEqual| x y)))))
          result))))
    (let (mList potentialSigList c sig)
      (declare (special |$domainShell|))
      (setq mList
        (loop for x in argl
          collect (|getArgumentMode| x env)))
      (setq potentialSigList
        (remdup
          (loop for item in (elt |$domainShell| 1)
            when (fn (caar item) (cadar item) opsig mList form)
              collect (cadar item))))
      (setq c (|#| potentialSigList))
      (cond
        ((eql 1 c) (car potentialSigList))
        ((eql 0 c)
          (when (equal (|#| (setq sig (|getSignatureFromMode| form env))) (|#| form))
            sig))
        ((> c 1)
          (setq sig (car potentialSigList))
          (|stackWarning|
            (cons '|signature of lhs not unique:|
              (append (|bright| sig) (list '|chosen|))))
          sig)
        (t nil))))))
```

—————

defun getArgumentMode

```
[get p??]
```

— **defun getArgumentMode** —

```
(defun |getArgumentMode| (x e)
  (if (stringp x) x (|get| x '|mode| e)))
```

—————

defplist compElt plist

We set up the `compElt` function to handle the `elt` keyword by setting the `special` keyword on the `elt` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|elt| 'special) '|compElt|))
```

—————

defun compElt

```
[compForm p600]
[isDomainForm p348]
[addDomain p252]
[getModemapListFromDomain p264]
[length p??]
[stackMessage p??]
[stackWarning p??]
[convert p597]
[opOf p??]
[getDeltaEntry p??]
[$One p??]
[$Zero p??]
```

— **defun compElt** —

```
(defun |compElt| (form mode env)
  (let (aDomain anOp mmList n modemap sig pred val)
    (declare (special |$One| |$Zero|))
    (setq anOp (third form))
    (setq aDomain (second form))
    (cond
      ((null (and (consp form) (eq (qfirst form) '|elt|)
                  (consp (qrest form)) (consp (qcddr form))
                  (eq (qcdddr form) nil))))
```

```

(|compForm| form mode env))
((eq aDomain '|Lisp|)
 (list (cond
        ((equal anOp '$Zero|) 0)
        ((equal anOp '$One|) 1)
        (t anOp))
      mode env))
(|isDomainForm| aDomain env)
(setq env (|addDomain| aDomain env))
(setq mmList (|getModemapListFromDomain| anOp 0 aDomain env))
(setq modemap
 (progn
  (setq n (|#| mmList))
  (cond
   ((eql 1 n) (elt mmList 0))
   ((eql 0 n)
    (|stackMessage|
     (list "Operation " anOp "missing from domain: "
           aDomain nil)))
   nil)
  (t
   (|stackWarning|
    (list "more than 1 modemap for: " anOp " with dc="
          aDomain " ==>" mmList ))
    (elt mmList 0))))))
(when modemap
 (setq sig (first modemap))
 (setq pred (caadr modemap))
 (setq val (cadadr modemap))
 (unless (and (not (eql (|#| sig) 2))
              (null (and (consp val) (eq (qfirst val) '|elt|))))
  (setq val (|genDeltaEntry| (cons (|opOf| anOp) modemap)))
  (|convert| (list (list '|call| val) (second sig) env) mode))))
(t
 (|compForm| form mode env))))

```

defplist compExit plist

We set up the `compExit` function to handle the `exit` keyword by setting the `special` keyword on the `exit` symbol property list.

— **postvars** —

```

(eval-when (eval load)
 (setf (get '|exit| 'special) '|compExit|))

```

defun compExit

```
[comp p587]
[modifyModeStack p622]
[stackMessageIfNone p??]
[$exitModeStack p??]
```

— defun compExit —

```
(defun |compExit| (form mode env)
  (let (exitForm index m1 u)
    (declare (special |$exitModeStack|))
    (setq index (1- (second form)))
    (setq exitForm (third form))
    (cond
      ((null |$exitModeStack|)
       (|comp| exitForm mode env))
      (t
       (setq m1 (elt |$exitModeStack| index))
       (setq u (|comp| exitForm m1 env))
       (cond
         (u
          (|modifyModeStack| (second u) index)
          (list (list ' |TAGGEDexit| index u) mode env))
         (t
          (|stackMessageIfNone|
           (list ' |cannot compile exit expression| exitForm ' |in mode| m1))))))))))
```

defplist compHas plist

We set up the `compHas` function to handle the `has` keyword by setting the `special` keyword on the `has` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get ' |has| 'special) ' |compHas|))
```

defun compHas

```
[chaseInferences p??]
[compHasFormat p313]
[coerce p355]
[$e p??]
[$e p??]
[$Boolean p??]
```

— defun compHas —

```
(defun |compHas| (pred mode |$e|)
  (declare (special |$e| |$Boolean|))
  (let (a b predCode)
    (setq a (second pred))
    (setq b (third pred))
    (setq |$e| (|chaseInferences| pred |$e|))
    (setq predCode (|compHasFormat| pred))
    (|coerce| (list predCode |$Boolean| |$e|) mode)))
```

—————

defun compHasFormat

```
[take p??]
[length p??]
[sublislis p??]
[comp p587]
[mkList p314]
[mkDomainConstructor p??]
[isDomainForm p348]
[$FormalMapVariableList p270]
[$EmptyMode p176]
[$e p??]
[$form p??]
[$EmptyEnvironment p??]
```

— defun compHasFormat —

```
(defun |compHasFormat| (pred)
  (let (olda b argl formals tmp1 a)
    (declare (special |$EmptyEnvironment| |$e| |$EmptyMode|
                      |$FormalMapVariableList| |$form|))
    (when (eq (car pred) '|has|) (car pred))
    (setq olda (second pred))
    (setq b (third pred))
```

```

(setq arg1 (rest |$form|))
(setq formals (take (|#| arg1) |$FormalMapVariableList|))
(setq a (sublislis arg1 formals olda))
(setq tmp1 (|comp| a |$EmptyModel| |$e|))
(when tmp1
  (setq a (car tmp1))
  (setq a (sublislis formals arg1 a))
  (cond
    ((and (consp b) (eq (qfirst b) 'attribute) (consp (qrest b))
      (eq (qcddr b) nil))
      (list '|HasAttribute| a (list 'quote (qsecond b))))
    ((and (consp b) (eq (qfirst b) 'signature) (consp (qrest b))
      (consp (qcddr b)) (eq (qcddr b) NIL))
      (list '|HasSignature| a
        (|mkList|
          (list (MKQ (qsecond b))
            (|mkList|
              (loop for type in (qthird b)
                collect (|mkDomainConstructor| type)))))))
    ((|isDomainForm| b |$EmptyEnvironment|)
      (list 'equal a b))
    (t
      (list '|HasCategory| a (|mkDomainConstructor| b))))))

```

defun mkList

— defun mkList —

```

(defun |mkList| (u)
  (when u (cons 'list u)))

```

defplist compIf plist

We set up the `compIf` function to handle the `if` keyword by setting the `special` keyword on the `if` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get 'if 'special) '|compIf|))

```

defun compIf

```
[canReturn p316]
[intersectionEnvironment p??]
[compBoolean p318]
[compFromIf p316]
[resolve p365]
[coerce p355]
[quotify p??]
[$Boolean p??]
```

— defun compIf —

```
(defun |compIf| (form mode env)
  (labels (
    (environ (bEnv cEnv b c env)
      (cond
        ((|canReturn| b 0 0 t)
          (if (|canReturn| c 0 0 t) (|intersectionEnvironment| bEnv cEnv) bEnv))
        ((|canReturn| c 0 0 t) cEnv)
        (t env))))
    (let (a b c tmp1 xa ma Ea Einv Tb xb mb Eb Tc xc mc Ec xbp x returnEnv)
      (declare (special |$Boolean|))
      (setq a (second form))
      (setq b (third form))
      (setq c (fourth form))
      (when (setq tmp1 (|compBoolean| a |$Boolean| env))
        (setq xa (first tmp1))
        (setq ma (second tmp1))
        (setq Ea (third tmp1))
        (setq Einv (fourth tmp1))
        (when (setq Tb (|compFromIf| b mode Ea))
          (setq xb (first Tb))
          (setq mb (second Tb))
          (setq Eb (third Tb))
          (when (setq Tc (|compFromIf| c (|resolve| mb mode) Einv))
            (setq xc (first Tc))
            (setq mc (second Tc))
            (setq Ec (third Tc))
            (when (setq xbp (|coerce| Tb mc))
              (setq x (list 'if xa (first xbp) xc))
              (setq returnEnv (environ (third xbp) Ec (first xbp) xc env))
              (list x mc returnEnv))))))))))
```

defun compFromIf

[comp p587]

— defun compFromIf —

```
(defun |compFromIf| (a m env)
  (if (eq a '|noBranch|)
      (list '|noBranch| m env)
      (|comp| a m env)))
```

—————

defun canReturn

[say p??]

[canReturn p316]

[systemErrorHere p??]

— defun canReturn —

```
(defun |canReturn| (expr level exitCount ValueFlag)
  (labels (
    (findThrow (gs expr level exitCount ValueFlag)
      (cond
        ((atom expr) nil)
        ((and (consp expr) (eq (qfirst expr) 'throw) (consp (qrest expr))
          (equal (qsecond expr) gs) (consp (qcddr expr))
          (eq (qcddr expr) nil))
        t)
      ((and (consp expr) (eq (qfirst expr) 'seq))
        (let (result)
          (loop for u in (qrest expr)
            do (setq result
              (or result
                (findThrow gs u (1+ level) exitCount ValueFlag))))
          result)))
    (t
      (let (result)
        (loop for u in (rest expr)
          do (setq result
            (or result
              (findThrow gs u level exitCount ValueFlag))))
        result))))))
  (let (op gs)
    (cond
      ((atom expr) (and ValueFlag (equal level exitCount)))
```



```

((eq (setq op (car expr)) 'quote) (and ValueFlag (equal level exitCount)))
((eq op '|TAGGEDexit|)
 (cond
  ((and (consp expr) (consp (qrest expr)) (consp (qcddr expr))
        (eq (qcdddr expr) nil))
   (|canReturn| (car (third expr)) level (second expr)
                 (equal (second expr) level))))
  ((and (equal level exitCount) (null ValueFlag))
   nil)
  ((eq op 'seq)
   (let (result)
     (loop for u in (rest expr)
            do (setq result (or result (|canReturn| u (1+ level) exitCount nil))))
    result))
  ((eq op '|TAGGEDreturn|) nil)
  ((eq op 'catch)
   (cond
    ((findThrow (second expr) (third expr) level
                 exitCount ValueFlag)
     t)
    (t
     (|canReturn| (third expr) level exitCount ValueFlag))))
  ((eq op 'cond)
   (cond
    ((equal level exitCount)
     (let (result)
       (loop for u in (rest expr)
              do (setq result (or result
                                   (|canReturn| (|last| u) level exitCount ValueFlag))))
      result))
    (t
     (let (outer)
       (loop for v in (rest expr)
              do (setq outer (or outer
                                   (let (inner)
                                     (loop for u in v
                                            do (setq inner
                                                  (or inner
                                                       (findThrow gs u level exitCount ValueFlag))))
                                    inner))))
        outer))))
  ((eq op 'if)
   (and (consp expr) (consp (qrest expr)) (consp (qcddr expr))
        (consp (qcdddr expr))
        (eq (qcddddd expr) nil))
   (cond
    ((null (|canReturn| (second expr) 0 0 t))
     (say "IF statement can not cause consequents to be executed")
     (|pp| expr)))
    (or (|canReturn| (second expr) level exitCount nil)

```

```

      (|canReturn| (third expr) level exitCount ValueFlag)
      (|canReturn| (fourth expr) level exitCount ValueFlag)))
((atom op)
 (let ((result t))
  (loop for u in expr
    do (setq result
      (and result (|canReturn| u level exitCount ValueFlag))))
  result))
((and (consp op) (eq (qfirst op) 'xlam) (consp (qrest op))
  (consp (qcddr op)) (eq (qcdddr op) nil))
 (let ((result t))
  (loop for u in expr
    do (setq result
      (and result (|canReturn| u level exitCount ValueFlag))))
  result))
(t (|systemErrorHere| "canReturn")))))))

```

defun compBoolean

[comp p587]
 [getSuccessEnvironment p318]
 [getInverseEnvironment p320]

— defun compBoolean —

```

(defun |compBoolean| (p mode env)
  (let (tmp1 pp)
    (when (setq tmp1 (OR (|comp| p mode env)))
      (setq pp (car tmp1))
      (setq mode (cadr tmp1))
      (setq env (caddr tmp1))
      (list pp mode (|getSuccessEnvironment| p env)
        (|getInverseEnvironment| p env)))))

```

defun getSuccessEnvironment

[isDomainForm p348]
 [put p??]
 [identp p??]
 [getProplist p??]
 [comp p587]

```
[consProplistOf p??]
[removeEnv p??]
[addBinding p??]
[get p??]
[$EmptyEnvironment p??]
[$EmptyMode p176]
```

— defun getSuccessEnvironment —

```
(defun |getSuccessEnvironment| (a env)
  (let (id currentProplist tt newProplist x m)
    (declare (special |$EmptyMode| |$EmptyEnvironment|))
    (cond
      ((and (consp a) (eq (qfirst a) '|has|) (CONSP (qrest a))
        (consp (qcddr a)) (eq (qcdddr a) nil))
        (if
          (and (identp (second a)) (|isDomainForm| (third a) |$EmptyEnvironment|))
          (|put| (second a) '|specialCase| (third a) env)
          env))
      ((and (consp a) (eq (qfirst a) '|is|) (consp (qrest a))
        (consp (qcddr a)) (eq (qcdddr a) nil))
        (setq id (qsecond a))
        (setq m (qthird a))
        (cond
          ((and (identp id) (|isDomainForm| m |$EmptyEnvironment|))
            (setq env (|put| id '|specialCase| m env))
            (setq currentProplist (|getProplist| id env))
            (setq tt (|comp| m |$EmptyMode| env))
            (when tt
              (setq env (caddr tt))
              (setq newProplist
                (|consProplistOf| id currentProplist '|value|
                  (cons m (cdr (|removeEnv| tt))))))
            (|addBinding| id newProplist env)))
          (t env)))
      ((and (consp a) (eq (qfirst a) '|case|) (consp (qrest a))
        (consp (qcddr a)) (eq (qcdddr a) nil)
        (identp (qsecond a)))
        (setq x (qsecond a))
        (setq m (qthird a))
        (|put| x '|condition| (cons a (|get| x '|condition| env)) env))
      (t env))))
```

defun getInverseEnvironment

```

[identp p??]
[isDomainForm p348]
[put p??]
[get p??]
[member p??]
[mkpf p??]
[delete p??]
[getUnionMode p321]
[$EmptyEnvironment p??]

```

— **defun getInverseEnvironment** —

```

(defun |getInverseEnvironment| (a env)
  (let (op argl x m oldpred tmp1 zz newpred)
    (declare (special |$EmptyEnvironment|))
    (cond
      ((atom a) env)
      (t
       (setq op (car a))
       (setq argl (cdr a))
       (cond
         ((eq op '|has|)
          (setq x (car argl))
          (setq m (cadr argl))
          (cond
            ((and (identp x) (|isDomainForm| m |$EmptyEnvironment|))
             (|put| x '|specialCase| m env))
            (t env)))
         ((and (consp a) (eq (qfirst a) '|case|) (consp (qrest a))
              (consp (qcddr a)) (eq (qcdddr a) nil)
              (identp (qsecond a)))
          (setq x (qsecond a))
          (setq m (qthird a))
          (setq tmp1 (|get| x '|condition| env))
          (cond
            ((and tmp1 (consp tmp1) (eq (qrest tmp1) nil) (consp (qfirst tmp1))
              (eq (qcaar tmp1) '|or|) (|member| a (qcdr tmp1)))
             (setq oldpred (qcdr tmp1))
             (|put| x '|condition| (list (mkpf (|delete| a oldpred) '|or|) env))
            (t
             (setq tmp1 (|getUnionMode| x env))
             (setq zz (|delete| m (qrest tmp1)))
             (loop for u in zz
                  when (and (consp u) (eq (qfirst u) '|:|)
                          (consp (qrest u)) (equal (qsecond u) m))
                  do (setq zz (|delete| u zz)))
             (setq newpred

```

```

      (mkpf (loop for mp in zz collect (list '|case| x mp)) 'or))
      (|put| x '|condition|
        (cons newpred (|get| x '|condition| env)) env))))
    (t env))))))

```

defun getUnionMode

[isUnionMode p321]
[getmode p??]

— defun getUnionMode —

```

(defun |getUnionMode| (x env)
  (let (m)
    (setq m (when (atom x) (|getmode| x env)))
    (when m (|isUnionMode| m env))))

```

defun isUnionMode

[getmode p??]
[get p??]

— defun isUnionMode —

```

(defun |isUnionMode| (m env)
  (let (mp v tmp1)
    (cond
      ((and (consp m) (eq (qfirst m) '|Union|)) m)
      ((progn
         (setq tmp1 (setq mp (|getmode| m env)))
         (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|)
              (consp (qrest tmp1)) (eq (qcddr tmp1) nil)
              (consp (qsecond tmp1))
              (eq (qcaadr tmp1) '|UnionCategory|)))
         (second mp))
      ((setq v (|get| (if (eq m '$) '|Rep| m) '|value| env))
       (when (and (consp (car v)) (eq (qfirst (car v)) '|Union|)) (car v))))))

```

defplist compImport plist

We set up the `compImport` function to handle the `import` keyword by setting the `special` keyword on the `import` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|import| 'special) '|compImport|))
```

—————

defun compImport

```
[addDomain p252]
[$NoValueMode p176]
```

— **defun compImport** —

```
(defun |compImport| (form mode env)
  (declare (ignore mode))
  (declare (special |$NoValueMode|))
  (dolist (dom (cdr form)) (setq env (|addDomain| dom env)))
  (list '|/throwAway| |$NoValueMode| env))
```

—————

defplist compIs plist

We set up the `compIs` function to handle the `is` keyword by setting the `special` keyword on the `is` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|is| 'special) '|compIs|))
```

—————

defun compIs

```
[comp p587]
[coerce p355]
[$Boolean p??]
```

[[\\$EmptyMode](#) p176]

— **defun compIs** —

```
(defun |compIs| (form mode env)
  (let (a b aval am tmp1 bval bm td)
    (declare (special |$Boolean| |$EmptyMode|))
    (setq a (second form))
    (setq b (third form))
    (when (setq tmp1 (|comp| a |$EmptyMode| env))
      (setq aval (first tmp1))
      (setq am (second tmp1))
      (setq env (third tmp1))
      (when (setq tmp1 (|comp| b |$EmptyMode| env))
        (setq bval (first tmp1))
        (setq bm (second tmp1))
        (setq env (third tmp1))
        (setq td (list (list '|domainEqual| aval bval) |$Boolean| env ))
        (|coerce| td mode))))))
```

—————

defplist compJoin plist

We set up the `compJoin` function to handle the `Join` keyword by setting the `special` keyword on the `Join` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|Join| 'special) '|compJoin|))
```

—————

defun compJoin

```
[nreverse0 p??]
[compForMode p325]
[stackSemanticError p??]
[nreverse0 p??]
[isCategoryForm p??]
[union p??]
[compJoin,getParms p??]
[wrapDomainSub p294]
[convert p597]
```

[\$Category p??]

— defun compJoin —

```
(defun |compJoin| (form mode env)
  (labels (
    (getParms (y env)
      (cond
        ((atom y)
          (when (|isDomainForm| y env) (list y)))
        ((and (consp y) (eq (qfirst y) 'length)
          (consp (qrest y)) (eq (qcddr y) nil))
          (list y (second y)))
        (t (list y))))))
  (let (arg1 catList p1 tmp3 tmp4 tmp5 body parameters catListp td)
    (declare (special |$Category|))
    (setq arg1 (cdr form))
    (setq catList
      (dolist (x arg1 (nreverse0 tmp3))
        (push (car (or (|compForMode| x |$Category| env) (return '|failed|)))
          tmp3)))
    (cond
      ((eq catList '|failed|)
        (|stackSemanticError| (list '|cannot form Join of: | arg1) nil))
      (t
        (setq catListp
          (dolist (x catList (nreverse0 tmp4))
            (setq tmp4
              (cons
                (cond
                  ((|isCategoryForm| x env)
                    (setq parameters
                      (|union|
                        (dolist (y (cdr x) tmp5)
                          (setq tmp5 (append tmp5 (getParms y env))))
                      parameters))
                  x)
                ((and (consp x) (eq (qfirst x) '|DomainSubstitutionMacro|)
                  (consp (qrest x)) (consp (qcddr x))
                  (eq (qcdddr x) nil))
                  (setq p1 (second x))
                  (setq body (third x))
                  (setq parameters (|union| p1 parameters)) body)
                ((and (consp x) (eq (qfirst x) '|mkCategory|))
                  x)
                ((and (atom x) (equal (|getmode| x env) |$Category|))
                  x)
                (t
                  (|stackSemanticError| (list '|invalid argument to Join: | x) nil)
                  x))
```



```

      tmp4))))
    (setq td (list (|wrapDomainSub| parameters (cons '|Join| catListp))
                  |$Category| env))
    (|convert| td mode))))))

```

defun compForMode

[comp p587]

[$\$compForModeIfTrue$ p??]

— defun compForMode —

```

(defun |compForMode| (x m e)
  (let (|$compForModeIfTrue|)
    (declare (special |$compForModeIfTrue|))
    (setq |$compForModeIfTrue| t)
    (|comp| x m e)))

```

defplist compLambda plist

We set up the `compLambda` function to handle the `+->` keyword by setting the `special` keyword on the `+->` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get '|+->| 'special) '|compLambda|))

```

defun compLambda

[argsToSig p621]

[compAtSign p361]

[stackAndThrow p??]

— defun compLambda —

```

(defun |compLambda| (form mode env)

```

```

(let (vl body tmp1 tmp2 tmp3 target args arg1 sig1 ress)
  (setq vl (second form))
  (setq body (third form))
  (cond
    ((and (consp vl) (eq (qfirst vl) '|:|))
     (progn
      (setq tmp1 (qrest vl))
      (and (consp tmp1)
        (progn
          (setq args (qfirst tmp1))
          (setq tmp2 (qrest tmp1))
          (and (consp tmp2)
            (eq (qrest tmp2) nil)
            (progn
              (setq target (qfirst tmp2))
              t))))))
    (when (and (consp args) (eq (qfirst args) '|@Tuple|))
      (setq args (qrest args)))
    (cond
      ((listp args)
       (setq tmp3 (|argsToSig| args))
       (setq arg1 (first tmp3))
       (setq sig1 (second tmp3))
       (cond
         (sig1
          (setq ress
            (compAtSign
             (list '@
              (list '+-> arg1 body)
              (cons '|Mapping| (cons target sig1))) mode env))
          ress)
         (t (|stackAndThrow| (list '|compLambda| form )))))
      (t (|stackAndThrow| (list '|compLambda| form )))))
    (t (|stackAndThrow| (list '|compLambda| form ))))))

```

defplist compLeave plist

We set up the `compLeave` function to handle the `leave` keyword by setting the `special` keyword on the `leave` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get '|leave| 'special) '|compLeave|))

```

defun compLeave

```
[comp p587]
[modifyModeStack p622]
[$exitModeStack p??]
[$leaveLevelStack p??]
```

— **defun compLeave** —

```
(defun |compLeave| (form mode env)
  (let (level x index u)
    (declare (special |$exitModeStack| |$leaveLevelStack|))
    (setq level (second form))
    (setq x (third form))
    (setq index
      (- (1- (|#| |$exitModeStack|)) (elt |$leaveLevelStack| (1- level))))
    (when (setq u (|comp| x (elt |$exitModeStack| index) env))
      (|modifyModeStack| (second u) index)
      (list (list ' |TAGGEDexit| index u) mode env ))))
```

—————

defplist compMacro plist

We set up the `compMacro` function to handle the `MDEF` keyword by setting the `special` keyword on the `MDEF` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'mdef 'special) '|compMacro|))
```

—————

defun compMacro

```
[formatUnabbreviated p??]
[sayBrightly p??]
[put p??]
[macroExpand p178]
[$macroIfTrue p??]
[$NoValueMode p176]
[$EmptyMode p176]
```

— **defun compMacro** —

```

(defun |compMacro| (form mode env)
  (let (|$macroIfTrue| lhs signature specialCases rhs prhs)
    (declare (special |$macroIfTrue| |$NoValueMode| |$EmptyMode|))
    (setq |$macroIfTrue| t)
    (setq lhs (second form))
    (setq signature (third form))
    (setq specialCases (fourth form))
    (setq rhs (fifth form))
    (setq prhs
      (cond
        ((and (consp rhs) (eq (qfirst rhs) 'category))
         (list "-- the constructor category"))
        ((and (consp rhs) (eq (qfirst rhs) '|Join|))
         (list "-- the constructor category"))
        ((and (consp rhs) (eq (qfirst rhs) 'capsule))
         (list "-- the constructor capsule"))
        ((and (consp rhs) (eq (qfirst rhs) '|add|))
         (list "-- the constructor capsule"))
        (t (|formatUnabbreviated| rhs))))
    (|sayBrightly|
     (cons "    processing macro definition "
       (append (|formatUnabbreviated| lhs)
         (cons " ==> "
           (append prhs (list " "))))))
    (when (or (equal mode |$EmptyMode|) (equal mode |$NoValueMode|))
      (list '|/throwAway| |$NoValueMode|
        (|put| (CAR lhs) '|macro| (|macroExpand| rhs env) env))))))

```

defplist compPretend plist

We set up the `compPretend` function to handle the `pretend` keyword by setting the `special` keyword on the `pretend` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get '|pretend| 'special) '|compPretend|))

```

defun compPretend

[addDomain p252]
 [comp p587]

```
[opOf p??]
[stackSemanticError p??]
[stackWarning p??]
[$newCompilerUnionFlag p??]
[$EmptyMode p176]
```

— **defun compPretend** —

```
(defun |compPretend| (form mode env)
  (let (x tt warningMessage td tp)
    (declare (special |$newCompilerUnionFlag| |$EmptyMode|))
    (setq x (second form))
    (setq tt (third form))
    (setq env (|addDomain| tt env))
    (when (setq td (or (|comp| x tt env) (|comp| x |$EmptyMode| env)))
      (when (equal (second td) tt)
        (setq warningMessage (list '|pretend| tt '| -- should replace by @|)))
      (cond
        ((and |$newCompilerUnionFlag|
              (eq (|opOf| (second td)) '|Union|)
              (not (eq (|opOf| mode) '|Union|)))
          (|stackSemanticError|
           (list '|cannot pretend | x '| of mode | (second td) '| to mode | mode)
           nil))
        (t
         (setq td (list (first td) tt (third td)))
         (when (setq tp (|coerce| td mode))
           (when warningMessage (|stackWarning| warningMessage)
             tp))))))
```

—————

deflist compQuote plist

We set up the `compQuote` function to handle the `QUOTE` keyword by setting the `special` keyword on the `QUOTE` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'quote 'special) '|compQuote|))
```

—————

defun compQuote

— defun compQuote —

```
(defun |compQuote| (form mode env)
  (list form mode env))
```

—————

defplist compReduce plist

We set up the `compReduce` function to handle the `REDUCE` keyword by setting the `special` keyword on the `REDUCE` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'reduce 'special) '|compReduce|))
```

—————

defun compReduce

```
[compReduce1 p330]
[$formalArgList p??]
```

— defun compReduce —

```
(defun |compReduce| (form mode env)
  (declare (special |$formalArgList|))
  (|compReduce1| form mode env |$formalArgList|))
```

—————

defun compReduce1

```
[systemError p??]
[nreverse0 p??]
[compIterator p??]
[comp p587]
[parseTran p101]
[getIdentity p??]
```

```

[$sideEffectsList p??]
[$until p??]
[$initList p??]
[$Boolean p??]
[$e p??]
[$sendTestList p??]

```

— defun compReduce1 —

```

(defun |compReduce1| (form mode env |$formalArgList|)
  (declare (special |$formalArgList|))
  (let (|$sideEffectsList| |$until| |$initList| |$sendTestList| collectForm
        collectOp body op itl acc afterFirst bodyVal part1 part2 part3 id
        identityCode untilCode finalCode tmp1 tmp2)
    (declare (special |$sideEffectsList| |$until| |$initList| |$Boolean| |$e|
                      |$sendTestList|))
    (setq op (second form))
    (setq collectForm (fourth form))
    (setq collectOp (first collectForm))
    (setq tmp1 (reverse (cdr collectForm)))
    (setq body (first tmp1))
    (setq itl (nreverse (cdr tmp1)))
    (when (stringp op) (setq op (intern op)))
    (cond
      ((null (member collectOp '(collect collectv collectvec)))
       (|systemError| (list '|illegal reduction form:| form)))
      (t
       (setq |$sideEffectsList| nil)
       (setq |$until| nil)
       (setq |$initList| nil)
       (setq |$sendTestList| nil)
       (setq |$e| env)
       (setq itl
        (dolist (x itl (nreverse0 tmp2))
          (setq tmp1 (or (|compIterator| x |$e|) (return '|failed|)))
          (setq |$e| (second tmp1))
          (push (elt tmp1 0) tmp2)))
        (unless (eq itl '|failed|)
          (setq env |$e|)
          (setq acc (gensym))
          (setq afterFirst (gensym))
          (setq bodyVal (gensym))
          (when (setq tmp1 (|comp| (list 'let bodyVal body ) mode env))
            (setq part1 (first tmp1))
            (setq mode (second tmp1))
            (setq env (third tmp1))
            (when (setq tmp1 (|comp| (list 'let acc bodyVal) mode env))
              (setq part2 (first tmp1))
              (setq env (third tmp1))

```

```

(when (setq tmp1
      (|comp| (list 'let acc (|parseTran| (list op acc bodyVal)))
              mode env))
  (setq part3 (first tmp1))
  (setq env (third tmp1))
  (when (setq identityCode
        (if (setq id (|getIdentity| op env))
            (car (|comp| id mode env))
            (list '|IdentityError| (mkq op))))
    (setq finalCode
      (cons 'progn
        (cons (list 'let afterFirst nil)
          (cons
            (cons 'repeat
              (append itl
                (list
                  (list 'progn part1
                    (list 'if afterFirst part3
                      (list 'progn part2 (list 'let afterFirst (mkq t)))) nil))))
              (list (list 'if afterFirst acc identityCode ))))))
        (when |$until|
          (setq tmp1 (|comp| |$until| |$Boolean| env))
          (setq untilCode (first tmp1))
          (setq env (third tmp1))
          (setq finalCode
            (subst (list 'until untilCode) '|$until| finalCode :test #'equal)))
          (list finalCode mode env ))))))))

```

defplist compRepeatOrCollect plist

We set up the `compRepeatOrCollect` function to handle the `COLLECT` keyword by setting the `special` keyword on the `COLLECT` symbol property list.

— postvars —

```

(eval-when (eval load)
  (setf (get 'collect 'special) '|compRepeatOrCollect|))

```

defplist compRepeatOrCollect plist

We set up the `compRepeatOrCollect` function to handle the `REPEAT` keyword by setting the `special` keyword on the `REPEAT` symbol property list.

— postvars —


```
(eval-when (eval load)
  (setf (get 'repeat 'special) '|compRepeatOrCollect|))
```

defun compRepeatOrCollect

```
[length p??]
[compIterator p??]
[modeIsAggregateOf p??]
[stackMessage p??]
[compOrCroak p586]
[comp p587]
[coerceExit p361]
[ p??]
[ p??]
[$until p??]
[$Boolean p??]
[$NoValueMode p176]
[$exitModeStack p??]
[$leaveLevelStack p??]
[$formalArgList p??]
```

— defun compRepeatOrCollect —

```
(defun |compRepeatOrCollect| (form mode env)
  (labels (
    (fn (form |$exitModeStack| |$leaveLevelStack| |$formalArgList| env)
      (declare (special |$exitModeStack| |$leaveLevelStack| |$formalArgList|))
      (let (|$until| body itl xp targetMode repeatOrCollect bodyMode bodyp mp tmp1
            untilCode ep itlp formp u mpp tmp2)
        (declare (special |$Boolean| |$until| |$NoValueMode| ))
        (setq |$until| nil)
        (setq repeatOrCollect (car form))
        (setq tmp1 (reverse (cdr form)))
        (setq body (car tmp1))
        (setq itl (nreverse (cdr tmp1)))
        (setq itlp
          (dolist (x itl (nreverse0 tmp2))
            (setq tmp1 (or (|compIterator| x env) (return '|failed|))))
            (setq xp (first tmp1))
            (setq env (second tmp1))
            (push xp tmp2))))
        (unless (eq itlp '|failed|)
          (setq targetMode (car |$exitModeStack|))
          (setq bodyMode
```

```

(if (eq repeatOrCollect 'collect)
  (cond
    ((eq targetMode '|$EmptyMode|)
     '|$EmptyMode|)
    ((setq u (|modeIsAggregateOf| '|List| targetMode env))
     (second u))
    ((setq u (|modeIsAggregateOf| '|PrimitiveArray| targetMode env))
     (setq repeatOrCollect 'collectv)
     (second u))
    ((setq u (|modeIsAggregateOf| '|Vector| targetMode env))
     (setq repeatOrCollect 'collectvec)
     (second u))
    (t
     (|stackMessage| "Invalid collect bodytype")
     '|failed|))
    '$NoValueMode|))
(unless (eq bodyMode '|failed|)
  (when (setq tmp1 (|compOrCroak| body bodyMode env))
    (setq bodyp (first tmp1))
    (setq mp (second tmp1))
    (setq ep (third tmp1))
    (when |$until|
      (setq tmp1 (|comp| |$until| |$Boolean| ep))
      (setq untilCode (first tmp1))
      (setq ep (third tmp1))
      (setq itlp
        (subst (list 'until untilCode) '|$until| itlp :test #'equal)))
    (setq formp (cons repeatOrCollect (append itlp (list bodyp))))
    (setq mpp
      (cond
        ((eq repeatOrCollect 'collect)
         (if (setq u (|modeIsAggregateOf| '|List| targetMode env))
             (car u)
             (list '|List| mp)))
        ((eq repeatOrCollect 'collectv)
         (if (setq u (|modeIsAggregateOf| '|PrimitiveArray| targetMode env))
             (car u)
             (list '|PrimitiveArray| mp)))
        ((eq repeatOrCollect 'collectvec)
         (if (setq u (|modeIsAggregateOf| '|Vector| targetMode env))
             (car u)
             (list '|Vector| mp)))
        (t mp)))
    (|coerceExit| (list formp mpp ep) targetMode))))))
(declare (special |$exitModeStack| |$leaveLevelStack| |$formalArgList|))
(fn form
  (cons mode |$exitModeStack|)
  (cons (|#| |$exitModeStack|) |$leaveLevelStack|)
  |$formalArgList|
  env)))

```

defplist compReturn plist

We set up the `compReturn` function to handle the `return` keyword by setting the `special` keyword on the `return` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get '|return| 'special) '|compReturn|))
```

defun compReturn

```
[stackSemanticError p??]
[userError p??]
[resolve p365]
[comp p587]
[modifyModeStack p622]
[$exitModeStack p??]
[$returnMode p??]
```

— **defun compReturn** —

```
(defun |compReturn| (form mode env)
  (let (level x index u xp mp ep)
    (declare (special |$returnMode| |$exitModeStack|))
    (setq level (second form))
    (setq x (third form))
    (cond
      ((null |$exitModeStack|)
       (|stackSemanticError|
        (list '|the return before| x '|is unnecessary|) nil)
       nil)
      ((not (eql level 1))
       (|userError| "multi-level returns not supported"))
      (t
       (setq index (max 0 (1- (|#| |$exitModeStack|))))
       (when (>= index 0)
        (setq |$returnMode|
              (|resolve| (elt |$exitModeStack| index) |$returnMode|)))
       (when (setq u (|comp| x |$returnMode| env))
        (setq xp (first u))
```

```

(setq mp (second u))
(setq ep (third u))
(when (>= index 0)
  (setq |$returnMode| (|resolve| mp |$returnMode|))
  (|modifyModeStack| mp index))
(list (list 'TAGGEDreturn| 0 u) mode ep))))))

```

defplist compSeq plist

We set up the `compSeq` function to handle the `SEQ` keyword by setting the `special` keyword on the `SEQ` symbol property list.

— **postvars** —

```

(eval-when (eval load)
  (setf (get 'seq 'special) '|compSeq|))

```

defun compSeq

```

[compSeq1 p336]
[$exitModeStack p??]

```

— **defun compSeq** —

```

(defun |compSeq| (form mode env)
  (declare (special |$exitModeStack|))
  (|compSeq1| (cdr form) (cons mode |$exitModeStack|) env))

```

defun compSeq1

```

[nreverse0 p??]
[compSeqItem p338]
[mkq p??]
[replaceExitEtc p337]
[$exitModeStack p??]
[$insideExpressionIfTrue p??]
[$finalEnv p??]

```

[[\\$NoValueMode](#) p176]

— **defun compSeq1** —

```
(defun |compSeq1| (form |$exitModeStack| env)
  (declare (special |$exitModeStack|))
  (let (|$insideExpressionIfTrue| |$finalEnv| tmp1 tmp2 c catchTag newform)
    (declare (special |$insideExpressionIfTrue| |$finalEnv| |$NoValueMode|))
    (setq |$insideExpressionIfTrue| nil)
    (setq |$finalEnv| nil)
    (when
      (setq c (dolist (x form (nreverse0 tmp2))
        (setq |$insideExpressionIfTrue| nil)
        (setq tmp1 (|compSeqItem| x |$NoValueMode| env))
        (unless tmp1 (return nil))
        (setq env (third tmp1))
        (push (first tmp1) tmp2)))
      (setq catchTag (mkq (gensym)))
      (setq newform
        (cons 'seq
          (|replaceExitEtc| c catchTag 'TAGGEDexit (elt |$exitModeStack| 0))))
      (list (list 'catch catchTag newform)
        (elt |$exitModeStack| 0) |$finalEnv|))))
```

—————

defun replaceExitEtc

[[rplac](#) p??]
 [[replaceExitEtc](#) p337]
 [[intersectionEnvironment](#) p??]
 [[convertOrCroak](#) p338]
 [[\\$finalEnv](#) p??]
 [[\\$finalEnv](#) p??]

— **defun replaceExitEtc** —

```
(defun |replaceExitEtc| (x tag opFlag opMode)
  (declare (special |$finalEnv|))
  (cond
    ((atom x) nil)
    ((and (consp x) (eq (qfirst x) 'quote)) nil)
    ((and (consp x) (equal (qfirst x) opFlag) (consp (qrest x))
      (consp (qcddr x)) (eq (qcdddr x) nil))
      (|rplac| (caaddr x) (|replaceExitEtc| (caaddr x) tag opFlag opMode))
      (cond
        ((eq1 (second x) 0)
```

```

(setq |$finalEnv|
  (if |$finalEnv|
    (|intersectionEnvironment| |$finalEnv| (third (third x)))
    (third (third x))))
(|rplac| (car x) 'throw)
(|rplac| (cadr x) tag)
(|rplac| (caddr x) (car (|convertOrCroak| (caddr x) opMode))))
(t
  (|rplac| (cadr x) (1- (cadr x)))))
((and (consp x) (consp (qrest x)) (consp (qcddr x))
  (eq (qcddr x) nil)
  (member (qfirst x) '(|TAGGEDreturn| |TAGGEDexit|)))
  (|rplac| (car (caddr x))
    (|replaceExitEtc| (car (caddr x)) tag opFlag opMode)))
(t
  (|replaceExitEtc| (car x) tag opFlag opMode)
  (|replaceExitEtc| (cdr x) tag opFlag opMode)))
x)

```

defun convertOrCroak

[convert p597]
[userError p??]

— defun convertOrCroak —

```

(defun |convertOrCroak| (tt m)
  (let (u)
    (if (setq u (|convert| tt m))
      u
      (|userError|
        (list 'CANNOT CONVERT: | (first tt) '|%1| ' OF MODE: | (second tt)
          '|%1| ' TO MODE: | m '|%1|))))))

```

defun compSeqItem

[comp p587]
[macroExpand p178]

— defun compSeqItem —

```
(defun |compSeqItem| (form mode env)
  (|comp| (|macroExpand| form env) mode env))
```

defplist compSetq plist

We set up the `compSetq` function to handle the `LET` keyword by setting the `special` keyword on the `LET` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'let 'special) '|compSetq|))
```

defplist compSetq plist

We set up the `compSetq` function to handle the `SETQ` keyword by setting the `special` keyword on the `SETQ` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'setq 'special) '|compSetq|))
```

defun compSetq

[`compSetq1` p339]

— **defun compSetq** —

```
(defun |compSetq| (form mode env)
  (|compSetq1| (second form) (third form) mode env))
```

defun compSetq1

[`setqSingle` p344]

[`compSetq1` `identp` (`vol5`)]

[compMakeDeclaration p622]
 [compSetq p339]
 [setqMultiple p341]
 [setqSetelt p344]
 [\$EmptyMode p176]

— defun compSetq1 —

```
(defun |compSetq1| (form val mode env)
  (let (x y ep op z)
    (declare (special |$EmptyMode|))
    (cond
      ((identp form) (|setqSingle| form val mode env))
      ((and (consp form) (eq (qfirst form) '|:|) (consp (qrest form))
        (consp (qcddr form)) (eq (qcdddr form) nil))
        (setq x (second form))
        (setq y (third form))
        (setq ep (third (|compMakeDeclaration| form |$EmptyMode| env)))
        (|compSetq| (list 'let x val) mode ep))
      ((consp form)
        (setq op (qfirst form))
        (setq z (qrest form))
        (cond
          ((eq op 'cons) (|setqMultiple| (|uncons| form) val mode env))
          ((eq op '|@Tuple|) (|setqMultiple| z val mode env))
          (t (|setqSetelt| form val mode env))))))
```

defun uncons

[uncons p340]

— defun uncons —

```
(defun |uncons| (x)
  (cond
    ((atom x) x)
    ((and (consp x) (eq (qfirst x) 'cons) (consp (qrest x))
      (consp (qcddr x)) (eq (qcdddr x) nil))
      (cons (second x) (|uncons| (third x)))))
```

defun setqMultiple

```

[nreverse0 p??]
[stackMessage p??]
[setqMultipleExplicit p343]
[genVariable p??]
[addBinding p??]
[compSetq1 p339]
[convert p597]
[put p??]
[genSomeVariable p??]
[length p??]
[mkprogn p??]
[$EmptyMode p176]
[$NoValueMode p176]
[$noEnv p??]

```

— defun setqMultiple —

```

(defun |setqMultiple| (nameList val m env)
  (labels (
    (decompose (tt len env)
      (declare (ignore len))
      (let (tmp1 z)
        (declare (special |$EmptyMode|))
        (cond
          ((and (consp tt) (eq (qfirst tt) '|Record|)
            (progn (setq z (qrest tt)) t))
          (loop for item in z
            collect (cons (second item) (third item))))
        ((progn
          (setq tmp1 (|comp| tt |$EmptyMode| env))
          (and (consp tmp1) (consp (qrest tmp1)) (consp (qsecond tmp1))
            (eq (qcaadr tmp1) '|RecordCategory|)
            (consp (qcddr tmp1)) (eq (qcdddr tmp1) nil)))
          (loop for item in z
            collect (cons (second item) (third item))))
        (t (|stackMessage| (list '|no multiple assigns to mode: | tt))))))
    (let (g m1 tt x mp selectorModePairs tmp2 assignList)
      (declare (special |$noEnv| |$EmptyMode| |$NoValueMode|))
      (cond
        ((and (consp val) (eq (qfirst val) '|cons|) (equal m |$NoValueMode|))
          (|setqMultipleExplicit| nameList (|uncons| val) m env))
        ((and (consp val) (eq (qfirst val) '|@Tuple|) (equal m |$NoValueMode|))
          (|setqMultipleExplicit| nameList (qrest val) m env))
        ; 1 create a gensym, %add to local environment, compile and assign rhs
        (t
          (setq g (|genVariable|))

```

```

(setq env (|addBinding| g nil env))
(setq tmp2 (|compSetq1| g val |$EmptyMode| env))
(when tmp2
  (setq tt tmp2)
  (setq m1 (cadr tmp2))
  (setq env (|put| g 'mode m1 env))
  (setq tmp2 (|convert| tt m))
; 1.1 --exit if result is a list
  (when tmp2
    (setq x (first tmp2))
    (setq mp (second tmp2))
    (setq env (third tmp2))
    (cond
      ((and (consp m1) (eq (qfirst m1) '|List|) (consp (qrest m1))
        (eq (qcddr m1) nil))
        (loop for y in nameList do
          (setq env
            (|put| y '|value| (list (|genSomeVariable|) (second m1) |$noEnv|
              env))))
        (|convert| (list (list 'progn x (list 'let nameList g) g) mp env) m))
      (t
        ; 2 --verify that the #nameList = number of parts of right-hand-side
        (setq selectorModePairs
          (decompose m1 (|#| nameList) env))
        (when selectorModePairs
          (cond
            ((not (eq1 (|#| nameList) (|#| selectorModePairs)))
              (|stackMessage|
                (list val '| must decompose into |
                  (|#| nameList) '| components| )))
            (t
              ; 3 --generate code
              (setq assignList
                (loop for x in nameList
                  for item in selectorModePairs
                    collect (car
                      (progn
                        (setq tmp2
                          (or (|compSetq1| x (list '|elt| g (first item))
                            (rest item) env)
                          (return '|failed|)))
                        (setq env (third tmp2))
                        tmp2))))
                (unless (eq assignList '|failed|)
                  (list (mkprogn (cons x (append assignList (list g)))) mp env)
                  ))))))))

```

defun setqMultipleExplicit

```
[stackMessage p??]
[genVariable p??]
[compSetq1 p339]
[last p??]
[$EmptyMode p176]
[$NoValueMode p176]
```

— defun setqMultipleExplicit —

```
(defun |setqMultipleExplicit| (nameList valList m env)
  (declare (ignore m))
  (let (gensymList assignList tmp1 reAssignList)
    (declare (special |$NoValueMode| |$EmptyMode|))
    (cond
      ((not (eq1 (|#| nameList) (|#| valList))))
      (|stackMessage|
        (list '|Multiple assignment error; # of items in: | nameList
              '|must = # in: | valList)))
    (t
      (setq gensymList
        (loop for name in nameList
              collect (|genVariable|)))
      (setq assignList
        (loop for g in gensymList
              for val in valList
              collect (progn
                (setq tmp1
                  (or (|compSetq1| g val |$EmptyMode| env)
                    (return '|failed|)))
                (setq env (third tmp1))
                tmp1)))
      (unless (eq assignList '|failed|)
        (setq reAssignList
          (loop for g in gensymList
                for name in nameList
                collect (progn
                  (setq tmp1
                    (or (|compSetq1| name g |$EmptyMode| env)
                      (return '|failed|)))
                  (setq env (third tmp1))
                  tmp1)))
        (unless (eq reAssignList '|failed|)
          (list
            (cons 'progn
              (append
                (loop for tt in assignList
                      collect (car tt))
```

```
(loop for tt in reAssignList
  collect (car tt)))
|$NoValueMode| (third (|last| reAssignList)))))))))
```

defun setqSetelt

[comp p587]

— defun setqSetelt —

```
(defun |setqSetelt| (form val mode env)
  (|comp| (cons '|setelt| (cons (car form) (append (cdr form) (list val))))
    mode env))
```

defun setqSingle

```
[setqSingle getProplist (vol5)]
[getmode p??]
[get p??]
[maxSuperType p348]
[comp p587]
[getmode p??]
[assignError p346]
[convert p597]
[setqSingle identp (vol5)]
[profileRecord p??]
[consProplistOf p??]
[removeEnv p??]
[setqSingle addBinding (vol5)]
[isDomainForm p348]
[isDomainInScope p??]
[stackWarning p??]
[augModemapsFromDomain1 p257]
[NRTassocIndex p346]
[isDomainForm p348]
[outputComp p347]
[$insideSetqSingleIfTrue p??]
[$QuickLet p??]
[$form p??]
```

[\$profileCompiler p??]
 [\$EmptyMode p176]
 [\$NoValueMode p176]

— defun setqSingle —

```
(defun |setqSingle| (form val mode env)
  (let (|$insideSetqSingleIfTrue| currentProplist mpp maxmpp td x mp tp key
        newProplist ep k newform)
    (declare (special |$insideSetqSingleIfTrue| |$QuickLet| |$form|
                      |$profileCompiler| |$EmptyMode| |$NoValueMode|))
    (setq |$insideSetqSingleIfTrue| t)
    (setq currentProplist (|getProplist| form env))
    (setq mpp
      (or (|get| form '|mode| env) (|getmode| form env)
          (if (equal mode |$NoValueMode|) |$EmptyMode| mode)))
    (when (setq td
      (cond
        ((setq td (|comp| val mpp env))
         td)
        ((and (null (|get| form '|mode| env))
              (not (equal mpp (setq maxmpp (|maxSuperType| mpp env))))
              (setq td (|comp| val maxmpp env)))
         td)
        ((and (setq td (|comp| val |$EmptyMode| env))
              (|getmode| (second td) env))
         (|assignError| val (second td) form mpp))))
      (when (setq tp (|convert| td mode))
        (setq x (first tp))
        (setq mp (second tp))
        (setq ep (third tp))
        (when (and |$profileCompiler| (identp form))
          (setq key (if (member form (cdr |$form|)) '|arguments| '|locals|))
          (|profileRecord| key form (second td)))
        (setq newProplist
          (|consProplistOf| form currentProplist '|value|
                            (|removeEnv| (cons val (cdr td)))))
        (setq ep (if (consp form) ep (|addBinding| form newProplist ep)))
        (when (|isDomainForm| val ep)
          (when (|isDomainInScope| form ep)
            (|stackWarning|
              (list '|domain| valued variable| form
                    '|has been reassigned within its scope| )))
          (setq ep (|augModemapsFromDomain1| form val ep)))
        (if (setq k (|NRTassocIndex| form))
          (setq newform (list 'setelt '$ k x))
          (setq newform
            (if |$QuickLet|
              (list 'let form x))
```

```

(list 'let form x
  (if (|isDomainForm| x ep)
    (list 'elt form 0)
    (car (|outputComp| form ep))))))
(list newform mp ep))))

```

defun NRTassocIndex

This function returns the index of domain entry *x* in the association list [*\$NRTaddForm p??*] [*\$NRTdeltaList p??*] [*\$found p??*] [*\$NRTbase p??*] [*\$NRTdeltaLength p??*]

— defun NRTassocIndex —

```

(defun |NRTassocIndex| (x)
  (let (k (i 0))
    (declare (special |$NRTdeltaLength| |$NRTbase| |$found| |$NRTdeltaList|
                      |$NRTaddForm|))
    (cond
      ((null x) x)
      ((equal x |$NRTaddForm|) 5)
      ((setq k
        (let (result)
          (loop for y in |$NRTdeltaList|
            when (and (incf i)
                      (eq (elt y 0) '|domain|)
                      (equal (elt y 1) x)
                      (setq |$found| y))
            do (setq result (or result i)))
          result))
        (- (+ |$NRTbase| |$NRTdeltaLength|) k))
      (t nil))))

```

defun assignError

[stackMessage p??]

— defun assignError —

```
(defun |assignError| (val mp form m)
  (let (message)
    (setq message
      (if val
        (list '|CANNOT ASSIGN: | val '|%1|
              '| OF MODE: | mp '|%1|
              '| TO: | form '|%1| '| OF MODE: | m)
        (list '|CANNOT ASSIGN: | val '|%1|
              '| TO: | form '|%1| '| OF MODE: | m)))
      (|stackMessage| message)))
```

defun outputComp

```
[comp p587]
[nreverse0 p??]
[outputComp p347]
[get p??]
[$Expression p??]
```

— defun outputComp —

```
(defun |outputComp| (x env)
  (let (arg1 v)
    (declare (special |$Expression|))
    (cond
      ((|comp| (list '|::| x |$Expression|) |$Expression| env))
      ((and (consp x) (eq (qfirst x) '|construct|))
        (setq arg1 (qrest x))
        (list (cons 'list
          (let (result tmp1)
            (loop for x in arg1
              do (setq result
                (cons (car
                  (progn
                    (setq tmp1 (|outputComp| x env))
                    (setq env (third tmp1))
                    tmp1))
                  result))))
            (nreverse0 result)))
          |$Expression| env))
      ((and (setq v (|get| x '|value| env))
        (consp (cadr v)) (eq (qfirst (cadr v)) '|Union|))
        (list (list '|coerceUn2E| x (cadr v)) |$Expression| env))
      (t (list x |$Expression| env))))))
```

defun maxSuperType

```
[get p??]
[maxSuperType p348]

— defun maxSuperType —

(defun |maxSuperType| (m env)
  (let (typ)
    (if (setq typ (|get| m '|SuperDomain| env))
        (|maxSuperType| typ env)
        m)))
```

defun isDomainForm

```
[isFunctor p253]
[isCategoryForm p??]
[isDomainConstructorForm p348]
[$SpecialDomainNames p??]

— defun isDomainForm —

(defun |isDomainForm| (d env)
  (let (tmp1)
    (declare (special |$SpecialDomainNames|))
    (or (member (ifcar d) |$SpecialDomainNames|) (|isFunctor| d)
        (and (progn
              (setq tmp1 (|getmode| d env))
              (and (consp tmp1) (eq (qfirst tmp1) '|Mapping|) (consp (qrest tmp1))))
          (|isCategoryForm| (qsecond tmp1) env))
        (|isCategoryForm| (|getmode| d env) env)
        (|isDomainConstructorForm| d env))))
```

defun isDomainConstructorForm

```
[isCategoryForm p??]
[eqsubstlist p??]
[$FormalMapVariableList p270]
```


— defun isDomainConstructorForm —

```
(defun |isDomainConstructorForm| (d env)
  (let (u)
    (declare (special |$FormalMapVariableList|))
    (when
      (and (consp d)
           (setq u (|get| (qfirst d) '|value| env))
           (consp u)
           (consp (qrest u))
           (consp (qsecond u))
           (eq (qcaadr u) '|Mapping|)
           (consp (qcdadr u)))
      (|isCategoryForm|
       (eqsubstlist (rest d) |$FormalMapVariableList| (cadadr u)) env))))
```

—————

defplist compString plist

We set up the `compString` function to handle the `String` keyword by setting the `special` keyword on the `String` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|String| 'special) '|compString|))
```

—————

defun compString

[[resolve p365](#)]
[\$StringCategory p??]

— defun compString —

```
(defun |compString| (form mode env)
  (declare (special |$StringCategory|))
  (list form (|resolve| |$StringCategory| mode) env))
```

—————

defplist compSubDomain plist

We set up the `compSubDomain` function to handle the `SubDomain` keyword by setting the `special` keyword on the `SubDomain` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'SubDomain 'special) '|compSubDomain|))
```

—————

defun compSubDomain

```
[compSubDomain1 p350]
[compCapsule p278]
[$addFormLhs p??]
[$NRTaddForm p??]
[$addForm p??]
[$addFormLhs p??]
```

— **defun compSubDomain** —

```
(defun |compSubDomain| (form mode env)
  (let (|$addFormLhs| |$addForm| domainForm predicate tmp1)
    (declare (special |$addFormLhs| |$addForm| |$NRTaddForm| |$addFormLhs|))
    (setq domainForm (second form))
    (setq predicate (third form))
    (setq |$addFormLhs| domainForm)
    (setq |$addForm| nil)
    (setq |$NRTaddForm| domainForm)
    (setq tmp1 (|compSubDomain1| domainForm predicate mode env))
    (setq |$addForm| (first tmp1))
    (setq env (third tmp1))
    (|compCapsule| (list 'capsule) mode env)))
```

—————

defun compSubDomain1

```
[compMakeDeclaration p622]
[addDomain p252]
[compOrCroak p586]
[stackSemanticError p??]
[lispize p351]
```

[evalAndRwriteLispForm p204]
 [\$CategoryFrame p??]
 [\$op p??]
 [\$lisplibSuperDomain p??]
 [\$Boolean p??]
 [\$EmptyMode p176]

— defun compSubDomain1 —

```
(defun |compSubDomain1| (domainForm predicate mode env)
  (let (u prefixPredicate opp dFp)
    (declare (special |$CategoryFrame| |$op| |$lisplibSuperDomain| |$Boolean|
                      |$EmptyMode|))
    (setq env (third
      (|compMakeDeclaration| (list '|:| '|#1| domainForm)
        |$EmptyMode| (|addDomain| domainForm env))))
    (setq u (|compOrCroak| predicate |$Boolean| env))
    (unless u
      (|stackSemanticError|
        (list '|predicate: | predicate
              '| cannot be interpreted with #1: | domainForm) nil))
    (setq prefixPredicate (|lispize| (first u)))
    (setq |$lisplibSuperDomain| (list domainForm predicate))
    (|evalAndRwriteLispForm| '|evalOnLoad2|
      (list 'setq '|$CategoryFrame|
        (list '|put|
          (setq opp (list 'quote |$op|))
            ''|SuperDomain|
          (setq dFp (list 'quote domainForm))
            (list '|put| dFp ''|SubDomain|
              (list 'cons (list 'quote (cons |$op| prefixPredicate))
                (list 'delasc opp (list '|get| dFp ''|SubDomain| '|$CategoryFrame|)))
                '|$CategoryFrame|))))
        (list domainForm mode env)))
```

defun lispize

[optimize p229]

— defun lispize —

```
(defun |lispize| (x)
  (car (|optimize| (list x))))
```

defplist compSubsetCategory plist

We set up the `compSubsetCategory` function to handle the `SubsetCategory` keyword by setting the `special` keyword on the `SubsetCategory` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|SubsetCategory| 'special) '|compSubsetCategory|))
```

defun compSubsetCategory

TPDHERE: See `LocalAlgebra` for an example call `[put p??]`

`[comp p587]`

`[$lhsOfColon p??]`

— defun compSubsetCategory —

```
(defun |compSubsetCategory| (form mode env)
  (let (cat r)
    (declare (special |$lhsOfColon|))
    (setq cat (second form))
    (setq r (third form))
    ; --1. put "Subsets" property on R to allow directly coercion to subset;
    ; -- allow automatic coercion from subset to R but not vice versa
    (setq env (|put| r '|Subsets| (list (list |$lhsOfColon| '|isFalse|)) env))
    ; --2. give the subset domain modemaps of cat plus 3 new functions
    (|comp|
      (list '|Join| cat
        (subst |$lhsOfColon| '$
          (list 'category '|domain|
            (list 'signature '|coerce| (list r '$))
            (list 'signature '|lift| (list r '$))
            (list 'signature '|reduce| (list '$ r))) :test #'equal))
        mode env)))
```

defplist compSuchthat plist

We set up the `compSuchthat` function to handle the `|` keyword by setting the `special` keyword on the `|` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '\| 'special) '|compSuchthat|))
```

defun compSuchthat

```
[comp p587]
[put p??]
[$Boolean p??]
```

— defun compSuchthat —

```
(defun |compSuchthat| (form mode env)
  (let (x p xp mp tmp1 pp)
    (declare (special |$Boolean|))
    (setq x (second form))
    (setq p (third form))
    (when (setq tmp1 (|comp| x mode env))
      (setq xp (first tmp1))
      (setq mp (second tmp1))
      (setq env (third tmp1))
      (when (setq tmp1 (|comp| p |$Boolean| env))
        (setq pp (first tmp1))
        (setq env (third tmp1))
        (setq env (|put| xp '|condition| pp env))
        (list xp mp env))))))
```

defplist compVector plist

We set up the `compVector` function to handle the `VECTOR` keyword by setting the `special` keyword on the `VECTOR` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get 'vector 'special) '|compVector|))
```

defun compVector

```
; null l => [$EmptyVector,m,e]
```

```
; Tl:= [[.,mUnder,e]:= comp(x,mUnder,e) or return "failed" for x in l]
; Tl="failed" => nil
; [["VECTOR",:[T.expr for T in Tl]],m,e]
```

```
[comp p587]
[$EmptyVector p??]
```

— **defun compVector** —

```
(defun |compVector| (form mode env)
  (let (tmp1 tmp2 t0 failed (newmode (second mode)))
    (declare (special |$EmptyVector|))
    (if (null form)
      (list |$EmptyVector| mode env)
      (progn
        (setq t0
          (do ((t3 form (cdr t3)) (x nil))
              ((or (atom t3) failed) (unless failed (nreverse0 tmp2)))
            (setq x (car t3))
            (if (setq tmp1 (|comp| x newmode env))
              (progn
                (setq newmode (second tmp1))
                (setq env (third tmp1))
                (push tmp1 tmp2))
              (setq failed t))))))
        (unless failed
          (list (cons 'vector
                     (loop for texpr in t0 collect (car texpr))) mode env))))))
```

—————

defplist compWhere plist

We set up the `compWhere` function to handle the `where` keyword by setting the `special` keyword on the `where` symbol property list.

— **postvars** —

```
(eval-when (eval load)
  (setf (get 'where| 'special) '|compWhere|))
```

—————

defun compWhere

```
[comp p587]
[macroExpand p178]
```

```
[deltaContour p??]
[addContour p??]
[$insideExpressionIfTrue p??]
[$insideWhereIfTrue p??]
[$EmptyMode p176]
```

— defun compWhere —

```
(defun |compWhere| (form mode eInit)
  (let (|$insideExpressionIfTrue| |$insideWhereIfTrue| newform exprList e
        eBefore tmp1 x eAfter del eFinal)
    (declare (special |$insideExpressionIfTrue| |$insideWhereIfTrue|
                      |$EmptyMode|))
    (setq newform (second form))
    (setq exprlist (cddr form))
    (setq |$insideExpressionIfTrue| nil)
    (setq |$insideWhereIfTrue| t)
    (setq e eInit)
    (when (dolist (item exprList t)
              (setq tmp1 (|comp| item |$EmptyMode| e))
              (unless tmp1 (return nil))
              (setq e (third tmp1))))
    (setq |$insideWhereIfTrue| nil)
    (setq tmp1 (|comp| (|macroExpand| newform (setq eBefore e)) mode e))
    (when tmp1
      (setq x (first tmp1))
      (setq mode (second tmp1))
      (setq eAfter (third tmp1))
      (setq del (|deltaContour| eAfter eBefore))
      (if del
        (setq eFinal (|addContour| del eInit))
        (setq eFinal eInit))
      (list x mode eFinal))))
```

6.6 Functions for coercion

defun coerce

The function `coerce` is used by the old compiler for coercions. The function `coerceInteractive` is used by the interpreter. One should always call the correct function, since the representation of basic objects may not be the same. [keyedSystemError p??]

```
[rplac p??]
[coerceEasy p356]
[coerceSubset p357]
```

```
[coerceHard p358]
[isSomeDomainVariable p??]
[stackMessage p??]
[$InteractiveMode p??]
[$Rep p??]
[$fromCoerceable p??]
```

— defun coerce —

```
(defun |coerce| (tt mode)
  (labels (
    (fn (x m1 m2)
      (list '|Cannot coerce| x '|%1| '|      of mode| m1
            '|%1| '|      to mode| m2))))
    (let (tp)
      (declare (special |$fromCoerceable$| |$Rep| |$InteractiveMode|))
      (if |$InteractiveMode|
        (|keyedSystemError|
         "Unexpected error or improper call to system function %1: %2"
         (list "coerce" "function coerce called from the interpreter."))
        (progn
         (|rplac| (cadr tt) (subst '$ |$Rep| (cadr tt) :test #'equal))
         (cond
          ((setq tp (|coerceEasy| tt mode)) tp)
          ((setq tp (|coerceSubset| tt mode)) tp)
          ((setq tp (|coerceHard| tt mode)) tp)
          ((or (eq (car tt) '|$fromCoerceable$|) (|isSomeDomainVariable| mode)) nil)
          (t (|stackMessage| (fn (first tt) (second tt) mode))))))))))
```

—————→

defun coerceEasy

```
[modeEqualSubst p367]
[$EmptyMode p176]
[$Exit p??]
[$NoValueMode p176]
[$Void p??]
```

— defun coerceEasy —

```
(defun |coerceEasy| (tt m)
  (declare (special |$EmptyMode| |$Exit| |$NoValueMode| |$Void|))
  (cond
   ((equal m |$EmptyMode|) tt)
   ((or (equal m |$NoValueMode|) (equal m |$Void|))
```



```

(list (car tt) m (third tt)))
((equal (second tt) m) tt)
((equal (second tt) |$NoValueMode|) tt)
((equal (second tt) |$Exit|)
 (list
  (list 'progn (car tt) (list '|userError| "Did not really exit."))
  m (third tt)))
((or (equal (second tt) |$EmptyMode|)
     (|modeEqualSubst| (second tt) m (third tt)))
 (list (car tt) m (third tt)))))

```

defun coerceSubset

```

[isSubset p??]
[lassoc p??]
[get p??]
[opOf p??]
[eval p??]
[isSubset p??]
[maxSuperType p348]

```

— defun coerceSubset —

```

(defun |coerceSubset| (arg1 mp)
  (let (x m env pred)
    (setq x (first arg1))
    (setq m (second arg1))
    (setq env (third arg1))
    (cond
     ((or (|isSubset| m mp env) (and (eq m '|Rep|) (eq mp '$)))
      (list x mp env))
     ((and (consp m) (eq (qfirst m) '|SubDomain|)
           (consp (qrest m)) (equal (qsecond m) mp))
      (list x mp env))
     ((and (setq pred (lassoc (|opOf| mp) (|get| (|opOf| m) '|SubDomain| env)))
           (integerp x) (|eval| (subst x '|#1| pred :test #'equal)))
      (list x mp env))
     ((and (setq pred (|isSubset| mp (|maxSuperType| m env) env))
           (integerp x) (|eval| (subst x '|* pred :test #'equal)))
      (list x mp env))
     (t nil))))

```

defun coerceHard

```
[modeEqual p366]
[get p??]
[getmode p??]
[isCategoryForm p??]
[extendsCategoryForm p??]
[coerceExtraHard p359]
[$e p??]
[$e p??]
[$String p349]
[$bootStrapMode p??]
```

— defun coerceHard —

```
(defun |coerceHard| (tt m)
  (let (|$e| mp tmp1 mpp)
    (declare (special |$e| |$String| |$bootStrapMode|))
    (setq |$e| (third tt))
    (setq mp (second tt))
    (cond
      ((and (stringp mp) (|modeEqual| m |$String|))
        (list (car tt) m |$e|))
      ((or (|modeEqual| mp m)
        (and (or (progn
          (setq tmp1 (|get| mp '|value| |$e|))
          (and (consp tmp1)
            (progn (setq mpp (qfirst tmp1)) t)))
          (progn
            (setq tmp1 (|getmode| mp |$e|))
            (and (consp tmp1)
              (eq (qfirst tmp1) '|Mapping|)
              (and (consp (qrest tmp1))
                (eq (qcddr tmp1) nil)
                (progn (setq mpp (qsecond tmp1)) t))))))
          (|modeEqual| mpp m))
        (and (or (progn
          (setq tmp1 (|get| m '|value| |$e|))
          (and (consp tmp1)
            (progn (setq mpp (qfirst tmp1)) t)))
          (progn
            (setq tmp1 (|getmode| m |$e|))
            (and (consp tmp1)
              (eq (qfirst tmp1) '|Mapping|)
              (and (consp (qrest tmp1))
                (eq (qcddr tmp1) nil)
                (progn (setq mpp (qsecond tmp1)) t))))))
          (|modeEqual| mpp mp)))
        (list (car tt) m (third tt)))
```

```

((and (stringp (car tt)) (equal (car tt) m))
  (list (car tt) m |$e|))
((|isCategoryForm| m |$e|)
  (cond
    ((eq |$bootStrapMode| t)
      (list (car tt) m |$e|))
    ((|extendsCategoryForm| (car tt) (cadr tt) m)
      (list (car tt) m |$e|))
    (t (|coerceExtraHard| tt m))))
(t (|coerceExtraHard| tt m))))

```

defun coerceExtraHard

[\[autoCoerceByModemap p364\]](#)
[\[isUnionMode p321\]](#)
[\[hasType p360\]](#)
[\[member p??\]](#)
[\[autoCoerceByModemap p364\]](#)
[\[coerce p355\]](#)
[\[\\$Expression p??\]](#)

— defun coerceExtraHard —

```

(defun |coerceExtraHard| (tt m)
  (let (x mp e tmp1 z ta tp tpp)
    (declare (special |$Expression|))
    (setq x (first tt))
    (setq mp (second tt))
    (setq e (third tt))
    (cond
      ((setq tp (|autoCoerceByModemap| tt m)) tp)
      ((and (progn
              (setq tmp1 (|isUnionMode| mp e))
              (and (consp tmp1) (eq (qfirst tmp1) '|Union|))
              (progn
                (setq z (qrest tmp1)) t)))
          (setq ta (|hasType| x e))
          (|member| ta z)
          (setq tp (|autoCoerceByModemap| tt ta))
          (setq tpp (|coerce| tp m)))
        tpp)
      ((and (consp mp) (eq (qfirst mp) '|Record|) (equal m |$Expression|))
        (list (list '|coerceRe2E| x (list 'elt (copy mp) 0)) m e))
      (t nil))))

```

defun hasType

[get p??]

— defun hasType —

```
(defun |hasType| (x e)
  (labels (
    (fn (x)
      (cond
        ((null x) nil)
        ((and (consp x) (consp (qfirst x)) (eq (qcaar x) '|case|)
              (consp (qcddar x)) (consp (qcddar x))
              (eq (qcdddar x) nil))
          (qcaddar x))
        (t (fn (cdr x))))))
    (fn (|get| x '|condition| e))))
```

defun coerceable

[pmatch p??]
 [sublis p??]
 [coerce p³⁵⁵]
 [\$fromCoerceable p??]

— defun coerceable —

```
(defun |coerceable| (m mp env)
  (let (sl)
    (declare (special |$fromCoerceable$|))
    (cond
      ((equal m mp) m)
      ((setq sl (|pmatch| mp m)) (sublis sl mp))
      ((|coerce| (list '|$fromCoerceable$| m env) mp) mp)
      (t nil))))
```

defun coerceExit

[resolve p365]

[replaceExitEsc p??]

[coerce p355]

[\$exitMode p??]

— defun coerceExit —

```
(defun |coerceExit| (arg1 mp)
  (let (x m e catchTag xp)
    (declare (special |$exitMode|))
    (setq x (first arg1))
    (setq m (second arg1))
    (setq e (third arg1))
    (setq mp (|resolve| m mp))
    (setq xp
      (|replaceExitEtc| x
        (setq catchTag (mkq (gensym))) '|TAGGEDexit| |$exitMode|))
    (|coerce| (list (list 'catch catchTag xp) m e) mp)))
```

defplist compAtSign plist

We set up the `compAtSign` function to handle the `@` keyword by setting the `special` keyword on the `@` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get '|@| 'special) 'compAtSign))
```

defun compAtSign

[addDomain p252]

[comp p587]

[coerce p355]

— defun compAtSign —

```
(defun compAtSign (form mode env)
  (let ((newform (second form)) (mprime (third form)) tmp)
```

```
(setq env (|addDomain| mprime env))
(when (setq tmp (|comp| newform mprime env)) (|coerce| tmp mode)))
```

defplist compCoerce plist

We set up the `compCoerce` function to handle the `::` keyword by setting the `special` keyword on the `::` symbol property list.

— postvars —

```
(eval-when (eval load)
  (setf (get ' "::" 'special) '|compCoerce|))
```

defun compCoerce

```
[addDomain p252]
[getmode p??]
[compCoerce1 p363]
[coerce p355]
```

— defun compCoerce —

```
(defun |compCoerce| (form mode env)
  (let (newform newmode tmp1 tmp4 z td)
    (setq newform (second form))
    (setq newmode (third form))
    (setq env (|addDomain| newmode env))
    (setq tmp1 (|getmode| newmode env))
    (cond
      ((setq td (|compCoerce1| newform newmode env))
        (|coerce| td mode))
      ((and (consp tmp1) (eq (qfirst tmp1) '|Mapping|)
        (consp (qrest tmp1)) (eq (qcddr tmp1) nil)
        (consp (qsecond tmp1))
        (eq (qcaadr tmp1) '|UnionCategory|))
        (setq z (qcdadr tmp1))
        (when
          (setq td
            (dolist (mode1 z tmp4)
              (setq tmp4 (or tmp4 (|compCoerce1| newform mode1 env))))
            (|coerce| (list (car td) newmode (third td)) mode))))))
```

defun compCoerce1

[comp p587]
 [resolve p365]
 [coerce p355]
 [coerceByModemap p363]
 [mkq p??]

— defun compCoerce1 —

```
(defun |compCoerce1| (form mode env)
  (let (m1 td tp gg pred code)
    (declare (special |$String| |$EmptyMode|))
    (when (setq td (or (|comp| form mode env) (|comp| form |$EmptyMode| env)))
      (setq m1 (if (stringp (second td)) |$String| (second td)))
      (setq mode (|resolve| m1 mode))
      (setq td (list (car td) m1 (third td)))
      (cond
        ((setq tp (|coerce| td mode)) tp)
        ((setq tp (|coerceByModemap| td mode)) tp)
        ((setq pred (|isSubset| mode (second td) env))
         (setq gg (gensym))
         (setq pred (subst gg '* pred :test #'equal))
         (setq code
          (list 'prog1
              (list 'let gg (first td))
              (cons '|check-subtype| (cons pred (list (mkq mode) gg))))))
        (list code mode (third td))))))
```

defun coerceByModemap

[modeEqual p366]
 [isSubset p??]
 [genDeltaEntry p??]

— defun coerceByModemap —

```
(defun |coerceByModemap| (arg1 mp)
  (let (x m env map cexpr u mm fn)
    (setq x (first arg1))
    (setq m (second arg1))
    (setq env (third arg1))
```

```

(setq u
(loop for modemap in (|getModemapList| '|coerce| 1 env)
do
  (setq map (first modemap))
  (setq cexpr (second modemap))
  when
    (and (consp map) (consp (qrest map))
         (consp (qcddr map))
         (eq (qcdddr map) nil)
         (or (|modeEqual| (second map) mp) (|isSubset| (second map) mp env))
         (or (|modeEqual| (third map) m) (|isSubset| m (third map) env)))
    collect modemap))
(when u
  (setq mm (first u))
  (setq fn (|genDeltaEntry| (cons '|coerce| mm)))
  (list (list '|call| fn x) mp env)))

```

defun autoCoerceByModemap

```

[getModemapList p263]
[modeEqual p366]
[member p??]
[get p??]
[stackMessage p??]
[$fromCoerceable p??]

```

— defun autoCoerceByModemap —

```

(defun |autoCoerceByModemap| (arg1 target)
  (let (x source e map cexpr u fn y)
    (declare (special |$fromCoerceable$|))
    (setq x (first arg1))
    (setq source (second arg1))
    (setq e (third arg1))
    (setq u
      (loop for modemap in (|getModemapList| '|autoCoerce| 1 e)
        do
          (setq map (first modemap))
          (setq cexpr (second modemap))
          when
            (and (consp map) (consp (qrest map)) (consp (qcddr map))
                 (eq (qcdddr map) nil)
                 (|modeEqual| (second map) target)
                 (|modeEqual| (third map) source))
            collect cexpr))
    (setq y (list (list '|call| fn x) mp env)))
  )

```



```

(when u
  (setq fn
    (let (result)
      (loop for item in u
        do
          (when (first item) (setq result (or result (second item))))))
    result))
(when fn
  (cond
    ((and (consp source) (eq (qfirst source) '|Union|)
      (|member| target (qrest source))))
    (cond
      ((and (setq y (|get| x '|condition| e))
        (let (result)
          (loop for u in y do
            (setq result
              (or result
                (and (consp u) (eq (qfirst u) '|case|) (consp (qrest u))
                  (consp (qcddr u))
                  (eq (qcdddr u) nil)
                  (equal (qthird u) target))))))
          result))
        (list (list '|call| fn x) target e))
      ((eq x '|$fromCoerceable$|) nil)
      (t
        (|stackMessage|
          (list '|cannot coerce: | x '|%1| '|          of mode: | source
            '|%1| '|          to: | target '| without a case statement|))))))
    (t
      (list (list '|call| fn x) target e))))))

```

defun resolve

[\[modeEqual p366\]](#)
[\[mkUnion p366\]](#)
[\[\\$String p349\]](#)
[\[\\$EmptyMode p176\]](#)
[\[\\$NoValueMode p176\]](#)

— defun resolve —

```

(defun |resolve| (din dout)
  (declare (special |$String| |$EmptyMode| |$NoValueMode|))
  (cond
    ((or (equal din |$NoValueMode|) (equal dout |$NoValueMode|)) |$NoValueMode|)

```

```

(equal dout |$EmptyModel|) din)
((and (not (equal din dout)) (or (stringp din) (stringp dout)))
 (cond
  ((|modeEqual| dout |$String|) dout)
  ((|modeEqual| din |$String|) nil)
  (t (|mkUnion| din dout))))
(t dout))

```

defun mkUnion

```

[union p??]
[$Rep p??]

```

— defun mkUnion —

```

(defun |mkUnion| (a b)
  (declare (special |$Rep|))
  (cond
   ((and (eq b '$) (consp |$Rep|) (eq (qfirst |$Rep|) '|Union|))
    (qrest |$Rep|))
   ((and (consp a) (eq (qfirst a) '|Union|))
    (cond
     ((and (consp b) (eq (qfirst b) '|Union|))
      (cons '|Union| (|union| (qrest a) (qrest b))))
     (t (cons '|Union| (|union| (list b) (qrest a))))))
   ((and (consp b) (eq (qfirst b) '|Union|))
    (cons '|Union| (|union| (list a) (qrest b))))
   (t (list '|Union| a b))))

```

defun This orders Unions

This orders Unions

— defun modeEqual —

```

(defun |modeEqual| (x y)
  (let (x1 y1)
    (cond
     ((or (atom x) (atom y)) (equal x y))
     ((not (eq1 (|#| x) (|#| y))) nil)
     ((and (consp x) (eq (qfirst x) '|Union|) (consp y) (eq (qfirst y) '|Union|))
      (setq x1 (qrest x))

```

```

(setq y1 (qrest y))
(loop for a in x1 do
  (loop for b in y1 do
    (when (|modeEqual| a b)
      (setq x1 (|delete| a x1))
      (setq y1 (|delete| b y1))
      (return nil))))
(unless (or x1 y1) t))
(t
 (let ((result t))
  (loop for u in x for v in y
    do (setq result (and result (|modeEqual| u v))))
  result))))

```

defun modeEqualSubst

```

[modeEqual p366]
[modeEqualSubst p367]
[length p??]

```

— defun modeEqualSubst —

```

(defun |modeEqualSubst| (m1 m env)
  (let (mp op z1 z2)
    (cond
      ((|modeEqual| m1 m) t)
      ((atom m1)
       (when (setq mp (car (|get| m1 '|value| env)))
         (|modeEqual| mp m)))
      ((and (consp m1) (consp m) (equal (qfirst m) (qfirst m1))
        (equal (|#| (qrest m1)) (|#| (qrest m))))
       (setq op (qfirst m1))
       (setq z1 (qrest m1))
       (setq z2 (qrest m))
       (let ((result t))
        (loop for xm1 in z1 for xm2 in z2
          do (setq result (and result (|modeEqualSubst| xm1 xm2 env))))
        result))
      (t nil))))

```

Chapter 7

Post Transformers

7.1 Direct called postparse routines

defun postTransform

[postTran p370]
[postTransform identp (vol5)]
[postTransformCheck p373]
[aplTran p405]

— defun postTransform —

```
(defun postTransform (y)
  (let (x tmp1 tmp2 tmp3 tmp4 tmp5 tt l u)
    (setq x y)
    (setq u (|postTran| x))
    (when
      (and (consp u) (eq (qfirst u) '|@Tuple|))
      (progn
        (setq tmp1 (qrest u))
        (and (consp tmp1)
          (progn (setq tmp2 (reverse tmp1)) t)
          (consp tmp2)
          (progn
            (setq tmp3 (qfirst tmp2))
            (and (consp tmp3)
              (eq (qfirst tmp3) '|:|)
              (progn
                (setq tmp4 (qrest tmp3))
                (and (consp tmp4)
                  (progn
                    (setq y (qfirst tmp4))
```

```

      (setq tmp5 (qrest tmp4))
      (and (consp tmp5)
            (eq (qrest tmp5) nil)
            (progn (setq tt (qfirst tmp5)) t))))))
      (progn (setq l (qrest tmp2)) t)
      (progn (setq l (nreverse l)) t)))
      (dolist (x l t) (unless (identp x) (return nil))))
      (setq u (list '|:| (cons 'listof (append l (list y)) tt)))
      (postTransformCheck u)
      (aplTran u)))

```

defun postTran

```

[postAtom p371]
[postTran p370]
[unTuple p413]
[postTranList p372]
[postForm p374]
[postOp p371]
[postScriptsForm p372]

```

— defun postTran —

```

(defun |postTran| (x)
  (let (op f tmp1 a tmp2 tmp3 b y)
    (if (atom x)
        (postAtom x)
        (progn
          (setq op (car x))
          (cond
            ((and (atom op) (setq f (get1 op '|postTran|)))
              (funcall f x))
            ((and (consp op) (eq (qfirst op) '|elt|))
              (progn
                (setq tmp1 (qrest op))
                (and (consp tmp1)
                     (progn
                      (setq a (qfirst tmp1))
                      (setq tmp2 (qrest tmp1))
                      (and (consp tmp2)
                           (eq (qrest tmp2) nil)
                           (progn (setq b (qfirst tmp2)) t))))))
              (cons (|postTran| op) (cdr (|postTran| (cons b (cdr x))))))
            ((and (consp op) (eq (qfirst op) '|Scripts|))
              (postScriptsForm op

```

```

      (dolist (y (rest x) tmp3)
        (setq tmp3 (append tmp3 (|unTuple| (|postTran| y))))))
      ((not (equal op (setq y (postOp op))))
       (cons y (postTranList (cdr x))))
      (t (postForm x))))))

```

defun postOp

— defun postOp —

```

(defun postOp (x)
  (declare (special $boot))
  (cond
    ((eq x '|:=|) 'let)
    ((eq x '|:-|) 'letd)
    ((eq x '|Attribute|) 'attribute)
    (t x)))

```

defun postAtom

[\$boot p??]

— defun postAtom —

```

(defun postAtom (x)
  (declare (special $boot))
  (cond
    ($boot x)
    ((eq1 x 0) '(|Zero|))
    ((eq1 x 1) '(|One|))
    ((eq x t) 't$)
    ((and (identp x) (getdatabase x 'niladic)) (list x))
    (t x)))

```

defun postTranList

[postTran p370]

— defun postTranList —

```
(defun postTranList (x)
  (loop for y in x collect (|postTran| y)))
```

—————

defun postScriptsForm

[getScriptName p408]

[length p??]

[postTranScripts p372]

— defun postScriptsForm —

```
(defun postScriptsForm (form argl)
  (let ((op (second form)) (a (third form)))
    (cons (getScriptName op a (|#| argl))
          (append (postTranScripts a) argl))))
```

—————

defun postTranScripts

[postTranScripts p372]

[postTran p370]

— defun postTranScripts —

```
(defun postTranScripts (a)
  (labels (
    (fn (x)
      (if (and (consp x) (eq (qfirst x) '|@Tuple|))
          (qrest x)
          (list x))))
    (let (tmp1 tmp2 tmp3)
      (cond
        ((and (consp a) (eq (qfirst a) '|PrefixSC|))
         (progn
          (setq tmp1 (qrest a))
```



```

      (and (consp tmp1) (eq (qrest tmp1) nil))))
    (postTranScripts (qfirst tmp1)))
  ((and (consp a) (eq (qfirst a) '|;|))
   (dolist (y (qrest a) tmp2)
    (setq tmp2 (append tmp2 (postTranScripts y)))))
  ((and (consp a) (eq (qfirst a) '|,|))
   (dolist (y (qrest a) tmp3)
    (setq tmp3 (append tmp3 (fn (|postTran| y))))))
  (t (list (|postTran| a)))))

```

defun postTransformCheck

[postcheck p373]
 [\$defOp p??]

— defun postTransformCheck —

```

(defun postTransformCheck (x)
  (let (|$defOp|)
    (declare (special |$defOp|))
    (setq |$defOp| nil)
    (postcheck x)))

```

defun postcheck

[setDefOp p404]
 [postcheck p373]

— defun postcheck —

```

(defun postcheck (x)
  (cond
    ((atom x) nil)
    ((and (consp x) (eq (qfirst x) 'def) (consp (qrest x)))
     (setDefOp (qsecond x))
     (postcheck (qcddr x)))
    ((and (consp x) (eq (qfirst x) 'quote)) nil)
    (t (postcheck (car x)) (postcheck (cdr x)))))

```

defun postError

```
[bumperrorcount p543]
[$defOp p??]
[$InteractiveMode p??]
[$postStack p??]
```

— defun postError —

```
(defun postError (msg)
  (let (xmsg)
    (declare (special |$defOp| |$postStack| |$InteractiveMode|))
    (bumperrorcount '|precompilation|)
    (setq xmsg
      (if (and (not (eq |$defOp| '|$defOp|)) (null |$InteractiveMode|))
          (cons |$defOp| (cons ": " msg))
          msg))
    (push xmsg |$postStack|)
    nil))
```

defun postForm

```
[postTranList p372]
[internl p??]
[postTran p370]
[postError p374]
[bright p??]
[$boot p??]
```

— defun postForm —

```
(defun postForm (u)
  (let (op argl arglp numOfArgs opp x)
    (declare (special $boot))
    (seq
      (setq op (car u))
      (setq argl (cdr u))
      (setq x
        (cond
          ((atom op)
           (setq arglp (postTranList argl))
           (setq opp
             (seq
              (exit op)
              (exit op))))
          (t
           (setq arglp (postTranList argl))
           (setq opp
             (seq
              (exit op)
              (exit op)))))))
    (exit op))
```

```

(when $boot (exit op))
(when (or (get1 op '|Led|) (get1 op '|Nud|) (eq op 'in)) (exit op))
(setq numOfArgs
  (cond
    ((and (consp arglp) (eq (qrest arglp) nil) (consp (qfirst arglp))
      (eq (qcaar arglp) '@Tuple|))
      (|#| (qcdar arglp)))
    (t 1)))
(internal '* (princ-to-string numOfArgs) (pname op)))
(cons opp arglp))
((and (consp op) (eq (qfirst op) '|Scripts|))
  (append (|postTran| op) (postTranList argl)))
(t
  (setq u (postTranList u))
  (cond
    ((and (consp u) (consp (qfirst u)) (eq (qcaar u) '@Tuple|))
      (postError
        (cons " "
          (append (|bright| u)
            (list "is illegal because tuples cannot be applied!" '|%1|
              " Did you misuse infix dot?")))))
    (t u)))
(cond
  ((and (consp x) (consp (qrest x)) (eq (qcddr x) nil)
    (consp (qsecond x)) (eq (qcaadr x) '@Tuple|))
    (cons (car x) (qcdadr x)))
  (t x))))

```

7.2 Indirect called postparse routines

In the `postTran` function there is the code:

```

((and (atom op) (setq f (get1 op '|postTran|)))
  (funcall f x))

```

The functions in this section are called through the symbol-plist of the symbol being parsed. The original list read:

<code>add</code>	<code>postAdd</code>
<code>@</code>	<code>postAtSign</code>
<code>:BF:</code>	<code>postBigFloat</code>
<code>Block</code>	<code>postBlock</code>
<code>CATEGORY</code>	<code>postCategory</code>
<code>COLLECT</code>	<code>postCollect</code>

:	postColon	
::	postColonColon	
,	postComma	
construct	postConstruct	
==	postDef	
=>	postExit	
if	postIf	
in	postin	;" the infix operator version of in"
IN	postIn	;" the iterator form of in"
Join	postJoin	
->	postMapping	
==>	postMDef	
pretend	postPretend	
QUOTE	postQUOTE	
Reduce	postReduce	
REPEAT	postRepeat	
Scripts	postScripts	
;	postSemiColon	
Signature	postSignature	
/	postSlash	
@Tuple	postTuple	
TupleCollect	postTupleCollect	
where	postWhere	
with	postWith	

defplist postAdd plist

— postvars —

```
(eval-when (eval load)
  (setf (get 'add| 'postTran|) 'postAdd|))
```

—————

defun postAdd

```
[postTran p370]
[postCapsule p377]
```

— defun postAdd —

```
(defun |postAdd| (arg)
  (if (null (cddr arg))
      (|postCapsule| (second arg))
      (list 'add| (|postTran| (second arg)) (|postCapsule| (third arg)))))
```

defun postCapsule

[checkWarning p547]
 [postBlockItem p377]
 [postBlockItemList p377]
 [postFlatten p386]

— defun postCapsule —

```
(defun |postCapsule| (x)
  (let (op)
    (cond
      ((null (and (consp x) (progn (setq op (qfirst x)) t)))
        (|checkWarning| (list "Apparent indentation error following add")))
      ((or (integerp op) (eq op '==))
        (list 'capsule (|postBlockItem| x)))
      ((eq op '|;|)
        (cons 'capsule (|postBlockItemList| (|postFlatten| x '|;|))))
      ((eq op '|if|)
        (list 'capsule (|postBlockItem| x)))
      (t (|checkWarning| (list "Apparent indentation error following add")))))
```

defun postBlockItemList

[postBlockItem p377]

— defun postBlockItemList —

```
(defun |postBlockItemList| (args)
  (let (result)
    (dolist (item args (nreverse result))
      (push (|postBlockItem| item) result))))
```

defun postBlockItem

[postTran p370]

— defun postBlockItem —

```

(defun |postBlockItem| (x)
  (let ((tmp1 t) tmp2 y tt z)
    (setq x (|postTran| x))
    (if
      (and (consp x) (eq (qfirst x) '|@Tuple|))
      (progn
        (and (consp (qrest x))
          (progn (setq tmp2 (reverse (qrest x))) t)
          (consp tmp2)
          (progn
            (and (consp (qfirst tmp2)) (eq (qcaar tmp2) '|:|)
              (progn
                (and (consp (qcdar tmp2))
                  (progn
                    (setq y (qcadar tmp2))
                    (and (consp (qcddar tmp2))
                      (eq (qcdddar tmp2) nil)
                      (progn (setq tt (qcaddar tmp2)) t)))))))
            (progn (setq z (qrest tmp2)) t)
            (progn (setq z (nreverse z)) T)))
        (do ((tmp6 nil (null tmp1)) (tmp7 z (cdr tmp7)) (x nil))
          ((or tmp6 (atom tmp7)) tmp1)
          (setq x (car tmp7))
          (setq tmp1 (and tmp1 (identp x)))))
      (cons '|:| (cons (cons 'listof (append z (list y))) (list tt)))
      x)))

```

defplist postAtSign plist

— postvars —

```

(eval-when (eval load)
  (setf (get '@ '|postTran|) '|postAtSign|))

```

defun postAtSign

[postTran p370]
 [postType p379]

— defun postAtSign —

```
(defun |postAtSign| (arg)
  (cons '@ (cons (|postTran| (second arg)) (|postType| (third arg)))))
```

defun postType

[postTran p370]
[unTuple p413]

— defun postType —

```
(defun |postType| (typ)
  (let (source target)
    (cond
      ((and (consp typ) (eq (qfirst typ) '->) (consp (qrest typ))
        (consp (qcddr typ)) (eq (qcdddr typ) nil))
       (setq source (qsecond typ))
       (setq target (qthird typ))
       (cond
         ((eq source '|constant|)
          (list (list (|postTran| target)) '|constant|))
         (t
          (list (cons '|Mapping|
                     (cons (|postTran| target)
                           (|unTuple| (|postTran| source)))))))
      ((and (consp typ) (eq (qfirst typ) '->)
        (consp (qrest typ)) (eq (qcddr typ) nil))
       (list (list '|Mapping| (|postTran| (qsecond typ)))))
      (t (list (|postTran| typ)))))
```

defplist postBigFloat plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|:BF:| '|postTran|) '|postBigFloat|))
```

defun postBigFloat

[postTran p370]

[\$boot p??]

[\$InteractiveMode p??]

— defun postBigFloat —

```

(defun |postBigFloat| (arg)
  (let (mant expon eltword)
    (declare (special $boot |$InteractiveMode|))
    (setq mant (second arg))
    (setq expon (cddr arg))
    (if $boot
        (times (float mant) (expt (float 10) expon))
        (progn
          (setq eltword (if |$InteractiveMode| '|$elt| '|elt|))
          (|postTran|
            (list (list eltword '(|Float|) '|float|)
                  (list '|,| (list '|,| mant expon) 10)))))))

```

—————

defplist postBlock plist

— postvars —

```

(eval-when (eval load)
  (setf (get '|Block| '|postTran|) '|postBlock|))

```

—————

defun postBlock

[postBlockItemList p377]

[postTran p370]

— defun postBlock —

```

(defun |postBlock| (arg)
  (let (tmp1 x y)
    (setq tmp1 (reverse (cdr arg)))
    (setq x (car tmp1))
    (setq y (nreverse (cdr tmp1)))

```



```
(cons 'seq
      (append (|postBlockItemList| y) (list (list 'exit| (|postTran| x))))))
```

deflist postCategory plist

— postvars —

```
(eval-when (eval load)
  (setf (get 'category '|postTran|) '|postCategory|))
```

defun postCategory

```
[postTran p370]
[nreverse0 p??]
[$insidePostCategoryIfTrue p??]
```

— defun postCategory —

```
(defun |postCategory| (u)
  (declare (special |$insidePostCategoryIfTrue|))
  (labels (
    (fn (arg)
      (let (|$insidePostCategoryIfTrue|)
        (declare (special |$insidePostCategoryIfTrue|))
        (setq |$insidePostCategoryIfTrue| t)
        (|postTran| arg))) )
    (let ((z (cdr u)) op tmp1)
      (if (null z)
          u
          (progn
            (setq op (if |$insidePostCategoryIfTrue| 'progn 'category))
            (cons op (dolist (x z (nreverse0 tmp1)) (push (fn x) tmp1))))))))
```

defun postCollect,finish

```
[postMakeCons p382]
[tuple2List p547]
```

[postTranList [p372](#)]

— defun postCollect,finish —

```
(defun |postCollect,finish| (op itl y)
  (let (tmp2 tmp5 newBody)
    (cond
      ((and (consp y) (eq (qfirst y) '|:|)
        (consp (qrest y)) (eq (qcddr y) nil))
       (list 'reduce '|append| 0 (cons op (append itl (list (qsecond y))))))
      ((and (consp y) (eq (qfirst y) '|Tuple|))
       (setq newBody
        (cond
          ((dolist (x (qrest y) tmp2)
            (setq tmp2
              (or tmp2 (and (consp x) (eq (qfirst x) '|:|)
                (consp (qrest x)) (eq (qcddr x) nil))))))
          (|postMakeCons| (qrest y)))
        (dolist (x (qrest y) tmp5)
          (setq tmp5 (or tmp5 (and (consp x) (eq (qfirst x) 'segment))))
          (|tuple2List| (qrest y)))
        (t (cons '|construct| (postTranList (qrest y))))))
       (list 'reduce '|append| 0 (cons op (append itl (list newBody))))
      (t (cons op (append itl (list y))))))
```

—————

defun postMakeCons

[postMakeCons [p382](#)]

[postTran [p370](#)]

— defun postMakeCons —

```
(defun |postMakeCons| (args)
  (let (a b)
    (cond
      ((null args) '|nil|)
      ((and (consp args) (consp (qfirst args)) (eq (qcaar args) '|:|)
        (consp (qcdar args)) (eq (qcddar args) nil))
       (setq a (qcadar args))
       (setq b (qrest args))
       (if b
        (list '|append| (|postTran| a) (|postMakeCons| b))
        (|postTran| a)))
      (t (list '|cons| (|postTran| (car args)) (|postMakeCons| (cdr args))))))
```

defplist postCollect plist

— postvars —

```
(eval-when (eval load)
  (setf (get 'collect '|postTran|) '|postCollect|))
```

defun postCollect

```
[postCollect,finish p381]
[postCollect p383]
[postIteratorList p384]
[postTran p370]
```

— defun postCollect —

```
(defun |postCollect| (arg)
  (let (constructOp tmp3 m itl x)
    (setq constructOp (car arg))
    (setq tmp3 (reverse (cdr arg)))
    (setq x (car tmp3))
    (setq m (nreverse (cdr tmp3)))
    (cond
      ((and (consp x) (consp (qfirst x)) (eq (qcaar x) '|elt|)
            (consp (qcddar x)) (consp (qcddar x))
            (eq (qcdddar x) nil)
            (eq (qcaddar x) '|construct|))
        (|postCollect|
         (cons (list '|elt| (qcadar x) 'collect)
               (append m (list (cons '|construct| (qrest x)))))))
      (t
       (setq itl (|postIteratorList| m))
       (setq x
        (if (and (consp x) (eq (qfirst x) '|construct|)
                  (consp (qrest x)) (eq (qcddr x) nil))
            (qsecond x)
            x))
       (|postCollect,finish| constructOp itl (|postTran| x)))))
```

defun postIteratorList

[postTran p370]
 [postInSeq p392]
 [postIteratorList p384]

— defun postIteratorList —

```
(defun |postIteratorList| (args)
  (let (z p y u a b)
    (cond
      ((consp args)
       (setq p (|postTran| (qfirst args)))
       (setq z (qrest args))
       (cond
         ((and (consp p) (eq (qfirst p) 'in) (consp (qrest p))
          (consp (qcddr p)) (eq (qcdddr p) nil))
          (setq y (qsecond p))
          (setq u (qthird p))
          (cond
            ((and (consp u) (eq (qfirst u) '|\|) (consp (qrest u))
             (consp (qcddr u)) (eq (qcdddr u) nil))
             (setq a (qsecond u))
             (setq b (qthird u))
             (cons (list 'in y (|postInSeq| a))
                   (cons (list '|\| b)
                         (|postIteratorList| z))))
            (t (cons (list 'in y (|postInSeq| u)) (|postIteratorList| z))))))
         (t (cons p (|postIteratorList| z))))))
    (t args)))
```

—

defplist postColon plist

— postvars —

```
(eval-when (eval load)
  (setf (get '[:| 'postTran|) 'postColon|))
```

—

defun postColon

[postTran p370]
[postType p379]

— defun postColon —

```
(defun |postColon| (u)
  (cond
    ((and (consp u) (eq (qfirst u) '|:|)
          (consp (qrest u)) (eq (qcddr u) nil))
      (list '|:| (|postTran| (qsecond u))))
    ((and (consp u) (eq (qfirst u) '|:|) (consp (qrest u))
          (consp (qcddr u)) (eq (qcdddr u) nil))
      (cons '|:| (cons (|postTran| (second u)) (|postType| (third u)))))))
```

—————

defplist postColonColon plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|::| '|postTran|) '|postColonColon|))
```

—————

defun postColonColon

[postForm p374]
[\$boot p??]

— defun postColonColon —

```
(defun |postColonColon| (u)
  (if (and $boot (consp u) (eq (qfirst u) '|::|) (consp (qrest u))
      (consp (qcddr u)) (eq (qcdddr u) nil))
      (intern (princ-to-string (third u)) (second u))
      (postForm u)))
```

—————

defplist postComma plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|,| 'postTran|) 'postComma|))
```

—————

defun postComma

[postTuple p402]
[comma2Tuple p386]

— defun postComma —

```
(defun |postComma| (u)
  (|postTuple| (|comma2Tuple| u)))
```

—————

defun comma2Tuple

[postFlatten p386]

— defun comma2Tuple —

```
(defun |comma2Tuple| (u)
  (cons '|@Tuple| (|postFlatten| u '|,|)))
```

—————

defun postFlatten

[postFlatten p386]

— defun postFlatten —

```
(defun |postFlatten| (x op)
  (let (a b)
    (cond
```

```
((and (consp x) (equal (qfirst x) op) (consp (qrest x))
      (consp (qcddr x)) (eq (qcdddr x) nil))
  (setq a (qsecond x))
  (setq b (qthird x))
  (append (|postFlatten| a op) (|postFlatten| b op)))
(t (list x))))
```

deflist postConstruct plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|construct| '|postTran|) '|postConstruct|))
```

defun postConstruct

```
[comma2Tuple p386]
[postTranSegment p388]
[postMakeCons p382]
[tuple2List p547]
[postTranList p372]
[postTran p370]
```

— defun postConstruct —

```
(defun |postConstruct| (u)
  (let (b a tmp4 tmp7)
    (cond
      ((and (consp u) (eq (qfirst u) '|construct|)
            (consp (qrest u)) (eq (qcddr u) nil))
       (setq b (qsecond u))
       (setq a
         (if (and (consp b) (eq (qfirst b) '|,|'))
             (|comma2Tuple| b)
             b))
       (cond
         ((and (consp a) (eq (qfirst a) 'segment) (consp (qrest a))
               (consp (qcddr a)) (eq (qcdddr a) nil))
          (list '|construct| (|postTranSegment| (second a) (third a))))
         ((and (consp a) (eq (qfirst a) '@Tuple|))
```

```

(cond
  ((dolist (x (qrest a) tmp4)
    (setq tmp4
      (or tmp4
        (and (consp x) (eq (qfirst x) '|:|)
          (consp (qrest x)) (eq (qcddr x) nil))))))
    (|postMakeCons| (qrest a)))
  ((dolist (x (qrest a) tmp7)
    (setq tmp7 (or tmp7 (and (consp x) (eq (qfirst x) 'segment)))))
    (|tuple2List| (qrest a)))
  (t (cons '|construct| (postTranList (qrest a)))))
(t (list '|construct| (|postTran| a))))
(t u)))

```

defun postTranSegment

[postTran p370]

— defun postTranSegment —

```

(defun |postTranSegment| (p q)
  (list 'segment (|postTran| p) (when q (|postTran| q))))

```

defplist postDef plist

— postvars —

```

(eval-when (eval load)
  (setf (get '|==| '|postTran|) '|postDef|))

```

defun postDef

[postMDef p395]

[recordHeaderDocumentation p470]

[postTran p370]

[postDefArgs p390]


```
[nreverse0 p??]
[$boot p??]
[$maxSignatureLineNumber p??]
[$headerDocumentation p??]
[$docList p??]
[$InteractiveMode p??]
```

— defun postDef —

```
(defun |postDef| (arg)
  (let (defOp rhs lhs targetType tmp1 op arg1 newLhs
        argTypeList typeList form specialCaseForm tmp4 tmp6 tmp8)
    (declare (special $boot |$maxSignatureLineNumber| |$headerDocumentation|
                      |$docList| |$InteractiveMode|))
    (setq defOp (first arg))
    (setq lhs (second arg))
    (setq rhs (third arg))
    (if (and (consp lhs) (eq (qfirst lhs) '|macro|)
            (consp (qrest lhs)) (eq (qcddr lhs) nil))
        (|postMDef| (list '==> (second lhs) rhs))
        (progn
          (unless $boot (|recordHeaderDocumentation| nil))
          (when (not (eql |$maxSignatureLineNumber| 0))
            (setq |$docList|
                  (cons (cons '|constructor| |$headerDocumentation|) |$docList|))
            (setq |$maxSignatureLineNumber| 0))
          (setq lhs (|postTran| lhs))
          (setq tmp1
            (if (and (consp lhs) (eq (qfirst lhs) '|:|') (cdr lhs) (list lhs nil)))
            (setf tmp1 (first tmp1)))
          (setq targetType (second tmp1))
          (when (and (null |$InteractiveMode|) (atom form)) (setf form (list form)))
          (setq newLhs
            (if (atom form)
                form
                (progn
                  (setf tmp1
                    (dolist (x form (nreverse0 tmp4))
                      (push
                        (if (and (consp x) (eq (qfirst x) '|:|') (consp (qrest x))
                                (consp (qcddr x)) (eq (qcdddr x) nil))
                            (second x)
                            x)
                        tmp4)))
                  (setf op (car tmp1))
                  (setf arg1 (cdr tmp1))
                  (cons op (|postDefArgs| arg1))))))
          (setf argTypeList
            (unless (atom form)
```

```

(dolist (x (cdr form) (nreverse0 tmp6))
  (push
    (when (and (consp x) (eq (qfirst x) '[:]) (consp (qrest x))
      (consp (qcddr x)) (eq (qcdddr x) nil))
      (third x))
    tmp6))))
(setq typeList (cons targetType argTypeList))
(when (atom form) (setq form (list form)))
(setq specialCaseForm (dolist (x form (nreverse tmp8)) (push nil tmp8)))
(list 'def newLhs typeList specialCaseForm (|postTran| rhs))))))

```

defun postDefArgs

[postError p374]
 [postDefArgs p390]

— defun postDefArgs —

```

(defun |postDefArgs| (args)
  (let (a b)
    (cond
      ((null args) args)
      ((and (consp args) (consp (qfirst args)) (eq (qcaar args) '[:])
        (consp (qcddr args)) (eq (qcdddr args) nil))
        (setq a (qcadar args))
        (setq b (qrest args))
        (cond
          (b (postError
            (list " Argument" a "of indefinite length must be last"))
            ((or (atom a) (and (consp a) (eq (qfirst a) 'quote)))
              a)
            (t
              (postError
                (list " Argument" a "of indefinite length must be a name")))))
          (t (cons (car args) (|postDefArgs| (cdr args)))))))

```

defplist postExit plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|=>' '|postTran|) '|postExit|))
```

defun postExit

[postTran p370]

— defun postExit —

```
(defun |postExit| (arg)
  (list 'if (|postTran| (second arg))
        (list '|exit| (|postTran| (third arg))
              '|noBranch|)))
```

defplist postIf plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|if|' '|postTran|) '|postIf|))
```

defun postIf

[nreverse0 p??]
 [postTran p370]
 [\$boot p??]

— defun postIf —

```
(defun |postIf| (arg)
  (let (tmp1)
    (if (null (and (consp arg) (eq (qfirst arg) '|if|)))
        arg
        (cons 'if
              (dolist (x (qrest arg) (nreverse0 tmp1))
                (push
```

```
(if (and (null (setq x (|postTran| x))) (null $boot)) 'noBranch| x)
tmp1))))))
```

defplist postin plist

— postvars —

```
(eval-when (eval load)
  (setf (get 'in| 'postTran|) 'postin|))
```

defun postin

```
[systemErrorHere p??]
[postTran p370]
[postInSeq p392]
```

— defun postin —

```
(defun |postin| (arg)
  (if (null (and (consp arg) (eq (qfirst arg) 'in|) (consp (qrest arg))
                (consp (qcddr arg)) (eq (qcddr arg) nil)))
      (|systemErrorHere| "postin")
      (list 'in| (|postTran| (second arg)) (|postInSeq| (third arg)))))
```

defun postInSeq

```
[postTranSegment p388]
[tuple2List p547]
[postTran p370]
```

— defun postInSeq —

```
(defun |postInSeq| (seq)
  (cond
    ((and (consp seq) (eq (qfirst seq) 'segment) (consp (qrest seq))
```

```

      (consp (qcddr seq)) (eq (qcdddr seq) nil))
    (|postTranSegment| (second seq) (third seq)))
  ((and (consp seq) (eq (qfirst seq) '@Tuple|))
    (|tuple2List| (qrest seq)))
  (t (|postTran| seq))))

```

defplist postIn plist

— postvars —

```

(eval-when (eval load)
  (setf (get 'in '|postTran|) '|postIn|))

```

defun postIn

```

[systemErrorHere p??]
[postTran p370]
[postInSeq p392]

```

— defun postIn —

```

(defun |postIn| (arg)
  (if (null (and (consp arg) (eq (qfirst arg) 'in) (consp (qrest arg))
    (consp (qcddr arg)) (eq (qcdddr arg) nil))))
    (|systemErrorHere| "postIn")
    (list 'in (|postTran| (second arg)) (|postInSeq| (third arg)))))

```

defplist postJoin plist

— postvars —

```

(eval-when (eval load)
  (setf (get '|Join| '|postTran|) '|postJoin|))

```

defun postJoin

[postTran p370]
 [postTranList p372]

— defun postJoin —

```
(defun |postJoin| (arg)
  (let (a l al)
    (setq a (|postTran| (cadr arg)))
    (setq l (postTranList (cddr arg)))
    (when (and (consp l) (eq (qrest l) nil) (consp (qfirst l))
              (member (qcaar l) '(attribute signature))))
      (setq l (list (list 'category (qfirst l)))))
    (setq al (if (and (consp a) (eq (qfirst a) '|@Tuple|)) (qrest a) (list a)))
    (cons '|Join| (append al l))))
```

—————

defplist postMapping plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|->| '|postTran|) '|postMapping|))
```

—————

defun postMapping

[postTran p370]
 [unTuple p413]

— defun postMapping —

```
(defun |postMapping| (u)
  (if (null (and (consp u) (eq (qfirst u) '|->|) (consp (qrest u))
                (consp (qcddr u)) (eq (qcdddr u) nil))))
      u
      (cons '|Mapping|
            (cons (|postTran| (third u))
                  (|unTuple| (|postTran| (second u)))))))
```

—————

defplist postMDef plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|==>' '|postTran|) '|postMDef|))
```

—————

defun postMDef

```
[postTran p370]
[throwkeyedmsg p??]
[nreverse0 p??]
[$InteractiveMode p??]
[$boot p??]
```

— defun postMDef —

```
(defun |postMDef| (arg)
  (let (rhs lhs tmp1 targetType form newLhs typeList tmp4 tmp5 tmp8)
    (declare (special |$InteractiveMode| $boot))
    (setq lhs (second arg))
    (setq rhs (third arg))
    (cond
      ((and |$InteractiveMode| (null $boot))
       (setq lhs (|postTran| lhs))
       (if (null (identp lhs))
           (|throwkeyedmsg| "The left-hand side of a => form must be a symbol." nil)
           (list 'mdef lhs nil nil (|postTran| rhs)))))
      (t
       (setq lhs (|postTran| lhs))
       (setq tmp1
        (if (and (consp lhs) (eq (qfirst lhs) '|:|') (cdr lhs) (list lhs nil)))
        (list lhs nil)))
       (setq form (first tmp1))
       (setq targetType (second tmp1))
       (setq form (if (atom form) (list form) form))
       (setq newLhs
        (dolist (x form (nreverse0 tmp4))
          (push
           (if (and (consp x) (eq (qfirst x) '|:|') (consp (qrest x))) (second x) x)
           tmp4)))
       (setq typeList
        (cons targetType
         (dolist (x (qrest form) (nreverse0 tmp5))
           (push
```

```

      (when (and (consp x) (eq (qfirst x) '|:|) (consp (qrest x))
                (consp (qcddr x)) (eq (qcddr x) nil))
            (third x))
      tmp5))))
(list 'mdef newLhs typeList
      (dolist (x form (nreverse0 tmp8)) (push nil tmp8))
      (|postTran| rhs))))))

```

defplist postPretend plist

— postvars —

```

(eval-when (eval load)
  (setf (get '|pretend| '|postTran|) '|postPretend|))

```

defun postPretend

```

[postTran p370]
[postType p379]

```

— defun postPretend —

```

(defun |postPretend| (arg)
  (cons '|pretend| (cons (|postTran| (second arg)) (|postType| (third arg)))))

```

defplist postQUOTE plist

— postvars —

```

(eval-when (eval load)
  (setf (get '|quote| '|postTran|) '|postQUOTE|))

```

defun postQUOTE

— defun postQUOTE —

```
(defun |postQUOTE| (arg) arg)
```

—————

defplist postReduce plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|Reduce| '|postTran|) '|postReduce|))
```

—————

defun postReduce

```
[postTran p370]
[postReduce p397]
[$InteractiveMode p??]
```

— defun postReduce —

```
(defun |postReduce| (arg)
  (let (op expr g)
    (setq op (second arg))
    (setq expr (third arg))
    (if (or |$InteractiveMode| (and (consp expr) (eq (qfirst expr) 'collect)))
        (list 'reduce op 0 (|postTran| expr))
        (|postReduce|
         (list '|Reduce| op
              (list 'collect
                   (list 'in (setq g (gensym)) expr)
                   (list '|construct| g))))))))
```

—————

defplist postRepeat plist

— postvars —

```
(eval-when (eval load)
  (setf (get 'repeat '|postTran|) '|postRepeat|))
```

—————

defun postRepeat

[postIteratorList p384]

[postTran p370]

— defun postRepeat —

```
(defun |postRepeat| (arg)
  (let (tmp1 x m)
    (setq tmp1 (reverse (cdr arg)))
    (setq x (car tmp1))
    (setq m (nreverse (cdr tmp1)))
    (cons 'repeat (append (|postIteratorList| m) (list (|postTran| x))))))
```

—————

defplist postScripts plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|Scripts| '|postTran|) '|postScripts|))
```

—————

defun postScripts

[getScriptName p408]

[postTranScripts p372]

— defun postScripts —

```
(defun |postScripts| (arg)
  (cons (getScriptName (second arg) (third arg) 0)
        (postTranScripts (third arg))))
```

defplist postSemiColon plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|;| '|postTran|) '|postSemiColon|))
```

defun postSemiColon

```
[postBlock p380]
[postFlattenLeft p399]
```

— defun postSemiColon —

```
(defun |postSemiColon| (u)
  (|postBlock| (cons '|Block| (|postFlattenLeft| u '|;|))))
```

defun postFlattenLeft

```
[postFlattenLeft p399]
```

— defun postFlattenLeft —

```
(defun |postFlattenLeft| (x op)
  (let (a b)
    (cond
      ((and (consp x) (equal (qfirst x) op) (consp (qrest x))
            (consp (qcddr x)) (eq (qcdddr x) nil))
       (setq a (qsecond x))
       (setq b (qthird x))
       (append (|postFlattenLeft| a op) (list b)))
      (t (list x)))))
```

defplist postSignature plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|Signature| '|postTran|) '|postSignature|))
```

defun postSignature

[postType p379]
 [removeSuperfluousMapping p400]
 [killColons p401]

— defun postSignature —

```
(defun |postSignature| (arg)
  (let (sig sig1 op)
    (setq op (second arg))
    (setq sig (third arg))
    (when (and (consp sig) (eq (qfirst sig) '->))
      (setq sig1 (|postType| sig))
      (setq op (postAtom (if (stringp op) (setq op (intern op)) op)))
      (cons 'signature
            (cons op (|removeSuperfluousMapping| (|killColons| sig1)))))))
```

defun removeSuperfluousMapping

— defun removeSuperfluousMapping —

```
(defun |removeSuperfluousMapping| (sig1)
  (if (and (consp sig1) (consp (qfirst sig1))) (eq (qcaar sig1) '|Mapping|))
      (cons (cdr (qfirst sig1)) (qrest sig1))
      sig1))
```

defun killColons

[killColons p401]

— defun killColons —

```
(defun |killColons| (x)
  (cond
    ((atom x) x)
    ((and (consp x) (eq (qfirst x) '|Record|)) x)
    ((and (consp x) (eq (qfirst x) '|Union|)) x)
    ((and (consp x) (eq (qfirst x) '|:|) (consp (qrest x))
          (consp (qcddr x)) (eq (qcddr x) nil))
     (|killColons| (third x)))
    (t (cons (|killColons| (car x)) (|killColons| (cdr x))))))
```

defplist postSlash plist

— postvars —

```
(eval-when (eval load)
  (setf (get '/ '|postTran|) '|postSlash|))
```

defun postSlash

[postTran p370]

— defun postSlash —

```
(defun |postSlash| (arg)
  (if (stringp (second arg))
      (|postTran| (list '|Reduce| (intern (second arg)) (third arg) ))
      (list '/ (|postTran| (second arg)) (|postTran| (third arg)))))
```

defplist postTuple plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|@Tuple| '|postTran|) '|postTuple|))
```

—————

defun postTuple

[postTranList p372]

— defun postTuple —

```
(defun |postTuple| (arg)
  (cond
    ((and (consp arg) (eq (qrest arg) nil) (eq (qfirst arg) '|@Tuple|))
     arg)
    ((and (consp arg) (eq (qfirst arg) '|@Tuple|) (consp (qrest arg)))
     (cons '|@Tuple| (postTranList (cdr arg))))))
```

—————

defplist postTupleCollect plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|TupleCollect| '|postTran|) '|postTupleCollect|))
```

—————

defun postTupleCollect

[postCollect p383]

— defun postTupleCollect —

```
(defun |postTupleCollect| (arg)
```

```
(let (constructOp tmp1 x m)
  (setq constructOp (car arg))
  (setq tmp1 (reverse (cdr arg)))
  (setq x (car tmp1))
  (setq m (nreverse (cdr tmp1)))
  (|postCollect| (cons constructOp (append m (list (list '|construct| x)))))))
```

defplist postWhere plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|where| '|postTran|) '|postWhere|))
```

defun postWhere

[postTran p370]
[postTranList p372]

— defun postWhere —

```
(defun |postWhere| (arg)
  (let (b x)
    (setq b (third arg))
    (setq x (if (and (consp b) (eq (qfirst b) '|Block|)) (qrest b) (list b)))
    (cons '|where| (cons (|postTran| (second arg)) (postTranList x)))))
```

defplist postWith plist

— postvars —

```
(eval-when (eval load)
  (setf (get '|with| '|postTran|) '|postWith|))
```

defun postWith

```
[postTran p370]
[$insidePostCategoryIfTrue p??]
```

— defun postWith —

```
(defun |postWith| (arg)
  (let (|$insidePostCategoryIfTrue| a)
    (declare (special |$insidePostCategoryIfTrue|))
    (setq |$insidePostCategoryIfTrue| t)
    (setq a (|postTran| (second arg)))
    (cond
      ((and (consp a) (member (qfirst a) '(signature attribute if)))
       (list 'category a))
      ((and (consp a) (eq (qfirst a) 'progn))
       (cons 'category (qrest a)))
      (t a))))
```

7.3 Support routines**defun setDefOp**

```
[$defOp p??]
[$topOp p??]
```

— defun setDefOp —

```
(defun setDefOp (f)
  (let (tmp1)
    (declare (special |$defOp| |$topOp|))
    (when (and (consp f) (eq (qfirst f) '|:|)
              (consp (setq tmp1 (qrest f)))))
      (setq f (qfirst tmp1)))
    (unless (atom f) (setq f (car f)))
    (if |$topOp|
      (setq |$defOp| f)
      (setq |$topOp| f))))
```

defun aplTran

```
[aplTran1 p405]
[containsBang p408]
[$genno p??]
[$boot p??]
```

— defun aplTran —

```
(defun aplTran (x)
  (let ($genno u)
    (declare (special $genno $boot))
    (cond
      ($boot x)
      (t
       (setq $genno 0)
       (setq u (aplTran1 x))
       (cond
         ((containsBang u)
          (|throwKeyedMsg|
           (format nil
            " AXIOM cannot now process ! in the way you have used it. ~
            Use parentheses, if appropriate.")
           nil))
         (t u))))))
```

—————

defun aplTran1

```
[aplTranList p407]
[aplTran1 p405]
[hasAplExtension p407]
[nreverse0 p??]
[$boot p??]
```

— defun aplTran1 —

```
(defun aplTran1 (x)
  (let (op argl1 argl f y opprime yprime tmp1 arglAssoc futureArgl g)
    (declare (special $boot))
    (if (atom x)
        x
        (progn
         (setq op (car x))
         (setq argl1 (cdr x))
         (setq argl (aplTranList argl1))
```

```

(cond
  ((eq op '!')
   (cond
    ((and (consp arg1)
          (progn
           (setq f (qfirst arg1))
           (setq tmp1 (qrest arg1))
           (and (consp tmp1)
                (eq (qrest tmp1) nil)
                (progn
                 (setq y (qfirst tmp1))
                 t))))))
    (cond
     ((and (consp y)
           (progn
            (setq opprime (qfirst y))
            (setq yprime (qrest y))
            t)
          (eq opprime '!'))
      (aplTran1 (cons op (cons op (cons f yprime))))))
    ($boot
     (cons 'collect
           (cons
            (list 'in (setq g (genvar)) (aplTran1 y))
            (list (list f g ) )))))
    (t
     (list 'map f (aplTran1 y) ))))
  (t x)))
((progn
  (setq tmp1 (hasAplExtension arg1))
  (and (consp tmp1)
       (progn
        (setq arglAssoc (qfirst tmp1))
        (setq futureArg1 (qrest tmp1))
        t)))
 (cons '|reshape|
       (cons
        (cons 'collect
              (append
               (do ((tmp3 arglAssoc (cdr tmp3)) (tmp4 nil))
                   ((or (atom tmp3)
                        (progn (setq tmp4 (car tmp3)) nil)
                        (progn
                         (setq g (car tmp4))
                         (setq a (cdr tmp4))
                         nil))
                    (nreverse0 tmp2))
               (push (list 'in g (list 'identity a))) tmp2)))
        (list (aplTran1 (cons op futureArg1))))
        (list (cdar arglAssoc))))))

```

```
(t (cons op arg1))))))
```

defun aplTranList

```
[aplTran1 p405]  
[aplTranList p407]
```

— defun aplTranList —

```
(defun aplTranList (x)  
  (if (atom x)  
      x  
      (cons (aplTran1 (car x)) (aplTranList (cdr x)))))
```

defun hasAplExtension

```
[nreverse0 p??]  
[deepestExpression p408]  
[genvar p??]  
[aplTran1 p405]
```

— defun hasAplExtension —

```
(defun hasAplExtension (arg1)  
  (let (tmp2 tmp3 y z g arglAssoc u)  
    (when  
      (dolist (x arg1 tmp2)  
        (setq tmp2 (or tmp2 (and (consp x) (eq (qfirst x) '!))))))  
    (setq u  
      (dolist (x arg1 (nreverse0 tmp3))  
        (push  
          (if (and (consp x) (eq (qfirst x) '!)  
                  (consp (qrest x)) (eq (qcddr x) nil))  
              (progn  
                (setq y (qsecond x))  
                (setq z (deepestExpression y))  
                (setq arglAssoc  
                  (cons (cons (setq g (genvar)) (aplTran1 z)) arglAssoc))  
                (subst g z y :test #'equal))  
              x)  
          tmp3)))
```

```
(cons arglAssoc u))))
```

defun deepestExpression

[deepestExpression p408]

— defun deepestExpression —

```
(defun deepestExpression (x)
  (if (and (consp x) (eq (qfirst x) '!))
      (consp (qrest x)) (eq (qcddr x) nil))
      (deepestExpression (qsecond x))
      x))
```

defun containsBang

[containsBang p408]

— defun containsBang —

```
(defun containsBang (u)
  (let (tmp2)
    (cond
      ((atom u) (eq u '!))
      ((and (consp u) (equal (qfirst u) 'quote)
            (consp (qrest u)) (eq (qcddr u) nil))
       nil)
      (t
       (dolist (x u tmp2)
         (setq tmp2 (or tmp2 (containsBang x)))))))
```

defun getScriptName

[getScriptName identp (vol5)]
 [postError p374]
 [internl p??]
 [decodeScripts p409]

```
[getScriptName pname (vol5)]
```

— **defun getScriptName** —

```
(defun getScriptName (op a numberOfFunctionalArgs)
  (when (null (identp op))
    (postError (list " " op " cannot have scripts" )))
  (internl '* (princ-to-string numberOfFunctionalArgs)
    (decodeScripts a) (pname op)))
```

defun decodeScripts

```
[strconc p??]
[decodeScripts p409]
```

— **defun decodeScripts** —

```
(defun decodeScripts (a)
  (labels (
    (fn (a)
      (let ((tmp1 0))
        (if (and (consp a) (eq (qfirst a) '|,|))
          (dolist (x (qrest a) tmp1) (setq tmp1 (+ tmp1 (fn x))))
          1))))
    (cond
      ((and (consp a) (eq (qfirst a) '|PrefixSC|)
        (consp (qrest a)) (eq (qcddr a) nil))
       (strconc (princ-to-string 0) (decodeScripts (qsecond a))))
      ((and (consp a) (eq (qfirst a) '|;|))
       (apply 'strconc (loop for x in (qrest a) collect (decodeScripts x))))
      ((and (consp a) (eq (qfirst a) '|,|))
       (princ-to-string (fn a)))
      (t
       (princ-to-string 1)))))
```

Chapter 8

DEF forms

defvar \$defstack

— initvars —

(defvar \$defstack nil)

—————

defvar \$is-spill

— initvars —

(defvar \$is-spill nil)

—————

defvar \$is-spill-list

— initvars —

(defvar \$is-spill-list nil)

—————

defvar \$vl

— initvars —

```
(defvar $vl nil)
```

—————

defvar \$is-gensymlist

— initvars —

```
(defvar $is-gensymlist nil)
```

—————

defvar initial-gensym

— initvars —

```
(defvar initial-gensym (list (gensym)))
```

—————

defvar \$is-eqlist

— initvars —

```
(defvar $is-eqlist nil)
```

—————

defun hackforis

[hackforis1 p[413](#)]

— defun hackforis —


```
(defun hackforis (l) (mapcar #'hackforis1 L))
```

defun hackforis1

```
[eqcar p??]
```

— defun hackforis1 —

```
(defun hackforis1 (x)
  (if (and (member (ifcar x) '(in on)) (eqcar (second x) 'is))
      (cons (first x) (cons (cons 'setq (cdadr x)) (cddr x)))
      x))
```

defun unTuple

— defun unTuple —

```
(defun |unTuple| (x)
  (if (and (consp x) (eq (qfirst x) '|@Tuple|))
      (qrest x)
      (list x)))
```

8.1 The PARSE code

defvar tmptok

— initvars —

```
(defvar |tmptok| nil)
```

defvar tok

— initvars —

```
(defvar tok nil)
```

—————

defvar ParseMode

— initvars —

```
(defvar |ParseMode| nil)
```

—————

defvar definition-name

— initvars —

```
(defvar definition-name nil)
```

—————

defvar lablasoc

— initvars —

```
(defvar lablasoc nil)
```

—————

defun PARSE-NewExpr

```
[match-string p451]
```

```
[action p464]
```

[PARSE-NewExpr processSynonyms (vol5)]
 [must p463]
 [current-symbol p457]
 [PARSE-Statement p419]
 [definition-name p414]

— defun PARSE-NewExpr —

```
(defun |PARSE-NewExpr| ()
  (or (and (match-string ")") (action (|processSynonyms|))
        (must (|PARSE-Command|)))
      (and (action (setq definition-name (current-symbol)))
            (|PARSE-Statement|))))
```

—————

defun PARSE-Command

[match-advance-string p452]
 [must p463]
 [PARSE-SpecialKeyWord p415]
 [PARSE-SpecialCommand p416]
 [push-reduction p465]

— defun PARSE-Command —

```
(defun |PARSE-Command| ()
  (and (match-advance-string ")") (must (|PARSE-SpecialKeyWord|))
        (must (|PARSE-SpecialCommand|))
        (push-reduction '|PARSE-Command| nil)))
```

—————

defun PARSE-SpecialKeyWord

[match-current-token p456]
 [action p464]
 [token-symbol p??]
 [current-token p458]
 [PARSE-SpecialKeyWord unAbbreviateKeyword (vol5)]
 [current-symbol p457]

— defun PARSE-SpecialKeyWord —

```
(defun |PARSE-SpecialKeyword| ()
  (and (match-current-token 'identifier)
       (action (setf (token-symbol (current-token))
                     (|unAbbreviateKeyword| (current-symbol))))))
```

defun PARSE-SpecialCommand

```
[match-advance-string p452]
[bang p??]
[optional p464]
[PARSE-Expression p422]
[push-reduction p465]
[PARSE-SpecialCommand p416]
[pop-stack-1 p548]
[PARSE-CommandTail p418]
[must p463]
[current-symbol p457]
[action p464]
[PARSE-TokenList p418]
[PARSE-TokenCommandTail p417]
[star p464]
[PARSE-PrimaryOrQM p419]
[PARSE-CommandTail p418]
[$noParseCommands p??]
[$tokenCommands p??]
```

— defun PARSE-SpecialCommand —

```
(defun |PARSE-SpecialCommand| ()
  (declare (special |$noParseCommands| |$tokenCommands|))
  (or (and (match-advance-string "show")
           (bang fil_test
              (optional
                (or (match-advance-string "?")
                    (|PARSE-Expression|))))
           (push-reduction '|PARSE-SpecialCommand|
                           (list '|show| (pop-stack-1)))
           (must (|PARSE-CommandTail|)))
      (and (member (current-symbol) |$noParseCommands|)
           (action (funcall (current-symbol))))
      (and (member (current-symbol) |$tokenCommands|)
           (|PARSE-TokenList|) (must (|PARSE-TokenCommandTail|)))
      (and (star repeater (|PARSE-PrimaryOrQM|))
           (must (|PARSE-CommandTail|)))))
```

defun PARSE-TokenCommandTail

```
[bang p??]
[optional p464]
[star p464]
[PARSE-TokenOption p417]
[atEndOfLine p??]
[push-reduction p465]
[PARSE-TokenCommandTail p417]
[pop-stack-2 p548]
[pop-stack-1 p548]
[action p464]
[PARSE-TokenCommandTail systemCommand (vol5)]
```

— **defun PARSE-TokenCommandTail** —

```
(defun |PARSE-TokenCommandTail| ()
  (and (bang fil_test (optional (star repeater (|PARSE-TokenOption|))))
    (|atEndOfLine|)
    (push-reduction '|PARSE-TokenCommandTail|
      (cons (pop-stack-2) (append (pop-stack-1) nil)))
    (action (|systemCommand| (pop-stack-1)))))
```

defun PARSE-TokenOption

```
[match-advance-string p452]
[must p463]
[PARSE-TokenList p418]
```

— **defun PARSE-TokenOption** —

```
(defun |PARSE-TokenOption| ()
  (and (match-advance-string "") (must (|PARSE-TokenList|))))
```

defun PARSE-TokenList

```
[star p464]
[isTokenDelimiter p455]
[push-reduction p465]
[current-symbol p457]
[action p464]
[advance-token p459]
```

— defun PARSE-TokenList —

```
(defun |PARSE-TokenList| ()
  (star repeater
    (and (not (|isTokenDelimiter|))
      (push-reduction '|PARSE-TokenList| (current-symbol))
      (action (advance-token)))))
```

defun PARSE-CommandTail

```
[bang p??]
[optional p464]
[star p464]
[push-reduction p465]
[PARSE-Option p419]
[PARSE-CommandTail p418]
[pop-stack-2 p548]
[pop-stack-1 p548]
[action p464]
[PARSE-CommandTail systemCommand (vol5)]
```

— defun PARSE-CommandTail —

```
(defun |PARSE-CommandTail| ()
  (and (bang fil_test (optional (star repeater (|PARSE-Option|))))
    (|atEndOfLine|)
    (push-reduction '|PARSE-CommandTail|
      (cons (pop-stack-2) (append (pop-stack-1) nil)))
    (action (|systemCommand| (pop-stack-1)))))
```

defun PARSE-PrimaryOrQM

[match-advance-string p452]
 [push-reduction p465]
 [PARSE-PrimaryOrQM p419]
 [PARSE-Primary p431]

— **defun PARSE-PrimaryOrQM** —

```
(defun |PARSE-PrimaryOrQM| ()
  (or (and (match-advance-string "?")
            (push-reduction '|PARSE-PrimaryOrQM| '?))
      (|PARSE-Primary|)))
```

—————

defun PARSE-Option

[match-advance-string p452]
 [must p463]
 [star p464]
 [PARSE-PrimaryOrQM p419]

— **defun PARSE-Option** —

```
(defun |PARSE-Option| ()
  (and (match-advance-string ")")
       (must (star repeater (|PARSE-PrimaryOrQM|))))))
```

—————

defun PARSE-Statement

[PARSE-Expr p423]
 [optional p464]
 [star p464]
 [match-advance-string p452]
 [must p463]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— **defun PARSE-Statement** —

```
(defun |PARSE-Statement| ()
  (and (|PARSE-Expr| 0)
    (optional
      (and (star repeater
        (and (match-advance-string ",")
          (must (|PARSE-Expr| 0))))
        (push-reduction '|PARSE-Statement|
          (cons '|Series|
            (cons (pop-stack-2)
              (append (pop-stack-1) nil))))))))))
```

defun PARSE-InfixWith

[[PARSE-With](#) p420]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-InfixWith —

```
(defun |PARSE-InfixWith| ()
  (and (|PARSE-With|)
    (push-reduction '|PARSE-InfixWith|
      (list '|Join| (pop-stack-2) (pop-stack-1)))))
```

defun PARSE-With

[match-advance-string p452]
 [must p463]
 [push-reduction p465]
 [pop-stack-1 p548]

— defun PARSE-With —

```
(defun |PARSE-With| ()
  (and (match-advance-string "with") (must (|PARSE-Category|))
    (push-reduction '|PARSE-With|
      (cons '|with| (cons (pop-stack-1) nil)))))
```

defun PARSE-Category

```

[match-advance-string p452]
[must p463]
[bang p??]
[optional p464]
[push-reduction p465]
[PARSE-Expression p422]
[PARSE-Category p421]
[pop-stack-3 p549]
[pop-stack-2 p548]
[pop-stack-1 p548]
[star p464]
[line-number p??]
[PARSE-Application p429]
[action p464]
[recordSignatureDocumentation p468]
[nth-stack p549]
[recordAttributeDocumentation p468]
[current-line p632]

```

— **defun PARSE-Category** —

```

(defun |PARSE-Category| ()
  (let (g1)
    (or (and (match-advance-string "if") (must (|PARSE-Expression|))
             (must (match-advance-string "then"))
             (must (|PARSE-Category|))
             (bang fil_test
              (optional
               (and (match-advance-string "else")
                    (must (|PARSE-Category|))))))
        (push-reduction '|PARSE-Category|
          (list '|if| (pop-stack-3) (pop-stack-2) (pop-stack-1))))
    (and (match-advance-string "(") (must (|PARSE-Category|))
         (bang fil_test
          (optional
           (star repeater
            (and (match-advance-string ";")
                  (must (|PARSE-Category|))))))
         (must (match-advance-string ")"))
         (push-reduction '|PARSE-Category|
          (cons 'category
                (cons (pop-stack-2)
                      (append (pop-stack-1) nil))))))
    (and (action (setq g1 (line-number current-line))
                (|PARSE-Application|)
                (must (or (and (match-advance-string ":")

```

```

      (must (|PARSE-Expression|))
      (push-reduction '|PARSE-Category|
        (list '|Signature| (pop-stack-2) (pop-stack-1) ))
      (action (|recordSignatureDocumentation|
        (nth-stack 1) g1)))
    (and (push-reduction '|PARSE-Category|
      (list '|Attribute| (pop-stack-1) ))
      (action (|recordAttributeDocumentation|
        (nth-stack 1) g1)))))))))

```

defun PARSE-Expression

[\[PARSE-Expr p423\]](#)
[\[PARSE-rightBindingPowerOf p425\]](#)
[\[make-symbol-of p458\]](#)
[\[push-reduction p465\]](#)
[\[pop-stack-1 p548\]](#)
[\[ParseMode p414\]](#)
[\[prior-token p97\]](#)

— defun PARSE-Expression —

```

(defun |PARSE-Expression| ()
  (declare (special prior-token))
  (and (|PARSE-Expr|
    (|PARSE-rightBindingPowerOf| (make-symbol-of prior-token)
      |ParseMode|))
    (push-reduction '|PARSE-Expression| (pop-stack-1))))

```

defun PARSE-Import

[\[match-advance-string p452\]](#)
[\[must p463\]](#)
[\[PARSE-Expr p423\]](#)
[\[bang p??\]](#)
[\[optional p464\]](#)
[\[star p464\]](#)
[\[push-reduction p465\]](#)
[\[pop-stack-2 p548\]](#)
[\[pop-stack-1 p548\]](#)

— defun PARSE-Import —

```
(defun |PARSE-Import| ()
  (and (match-advance-string "import") (must (|PARSE-Expr| 1000))
    (bang fil_test
      (optional
        (star repeater
          (and (match-advance-string ",")
            (must (|PARSE-Expr| 1000))))))
      (push-reduction '|PARSE-Import|
        (cons '|import|
          (cons (pop-stack-2) (append (pop-stack-1) nil)))))))
```

—————

defun PARSE-Expr

```
[PARSE-NudPart p424]
[PARSE-LedPart p423]
[optional p464]
[star p464]
[push-reduction p465]
[pop-stack-1 p548]
```

— defun PARSE-Expr —

```
(defun |PARSE-Expr| (rbp)
  (declare (special rbp))
  (and (|PARSE-NudPart| rbp)
    (optional (star opt_expr (|PARSE-LedPart| rbp)))
    (push-reduction '|PARSE-Expr| (pop-stack-1))))
```

—————

defun PARSE-LedPart

```
[PARSE-Operation p424]
[push-reduction p465]
[pop-stack-1 p548]
```

— defun PARSE-LedPart —

```
(defun |PARSE-LedPart| (rbp)
```

```
(declare (special rbp))
(and (|PARSE-Operation| ' |Led| rbp)
      (push-reduction ' |PARSE-LedPart| (pop-stack-1))))
```

defun PARSE-NudPart

```
[PARSE-Operation p424]
[PARSE-Reduction p428]
[PARSE-Form p429]
[push-reduction p465]
[pop-stack-1 p548]
[rbp p??]
```

— defun PARSE-NudPart —

```
(defun |PARSE-NudPart| (rbp)
  (declare (special rbp))
  (and (or (|PARSE-Operation| ' |Nud| rbp) (|PARSE-Reduction|)
           (|PARSE-Form|))
        (push-reduction ' |PARSE-NudPart| (pop-stack-1))))
```

defun PARSE-Operation

```
[match-current-token p456]
[current-symbol p457]
[PARSE-leftBindingPowerOf p425]
[lt p??]
[getl p??]
[action p464]
[PARSE-rightBindingPowerOf p425]
[PARSE-getSemanticForm p425]
[elemn p??]
[ParseMode p414]
[rbp p??]
[tmptok p413]
```

— defun PARSE-Operation —

```
(defun |PARSE-Operation| (|ParseMode| rbp)
  (declare (special |ParseMode| rbp |tmptok|)))
```

```
(and (not (match-current-token 'identifier))
      (get1 (setq |tmptok| (current-symbol)) |ParseMode|)
      (lt rbp (|PARSE-leftBindingPowerOf| |tmptok| |ParseMode|))
      (action (setq rbp (|PARSE-rightBindingPowerOf| |tmptok| |ParseMode|)))
      (|PARSE-getSemanticForm| |tmptok| |ParseMode|
        (elemn (get1 |tmptok| |ParseMode|) 5 nil))))
```

defun PARSE-leftBindingPowerOf

```
[get1 p??]
[elemn p??]
```

— defun PARSE-leftBindingPowerOf —

```
(defun |PARSE-leftBindingPowerOf| (x ind)
  (declare (special x ind))
  (let ((y (get1 x ind))) (if y (elemn y 3 0) 0)))
```

defun PARSE-rightBindingPowerOf

```
[get1 p??]
[elemn p??]
```

— defun PARSE-rightBindingPowerOf —

```
(defun |PARSE-rightBindingPowerOf| (x ind)
  (declare (special x ind))
  (let ((y (get1 x ind))) (if y (elemn y 4 105) 105)))
```

defun PARSE-getSemanticForm

```
[PARSE-Prefix p426]
[PARSE-Infix p426]
```

— defun PARSE-getSemanticForm —

```
(defun |PARSE-getSemanticForm| (x ind y)
  (declare (special x ind y))
  (or (and y (eval y)) (and (eq ind '|Nud|) (|PARSE-Prefix|))
      (and (eq ind '|Led|) (|PARSE-Infix|))))
```

defun PARSE-Prefix

[push-reduction p465]
 [current-symbol p457]
 [action p464]
 [advance-token p459]
 [optional p464]
 [PARSE-TokTail p427]
 [must p463]
 [PARSE-Expression p422]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-Prefix —

```
(defun |PARSE-Prefix| ()
  (and (push-reduction '|PARSE-Prefix| (current-symbol))
       (action (advance-token)) (optional (|PARSE-TokTail|))
       (must (|PARSE-Expression|))
       (push-reduction '|PARSE-Prefix|
                        (list (pop-stack-2) (pop-stack-1)))))
```

defun PARSE-Infix

[push-reduction p465]
 [current-symbol p457]
 [action p464]
 [advance-token p459]
 [optional p464]
 [PARSE-TokTail p427]
 [must p463]
 [PARSE-Expression p422]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-Infix —

```
(defun |PARSE-Infix| ()
  (and (push-reduction '|PARSE-Infix| (current-symbol))
        (action (advance-token)) (optional (|PARSE-TokTail|))
        (must (|PARSE-Expression|))
        (push-reduction '|PARSE-Infix|
                          (list (pop-stack-2) (pop-stack-2) (pop-stack-1) )))))
```

—————

defun PARSE-TokTail

```
[current-symbol p457]
[current-char p460]
[char-eq p461]
[copy-token p??]
[action p464]
[PARSE-Qualification p427]
[$boot p??]
```

— defun PARSE-TokTail —

```
(defun |PARSE-TokTail| ()
  (let (g1)
    (and (null $boot) (eq (current-symbol) '$)
          (or (alpha-char-p (current-char))
                (char-eq (current-char) "$")
                (char-eq (current-char) "%")
                (char-eq (current-char) "("))
          (action (setq g1 (copy-token prior-token)))
          (|PARSE-Qualification|) (action (setq prior-token g1)))))
```

—————

defun PARSE-Qualification

```
[match-advance-string p452]
[must p463]
[PARSE-Primary1 p431]
[push-reduction p465]
[dollarTran p462]
[pop-stack-1 p548]
```

— defun PARSE-Qualification —

```
(defun |PARSE-Qualification| ()
  (and (match-advance-string "$") (must (|PARSE-Primary1|))
    (push-reduction '|PARSE-Qualification|
      (|dollarTran| (pop-stack-1) (pop-stack-1))))))
```

defun PARSE-Reduction

```
[PARSE-ReductionOp p428]
[must p463]
[PARSE-Expr p423]
[push-reduction p465]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-Reduction —

```
(defun |PARSE-Reduction| ()
  (and (|PARSE-ReductionOp|) (must (|PARSE-Expr| 1000))
    (push-reduction '|PARSE-Reduction|
      (list '|Reduce| (pop-stack-2) (pop-stack-1) )))))
```

defun PARSE-ReductionOp

```
[getl p??]
[current-symbol p457]
[match-next-token p457]
[action p464]
[advance-token p459]
```

— defun PARSE-ReductionOp —

```
(defun |PARSE-ReductionOp| ()
  (and (getl (current-symbol) '|Led|)
    (match-next-token 'special-char (code-char 47))
    (push-reduction '|PARSE-ReductionOp| (current-symbol))
    (action (advance-token)) (action (advance-token)))))
```

defun PARSE-Form

[match-advance-string p452]
 [bang p??]
 [optional p464]
 [must p463]
 [push-reduction p465]
 [pop-stack-1 p548]
 [PARSE-Application p429]

— defun PARSE-Form —

```
(defun |PARSE-Form| ()
  (or (and (match-advance-string "iterate")
    (bang fil_test
      (optional
        (and (match-advance-string "from")
          (must (|PARSE-Label|))
          (push-reduction '|PARSE-Form|
            (list (pop-stack-1))))))
    (push-reduction '|PARSE-Form|
      (cons '|iterate| (append (pop-stack-1) nil))))
    (and (match-advance-string "yield") (must (|PARSE-Application|))
      (push-reduction '|PARSE-Form|
        (list '|yield| (pop-stack-1))))
      (|PARSE-Application|)))
```

—————

defun PARSE-Application

[PARSE-Primary p431]
 [optional p464]
 [star p464]
 [PARSE-Selector p430]
 [PARSE-Application p429]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-Application —

```
(defun |PARSE-Application| ()
  (and (|PARSE-Primary|) (optional (star opt_expr (|PARSE-Selector|)))
    (optional
      (and (|PARSE-Application|))
```

```
(push-reduction '|PARSE-Application|
  (list (pop-stack-2) (pop-stack-1))))))
```

defun PARSE-Label

```
[match-advance-string p452]
[must p463]
[PARSE-Name p438]
```

— defun PARSE-Label —

```
(defun |PARSE-Label| ()
  (and (match-advance-string "<<") (must (|PARSE-Name|))
    (must (match-advance-string ">>"))))
```

defun PARSE-Selector

```
[current-symbol p457]
[char-ne p461]
[current-char p460]
[match-advance-string p452]
[must p463]
[PARSE-PrimaryNoFloat p431]
[push-reduction p465]
[pop-stack-2 p548]
[pop-stack-1 p548]
[PARSE-Float p432]
[PARSE-Primary p431]
[$boot p??]
```

— defun PARSE-Selector —

```
(defun |PARSE-Selector| ()
  (declare (special $boot))
  (or (and nonblank (eq (current-symbol) '|.|)
    (char-ne (current-char) '|' |) (match-advance-string ".")
    (must (|PARSE-PrimaryNoFloat|))
    (must (or (and $boot
      (push-reduction '|PARSE-Selector|
        (list 'elt (pop-stack-2) (pop-stack-1))))
```

```

      (push-reduction '|PARSE-Selector|
        (list (pop-stack-2) (pop-stack-1))))))
    (and (or (|PARSE-Float|)
      (and (match-advance-string ".")
        (must (|PARSE-Primary|))))
      (must (or (and $boot
        (push-reduction '|PARSE-Selector|
          (list 'elt (pop-stack-2) (pop-stack-1))))
        (push-reduction '|PARSE-Selector|
          (list (pop-stack-2) (pop-stack-1)))))))

```

defun PARSE-PrimaryNoFloat

[PARSE-Primary1 p431]
 [optional p464]
 [PARSE-TokTail p427]

— defun PARSE-PrimaryNoFloat —

```

(defun |PARSE-PrimaryNoFloat| ()
  (and (|PARSE-Primary1|) (optional (|PARSE-TokTail|))))

```

defun PARSE-Primary

[PARSE-Float p432]
 [PARSE-PrimaryNoFloat p431]

— defun PARSE-Primary —

```

(defun |PARSE-Primary| ()
  (or (|PARSE-Float|) (|PARSE-PrimaryNoFloat|)))

```

defun PARSE-Primary1

[PARSE-VarForm p437]
 [optional p464]

[\[current-symbol p457\]](#)
[\[PARSE-Primary1 p431\]](#)
[\[must p463\]](#)
[\[pop-stack-2 p548\]](#)
[\[pop-stack-1 p548\]](#)
[\[push-reduction p465\]](#)
[\[PARSE-Quad p436\]](#)
[\[PARSE-String p437\]](#)
[\[PARSE-IntegerTok p436\]](#)
[\[PARSE-FormalParameter p436\]](#)
[\[match-string p451\]](#)
[\[PARSE-Data p439\]](#)
[\[match-advance-string p452\]](#)
[\[PARSE-Expr p423\]](#)
[\[PARSE-Sequence p442\]](#)
[\[PARSE-Enclosure p435\]](#)
[\[\\$boot p??\]](#)

— **defun PARSE-Primary1** —

```

(defun |PARSE-Primary1| ()
  (declare (special $boot))
  (or (and (|PARSE-VarForm|)
    (optional
      (and nonblank (eq (current-symbol) '|(|)
        (must (|PARSE-Primary1|))
        (push-reduction '|PARSE-Primary1|
          (list (pop-stack-2) (pop-stack-1))))))
    (|PARSE-Quad|) (|PARSE-String|) (|PARSE-IntegerTok|)
    (|PARSE-FormalParameter|)
    (and (match-string "")
      (must (or (and $boot (|PARSE-Data|))
        (and (match-advance-string "")
          (must (|PARSE-Expr| 999))
          (push-reduction '|PARSE-Primary1|
            (list 'quote (pop-stack-1)))))))
    (|PARSE-Sequence|) (|PARSE-Enclosure|)))

```

defun PARSE-Float

[\[PARSE-FloatBase p433\]](#)
[\[must p463\]](#)
[\[PARSE-FloatExponent p434\]](#)
[\[push-reduction p465\]](#)

```
[make-float p??]
[pop-stack-4 p549]
[pop-stack-3 p549]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-Float —

```
(defun |PARSE-Float| ()
  (and (|PARSE-FloatBase|)
    (must (or (and nonblank (|PARSE-FloatExponent|))
      (push-reduction '|PARSE-Float| 0)))
    (push-reduction '|PARSE-Float|
      (make-float (pop-stack-4) (pop-stack-2) (pop-stack-2)
        (pop-stack-1))))))
```

—————→

defun PARSE-FloatBase

```
[current-symbol p457]
[char-eq p461]
[current-char p460]
[char-ne p461]
[next-char p460]
[PARSE-IntegerTok p436]
[must p463]
[PARSE-FloatBasePart p434]
[PARSE-IntegerTok p436]
[push-reduction p465]
[PARSE-FloatBase digitp (vol5)]
```

— defun PARSE-FloatBase —

```
(defun |PARSE-FloatBase| ()
  (or (and (integerp (current-symbol)) (char-eq (current-char) ".")
    (char-ne (next-char) ".") (|PARSE-IntegerTok|)
    (must (|PARSE-FloatBasePart|)))
    (and (integerp (current-symbol))
      (char-eq (char-upcase (current-char)) 'e)
      (|PARSE-IntegerTok|) (push-reduction '|PARSE-FloatBase| 0)
      (push-reduction '|PARSE-FloatBase| 0))
    (and (digitp (current-char)) (eq (current-symbol) '|.|)
      (push-reduction '|PARSE-FloatBase| 0)
      (|PARSE-FloatBasePart|))))
```

defun PARSE-FloatBasePart

```
[match-advance-string p452]
[must p463]
[PARSE-FloatBasePart digitp (vol5)]
[current-char p460]
[push-reduction p465]
[token-nonblank p??]
[current-token p458]
[PARSE-IntegerTok p436]
```

— **defun PARSE-FloatBasePart** —

```
(defun |PARSE-FloatBasePart| ()
  (and (match-advance-string ".")
    (must (or (and (digitp (current-char))
                  (push-reduction '|PARSE-FloatBasePart|
                                (token-nonblank (current-token)))
                (|PARSE-IntegerTok|))
          (and (push-reduction '|PARSE-FloatBasePart| 0)
                (push-reduction '|PARSE-FloatBasePart| 0))))))
```

defun PARSE-FloatExponent

```
[current-symbol p457]
[current-char p460]
[action p464]
[advance-token p459]
[PARSE-IntegerTok p436]
[match-advance-string p452]
[must p463]
[push-reduction p465]
[PARSE-FloatExponent identp (vol5)]
[floatexpid p462]
```

— **defun PARSE-FloatExponent** —

```
(defun |PARSE-FloatExponent| ()
  (let (g1)
    (or (and (member (current-symbol) '(e |e|))
              (find (current-char) "+-") (action (advance-token))
```

```
(must (or (|PARSE-IntegerTok|)
           (and (match-advance-string "+")
                 (must (|PARSE-IntegerTok|)))
           (and (match-advance-string "-")
                 (must (|PARSE-IntegerTok|))
                 (push-reduction '|PARSE-FloatExponent|
                                (- (pop-stack-1))))
           (push-reduction '|PARSE-FloatExponent| 0))))
(and (identp (current-symbol))
     (setq g1 (floatexpid (current-symbol)))
     (action (advance-token))
     (push-reduction '|PARSE-FloatExponent| g1))))
```

- [match-advance-string p452]
- [must p463]
- [PARSE-Expr p423]
- [push-reduction p465]
- [pop-stack-1 p548]

— defun PARSE-Enclosure —

```
(defun |PARSE-Enclosure| ()
  (or (and (match-advance-string "(")
    (must (or (and (|PARSE-Expr| 6)
      (must (match-advance-string ")"))))
    (and (match-advance-string ")")
      (push-reduction '|PARSE-Enclosure|
        (list '|@Tuple|))))))
  (and (match-advance-string "{")
    (must (or (and (|PARSE-Expr| 6)
      (must (match-advance-string "}"))
      (push-reduction '|PARSE-Enclosure|
        (cons '|brace|
          (list (list '|construct| (pop-stack-1)))))))
    (and (match-advance-string ")")
      (push-reduction '|PARSE-Enclosure|
        (list '|brace|)))))))))
```

defun PARSE-IntegerTok

[parse-number p545]

— defun PARSE-IntegerTok —

(defun |PARSE-IntegerTok| () (parse-number))

—————

defun PARSE-FormalParameter

[PARSE-FormalParameterTok p436]

— defun PARSE-FormalParameter —

(defun |PARSE-FormalParameter| () (|PARSE-FormalParameterTok|))

—————

defun PARSE-FormalParameterTok

[parse-argument-designator p546]

— defun PARSE-FormalParameterTok —

(defun |PARSE-FormalParameterTok| () (parse-argument-designator))

—————

defun PARSE-Quad

[match-advance-string p452]

[push-reduction p465]

[PARSE-GlyphTok p441]

[\$boot p??]

— defun PARSE-Quad —

```
(defun |PARSE-Quad| ()
  (or (and (match-advance-string "$")
            (push-reduction '|PARSE-Quad| '$))
```



```
(and $boot (|PARSE-GlyphTok| '|.|)
  (push-reduction '|PARSE-Quad| '|.|))))
```

defun PARSE-String

[parse-spadstring p544]

— defun PARSE-String —

```
(defun |PARSE-String| () (parse-spadstring))
```

defun PARSE-VarForm

[PARSE-Name p438]

[optional p464]

[PARSE-Scripts p437]

[push-reduction p465]

[pop-stack-2 p548]

[pop-stack-1 p548]

— defun PARSE-VarForm —

```
(defun |PARSE-VarForm| ()
  (and (|PARSE-Name|)
    (optional
      (and (|PARSE-Scripts|)
        (push-reduction '|PARSE-VarForm|
          (list '|Scripts| (pop-stack-2) (pop-stack-1))))))
    (push-reduction '|PARSE-VarForm| (pop-stack-1))))
```

defun PARSE-Scripts

[match-advance-string p452]

[must p463]

[PARSE-ScriptItem p438]

— defun PARSE-Scripts —

```
(defun |PARSE-Scripts| ()
  (and nonblank (match-advance-string "[") (must (|PARSE-ScriptItem|))
    (must (match-advance-string "]")))))
```

defun PARSE-ScriptItem

```
[PARSE-Expr p423]
[optional p464]
[star p464]
[match-advance-string p452]
[must p463]
[PARSE-ScriptItem p438]
[push-reduction p465]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-ScriptItem —

```
(defun |PARSE-ScriptItem| ()
  (or (and (|PARSE-Expr| 90)
    (optional
      (and (star repeater
        (and (match-advance-string ";")
          (must (|PARSE-ScriptItem|))))
        (push-reduction '|PARSE-ScriptItem|
          (cons '|;|
            (cons (pop-stack-2)
              (append (pop-stack-1) nil)))))))
      (and (match-advance-string ";") (must (|PARSE-ScriptItem|))
        (push-reduction '|PARSE-ScriptItem|
          (list '|PrefixSC| (pop-stack-1)))))))
```

defun PARSE-Name

```
[parse-identifier p545]
[push-reduction p465]
[pop-stack-1 p548]
```

— defun PARSE-Name —

```
(defun |PARSE-Name| ()
  (and (parse-identifier) (push-reduction '|PARSE-Name| (pop-stack-1))))
```

defun PARSE-Data

```
[action p464]
[PARSE-Sexpr p439]
[push-reduction p465]
[translabel p541]
[pop-stack-1 p548]
[labasoc p??]
```

— defun PARSE-Data —

```
(defun |PARSE-Data| ()
  (declare (special lablasoc))
  (and (action (setq lablasoc nil)) (|PARSE-Sexpr|)
    (push-reduction '|PARSE-Data|
      (list 'quote (translabel (pop-stack-1) lablasoc)))))
```

defun PARSE-Sexpr

```
[PARSE-Sexpr1 p439]
```

— defun PARSE-Sexpr —

```
(defun |PARSE-Sexpr| ()
  (and (action (advance-token)) (|PARSE-Sexpr1|)))
```

defun PARSE-Sexpr1

```
[PARSE-AnyId p441]
[optional p464]
[PARSE-NBGlyphTok p441]
[must p463]
[PARSE-Sexpr1 p439]
```

[\[action p464\]](#)
[\[pop-stack-2 p548\]](#)
[\[nth-stack p549\]](#)
[\[match-advance-string p452\]](#)
[\[push-reduction p465\]](#)
[\[PARSE-IntegerTok p436\]](#)
[\[pop-stack-1 p548\]](#)
[\[PARSE-String p437\]](#)
[\[bang p??\]](#)
[\[star p464\]](#)
[\[PARSE-GlyphTok p441\]](#)

— defun PARSE-Sexpr1 —

```

(defun |PARSE-Sexpr1| ()
  (or (and (|PARSE-AnyId|)
    (optional
      (and (|PARSE-NBGlyphTok| '=) (must (|PARSE-Sexpr1|))
        (action (setq lablasoc
          (cons (cons (pop-stack-2)
            (nth-stack 1))
            lablasoc))))))
    (and (match-advance-string "'") (must (|PARSE-Sexpr1|))
      (push-reduction '|PARSE-Sexpr1|
        (list 'quote (pop-stack-1))))
    (|PARSE-IntegerTok|)
    (and (match-advance-string "-") (must (|PARSE-IntegerTok|))
      (push-reduction '|PARSE-Sexpr1| (- (pop-stack-1))))
    (|PARSE-String|)
    (and (match-advance-string "<")
      (bang fil_test (optional (star repeater (|PARSE-Sexpr1|))))
      (must (match-advance-string ">"))
      (push-reduction '|PARSE-Sexpr1| (list2vec (pop-stack-1))))
    (and (match-advance-string "(")
      (bang fil_test
        (optional
          (and (star repeater (|PARSE-Sexpr1|))
            (optional
              (and (|PARSE-GlyphTok| '|.|)
                (must (|PARSE-Sexpr1|))
                (push-reduction '|PARSE-Sexpr1|
                  (nconc (pop-stack-2) (pop-stack-1))))))))
      (must (match-advance-string ")")))))

```

defun PARSE-NBGlyphTok

```
[match-current-token p456]
[action p464]
[advance-token p459]
[tok p414]
```

— defun PARSE-NBGlyphTok —

```
(defun |PARSE-NBGlyphTok| (|tok|)
  (declare (special |tok|))
  (and (match-current-token 'glyph |tok|) nonblank (action (advance-token))))
```

—————

defun PARSE-GlyphTok

```
[match-current-token p456]
[action p464]
[advance-token p459]
[tok p414]
```

— defun PARSE-GlyphTok —

```
(defun |PARSE-GlyphTok| (|tok|)
  (declare (special |tok|))
  (and (match-current-token 'glyph |tok|) (action (advance-token))))
```

—————

defun PARSE-AnyId

```
[parse-identifier p545]
[match-string p451]
[push-reduction p465]
[current-symbol p457]
[action p464]
[advance-token p459]
[parse-keyword p546]
```

— defun PARSE-AnyId —

```
(defun |PARSE-AnyId| ()
  (or (parse-identifier)
```

```
(or (and (match-string "$")
  (push-reduction '|PARSE-AnyId| (current-symbol))
  (action (advance-token)))
  (parse-keyword))))
```

defun PARSE-Sequence

[PARSE-OpenBracket p443]
 [must p463]
 [PARSE-Sequence1 p442]
 [match-advance-string p452]
 [PARSE-OpenBrace p443]
 [push-reduction p465]
 [pop-stack-1 p548]

— defun PARSE-Sequence —

```
(defun |PARSE-Sequence| ()
  (or (and (|PARSE-OpenBracket|) (must (|PARSE-Sequence1|))
    (must (match-advance-string "]")))
    (and (|PARSE-OpenBrace|) (must (|PARSE-Sequence1|))
    (must (match-advance-string "}")))
    (push-reduction '|PARSE-Sequence|
      (list '|brace| (pop-stack-1))))))
```

defun PARSE-Sequence1

[PARSE-Expression p422]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]
 [optional p464]
 [PARSE-IteratorTail p444]

— defun PARSE-Sequence1 —

```
(defun |PARSE-Sequence1| ()
  (and (or (and (|PARSE-Expression|)
    (push-reduction '|PARSE-Sequence1|
      (list (pop-stack-2) (pop-stack-1))))
```

```

(push-reduction '|PARSE-Sequence1| (list (pop-stack-1))))
(optional
  (and (|PARSE-IteratorTail|)
    (push-reduction '|PARSE-Sequence1|
      (cons 'collect
        (append (pop-stack-1)
          (list (pop-stack-1))))))))))

```

defun PARSE-OpenBracket

```

[getToken p455]
[current-symbol p457]
[eqcar p??]
[push-reduction p465]
[action p464]
[advance-token p459]

```

— defun PARSE-OpenBracket —

```

(defun |PARSE-OpenBracket| ()
  (let (g1)
    (and (eq (|getToken| (setq g1 (current-symbol))) '[])
      (must (or (and (eqcar g1 '|elt|)
        (push-reduction '|PARSE-OpenBracket|
          (list '|elt| (second g1) '|construct|)))
        (push-reduction '|PARSE-OpenBracket| '|construct|)))
      (action (advance-token)))))

```

defun PARSE-OpenBrace

```

[getToken p455]
[current-symbol p457]
[eqcar p??]
[push-reduction p465]
[action p464]
[advance-token p459]

```

— defun PARSE-OpenBrace —

```

(defun |PARSE-OpenBrace| ()

```

```

(let (g1)
  (and (eq (|getToken| (setq g1 (current-symbol))) '{)
    (must (or (and (eqcar g1 '|elt|)
                  (push-reduction '|PARSE-OpenBrace|
                                (list '|elt| (second g1) '|brace|)))
              (push-reduction '|PARSE-OpenBrace| '|construct|)))
    (action (advance-token)))))

```

defun PARSE-IteratorTail

```

[match-advance-string p452]
[bang p??]
[optional p464]
[star p464]
[PARSE-Iterator p444]

```

— defun PARSE-IteratorTail —

```

(defun |PARSE-IteratorTail| ()
  (or (and (match-advance-string "repeat")
           (bang fil_test (optional (star repeater (|PARSE-Iterator|)))))
      (star repeater (|PARSE-Iterator|))))

```

defun PARSE-Iterator

```

[match-advance-string p452]
[must p463]
[PARSE-Primary p431]
[PARSE-Expression p422]
[PARSE-Expr p423]
[pop-stack-3 p549]
[pop-stack-2 p548]
[pop-stack-1 p548]
[optional p464]

```

— defun PARSE-Iterator —

```

(defun |PARSE-Iterator| ()
  (or (and (match-advance-string "for") (must (|PARSE-Primary|))
          (must (match-advance-string "in")))

```



```

      (must (|PARSE-Expression|))
      (must (or (and (match-advance-string "by")
                     (must (|PARSE-Expr| 200))
                     (push-reduction '|PARSE-Iterator|
                                     (list 'inby (pop-stack-3)
                                           (pop-stack-2) (pop-stack-1))))
                (push-reduction '|PARSE-Iterator|
                                (list 'in (pop-stack-2) (pop-stack-1)))))
      (optional
        (and (match-advance-string "|")
              (must (|PARSE-Expr| 111))
              (push-reduction '|PARSE-Iterator|
                              (list '|\\| (pop-stack-1)))))
      (and (match-advance-string "while") (must (|PARSE-Expr| 190))
            (push-reduction '|PARSE-Iterator|
                            (list 'while (pop-stack-1))))
      (and (match-advance-string "until") (must (|PARSE-Expr| 190))
            (push-reduction '|PARSE-Iterator|
                            (list 'until (pop-stack-1)))))

```

The PARSE implicit routines

These symbols are not explicitly referenced in the source. Nevertheless, they are called during runtime. For example, PARSE-SemiColon is called in the chain:

```

PARSE-Enclosure {loc0=nil,loc1="(V ==> Vector; " } [ihs=35]
  PARSE-Expr
    PARSE-LedPart
      PARSE-Operation
        PARSE-getSemanticForm
          PARSE-SemiColon

```

so there is a bit of indirection involved in the call.

defun PARSE-Suffix

```

[push-reduction p465]
[current-symbol p457]
[action p464]
[advance-token p459]
[optional p464]
[PARSE-TokTail p427]
[pop-stack-1 p548]

```

— defun PARSE-Suffix —

```
(defun |PARSE-Suffix| ()
  (and (push-reduction '|PARSE-Suffix| (current-symbol))
        (action (advance-token)) (optional (|PARSE-TokTail|))
        (push-reduction '|PARSE-Suffix|
          (list (pop-stack-1) (pop-stack-1))))))
```

—————

defun PARSE-SemiColon

```
[match-advance-string p452]
[must p463]
[PARSE-Expr p423]
[push-reduction p465]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-SemiColon —

```
(defun |PARSE-SemiColon| ()
  (and (match-advance-string ";")
        (must (or (|PARSE-Expr| 82)
          (push-reduction '|PARSE-SemiColon| '|/throwAway|)))
        (push-reduction '|PARSE-SemiColon|
          (list '|;| (pop-stack-2) (pop-stack-1))))))
```

—————

defun PARSE-Return

```
[match-advance-string p452]
[must p463]
[PARSE-Expression p422]
[push-reduction p465]
[pop-stack-1 p548]
```

— defun PARSE-Return —

```
(defun |PARSE-Return| ()
  (and (match-advance-string "return") (must (|PARSE-Expression|))
        (push-reduction '|PARSE-Return|
```

```
(list 'return (pop-stack-1))))
```


defun PARSE-Exit

[match-advance-string p452]
[must p463]
[PARSE-Expression p422]
[push-reduction p465]
[pop-stack-1 p548]

— defun PARSE-Exit —

```
(defun |PARSE-Exit| ()
  (and (match-advance-string "exit")
       (must (or (|PARSE-Expression|)
                  (push-reduction '|PARSE-Exit| '|$NoValue|)))
       (push-reduction '|PARSE-Exit|
                        (list '|exit| (pop-stack-1)))))
```

defun PARSE-Leave

- [match-advance-string p452]
- [PARSE-Expression p422]
- [must p463]
- [push-reduction p465]
- [PARSE-Label p430]
- [pop-stack-1 p548]

— defun PARSE-Leave —

```
(defun |PARSE-Leave| ()
  (and (match-advance-string "leave")
    (must (or (|PARSE-Expression|)
      (push-reduction ' |PARSE-Leave| ' |$NoValue|)))
    (must (or (and (match-advance-string "from")
      (must (|PARSE-Label|)
        (push-reduction ' |PARSE-Leave|
          (list ' |leaveFrom| (pop-stack-1) (pop-stack-1))))
      (push-reduction ' |PARSE-Leave|
        (list ' |leave| (pop-stack-1)))))))
```

defun PARSE-Seg

```
[PARSE-GlyphTok p441]
[bang p??]
[optional p464]
[PARSE-Expression p422]
[push-reduction p465]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-Seg —

```
(defun |PARSE-Seg| ()
  (and (|PARSE-GlyphTok| '|.|.|)
    (bang fil_test (optional (|PARSE-Expression|)))
    (push-reduction '|PARSE-Seg|
      (list 'segment (pop-stack-2) (pop-stack-1))))))
```

defun PARSE-Conditional

```
[match-advance-string p452]
[must p463]
[PARSE-Expression p422]
[bang p??]
[optional p464]
[PARSE-ElseClause p449]
[push-reduction p465]
[pop-stack-3 p549]
[pop-stack-2 p548]
[pop-stack-1 p548]
```

— defun PARSE-Conditional —

```
(defun |PARSE-Conditional| ()
  (and (match-advance-string "if") (must (|PARSE-Expression|))
    (must (match-advance-string "then")) (must (|PARSE-Expression|))
    (bang fil_test
      (optional
        (and (match-advance-string "else")
          (must (|PARSE-ElseClause|))))))
    (push-reduction '|PARSE-Conditional|
```

```
(list 'if| (pop-stack-3) (pop-stack-2) (pop-stack-1))))))
```

defun PARSE-ElseClause

[current-symbol p457]
 [PARSE-Conditional p448]
 [PARSE-Expression p422]

— defun PARSE-ElseClause —

```
(defun |PARSE-ElseClause| ()
  (or (and (eq (current-symbol) 'if|) (|PARSE-Conditional|))
      (|PARSE-Expression|)))
```

defun PARSE-Loop

[star p464]
 [PARSE-Iterator p444]
 [must p463]
 [match-advance-string p452]
 [PARSE-Expr p423]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-Loop —

```
(defun |PARSE-Loop| ()
  (or (and (star repeater (|PARSE-Iterator|))
          (must (match-advance-string "repeat"))
          (must (|PARSE-Expr| 110))
          (push-reduction '|PARSE-Loop|
            (cons 'repeat
              (append (pop-stack-2) (list (pop-stack-1))))))
      (and (match-advance-string "repeat") (must (|PARSE-Expr| 110))
          (push-reduction '|PARSE-Loop|
            (list 'repeat (pop-stack-1))))))
```

defun PARSE-LabelExpr

[PARSE-Label p430]
 [must p463]
 [PARSE-Expr p423]
 [push-reduction p465]
 [pop-stack-2 p548]
 [pop-stack-1 p548]

— defun PARSE-LabelExpr —

```
(defun |PARSE-LabelExpr| ()
  (and (|PARSE-Label|) (must (|PARSE-Expr| 120))
    (push-reduction '|PARSE-LabelExpr|
      (list 'label (pop-stack-2) (pop-stack-1))))))
```

—————

defun PARSE-FloatTok

[parse-number p545]
 [push-reduction p465]
 [pop-stack-1 p548]
 [bfp- p??]
 [\$boot p??]

— defun PARSE-FloatTok —

```
(defun |PARSE-FloatTok| ()
  (declare (special $boot))
  (and (parse-number)
    (push-reduction '|PARSE-FloatTok|
      (if $boot (pop-stack-1) (bfp- (pop-stack-1))))))
```

—————

8.2 The PARSE support routines

This section is broken up into 3 levels:

- String grabbing: Match String, Match Advance String
- Token handling: Current Token, Next Token, Advance Token

- Character handling: Current Char, Next Char, Advance Char
- Line handling: Next Line, Print Next Line
- Error Handling
- Floating Point Support
- Dollar Translation

String grabbing

String grabbing is the art of matching initial segments of the current line, and removing them from the line before the get tokenized if they match (or removing the corresponding current tokens).

defun match-string

The match-string function returns length of X if X matches initial segment of inputstream.

[unget-tokens p456]
 [skip-blanks p451]
 [line-past-end-p p633]
 [current-char p460]
 [initial-substring-p p453]
 [subseq p??]
 [\$line p631]
 [line p631]

— defun match-string —

```
(defun match-string (x)
  (unget-tokens) ; So we don't get out of synch with token stream
  (skip-blanks)
  (if (and (not (line-past-end-p current-line)) (current-char) )
      (initial-substring-p x
        (subseq (line-buffer current-line) (line-current-index current-line))))))
```

—————

defun skip-blanks

[current-char p460]
 [token-lookahead-type p452]
 [advance-char p??]

— defun skip-blanks —

```
(defun skip-blanks ()
  (loop (let ((cc (current-char)))
    (if (not cc) (return nil))
    (if (eq (token-lookahead-type cc) 'white)
      (if (not (advance-char)) (return nil))
      (return t)))))
```

— **initvars** —

```
(defvar Escape-Character #\\ "Superquoting character.")
```

defun token-lookahead-type

[Escape-Character p??]

— **defun token-lookahead-type** —

```
(defun token-lookahead-type (char)
  "Predicts the kind of token to follow, based on the given initial character."
  (declare (special Escape-Character))
  (cond
    ((not char) 'eof)
    ((or (char= char Escape-Character) (alpha-char-p char)) 'id)
    ((digitp char) 'num)
    ((char= char #\') 'string)
    ((char= char #\[) 'bstring)
    ((member char '(#\Space #\Tab #\Return) :test #'char=) 'white)
    (t 'special-char)))
```

defun match-advance-string

The match-string function returns length of X if X matches initial segment of inputstream. If it is successful, advance inputstream past X. [quote-if-string p453]

[current-token p458]

[match-string p451]

[line-current-index p??]

[line-past-end-p p633]

[line-current-char p??]
 [\$token p97]
 [\$line p631]

— defun match-advance-string —

```
(defun match-advance-string (x)
  (let ((y (if (>= (length (string x))
                  (length (string (quote-if-string (current-token)))))
            (match-string x)
            nil))) ; must match at least the current token
    (when y
      (incf (line-current-index current-line) y)
      (if (not (line-past-end-p current-line))
          (setf (line-current-char current-line)
                (elt (line-buffer current-line)
                    (line-current-index current-line)))
          (setf (line-current-char current-line) #\space))
      (setq prior-token
            (make-token :symbol (intern (string x))
                        :type 'identifier
                        :nonblank nonblank))
      t)))
```

—————

defun initial-substring-p

[string-not-greaterp p??]

— defun initial-substring-p —

```
(defun initial-substring-p (part whole)
  "Returns length of part if part matches initial segment of whole."
  (let ((x (string-not-greaterp part whole)))
    (and x (= x (length part)) x)))
```

—————

defun quote-if-string

[token-type p??]
 [strconc p??]
 [token-symbol p??]
 [underscore p455]

```
[token-nonblank p??]
[pack p??]
[escape-keywords p454]
[$boot p??]
[$spad p571]
```

— defun quote-if-string —

```
(defun quote-if-string (token)
  (declare (special $boot $spad))
  (when token ;only use token-type on non-null tokens
    (case (token-type token)
      (bstring (strconc "[" (token-symbol token) "]*"))
      (string (strconc "'" (token-symbol token) "'"))
      (spadstring (strconc "\"" (underscore (token-symbol token)) "\""))
      (number (format nil "~v,'OD" (token-nonblank token)
                        (token-symbol token)))
      (special-char (string (token-symbol token)))
      (identifier (let ((id (symbol-name (token-symbol token)))
                        (pack (package-name (symbol-package
                                             (token-symbol token)))))
                    (if (or $boot $spad)
                        (if (string= pack "BOOT")
                            (escape-keywords (underscore id) (token-symbol token))
                            (concatenate 'string
                                           (underscore pack) "'" (underscore id)))
                        id)))
                    (token-symbol token))))))
```

—————

defun escape-keywords

```
[$keywords p??]
```

— defun escape-keywords —

```
(defun escape-keywords (pname id)
  (declare (special keywords))
  (if (member id keywords)
      (concatenate 'string "_" pname)
      pname))
```

—————

defun isTokenDelimiter

NIL needed below since END_UNIT is not generated by current parser [current-symbol p457]

— defun isTokenDelimiter —

```
(defun |isTokenDelimiter| ()
  (member (current-symbol) '(\) end\_unit nil)))
```

—————

defun underscore

[vector-push p??]

— defun underscore —

```
(defun underscore (string)
  (if (every #'alpha-char-p string)
      string
      (let* ((size (length string))
              (out-string (make-array (* 2 size)
                                      :element-type 'string-char
                                      :fill-pointer 0))
              next-char)
        (dotimes (i size)
          (setq next-char (char string i))
          (unless (alpha-char-p next-char) (vector-push #\_ out-string))
          (vector-push next-char out-string))
        out-string)))
```

—————

Token Handling**defun getToken**

[eqcar p??]

— defun getToken —

```
(defun |getToken| (x)
  (if (eqcar x '|elt|) (third x) x))
```

—————

defun unget-tokens

[quote-if-string p453]
 [line-current-segment p634]
 [strconc p??]
 [line-number p??]
 [token-nonblank p??]
 [line-new-line p634]
 [line-number p??]
 [valid-tokens p98]

— defun unget-tokens —

```
(defun unget-tokens ()
  (case valid-tokens
    (0 t)
    (1 (let* ((cursym (quote-if-string current-token))
              (curline (line-current-segment current-line))
              (revised-line (strconc cursym curline (copy-seq " "))))
        (line-new-line revised-line current-line (line-number current-line))
        (setq nonblank (token-nonblank current-token))
        (setq valid-tokens 0)))
    (2 (let* ((cursym (quote-if-string current-token))
              (nextsym (quote-if-string next-token))
              (curline (line-current-segment Current-Line))
              (revised-line
               (strconc (if (token-nonblank current-token) "" " ")
                        cursym
                        (if (token-nonblank next-token) "" " ")
                        nextsym curline " ")))
        (setq nonblank (token-nonblank current-token))
        (line-new-line revised-line current-line (line-number current-line))
        (setq valid-tokens 0)))
    (t (error "How many tokens do you think you have?"))))
```

defun match-current-token

This returns the current token if it has EQ type and (optionally) equal symbol. [current-token p458]
 [match-token p457]

— defun match-current-token —

```
(defun match-current-token (type &optional (symbol nil))
```

```
(match-token (current-token) type symbol))
```

defun match-token

```
[token-type p??]  
[token-symbol p??]
```

— defun match-token —

```
(defun match-token (token type &optional (symbol nil))  
  (when (and token (eq (token-type token) type))  
    (if symbol  
        (when (equal symbol (token-symbol token)) token)  
        token)))
```

defun match-next-token

This returns the next token if it has equal type and (optionally) equal symbol. [next-token p459]
[match-token p457]

— defun match-next-token —

```
(defun match-next-token (type &optional (symbol nil))  
  (match-token (next-token) type symbol))
```

defun current-symbol

```
[make-symbol-of p458]  
[current-token p458]
```

— defun current-symbol —

```
(defun current-symbol ()  
  (make-symbol-of (current-token)))
```

defun make-symbol-of[[\\$token](#) [p97](#)]

— defun make-symbol-of —

```
(defun make-symbol-of (token)
  (let ((u (and token (token-symbol token))))
    (cond
      ((not u) nil)
      ((characterp u) (intern (string u)))
      (u))))
```

defun current-tokenThis returns the current token getting a new one if necessary. [[try-get-token](#) [p458](#)][[valid-tokens](#) [p98](#)][[current-token](#) [p458](#)]

— defun current-token —

```
(defun current-token ()
  (declare (special valid-tokens current-token))
  (if (> valid-tokens 0)
      current-token
      (try-get-token current-token)))
```

defun try-get-token[[get-token](#) [p460](#)][[valid-tokens](#) [p98](#)]

— defun try-get-token —

```
(defun try-get-token (token)
  (declare (special valid-tokens))
  (let ((tok (get-token token)))
    (when tok
      (incf valid-tokens)
      token)))
```

defun next-token

This returns the token after the current token, or NIL if there is none after. [try-get-token p458]

[current-token p458]
 [valid-tokens p98]
 [next-token p459]

— **defun next-token** —

```
(defun next-token ()
  (declare (special valid-tokens next-token))
  (current-token)
  (if (> valid-tokens 1)
      next-token
      (try-get-token next-token)))
```

defun advance-token

This makes the next token be the current token. [current-token p458]

[copy-token p??]
 [try-get-token p458]
 [valid-tokens p98]
 [current-token p458]

— **defun advance-token** —

```
(defun advance-token ()
  (current-token) ;don't know why this is needed
  (case valid-tokens
    (0 (try-get-token (current-token)))
    (1 (decf valid-tokens)
        (setq prior-token (copy-token current-token))
        (try-get-token current-token))
    (2 (setq prior-token (copy-token current-token))
        (setq current-token (copy-token next-token))
        (decf valid-tokens))))
```

defvar XTokenReader

— initvars —

```
(defvar XTokenReader 'get-meta-token "Name of tokenizing function")
```

—————

defun get-token

[XTokenReader p460]
 [XTokenReader p460]

— defun get-token —

```
(defun get-token (token)
  (funcall XTokenReader token))
```

—————

Character handling**defun current-char**

This returns the current character of the line, initially blank for an unread line. [line p631]
 [current-line p632]

— defun current-char —

```
(defun current-char ()
  (if (line-past-end-p current-line)
      #\return
      (line-current-char current-line)))
```

—————

defun next-char

This returns the character after the current character, blank if at end of line. The blank-at-end-of-line assumption is allowable because we assume that end-of-line is a token separator, which blank is equivalent to. [line-at-end-p p632]

[line-next-char p633]

[current-line p632]

— defun next-char —

```
(defun next-char ()
  (if (line-at-end-p current-line)
      #\return
      (line-next-char current-line)))
```

—————

defun char-eq

— defun char-eq —

```
(defun char-eq (x y)
  (char= (character x) (character y)))
```

—————

defun char-ne

— defun char-ne —

```
(defun char-ne (x y)
  (char/= (character x) (character y)))
```

—————

Error handling

defvar meta-error-handler

— initvars —

```
(defvar meta-error-handler 'meta-meta-error-handler)
```

—————

defun meta-syntax-error

[meta-error-handler p461]
 [meta-error-handler p461]

— defun meta-syntax-error —

```
(defun meta-syntax-error (&optional (wanted nil) (parsing nil))
  (declare (special meta-error-handler))
  (funcall meta-error-handler wanted parsing))
```

—————

Floating Point Support**defun floatexpid**

TPDHERE: The use of and in spadreduce is undefined. rewrite this to loop

[floatexpid identp (vol5)]
 [floatexpid pname (vol5)]
 [spadreduce p??]
 [collect p383]
 [step p??]
 [maxindex p??]
 [floatexpid digitp (vol5)]

— defun floatexpid —

```
(defun floatexpid (x &aux s)
  (when (and (identp x) (char= (char-upcase (elt (setq s (pname x)) 0)) #\E)
    (> (length s) 1)
    (spadreduce and 0 (collect (step i 1 1 (maxindex s))
                                (digitp (elt s i)))))
    (read-from-string s t nil :start 1)))
```

—————

Dollar Translation**defun dollarTran**

[\$InteractiveMode p??]

— defun dollarTran —

```
(defun |dollarTran| (dom rand)
  (let ((eltWord (if |$InteractiveMode| '|$elt| '|elt|)))
    (declare (special |$InteractiveMode|))
    (if (and (not (atom rand)) (cdr rand))
        (cons (list eltWord dom (car rand)) (cdr rand))
        (list eltWord dom rand))))
```

Applying metagrammatical elements of a production (e.g., Star).

- **must** means that if it is not present in the token stream, it is a syntax error.
- **optional** means that if it is present in the token stream, that is a good thing, otherwise don't worry (like [foo] in BNF notation).
- **action** is something we do as a consequence of successful parsing; it is inserted at the end of the conjunction of requirements for a successful parse, and so should return T.
- **sequence** consists of a head, which if recognized implies that the tail must follow. Following tail are actions, which are performed upon recognizing the head and tail.

defmacro Bang

If the execution of prod does not result in an increase in the size of the stack, then stack a NIL. Return the value of prod.

— **defmacro bang** —

```
(defmacro bang (lab prod)
  '(progn
    (setf (stack-updated reduce-stack) nil)
    (let* ((prodvalue ,prod) (updated (stack-updated reduce-stack)))
      (unless updated (push-reduction ',lab nil))
      prodvalue)))
```

defmacro must

[meta-syntax-error p462]

— **defmacro must** —

```
(defmacro must (dothis &optional (this-is nil) (in-rule nil))
  '(or ,dothis (meta-syntax-error ,this-is ,in-rule)))
```

defun action

— defun action —

```
(defun action (dothis) (or dothis t))
```

defun optional

— defun optional —

```
(defun optional (dothis) (or dothis t))
```

defmacro star

Succeeds if there are one or more of PROD, stacking as one unit the sub-reductions of PROD and labelling them with LAB. E.G., (Star IDs (parse-id)) with A B C will stack (3 IDs (A B C)), where (parse-id) would stack (1 ID (A)) when applied once. [stack-size p??]
 [push-reduction p465]
 [pop-stack-1 p548]

— defmacro star —

```
(defmacro star (lab prod)
  '(prog ((oldstacksize (stack-size reduce-stack)))
    (if (not ,prod) (return nil))
    loop
    (if (not ,prod)
      (let* ((newstacksize (stack-size reduce-stack))
              (number-of-new-reductions (- newstacksize oldstacksize)))
        (if (> number-of-new-reductions 0)
          (return (do ((i 0 (1+ i)) (accum nil))
                      ((= i number-of-new-reductions)
                       (push-reduction ',lab accum)
                       (return t)))
                    (push (pop-stack-1) accum)))
        (return t))))
```

```
(go loop))))
```

Stacking and retrieving reductions of rules.

defvar reduce-stack

Stack of results of reduced productions. [stack p95]

— **initvars** —

```
(defvar reduce-stack (make-stack) )
```

defmacro reduce-stack-clear

— **defmacro reduce-stack-clear** —

```
(defmacro reduce-stack-clear () '(stack-load nil reduce-stack))
```

defun push-reduction

```
[stack-push p96]
[make-reduction p??]
[reduce-stack p465]
```

— **defun push-reduction** —

```
(defun push-reduction (rule redn)
  (stack-push (make-reduction :rule rule :value redn) reduce-stack))
```

Chapter 9

Comment Recording

This is the graph of the functions used for recording comments. The syntax is a graphviz dot file. To generate this graph as a JPEG file, type:

```
tangle v9CommentRecording.dot bookvol9.pamphlet >v9cr.dot
dot -Tjpg v9cr.dot >v9cr.jpg
```

— v9CommentRecording.dot —

```
digraph pic {
    fontsize=10;
    bgcolor="#ECEA81";
    node [shape=box, color=white, style=filled];

    "postDef" [color="#FFFFFF"]
    "PARSE-Category" [color="#FFFFFF"]

    "recordAttributeDocumentation" [color="#FF6600"]
    "recordSignatureDocumentation" [color="#FF6600"]
    "recordDocumentation" [color="#2222DD"]
    "collectComBlock" [color="#22EE22"]
    "recordHeaderDocumentation" [color="#FFFF66"]
    "collectAndDeleteAssoc" [color="#FFFF66"]

    "postDef" -> "recordHeaderDocumentation"
    "PARSE-Category" -> "recordSignatureDocumentation"
    "PARSE-Category" -> "recordAttributeDocumentation"

    "recordAttributeDocumentation" -> "recordDocumentation"
    "recordSignatureDocumentation" -> "recordDocumentation"
    "recordDocumentation" -> "recordHeaderDocumentation"
    "recordDocumentation" -> "collectComBlock"
```

```
"collectComBlock" -> "collectAndDeleteAssoc"
}
```

9.1 Comment Recording Layer 0 – API

defun recordSignatureDocumentation

This function is called externally by PARSE-Category. [recordDocumentation p469]
[postTransform p369]

— defun recordSignatureDocumentation —

```
(defun |recordSignatureDocumentation| (opSig lineno)
  (|recordDocumentation| (cdr (postTransform opSig)) lineno))
```

defun recordAttributeDocumentation

This function is called externally by PARSE-Category. [opOf p??]
[pname p??]
[upper-case-p p??]
[recordDocumentation p469]
[ifcdr p??]
[postTransform p369]

— defun recordAttributeDocumentation —

```
(defun |recordAttributeDocumentation| (arg lineno)
  (let (att name)
    (setq att (cadr arg))
    (setq name (|opOf| att))
    (cond
      ((upper-case-p (elt (pname name) 0)) nil)
      (t
       (|recordDocumentation|
        (list name (cons 'attribute (ifcdr (postTransform att)))) lineno))))))
```

9.2 Comment Recording Layer 1

defun recordDocumentation

[recordHeaderDocumentation p470]
 [collectComBlock p469]
 [\$maxSignatureLineNumber p??]
 [\$docList p??]

— defun recordDocumentation —

```
(defun |recordDocumentation| (key lineno)
  (let (u)
    (declare (special |$docList| |$maxSignatureLineNumber|))
    (|recordHeaderDocumentation| lineno)
    (setq u (|collectComBlock| lineno))
    (setq |$maxSignatureLineNumber| lineno)
    (setq |$docList| (cons (cons key u) |$docList|))))
```

—————

9.3 Comment Recording Layer 2

defun collectComBlock

[collectAndDeleteAssoc p470]
 [\$comblocklist p551]

— defun collectComBlock —

```
(defun |collectComBlock| (x)
  (let (val u)
    (declare (special $comblocklist))
    (cond
      ((and (consp $comblocklist)
            (consp (qcar $comblocklist))
            (equal (qcaar $comblocklist) x))
       (setq val (qcdar $comblocklist))
       (setq u (append val (|collectAndDeleteAssoc| x)))
       (setq $comblocklist (cdr $comblocklist))
       u)
      (t (|collectAndDeleteAssoc| x)))))
```

—————

9.4 Comment Recording Layer 3

defun recordHeaderDocumentation

This function is called externally by postDef. [assocright p??]
 [\$maxSignatureLineNumber p??]
 [\$comblocklist p551]
 [\$headerDocumentation p??]
 [\$headerDocumentation p??]
 [\$comblocklist p551]

— defun recordHeaderDocumentation —

```
(defun |recordHeaderDocumentation| (lineno)
  (let (al)
    (declare (special |$headerDocumentation| |$maxSignatureLineNumber|
                     $comblocklist))
    (when (eql |$maxSignatureLineNumber| 0)
      (setq al
        (loop for p in $comblocklist
              when (or (null (car p)) (null lineno) (> lineno (car p)))
              collect p))
      (setq $comblocklist (setdifference $comblocklist al))
      (setq |$headerDocumentation| (assocright al))
      (when |$headerDocumentation| (setq |$maxSignatureLineNumber| 1))
      |$headerDocumentation|)))
```

defun collectAndDeleteAssoc

u is (.. (x . a) .. (x . b) ..) ==> (a b ..)

deleting entries from u assumes that the first element is useless [\$comblocklist p551]

— defun collectAndDeleteAssoc —

```
(defun |collectAndDeleteAssoc| (x)
  (let (r res s)
    (declare (special $comblocklist))
    (maplist
      #'(lambda (y)
        (when (setq s (cdr y))
          (do ()
            ((null (and s (consp (car s)) (equal (qcar (car s)) x))) nil)
            (setq r (qcdr (car s))))
```

```
      (setq res (append res r))
      (setq s (cdr s))
      (rplacd y s)))
$comblocklist)
res))
```

—————

Chapter 10

Category handling

defun getConstructorExports

— defun getConstructorExports —

```
(defun |getConstructorExports| (&rest arg)
  (let (options conform)
    (setq conform (car arg))
    (setq options (cdr arg))
    (|categoryParts| conform
      (getdatabase (|op0f| conform) 'constructorcategory)
      (ifcar options))))
```

—————

Chapter 11

Building libdb.text

defun extendLocalLibdb

```
[buildLibdb p476]
[union p??]
[purgeNewConstructorLines p479]
[dbReadLines p479]
[dbWriteLines p479]
[extendLocalLibdb deleteFile (vol5)]
[msort p??]
[$createLocalLibDb p??]
[$newConstructorList p??]
[$newConstructorList p??]
```

— **defun extendLocalLibdb** —

```
(defun |extendLocalLibdb| (conlist)
  (let (localLibdb oldlines newlines)
    (declare (special |$createLocalLibDb| |$newConstructorList|))
    (cond
      ((null |$createLocalLibDb|) nil)
      ((null conlist) nil)
      (t
       (|buildLibdb| conlist)
       (setq |$newConstructorList| (|union| conlist |$newConstructorList|))
       (setq localLibdb "libdb.text")
       (cond
         ((null (probe-file "libdb.text"))
          (rename-file "temp.text" "libdb.text"))
         (t
          (setq oldlines
                 (|purgeNewConstructorLines| (|dbReadLines| localLibdb) conlist))
```

```
(setq newlines (|dbReadLines| "temp.text"))
(|dbWriteLines| (msort (|union| oldlines newlines)) "libdb.text")
(|deleteFile| "temp.text")))))))
```

defun buildLibdb

This function appears to have two use cases, one in which the `domainList` variable is undefined, in which case it writes out all of the constructors, and the other case where it writes out a single constructor. Formal for `libdb.text`:

```
constructors      Cname\#\I\sig \args  \abb \comments (C is C, D, P, X)
operations        Op  \#\E\sig \conname\pred\comments (E is one of U/E)
attributes        Aname\#\E\args\conname\pred\comments
I = <x if exposed><d if category with a default package>
```

```
[dsetq p??]
[ifcar p??]
[buildLibdb deleteFile (vol5)]
[buildLibdb make-outstream (vol5)]
[writedb p??]
[buildLibdbString p478]
[buildLibdb allConstructors (vol5)]
[buildLibdbConEntry p480]
[getConstructorExports p473]
[buildLibOps p482]
[buildLibAttrs p483]
[shut p??]
[obey p??]
[deleteFile p??]
[$outStream p??]
[$conform p??]
[$kind p??]
[$doc p??]
[$exposed? p??]
[$conform p??]
[$conname p??]
[$outStream p??]
[$DefLst p??]
[$PakLst p??]
[$catLst p??]
[$DomLst p??]
[$AttrLst p??]
[$OpLst p??]
```


— defun buildLibdb —

```

(defun |buildLibdb| (&rest G168131 &AUX options)
  (dsetq options G168131)
  (let (|$OpLst| |$AttrLst| |$DomLst| |$CatLst| |$PakLst| |$DefLst|
        |$outStream| |$conname| |$conform| |$exposed?| |$doc|
        |$kind| domainList comments constructorList tmp1 attrlist oplist)
    (declare (special |$OpLst| |$AttrLst| |$DomLst| |$CatLst| |$PakLst|
                      |$DefLst| |$outStream| |$conname| |$conform|
                      |$exposed?| |$doc| |$kind|))
    (setq domainList (ifcar options))
    (setq |$OpLst| nil)
    (setq |$AttrLst| nil)
    (setq |$DomLst| nil)
    (setq |$CatLst| nil)
    (setq |$PakLst| nil)
    (setq |$DefLst| nil)
    (|deleteFile| "temp.text")
    (setq |$outStream| (make-outstream "temp.text"))
    (unless domainList
      (setq comments
        (concatenate 'string
          "\\spad{Union(A,B,...,C)} is a primitive type in AXIOM used to "
          "represent objects of type \\spad{A} or of type \\spad{B} or...or "
          "of type \\spad{C}.")
        (|writedb|
          (|buildLibdbString|
            (list "dUnion" 1 "x" "special" "(A,B,...,C)" 'UNION comments))))
      (setq comments
        (concatenate 'string
          "\\spad{Record(a:A,b:B,...,c:C)} is a primitive type in AXIOM used "
          "to represent composite objects made up of objects of type "
          "\\spad{A}, \\spad{B},..., \\spad{C} which are indexed by \"keys\" "
          "(identifiers) \\spad{a},\\spad{b},...,\\spad{c}.")
        (|writedb|
          (|buildLibdbString|
            (list "dRecord" 1 "x" "special" "(a:A,b:B,...,c:C)" 'RECORD comments))))
      (setq comments
        (concatenate 'string
          "\\spad{Mapping(T,S)} is a primitive type in AXIOM used to represent "
          "mappings from source type \\spad{S} to target type \\spad{T}. "
          "Similarly, \\spad{Mapping(T,A,B)} denotes a mapping from source "
          "type \\spad{(A,B)} to target type \\spad{T}.")
        (|writedb|
          (|buildLibdbString|
            (list "dMapping" 1 "x" "special" "(T,S)" 'MAPPING comments))))
      (setq comments
        (concatenate 'string
          "\\spad{Enumeration(a,b,...,c)} is a primitive type in AXIOM used to "

```

```

    "represent the object composed of the symbols \\spad{a},\\spad{b},"
    "..., and \\spad{c}.")
  (|writedb|
    (|buildLibdbString|
      (list "dEnumeration" 1 "x" "special" "(a,b,...,c)" 'ENUM comments))))
  (setq |$conname| nil)
  (setq |$conform| nil)
  (setq |$exposed?| nil)
  (setq |$doc| nil)
  (setq |$kind| nil)
  (setq constructorList (or domainList (|allConstructors|)))
  (loop for con in constructorList do
    (|writedb| (|buildLibdbConEntry| con))
    (setq tmp1 (|getConstructorExports| |$conform|))
    (setq attrlist (car tmp1))
    (setq oplist (cdr tmp1))
    (|buildLibOps| oplist)
    (|buildLibAttrs| attrlist))
  (shut |$outStream|)
  (unless domainList
    (obey "sort \"temp.text\" > \"libdb.text\"")
    (rename-file "libdb.text" "olibdb.text")
    (|deleteFile| "temp.text"))))

```

defun buildLibdbString

[strconc p??]

— defun buildLibdbString —

```

(defun |buildLibdbString| (arg)
  (let (x u)
    (setq x (car arg))
    (setq u (cdr arg))
    (strconc (princ-to-string x)
      (let ((result ""))
        (loop for y in u
          collect (setq result (strconc result (strconc "\"" (princ-to-string y))))
          result))))

```

defun dbReadLines

[eofp p??]
[readline p??]

— defun dbReadLines —

```
(defun |dbReadLines| (target)
  (let (instream lines)
    (setq instream (open target))
    (setq lines
      (loop while (not (eofp instream))
        collect (readline instream)))
    (close instream)
    lines))
```

defun purgeNewConstructorLines

[screenLocalLine p[484](#)]

— defun purgeNewConstructorLines —

```
(defun |purgeNewConstructorLines| (lines conlist)
  (loop for x in lines
    when (null (|screenLocalLine| x conlist))
    collect x))
```

defun dbWriteLines

[ifcar p??]
[getTempPath p??]
[make-outstream p??]
[writedb p??]
[shut p??]
[\$outStream p??]
[\$outStream p??]

— defun dbWriteLines —

```
(defun |dbWriteLines| (&rest G176369 &aux options s)
```

```

(dsetq (s . options) G176369)
(let (|$outStream| pathname)
(declare (special |$outStream|))
  (setq pathname (or (ifcar options) (|getTempPath| ' |source|)))
  (setq |$outStream| (make-outstream pathname))
  (loop for x in s do (|writedb| x))
  (shut |$outStream|)
  pathname))

```

defun buildLibdbConEntry

```

[getdatabase p??]
[dbMkForm p??]
[msubst p??]
[isExposedConstructor p??]
[pname p??]
[maxindex p??]
[downcase p??]
[lassoc p??]
[libdbTrim p??]
[concatWithBlanks p??]
[form2HtString p??]
[libConstructorSig p??]
[strconc p??]
[buildLibdbString p478]
[length p??]
[$exposed? p??]
[$kind p??]
[$conform p??]
[$kind p??]
[$doc p??]
[$exposed? p??]
[$conname p??]

```

— defun buildLibdbConEntry —

```

(defun |buildLibdbConEntry| (conname)
  (let (abb conform pname kind arg1 tmp1 conComments argpart sigpart header)
    (declare (special |$exposed?| |$doc| |$kind| |$conname| |$conform|))
    (cond
      ((null (getdatabase conname 'constructormodemap)) nil)
      (t
       (setq abb (getdatabase conname 'abbreviation))
       (setq |$conname| conname)

```

```

(setq conform (or (getdatabase conname 'constructorform) (list conname)))
(setq |$conform| (|dbMkForm| (msubst 't 'T$ conform)))
(cond
  ((null |$conform|) nil)
  (t
   (setq |$exposed?| (if (|isExposedConstructor| conname) "x" "n"))
   (setq |$doc| (getdatabase conname 'documentation))
   (setq pname (pname conname))
   (setq kind (getdatabase conname 'constructorkind))
   (cond
     ((and (eq kind '|domain|)
            (progn
              (setq tmp1 (getdatabase conname 'constructormodemap))
              (and (consp tmp1)
                    (consp (qcar tmp1))
                    (consp (qcdar tmp1))))
              (consp (qcadar tmp1)) (eq (qcaadar tmp1) 'category)
              (progn
                (and (consp (qcdadar tmp1))
                      (eq (qcar (qcdadar tmp1)) '|package|))))
             (setq kind '|package|))))
     (t
      (setq |$kind|
        (if (char= (elt pname (maxindex pname)) #\&)
            '|x|
            (downcase (elt (pname kind) 0))))
      (setq arg1 (cdr |$conform|))
      (setq conComments
        (cond
          ((progn
              (setq tmp1 (lassoc '|constructor| |$doc|))
              (and (consp tmp1)
                    (eq (qcdr tmp1) nil)
                    (consp (qcar tmp1))
                    (equal (qcaar tmp1) nil)))
              (|libdbTrim| (|concatWithBlanks| (qcdar tmp1))))
           (t "")))
      (setq argpart (substring (|form2HtString| (cons '|f| arg1)) 1 nil))
      (setq sigpart (|libConstructorSig| |$conform|))
      (setq header (strconc |$kind| (pname conname)))
      (|buildLibdbString|
        (list header (|#| arg1) |$exposed?|
              sigpart argpart abb conComments))))))

```

defun buildLibOps[buildLibOp p⁴⁸²]

— defun buildLibOps —

```
(defun |buildLibOps| (oplist)
  (loop for item in oplist
        do (|buildLibOp| (car item) (cadr item) (cddr item))))
```

—————

defun buildLibOp

```
[sublislis p??]
[msubst p??]
[form2LispString p??]
[strconc p??]
[libdbTrim p??]
[concatWithBlanks p??]
[lassoc p??]
[checkCommentsForBraces p??]
[writedb p??]
[buildLibdbString p478]
[$kind p??]
[$doc p??]
[$exposed? p??]
[$conform p??]
```

— defun buildLibOp —

```
(defun |buildLibOp| (op sig pred)
  (let (nsig sigpart predString s sop header conform comments)
    (declare (special |$kind| |$doc| |$exposed?| |$conform|))
    (setq nsig (sublislis (cdr |$conform|) |$FormalMapVariableList| sig))
    (setq pred (sublislis (cdr |$conform|) |$FormalMapVariableList| pred))
    (setq nsig (msubst 't 't$ nsig))
    (setq pred (msubst 't 't$ pred))
    (setq sigpart (|form2LispString| (cons '|Mapping| nsig)))
    (setq predString (if (eq pred t) "" (|form2LispString| pred)))
    (setq sop
      (cond
        ((string= (setq s (princ-to-string op)) "One") "1")
        ((string= s "Zero") "0")
        (t s)))
    (setq header (strconc "o" sop))
```

```
(setq conform (strconc |$kind| (|form2LispString| |$conform|)))
(setq comments
  (|libdbTrim| (|concatWithBlanks| (lassoc sig (lassoc op |$doc|)))))
(|checkCommentsForBraces| '|operation| sop sigpart comments)
(|writedb|
  (|buildLibdbString|
    (list header (|#| (cdr sig)) |$exposed?| sigpart
      conform predString comments)))))
```

defun buildLibAttrs

[buildLibAttr p483]

— defun buildLibAttrs —

```
(defun |buildLibAttrs| (attrlist)
  (let (name arg1 pred)
    (loop for item in attrlist
      do (|buildLibAttr| (car item) (cadr item) (cddr item)))))
```

defun buildLibAttr

attributes AKname\#\args\conname\pred\comments (K is U or C)

```
[form2LispString p??]
[sublislis p??]
[concatWithBlanks p??]
[lassoc p??]
[checkCommentsForBraces p??]
[writedb p??]
[buildLibdbString p478]
[length p??]
[$conform p??]
[$FormalMapVariableList p270]
[$kind p??]
[$doc p??]
[$exposed? p??]
[$conname p??]
```

— defun buildLibAttr —

```
(defun |buildLibAttr| (name argl pred)
  (let (argPart predString header conname comments)
    (declare (special |$kind| |$conname| |$doc| |$conform|
                     |$FormalMapVariableList| |$exposed?|)))
    (setq header (strconc "a" (princ-to-string name)))
    (setq argPart (substring (|form2LispString| (cons '|f| argl)) 1 nil))
    (setq pred (sublislis (cdr |$conform|) |$FormalMapVariableList| pred))
    (setq predString (if (eq pred t) "" (|form2LispString| pred)))
    (setq header (strconc "a" (princ-to-string name)))
    (setq conname (strconc |$kind| (|form2LispString| |$conname|)))
    (setq comments
      (|concatWithBlanks| (lassoc (cons '|attribute| argl) (lassoc name |$doc|))))
    (|checkCommentsForBraces| '|attribute| (princ-to-string name) argl comments)
    (|writedb|
      (|buildLibdbString|
        (list header (|#| argl) |$exposed?| argPart
              conname predString comments))))))
```

defun screenLocalLine

```
[dbPart p??]
[charPosition p??]
[dbName p??]
[dbKind p??]
```

— defun screenLocalLine —

```
(defun |screenLocalLine| (line conlist)
  (let (s k con)
    (setq k (|dbKind| line))
    (setq con
      (intern (cond ((or (char= k #\o) (char= k #\a))
                     (setq s (|dbPart| line 5 1))
                     (setq k (|charPosition| #\ ( s 1))
                     (substring s 1 (1- k)))
                     (t (|dbName| line))))))
    (member con conlist)))
```

Chapter 12

Comment Syntax Checking

This is the graph of the functions used for comment syntax checking. The syntax is a graphviz dot file. To generate this graph as a JPEG file, type:

```
tangle v9CommentSyntaxChecking.dot bookvol9.pamphlet >v9csc.dot
dot -Tjpg v9csc.dot >v9csc.jpg
```

— v9CommentSyntaxChecking.dot —

```
digraph hierarchy {
    fontsize=10;
    bgcolor="#ECEA81";
    node [shape=box, color=white, style=filled];

    "compileDocumentation" [color="#FFFFFF"]
    "finalizeLisplib"      [color="#FFFFFF"]

    {rank=same; "compileDocumentation" "finalizeLisplib"}

    "checkAddBackSlashes" [color="#FFFF66"]
    "checkAddMacros"      [color="#FFFF66"]
    "checkAddPeriod"      [color="#FFFF66"]
    "checkAddSpaceSegments" [color="#FFFF66"]
    "checkAddSpaces"      [color="#FFFF66"]
    "checkAlphabetic"      [color="#FFFF66"]
    "checkIeEgfun"         [color="#FFFF66"]
    "checkIsValidType"     [color="#FFFF66"]
    "checkLookForLeftBrace" [color="#FFFF66"]
    "checkLookForRightBrace" [color="#FFFF66"]
    "checkNumOfArgs"       [color="#FFFF66"]
    "checkSayBracket"      [color="#FFFF66"]
    "checkSkipBlanks"      [color="#FFFF66"]
}
```

```

"checkSplitBackslash" [color="#FFFF66"]
"checkSplitOn" [color="#FFFF66"]
"checkSplitPunctuation" [color="#FFFF66"]
"firstNonBlankPosition" [color="#FFFF66"]
"getMatchingRightPren" [color="#FFFF66"]
"hasNoVowels" [color="#FFFF66"]
"htcharPosition" [color="#FFFF66"]
"newWordFrom" [color="#FFFF66"]
"removeBackslashes" [color="#FFFF66"]
"whoOwns" [color="#FFFF66"]

{rank=same;
  "checkAddBackSlashes"
  "checkAddMacros"
  "checkAddPeriod"
  "checkAddSpaceSegments"
  "checkAddSpaces"
  "checkAlphabetic"
  "checkIeEgfun"
  "checkIsValidType"
  "checkLookForLeftBrace"
  "checkLookForRightBrace"
  "checkNumOfArgs"
  "checkSayBracket"
  "checkSkipBlanks"
  "checkSplitBackslash"
  "checkSplitOn"
  "checkSplitPunctuation"
  "firstNonBlankPosition"
  "getMatchingRightPren"
  "hasNoVowels"
  "htcharPosition"
  "newWordFrom"
  "removeBackslashes"
  "whoOwns"
}

"checkAddIndented" [color="#22EE22"]
"checkDocMessage" [color="#22EE22"]
"checkExtract" [color="#22EE22"]
"checkGetArgs" [color="#22EE22"]
"checkGetMargin" [color="#22EE22"]
"checkGetParse" [color="#22EE22"]
"checkGetStringBeforeRightBrace" [color="#22EE22"]
"checkIeEg" [color="#22EE22"]
"checkIndentedLines" [color="#22EE22"]
"checkSkipIdentifierToken" [color="#22EE22"]
"checkSkipOpToken" [color="#22EE22"]
"checkSplitBrace" [color="#22EE22"]
"checkTrimCommented" [color="#22EE22"]

```

```

"newString2Words" [color="#22EE22"]

{rank=same;
  "checkAddIndented"
  "checkDocMessage"
  "checkExtract"
  "checkGetArgs"
  "checkGetMargin"
  "checkGetParse"
  "checkGetStringBeforeRightBrace"
  "checkIeEg"
  "checkIndentedLines"
  "checkSkipIdentifierToken"
  "checkSkipOpToken"
  "checkSplitBrace"
  "checkTrimCommented"
  "newString2Words"
}

"checkDocError" [color="#2222DD"]
"checkRemoveComments" [color="#2222DD"]
"checkSkipToken" [color="#2222DD"]
"checkSplit2Words" [color="#2222DD"]

{rank=same;
  "checkDocError"
  "checkRemoveComments"
  "checkSkipToken"
  "checkSplit2Words"
}

"checkBeginEnd" [color="#FF6600"]
"checkDecorate" [color="#FF6600"]
"checkDecorateForHt" [color="#FF6600"]
"checkDocError1" [color="#FF6600"]
"checkFixCommonProblem" [color="#FF6600"]
"checkGetLispFunctionName" [color="#FF6600"]
"checkHTargs" [color="#FF6600"]
"checkRecordHash" [color="#FF6600"]
"checkTexht" [color="#FF6600"]
"checkTransformFirsts" [color="#FF6600"]
"checkTrim" [color="#FF6600"]

{rank=same;
  "checkBeginEnd"
  "checkDecorate"
  "checkDecorateForHt"
  "checkDocError1"
  "checkFixCommonProblem"
  "checkGetLispFunctionName"

```

```

    "checkHTargs"
    "checkRecordHash"
    "checkTexht"
    "checkTransformFirsts"
    "checkTrim"
}

"checkArguments" [color="#0066FF"]
"checkBalance" [color="#0066FF"]

{rank=same;
  "checkArguments"
  "checkBalance"
}

"checkComments" [color="#006600"]
"checkRewrite" [color="#006600"]

{rank=same;
  "checkComments"
  "checkRewrite"
}

"transformAndRecheckComments" [color="#448822"]

"transDoc" [color="#448822"]

"transDocList" [color="#448822"]

"finalizeDocumentation" [color="#448822"]

"checkAddIndented" -> "firstNonBlankPosition"
"checkAddIndented" -> "checkAddSpaceSegments"
"checkArguments" -> "checkHTargs"
"checkBalance" -> "checkBeginEnd"
"checkBalance" -> "checkDocError"
"checkBalance" -> "checkSayBracket"
"checkBeginEnd" -> "checkDocError"
"checkComments" -> "checkGetMargin"
"checkComments" -> "checkTransformFirsts"
"checkComments" -> "checkIndentedLines"
"checkComments" -> "checkGetArgs"
"checkComments" -> "newString2Words"
"checkComments" -> "checkAddSpaces"
"checkComments" -> "checkIeEg"
"checkComments" -> "checkSplit2Words"
"checkComments" -> "checkBalance"
"checkComments" -> "checkArguments"
"checkComments" -> "checkFixCommonProblem"
"checkComments" -> "checkDecorate"

```

```

"checkComments" -> "checkAddPeriod"
"checkDecorate" -> "checkDocError"
"checkDecorate" -> "checkAddBackSlashes"
"checkDecorate" -> "hasNoVowels"
"checkDecorateForHt" -> "checkDocError"
"checkDocError" -> "checkDocMessage"
"checkDocError1" -> "checkDocError"
"checkDocMessage" -> "whoOwns"
"checkExtract" -> "firstNonBlankPosition"
"checkFixCommonProblem" -> "checkDocError"
"checkGetArgs" -> "firstNonBlankPosition"
"checkGetArgs" -> "getMatchingRightPren"
"checkGetLispFunctionName" -> "checkDocError"
"checkGetMargin" -> "firstNonBlankPosition"
"checkGetParse" -> "removeBackslashes"
"checkHTargs" -> "checkLookForLeftBrace"
"checkHTargs" -> "checkLookForRightBrace"
"checkHTargs" -> "checkDocError"
"checkIeEg" -> "checkIeEgfun"
"checkIndentedLines" -> "firstNonBlankPosition"
"checkIndentedLines" -> "checkAddSpaceSegments"
"checkRecordHash" -> "checkLookForLeftBrace"
"checkRecordHash" -> "checkLookForRightBrace"
"checkRecordHash" -> "checkGetLispFunctionName"
"checkRecordHash" -> "checkGetStringBeforeRightBrace"
"checkRecordHash" -> "checkGetParse"
"checkRecordHash" -> "checkDocError"
"checkRecordHash" -> "checkNumOfArgs"
"checkRecordHash" -> "checkIsValidType"
"checkRemoveComments" -> "checkTrimCommented"
"checkRewrite" -> "checkRemoveComments"
"checkRewrite" -> "checkAddIndented"
"checkRewrite" -> "checkGetArgs"
"checkRewrite" -> "newString2Words"
"checkRewrite" -> "checkAddSpaces"
"checkRewrite" -> "checkSplit2Words"
"checkRewrite" -> "checkAddMacros"
"checkRewrite" -> "checkTexht"
"checkRewrite" -> "checkArguments"
"checkRewrite" -> "checkFixCommonProblem"
"checkRewrite" -> "checkRecordHash"
"checkRewrite" -> "checkDecorateForHt"
"checkSkipIdentifierToken" -> "checkAlphabetic"
"checkSkipOpToken" -> "checkAlphabetic"
"checkSkipToken" -> "checkSkipIdentifierToken"
"checkSkipToken" -> "checkSkipOpToken"
"checkSplit2Words" -> "checkSplitBrace"
"checkSplitBrace" -> "checkSplitBackslash"
"checkSplitBrace" -> "checkSplitOn"
"checkSplitBrace" -> "checkSplitPunctuation"

```

```

"checkTexht" -> "checkDocError"
"checkTransformFirst" -> "checkSkipToken"
"checkTransformFirst" -> "checkSkipBlanks"
"checkTransformFirst" -> "getMatchingRightPren"
"checkTransformFirst" -> "checkDocError"
"checkTrim" -> "checkDocError"
"checkTrimCommented" -> "htcharPosition"
"finalizeDocumentation" -> "transDocList"
"newString2Words" -> "newWordFrom"
"transDoc" -> "checkDocError1"
"transDoc" -> "checkTrim"
"transDoc" -> "checkExtract"
"transDoc" -> "transformAndRecheckComments"
"transDocList" -> "transDoc"
"transDocList" -> "checkDocError"
"transDocList" -> "checkDocError1"
"transformAndRecheckComments" -> "checkComments"
"transformAndRecheckComments" -> "checkRewrite"

"compileDocumentation" -> "finalizeDocumentation"
"finalizeLisplib" -> "finalizeDocumentation"
}

```

12.1 Comment Checking Layer 0 – API

defun finalizeDocumentation

```

[bright p??]
[sayMSG p??]
[strconc p??]
[sayKeyedMsg p??]
[form2String p??]
[formatOpSignature p??]
[transDocList p493]
[assocleft p??]
[remdup p??]
[macroExpand p178]
[sublislis p??]
[$e p??]
[$lisplibForm p??]
[$docList p??]
[$op p??]
[$comblocklist p551]
[$FormalMapVariableList p270]

```

— defun finalizeDocumentation —

```
(defun |finalizeDocumentation| ()
  (labels (
    (fn (x env)
      (declare (special |$lisplibForm| |$FormalMapVariableList|))
      (cond
        ((atom x) (list x nil))
        (t
         (when (> (|#| x) 2) (setq x (take 2 x)))
         (sublislis |$FormalMapVariableList| (cdr |$lisplibForm|)
                    (|macroExpand| x env))))))
    (hn (u)
      ; ((op,sig,doc), ...) --> ((op ((sig doc) ...)) ...)
      (let (opList op1 sig doc)
        (setq oplist (remdup (assocleft u)))
        (loop for op in opList
          collect
            (cons op
              (loop for item in u
                do (setq op1 (first item))
                  (setq sig (second item))
                  (setq doc (third item))
                  when (equal op op1)
                    collect
                      (list sig doc))))))
        (let (unusedCommentLineNumbers docList u noHeading attributes
              signatures name bigcnt op s litcnt a n r sig)
          (declare (special |$e| |$lisplibForm| |$docList| |$op| $comblocklist))
          (setq unusedCommentLineNumbers
            (loop for x in $comblocklist
              when (cdr x)
                collect x))
          (setq docList (subst '$ '% (|transDocList| |$op| |$docList|) :test #'equal))
          (cond
            ((setq u
              (loop for item in docList
                when (null (cdr item))
                  collect (car item)))
              (loop for y in u
                do
                  (cond
                    ((eq y '|constructor|) (setq noHeading t))
                    ((and (consp y) (consp (qcdr y)) (eq (qcddr y) nil)
                      (consp (qcadr y)) (eq (qcaadr y) '|attribute|))
                     (setq attributes (cons (cons (qcar y) (qcadr y)) attributes)))
                    (t (setq signatures (cons y signatures))))
                  (setq name (CAR |$lisplibForm|))
                  (when (or noHeading signatures attributes unusedCommentLineNumbers)
```

```

(|sayKeyedMsg| "Constructor documentation warnings (++ comments):" nil)
(setq bigcnt 1)
(when (or noHeading signatures attributes)
  (|sayKeyedMsg|
    " %1 The constructor %2 has missing documentation."
    (list (strconc (princ-to-string bigcnt) ".") name))
  (setq bigcnt (1+ bigcnt))
  (setq litcnt 1)
  (when noHeading
    (|sayKeyedMsg|
      " %x3 %1 The constructor %2 is missing the heading description."
      (list (strconc "(" (princ-to-string litcnt) ")") name))
      (setq litcnt (1+ litcnt)))
  (when signatures
    (|sayKeyedMsg|
      " %x3 %1 The following functions do not have documentation:"
      (list (strconc "(" (princ-to-string litcnt) ")"))
      (setq litcnt (1+ litcnt))
      (loop for item in signatures
        do
          (setq op (first item))
          (setq sig (second item))
          (setq s (|formatOpSignature| op sig))
          (|sayMSG|
            (if (atom s)
              (list '|%x9| s)
              (cons '|%x9| s))))))
  (when attributes
    (|sayKeyedMsg|
      " %x3 %1 The following attributes do not have documentation:"
      (list (strconc "(" (princ-to-string litcnt) ")"))
      (setq litcnt (1+ litcnt))
      (DO ((G166491 attributes
              (CDR G166491))
          (x NIL))
        ((OR (ATOM G166491)
              (PROGN
                (SETQ x (CAR G166491))
                NIL))
          NIL)
        (SEQ (EXIT
              (PROGN
                (setq a (|form2String| x))
                (|sayMSG|
                  (COND
                    ((ATOM a)
                     (CONS '|%x9| (CONS a NIL)))
                    ('T (CONS '|%x9| a))))))))))
  (when unusedCommentLineNumbers
    (|sayKeyedMsg|

```



```

      " %1 The constructor %2 has incorrectly placed documentation."
      (list (strconc (princ-to-string bigcnt) ".") name))
    (loop for item in unusedCommentLineNumbers
      do
        (setq r (second item))
        (|sayMSG| (cons " " (append (|bright| n) (list " " r))))))
    (hn
      (loop for item in docList
        collect (append (fn (car item) |$e|) (cdr item))))))

```

12.2 Comment Checking Layer 1

defun transDocList

```

[|sayBrightly| p??]
[transDoc p494]
[checkDocError p515]
[checkDocError1 p505]
[$constructorName p??]

```

— defun transDocList —

```

(defun |transDocList| (|$constructorName| doclist)
  (declare (special |$constructorName|))
  (let (commentList conEntry acc)
    (|sayBrightly|
      (list " Processing " |$constructorName| " for Browser database:"))
    (setq commentList (|transDoc| |$constructorName| doclist))
    (setq acc nil)
    (loop for entry in commentList
      do
        (cond
          ((and (consp entry) (eq (qcar entry) '|constructor|)
              (consp (qcdr entry)) (eq (qcddr entry) nil))
            (if conEntry
              (|checkDocError| (list "Spurious comments: " (qcadr entry)))
              (setq conEntry entry)))
            (t (setq acc (cons entry acc)))))
    (if conEntry
      (cons conEntry acc)
      (progn
        (|checkDocError1| (list "Missing Description"))
        acc))))

```

12.3 Comment Checking Layer 2

defun transDoc

```
[checkDocError1 p505]
[checkTrim p515]
[checkExtract p519]
[transformAndRecheckComments p495]
[nreverse p??]
[$x p??]
[$attribute? p??]
[$x p??]
[$attribute? p??]
[$argl p??]
```

— defun transDoc —

```
(defun |transDoc| (conname doclist)
  (declare (ignore conname))
  (let (|$x| |$attribute?| |$argl| rlist lines u v longline acc)
    (declare (special |$x| |$attribute?| |$argl|))
    (setq |$x| nil)
    (setq rlist (reverse doclist))
    (loop for item in rlist
      do
        (setq |$x| (car item))
        (setq lines (cdr item))
        (setq |$attribute?|
          (and (consp |$x|) (consp (qcdr |$x|)) (eq (qcddr |$x|) nil)
              (consp (qcadr |$x|)) (eq (qcdadr |$x|) nil)
              (eq (qcaadr |$x|) '|attribute|)))
        (cond
          ((null lines)
            (unless |$attribute?| (|checkDocError1| (list "Not documented!!!!"))))
          (t
            (setq u
              (|checkTrim| |$x|
                (cond
                  ((stringp lines) (list lines))
                  ((eq |$x| '|constructor|) (car lines))
                  (t lines))))
              (setq |$argl| nil) ;; possibly unused -- tpd
              (setq longline
                (cond
                  ((eq |$x| '|constructor|)
```

```

      (setq v
        (or
          (|checkExtract| "Description:" u)
          (and u (|checkExtract| "Description:"
            (cons (strconc "Description: " (car u)) (cdr u))))))
      (|transformAndRecheckComments| '|constructor| (or v u)))
    (t (|transformAndRecheckComments| |$x| u)))
  (setq acc (cons (list |$x| longline) acc))))
(nreverse acc)))

```

12.4 Comment Checking Layer 3

defun transformAndRecheckComments

```

[sayBrightly p??]
[checkComments p496]
[checkRewrite p497]
[$exposeFlagHeading p??]
[$checkingXmptex? p??]
[$x p??]
[$name p??]
[$origin p??]
[$recheckingFlag p??]
[$exposeFlagHeading p??]

```

— defun transformAndRecheckComments —

```

(defun |transformAndRecheckComments| (name lines)
  (let (|$x| |$name| |$origin| |$recheckingFlag| |$exposeFlagHeading| u)
    (declare (special |$x| |$name| |$origin| |$recheckingFlag|
      |$exposeFlagHeading| |$exposeFlag| |$checkingXmptex?|))
    (setq |$checkingXmptex?| nil)
    (setq |$x| name)
    (setq |$name| '|GlossaryPage|)
    (setq |$origin| '|gloss|)
    (setq |$recheckingFlag| nil)
    (setq |$exposeFlagHeading| (list "-----" name "-----"))
    (unless |$exposeFlag| (|sayBrightly| |$exposeFlagHeading|))
    (setq u (|checkComments| name lines))
    (setq |$recheckingFlag| t)
    (|checkRewrite| name (list u))
    (setq |$recheckingFlag| nil)
    u))

```

12.5 Comment Checking Layer 4

defun checkComments

```
[checkGetMargin p520]
[checkTransformFirsts p512]
[checkIndentedLines p522]
[checkGetArgs p520]
[newString2Words p525]
[checkAddSpaces p528]
[checkIeEg p522]
[checkSplit2Words p517]
[checkBalance p499]
[checkArguments p499]
[checkFixCommonProblems p??]
[checkDecorate p502]
[strconc p??]
[checkAddPeriod p527]
[pp p??]
[$attribute? p??]
[$checkErrorFlag p??]
[$argl p??]
[$checkErrorFlag p??]
```

— defun checkComments —

```
(defun |checkComments| (nameSig lines)
  (let (|$checkErrorFlag| margin w verbatim u2 okBefore u v res)
    (declare (special |$checkErrorFlag| |$argl| |$attribute?|))
    (setq |$checkErrorFlag| nil)
    (setq margin (|checkGetMargin| lines))
    (cond
      ((and (or (null (boundp '|$attribute?|)) (null |$attribute?|))
            (not (eq nameSig '|constructor|))))
        (setq lines
          (cons
            (|checkTransformFirsts| (car nameSig) (car lines) margin)
            (cdr lines))))))
    (setq u (|checkIndentedLines| lines margin))
    (setq |$argl| (|checkGetArgs| (car u)))
    (setq u2 nil)
    (setq verbatim nil)
    (loop for x in u
      do (setq w (|newString2Words| x))
```

```

(cond
  (verbatim
    (cond
      ((and w (equal (car w) "\\end{verbatim}"))
        (setq verbatim nil)
        (setq u2 (append u2 w)))
      (t
        (setq u2 (append u2 (list x))))))
    ((and w (equal (car w) "\\begin{verbatim}"))
      (setq verbatim t)
      (setq u2 (append u2 w)))
    (t (setq u2 (append u2 w)))))
(setq u u2)
(setq u (|checkAddSpaces| u))
(setq u (|checkIeEg| u))
(setq u (|checkSplit2Words| u))
(|checkBalance| u)
(setq okBefore (null |$checkErrorFlag|))
(|checkArguments| u)
(when |$checkErrorFlag| (setq u (|checkFixCommonProblem| u)))
(setq v (|checkDecorate| u))
(setq res
  (let ((result ""))
    (loop for y in v
      do (setq result (strconc result y)))
    result))
(setq res (|checkAddPeriod| res))
(when |$checkErrorFlag| (|pp| res))
res))

```

defun checkRewrite

```

[checkRemoveComments p516]
[checkAddIndented p518]
[checkGetArgs p520]
[newString2Words p525]
[checkAddSpaces p528]
[checkSplit2Words p517]
[checkAddMacros p527]
[checkTexht p511]
[checkArguments p499]
[checkFixCommonProblem p505]
[checkRecordHash p507]
[checkDecorateForHt p504]
[$checkErrorFlag p??]

```

```
[$arg1 p??]
[$checkingXmptex? p??]
```

— defun checkRewrite —

```
(defun |checkRewrite| (name lines)
  (declare (ignore name))
  (prog (|$checkErrorFlag| margin w verbatim u2 okBefore u)
    (declare (special |$checkErrorFlag| |$arg1| |$checkingXmptex?|))
    (setq |$checkErrorFlag| t)
    (setq margin 0)
    (setq lines (|checkRemoveComments| lines))
    (setq u lines)
    (when |$checkingXmptex?|
      (setq u
        (loop for x in u
          collect (|checkAddIndented| x margin))))
    (setq |$arg1| (|checkGetArgs| (car u)))
    (setq u2 nil)
    (setq verbatim nil)
    (loop for x in u
      do
        (setq w (|newString2Words| x))
        (cond
          (verbatim
            (cond
              ((and w (equal (car w) "\\end{verbatim}"))
               (setq verbatim nil)
               (setq u2 (append u2 w)))
              (t
               (setq u2 (append u2 (list x))))))
            ((and w (equal (car w) "\\begin{verbatim}"))
             (setq verbatim t)
             (setq u2 (append u2 w)))
            (t (setq u2 (append u2 w))))))
    (setq u u2)
    (setq u (|checkAddSpaces| u))
    (setq u (|checkSplit2Words| u))
    (setq u (|checkAddMacros| u))
    (setq u (|checkTexht| u))
    (setq okBefore (null |$checkErrorFlag|))
    (|checkArguments| u)
    (when |$checkErrorFlag| (setq u (|checkFixCommonProblem| u)))
    (|checkRecordHash| u)
    (|checkDecorateForHt| u)))
```

—

12.6 Comment Checking Layer 5

defun checkArguments

```
[hget p??]
[checkHTargs p506]
[$htMacroTable p??]
```

— defun checkArguments —

```
(defun |checkArguments| (u)
  (let (x k)
    (declare (special |$htMacroTable|))
    (loop while u
      do (setq x (car u))
        (cond
          ((null (setq k (hget |$htMacroTable| x))) '|skip|)
          ((eql k 0) '|skip|)
          ((> k 0) (|checkHTargs| x (cdr u) k nil))
          (t (|checkHTargs| x (cdr u) (- k) t)))
        (pop u))
      u))
```

—————

defun checkBalance

```
[checkBeginEnd p500]
[assoc p??]
[rassoc p??]
[checkDocError p515]
[checkSayBracket p532]
[nreverse p??]
[$checkPrenAlist p??]
```

— defun checkBalance —

```
(defun |checkBalance| (u)
  (let (x openClose open top restStack stack)
    (declare (special |$checkPrenAlist|))
    (|checkBeginEnd| u)
    (setq stack nil)
    (loop while u
      do
        (setq x (car u))
        (cond
```

```

((setq openClose (|assoc| x |$checkPrenAlist|))
 (setq stack (cons (car openClose) stack)))
((setq open (|rassoc| x |$checkPrenAlist|))
 (cond
  ((consp stack)
   (setq top (qcar stack))
   (setq restStack (qcdr stack))
   (when (not (eq open top))
    (|checkDocError|
     (list "Mismatch: left " (|checkSayBracket| top)
           " matches right " (|checkSayBracket| open))))
   (setq stack restStack))
  (t
   (|checkDocError|
    (list "Missing left " (|checkSayBracket| open))))))
(pop u))
(when stack
 (loop for x in (nreverse stack)
  do
   (|checkDocError| (list "Missing right " (|checkSayBracket| x)))))
u))

```

12.7 Comment Checking Layer 6

defun checkBeginEnd

```

[length p??]
[hget p??]
[ifcar p??]
[ifcdr p??]
[substring? p??]
[checkDocError p515]
[member p??]
[$beginEndList p??]
[$htMacroTable p??]

```

— defun checkBeginEnd —

```

(defun |checkBeginEnd| (u)
 (let (x y beginEndStack)
 (declare (special |$beginEndList| |$htMacroTable|))
 (loop while u
  do
   (setq x (car u))

```



```

(cond
  ((and (stringp x) (equal (elt x 0) #\\) (> (|#| x) 2)
    (null (hget |$htMacroTable| x)) (null (equal x "\\spadignore"))
    (equal (ifcar (ifcdr u)) #\\{)
    (null (or (|substring?| "\\radiobox" x 0)
      (|substring?| "\\inputbox" x 0))))
    (|checkDocError| (list 'Unexpected HT command: | x)))
  ((equal x "\\beginitems")
    (setq beginEndStack (cons 'items| beginEndStack)))
  ((equal x "\\begin")
    (cond
      ((and (consp u) (consp (qcdr u)) (equal (qcar (qcdr u)) #\\{)
        (consp (qcddr u)) (equal (car (qcddr u)) #\\}))
        (setq y (qcaddr u))
        (cond
          ((null (|member| y |$beginEndList|))
            (|checkDocError| (list "Unknown begin type: \\begin{" y " "}))))
          (setq beginEndStack (cons y beginEndStack))
          (setq u (qcddr u)))
        (t (|checkDocError| (list "Improper \\begin command")))))
      ((equal x "\\item")
        (cond
          ((|member| (ifcar beginEndStack) '("items" "menu")) nil)
          (null beginEndStack)
            (|checkDocError| (list "\\item appears outside a \\begin-\\end"))
          (t
            (|checkDocError|
              (list "\\item appears within a \\begin{"
                (ifcar beginEndStack) "}" ..))))))
      ((equal x "\\end")
        (cond
          ((and (consp u) (consp (qcdr u)) (equal (qcar (qcdr u)) #\\{)
            (consp (qcddr u)) (equal (car (qcddr u)) #\\}))
            (setq y (qcaddr u))
            (cond
              ((equal y (ifcar beginEndStack))
                (setq beginEndStack (cdr beginEndStack))
                (setq u (qcddr u)))
              (t
                (|checkDocError|
                  (list "Trying to match \\begin{" (ifcar beginEndStack)
                    "}" with \\end{" y " "}")))))
              (t
                (|checkDocError| (list "Improper \\end command")))))
          (pop u))
        (cond
          (beginEndStack
            (|checkDocError| (list "Missing \\end{" (car beginEndStack) " "})))
          (t 'ok))))))

```

defun checkDecorate

```
[checkDocError p515]
[member p??]
[checkAddBackSlashes p526]
[hasNoVowels p537]
[$checkingXmptex? p??]
[$charExclusions p??]
[$argl p??]
```

— defun checkDecorate —

```
(defun |checkDecorate| (u)
  (let (x count mathSymbolsOk spadflag verbatim v xcount acc)
    (declare (special |$argl| |$charExclusions| |$checkingXmptex?|))
    (setq count 0)
    (loop while u
      do
        (setq x (car u))
        (cond
          ((null verbatim)
            (cond
              ((string= x "\\em")
                (cond
                  ((> count 0)
                    (setq mathSymbolsOk (1- count))
                    (setq spadflag (1- count))))
                (t
                 (|checkDocError| (list "\\em must be enclosed in braces")))))
            (when (|member| x '("\\spadpaste" "\\spad" "\\spadop"))
              (setq mathSymbolsOk count))
            (cond
              ((|member| x '("\\s" "\\spadtype" "\\spadsys" "\\example" "\\andexample"
                "\\spadop" "\\spad" "\\spadignore" "\\spadpaste"
                "\\spadcommand" "\\footnote"))
                (setq spadflag count))
              ((equal x #\{)
                (setq count (1+ count)))
              ((equal x #\})
                (setq count (1- count))
                (when (eql mathSymbolsOk count) (setq mathSymbolsOk nil))
                (when (eql spadflag count) (setq spadflag nil)))
              (and (null mathSymbolsOk)
                (|member| x '("+ " *" "=" "==" "->"))
                (when |$checkingXmptex?|
                  (|checkDocError|
```

```

      (list '|Symbol | x " appearing outside \\spad{}"))))))))
(setq acc
 (cond
  ((string= x "\\end{verbatim}")
   (setq verbatim nil)
   (cons x acc))
  (verbatim (cons x acc))
  ((string= x "\\begin{verbatim}")
   (setq verbatim t)
   (cons x acc))
  ((and (string= x "\\begin")
        (equal (car (setq v (ifcdr u))) #\{)
        (string= (car (setq v (ifcdr v))) "detail")
        (equal (car (setq v (ifcdr v))) #\}))
        (setq u v)
        (cons "\\blankline " acc))
  ((and (string= x "\\end")
        (equal (car (setq v (ifcdr u))) #\{)
        (string= (car (setq v (ifcdr v))) "detail")
        (equal (car (setq v (ifcdr v))) #\}))
        (setq u v)
        acc)
  ((or (char= x #\$) (string= x "$"))
   (cons "\\$" acc))
  ((or (char= x #%) (string= x "%"))
   (cons "\\%" acc))
  ((or (char= x #\,) (string= x ","))
   (cons ",{" acc))
  ((string= x "\\spad")
   (cons "\\spad" acc))
  ((and (stringp x) (digitp (elt x 0)))
   (cons x acc))
  ((and (null spadflag)
        (or (and (charp x)
                  (alpha-char-p x)
                  (null (member x |$charExclusions|)))
            (|member| x |$argl|)))
   (cons #\} (cons x (cons #\{ (cons "\\spad" acc)))))
  ((and (null spadflag)
        (or (and (stringp x)
                  (null (equal (elt x 0) #\\))
                  (digitp (elt x (maxindex x))))
            (|member| x '("true" "false"))))
   (cons #\} (cons x (cons #\{ (cons "\\spad" acc)))))
  (t
   (setq xcount (|#| x))
   (cond
    ((and (eql xcount 3)
          (char= (elt x 1) #\t)
          (char= (elt x 2) #\h))

```

```

      (cons "th" (cons #\}
        (cons (elt x 0) (cons #\{ (cons "\\spad" acc))))))
    ((and (eql xcount 4)
      (char= (elt x 1) #\-)
      (char= (elt x 2) #\t)
      (char= (elt x 3) #\h))
      (cons "-th" (cons #\}
        (cons (elt x 0) (cons #\{ (cons "\\spad" acc))))))
    ((or (and (eql xcount 2)
      (char= (elt x 1) #\i))
      (and (null spadflag)
        (> xcount 0)
        (> 4 xcount)
        (null (|member| x '("th" "rd" "st"))
          (|hasNoVowels| x)))
      (cons #\}
        (cons x (cons #\{ (cons "\\spad" acc))))))
    (t
      (cons (|checkAddBackSlashes| x) acc))))
  (setq u (cdr u)))
(nreverse acc))

```

defun checkDecorateForHt

```

[checkDocError p515]
[member p??]
[$checkingXmptex? p??]

```

— defun checkDecorateForHt —

```

(defun |checkDecorateForHt| (u)
  (let (x count spadflag)
    (declare (special |$checkingXmptex?|))
    (setq count 0)
    (setq spadflag nil)
    (loop while u
      do
        (setq x (car u))
        (when (equal x "\\em")
          (if (> count 0)
            (setq spadflag (1- count))
            (|checkDocError| (list "\\em must be enclosed in braces"))))
        (cond
          ((|member| x '("\\s" "\\spadop" "\\spadtype" "\\spad" "\\spadpaste"
            "\\spadcommand" "\\footnote"))

```

```

      (setq spadflag count))
    ((equal x #\{)
      (setq count (1+ count)))
    ((equal x #\})
      (setq count (1- count))
      (when (equal spadflag count) (setq spadflag nil)))
    ((and (null spadflag) (|member| x '("+ " *" "=" "==" "->"))
      (when (|$checkingXmptex?|
        (|checkDocError| (list '|Symbol| x " appearing outside \spad{}"))))
      (t nil))
    (when (or (equal x "$") (equal x "%"))
      (|checkDocError| (list "Unescaped " x)))
    (pop u))
  u))

```

defun checkDocError1

```

[checkDocError p515]
[$compileDocumentation p169]

```

— defun checkDocError1 —

```

(defun |checkDocError1| (u)
  (declare (special |$compileDocumentation|))
  (if (and (boundp '|$compileDocumentation|) |$compileDocumentation|)
      nil
      (|checkDocError| u)))

```

defun checkFixCommonProblem

```

[member p??]
[ifcar p??]
[ifcdr p??]
[checkDocError p515]
[$HTspadmacros p??]

```

— defun checkFixCommonProblem —

```

(defun |checkFixCommonProblem| (u)
  (let (x next acc)
    (declare (special |$HTspadmacros|))

```

```

(loop while u
  do
    (setq x (car u))
    (cond
      ((and (equal x #\{)
            (|member| (setq next (ifcar (cdr u))) |$HTspadmacros|)
            (not (equal (ifcar (ifcdr (cdr u))) #\{))))
        (|checkDocError| (list "Reversing " next " and left brace")))
      (setq acc (cons #\{ (cons next acc)))
      (setq u (cddr u)))
    (t
      (setq acc (cons x acc))
      (setq u (cdr u))))))
(nreverse acc))

```

defun checkGetLispFunctionName

[charPosition p??]
 [checkDocError p515]

— defun checkGetLispFunctionName —

```

(defun |checkGetLispFunctionName| (s)
  (let (n k j)
    (setq n (|#| s))
    (cond
      ((and (setq k (|charPosition| #\| s 1))
            (> n k)
            (setq j (|charPosition| #\| s (1+ k)))
            (> n j))
        (substring s (1+ k) (1- (- j k)))))
    (t
      (|checkDocError| (cons "Ill-formed lisp expression : " (list s))
        '|illformed|))))

```

defun checkHTargs

Note that u should start with an open brace. [checkLookForLeftBrace p531]
 [checkLookForRightBrace p531]
 [checkDocError p515]
 [checkHTargs p506]

```
[ifcdr p??]
```

— **defun checkHTargs** —

```
(defun |checkHTargs| (keyword u nargs inteerValue?)
  (cond
    ((eq1 nargs 0) 'ok)
    ((null (setq u (|checkLookForLeftBrace| u)))
     (|checkDocError| (list "Missing argument for " keyword)))
    ((null (setq u (|checkLookForRightBrace| (ifcdr u))))
     (|checkDocError| (list "Missing right brace for " keyword)))
    (t
     (|checkHTargs| keyword (cdr u) (1- nargs) inteerValue?))))
```

—————

defun checkRecordHash

```
[member p??]
[checkLookForLeftBrace p531]
[checkLookForRightBrace p531]
[ifcdr p??]
[intern p??]
[hget p??]
[hput p??]
[checkGetLispFunctionName p506]
[checkGetStringBeforeRightBrace p521]
[checkGetParse p521]
[checkDocError p515]
[opOf p??]
[spadSysChoose p510]
[checkNumOfArgs p532]
[checkIsValidType p530]
[form2HtString p??]
[getl p??]
[$HTlinks p??]
[$htHash p??]
[$HTlisplinks p??]
[$lispHash p??]
[$glossHash p??]
[$currentSysList p??]
[$setOptions p??]
[$sysHash p??]
[$name p??]
[$origin p??]
```

```

[$sysHash p??]
[$glossHash p??]
[$lispHash p??]
[$htHash p??]

```

— defun checkRecordHash —

```

(defun |checkRecordHash| (u)
  (let (p q htname entry s parse n key x)
    (declare (special |$origin| |$name| |$sysHash| |$setOptions| |$glossHash|
                      |$currentSysList| |$lispHash| |$HTlisplinks| |$htHash|
                      |$HTlinks|))
    (loop while u
      do
        (setq x (car u))
        (when (and (stringp x) (equal (elt x 0) #\)))
          (cond
            ((and (|member| x |$HTlinks|)
                  (setq u (|checkLookForLeftBrace| (ifcdr u)))
                  (setq u (|checkLookForRightBrace| (ifcdr u)))
                  (setq u (|checkLookForLeftBrace| (ifcdr u)))
                  (setq u (ifcdr u)))
              (setq htname (|intern| (ifcar u)))
              (setq entry (or (hget |$htHash| htname) (list nil)))
              (hput |$htHash| htname
                (cons (car entry) (cons (cons |$name| |$origin|) (cdr entry)))))
            ((and (|member| x |$HTlisplinks|)
                  (setq u (|checkLookForLeftBrace| (ifcdr u)))
                  (setq u (|checkLookForRightBrace| (ifcdr u)))
                  (setq u (|checkLookForLeftBrace| (ifcdr u)))
                  (setq u (ifcdr u)))
              (setq htname
                (|intern|
                 (|checkGetLispFunctionName|
                  (|checkGetStringBeforeRightBrace| u))))
              (setq entry (or (hget |$lispHash| htname) (list nil)))
              (hput |$lispHash| htname
                (cons (car entry) (cons (cons |$name| |$origin|) (cdr entry)))))
            ((and (or (setq p (|member| x '("\gloss" "\spadglos")))
                      (setq q (|member| x '("\glossSee" "\spadglosSee"))))
                  (setq u (|checkLookForLeftBrace| (ifcdr u)))
                  (setq u (ifcdr u)))
              (when q
                (setq u (|checkLookForRightBrace| u))
                (setq u (|checkLookForLeftBrace| (ifcdr u)))
                (setq u (ifcdr u)))
              (setq htname (|intern| (|checkGetStringBeforeRightBrace| u)))
              (setq entry
                (or (hget |$glossHash| htname) (list nil)))

```



```

      (hput |$glossHash| htname
        (cons (car entry) (cons (cons |$name| |$origin|) (cdr entry)))))
((and (boot-equal x "\\spadsys")
      (setq u (|checkLookForLeftBrace| (ifcdr u)))
      (setq u (ifcdr u)))
 (setq s (|checkGetStringBeforeRightBrace| u))
 (when (char= (elt s 0) #\\) (setq s (substring s 1 nil)))
 (setq parse (|checkGetParse| s))
 (cond
  ((null parse)
   (|checkDocError| (list "Unparseable \\spadtype: " s)))
  ((null (|member| (|opOf| parse) |$currentSysList|))
   (|checkDocError| (list "Bad system command: " s)))
  ((or (atom parse)
       (null (and (consp parse) (eq (qcar parse) '|set|)
                  (consp (qcdr parse))
                  (eq (qcddr parse) nil)))))
   '|ok|)
 (null (|spadSysChoose| |$setOptions| (qcadr parse)))
 (progn
  (|checkDocError| (list "Incorrect \\spadsys: " s))
  (setq entry (or (hget |$sysHash| htname) (list nil)))
  (hput |$sysHash| htname
    (cons (car entry) (cons (cons |$name| |$origin|) (cdr entry)))))
((and (boot-equal x "\\spadtype")
      (setq u (|checkLookForLeftBrace| (ifcdr u)))
      (setq u (ifcdr u)))
 (setq s (|checkGetStringBeforeRightBrace| u))
 (setq parse (|checkGetParse| s))
 (cond
  ((null parse)
   (|checkDocError| (list "Unparseable \\spadtype: " s)))
  (t
   (setq n (|checkNumOfArgs| parse))
   (cond
    ((null n)
     (|checkDocError| (list "Unknown \\spadtype: " s)))
    ((and (atom parse) (> n 0))
     '|skip|)
    ((null (setq key (|checkIsValidType| parse)))
     (|checkDocError| (list "Unknown \\spadtype: " s)))
    ((atom key) '|ok|)
    (t
     (|checkDocError|
      (list "Wrong number of arguments: " (|form2HtString| key))))))
 (|member| x '("\\spadop" "\\keyword"))
 (setq u (|checkLookForLeftBrace| (ifcdr u)))
 (setq u (ifcdr u)))
 (setq x (|intern| (|checkGetStringBeforeRightBrace| u)))
 (when (null (or (get1 x '|Led|) (get1 x '|Nud|))))

```

```

      (|checkDocError| (list "Unknown \\spadop: " x))))))
    (pop u))
  '|done|))

```

defun spadSysChoose

```

[lassoc p??]
[spadSysBranch p510]

```

— defun spadSysChoose —

```

(defun |spadSysChoose| (tree form) ; tree is ((word . tree) ...)
  (let (lookupOn newTree)
    (cond
      ((null form) t)
      ((null tree) nil)
      (t
       (if (and (consp form) (consp (qcdr form)) (eq (qcddr form) nil))
           (setq lookupOn (qcar form))
           (setq lookupOn form))
       (when (setq newTree (lassoc lookupOn tree))
         (|spadSysBranch| newTree (cadr form)))))))

```

defun spadSysBranch

```

[spadSysChoose p510]
[member p??]
[systemError p??]

```

— defun spadSysBranch —

```

(defun |spadSysBranch| (tree arg) ; tree is (msg kind TREEorSomethingElse ...)
  (let (kind)
    (cond
      ((null arg) t)
      (t
       (setq kind (elt tree 2))
       (cond
         ((eq kind 'tree) (|spadSysChoose| (elt tree 4) arg))
         ((eq kind 'literals) (|member| arg (ELT tree 4)))
         ((eq kind 'integer) (integerp arg))

```

```
((eq kind 'function) (atom arg))
(t (|systemError| "unknown tree branch")))))))
```

defun checkTexht

```
[ifcdr p??]
[ifcar p??]
[checkDocError p515]
```

— defun checkTexht —

```
(defun |checkTexht| (u)
  (let (count y x acc)
    (setq count 0)
    (loop while u
      do
        (setq x (car u))
        (when (and (string= x "\\texht") (setq u (ifcdr u)))
          (unless (equal (ifcar u) #{\})
            (|checkDocError| "First left brace after \\texht missing"))
          ; drop first argument including braces of \texht
          (setq count 1)
          (loop while
            (or (not (equal (setq y (ifcar (setq u (cdr u)))) #\}))
              (> count 1))
          do
            (when (equal y #{\}) (setq count (1+ count)))
            (when (equal y #\}) (setq count (1- count)))
            ; drop first right brace of 1st arg
            (setq x (ifcar (setq u (cdr u)))))
        (when (and (string= x "\\httex")
          (setq u (ifcdr u))
          (equal (ifcar u) #{\}))
          ; left brace: add it
          (setq acc (cons (ifcar u) acc))
          (loop while
            (not (equal (setq y (ifcar (setq u (cdr u)))) #\}))
            do (setq acc (cons y acc)))
          ; right brace: add it
          (setq acc (cons (ifcar u) acc))
          ; left brace: forget it
          (setq x (ifcar (setq u (cdr u)))))
        (loop while (not (equal (ifcar (setq u (cdr u))) #\}))
          do 'skip|)
        ; forget right brace: move to next char
```

```

      (setq x (ifcar (setq u (cdr u))))
    (setq acc (cons x acc))
    (setq u (cdr u)))
  (nreverse acc)))

```

defun checkTransformFirsts

```

[pname p??]
[leftTrim p??]
[fillerSpaces p??]
[checkTransformFirsts p512]
[maxindex p??]
[checkSkipToken p517]
[checkSkipBlanks p533]
[getMatchingRightPren p536]
[checkDocError p515]
[strconc p??]
[getl p??]
[lassoc p??]
[$checkPrenAlist p??]

```

— defun checkTransformFirsts —

```

(defun |checkTransformFirsts| (opname u margin)
  (prog (namestring s m infixOp p open close z n i prefixOp j k firstWord)
    (declare (special |$checkPrenAlist|))
    (return
      (progn
        ; case 1: \spad{...
        ; case 2: form(args)
        (setq u (string-trim '(#\space) u)) ; spaces confuse us
        (setq namestring (pname opname))
        (cond
          ((equal namestring "Zero") (setq namestring "0"))
          ((equal namestring "One") (setq namestring "1"))
          (t nil))
        (cond
          (> margin 0)
          (setq s (|leftTrim| u))
          (strconc (|fillerSpaces| margin) (|checkTransformFirsts| opname s 0)))
        (t
          (setq m (maxindex u))
          (cond
            (> 2 m) u)

```

```

(equal (elt u 0) #\)) u)
(alpha-char-p (elt u 0))
(setq i (or (|checkSkipToken| u 0 m) (return u)))
(setq j (or (|checkSkipBlanks| u i m) (return u)))
(setq open (elt u j))
(cond
  ((or (and (equal open #\[) (setq close #\]))
        (and (equal open #\() (setq close #\))))
   (setq k (|getMatchingRightPren| u (1+ j) open close))
   (cond
     ((not (equal namestring (setq firstWord (substring u 0 i))))
      (|checkDocError|
       (list "(1) Improper first word in comments: " firstWord))
       u)
     ((null k)
      (cond
        ((equal open #\[)
         (|checkDocError|
          (list "Missing close bracket on first line: " u)))
        (t
         (|checkDocError|
          (list "Missing close parenthesis on first line: " u))))
      u)
     (t
      (strconc "\\spad{" (substring u 0 (1+ k)) "}"
                (substring u (1+ k) nil))))))
  (t
   (setq k (or (|checkSkipToken| u j m) (return u)))
   (setq infixOp (intern (substring u j (- k j))))
   (cond
; case 3: form arg
    ((null (get1 infixOp '|Led|))
     (cond
      ((not (equal namestring (setq firstWord (substring u 0 i))))
       (|checkDocError|
        (list "(2) Improper first word in comments: " firstWord))
        u)
      ((and (eql (|#| (setq p (pname infixOp))) 1)
              (setq open (elt p 0))
              (setq close (lassoc open |$checkPrenAlist|)))
       (setq z (|getMatchingRightPren| u (1+ k) open close))
       (when (> z (maxindex u)) (setq z (1- k)))
       (strconc "\\spad{" (substring u 0 (1+ z)) "}"
                 (substring u (1+ z) nil)))
      (t
       (strconc "\\spad{" (substring u 0 k) "}"
                 (substring u k nil))))))
    (t
     (setq z (or (|checkSkipBlanks| u k m) (return u)))
     (setq n (or (|checkSkipToken| u z m) (return u)))

```

```

(cond
  ((not (equal namestring (pname infixOp)))
    (|checkDocError|
      (list "(3) Improper initial operator in comments: " infixOp))
    u)
  (t
    (strconc "\\spad{" (substring u 0 n) "}"
      (substring u n nil)))))))))

; case 4: arg op arg
(t
  (setq i (or (|checkSkipToken| u 0 m) (return u)))
  (cond
    ((not (equal namestring (setq firstWord (substring u 0 i))))
      (|checkDocError|
        (list "(4) Improper first word in comments: " firstWord))
      u)
    (t
      (setq prefixOp (intern (substring u 0 i)))
      (cond
        ((null (get1 prefixOp '|Nud|)) u)
        (t
          (setq j (or (|checkSkipBlanks| u i m) (return u)))
          (cond
            ((equal (elt u j) #\ ( )
              (setq j
                (|getMatchingRightPren| u (1+ j) #\ ( #\ ) ))
              (cond
                ((> j m) u)
                (t
                  (strconc "\\spad{" (substring u 0 (1+ j)) "}"
                    (substring u (1+ j) nil))))))
              (t
                (setq k (or (|checkSkipToken| u j m) (return u)))
                (cond
                  ((not (equal namestring (setq firstWord (substring u 0 i))))
                    (|checkDocError|
                      (list "(5) Improper first word in comments: " firstWord))
                    u)
                  (t
                    (strconc "\\spad{" (substring u 0 k) "}"
                      (substring u k nil)))))))))))))))))

```

defun checkTrim

```
[charPosition p??]
[systemError p??]
[checkDocError p515]
[$charBlank p??]
[$x p??]
[$charPlus p??]
```

— defun checkTrim —

```
(defun |checkTrim| (|$x| lines)
  (declare (special |$x|))
  (labels (
    (trim (s)
      (let (k)
        (setq k (wherePP s))
        (substring s (+ k 2) nil)))
    (wherePP (u)
      (let (k)
        (setq k (|charPosition| #\+ u 0))
        (if (or (eql k (|#| u))
            (not (eql (|charPosition| #\+ u (1+ k)) (1+ k))))
            (|systemError| " Improper comment found")
            k))))
    (let (j s)
      (setq s (list (wherePP (car lines))))
      (loop for x in (rest lines)
        do
          (setq j (wherePP x))
          (unless (member j s)
            (|checkDocError| (list |$x| " has varying indentation levels"))
            (setq s (cons j s))))
      (loop for y in lines
        collect (trim y))))
```

—————

12.8 Comment Checking Layer 7**defun checkDocError**

```
[checkDocMessage p518]
[concat p??]
[saybrightly1 p??]
[sayBrightly p??]
```

```

[$checkErrorFlag p??]
[$recheckingFlag p??]
[$constructorName p??]
[$exposeFlag p??]
[$exposeFlagHeading p??]
[$outStream p??]
[$checkErrorFlag p??]
[$exposeFlagHeading p??]

```

— defun checkDocError —

```

(defun |checkDocError| (u)
  (let (msg)
    (declare (special |$outStream| |$exposeFlag| |$exposeFlagHeading|
                     |$constructorName| |$recheckingFlag| |$checkErrorFlag|))
    (setq |$checkErrorFlag| t)
    (setq msg
      (cond
        (|$recheckingFlag|
         (if |$constructorName|
             (|checkDocMessage| u)
             (|concat| "> " u)))
        (|$constructorName| (|checkDocMessage| u))
        (t u)))
    (when (and |$exposeFlag| |$exposeFlagHeading|)
      (saybrightly1 |$exposeFlagHeading| |$outStream|)
      (|sayBrightly| |$exposeFlagHeading|)
      (setq |$exposeFlagHeading| nil))
    (|sayBrightly| msg)
    (when |$exposeFlag| (saybrightly1 msg |$outStream|))))

```

—————

defun checkRemoveComments

[[checkTrimCommented p525](#)]

— defun checkRemoveComments —

```

(defun |checkRemoveComments| (lines)
  (let (line acc)
    (loop while lines
      do
        (setq line (|checkTrimCommented| (car lines)))
        (when (>= (|firstNonBlankPosition| line) 0) (push line acc))
        (pop lines))
    (reverse acc)))

```

defun checkSkipToken

[checkSkipIdentifierToken p523]
 [checkSkipOpToken p523]

— defun checkSkipToken —

```
(defun |checkSkipToken| (u i m)
  (let ((str (string-trim '(\space) u))) ; ignore leading spaces
    (if (alpha-char-p (elt str i))
        (|checkSkipIdentifierToken| str i m)
        (|checkSkipOpToken| str i m))))
```

defun checkSplit2Words

[checkSplitBrace p524]

— defun checkSplit2Words —

```
(defun |checkSplit2Words| (u)
  (let (x verbatim z acc)
    (setq acc nil)
    (loop while u
      do
        (setq x (car u))
        (setq acc
          (cond
            ((string= x "\\end{verbatim}")
             (setq verbatim nil)
             (cons x acc))
            (verbatim (cons x acc))
            ((string= x "\\begin{verbatim}")
             (setq verbatim t)
             (cons x acc))
            ((setq z (|checkSplitBrace| x))
             (append (nreverse z) acc))
            (t (cons x acc)))))
    (pop u))
  (nreverse acc)))
```

12.9 Comment Checking Layer 8

defun checkAddIndented

[firstNonBlankPosition p⁵³⁶]

[strconc p??]

[checkAddSpaceSegments p⁵²⁸]

TPDHERE: Note that this function was missing without error, so may be junk

— defun checkAddIndented —

```
(defun |checkAddIndented| (x margin)
  (let (k)
    (setq k (|firstNonBlankPosition| x))
    (cond
      ((eql k -1) "\\blankline ")
      ((eql margin k) x)
      (t
       (strconc "\\indented{" (princ-to-string (- k margin)) "}{"
        (|checkAddSpaceSegments| (substring x k nil) 0) "}"))))
```

defun checkDocMessage

[getdatabase p??]

[whoOwns p⁵³⁹]

[concat p??]

[\$x p??]

[\$constructorName p??]

— defun checkDocMessage —

```
(defun |checkDocMessage| (u)
  (let (sourcefile person middle)
    (declare (special |$constructorName| |$x|))
    (setq sourcefile (getdatabase |$constructorName| 'sourcefile))
    (setq person (or (|whoOwns| |$constructorName|) "----"))
    (setq middle
      (if (boundp '|$x|)
        (list "(" |$x| "): ")
        (list ": ")))
    (|concat| person ">" sourcefile "-->" |$constructorName| middle u)))
```

defun checkExtract

[firstNonBlankPosition p⁵³⁶]
 [substring? p??]
 [charPosition p??]
 [length p??]

— defun checkExtract —

```
(defun |checkExtract| (header lines)
  (let (line u margin firstLines m k j i acc)
    (loop while lines
      do
        (setq line (car lines))
        (setq k (|firstNonBlankPosition| line)) ; gives margin of description
        (if (|substring?| header line k)
          (return nil)
          (setq lines (cdr lines))))
    (cond
      ((null lines) nil)
      (t
       (setq u (car lines))
       (setq j (|charPosition| #\: u k))
       (setq margin k)
       (setq firstLines
        (if (not (eql (setq k (|firstNonBlankPosition| u (1+ j))) -1))
            (cons (substring u (1+ j) nil) (cdr lines))
            (cdr lines)))
        ; now look for another header; if found skip all rest of these lines
        (setq acc nil)
        (loop for line in firstLines
          do
            (setq m (|#| line))
            (cond
              ((eql (setq k (|firstNonBlankPosition| line)) -1) '|skip|)
              ((> k margin) '|skip|)
              ((null (upper-case-p (elt line k))) '|skip|)
              ((equal (setq j (|charPosition| #\: line k)) m) '|skip|)
              ((> j (setq i (|charPosition| #\space line (1+ k))) '|skip|)
               (t (return nil)))
              (setq acc (cons line acc)))
            (nreverse acc))))))
```

defun checkGetArgs

```
[maxindex p??]
[firstNonBlankPosition p536]
[checkGetArgs p520]
[stringPrefix? p??]
[getMatchingRightPren p536]
[charPosition p??]
[trimString p??]
```

— defun checkGetArgs —

```
(defun |checkGetArgs| (u)
  (let (m k acc i)
    (cond
      ((null (stringp u)) nil)
      (t
       (setq m (maxindex u))
       (setq k (|firstNonBlankPosition| u))
       (cond
         ((> k 0)
          (|checkGetArgs| (substring u k nil)))
         ((|stringPrefix?| "\\spad{" u)
          (setq k (or (|getMatchingRightPren| u 6 #{ #\\}) m))
          (|checkGetArgs| (substring u 6 (- k 6))))
         ((> (setq i (|charPosition| #\\( u 0)) m)
          nil)
          (not (eq1 (elt u m) #\\)))
          nil)
        (t
         (do ()
           ((null (> m (setq k (|charPosition| #\\, u (1+ i))))) nil)
           (setq acc
            (cons (|trimString| (substring u (1+ i) (1- (- k i)))) acc))
            (setq i k))
            (nreverse (cons (substring u (1+ i) (1- (- m i))) acc))))))))
```

— —

defun checkGetMargin

```
[firstNonBlankPosition p536]
```

— defun checkGetMargin —

```
(defun |checkGetMargin| (lines)
  (let (x k margin)
```

```
(loop while lines
  do
    (setq x (car lines))
    (setq k (|firstNonBlankPosition| x))
    (unless (= k -1) (setq margin (if margin (min margin k) k)))
    (pop lines))
  (or margin 0)))
```

defun checkGetParse

```
[ncParseFromString p??]
[removeBackslashes p539]
```

— defun checkGetParse —

```
(defun |checkGetParse| (s)
  (|ncParseFromString| (|removeBackslashes| s)))
```

defun checkGetStringBeforeRightBrace

— defun checkGetStringBeforeRightBrace —

```
(defun |checkGetStringBeforeRightBrace| (u)
  (prog (x acc)
    (return
      (loop while u
        do
          (setq x (car u))
          (cond
            ((equal x #\})
              (let ((result ""))
                (loop for item in acc
                  do (setq result (concatenate 'string item result)))
                (return result)))
            (t
              (setq acc (cons x acc))
              (setq u (cdr u))))))))
```

defun checkIeEg

[checkIeEgfun p529]
 [nreverse p??]

— defun checkIeEg —

```
(defun |checkIeEg| (u)
  (let (x verbatim z acc)
    (setq acc nil)
    (setq verbatim nil)
    (loop while u
      do
        (setq x (car u))
        (setq acc
          (cond
            ((equal x "\\end{verbatim}")
             (setq verbatim nil)
             (cons x acc))
            (verbatim (cons x acc))
            ((equal x "\\begin{verbatim}")
             (setq verbatim t)
             (cons x acc))
            ((setq z (|checkIeEgfun| x))
             (append (nreverse z) acc))
            (t (cons x acc))))
        (setq u (cdr u)))
    (nreverse acc)))
```

—————

defun checkIndentedLines

[firstNonBlankPosition p536]
 [strconc p??]
 [checkAddSpaceSegments p528]
 [\$charFauxNewline p??]

— defun checkIndentedLines —

```
(defun |checkIndentedLines| (u margin)
  (let (k s verbatim u2)
    (declare (special |$charFauxNewline|))
    (loop for x in u
      do
        (setq k (|firstNonBlankPosition| x))
        (cond
```

```

((eq1 k -1)
 (if verbatim
  (setq u2 (append u2 (list |$charFauxNewline|)))
  (setq u2 (append u2 (list "\\blankline "))))))
(t
 (setq s (substring x k nil))
 (cond
  ((string= s "\\begin{verbatim}")
   (setq verbatim t)
   (setq u2 (append u2 (list s))))
  ((string= s "\\end{verbatim}")
   (setq verbatim nil)
   (setq u2 (append u2 (list s))))
  (verbatim
   (setq u2 (append u2 (list (substring x margin nil))))))
 ((eq1 margin k)
  (setq u2 (append u2 (list s))))
 (t
  (setq u2
   (append u2
    (list (strconc "\\indented{" (princ-to-string (- k margin))
                  "}{ " (|checkAddSpaceSegments| s 0) "}")
          )))))
u2))

```

defun checkSkipIdentifierToken

[checkAlphabetic p529]

— defun checkSkipIdentifierToken —

```

(defun |checkSkipIdentifierToken| (u i m)
  (do ()
    ((null (and (> m i) (|checkAlphabetic| (elt u i)))) nil)
    (setq i (1+ i)))
  (unless (= i m) i))

```

defun checkSkipOpToken

[checkAlphabetic p529]

[member p??]

[\$charDelimiters p??]

— defun checkSkipOpToken —

```
(defun |checkSkipOpToken| (u i m)
  (declare (special |$charDelimiters|))
  (do ()
    ((null (and (> m i)
                (null (|checkAlphabetic| (elt u i)))
                (null (|member| (elt u i) |$charDelimiters|))))
      nil)
    (setq i (1+ i)))
  (unless (= i m) i))
```

—————

defun checkSplitBrace

```
[charp p??]
[length p??]
[checkSplitBackslash p533]
[checkSplitBrace p524]
[checkSplitOn p534]
[checkSplitPunctuation p535]
```

— defun checkSplitBrace —

```
(defun |checkSplitBrace| (x)
  (let (m u)
    (cond
      ((charp x) (list x))
      ((eql (|#| x) 1) (list (elt x 0)))
      ((and (setq u (|checkSplitBackslash| x)) (cdr u))
        (let (result)
          (loop for y in u do (append result (|checkSplitBrace| y)))
          result))
      (t
        (setq m (maxindex x))
        (cond
          ((and (setq u (|checkSplitOn| x)) (cdr u))
            (let (result)
              (loop for y in u do (append result (|checkSplitBrace| y)))
              result))
          ((and (setq u (|checkSplitPunctuation| x)) (cdr u))
            (let (result)
              (loop for y in u do (append result (|checkSplitBrace| y)))
              result))
          (t (list x))))))
```


defun checkTrimCommented

[length p??]
 [htcharPosition p537]

— defun checkTrimCommented —

```
(defun |checkTrimCommented| (line)
  (let (n k)
    (setq n (|#| line))
    (setq k (|htcharPosition| #\% line 0))
    (cond
      ((eq1 k 0) "")
      ((or (>= k (1- n)) (not (eq1 (elt line (1+ k)) #\%))) line)
      ((> (|#| line) k) (substring line 0 k))
      (t line))))
```

defun newString2Words

[newWordFrom p538]
 [nreverse0 p??]

— defun newString2Words —

```
(defun |newString2Words| (z)
  (let (m tmp1 w i result)
    (cond
      ((null (stringp z)) (list z))
      (t
       (setq m (maxindex z))
       (cond
         ((eq1 m -1) nil)
         (t
          (setq i 0)
          (do () ; [w while newWordFrom(l,i,m) is [w,i]]
              ((null (progn
                        (setq tmp1 (|newWordFrom| z i m))
                        (and (consp tmp1)
                           (progn
                              (setq w (qcar tmp1))
```

```

      (and (consp (qcdr tmp1))
            (eq (qcddr tmp1) nil)
            (progn
              (setq i (qcadr tmp1))
              t))))))
    (nreverse0 result))
  (setq result (cons (qcar tmp1) result))))))

```

12.10 Comment Checking Layer 9

defun checkAddBackSlashes

```

[strconc p??]
[maxindex p??]
[checkAddBackSlashes p526]
[$charEscapeList p??]

```

— defun checkAddBackSlashes —

```

(defun |checkAddBackSlashes| (s)
  (let (c m char insertIndex k)
    (declare (special |$charEscapeList|))
    (cond
      ((or (and (charp s) (setq c s))
            (and (eql (|#| s) 1) (setq c (elt s 0)))))
      (if (member s |$charEscapeList|)
          (strconc #\\ c)
          s))
      (t
       (setq k 0)
       (setq m (maxindex s))
       (setq insertIndex nil)
       (loop while (< k m)
         do
           (setq char (elt s k))
           (cond
             ((char= char #\\) (setq k (+ k 2)))
             ((member char |$charEscapeList|) (return (setq insertIndex k))))
           (setq k (1+ k)))
       (cond
         (insertIndex
          (|checkAddBackSlashes|
           (strconc (substring s 0 insertIndex) #\\ (elt s k)
                    (substring s (1+ insertIndex) nil))))

```

```
(T s))))))
```

defun checkAddMacros

```
[lassoc p??]
[nreverse p??]
[$HTmacs p??]
```

— defun checkAddMacros —

```
(defun |checkAddMacros| (u)
  (let (x verbatim y acc)
    (declare (special |$HTmacs|))
    (loop while u
      do
        (setq x (car u))
        (setq acc
          (cond
            ((string= x "\\end{verbatim}")
             (setq verbatim nil)
             (cons x acc))
            (verbatim
             (cons x acc))
            ((string= x "\\begin{verbatim}")
             (setq verbatim t)
             (cons x acc))
            ((setq y (lassoc x |$HTmacs|))
             (append y acc))
            (t (cons x acc))))
        (pop u))
    (nreverse acc)))
```

defun checkAddPeriod

```
[setelt p??]
[maxindex p??]
```

— defun checkAddPeriod —

```
(defun |checkAddPeriod| (s)
  (let (m lastChar)
```

```

(setq m (maxindex s))
(setq lastChar (elt s m))
(cond
  ((or (char= lastChar #\!) (char= lastChar #\?) (char= lastChar #\.)) s)
  ((or (char= lastChar #\,) (char= lastChar #\;))
    (setelt s m #\.)
    s)
  (t s))))

```

defun checkAddSpaceSegments

[checkAddSpaceSegments p528]
 [maxindex p??]
 [charPosition p??]
 [strconc p??]
 [\$charBlank p??]

— defun checkAddSpaceSegments —

```

(defun |checkAddSpaceSegments| (u k)
  (let (m i j n)
    (setq m (maxindex u))
    (setq i (|charPosition| #\space u k))
    (cond
      ((> i m) u)
      (t
       (setq j i)
       (loop while (and (incf j) (char= (elt u j) #\space)))
       (setq n (- j i)) ; number of blanks
       (if (> n 1)
         (strconc (substring u 0 i) "\\space{" (princ-to-string n) "}"
                   (|checkAddSpaceSegments| (substring u (+ i n) nil) 0))
         (|checkAddSpaceSegments| u j))))))

```

defun checkAddSpaces

[\$charBlank p??]
 [\$charFauxNewline p??]

— defun checkAddSpaces —

```
(defun |checkAddSpaces| (u)
  (let (u2 space i)
    (declare (special |$charFauxNewline|))
    (cond
      ((null u) nil)
      ((null (cdr u)) u)
      (t
       (setq space #\space)
       (setq i 0)
       (loop for f in u
         do
           (incf i)
           (when (string= f "\\begin{verbatim}")
             (setq space |$charFauxNewline|)
             (unless u2 (setq u2 (list space))))
           (if (> i 1)
             (setq u2 (append u2 (list space f)))
             (setq u2 (append u2 (list f))))
           (when (string= f "\\end{verbatim}")
             (setq u2 (append u2 (list space)))
             (setq space #\space)))
         u2))))
```

defun checkAlphabetic

[\$charIdentifierEndings p??]

— defun checkAlphabetic —

```
(defun |checkAlphabetic| (c)
  (declare (special |$charIdentifierEndings|))
  (or (alpha-char-p c) (digitp c) (member c |$charIdentifierEndings|)))
```

defun checkIeEgfun

[maxindex p??]
[checkIeEgFun p??]

— defun checkIeEgfun —

```
(defun |checkIeEgfun| (x)
```

```

(let (m key firstPart result)
  (cond
    ((characterp x) nil)
    ((equal x "") nil)
    (t
     (setq m (maxindex x))
     (loop for k from 0 to (- m 3)
           do
            (cond
              ((and
                (equal (elt x (1+ k)) #\.)
                (equal (elt x (+ k 3)) #\.)
                (or
                 (and
                  (equal (elt x k) #\i)
                  (equal (elt x (+ k 2)) #\e)
                  (setq key "that is"))
                 (and
                  (equal (elt x k) #\e)
                  (equal (elt x (+ k 2)) #\g)
                  (setq key "for example")))))
              (progn
               (setq firstPart (when (> k 0) (cons (substring x 0 k) nil)))
               (setq result
                (append firstPart
                 (cons "\\spadignore{"
                  (cons (substring x k 4)
                   (cons "}"
                    (|checkIeEgfun| (substring x (+ k 4) nil))))))))))
            result))))

```

defun checkIsValidType

This function returns ok if correct, form is wrong number of arguments, nil if unknown
 [length p??]
 [checkIsValidType p[530](#)]
 [constructor? p??]
 [abbreviation? p??]
 [getdatabase p??]

— defun checkIsValidType —

```

(defun |checkIsValidType| (form)
  (labels (
    (fn (form coSig)

```

```

(cond
  ((not (eql (|#| form) (|#| coSig))) form)
  ((let (result)
      (loop for x in (rest form)
            for flag in (rest coSig)
            do (when flag (setq result (or result (null (|checkIsValidType| x))))))
      result)
    nil)
  (t '|ok|))))
(let (op args conname)
  (cond
    ((atom form) '|ok|)
    (t
     (setq op (car form))
     (setq args (cdr form))
     (setq conname
      (if (|constructor?| op)
          op
          (|abbreviation?| op)))
     (when conname (fn form (getdatabase conname 'cosig)))))))

```

defun checkLookForLeftBrace

[\$charBlank p??]

— defun checkLookForLeftBrace —

```

(defun |checkLookForLeftBrace| (u)
  (loop while u
    do
      (cond
        ((equal (car u) #\{) (return (car u)))
        ((not (eql (car u) #\space)) (return nil))
        (t (pop u))))
    u)

```

defun checkLookForRightBrace

This returns a line beginning with right brace

— defun checkLookForRightBrace —

```
(defun |checkLookForRightBrace| (u)
  (let (found count)
    (setq count 0)
    (loop while u
      do
        (cond
          ((equal (car u) #\})
            (if (eql count 0)
              (return (setq found u))
              (setq count (1- count)))))
          ((equal (car u) #\{)
            (setq count (1+ count)))))
    (pop u))
  found))
```

defun checkNumOfArgs

A nil return implies that the argument list length does not match [opOf p??]

[constructor? p??]

[abbreviation? p??]

[getdatabase p??]

— defun checkNumOfArgs —

```
(defun |checkNumOfArgs| (conform)
  (let (conname)
    (setq conname (|opOf| conform))
    (when (or (|constructor?| conname) (setq conname (|abbreviation?| conname)))
      (|#| (getdatabase conname 'constructorargs)))))
```

defun checkSayBracket

— defun checkSayBracket —

```
(defun |checkSayBracket| (x)
  (cond
    ((or (char= x #\() (char= x #\))) "pren")
    ((or (char= x #\{) (char= x #\})) "brace")
    (t "bracket")))
```


defun checkSkipBlanks[`$charBlank p??`]— **defun checkSkipBlanks** —

```
(defun |checkSkipBlanks| (u i m)
  (do ()
    ((null (and (> m i) (equal (elt u i) #\space))) nil)
    (setq i (1+ i)))
  (unless (= i m) i))
```

defun checkSplitBackslash[`checkSplitBackslash p`[533](#)][`maxindex p??`][`charPosition p??`]— **defun checkSplitBackslash** —

```
(defun |checkSplitBackslash| (x)
  (let (m k u v)
    (cond
      ((null (stringp x)) (list x))
      (t
       (setq m (maxindex x))
       (cond
         ((> m (setq k (|charPosition| #\\ x 0)))
          (cond
            ((or (eql m 1) (alpha-char-p (elt x (1+ k)))) ;starts with backslash so
             (if (> m (setq k (|charPosition| #\\ x 1)))
                 ; yes, another backslash
                 (cons (substring x 0 k) (|checkSplitBackslash| (substring x k nil)))
                 ; no, just return the line
                 (list x)))
            ((eql k 0)
             ; starts with backspace but x.1 is not a letter; break it up
             (cons (substring x 0 2)
                   (|checkSplitBackslash| (substring x 2 nil))))
          (t
           (setq u (substring x 0 k))
           (setq v (substring x k 2))
```

```

      (if (= (1+ k) m)
        (list u v)
        (cons u
              (cons v
                    (|checkSplitBackslash|
                     (substring x (+ k 2) nil)))))))
    (t (list x))))))

```

defun checkSplitOn

```

[checkSplitOn p534]
[charp p??]
[maxindex p??]
[charPosition p??]
[$charSplitList p??]

```

— defun checkSplitOn —

```

(defun |checkSplitOn| (x)
  (let (m char k z)
    (declare (special |$charSplitList|))
    (cond
      ((charp x) (list x))
      (t
       (setq z |$charSplitList|)
       (setq m (maxindex x))
       (loop while z
         do
           (setq char (car z))
           (cond
             ((and (eql m 0) (equal (elt x 0) char))
              (return (setq k -1)))
             (t
              (setq k (|charPosition| char x 0))
              (cond
                ((and (> k 0) (equal (elt x (1- k)) #\\)) (list x))
                ((<= k m) (return k))))))
           (pop z))
       (cond
         ((null z) (list x))
         ((eql k -1) (list char))
         ((eql k 0) (list char (substring x 1 nil)))
         ((eql k (maxindex x)) (list (substring x 0 k) char))
         (t
          (cons (substring x 0 k)

```

```
(cons char (|checkSplitOn| (substring x (1+ k) nil)))))))))
```

defun checkSplitPunctuation

```
[charp p??]  
[maxindex p??]  
[checkSplitPunctuation p535]  
[charPosition p??]  
[hget p??]  
[$htMacroTable p??]
```

— defun checkSplitPunctuation —

```
(defun |checkSplitPunctuation| (x)
  (let (m lastchar v k u)
    (declare (special |$htMacroTable|))
    (cond
      ((charp x) (list x))
      (t
       (setq m (maxindex x))
       (cond
         ((> 1 m) (list x))
         (t
          (setq lastchar (elt x m))
          (cond
            ((and (equal lastchar #\.)
                  (equal (elt x (1- m)) #\.))
             (cond
               ((eql m 1) (list x))
               ((and (> m 3) (equal (elt x (- m 2)) #\.)
                     (append (|checkSplitPunctuation| (substring x 0 (- m 2)))
                             (list "..."))))
              (t
               (append (|checkSplitPunctuation| (substring x 0 (1- m)))
                       (list "..")))))
            ((or (equal lastchar #\.)
                  (equal lastchar #\;)
                  (equal lastchar #\,))
             (list (substring x 0 m) lastchar))
            ((and (> m 1) (equal (elt x (1- m)) #\''))
             (list (substring x 0 (1- m)) (substring x (1- m) nil)))
            (> m (setq k (|charPosition| #\\ x 0)))
            (cond
              ((eql k 0)
               (cond
```

```

      ((or (eql m 1) (hget |$htMacroTable| x) (alpha-char-p (elt x 1)))
       (list x))
      (t
       (setq v (substring x 2 nil))
       (cons (substring x 0 2) (|checkSplitPunctuation| v))))
      (t
       (setq u (substring x 0 k))
       (setq v (substring x k nil))
       (append (|checkSplitPunctuation| u)
               (|checkSplitPunctuation| v))))
      (> m (setq k (|charPosition| #\ - x 1)))
      (setq u (substring x (1+ k) nil))
      (cons (substring x 0 k)
            (cons #\ - (|checkSplitPunctuation| u))))
      (t
       (list x)))))))))

```

defun firstNonBlankPosition

[maxindex p??]

— defun firstNonBlankPosition —

```

(defun |firstNonBlankPosition| (&rest therest)
  (let ((x (car therest)) (options (cdr therest)) start k)
    (setq start (or (ifcar options) 0))
    (setq k -1)
    (loop for i from start to (maxindex x)
          do (when (not (eql (elt x i) #\space)) (return (setq k i))))
    k))

```

defun getMatchingRightPren

[maxindex p??]

— defun getMatchingRightPren —

```

(defun |getMatchingRightPren| (u j open close)
  (let (m c found count)
    (setq count 0)
    (setq m (maxindex u))

```

```

(loop for i from j to m
  do
    (setq c (elt u i))
    (cond
      ((equal c close)
       (if (eq1 count 0)
         (return (setq found i))
         (setq count (1- count))))
      ((equal c open)
       (setq count (1+ count))))
  found))

```

defun hasNoVowels

[maxindex p??]

— defun hasNoVowels —

```

(defun |hasNoVowels| (x)
  (labels (
    (isVowel (c)
      (or (eq c #\a) (eq c #\e) (eq c #\i) (eq c #\o) (eq c #\u)
          (eq c #\A) (eq c #\E) (eq c #\I) (eq c #\O) (eq c #\U))))
    (let (max)
      (setq max (maxindex x))
      (cond
        ((char= (elt x max) #\y) nil)
        (t
         (let ((result t))
           (loop for i from 0 to max
             do (setq result (and result (null (isVowel (elt x i))))))
           result))))))

```

defun htcharPosition

[length p??]

[charPosition p??]

[htcharPosition p537]

— defun htcharPosition —

```
(defun |htcharPosition| (char line i)
  (let (m k)
    (setq m (|#| line))
    (setq k (|charPosition| char line i))
    (cond
      ((eq1 k m) k)
      (> k 0)
      (if (not (eq1 (elt line (1- k)) #\))
          k
          (|htcharPosition| char line (1+ k))))
      (t 0))))
```

defun newWordFrom

```
[ $stringFauxNewline p?? ]
[ $charBlank p?? ]
[ $charFauxNewline p?? ]
```

— defun newWordFrom —

```
(defun |newWordFrom| (z i m)
  (let (ch done buf)
    (declare (special |$charFauxNewline| |$stringFauxNewline|))
    (loop while (and (<= i m) (char= (elt z i) #\space)) do (incf i))
    (cond
      (> i m) nil)
    (t
     (setq buf "")
     (setq ch (elt z i))
     (cond
       ((equal ch |$charFauxNewline|)
        (list |$stringFauxNewline| (1+ i)))
       (t
        (setq done nil)
        (loop while (and (<= i m) (null done))
          do
            (setq ch (elt z i))
            (cond
              ((or (equal ch #\space) (equal ch |$charFauxNewline|))
               (setq done t))
              (t
               (setq buf (strconc buf ch))
               (setq i (1+ i))))
            (list buf i)))))))
```

defun removeBackslashes

```
[charPosition p??]
[removeBackslashes p539]
[strconc p??]
[length p??]
```

— defun removeBackslashes —

```
(defun |removeBackslashes| (s)
  (let (k)
    (cond
      ((string= s "" ) "")
      ((> (|#| s) (setq k (|charPosition| #\\ s 0)))
       (if (eql k 0)
           (|removeBackslashes| (substring s 1 nil))
           (strconc (substring s 0 k)
                     (|removeBackslashes| (substring s (1+ k) nil))))))
    (t s))))
```

defun whoOwns

This function always returns nil in the current system. Since it has no side effects we define it to return nil. [getdatabase p??]

```
[strconc p??]
[awk p??]
[shut p??]
[$exposeFlag p??]
```

— defun whoOwns —

```
(defun |whoOwns| (con)
  (declare (ignore con))
  nil)
; (let (filename quoteChar instream value)
; (declare (special |$exposeFlag|))
; (cond
; ((null |$exposeFlag|) nil)
; (t
; (setq filename (getdatabase con 'sourcefile))
; (setq quoteChar #\\")
; (obey (strconc "awk '$2 == " quoteChar filename quoteChar
```

```
;          " {print $1}' whofiles > /tmp/temp"))
;  (setq instream (make-instream "/tmp/temp"))
;  (setq value (unless (eofp instream) (readline instream)))
;  (shut instream)
;  value))))
```

Chapter 13

Utility Functions

defun translabel

[translabel1 p541]

— defun translabel —

```
(defun translabel (x al)
  (translabel1 x al) x)
```

—————

defun translabel1

[refvecp p??]

[maxindex p??]

[translabel1 p541]

[lassoc p??]

— defun translabel1 —

```
(defun translabel1 (x al)
  "Transforms X according to AL = ((<label> . Sexpr) ..)."
  (cond
    ((refvecp x)
     (do ((i 0 (1+ i)) (k (maxindex x)))
         ((> i k)
          (if (let ((y (lassoc (elt x i) al))) (setelt x i y))
              (translabel1 (elt x i) al))))
     ((atom x) nil)
    ((let ((y (lassoc (first x) al)))
```

```
(if y (setf (first x) y) (translabel1 (cdr x) al))))
((translabel1 (first x) al) (translabel1 (cdr x) al))))
```

defun displayPreCompilationErrors

```
[length p??]
[remdup p??]
[sayBrightly p??]
[sayMath p??]
[$postStack p??]
[$topOp p??]
[$InteractiveMode p??]
```

— defun displayPreCompilationErrors —

```
(defun |displayPreCompilationErrors| ()
  (let (n errors heading)
    (declare (special |$postStack| |$topOp| |$InteractiveMode|))
    (setq n (|#| (setq |$postStack| (remdup (nreverse |$postStack|)))))
    (unless (eql n 0)
      (setq errors (cond ((> n 1) "errors") (t "error")))
      (cond
        (|$InteractiveMode|
         (|sayBrightly| (list " Semantic " errors " detected: ")))
        (t
         (setq heading
           (if (not (eq |$topOp| '|$topOp|))
               (list " " |$topOp| " has")
               (list " You have")))
         (|sayBrightly|
          (append heading (list n "precompilation " errors ":" )))))
      (cond
        ((> n 1)
         (let ((i 1))
           (dolist (x |$postStack|)
             (|sayMath| (cons " " (cons i (cons " " x))))))
          (t (|sayMath| (cons " " (car |$postStack|)))))
        (terpri))))
```

defun bumperrorcount

[[\\$InteractiveMode p??](#)]
 [[\\$spad-errors p??](#)]

— defun bumperrorcount —

```
(defun bumperrorcount (kind)
  (declare (special |$InteractiveMode| $spad_errors))
  (unless |$InteractiveMode|
    (let ((index (case kind
                    (|syntax| 0)
                    (|precompilation| 1)
                    (|semantic| 2)
                    (t (error (break "BUMPERRORCOUNT: kind=~s~%" kind))))))
      (setelt $spad_errors index (1+ (elt $spad_errors index))))))
```

—————

defun parseTranCheckForRecord

[[postError p374](#)]
 [[parseTran p101](#)]

— defun parseTranCheckForRecord —

```
(defun |parseTranCheckForRecord| (x op)
  (declare (ignore op))
  (let (tmp3)
    (setq x (|parseTran| x))
    (cond
      ((and (consp x) (eq (qfirst x) '|Record|))
        (cond
          ((do ((z nil tmp3) (tmp4 (qrest x) (cdr tmp4)) (y nil))
              ((or z (atom tmp4)) tmp3)
              (setq y (car tmp4))
              (cond
                ((null (and (consp y) (eq (qfirst y) '|:|) (consp (qrest y))
                           (consp (qcddr y)) (eq (qcddr y) nil)))
                  (setq tmp3 (or tmp3 y))))
            (postError (list " Constructor" x "has missing label" )))
          (t x)))
      (t x))))
```

—————

defun makeSimplePredicateOrNil

```
[isSimple p??]
[isAlmostSimple p??]
[wrapSEQExit p??]
```

— **defun makeSimplePredicateOrNil** —

```
(defun |makeSimplePredicateOrNil| (p)
  (let (u g)
    (cond
      ((|isSimple| p) nil)
      ((setq u (|isAlmostSimple| p)) u)
      (t (|wrapSEQExit| (list (list 'let (setq g (gensym)) p) g))))))
```

—————→

defun parse-spadstring

```
[match-current-token p456]
[token-symbol p??]
[push-reduction p465]
[advance-token p459]
```

— **defun parse-spadstring** —

```
(defun parse-spadstring ()
  (let* ((tok (match-current-token 'spadstring))
        (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'spadstring-token (copy-tree symbol))
      (advance-token)
      t)))
```

—————→

defun parse-string

```
[match-current-token p456]
[token-symbol p??]
[push-reduction p465]
[advance-token p459]
```

— **defun parse-string** —

```
(defun parse-string ()
  (let* ((tok (match-current-token 'string))
         (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'string-token (copy-tree symbol))
      (advance-token)
      t)))
```

defun parse-identifier

[match-current-token p456]
 [token-symbol p??]
 [push-reduction p465]
 [advance-token p459]

— defun parse-identifier —

```
(defun parse-identifier ()
  (let* ((tok (match-current-token 'identifier))
         (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'identifier-token (copy-tree symbol))
      (advance-token)
      t)))
```

defun parse-number

[match-current-token p456]
 [token-symbol p??]
 [push-reduction p465]
 [advance-token p459]

— defun parse-number —

```
(defun parse-number ()
  (let* ((tok (match-current-token 'number))
         (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'number-token (copy-tree symbol))
      (advance-token)
      t)))
```

defun parse-keyword

[match-current-token p456]
 [token-symbol p??]
 [push-reduction p465]
 [advance-token p459]

— defun parse-keyword —

```
(defun parse-keyword ()
  (let* ((tok (match-current-token 'keyword))
         (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'keyword-token (copy-tree symbol))
      (advance-token)
      t)))
```

defun parse-argument-designator

[push-reduction p465]
 [match-current-token p456]
 [token-symbol p??]
 [advance-token p459]

— defun parse-argument-designator —

```
(defun parse-argument-designator ()
  (let* ((tok (match-current-token 'argument-designator))
         (symbol (if tok (token-symbol tok))))
    (when tok
      (push-reduction 'argument-designator-token (copy-tree symbol))
      (advance-token)
      t)))
```

defun print-package

[[\\$out-stream p??](#)]

— defun print-package —

```
(defun print-package (package)
  (declare (special out-stream))
  (format out-stream "~&~%(IN-PACKAGE ~S )~%~%" package))
```

defun checkWarning

[[postError p374](#)]

[[concat p??](#)]

— defun checkWarning —

```
(defun |checkWarning| (msg)
  (postError (|concat| "Parsing error: " msg)))
```

defun tuple2List

[[tuple2List p547](#)]

[[postTranSegment p388](#)]

[[postTran p370](#)]

[[\\$boot p??](#)]

[[\\$InteractiveMode p??](#)]

— defun tuple2List —

```
(defun |tuple2List| (arg)
  (let (u p q)
    (declare (special |$InteractiveMode| $boot))
    (when (consp arg)
      (setq u (|tuple2List| (qrest arg)))
      (cond
        ((and (consp (qfirst arg)) (eq (qcaar arg) 'segment)
              (consp (qcdar arg))
              (consp (qcddar arg))
              (eq (qcdddar arg) nil))
```

```

(setq p (qcadar arg))
(setq q (qcaddar arg))
(cond
  ((null u) (list '|construct| (|postTranSegment| p q)))
  ((and |$InteractiveMode| (null $boot))
   (cons '|append|
         (cons (list '|construct| (|postTranSegment| p q))
               (list (|tuple2List| (qrest arg)))))))
  (t
   (cons '|nconc|
         (cons (list '|construct| (|postTranSegment| p q))
               (list (|tuple2List| (qrest arg)))))))
  ((null u) (list '|construct| (|postTran| (qfirst arg))))
  (t (list '|cons| (|postTran| (qfirst arg)) (|tuple2List| (qrest arg))))))

```

defmacro pop-stack-1

[reduction-value p??]
 [Pop-Reduction p550]

— defmacro pop-stack-1 —

```
(defmacro pop-stack-1 () '(reduction-value (Pop-Reduction)))
```

defmacro pop-stack-2

[stack-push p96]
 [reduction-value p??]
 [Pop-Reduction p550]

— defmacro pop-stack-2 —

```

(defmacro pop-stack-2 ()
  '(let* ((top (Pop-Reduction)) (next (Pop-Reduction)))
    (stack-push top Reduce-Stack)
    (reduction-value next)))

```

defmacro pop-stack-3

[stack-push p96]
 [reduction-value p??]
 [Pop-Reduction p550]

— **defmacro pop-stack-3** —

```
(defmacro pop-stack-3 ()
  '(let* ((top (Pop-Reduction)) (next (Pop-Reduction)) (nnext (Pop-Reduction)))
    (stack-push next Reduce-Stack)
    (stack-push top Reduce-Stack)
    (reduction-value nnext)))
```

defmacro pop-stack-4

[stack-push p96]
 [reduction-value p??]
 [Pop-Reduction p550]

— **defmacro pop-stack-4** —

```
(defmacro pop-stack-4 ()
  '(let* ((top (Pop-Reduction))
    (next (Pop-Reduction))
    (nnext (Pop-Reduction))
    (nnnext (Pop-Reduction)))
    (stack-push nnnext Reduce-Stack)
    (stack-push next Reduce-Stack)
    (stack-push top Reduce-Stack)
    (reduction-value nnnext)))
```

defmacro nth-stack

[stack-store p??]
 [reduction-value p??]

— **defmacro nth-stack** —

```
(defmacro nth-stack (x)
```

```
'(reduction-value (nth (1- ,x) (stack-store Reduce-Stack))))
```

defun Pop-Reduction

[stack-pop [p97](#)]

— defun Pop-Reduction —

```
(defun Pop-Reduction () (stack-pop Reduce-Stack))
```

defun addclose

[suffix p??]

— defun addclose —

```
(defun addclose (line char)
  (cond
    ((char= (char line (maxindex line)) #\; )
     (setelt line (maxindex line) char)
     (if (char= char #\;) line (suffix #\; line)))
    ((suffix char line))))
```

defun blankp

— defun blankp —

```
(defun blankp (char)
  (or (eq char #\Space) (eq char #\tab)))
```

defun drop

Return a pointer to the Nth cons of X, counting 0 as the first cons. [drop p551]
 [take p??]
 [croak p??]

— defun drop —

```
(defun drop (n x &aux m)
  (cond
    ((eq1 n 0) x)
    ((> n 0) (drop (1- n) (cdr x)))
    ((>= (setq m (+ (length x) n)) 0) (take m x))
    ((croak (list "Bad args to DROP" n x)))))
```

—————

defun escaped

— defun escaped —

```
(defun escaped (str n)
  (and (> n 0) (eq (char str (1- n)) #\_)))
```

—————

defvar \$comblocklist

— initvars —

```
(defvar $comblocklist nil "a dynamic lists of comments for this block")
```

—————

defun fincomblock

- NUM is the line number of the current line
- OLDNUMS is the list of line numbers of previous lines
- OLDLOCS is the list of previous indentation locations

- NCBLOCK is the current comment block

```
[preparse-echo p95]
[$comblocklist p551]
[$EchoLineStack p??]
```

— **defun fincomblock** —

```
(defun fincomblock (num oldnums oldlocs ncblock linelist)
  (declare (special $EchoLineStack $comblocklist))
  (push
   (cond
    ((eql (car ncblock) 0) (cons (1- num) (reverse (cdr ncblock))))
    ;; comment for constructor itself paired with 1st line -1
    (t
     (when $EchoLineStack
      (setq num (pop $EchoLineStack))
      (preparse-echo linelist)
      (setq $EchoLineStack (list num)))
     (cons
      ;; scan backwards for line to left of current
      (do ((onums oldnums (cdr onums))
          (olocs oldlocs (cdr olocs))
          (sloc (car ncblock)))
          ((null onums) nil)
          (when (and (numberp (car olocs)) (<= (car olocs) sloc))
            (return (car onums))))
      (reverse (cdr ncblock)))))
   $comblocklist))
```

—————

defun indent-pos

```
[next-tab-loc p553]
```

— **defun indent-pos** —

```
(defun indent-pos (str)
  (do ((i 0 (1+ i)) (pos 0))
      ((>= i (length str)) nil)
      (case (char str i)
        (#\space (incf pos))
        (#\tab (setq pos (next-tab-loc pos)))
        (otherwise (return pos)))))
```

—————

defun infixtok

[string2id-n p??]

— defun infixtok —

```
(defun infixtok (s)
  (member (string2id-n s 1) '(|then| |else|) :test #'eq))
```

—————

defun is-console

[fp-output-stream p??]

[*terminal-io* p??]

is-console : Stream → Boolean

— defun is-console —

```
(defun is-console (stream)
  (and (stream-p stream) (output-stream-p stream)
    (eq (system:fp-output-stream stream)
        (system:fp-output-stream *terminal-io*))))
```

—————

defun next-tab-loc

— defun next-tab-loc —

```
(defun next-tab-loc (i)
  (* (1+ (truncate i 8)) 8))
```

—————

defun nonblankloc[blankp [p550](#)]

— defun nonblankloc —

```
(defun nonblankloc (str)
  (position-if-not #'blankp str))
```

defun parseprint

— defun parseprint —

```
(defun parseprint (l)
  (when l
    (format t "~&~%          ***      PREPARSE      ***~%~%"
      (dolist (x l) (format t "~5d. ~a~%" (car x) (cdr x)))
      (format t "~%"))))
```

defun skip-to-endif

```
[initial-substring p635]
[preparseReadLine p??]
[preparseReadLine1 p93]
[skip-to-endif p554]
```

— defun skip-to-endif —

```
(defun skip-to-endif (x)
  (let (line ind tmp1)
    (setq tmp1 (preparseReadLine1))
    (setq ind (car tmp1))
    (setq line (cdr tmp1))
    (cond
      ((not (stringp line)) (cons ind line))
      ((initial-substring line ")endif") (preparseReadLine x))
      ((initial-substring line ")fin") (cons ind nil))
      (t (skip-to-endif x)))))
```

Chapter 14

The Compiler

defvar \$newConlist

A list of new constructors discovered during compile. These are used in a call to `extendLocalLibdb` when a user compiles new local code.

— **initvars** —

```
(defvar |$newConlist| nil
  "A list of new constructors discovered during compile ")
```

—————

14.1 Compiling EQ.spad

Given the top level command:

```
)co EQ
```

The default call chain looks like:

```
1> (|compiler| ...)
2> (|compileSpad2Cmd| ...)
   Compiling AXIOM source code from file /tmp/A.spad
3> (|compilerDoit| ...)
4> (|/RQ,LIB|)
5> (/RF-1 ...)
6> (SPAD ...)
   AXSERV abbreviates package AxiomServer
7> (S-PROCESS ...)
8> (|compTopLevel| ...)
```

```

9> (|compOrCroak| ...)
10> (|compOrCroak1| ...)
11> (|comp| ...)
12> (|compNoStacking| ...)
13> (|comp2| ...)
14> (|comp3| ...)
15> (|compExpression| ...)
* 16> (|compWhere| ...)
17> (|comp| ...)
18> (|compNoStacking| ...)
19> (|comp2| ...)
20> (|comp3| ...)
21> (|compExpression| ...)
22> (|compSeq| ...)
23> (|compSeq1| ...)
24> (|compSeqItem| ...)
25> (|comp| ...)
26> (|compNoStacking| ...)
27> (|comp2| ...)
28> (|comp3| ...)
29> (|compExpression| ...)
<29 (|compExpression| ...)
<28 (|comp3| ...)
<27 (|comp2| ...)
<26 (|compNoStacking| ...)
<25 (|comp| ...)
<24 (|compSeqItem| ...)
24> (|compSeqItem| ...)
25> (|comp| ...)
26> (|compNoStacking| ...)
27> (|comp2| ...)
28> (|comp3| ...)
29> (|compExpression| ...)
30> (|compExit| ...)
31> (|comp| ...)
32> (|compNoStacking| ...)
33> (|comp2| ...)
34> (|comp3| ...)
35> (|compExpression| ...)
<35 (|compExpression| ...)
<34 (|comp3| ...)
<33 (|comp2| ...)
<32 (|compNoStacking| ...)
<31 (|comp| ...)
31> (|modifyModeStack| ...)
<31 (|modifyModeStack| ...)
<30 (|compExit| ...)
<29 (|compExpression| ...)
<28 (|comp3| ...)
<27 (|comp2| ...)

```



```

<26 (|compNoStacking| ...)
<25 (|comp| ...)
<24 (|compSeqItem| ...)
24> (|replaceExitEtc| ...)
25> (|replaceExitEtc,fn| ...)
26> (|replaceExitEtc| ...)
27> (|replaceExitEtc,fn| ...)
28> (|replaceExitEtc| ...)
29> (|replaceExitEtc,fn| ...)
<29 (|replaceExitEtc,fn| ...)
<28 (|replaceExitEtc| ...)
28> (|replaceExitEtc| ...)
29> (|replaceExitEtc,fn| ...)
<29 (|replaceExitEtc,fn| ...)
<28 (|replaceExitEtc| ...)
<27 (|replaceExitEtc,fn| ...)
<26 (|replaceExitEtc| ...)
26> (|replaceExitEtc| ...)
27> (|replaceExitEtc,fn| ...)
28> (|replaceExitEtc| ...)
29> (|replaceExitEtc,fn| ...)
30> (|replaceExitEtc| ...)
31> (|replaceExitEtc,fn| ...)
32> (|replaceExitEtc| ...)
33> (|replaceExitEtc,fn| ...)
<33 (|replaceExitEtc,fn| ...)
<32 (|replaceExitEtc| ...)
32> (|replaceExitEtc| ...)
33> (|replaceExitEtc,fn| ...)
<33 (|replaceExitEtc,fn| ...)
<32 (|replaceExitEtc| ...)
<31 (|replaceExitEtc,fn| ...)
<30 (|replaceExitEtc| ...)
30> (|convertOrCroak| ...)
31> (|convert| ...)
<31 (|convert| ...)
<30 (|convertOrCroak| ...)
<29 (|replaceExitEtc,fn| ...)
<28 (|replaceExitEtc| ...)
28> (|replaceExitEtc| ...)
29> (|replaceExitEtc,fn| ...)
<29 (|replaceExitEtc,fn| ...)
<28 (|replaceExitEtc| ...)
<27 (|replaceExitEtc,fn| ...)
<26 (|replaceExitEtc| ...)
<25 (|replaceExitEtc,fn| ...)
<24 (|replaceExitEtc| ...)
<23 (|compSeq1| ...)
<22 (|compSeq| ...)
<21 (|compExpression| ...)

```

```

    <20 (|comp3| ...)
    <19 (|comp2| ...)
    <18 (|compNoStacking| ...)
  <17 (|comp| ...)
17> (|comp| ...)
    18> (|compNoStacking| ...)
    19> (|comp2| ...)
    20> (|comp3| ...)
    21> (|compExpression| ...)
    22> (|comp| ...)
    23> (|compNoStacking| ...)
    24> (|comp2| ...)
    25> (|comp3| ...)
        26> (|compColon| ...)
        <26 (|compColon| ...)
        <25 (|comp3| ...)
        <24 (|comp2| ...)
        <23 (|compNoStacking| ...)
    <22 (|comp| ...)

```

In order to explain the compiler we will walk through the compilation of EQ.spad, which handles equations as mathematical objects. We start the system. Most of the structure in Axiom are circular so we have to the **print-cycle** to true.

```
root@spiff:/tmp# axiom -nox
```

```
(1) -> )lisp (setq *print-circle* t)
```

```
Value = T
```

We trace the function we find interesting:

```
(1) -> )lisp (trace |compiler|)
```

```
Value = (|compiler|)
```

14.2 The top level compiler command

This is the graph of the functions used for compDefine. The syntax is a graphviz dot file. To generate this graph as a JPEG file, type:

```
tangle v9compDefine.dot bookvol9.pamphlet >v9compdefine.dot
dot -Tjpg v9compiler.dot >v9compiler.jpg
```

— v9compiler.dot —

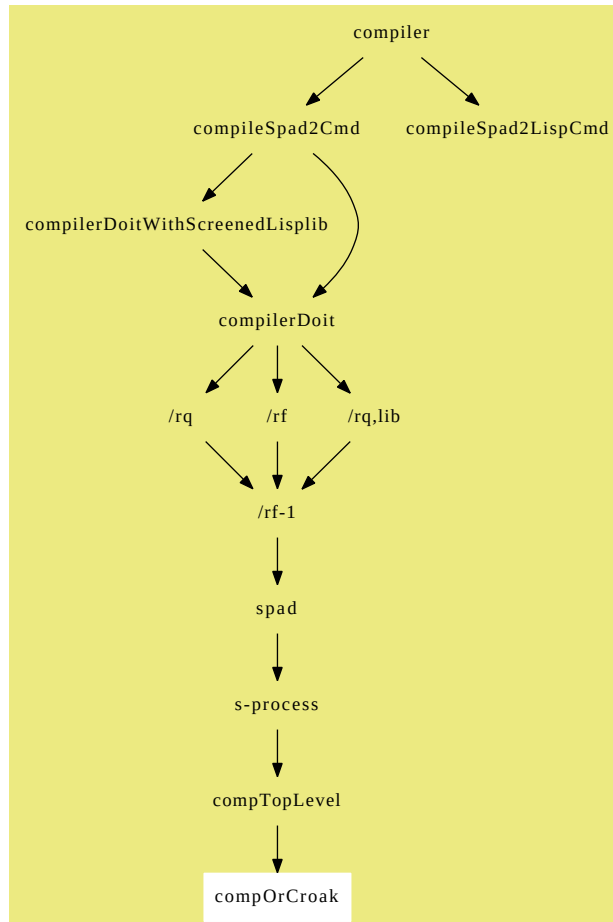
```

digraph pic {
    fontsize=10;
    bgcolor="#ECEA81";
    node [shape=box, color=white, style=filled];

    "compiler"                [color="#ECEA81"]
    "compileSpad2Cmd"         [color="#ECEA81"]
    "compileSpad2LispCmd"     [color="#ECEA81"]
    "compilerDoitWithScreenedLisplib" [color="#ECEA81"]
    "compilerDoit"            [color="#ECEA81"]
    "/rq"                     [color="#ECEA81"]
    "/rf"                      [color="#ECEA81"]
    "/rf-1"                   [color="#ECEA81"]
    "/rq,lib"                 [color="#ECEA81"]
    "spad"                    [color="#ECEA81"]
    "s-process"               [color="#ECEA81"]
    "compTopLevel"            [color="#ECEA81"]
    "compOrCroak"             [color="#FFFFFF"]

    "compiler" -> "compileSpad2Cmd"
    "compiler" -> "compileSpad2LispCmd"
    "compileSpad2Cmd" -> "compilerDoitWithScreenedLisplib"
    "compileSpad2Cmd" -> "compilerDoit"
    "compilerDoitWithScreenedLisplib" -> "compilerDoit"
    "compilerDoit" -> "/rq"
    "compilerDoit" -> "/rf"
    "compilerDoit" -> "/rq,lib"
    "/rq" -> "/rf-1"
    "/rf" -> "/rf-1"
    "/rq,lib" -> "/rf-1"
    "/rf-1" -> "spad"
    "spad" -> "s-process"
    "s-process" -> "compTopLevel"
    "compTopLevel" -> "compOrCroak"
}

```



defun compiler

We compile the spad file. We can see that the **compiler** function gets a list

```
(1) -> )co EQ
```

```
1> (|compiler| (EQ))
```

In order to find this file, the **pathname** and **pathnameType** functions are used to find the location and pathname to the file. They **pathnameType** function eventually returns the fact that this is a spad source file. Once that is known we call the **compileSpad2Cmd** function with a list containing the full pathname as a string.

```
1> (|compiler| (EQ))
2> (|pathname| (EQ))
<2 (|pathname| #p"EQ")
```

```

2> (|pathnameType| #p"EQ")
3> (|pathname| #p"EQ")
<3 (|pathname| #p"EQ")
<2 (|pathnameType| NIL)
2> (|pathnameType| "/tmp/EQ.spad")
3> (|pathname| "/tmp/EQ.spad")
<3 (|pathname| #p"/tmp/EQ.spad")
<2 (|pathnameType| "spad")
2> (|pathnameType| "/tmp/EQ.spad")
3> (|pathname| "/tmp/EQ.spad")
<3 (|pathname| #p"/tmp/EQ.spad")
<2 (|pathnameType| "spad")
2> (|pathnameType| "/tmp/EQ.spad")
3> (|pathname| "/tmp/EQ.spad")
<3 (|pathname| #p"/tmp/EQ.spad")
<2 (|pathnameType| "spad")
2> (|compileSpad2Cmd| ("/tmp/EQ.spad"))

```

```

[compiler helpSpad2Cmd (vol5)]
[compiler selectOptionLC (vol5)]
[compiler pathname (vol5)]
[compiler mergePathnames (vol5)]
[compiler pathnameType (vol5)]
[compiler namestring (vol5)]
[throwKeyedMsg p??]
[findfile p??]
[compileSpad2Cmd p562]
[compileSpadLispCmd p565]
[$newConlist p555]
[$options p??]
[/editfile p??]

```

compiler : (CONS SYMBOL NIL) → Prompt
— defun compiler —

```

(defun |compiler| (args)
  "The top level compiler command"
  (let (|$newConlist| ef thefile pathname pathtype)
    (declare (special |$newConlist| |$options| /editfile))
    (setq |$newConlist| nil)
    (if (and (null args) (null |$options|) (null /editfile))
        (|helpSpad2Cmd| '(|compiler|))
        (progn
          (unless args (setq args (cons /editfile nil)))
          (setq pathname (|pathname| args))
          (setq pathtype (|pathnameType| pathname))
          (cond
            ; have we been given a spad file?
            ((string= pathtype "spad")

```

```

(if (null (setq thefile ($findfile pathname '(|spad|))))
  (|throwKeyedMsg| "The spad file %1 is needed but does not exist."
    (cons (namestring pathname) nil))
  (|compileSpad2Cmd| (list thefile)))
; have we been given an nrllib?
((string= pathtype "nrllib")
 (if (null (setq thefile ($findfile pathname '(|nrllib|))))
   (|throwKeyedMsg| "The nrllib file %1 is needed but does not exist."
     (cons (namestring pathname) nil))
   (|compileSpadLispCmd| (list thefile))))
(t
 (setq thefile ($findfile pathname '(|spad|)))
 (cond
  ((and thefile (string= (|pathnameType| thefile) "spad"))
   (|compileSpad2Cmd| (list thefile)))
  (t
   (setq ef (|pathname| /editfile))
   (setq ef (|mergePathnames| pathname ef))
   (cond
    ((equal ef pathname)
     (|throwKeyedMsg|
      (format nil "Only AXIOM source files with file extension ~
                  .spad can be compiled.")
      nil))
    (t
     (setq pathname ef)
     (cond
      ((string= (|pathnameType| pathname) "spad")
       (|compileSpad2Cmd| args))
      (t
       (setq thefile ($findfile pathname '(|spad|)))
       (cond
        ((and thefile (string= (|pathnameType| thefile) "spad"))
         (|compileSpad2Cmd| (cons thefile nil)))
        (t (|throwKeyedMsg|
              (format nil "Only AXIOM source files with file extension ~
                          .spad can be compiled.")
              nil))))))))))))))

```

defun compileSpad2Cmd

This function sets up a constructor name (or nil if the whole file is to be compiled) and a set of options for the next layer of compile commands. For example, the call

```
)co dh )constructor DHMATRIX )functions identity
```

will look for the file `dh.spad` and issue a call

```
(|compilerDoitWithScreenedLisplib| (|DenavitHartenbergMatrix|) (|c| |lib|))
```

The argument to this function, as noted above, is a list containing the string pathname to the file.

```
2> (|compileSpad2Cmd| ("/tmp/EQ.spad"))
```

Again we find a lot of redundant work. We finally end up calling **compilerDoit** with a constructed argument list:

```
2> (|compilerDoit| NIL (|rq| |lib|))

[compileSpad2Cmd pathname (vol5)]
[compileSpad2Cmd pathnameType (vol5)]
[compileSpad2Cmd namestring (vol5)]
[compileSpad2Cmd updateSourceFiles (vol5)]
[compileSpad2Cmd selectOptionLC (vol5)]
[compileSpad2Cmd terminateSystemCommand (vol5)]
[throwKeyedMsg p??]
[compileSpad2Cmd sayKeyedMsg (vol5)]
[error p??]
[strconc p??]
[object2String p??]
[browserAutoloadOnceTrigger p??]
[spad2AsTranslatorAutoloadOnceTrigger p??]
[compilerDoitWithScreenedLisplib p??]
[compilerDoit p568]
[extendLocalLibdb p475]
[spadPrompt p??]
[$newComp p??]
[$scanIfTrue p??]
[$compileOnlyCertainItems p??]
[$f p??]
[$m p??]
[$QuickLet p??]
[$QuickCode p??]
[$sourceFileTypes p??]
[$InteractiveMode p??]
[$options p??]
[$newConlist p555]
[/editfile p??]
```

compileSpad2Cmd : (CONS PathnameString NIL) → Prompt
 — defun compileSpad2Cmd —

```

(defun |compileSpad2Cmd| (args)
  (let (|$newComp| |$scanIfTrue|
        |$compileOnlyCertainItems| |$f| |$m| |$QuickLet| |$QuickCode|
        |$sourceFileTypes| |$InteractiveModel| path optlist fun optname
        optargs fullopt constructor)
    (declare (special |$newComp| |$scanIfTrue|
                      |$compileOnlyCertainItems| |$f| |$m| |$QuickLet| |$QuickCode|
                      |$sourceFileTypes| |$InteractiveModel| /editfile |$options|
                      |$newConlist|))
    (setq path (|pathname| args))
    (cond
     ((not (string= (|pathnameType| path) "spad"))
      (|throwKeyedMsg|
       (format nil "The old AXIOM system compiler can only compile files ~
                    with file extension '.spad'."))
      nil))
     ((null (probe-file path))
      (|throwKeyedMsg| "The file %1 is needed but does not exist."
        (cons (|namestring| args) nil)))
     (t
      (setq /editfile path)
      (|updateSourceFiles| path)
      (|sayKeyedMsg| "Compiling AXIOM source code from %1"
        (list (file-namestring (|namestring| args))))
      (setq optlist '(|break| |constructor| |functions| |library| |lisp|
                     |nolib| |nolibrary| |noquiet| |vartrace| |quiet|))
      (setq |$QuickLet| t)
      (setq |$QuickCode| t)
      (setq fun '(|rq| |lib|))
      (setq |$sourceFileTypes| '("SPAD"))
      (dolist (opt |$options|)
        (setq optname (car opt))
        (setq optargs (cdr opt))
        (setq fullopt (|selectOptionLC| optname optlist nil))
        (case fullopt

          ; library exposes the result
          (|library| (setelt fun 1 '|lib|))

          ; nolibrary compiles but does not expose the result
          (|nolib| (setelt fun 1 '|nolib|))

          ; quiet suppresses compiler output
          (|quiet| (when (not (eq (elt fun 0) '|c|)) (setelt fun 0 '|rq|)))

          ; noquiet shows compiler output
          (|noquiet| (when (not (eq (elt fun 0) '|c|)) (setelt fun 0 '|rf|)))

          ; compiled code will not cause a break if it fails
          (|nolib| (setq |$scanIfTrue| t))

```



```

; compiled code will cause a break if it fails
(|break| (setq |$scanIfTrue| nil))

; allow variable tracing, otherwise lets are inlined
(|vartrace| (setq |$QuickLet| nil))

; compile functions from a domain, e.g.
; )co dh )constructor DHMATRIX )functions identity
(|functions|
  (if (null optargs)
    (|throwKeyedMsg| ")functions requires and argument and you do not give one." nil)
    (setq |$compileOnlyCertainItems| optargs)))

; compile a single constructor from a file
; )co dh )constructor DHMATRIX
(|constructor|
  (if (null optargs)
    (|throwKeyedMsg| ")constructor requires and argument and you do not give one." nil)
    (progn
      (setelt fun 0 '|c|)
      (setq constructor (mapcar #'|unabbrev| optargs)))))
(t
  (|throwKeyedMsg|
    "%1 is unknown or unavailable for the )compile command."
    (list (concatenate 'string ") " (string optname)))))
(setq |$InteractiveMode| nil)
(cond
  (|$compileOnlyCertainItems|
    (if (null constructor)
      (|sayKeyedMsg|
        (format nil "The )constructor option to )compile must also be ~
                    specified when the )functions option is used.")
        nil)
      (|compilerDoitWithScreenedLisplib| constructor fun)))
  (t (|compilerDoit| constructor fun)))
(|extendLocalLibdb| |$newConlist|)
(|terminateSystemCommand|)
(|spadPrompt|))))

```

defun compileSpadLispCmd

```

[compileSpadLispCmd pathname (vol5)]
[compileSpadLispCmd pathnameType (vol5)]
[compileSpadLispCmd selectOptionLC (vol5)]
[compileSpadLispCmd namestring (vol5)]

```

```

[compileSpadLispCmd terminateSystemCommand (vol5)]
[compileSpadLispCmd fnameMake (vol5)]
[compileSpadLispCmd pathnameDirectory (vol5)]
[compileSpadLispCmd pathnameName (vol5)]
[compileSpadLispCmd fnameReadable? (vol5)]
[compileSpadLispCmd localdatabase (vol5)]
[throwKeyedMsg p??]
[object2String p??]
[compileSpadLispCmd sayKeyedMsg (vol5)]
[recompile-lib-file-if-necessary p625]
[spadPrompt p??]
[$options p??]

```

— defun compileSpadLispCmd —

```

(defun |compileSpadLispCmd| (args)
  (let (path optlist optname optargs beQuiet dolibrary lsp)
    (declare (special |$options|))
    (setq path (|pathname| (|fnameMake| (car args) "code" "lsp")))
    (cond
      ((null (probe-file path))
        (|throwKeyedMsg| "The file %1 is needed but does not exist."
          (cons (|namestring| args) nil)))
      (t
        (setq optlist '(|quiet| |noquiet| |library| |nolibrary|))
        (setq beQuiet nil)
        (setq dolibrary t)
        (dolist (opt |$options|)
          (setq optname (car opt))
          (setq optargs (cdr opt))
          (case (|selectOptionLC| optname optlist nil)
            (|quiet| (setq beQuiet t))
            (|noquiet| (setq beQuiet nil))
            (|library| (setq dolibrary t))
            (|nolibrary| (setq dolibrary nil))
            (t
              (|throwKeyedMsg|
                "%1 is an unknown or unavailable for the )compile command."
                (list (strconc ")") (|object2String| optname))))))
        (setq lsp
          (|fnameMake|
            (|pathnameDirectory| path)
            (|pathnameName| path)
            (|pathnameType| path)))
        (cond
          ((|fnameReadable?| lsp)
            (unless beQuiet
              (|sayKeyedMsg| "Compiling Lisp source code from file %1"
                (list (|namestring| lsp))))

```

```

(recompile-lib-file-if-necessary lsp))
(t
 (|sayKeyedMsg| "The file %1 is needed but does not exist."
  (list (|namestring| lsp))))))
(cond
 (dolibrary
  (unless beQuiet (|sayKeyedMsg| "Issuing )library command for %1"
   (list (|pathnameName| path))))
 (localdatabase (list (|pathnameName| (car args))) nil))
 (null beQuiet)
 (|sayKeyedMsg|
  "The )library system command was not called after compilation." nil))
(t nil))
(|terminateSystemCommand|)
(|spadPrompt|))))))

```

compilerDoitWithScreenedLisplib

```

compilerDoitWithScreenedLisplib [embed p??]
[rewrite p??]
[compilerDoit p568]
[unembed p??]
[$saveableItems p??]
[$libFile p??]

```

— defun compilerDoitWithScreenedLisplib —

```

(defun |compilerDoitWithScreenedLisplib| (constructor fun)
 (declare (special |$saveableItems| |$libFile|))
 (embed 'rewrite
  '(lambda (key value stream)
    (cond
     ((and (eq stream |$libFile|)
      (not (member key |$saveableItems|)))
      value)
     ((not nil) (rewrite key value stream))))))
 (unwind-protect
  (|compilerDoit| constructor fun)
  (unembed 'rewrite)))

```

defun compilerDoit

This trivial function cases on the second argument to decide which combination of operations was requested. For this case we see:

```
(1) -> )co EQ
      Compiling AXIOM source code from file /tmp/EQ.spad using old system
      compiler.
1> (|compilerDoit| NIL (|rq| |lib|))
2> (|/RQ,LIB|)

... [snip]...

      <2 (|/RQ,LIB| T)
      <1 (|compilerDoit| T)
(1) ->
```

```
[compilerDoit /rq (vol5)]
[compilerDoit /rf (vol5)]
[compilerDoit member (vol5)]
[sayBrightly p??]
[opOf p??]
[/RQ,LIB p569]
[$byConstructors p626]
[$constructorsSeen p626]
```

— defun compilerDoit —

```
(defun |compilerDoit| (constructor fun)
  (let (|$byConstructors| |$constructorsSeen|)
    (declare (special |$byConstructors| |$constructorsSeen|))
    (cond
      ((equal fun '(|rf| |lib|)) (|/RQ,LIB|)) ; Ignore "noquiet"
      ((equal fun '(|rf| |nolib|)) (/rf))
      ((equal fun '(|rq| |lib|)) (|/RQ,LIB|))
      ((equal fun '(|rq| |nolib|)) (/rq))
      ((equal fun '(|c| |lib|))
        (setq |$byConstructors| (loop for x in constructor collect (|opOf| x)))
        (|/RQ,LIB|)
        (dolist (x |$byConstructors|)
          (unless (|member| x |$constructorsSeen|)
            (|sayBrightly| '(">>> Warning " ,x " was not found"))))))))
```

defun /rq

Compile with quiet output [rf-1 p570]
 [echo-meta p??]

— defun /rq —

```
(defun /rq (&rest foo &aux (echo-meta nil))
  (declare (special Echo-Meta) (ignore foo))
  (/rf-1 nil))
```

—————

defun /rf

Compile with noisy output [rf-1 p570]
 [echo-meta p??]

— defun /rf —

```
(defun /rf (&rest foo &aux (echo-meta t))
  (declare (special echo-meta) (ignore foo))
  (/rf-1 nil))
```

—————

defun /RQ,LIB

This function simply calls /rf-1.

```
(2) -> )co EQ
      Compiling AXIOM source code from file /tmp/EQ.spad using old system
      compiler.
1> (|compilerDoit| NIL (|rq| |lib|))
2> (|RQ,LIB|)
3> (/RF-1 NIL)
...[snip]...
<3 (/RF-1 T)
<2 (|RQ,LIB| T)
<1 (|compilerDoit| T)
```

[rf-1 p570]
 [/RQ,LIB echo-meta (vol5)]
 [\$lisplib p??]

— defun /RQ,LIB —

```
(defun |/RQ,LIB| (&rest foo &aux (echo-meta nil) ($lisplib t))
  (declare (special echo-meta $lisplib) (ignore foo))
  (/rf-1 nil))
```

defun /rf-1

Since this function is called with nil we fall directly into the call to the function **spad**:

```
(2) -> )co EQ
      Compiling AXIOM source code from file /tmp/EQ.spad using old system
      compiler.
1> (|compilerDoit| NIL (|rq| |lib|))
2> (|/RQ,LIB|)
3> (/RF-1 NIL)
4> (SPAD "/tmp/EQ.spad")
...[snip]...
<4 (SPAD T)
<3 (/RF-1 T)
<2 (|/RQ,LIB| T)
<1 (|compilerDoit| T)
```

```
[/rf-1 makeInputFilename (vol5)]
[ncINTERPFILE p624]
calls/rf-1spad [/editfile p??]
[echo-meta p??]
```

— defun /rf-1 —

```
(defun /rf-1 (ignore)
  (declare (ignore ignore))
  (let* ((input-file (makeInputFilename /editfile))
        (type (pathname-type input-file)))
    (declare (special echo-meta /editfile))
    (cond
      ((string= type "lisp") (load input-file))
      ((string= type "input") (|ncINTERPFILE| input-file echo-meta))
      (t (spad input-file)))))
```

defun spad

Here we begin the actual compilation process.

```

1> (SPAD "/tmp/EQ.spad")
2> (|makeInitialModemapFrame|)
<2 (|makeInitialModemapFrame| ((NIL)))
2> (INIT-BOOT/SPAD-READER)
<2 (INIT-BOOT/SPAD-READER NIL)
2> (OPEN "/tmp/EQ.spad" :DIRECTION :INPUT)
<2 (OPEN #<input stream "/tmp/EQ.spad">)
2> (INITIALIZE-PREPARSE #<input stream "/tmp/EQ.spad">)
<2 (INITIALIZE-PREPARSE ")abbrev domain EQ Equation")
2> (PREPARSE #<input stream "/tmp/EQ.spad">)
EQ abbreviates domain Equation
<2 (PREPARSE (# # # # # # # ...))
2> (|PARSE-NewExpr|)
<2 (|PARSE-NewExpr| T)
2> (S-PROCESS (|where| # #))
...[snip]...
3> (OPEN "/tmp/EQ.erlib/info" :DIRECTION :OUTPUT)
<3 (OPEN #<output stream "/tmp/EQ.erlib/info">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.lsp")
<3 (OPEN #<input stream "/tmp/EQ.nrllib/EQ.lsp">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.data" :DIRECTION :OUTPUT)
<3 (OPEN #<output stream "/tmp/EQ.nrllib/EQ.data">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.c" :DIRECTION :OUTPUT)
<3 (OPEN #<output stream "/tmp/EQ.nrllib/EQ.c">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.h" :DIRECTION :OUTPUT)
<3 (OPEN #<output stream "/tmp/EQ.nrllib/EQ.h">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.fn" :DIRECTION :OUTPUT)
<3 (OPEN #<output stream "/tmp/EQ.nrllib/EQ.fn">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.o" :DIRECTION :OUTPUT :IF-EXISTS :APPEND)
<3 (OPEN #<output stream "/tmp/EQ.nrllib/EQ.o">)
3> (OPEN #p"/tmp/EQ.nrllib/EQ.data")
<3 (OPEN #<input stream "/tmp/EQ.nrllib/EQ.data">)
3> (OPEN "/tmp/EQ.nrllib/index.kaf")
<3 (OPEN #<input stream "/tmp/EQ.nrllib/index.kaf">)
<2 (S-PROCESS NIL)
<1 (SPAD T)
1> (OPEN "temp.text" :DIRECTION :OUTPUT)
<1 (OPEN #<output stream "temp.text">)
1> (OPEN "libdb.text")
<1 (OPEN #<input stream "libdb.text">)
1> (OPEN "temp.text")
<1 (OPEN #<input stream "temp.text">)
1> (OPEN "libdb.text" :DIRECTION :OUTPUT)
<1 (OPEN #<output stream "libdb.text">)

```

The major steps in this process involve the **preparse** function. (See book volume 5 for more details). The **preparse** function returns a list of pairs of the form: ((linenumber . linestring) (linenumber . linestring)) For instance, for the file `EQ.spad`, we get:

```
<2 (PREPARSE (
(19 . "Equation(S: Type): public == private where")
(20 . " (Ex ==> OutputForm;")
(21 . "   public ==> Type with")
(22 . "   (\\"=\\": (S, S) -> $;")
...[skip]...
(202 . "           inv eq == [inv lhs eq, inv rhs eq]);")
(203 . "       if S has ExpressionSpace then")
(204 . "           subst(eq1,eq2) ==")
(205 . "               (eq3 := eq2 pretend Equation S;")
(206 . "               [subst(lhs eq1,eq3),subst(rhs eq1,eq3)])))))
```

```
[spad-reader p??]
[spad addBinding (vol5)]
[spad makeInitialModemapFrame (vol5)]
[spad init-boot/spad-reader (vol5)]
[initialize-preparse p82]
[preparse p86]
[PARSE-NewExpr p414]
[pop-stack-1 p548]
[s-process p574]
[ioclear p??]
[spad shut (vol5)]
[$noSubsumption p??]
[$InteractiveFrame p??]
[$InitialDomainsInScope p??]
[$InteractiveMode p??]
[$spad p571]
[$boot p??]
[curoutstream p??]
[*fileactq-apply* p??]
[line p631]
[optionlist p??]
[echo-meta p??]
[/editfile p??]
[*comp370-apply* p??]
[*eof* p??]
[file-closed p??]
[boot-line-stack p??]
[spad-reader p??]
```

— defun spad —


```

(defun spad (&optional (*spad-input-file* nil) (*spad-output-file* nil)
  &aux (*comp370-apply* #'print-defun)
        (*fileactq-apply* #'print-defun)
        ($spad t) ($boot nil) (optionlist nil) (*eof* nil)
        (file-closed nil) (/editfile *spad-input-file*)
        (|$noSubsumption| |$noSubsumption|) in-stream out-stream)
  (declare (special echo-meta /editfile *comp370-apply* *eof* curoutstream
    file-closed |$noSubsumption| |$InteractiveFrame|
    |$InteractiveMode| optionlist
    boot-line-stack *fileactq-apply* $spad $boot))
  ;; only rebind |$InteractiveFrame| if compiling
  (progv (if (not |$InteractiveMode|) '(|$InteractiveFrame|))
    (if (not |$InteractiveMode|)
      (list (|addBinding| '|$DomainsInScope|
        '((fluid . |true|))
        (|addBinding| '|$Information| nil
          (|makeInitialModemapFrame|))))))
  (init-boot/spad-reader)
  (unwind-protect
    (progn
      (setq in-stream (if *spad-input-file*
        (open *spad-input-file* :direction :input
          *standard-input*))
        (initialize-prepare in-stream)
        (setq out-stream (if *spad-output-file*
          (open *spad-output-file* :direction :output
            *standard-output*))
            (when *spad-output-file*
              (format out-stream "~&;; -- Mode:Lisp; Package:Boot  --~%~%")
              (print-package "BOOT"))
              (setq curoutstream out-stream)
              (loop
                (if (or *eof* file-closed) (return nil))
                (catch 'spad_reader
                  (if (setq boot-line-stack (prepare in-stream))
                    (let ((line (cdar boot-line-stack)))
                      (declare (special line))
                      (|PARSE-NewExpr|)
                      (let ((parseout (pop-stack-1)) )
                        (when parseout
                          (let ((*standard-output* out-stream))
                            (s-process parseout))
                            (format out-stream "~&")))))
                    )))
                  (ioclear in-stream out-stream)))
              (if *spad-input-file* (shut in-stream))
              (if *spad-output-file* (shut out-stream)))
    t))

```

defun Interpreter interface to the compiler

And the **s-process** function which returns a parsed version of the input.

```

2> (S-PROCESS
(|where|
(== (|:| (|Equation| (|:| S |Type|)) |public|) |private|)
(|;|
(|;|
(==> |Ex| |OutputForm|)
(==> |public|
(|Join| |Type|
(|with|
(CATEGORY
(|Signature| "=" (-> (|,| S S) $))
(|Signature| |equation| (-> (|,| S S) $))
(|Signature| |swap| (-> $ $))
(|Signature| |lhs| (-> $ S))
(|Signature| |rhs| (-> $ S))
(|Signature| |map| (-> (|,| (-> S S) $) $))
(|if| (|has| S (|InnerEvalable| (|,| |Symbol| S)))
(|Attribute| (|InnerEvalable| (|,| |Symbol| S)))
NIL)
(|if| (|has| S |SetCategory|)
(CATEGORY
(|Attribute| |SetCategory|)
(|Attribute| (|CoercibleTo| |Boolean|))
(|if| (|has| S (|Evalable| S))
(CATEGORY
(|Signature| |eval| (-> (|,| $ $) $))
(|Signature| |eval| (-> (|,| $ (|List| $)) $)))
NIL))
NIL)
(|if| (|has| S |AbelianSemiGroup|)
(CATEGORY
(|Attribute| |AbelianSemiGroup|)
(|Signature| "+" (-> (|,| S $) $))
(|Signature| "+" (-> (|,| $ S) $)))
NIL)
(|if| (|has| S |AbelianGroup|)
(CATEGORY
(|Attribute| |AbelianGroup|)
(|Signature| |leftZero| (-> $ $))
(|Signature| |rightZero| (-> $ $))
(|Signature| "-" (-> (|,| S $) $))
(|Signature| "-" (-> (|,| $ S) $))) NIL)
(|if| (|has| S |SemiGroup|)

```

```
(CATEGORY
  (|Attribute| |SemiGroup|)
  (|Signature| "*" (-> (|,| S $) $))
  (|Signature| "*" (-> (|,| $ S) $)))
NIL)
(if| (|has| S |Monoid|)
(CATEGORY
  (|Attribute| |Monoid|)
  (|Signature| |leftOne| (-> $ (|Union| (|,| $ "failed"))))
  (|Signature| |rightOne| (-> $ (|Union| (|,| $ "failed")))))
NIL)
(if| (|has| S |Group|)
(CATEGORY
  (|Attribute| |Group|)
  (|Signature| |leftOne| (-> $ (|Union| (|,| $ "failed"))))
  (|Signature| |rightOne| (-> $ (|Union| (|,| $ "failed")))))
NIL)
(if| (|has| S |Ring|)
(CATEGORY
  (|Attribute| |Ring|)
  (|Attribute| (|BiModule| (|,| S S))))
NIL)
(if| (|has| S |CommutativeRing|)
(|Attribute| (|Module| S))
NIL)
(if| (|has| S |IntegralDomain|)
(|Signature| |factorAndSplit| (-> $ (|List| $)))
NIL)
(if| (|has| S (|PartialDifferentialRing| |Symbol|))
(|Attribute| (|PartialDifferentialRing| |Symbol|))
NIL)
(if| (|has| S |Field|)
(CATEGORY
  (|Attribute| (|VectorSpace| S))
  (|Signature| "/" (-> (|,| $ $) $))
  (|Signature| |inv| (-> $ $)))
NIL)
(if| (|has| S |ExpressionSpace|)
(|Signature| |subst| (-> (|,| $ $) $))
NIL)
))))
(==> |private|
(|add|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
```

```

(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(|:=| |Rep|
(|Record| (|,| (|:| |lhs| S) (|:| |rhs| S))))
(|,| |eq1| (|:| |eq2| $)))
(|:| |s| S))
(|if| (|has| S |IntegralDomain|)
(==
(|factorAndSplit| |eq|)
(|;|
(=> (|has| S (|:| |factor| (-> S (|Factored| S))))
(|;|
(|:=| |eq0| (|rightZero| |eq|))
(COLLECT
(IN |rcf| (|factors| (|factor| (|lhs| |eq0|))))
(|construct|
(|equation| (|,| (|rcf| |factor|) 0))))))
(|construct| |eq|)))
NIL))
(==
(= (|:| |l| S) (|:| |r| S))
(|construct| (|,| |l| |r|))))
(==
(|equation| (|,| |l| |r|))
(|construct| (|,| |l| |r|))))
(== (|lhs| |eqn|) (|eqn| |lhs|)))
(== (|rhs| |eqn|) (|eqn| |rhs|)))
(==
(|swap| |eqn|)
(|construct| (|,| (|rhs| |eqn|) (|lhs| |eqn|))))))
(==
(|map| (|,| |fn| |eqn|))
(|equation|
(|,| (|fn| (|eqn| |lhs|)) (|fn| (|eqn| |rhs|))))))
(|if| (|has| S (|InnerEvalable| (|,| |Symbol| S)))
(|;|
(|;|

```

```

(|;|
(|;|
  (|:| (|:| |s| |Symbol|) (|:| |ls| (|List| |Symbol|)))
  (|:| |x| S))
(|:| |lx| (|List| S)))
(==
  (|eval| (|,| (|,| |eqn| |s|) |x|))
  (=
    (|eval| (|,| (|,| (|eqn| |lhs|) |s|) |x|))
    (|eval| (|,| (|,| (|eqn| |rhs|) |s|) |x|))))))
(==
  (|eval| (|,| (|,| |eqn| |ls|) |lx|))
  (=
    (|eval| (|,| (|,| (|eqn| |lhs|) |ls|) |lx|))
    (|eval| (|,| (|,| (|eqn| |rhs|) |ls|) |lx|))))))
NIL))
(|if| (|has| S (|Evalable| S))
(|;|
(==
  (|:| (|eval| (|,| (|:| |eqn1| $) (|:| |eqn2| $))) $)
  (=
    (|eval|
      (|,| (|eqn1| |lhs|) (|pretend| |eqn2| (|Equation| S))))
    (|eval|
      (|,| (|eqn1| |rhs|) (|pretend| |eqn2| (|Equation| S))))))
  (==
    (|:|
      (|eval| (|,| (|:| |eqn1| $) (|:| |eqn2| (|List| $)))) $)
    (=
      (|eval|
        (|,|
          (|eqn1| |lhs|)
          (|pretend| |eqn2| (|List| (|Equation| S))))
        (|eval|
          (|,|
            (|eqn1| |rhs|)
            (|pretend| |eqn2| (|List| (|Equation| S))))))
      NIL))
(|if| (|has| S |SetCategory|)
(|;|
(|;|
(==
  (= |eq1| |eq2|)
  (|and|
    (@ (= (|eq1| |lhs|) (|eq2| |lhs|)) |Boolean|)
    (@ (= (|eq1| |rhs|) (|eq2| |rhs|)) |Boolean|)))
  (==
    (|:| (|coerce| (|:| |eqn| $)) |Ex|)
    (= (|::| (|eqn| |lhs|) |Ex|) (|::| (|eqn| |rhs|) |Ex|))))
  (==

```

```

(|:| (|coerce| (|:| |eqn| $)) |Boolean|)
(= (|eqn| |lhs|) (|eqn| |rhs|)))
NIL))
(|if| (|has| S |AbelianSemiGroup|)
(|;|
(|;|
(|;|
(==
(+ |eq1| |eq2|)
(=
(+ (|eq1| |lhs|) (|eq2| |lhs|))
(+ (|eq1| |rhs|) (|eq2| |rhs|))))
(== (+ |s| |eq2|) (+ (|construct| (|,| |s| |s|)) |eq2|)))
(== (+ |eq1| |s|) (+ |eq1| (|construct| (|,| |s| |s|)))))
NIL))
(|if| (|has| S |AbelianGroup|)
(|;|
(|;|
(|;|
(|;|
(|;|
(|;|
(== (- |eq1|) (= (- (|lhs| |eq1|)) (- (|rhs| |eq1|))))
(== (- |s| |eq2|) (- (|construct| (|,| |s| |s|)) |eq2|)))
(== (- |eq1| |s|) (- |eq1| (|construct| (|,| |s| |s|))))
(== (|leftZero| |eq1|) (= 0 (- (|rhs| |eq1|) (|lhs| |eq1|))))
(== (|rightZero| |eq1|) (= (- (|lhs| |eq1|) (|rhs| |eq1|)) 0)))
(== 0 (|equation| (|,| (|elt| S 0) (|elt| S 0)))))
(==
(- |eq1| |eq2|)
(=
(- (|eq1| |lhs|) (|eq2| |lhs|))
(- (|eq1| |rhs|) (|eq2| |rhs|))))
NIL))
(|if| (|has| S |SemiGroup|)
(|;|
(|;|
(|;|
(==
(* (|:| |eq1| $) (|:| |eq2| $))
(=
(* (|eq1| |lhs|) (|eq2| |lhs|))
(* (|eq1| |rhs|) (|eq2| |rhs|))))
(==
(* (|:| |1| S) (|:| |eqn| $))
(= (* |1| (|eqn| |lhs|)) (* |1| (|eqn| |rhs|)))))
(==
(* (|:| |1| S) (|:| |eqn| $))
(= (* |1| (|eqn| |lhs|)) (* |1| (|eqn| |rhs|)))))
(==
(* (|:| |eqn| $) (|:| |1| S))

```

```

(= (* (|eqn| |lhs| |l|) (* (|eqn| |rhs| |l|))))
NIL))
(|if| (|has| S |Monoid|)
(|;|
(|;|
(|;|
(== 1 (|equation| (|,| (|elt| S 1) (|elt| S 1))))
(==
(|recip| |eq|)
(|;|
(|;|
(=> (|case| (|:=| |lh| (|recip| (|lhs| |eq|))) "failed")
"failed")
(=> (|case| (|:=| |rh| (|recip| (|rhs| |eq|))) "failed")
"failed"))
(|construct| (|,| (|::| |lh| S) (|::| |rh| S))))))
(==
(|leftOne| |eq|)
(|;|
(=> (|case| (|:=| |re| (|recip| (|lhs| |eq|))) "failed")
"failed")
(= 1 (* (|rhs| |eq|) |re|))))
(==
(|rightOne| |eq|)
(|;|
(=> (|case| (|:=| |re| (|recip| (|rhs| |eq|))) "failed")
"failed")
(= (* (|lhs| |eq|) |re| 1))))
NIL))
(|if| (|has| S |Group|)
(|;|
(|;|
(==
(|inv| |eq|)
(|construct| (|,| (|inv| (|lhs| |eq|)) (|inv| (|rhs| |eq|))))
(== (|leftOne| |eq|) (= 1 (* (|rhs| |eq|) (|inv| (|rhs| |eq|)))))
(== (|rightOne| |eq|) (= (* (|lhs| |eq|) (|inv| (|rhs| |eq|)) 1)))
NIL))
(|if| (|has| S |Ring|)
(|;|
(==
(|characteristic| (|@Tuple|))
(|elt| S |characteristic|) (|@Tuple|))
(== (* (|:| |i| |Integer|) (|:| |eq| $)) (* (|::| |i| S) |eq|)))
NIL))
(|if| (|has| S |IntegralDomain|)
(==
(|factorAndSplit| |eq|)
(|;|
(|;|

```

```

(=>
  (|has| S (|:| |factor| (-> S (|Factored| S))))
  (|;|
    (|:=| |eq0| (|rightZero| |eq|))
    (COLLECT
      (IN |rcf| (|factors| (|factor| (|lhs| |eq0|))))
      (|construct| (|equation| (|,| (|rcf| |factor| 0))))))
(=>
  (|has| S (|Polynomial| |Integer|))
  (|;|
    (|;|
      (|:=| |eq0| (|rightZero| |eq|))
      (==> MF
        (|MultivariateFactorize|
          (|,|
            (|,| (|,| |Symbol| (|IndexedExponents| |Symbol|)) |Integer|
              (|Polynomial| |Integer|))))
          (|:=|
            (|:| |p| (|Polynomial| |Integer|))
            (|pretend| (|lhs| |eq0|) (|Polynomial| |Integer|))))
        (COLLECT
          (IN |rcf| (|factors| ((|elt| MF |factor|) |p|)))
          (|construct|
            (|equation| (|,| (|pretend| (|rcf| |factor|) S) 0))))))
      (|construct| |eq|)))
  NIL))
(|if| (|has| S (|PartialDifferentialRing| |Symbol|))
  (==
    (|:| (|differentiate| (|,| (|:| |eq| $) (|:| |sym| |Symbol|))) $)
    (|construct|
      (|,|
        (|differentiate| (|,| (|lhs| |eq|) |sym|))
        (|differentiate| (|,| (|rhs| |eq|) |sym|))))
      NIL))
  (|if| (|has| S |Field|)
    (|;|
      (|;|
        (== (|dimension| (|@Tuple|)) (|::| 2 |CardinalNumber|))
        (==
          (/ (|:| |eq1| $) (|:| |eq2| $))
          (= (/ (|eq1| |lhs|) (|eq2| |lhs|)) (/ (|eq1| |rhs|) (|eq2| |rhs|))))
          (==
            (|inv| |eq|)
            (|construct| (|,| (|inv| (|lhs| |eq|)) (|inv| (|rhs| |eq|))))))
          NIL))
    (|if| (|has| S |ExpressionSpace|)
      (==
        (|subst| (|,| |eq1| |eq2|))
        (|;|

```



```

(|:=| |eq3| (|pretend| |eq2| (|Equation| S)))
(|construct|
  (|,|
    (|subst| (|,| (|lhs| |eq1|) |eq3|))
    (|subst| (|,| (|rhs| |eq1|) |eq3|))))))
NIL))))))

```

```

[curstrm p??]
[def-rename p585]
[new2OldLisp p78]
[parseTransform p101]
[postTransform p369]
[displayPreCompilationErrors p542]
[prettyprint p??]
[s-process processInteractive (vol5)]
[compTopLevel p583]
[def-process p??]
[displaySemanticErrors p??]
[terpri p??]
[get-internal-run-time p??]
[$Index p??]
[$macroassoc p??]
[$newspad p??]
[$PolyMode p??]
[$EmptyMode p176]
[$compUniquelyIfTrue p??]
[$currentFunction p??]
[$postStack p??]
[$topOp p??]
[$semanticErrorStack p??]
[$warningStack p??]
[$exitMode p??]
[$exitModeStack p??]
[$returnMode p??]
[$leaveMode p??]
[$leaveLevelStack p??]
[$top-level p??]
[$insideFunctorIfTrue p??]
[$insideExpressionIfTrue p??]
[$insideCoerceInteractiveHardIfTrue p??]
[$insideWhereIfTrue p??]
[$insideCategoryIfTrue p??]
[$insideCapsuleFunctionIfTrue p??]
[$form p??]
[$DomainFrame p??]

```

```

[$e p??]
[$EmptyEnvironment p??]
[$genFVar p??]
[$genSDVar p??]
[$VariableCount p??]
[$previousTime p??]
[$LocalFrame p??]
[$Translation p??]
[$TranslateOnly p??]
[$PrintOnly p??]
[$currentLine p??]
[$InteractiveFrame p??]
[curoutstream p??]

```

— defun s-process —

```

(defun s-process (x)
  (prog ((|$Index| 0)
        ($macroassoc ())
        ($newspad t)
        (|$PolyMode| |$EmptyMode|)
        (|$compUniquelyIfTrue| nil)
        |$currentFunction|
        (|$postStack| nil)
        |$topOp|
        (|$semanticErrorStack| ())
        (|$warningStack| ())
        (|$exitMode| |$EmptyMode|)
        (|$exitModeStack| ())
        (|$returnMode| |$EmptyMode|)
        (|$leaveMode| |$EmptyMode|)
        (|$leaveLevelStack| ())
        $top_level |$insideFunctorIfTrue| |$insideExpressionIfTrue|
        |$insideCoerceInteractiveHardIfTrue| |$insideWhereIfTrue|
        |$insideCategoryIfTrue| |$insideCapsuleFunctionIfTrue| |$form|
        (|$DomainFrame| '((NIL)))
        (|$e| |$EmptyEnvironment|)
        (|$genFVar| 0)
        (|$genSDVar| 0)
        (|$VariableCount| 0)
        (|$previousTime| (get-internal-run-time))
        (|$LocalFrame| '((NIL)))
        (curstrm curoutstream) |$s| |$x| |$m| u)
    (declare (special |$Index| $macroassoc $newspad |$PolyMode| |$EmptyMode|
                     |$compUniquelyIfTrue| |$currentFunction| |$postStack| |$topOp|
                     |$semanticErrorStack| |$warningStack| |$exitMode| |$exitModeStack|
                     |$returnMode| |$leaveMode| |$leaveLevelStack| $top_level
                     |$insideFunctorIfTrue| |$insideExpressionIfTrue|
                     |$insideCoerceInteractiveHardIfTrue| |$insideWhereIfTrue|

```

```

    |$insideCategoryIfTrue| |$insideCapsuleFunctionIfTrue| |$form| | | | |
    |$DomainFrame| |$e| |$EmptyEnvironment| |$genFVar| |$genSDVar|
    |$VariableCount| |$previousTime| |$LocalFrame|
    curstrm |$s| |$x| |$m| curoutstream $traceflag |$Translation|
    |$TranslateOnly| |$PrintOnly| |$currentLine| |$InteractiveFrame|))
(setq $traceflag t)
(if (not x) (return nil))
(if $boot
    (setq x (def-rename (new2OldLisp x)))
    (setq x (|parseTransform| (postTransform x))))
(when |$TranslateOnly| (return (setq |$Translation| x)))
(when |$postStack| (|displayPreCompilationErrors|) (return nil))
(when |$PrintOnly|
    (format t "~S  =====>~%" |$currentLine|)
    (return (prettyprint x)))
(if (not $boot)
    (if |$InteractiveModel|
        (|processInteractive| x nil)
        (when (setq u (|compTopLevel| x |$EmptyMode| |$InteractiveFrame|))
            (setq |$InteractiveFrame| (third u))))
    (def-process x))
(when |$semanticErrorStack| (|displaySemanticErrors|))
(terpri)))

```

defun compTopLevel

```

[compOrCroak p586]
[$NRTderivedTargetIfTrue p??]
[$killOptimizeIfTrue p??]
[$forceAdd p??]
[$compTimeSum p??]
[$resolveTimeSum p??]
[$packagesUsed p??]
[$envHashTable p??]

```

— defun compTopLevel —

```

(defun |compTopLevel| (form mode env)
  (let (|$NRTderivedTargetIfTrue| |$killOptimizeIfTrue| |$forceAdd|
        |$compTimeSum| |$resolveTimeSum| |$packagesUsed| |$envHashTable|
        t1 t2 t3 val newmode)
    (declare (special |$NRTderivedTargetIfTrue| |$killOptimizeIfTrue|
                      |$forceAdd| |$compTimeSum| |$resolveTimeSum|
                      |$packagesUsed| |$envHashTable| ))
    (setq |$NRTderivedTargetIfTrue| nil)

```

```

(setq |$killOptimizeIfTrue| nil)
(setq |$forceAdd| nil)
(setq |$compTimeSum| 0)
(setq |$resolveTimeSum| 0)
(setq |$packagesUsed| NIL)
(setq |$envHashTable| (make-hashtable 'equal))
(dolist (u (car (car env)))
  (dolist (v (cdr u))
    (hput |$envHashTable| (cons (car u) (cons (car v) nil)) t)))
(cond
  ((or (and (consp form) (eq (qfirst form) 'def))
       (and (consp form) (eq (qfirst form) '|where|))
       (progn
         (setq t1 (qrest form))
         (and (consp t1)
              (progn
                (setq t2 (qfirst t1))
                (and (consp t2) (eq (qfirst t2) 'def)))))))
   (setq t3 (|compOrCroak| form mode env))
   (setq val (car t3))
   (setq newmode (second t3))
   (cons val (cons newmode (cons env nil))))
(t (|compOrCroak| form mode env))))

```

defun print-defun

```

[is-console p553]
[print-full p??]
[vmlisp::optionlist p??]
[$PrettyPrint p??]

```

— defun print-defun —

```

(defun print-defun (name body)
  (let* ((sp (assoc 'vmlisp::compiler-output-stream vmlisp::optionlist))
         (st (if sp (cdr sp) *standard-output*)))
    (declare (special vmlisp::optionlist |$PrettyPrint|))
    (when (and (is-console st) (symbolp name) (fboundp name)
               (not (compiled-function-p (symbol-function name)))))
      (compile name))
    (when (or |$PrettyPrint| (not (is-console st)))
      (print-full body st) (force-output st))))

```

defun def-rename

[def-rename p585]

— defun def-rename —

```
(defun def-rename (x)
  (cond
    ((symbolp x)
     (let ((y (get x 'rename))) (if y (first y) x)))
    ((and (listp x) x)
     (if (eqcar x 'quote)
         x
         (cons (def-rename (first x)) (def-rename (cdr x))))))
  (x)))
```

Given:

```
CohenCategory(): Category == SetCategory with
```

```
kind:(CExpr)->Boolean
operand:(CExpr,Integer)->CExpr
numberOfOperand:(CExpr)->Integer
construct:(CExpr,CExpr)->CExpr
```

the resulting call looks like:

```
(|compOrCroak|
  (DEF (|CohenCategory|)
    ((|Category|)
      (NIL)
      (|Join|
        (|SetCategory|
          (CATEGORY |package|
            (SIGNATURE |kind| ((|Boolean|) |CExpr|))
            (SIGNATURE |operand| (|CExpr| |CExpr| (|Integer|)))
            (SIGNATURE |numberOfOperand| ((|Integer|) |CExpr|))
            (SIGNATURE |construct| (|CExpr| |CExpr| |CExpr|))))))
    |$EmptyMode|
    (((
      (|$DomainsInScope|
        (FLUID . |true|)
        (special |$EmptyMode| |$NoValueMode|)))))))
```

This compiler call expects the first argument *x* to be a DEF form to compile, The second argument, *m*, is the mode. The third argument, *e*, is the environment.

defun compOrCroak

[compOrCroak1 p586]

— defun compOrCroak —

```
(defun |compOrCroak| (form mode env)
  (|compOrCroak1| form mode env nil nil))
```

—————

This results in a call to the inner function with

```
(|compOrCroak1|
  (DEF (|CohenCategory|)
    ((|Category|))
    (NIL)
    (|Join|
      (|SetCategory|)
      (CATEGORY |package|
        (SIGNATURE |kind| ((|Boolean|) |CEExpr|))
        (SIGNATURE |operand| (|CEExpr| |CEExpr| (|Integer|)))
        (SIGNATURE |numberOfOperand| ((|Integer|) |CEExpr|))
        (SIGNATURE |construct| (|CEExpr| |CEExpr| |CEExpr|))))
    |$EmptyMode|
    (((
      |$DomainsInScope|
      (FLUID . |true|)
      (special |$EmptyMode| |$NoValueMode|))))))
  NIL
  NIL
  |comp|)
```

The inner function augments the environment with information from the compiler stack `$compStack` and `$compErrorMessageStack`. Note that these variables are passed in the argument list so they get preserved on the call stack. The calling function gets called for every inner form so we use this implicit stacking to retain the information.

defun compOrCroak1

[comp p587]

[compOrCroak1,compactify p624]

[stackSemanticError p??]

[mkErrorExpr p??]

[displaySemanticErrors p??]

[say p??]

[displayComp p??]

```
[userError p??]
[$compStack p??]
[$compErrorMessageStack p??]
[$level p??]
[$s p??]
[$scanIfTrue p??]
[$exitModeStack p??]
[compOrCroak p586]
```

— **defun compOrCroak1** —

```
(defun |compOrCroak1| (form mode env |$compStack| |$compErrorMessageStack|)
  (declare (special |$compStack| |$compErrorMessageStack|))
  (let (td errorMessage)
    (declare (special |$level| |$s| |$scanIfTrue| |$exitModeStack|))
    (cond
      ((setq td (catch '|compOrCroak| (|comp| form mode env))) td)
      (t
       (setq |$compStack|
              (cons (list form mode env |$exitModeStack|) |$compStack|))
       (setq |$s| (|compOrCroak1,compactify| |$compStack|))
       (setq |$level| (|#| |$s|))
       (setq errorMessage
              (if |$compErrorMessageStack|
                  (car |$compErrorMessageStack|)
                  '|unspecified error|))
       (cond
          (|$scanIfTrue|
           (|stackSemanticError| errorMessage (|mkErrorExpr| |$level|))
           (list '|failedCompilation| mode env ))
          (t
           (|displaySemanticErrors|)
           (say "***** comp fails at level " |$level| " with expression: *****")
           (|displayComp| |$level|)
           (|userError| errorMessage))))))
```

—————

defun comp

```
[compNoStacking p588]
[$compStack p??]
[$exitModeStack p??]
```

— **defun comp** —

```
(defun |comp| (form mode env)
```

```
(let (td)
  (declare (special |$compStack| |$exitModeStack|))
  (if (setq td (|compNoStacking| form mode env))
      (setq |$compStack| nil)
      (push (list form mode env |$exitModeStack|) |$compStack|))
  td))
```

defun compNoStacking

\$Representation is bound in `compDefineFunctor`, set by `doIt`. This hack says that when something is undeclared, `$` is preferred to the underlying representation – RDJ 9/12/83
[\[comp2 p589\]](#)
[\[compNoStacking1 p588\]](#)
[\[\\$compStack p??\]](#)
[\[\\$Representation p??\]](#)
[\[\\$EmptyMode p176\]](#)

— defun compNoStacking —

```
(defun |compNoStacking| (form mode env)
  (let (td)
    (declare (special |$compStack| |$Representation| |$EmptyMode|))
    (if (setq td (|comp2| form mode env))
        (if (and (equal mode |$EmptyMode|) (equal (second td) |$Representation|))
            (list (car td) '$ (third td))
            td)
        (|compNoStacking1| form mode env |$compStack|))))
```

defun compNoStacking1

[\[get p??\]](#)
[\[comp2 p589\]](#)
[\[\\$compStack p??\]](#)

— defun compNoStacking1 —

```
(defun |compNoStacking1| (form mode env |$compStack|)
  (declare (special |$compStack|))
  (let (u td)
    (if (setq u (|get| (if (eq mode '$) '|Rep| mode) '|value| env))
```



```
(if (setq td (|comp2| form (car u) env))
    (list (car td) mode (third td))
    nil)
nil)))
```

defun comp2

```
[comp3 p589]
[isDomainForm p348]
[isFunctor p253]
[insert p??]
[opOf p??]
[addDomain p252]
[$bootStrapMode p??]
[$packagesUsed p??]
[$lisplib p??]
```

— defun comp2 —

```
(defun |comp2| (form mode env)
  (let (tmp1)
    (declare (special |$bootStrapMode| |$packagesUsed| $lisplib))
    (when (setq tmp1 (|comp3| form mode env))
      (destructuring-bind (y mprime env) tmp1
        (when (and $lisplib (|isDomainForm| form env) (|isFunctor| form))
          (setq |$packagesUsed| (|insert| (list (|opOf| form)) |$packagesUsed|)))
          ; isDomainForm test needed to prevent error while compiling Ring
          ; $bootStrapMode-test necessary for compiling Ring in $bootStrapMode
          (if (and (not (equal mode mprime))
                  (or |$bootStrapMode| (|isDomainForm| mprime env)))
              (list y mprime (|addDomain| mprime env))
              (list y mprime env)))))))
```

defun comp3

```
[addDomain p252]
[compWithMappingMode p614]
[compAtom p595]
[getmode p??]
[applyMapping p591]
```

```

[compApply p592]
[compColon p294]
[compCoerce p362]
[stringPrefix? p??]
[comp3 pname (vol5)]
[compTypeOf p593]
[compExpression p139]
[comp3 member (vol5)]
[getDomainsInScope p254]
[$e p??]
[$insideCompTypeOf p??]

```

— defun comp3 —

```

(defun |comp3| (form mode |$e|)
  (declare (special |$e|))
  (let (env op ml u tt tmp1)
    (declare (special |$insideCompTypeOf|))
    (setq |$e| (|addDomain| mode |$e|))
    (setq env |$e|)
    (cond
      ((and (consp mode) (eq (qfirst mode) '|Mapping|))
        (|compWithMappingMode| form mode env))
      ((and (consp mode) (eq (qfirst mode) 'quote)
        (consp (qcdr mode)) (eq (qcddr mode) nil))
        (when (equal form (qcadr mode)) (list form mode |$e|)))
      ((stringp mode)
        (when (and (atom form)
          (or (equal mode form) (equal mode (princ-to-string form))))
          (list mode mode env )))
      ((or (null form) (atom form)) (|compAtom| form mode env))
      (t
        (setq op (car form))
        (cond
          ((and (progn
            (setq tmp1 (|getmode| op env))
            (and (consp tmp1)
              (eq (qfirst tmp1) '|Mapping|)
              (progn (setq ml (qrest tmp1)) t)))
            (setq u (|applyMapping| form mode env ml)))
            u)
          ((and (consp op) (eq (qfirst op) 'kappa)
            (consp (qcdr op)) (consp (qcddr op))
            (consp (qcdddr op)) (eq (qcdddr op) nil))
            (|compApply| (qcadr op) (qcaddr op) (qcaddr op) (cdr form) mode env))
          ((eq op '|:|) (|compColon| form mode env))
          ((eq op '|::|) (|compCoerce| form mode env))
          ((and (null (eq |$insideCompTypeOf| t))
            (|stringPrefix?| "TypeOf" (pname op)))
            (|stringPrefix?| "TypeOf" (pname op)))

```

```
(|compTypeOf| form mode env))
(t
 (setq tt (|compExpression| form mode env))
 (cond
  ((and (consp tt) (consp (qcdr tt)) (consp (qcddr tt))
        (eq (qcdddr tt) nil)
        (null (|member| (qcadr tt) (|getDomainsInScope| (qcaddr tt)))))
   (list (qcar tt) (qcadr tt) (|addDomain| (qcadr tt) (qcaddr tt)))
   (t tt))))))
```

defun applyMapping

```
[isCategoryForm p??]
[sublis p??]
[comp p587]
[convert p597]
[member p??]
[get p??]
[getAbbreviation p302]
[encodeItem p185]
[$FormalMapVariableList p270]
[$form p??]
[$op p??]
[$prefix p??]
[$formalArgList p??]
```

— defun applyMapping —

```
(defun |applyMapping| (t0 m e ml)
  (prog (op argl mlp temp1 arglp nprefix opp form pairlis)
    (declare (special |$FormalMapVariableList| |$form| |$op| |$prefix|
                      |$formalArgList|))
    (return
     (progn
      (setq op (car t0))
      (setq argl (cdr t0))
      (cond
       ((not (eql (|#| argl) (1- (|#| ml)))) nil)
       ((|isCategoryForm| (car ml) e)
        (setq pairlis
         (loop for a in argl for v in |$FormalMapVariableList|
              collect (cons v a)))
        (setq mlp (sublis pairlis ml))
        (setq arglp
```

```

(loop for x in argl for mp in (rest mlp)
  collect (car
    (progn
      (setq temp1 (or (|comp| x mp e) (return '|failed|)))
      (setq e (caddr temp1))
      temp1))))
(when (eq arglp '|failed|) (return nil))
(setq form (cons op arglp))
(|convert| (list form (car mlp) e) m))
(t
  (setq arglp
    (loop for x in argl for mp in (rest ml)
      collect (car
        (progn
          (setq temp1 (or (|comp| x mp e) (return '|failed|)))
          (setq e (caddr temp1))
          temp1))))
    (when (eq arglp '|failed|) (return nil))
    (setq form
      (cond
        ((and (null (|member| op |$formalArgList|))
              (atom op)
              (null (|get| op '|value| e))))
          (setq nprefix
            (or |$prefix| (|getAbbreviation| |$op| (|#| (cdr |$form|)))))
          (setq opp
            (intern (strconc
              (|encodeItem| nprefix) '|;'| (|encodeItem| op))))
          (cons opp (append arglp (list '$))))
        (t
          (cons '|call| (cons (list '|applyFun| op) arglp)))))
    (setq pairlis
      (loop for a in arglp for v in |$FormalMapVariableList|
        collect (cons v a)))
    (|convert| (list form (sublis pairlis (car ml)) e) m))))))

```

defun compApply

```

[comp p587]
[Pair p??]
[removeEnv p??]
[resolve p365]
[AddContour p??]
[$EmptyMode p176]

```

— defun compApply —

```
(defun |compApply| (sig varl body argl m e)
  (let (temp1 argtl contour code mq bodyq)
    (declare (special |$EmptyMode|))
    (setq argtl
      (loop for x in argl
        collect (progn
          (setq temp1 (|comp| x |$EmptyMode| e))
          (setq e (caddr temp1))
          temp1)))
    (setq contour
      (loop for x in varl
        for mq in (cdr sig)
        for a in argl
        collect
          (|Pair| x
            (list
              (list '|mode| mq)
              (list '|value| (|removeEnv| (|comp| a mq e)))))))
    (setq code
      (cons (list '|lambda| varl bodyq)
        (loop for tt in argtl
          collect (car tt))))
    (setq mq (|resolve| m (car sig)))
    (setq bodyq (car (|comp| body mq (|addContour| contour e))))
    (list code mq e)))
```

defun compTypeOf

```
[eqsubstlist p??]
[get p??]
[put p??]
[comp3 p589]
[$insideCompTypeOf p??]
[$FormalMapVariableList p270]
```

— defun compTypeOf —

```
(defun |compTypeOf| (form mode env)
  (let (|$insideCompTypeOf| op argl newModemap)
    (declare (special |$insideCompTypeOf| |$FormalMapVariableList|))
    (setq op (car form))
    (setq argl (cdr form))
    (setq |$insideCompTypeOf| t)
```

```
(setq newModemap
  (eqsubstlist argl |$FormalMapVariableList| (|get| op '|modemap| env)))
(setq env (|put| op '|modemap| newModemap env))
(|comp3| form mode env)))
```

defun compColonInside

```
[addDomain p252]
[comp p587]
[coerce p355]
[stackWarning p??]
[opOf p??]
[stackSemanticError p??]
[$newCompilerUnionFlag p??]
[$EmptyMode p176]
```

— defun compColonInside —

```
(defun |compColonInside| (form mode env mprime)
  (let (mpp warningMessage td tprime)
    (declare (special |$newCompilerUnionFlag| |$EmptyMode|))
    (setq env (|addDomain| mprime env))
    (when (setq td (|comp| form |$EmptyMode| env))
      (cond
        ((equal (setq mpp (second td)) mprime)
          (setq warningMessage
            (list '|:| mprime '| -- should replace by @|))))
        (setq td (list (car td) mprime (third td)))
        (when (setq tprime (|coerce| td mode))
          (cond
            (warningMessage (|stackWarning| warningMessage))
            ((and |$newCompilerUnionFlag| (eq (|opOf| mpp) '|Union|))
              (setq tprime
                (|stackSemanticError|
                  (list '|cannot pretend | form '| of mode | mpp '| to mode | mprime )
                    nil))))
            (t
              (|stackWarning|
                (list '|:| mprime '| -- should replace by pretend|))))
            tprime))))))
```

defun compAtom

```
[compAtomWithModemap p596]
[get p??]
[modelsAggregateOf p??]
[compList p599]
[compVector p353]
[convert p597]
[isSymbol p??]
[compSymbol p598]
[primitiveType p597]
[primitiveType p597]
[$Expression p??]
```

— **defun compAtom** —

```
(defun |compAtom| (form mode env)
  (prog (tmp1 tmp2 r td tt)
    (declare (special |$Expression|))
    (return
      (cond
        ((setq td
          (|compAtomWithModemap| form mode env (|get| form '|modemap| env))) td)
        ((eq form '|nil|)
          (setq td
            (cond
              ((progn
                (setq tmp1 (|modeIsAggregateOf| '|List| mode env))
                (and (consp tmp1)
                  (progn
                    (setq tmp2 (qrest tmp1))
                    (and (consp tmp2)
                      (eq (qrest tmp2) nil)
                      (progn
                        (setq r (qfirst tmp2)) t))))))
                (|compList| form (list '|List| r) env))
              ((progn
                (setq tmp1 (|modeIsAggregateOf| '|Vector| mode env))
                (and (consp tmp1)
                  (progn
                    (setq tmp2 (qrest tmp1))
                    (and (consp tmp2) (eq (qrest tmp2) nil)
                      (progn
                        (setq r (qfirst tmp2)) t))))))
                (|compVector| form (list '|Vector| r) env))))
          (when td (|convert| td mode)))
        (t
          (setq tt
            (cond
```

```

(|isSymbol| form) (or (|compSymbol| form mode env) (return nil)))
((and (equal mode |$Expression|)
      (|primitiveType| form)) (list form mode env ))
((stringp form) (list form form env ))
(t (list form (or (|primitiveType| form) (return nil)) env ))))
(|convert| tt mode))))))

```

defun compAtomWithModemap

[transImplementation p597]

[modeEqual p366]

[convert p597]

[\$NoValueMode p176]

— defun compAtomWithModemap —

```

(defun |compAtomWithModemap| (x m env v)
  (let (tt transimp y)
    (declare (special |$NoValueMode|))
    (cond
      ((setq transimp
        (loop for map in v
          when ; map is [[.,target],[.,fn]]
            (and (consp map) (consp (qcar map)) (consp (qcdr map))
                  (eq (qcddr map) nil)
                  (consp (qcdr map)) (eq (qcddr map) nil)
                  (consp (qcadr map)) (consp (qcddadr map))
                  (eq (qcddadr map) nil)))
        collect
          (list (|transImplementation| x map (qcadr map)) (qcadr map) env)))
      (cond
        ((setq tt
          (let (result)
            (loop for item in transimp
              when (|modeEqual| m (cadr item))
                do (setq result (or result item)))
            result))
          tt)
        ((eq 1 (|#| (setq transimp
          (loop for ta in transimp
            when (setq y (|convert| ta m))
              collect y))))
          (car transimp)))
      ((and (< 0 (|#| transimp)) (equal m |$NoValueMode|))
        (car transimp))
    )
  )

```



```
(t nil))))))
```

defun transImplementation

```
[genDeltaEntry p??]
```

— defun transImplementation —

```
(defun |transImplementation| (op map fn)
  (setq fn (|genDeltaEntry| (cons op map)))
  (if (and (consp fn) (eq (qcar fn) 'xlam))
      (cons fn nil)
      (cons '|call| (cons fn nil))))
```

defun convert

```
[resolve p365]
[coerce p355]
```

— defun convert —

```
(defun |convert| (td mode)
  (let (res)
    (when (setq res (|resolve| (second td) mode))
      (|coerce| td res))))
```

defun primitiveType

```
[$DoubleFloat p??]
[$NegativeInteger p??]
[$PositiveInteger p??]
[$NonNegativeInteger p??]
[$String p349]
[$EmptyMode p176]
```

— defun primitiveType —

```
(defun |primitiveType| (form)
  (declare (special |$DoubleFloat| |$NegativeInteger| |$PositiveInteger|
                  |$NonNegativeInteger| |$String| |$EmptyMode|))
  (cond
    ((null form) |$EmptyMode|)
    ((stringp form) |$String|)
    ((integerp form)
     (cond
       ((eql form 0) |$NonNegativeInteger|)
       ((> form 0) |$PositiveInteger|)
       (t |$NegativeInteger|)))
    ((floatp form) |$DoubleFloat|)
    (t nil)))
```

defun compSymbol

```
[isFluid p??]
[getmode p??]
[get p??]
[NRTgetLocalIndex p215]
[compSymbol member (vol5)]
[isFunction p??]
[errorRef p??]
[stackMessage p??]
[$Symbol p??]
[$Expression p??]
[$FormalMapVariableList p270]
[$compForModeIfTrue p??]
[$formalArgList p??]
[$NoValueMode p176]
[$functorLocalParameters p??]
[$Boolean p??]
[$NoValue p??]
```

— defun compSymbol —

```
(defun |compSymbol| (form mode env)
  (let (v mprime newmode)
    (declare (special |$Symbol| |$Expression| |$FormalMapVariableList|
                      |$compForModeIfTrue| |$formalArgList| |$NoValueMode|
                      |$functorLocalParameters| |$Boolean| |$NoValue|))
    (cond
      ((eq form '|$NoValue|) (list '|$NoValue| |$NoValueMode| env ))
      ((|isFluid| form)
```

```

(setq newmode (|getmode| form env))
(when newmode (list form (|getmode| form env) env)))
((eq form '|true|) (list '(quote t) |$Boolean| env ))
((eq form '|false|) (list nil |$Boolean| env ))
((or (equal form mode)
      (|get| form '|isLiteral| env)) (list (list 'quote form) form env))
((setq v (|get| form '|value| env))
 (cond
  ((member form |$functorLocalParameters|)
   ; s will be replaced by an ELT form in beforeCompile
   (|NRTgetLocalIndex| form)
   (list form (second v) env))
  (t
   ; form has been SETQd
   (list form (second v) env))))
((setq mprime (|getmode| form env))
 (cond
  ((and (null (|member| form |$formalArgList|))
        (null (member form |$FormalMapVariableList|))
        (null (|isFunction| form env))
        (null (eq |$compForModeIfTrue| t))))
   (|errorRef| form)))
 (list form mprime env ))
(member form |$FormalMapVariableList|)
(|stackMessage| (list '|no mode found for| form )))
((or (equal mode |$Expression|) (equal mode |$Symbol|))
 (list (list 'quote form) mode env ))
((null (|isFunction| form env)) (|errorRef| form))))

```

defun compList

[comp p587]

— defun compList —

```

(defun |compList| (form mode env)
  (let (tmp1 tmp2 t0 failed (newmode (second mode)))
    (if (null form)
        (list nil mode env)
        (progn
         (setq t0
          (do ((t3 form (cdr t3)) (x nil))
              ((or (atom t3) failed) (unless failed (nreverse0 tmp2)))
              (setq x (car t3)))
          (if (setq tmp1 (|comp| x newmode env))
              (progn

```

```

      (setq newmode (second tmp1))
      (setq env (third tmp1))
      (push tmp1 tmp2))
      (setq failed t))))
(unless failed
 (cons
  (cons 'list (loop for texpr in t0 collect (car texpr)))
  (list (list '|List| newmode) env))))))

```

defun compForm

[compForm1 [p600](#)]
 [compArgumentsAndTryAgain [p614](#)]
 [stackMessageIfNone p??]

— defun compForm —

```

(defun |compForm| (form mode env)
  (cond
   ((|compForm1| form mode env))
   ((|compArgumentsAndTryAgain| form mode env))
   (t (|stackMessageIfNone| (list '|cannot compile| form)))))

```

defun compForm1

This function is called if a keyword is found in a compile form but there is no handler listed for the form (See [6](#)). [length p??]

[outputComp [p347](#)]
 [compOrCroak [p586](#)]
 [compExpressionList [p607](#)]
 [coerceable [p360](#)]
 [comp [p587](#)]
 [coerce [p355](#)]
 [compForm2 [p607](#)]
 [augModemapsFromDomain1 [p257](#)]
 [getFormModemaps [p604](#)]
 [nreverse0 p??]
 [addDomain [p252](#)]
 [compToApply [p602](#)]
 [\$NumberOfArgsIfInteger p??]


```

      (setq env (|augModemapsFromDomain1| domain domain env))
      (dolist (x (|getFormModemaps| (cons opprime arg1) env)
                (nreverse0 tmp6))
        (when
          (and (consp x)
               (and (consp (qfirst x)) (equal (qcaar x) domain)))
            (push x tmp6))))))
    ans)
  ((setq ans
    (|compForm2| (cons opprime arg1) mode
      (setq env (|addDomain| domain env))
      (dolist (x (|getFormModemaps| (cons opprime arg1) env)
                (nreverse0 tmp5))
        (when
          (and (consp x)
               (and (consp (qfirst x)) (equal (qcaar x) domain)))
            (push x tmp5))))))
    ans)
    ((and (eq opprime '|construct|) (|coerceable| domain mode env))
      (when (setq td (|comp| (cons opprime arg1) domain env))
        (|coerce| td mode)))
      (t nil)))
  (t
    (setq env (|addDomain| mode env))
    (cond
      ((and (setq mmList (|getFormModemaps| form env))
            (setq td (|compForm2| form mode env mmList)))
        td)
      (t
        (|compToApply| op arg1 mode env))))))

```

defun compToApply

[compNoStacking p588]
 [compApplication p603]
 [\$EmptyMode p176]

— defun compToApply —

```

(defun |compToApply| (op arg1 m e)
  (let (tt m1)
    (declare (special |$EmptyMode|))
    (setq tt (|compNoStacking| op |$EmptyMode| e))
    (when tt
      (setq m1 (cadr tt))

```

```
(cond
  ((and (consp (car tt)) (eq (qcar (car tt)) 'quote)
        (consp (qcdr (car tt))) (eq (qcddr (car tt)) nil)
        (equal (qcadr (car tt)) m1))
    nil)
  (t
   (|compApplication| op argl m (caddr tt) tt))))))
```

defun compApplication

```
[eltForm p??]
[resolve p365]
[coerce p355]
[strconc p??]
[encodeItem p185]
[getAbbreviation p302]
[length p??]
[member p??]
[comp p587]
[isCategoryForm p??]
[$Category p??]
[$formatArgList p??]
[$op p??]
[$form p??]
[$prefix p??]
```

— defun compApplication —

```
(defun |compApplication| (op argl m env tt)
  (let (argml retm temp1 argTl nprefix opp form eltForm)
    (declare (special |$form| |$op| |$prefix| |$formalArgList| |$Category|))
    (cond
      ((and (consp (cadr tt)) (eq (qcar (cadr tt)) '|Mapping|)
            (consp (qcdr (cadr tt))))
        (setq retm (qcadr (cadr tt)))
        (setq argml (qcddr (cadr tt)))
        (cond
          ((not (eql (|#| argl) (|#| argml))) nil)
          (t
           (setq retm (|resolve| m retm))
           (cond
             ((or (equal retm |$Category|) (|isCategoryForm| retm env))
              nil)
             (t
```

```

(setq argTl
  (loop for x in argl for m in argml
    collect (progn
      (setq temp1 (or (|comp| x m env) (return '|failed|)))
      (setq env (caddr temp1))
      temp1)))
(cond
  ((eq argTl '|failed|) nil)
  (t
   (setq form
     (cond
      ((and
        (null
         (or (|member| op |$formalArgList|)
              (|member| (car tt) |$formalArgList|)))
         (atom (car tt)))
       (setq nprefix
         (or |$prefix| (|getAbbreviation| |$op| (|#| (cdr |$form|)))))
       (setq opp
         (intern
          (strconc (|encodeItem| nprefix) '|;| (|encodeItem| (car tt)))))
       (cons opp
        (append
         (loop for item in argTl collect (car item))
         (list '$))))
      (t
       (cons '|call|
        (cons (list '|applyFun| (car tt))
              (loop for item in argTl collect (car item))))))
      (|coerce| (list form retm env) (|resolve| retm m))))))
((eq op '|elt|) nil)
(t
 (setq eltForm (cons '|elt| (cons op argl)))
 (|comp| eltForm m env))))

```

defun getFormModemaps

```

[getFormModemaps p604]
[nreverse0 p??]
[get p??]
[eltModemapFilter p605]
[last p??]
[length p??]
[stackMessage p??]
[$insideCategoryPackageIfTrue p??]

```


— defun getFormModemaps —

```
(defun |getFormModemaps| (form env)
  (let (op argl domain op1 modemapList nargs finalModemapList)
    (declare (special |$insideCategoryPackageIfTrue|))
    (setq op (car form))
    (setq argl (cdr form))
    (cond
      ((and (consp op) (eq (qfirst op) '|elt|) (CONSP (qrest op))
        (consp (qcddr op)) (eq (qcdddr op) nil))
       (setq op1 (third op))
       (setq domain (second op))
       (loop for x in (|getFormModemaps| (cons op1 argl) env)
         when (and (consp x) (consp (qfirst x)) (equal (qcaar x) domain))
         collect x))
      ((null (atom op)) nil)
      (t
       (setq modemapList (|get| op '|modemap| env))
       (when |$insideCategoryPackageIfTrue|
        (setq modemapList
          (loop for x in modemapList
            when (and (consp x) (consp (qfirst x)) (not (eq (qcaar x) '$)))
            collect x))))))
    (cond
      ((eq op '|elt|)
       (setq modemapList (|eltModemapFilter| (|last| argl) modemapList env))
       ((eq op '|setelt|)
        (setq modemapList (|seteltModemapFilter| (CADR argl) modemapList env))))
      (setq nargs (|#| argl))
      (setq finalModemapList
        (loop for mm in modemapList
          when (equal (|#| (cddar mm)) nargs)
          collect mm))
      (when (and modemapList (null finalModemapList))
        (|stackMessage|
         (list '|no modemap for| op '|with | nargs '| arguments|)))
      finalModemapList))
```

defun eltModemapFilter

```
[isConstantId p??]
[stackMessage p??]
```

— defun eltModemapFilter —

```
(defun |eltModemapFilter| (name mmList env)
  (let (z)
    (if (|isConstantId| name env)
      (cond
        ((setq z
          (loop for mm in mmList
            when (and (consp mm) (consp (qfirst mm)) (consp (qcdar mm))
              (consp (qcddar mm))
              (consp (qcdddar mm))
              (equal (fourth (first mm)) name))
          collect mm))
        z)
      (t
        (|stackMessage|
          (list '|selector variable: | name '| is undeclared and unbound|))
          nil))
      mmList)))
```

defun seteltModemapFilter

```
[isConstantId p??]
[stackMessage p??]
```

— defun seteltModemapFilter —

```
(defun |seteltModemapFilter| (name mmList env)
  (let (z)
    (if (|isConstantId| name env)
      (cond
        ((setq z
          (loop for mm in mmList
            when (equal (car (cdddar mm)) name)
            collect mm))
        z)
      (t
        (|stackMessage|
          (list '|selector variable: | name '| is undeclared and unbound|))
          nil))
      mmList)))
```

defun compExpressionList

```
[nreverse0 p??]
[comp p587]
[convert p597]
[$Expression p??]
```

— defun compExpressionList —

```
(defun |compExpressionList| (argl m env)
  (let (tmp1 tlst)
    (declare (special |$Expression|))
    (setq tlst
      (prog (result)
        (return
          (do ((tmp2 argl (cdr tmp2)) (x nil))
              ((or (atom tmp2)) (nreverse0 result))
            (setq x (car tmp2))
            (setq result
              (cons
                (progn
                  (setq tmp1 (or (|comp| x |$Expression| env) (return '|failed|)))
                  (setq env (third tmp1))
                  tmp1)
                result))))))
    (unless (eq tlst '|failed|)
      (|convert|
        (list (cons 'list
          (prog (result)
            (return
              (do ((tmp3 tlst (cdr tmp3)) (y nil))
                  ((or (atom tmp3)) (nreverse0 result))
                (setq y (car tmp3))
                (setq result (cons (car y) result))))))
          |$Expression| env)
        m))))
```

—————

defun compForm2

```
[take p??]
[length p??]
[nreverse0 p??]
[sublis p??]
[assoc p??]
[PredImplies p??]
```



```

      (setq v (|assoc| (cons dc nsig) modemapList))
      (consp v)
      (progn
        (setq tmp6 (qrest v))
        (and (consp tmp6) (eq (qrest tmp6) nil)
          (progn
            (setq tmp7 (qfirst tmp6))
            (and (consp tmp7)
              (progn
                (setq ncond (qfirst tmp7))
                t))))))
      (setq deleteList (cons u deleteList))
      (unless (|PredImplies| ncond cond)
        (setq newList (push '(', (car u) (,cond (elt ,dc nil))) newList))))))
(when deleteList
  (setq modemapList
    (remove-if #'(lambda (x) (member x deletelist)) modemapList)))
; it is important that subsumed ops (newList) be considered last
(when newList (setq modemapList (append modemapList newList)))
(setq tl
  (loop for x in arg1
    while (and (|isSimple| x)
      (setq td (|compUniquely| x |$EmptyMode| env)))
    collect td
    do (setq env (third td))))
(cond
  ((some #'identity tl)
    (setq partialModeList (loop for x in tl collect (when x (second x))))
    (or
      (|compFormPartiallyBottomUp| form mode env modemapList partialModeList)
      (|compForm3| form mode env modemapList)))
  (t (|compForm3| form mode env modemapList))))

```

defun compForm3

[compFormWithModemap p??]
 [compUniquely [p613](#)]
 [\$compUniquelyIfTrue p??]

— defun compForm3 —

```

(defun |compForm3| (form mode env modemapList)
  (let (op arg1 mml tt)
    (declare (special |$compUniquelyIfTrue|))
    (setq op (car form))

```

```

(setq arg1 (cdr form))
(setq tt
  (let (result)
    (maplist #'(lambda (mlst)
      (setq result (or result
        (|compFormWithModemap| form mode env (car (setq mml mlst))))))
      modemapList)
    result))
(when (|$compUniquelyIfTrue|
  (if (let (result)
    (mapcar #'(lambda (mm)
      (setq result (or result (|compFormWithModemap| form mode env mm))))
      (rest mml))
      result)
    (throw '|$compUniquely| nil)
    tt))
  tt))

```

defun compFocompFormWithModemap

```

[isCategoryForm p??]
[isFunctor p253]
[substituteIntoFunctorModemap p612]
[listOfSharpVars p??]
[coerceable p360]
[compApplyModemap p259]
[isCategoryForm p??]
[identp p??]
[get p??]
[last p??]
[convert p597]
[$Category p??]
[$FormalMapVariableList p270]

```

— defun compFormWithModemap —

```

(defun |compFormWithModemap| (form m env modemap)
  (prog (op arg1 sv target cexpr targetp map temp1 f transimp sl mp formp z c
    xp ep tt)
    (declare (special |$Category| |$FormalMapVariableList|))
    (return
      (progn
        (setq op (car form))
        (setq arg1 (cdr form))

```

```

(setq map (car modemap))
(setq target (cadar modemap))
(when (and (|isCategoryForm| target env) (|isFunction| op))
  (setq temp1 (or (|substituteIntoFunctorModemap| arg1 modemap env)
    (return nil)))
  (setq modemap (car temp1))
  (setq env (cadr temp1))
  (setq map (car modemap))
  (setq target (cadar modemap))
  (setq cexpr (cdr modemap))
  modemap)
(setq sv (|listOfSharpVars| map))
(when sv
  (loop for x in arg1 for ss in |$FormalMapVariableList|
    do (when (|member| ss sv)
      (setq modemap (subst x ss modemap :test #'equal))
      (setq map (car modemap))
      (setq target (cadar modemap))
      (setq cexpr (cdr modemap))
      modemap)))
(cond
  ((null (setq targetp (|coerceable| target m env))) nil)
  (t
   (setq map (cons targetp (cdr map)))
   (setq temp1 (or (|compApplyModemap| form modemap env nil)
     (return nil)))
   (setq f (car temp1))
   (setq transimp (cadr temp1))
   (setq sl (caddr temp1))
   (setq mp (sublis sl (elt map 1)))
   (setq xp
    (progn
     (setq formp (cons f (loop for tt in transimp collect (car tt))))
     (cond
      ((or (equal mp |$Category|) (|isCategoryForm| mp env)) formp)
      ((and (eq op '|elt|) (consp f) (eq (qcar f) 'xlam)
        (identp (car arg1))
        (setq c (|get| (car arg1) '|condition| env))
        (consp c) (eq (qcdr c) nil)
        (consp (qcar c)) (eq (qcaar c) '|case|)
        (consp (qcdar c)) (equal (qcadar c) z)
        (consp (qcddar c)) (eq (qcdr (qcddar c)) nil)
        (or (and (consp (qcaddar c))
          (eq (qcar (qcaddar c)) '|:|)
          (consp (qcdr (qcaddar c)))
          (equal (qcadr (qcaddar c)) (cadr arg1))
          (consp (qcddr (qcaddar c)))
          (eq (qcdddr (qcaddar c)) nil)
          (equal (qcaddr (qcaddar c)) m))
        (eq (qcaddar c) (cadr arg1))))))

```

```

      (list 'cdr (car argl)))
    (t (cons '|call| formp))))))
  (setq ep
    (if transimp
      (caddr (|last| transimp))
      env))
  (setq tt (list xp mp ep))
  (|convert| tt m))))))

```

defun substituteIntoFunctorModemap

```

[keyedSystemError p??]
[eqsubstlist p??]
[compOrCroak p586]
[sublis p??]

```

— defun substituteIntoFunctorModemap —

```

(defun |substituteIntoFunctorModemap| (argl modemap env)
  (let (dc sig tmp1 tl substitutionList)
    (setq dc (caar modemap))
    (setq sig (cdar modemap))
    (cond
      ((not (eql (|#| dc) (|#| sig)))
        (|keyedSystemError|
          "Unexpected error or improper call to system function %1: %2"
          (list "substituteIntoFunctorModemap" "Incompatible maps"))))
      ((equal (|#| argl) (|#| (cdr sig)))
        (setq sig (eqsubstlist argl (cdr dc) sig))
        (setq tl
          (loop for a in argl for m in (rest sig)
            collect (progn
              (setq tmp1 (|compOrCroak| a m env))
              (setq env (caddr tmp1))
              tmp1)))
        (setq substitutionList
          (loop for x in (rest dc) for tt in tl
            collect (cons x (car tt))))
        (list (sublis substitutionList modemap) env))
      (t nil))))

```

defun compFormPartiallyBottomUp

[compForm3 p609]
 [compFormMatch p613]

— defun compFormPartiallyBottomUp —

```
(defun |compFormPartiallyBottomUp| (form mode env modemapList partialModeList)
  (let (mmList)
    (when (setq mmList (loop for mm in modemapList
                            when (|compFormMatch| mm partialModeList)
                            collect mm))
      (|compForm3| form mode env mmList))))
```

—————

defun compFormMatch

— defun compFormMatch —

```
(defun |compFormMatch| (mm partialModeList)
  (labels (
    (ismatch (a b)
      (cond
        ((null b) t)
        ((null (car b)) (|compFormMatch,match| (cdr a) (cdr b)))
        ((and (equal (car a) (car b)) (ismatch (cdr a) (cdr b))))))
    (and (consp mm) (consp (qfirst mm)) (consp (qcddar mm))
      (ismatch (qcddar mm) partialModeList))))
```

—————

defun compUniquely

[compUniquely p613]
 [comp p587]
 [\$compUniquelyIfTrue p??]

— defun compUniquely —

```
(defun |compUniquely| (x m env)
  (let (|$compUniquelyIfTrue|)
    (declare (special |$compUniquelyIfTrue|))
    (setq |$compUniquelyIfTrue| t)
```

```
(catch '|compUniquely| (|comp| x m env)))
```

defun compArgumentsAndTryAgain

[comp p587]

[compForm1 p600]

[\$EmptyMode p176]

— defun compArgumentsAndTryAgain —

```
(defun |compArgumentsAndTryAgain| (form mode env)
  (let (arg1 tmp1 a tmp2 tmp3 u)
    (declare (special |$EmptyMode|))
    (setq arg1 (cdr form))
    (cond
      ((and (consp form) (eq (qfirst form) '|elt|)
        (progn
          (setq tmp1 (qrest form))
          (and (consp tmp1)
            (progn
              (setq a (qfirst tmp1))
              (setq tmp2 (qrest tmp1))
              (and (consp tmp2) (eq (qrest tmp2) nil))))))
        (when (setq tmp3 (|comp| a |$EmptyMode| env))
          (setq env (third tmp3))
          (|compForm1| form mode env)))
      (t
        (setq u
          (dolist (x arg1)
            (setq tmp3 (or (|comp| x |$EmptyMode| env) (return '|failed|)))
            (setq env (third tmp3))
            tmp3))
          (unless (eq u '|failed|)
            (|compForm1| form mode env))))))
```

defun compWithMappingMode

[compWithMappingMode1 p615]

[\$formalArgList p??]

— defun compWithMappingMode —

```
(defun |compWithMappingMode| (form mode oldE)
  (declare (special |$formalArgList|))
  (|compWithMappingMode1| form mode oldE |$formalArgList|))
```

defun compWithMappingMode1

```
[isFunction p253]
[get p??]
[extendsCategoryForm p??]
[compLambda p325]
[stackAndThrow p??]
[take p??]
[compMakeDeclaration p622]
[hasFormalMapVariable p621]
[comp p587]
[extractCodeAndConstructTriple p620]
[optimizeFunctionDef p228]
[comp-tran p??]
[freelist p623]
[$formalArgList p??]
[$killOptimizeIfTrue p??]
[$funname p??]
[$funnameTail p??]
[$QuickCode p??]
[$EmptyMode p176]
[$FormalMapVariableList p270]
[$CategoryFrame p??]
[$formatArgList p??]
```

— defun compWithMappingMode1 —

```
(defun |compWithMappingMode1| (form mode oldE |$formalArgList|)
  (declare (special |$formalArgList|))
  (prog (|$killOptimizeIfTrue| $funname $funnameTail mprime sl tmp1 tmp2
        tmp3 tmp4 tmp5 tmp6 target argModeList nx oldstyle ress vl1 vl e tt
        u frees i scode locals body vec expandedFunction fname uu)
    (declare (special |$killOptimizeIfTrue| $funname $funnameTail
                      |$QuickCode| |$EmptyMode| |$FormalMapVariableList|
                      |$CategoryFrame| |$formatArgList|))
    (return
     (seq
      (progn
       (setq mprime (second mode))
       (setq sl (cddr mode))
```

```
(setq |$killOptimizeIfTrue| t)
(setq e oldE)
(cond
  ((|isFunctor| form)
    (cond
      ((and (progn
              (setq tmp1 (|get| form '|modemap| |$CategoryFrame|))
              (and (consp tmp1)
                    (progn
                      (setq tmp2 (qfirst tmp1))
                      (and (consp tmp2)
                            (progn
                              (setq tmp3 (qfirst tmp2))
                              (and (consp tmp3)
                                    (progn
                                      (setq tmp4 (qrest tmp3))
                                      (and (consp tmp4)
                                            (progn
                                              (setq target (qfirst tmp4))
                                              (setq argModeList (qrest tmp4))
                                              t))))))
                              (progn
                                (setq tmp5 (qrest tmp2))
                                (and (consp tmp5) (eq (qrest tmp5) nil))))))))
                (progn
                  (setq tmp5 (qrest tmp2))
                  (and (consp tmp5) (eq (qrest tmp5) nil)))))))
        (prog (t1)
              (setq t1 t)
              (return
                (do ((t2 nil (null t1))
                    (t3 argModeList (cdr t3))
                    (newmode nil)
                    (t4 sl (cdr t4))
                    (s nil))
                    ((or t2 (atom t3)
                        (progn (setq newmode (car t3)) nil)
                        (atom t4)
                        (progn (setq s (car t4)) nil))
                     t1)
                  (seq (exit
                        (setq t1
                          (and t1 (|extendsCategoryForm| '$ s newmode))))))
                    (|extendsCategoryForm| '$ target mprime))
                (return (list form mode e )))
              (t nil)))
    (t
      (when (stringp form) (setq form (intern form)))
      (setq res nil)
      (setq oldstyle t)
      (cond
        ((and (consp form)
              (eq (qfirst form) '+->))
```

```

(progn
  (setq tmp1 (qrest form))
  (and (consp tmp1)
    (progn
      (setq vl (qfirst tmp1))
      (setq tmp2 (qrest tmp1))
      (and (consp tmp2)
        (eq (qrest tmp2) nil)
        (progn (setq nx (qfirst tmp2)) t))))))
(setq oldstyle nil)
(cond
  ((and (consp vl) (eq (qfirst vl) '|:|))
    (setq ress (|compLambda| form mode oldE))
    ress)
  (t
    (setq vl
      (cond
        ((and (consp vl)
          (eq (qfirst vl) '|@Tuple|)
          (progn (setq vl1 (qrest vl)) t))
          vl1)
        (t vl))))
    (setq vl
      (cond
        ((symbolp vl) (cons vl nil))
        ((and
          (listp vl)
          (prog (t5)
            (setq t5 t)
            (return
              (do ((t7 nil (null t5))
                (t6 vl (cdr t6))
                (v nil))
                ((or t7 (atom t6) (progn (setq v (car t6)) nil)) t5)
              (seq
                (exit
                  (setq t5 (and t5 (symbolp v))))))))))
          vl)
        (t
          (|stackAndThrow| (cons '|bad +-> arguments:| (list vl ))))))
    (setq |$formatArgList| (append vl |$formalArgList|))
    (setq form nx))))
(t
  (setq vl (take (|#| sl) |$FormalMapVariableList|)))
(cond
  (ress ress)
  (t
    (do ((t8 sl (cdr t8)) (m nil) (t9 vl (cdr t9)) (v nil))
      ((or (atom t8)
        (progn (setq m (car t8)) nil)

```

```

        (atom t9)
        (progn (setq v (car t9)) nil))
    nil)
  (seq (exit (progn
    (setq tmp6
      (|compMakeDeclaration| (list '(:| v m ) |$EmptyMode| e))
      (setq e (third tmp6))
      tmp6))))
  (cond
    ((and oldstyle
      (null (null vl))
      (null (|hasFormalMapVariable| form vl)))
    (return
      (progn
        (setq tmp6 (or (|comp| (cons form vl) mprime e) (return nil)))
        (setq u (car tmp6))
        (|extractCodeAndConstructTriple| u mode oldE))))
    ((and (null vl) (setq tt (|comp| (cons form nil) mprime e)))
    (return
      (progn
        (setq u (car tt))
        (|extractCodeAndConstructTriple| u mode oldE))))
  (t
    (setq tmp6 (or (|comp| form mprime e) (return nil)))
    (setq u (car tmp6))
    (setq uu (|optimizeFunctionDef| '(nil (lambda ,vl ,u))))
    ; -- At this point, we have a function that we would like to pass.
    ; -- Unfortunately, it makes various free variable references outside
    ; -- itself. So we build a mini-vector that contains them all, and
    ; -- pass this as the environment to our inner function.
    (setq $funname nil)
    (setq $funnameTail (list nil))
    (setq expandedFunction (comp-tran (second uu)))
    (setq frees (freelist expandedFunction vl nil e))
    (setq expandedFunction
      (cond
        ((eq1 (|#| frees) 0)
        (cons 'lambda (cons (append vl (list '$$))
          (caddr expandedFunction))))
        ((eq1 (|#| frees) 1)
        (setq vec (caar frees))
        (cons 'lambda (cons (append vl (list vec))
          (caddr expandedFunction))))
      )
    (t
      (setq scode nil)
      (setq vec nil)
      (setq locals nil)
      (setq i -1)
      (do ((t0 frees (cdr t0)) (v nil))
        ((or (atom t0) (progn (setq v (car t0)) nil)) nil)

```

```

(seq
  (exit
    (progn
      (setq i (plus i 1))
      (setq vec (cons (car v) vec))
      (setq scode
        (cons
          (cons 'setq
            (cons (car v)
              (cons
                (cons
                  (cond
                    (|$QuickCode| 'qrefelt)
                    (t 'elt))
                  (cons '$$ (cons i nil)))
                nil)))
            scode))
          (setq locals (cons (car v) locals))))))
      (setq body (cddr expandedFunction))
    (cond
      (locals
        (cond
          ((and (consp body)
            (progn
              (setq tmp1 (qfirst body))
              (and (consp tmp1)
                (eq (qfirst tmp1) 'declare))))
            (setq body
              (cons (car body)
                (cons
                  (cons 'prog
                    (cons locals
                      (append scode
                        (cons
                          (cons 'return
                            (cons
                              (cons 'progn
                                (cdr body))
                                nil))
                              nil)))
                        nil))))
                nil))))
          (t
            (setq body
              (cons
                (cons 'prog
                  (cons locals
                    (append scode
                      (cons
                        (cons 'return
                          (cons
                            (cons
                              (cons 'progn
                                (cdr body))
                                nil))
                              nil)))
                        nil))))
                nil))))
        )
      )
    )
  )
)

```

```

        (cons 'progn body)
        nil))
      nil)))
    nil))))))
    (setq vec (cons 'vector (nreverse vec)))
    (cons 'lambda (cons (append vl (list '$$)) body))))))
  (setq fname (list 'closedfn expandedFunction))
  (setq uu
    (cond
      (frees (list 'cons fname vec))
      (t (list 'list fname))))
  (list uu mode oldE)))))))))

```

defun extractCodeAndConstructTriple

— defun extractCodeAndConstructTriple —

```

(defun |extractCodeAndConstructTriple| (form mode oldE)
  (let (tmp1 a fn op env)
    (cond
      ((and (consp form) (eq (qfirst form) '|call|))
        (progn
          (setq tmp1 (qrest form))
          (and (consp tmp1)
              (progn (setq fn (qfirst tmp1)) t))))
      (cond
        ((and (consp fn) (eq (qfirst fn) '|applyFun|))
          (progn
            (setq tmp1 (qrest fn))
            (and (consp tmp1) (eq (qrest tmp1) nil)
                (progn (setq a (qfirst tmp1)) t))))
        (setq fn a)))
      (list fn mode oldE))
    (t
      (setq op (car form))
      (setq env (car (reverse (cdr form))))
      (list (list 'cons (list 'function op) env) mode oldE))))))

```

defun hasFormalMapVariable

```
[hasFormalMapVariable ScanOrPairVec (vol5)]
[$formalMapVariables p??]
```

— defun hasFormalMapVariable —

```
(defun |hasFormalMapVariable| (x vl)
  (let (|$formalMapVariables|)
    (declare (special |$formalMapVariables|))
    (when (setq |$formalMapVariables| vl)
      (|ScanOrPairVec| #'(lambda (y) (member y |$formalMapVariables|)) x))))
```

defun argsToSig

— defun argsToSig —

```
(defun |argsToSig| (args)
  (let (tmp1 v tmp2 tt sig1 arg1 bad)
    (cond
      ((and (consp args) (eq (qfirst args) '|:|))
        (progn
          (setq tmp1 (qrest args))
          (and (consp tmp1)
            (progn
              (setq v (qfirst tmp1))
              (setq tmp2 (qrest tmp1))
              (and (consp tmp2)
                (eq (qrest tmp2) nil)
                (progn
                  (setq tt (qfirst tmp2))
                  t)))))))
      (list (list v) (list tt)))
    (t
      (setq sig1 nil)
      (setq arg1 nil)
      (setq bad nil)
      (dolist (arg args)
        (cond
          ((and (consp arg) (eq (qfirst arg) '|:|))
            (progn
              (setq tmp1 (qrest arg))
              (and (consp tmp1)
                (progn
```

```

      (setq v (qfirst tmp1))
      (setq tmp2 (qrest tmp1))
      (and (consp tmp2) (eq (qrest tmp2) nil)
        (progn
          (setq tt (qfirst tmp2))
          t))))))
    (setq sig1 (cons tt sig1))
    (setq arg1 (cons v arg1))
    (t (setq bad t))))
  (cond
    (bad (list nil nil ))
    (t (list (reverse arg1) (reverse sig1))))))

```

defun compMakeDeclaration

[compColon p294]
 [\$insideExpressionIfTrue p??]

— defun compMakeDeclaration —

```

(defun |compMakeDeclaration| (form mode env)
  (let (|$insideExpressionIfTrue|)
    (declare (special |$insideExpressionIfTrue|))
    (setq |$insideExpressionIfTrue| nil)
    (|compColon| form mode env)))

```

defun modifyModeStack

[say p??]
 [copy p??]
 [setelt p??]
 [resolve p365]
 [\$reportExitModeStack p??]
 [\$exitModeStack p??]

— defun modifyModeStack —

```

(defun |modifyModeStack| (m index)
  (declare (special |$exitModeStack| |$reportExitModeStack|))
  (if |$reportExitModeStack|
    (say "exitModeStack: " (copy |$exitModeStack|)

```

```
" ==> "
(progn
  (setelt |$exitModeStack| index
    (|resolve| m (elt |$exitModeStack| index)))
  |$exitModeStack|))
(setelt |$exitModeStack| index
  (|resolve| m (elt |$exitModeStack| index))))
```

defun Create a list of unbound symbols

We walk argument *u* looking for symbols that are unbound. If we find a symbol we add it to the free list. If it occurs in a prog then it is bound and we remove it from the free list. Multiple instances of a single symbol in the free list are represented by the alist (symbol . count) [freelist p623]

```
[freelist assq (vol5)]
[freelist identp (vol5)]
[getmode p??]
[unionq p??]
```

— defun freelist —

```
(defun freelist (u bound free e)
  (let (v op)
    (if (atom u)
      (cond
        ((null (identp u)) free)
        ((member u bound) free)
        ; more than 1 free becomes alist (name . number)
        ((setq v (assq u free)) (rplacd v (+ 1 (cdr v))) free)
        ((null (|getmode| u e)) free)
        (t (cons (cons u 1) free)))
      (progn
        (setq op (car u))
        (cond
          ((member op '(quote go function)) free)
          ((eq op 'lambda) ; lambdas bind symbols
            (setq bound (unionq bound (second u)))
            (dolist (v (cddr u))
              (setq free (freelist v bound free e))))
          ((eq op 'prog) ; progs bind symbols
            (setq bound (unionq bound (second u)))
            (dolist (v (cddr u))
              (unless (atom v)
                (setq free (freelist v bound free e)))))
          ((eq op 'seq)
```

```

(dolist (v (cdr u))
  (unless (atom v)
    (setq free (freelist v bound free e))))
((eq op 'cond)
  (dolist (v (cdr u))
    (dolist (vv v)
      (setq free (freelist vv bound free e))))))
(t
  (when (atom op) (setq u (cdr u))) ; atomic functions aren't descended
  (dolist (v u)
    (setq free (freelist v bound free e))))
free)))

```

defun compOrCroak1,compactify

[compOrCroak1,compactify p624]
 [lassoc p??]

— defun compOrCroak1,compactify —

```

(defun |compOrCroak1,compactify| (al)
  (cond
    ((null al) nil)
    ((lassoc (caar al) (cdr al)) (|compOrCroak1,compactify| (cdr al)))
    (t (cons (car al) (|compOrCroak1,compactify| (cdr al))))))

```

defun Compiler/Interpreter interface

[ncINTERPFILE SpadInterpretStream (vol5)]
 [\$EchoLines p??]
 [\$ReadingFile p??]

The **SpadInterpretStream** function call takes three arguments. The first argument The second argument **source** is the name of a file to include. The third argument **interactive?**, when false, will read from the file rather than the console.

— defun ncINTERPFILE —

```

(defun |ncINTERPFILE| (file echo)
  (let ((|$EchoLines| echo) (|$ReadingFile| t))
    (declare (special |$EchoLines| |$ReadingFile|))
    (|SpadInterpretStream| 1 file nil)))

```

defun recompile-lib-file-if-necessary

[compile-lib-file p625]
 [*lisp-bin-filetype* p??]

— defun recompile-lib-file-if-necessary —

```
(defun recompile-lib-file-if-necessary (lfile)
  (let* ((bfile (make-pathname :type *lisp-bin-filetype* :defaults lfile))
         (bdate (and (probe-file bfile) (file-write-date bfile)))
         (ldate (and (probe-file lfile) (file-write-date lfile))))
    (declare (special *lisp-bin-filetype*))
    (unless (and ldate bdate (> bdate ldate))
      (compile-lib-file lfile)
      (list bfile))))
```

defun spad-fixed-arg

— defun spad-fixed-arg —

```
(defun spad-fixed-arg (fname )
  (and (equal (symbol-package fname) (find-package "BOOT"))
       (not (get fname 'compiler::spad-var-arg))
       (search ";" (symbol-name fname))
       (or (get fname 'compiler::fixed-args)
           (setf (get fname 'compiler::fixed-args) t)))
  nil)
```

defun compile-lib-file

— defun compile-lib-file —

```
(defun compile-lib-file (fn &rest opts)
  (unwind-protect
    (progn
      (trace (compiler::fast-link-proclaimed-type-p
```

```

      :exitcond nil
      :entrycond (spad-fixed-arg (car system::arglist))))
(trace (compiler::t1defun
      :exitcond nil
      :entrycond (spad-fixed-arg (caar system::arglist))))
(apply #'compile-file fn opts))
(untrace compiler::fast-link-proclaimed-type-p compiler::t1defun)))

```

defun compileFileQuietly

if `$InteractiveMode` then use a null outputstream [`$InteractiveMode p??`]
 [`*standard-output* p??`]

— defun compileFileQuietly —

```

(defun |compileFileQuietly| (fn)
  (let (
    (*standard-output*
     (if |$InteractiveMode| (make-broadcast-stream)
        *standard-output*)))
    (declare (special *standard-output* |$InteractiveMode|))
    (compile-file fn)))

```

defvar \$byConstructors

— initvars —

```

(defvar |$byConstructors| () "list of constructors to be compiled")

```

defvar \$constructorsSeen

— initvars —

```

(defvar |$constructorsSeen| () "list of constructors found")

```

Chapter 15

Level 1

defvar current-fragment

A string containing remaining chars from readline; needed because Symbolics read-line returns embedded newlines in a c-m-Y.

— **initvars** —

```
(defvar current-fragment nil)
```

—————

defun read-a-line

The **read-a-line** function reads a line from the current stream, potentially setting *eof*. It then recursively calls itself [Line-New-Line p??]

[*eof* p??]

[File-Closed p??]

read-a-line : **FileStream** → **String** where FileStream might be

```
#<input stream "/research/t1/src/algebra/EQ.spad">
```

and the returned string might be

```
"abbrev domain EQ Equation"
```

— **defun read-a-line** —

```
(defun read-a-line (&optional (stream t))
```

```
(if (stream-eof stream)
    (progn
      (setq File-Closed t)
      (setq *eof* t)
      (Line-New-Line (make-string 0) Current-Line)
      nil)
    (read-line stream)))
```

Chapter 16

Level 0

16.1 Line Handling

Line Buffer

The philosophy of lines is that

- NEXT LINE will always return a non-blank line or fail.
- Every line is terminated by a blank character.

Hence there is always a current character, because there is never a non-blank line, and there is always a separator character between tokens on separate lines. Also, when a line is read, the character pointer is always positioned ON the first character.

defstruct line

— initvars —

```
(defstruct line "Line of input file to parse."  
  (buffer (make-string 0) :type string)  
  (current-char #\Return :type character)  
  (current-index 1 :type fixnum)  
  (last-index 0 :type fixnum)  
  (number 0 :type fixnum))
```

—————

defvar current-line

The current input line.

— **initvars** —

```
(defvar current-line (make-line))
```

—————

defmacro line-clear

[[\\$line p631](#)]

— **defmacro line-clear** —

```
(defmacro line-clear (line)
  '(let ((l ,line))
    (setf (line-buffer l) (make-string 0))
    (setf (line-current-char l) #\return)
    (setf (line-current-index l) 1)
    (setf (line-last-index l) 0)
    (setf (line-number l) 0)))
```

—————

defun line-print

[[\\$line p631](#)]

[[\\$out-stream p??](#)]

— **defun line-print** —

```
(defun line-print (line)
  (declare (special out-stream))
  (format out-stream "~&~5D> ~A~%" (Line-Number line) (Line-Buffer line))
  (format out-stream "~v@T~%" (+ 7 (Line-Current-Index line))))
```

—————

defun line-at-end-p

[[\\$line p631](#)]

— **defun line-at-end-p** —

```
(defun line-at-end-p (line)
  "Tests if line is empty or positioned past the last character."
  (>= (line-current-index line) (line-last-index line)))
```

defun line-past-end-p

[[line](#) [p631](#)]

— defun line-past-end-p —

```
(defun line-past-end-p (line)
  "Tests if line is empty or positioned past the last character."
  (> (line-current-index line) (line-last-index line)))
```

defun line-next-char

[[line](#) [p631](#)]

— defun line-next-char —

```
(defun line-next-char (line)
  (elt (line-buffer line) (1+ (line-current-index line))))
```

defun line-advance-char

[[line](#) [p631](#)]

— defun line-advance-char —

```
(defun line-advance-char (line)
  (setf (line-current-char line)
        (elt (line-buffer line) (incf (line-current-index line)))))
```

defun line-current-segment

[\$line p631]

— defun line-current-segment —

```
(defun line-current-segment (line)
  "Buffer from current index to last index."
  (if (line-at-end-p line)
      (make-string 0)
      (subseq (line-buffer line)
               (line-current-index line)
               (line-last-index line))))
```

—————

defun line-new-line

[\$line p631]

— defun line-new-line —

```
(defun line-new-line (string line &optional (linenum nil))
  "Sets string to be the next line stored in line."
  (setf (line-last-index line) (1- (length string)))
  (setf (line-current-index line) 0)
  (setf (line-current-char line)
        (or (and (> (length string) 0) (elt string 0)) #\Return))
  (setf (line-buffer line) string)
  (setf (line-number line) (or linenum (1+ (line-number line)))))
```

—————

defun next-line

[\$in-stream p??]

[\$line-handler p??]

— defun next-line —

```
(defun next-line (&optional (in-stream t))
  (declare (special in-stream line-handler))
  (funcall Line-Handler in-stream))
```

—————

defun Advance-Char

[Line-At-End-P p??]
 [Line-Advance-Char p??]
 [next-line p634]
 [current-char p460]
 [\$in-stream p??]
 [\$line p631]

— defun Advance-Char —

```
(defun Advance-Char ()
  "Advances IN-STREAM, invoking Next Line if necessary."
  (declare (special in-stream))
  (loop
    (cond
      ((not (Line-At-End-P Current-Line))
       (return (Line-Advance-Char Current-Line)))
      ((next-line in-stream)
       (return (current-char)))
      ((return nil)))))
```

—————

defun storeblanks

— defun storeblanks —

```
(defun storeblanks (line n)
  (do ((i 0 (1+ i)))
      ((= i n) line)
    (setf (char line i) #\ )))
```

—————

defun initial-substring

— defun initial-substring —

```
(defun initial-substring (pattern line)
  (let ((ind (mismatch pattern line)))
    (or (null ind) (eql ind (size pattern)))))
```

defun get-a-line

[is-console p553]
 [get-a-line mkprompt (vol5)]
 [read-a-line p629]

get-a-line : **FileStream** $\rightarrow$ **String** where FileStream might be

```
#<input stream "/research/t1/src/algebra/EQ.spad">
```

and the returned string might be

```
">abbrev domain EQ Equation"
```

— defun get-a-line —

```
(defun get-a-line (stream)
  (when (is-console stream) (princ (mkprompt)))
  (let ((l1 (read-a-line stream)))
    (if (and (stringp l1) (adjustable-array-p l1))
        (make-array (array-dimensions l1) :element-type 'string-char
                     :adjustable t :initial-contents l1)
        l1)))
```

Chapter 17

Signatures

Chapter 18

The Chunks

— Compiler —

```
(in-package "BOOT")

\getchunk{initvars}

\getchunk{LEDNUDTables}
\getchunk{GLIPHTable}
\getchunk{RENAMETOKTable}
\getchunk{GENERICTable}

\getchunk{defmacro bang}
\getchunk{defmacro line-clear}
\getchunk{defmacro must}
\getchunk{defmacro nth-stack}
\getchunk{defmacro pop-stack-1}
\getchunk{defmacro pop-stack-2}
\getchunk{defmacro pop-stack-3}
\getchunk{defmacro pop-stack-4}
\getchunk{defmacro reduce-stack-clear}
\getchunk{defmacro stack-/empty}
\getchunk{defmacro star}

\getchunk{defun action}
\getchunk{defun addArgumentConditions}
\getchunk{defun addclose}
\getchunk{defun addConstructorModemaps}
\getchunk{defun addDomain}
\getchunk{defun addEltModemap}
\getchunk{defun addEmptyCapsuleIfNecessary}
\getchunk{defun addModemapKnown}
```

```

\getchunk{defun addModemap}
\getchunk{defun addModemap0}
\getchunk{defun addModemap1}
\getchunk{defun addNewDomain}
\getchunk{defun add-parens-and-semis-to-line}
\getchunk{defun addSuffix}
\getchunk{defun Advance-Char}
\getchunk{defun advance-token}
\getchunk{defun alistSize}
\getchunk{defun allLASSOCs}
\getchunk{defun aplTran}
\getchunk{defun aplTran1}
\getchunk{defun aplTranList}
\getchunk{defun applyMapping}
\getchunk{defun argsToSig}
\getchunk{defun assignError}
\getchunk{defun AssocBarGensym}
\getchunk{defun augLisplibModemapsFromCategory}
\getchunk{defun augmentLisplibModemapsFromFunctor}
\getchunk{defun augModemapsFromCategory}
\getchunk{defun augModemapsFromCategoryRep}
\getchunk{defun augModemapsFromDomain}
\getchunk{defun augModemapsFromDomain1}
\getchunk{defun autoCoerceByModemap}

\getchunk{defun blankp}
\getchunk{defun bootStrapError}
\getchunk{defun buildLibAttr}
\getchunk{defun buildLibAttrs}
\getchunk{defun buildLibdb}
\getchunk{defun buildLibdbConEntry}
\getchunk{defun buildLibdbString}
\getchunk{defun buildLibOp}
\getchunk{defun buildLibOps}
\getchunk{defun bumpererrorcount}

\getchunk{defun canReturn}
\getchunk{defun char-eq}
\getchunk{defun char-ne}
\getchunk{defun checkAddBackSlashes}
\getchunk{defun checkAddIndented}
\getchunk{defun checkAddMacros}
\getchunk{defun checkAddPeriod}
\getchunk{defun checkAddSpaces}
\getchunk{defun checkAddSpaceSegments}
\getchunk{defun checkAlphabetic}
\getchunk{defun checkAndDeclare}
\getchunk{defun checkArguments}
\getchunk{defun checkBalance}
\getchunk{defun checkBeginEnd}

```

```

\getchunk{defun checkComments}
\getchunk{defun checkDecorate}
\getchunk{defun checkDecorateForHt}
\getchunk{defun checkDocError}
\getchunk{defun checkDocError1}
\getchunk{defun checkDocMessage}
\getchunk{defun checkExtract}
\getchunk{defun checkFixCommonProblem}
\getchunk{defun checkGetArgs}
\getchunk{defun checkGetLispFunctionName}
\getchunk{defun checkGetMargin}
\getchunk{defun checkGetParse}
\getchunk{defun checkGetStringBeforeRightBrace}
\getchunk{defun checkHTargs}
\getchunk{defun checkIeEg}
\getchunk{defun checkIeEgfun}
\getchunk{defun checkIndentedLines}
\getchunk{defun checkIsValidType}
\getchunk{defun checkLookForLeftBrace}
\getchunk{defun checkLookForRightBrace}
\getchunk{defun checkNumOfArgs}
\getchunk{defun checkRecordHash}
\getchunk{defun checkRemoveComments}
\getchunk{defun checkRewrite}
\getchunk{defun checkSayBracket}
\getchunk{defun checkSkipBlanks}
\getchunk{defun checkSkipIdentifierToken}
\getchunk{defun checkSkipOpToken}
\getchunk{defun checkSkipToken}
\getchunk{defun checkSplitBackslash}
\getchunk{defun checkSplitBrace}
\getchunk{defun checkSplitOn}
\getchunk{defun checkSplitPunctuation}
\getchunk{defun checkSplit2Words}
\getchunk{defun checkTexht}
\getchunk{defun checkTransformFirsts}
\getchunk{defun checkTrim}
\getchunk{defun checkTrimCommented}
\getchunk{defun checkWarning}
\getchunk{defun coerce}
\getchunk{defun coerceable}
\getchunk{defun coerceByModemap}
\getchunk{defun coerceEasy}
\getchunk{defun coerceExit}
\getchunk{defun coerceExtraHard}
\getchunk{defun coerceHard}
\getchunk{defun coerceSubset}
\getchunk{defun collectAndDeleteAssoc}
\getchunk{defun collectComBlock}
\getchunk{defun comma2Tuple}

```

```
\getchunk{defun comp}  
\getchunk{defun comp2}  
\getchunk{defun comp3}  
\getchunk{defun compAdd}  
\getchunk{defun compAndDefine}  
\getchunk{defun compApplication}  
\getchunk{defun compApply}  
\getchunk{defun compApplyModemap}  
\getchunk{defun compArgumentConditions}  
\getchunk{defun compArgumentsAndTryAgain}  
\getchunk{defun compAtom}  
\getchunk{defun compAtomWithModemap}  
\getchunk{defun compAtSign}  
\getchunk{defun compBoolean}  
\getchunk{defun compCapsule}  
\getchunk{defun compCapsuleInner}  
\getchunk{defun compCapsuleItems}  
\getchunk{defun compCase}  
\getchunk{defun compCase1}  
\getchunk{defun compCat}  
\getchunk{defun compCategory}  
\getchunk{defun compCategoryItem}  
\getchunk{defun compCoerce}  
\getchunk{defun compCoerce1}  
\getchunk{defun compColon}  
\getchunk{defun compColonInside}  
\getchunk{defun compCons}  
\getchunk{defun compCons1}  
\getchunk{defun compConstruct}  
\getchunk{defun compConstructorCategory}  
\getchunk{defun compDefine}  
\getchunk{defun compDefine1}  
\getchunk{defun compDefineAddSignature}  
\getchunk{defun compDefineCapsuleFunction}  
\getchunk{defun compDefineCategory}  
\getchunk{defun compDefineCategory1}  
\getchunk{defun compDefineCategory2}  
\getchunk{defun compDefineFunctor}  
\getchunk{defun compDefineFunctor1}  
\getchunk{defun compDefineLisplib}  
\getchunk{defun compDefWhereClause}  
\getchunk{defun compElt}  
\getchunk{defun compExit}  
\getchunk{defun compExpression}  
\getchunk{defun compExpressionList}  
\getchunk{defun compForm}  
\getchunk{defun compForm1}  
\getchunk{defun compForm2}  
\getchunk{defun compForm3}  
\getchunk{defun compFormMatch}
```

```

\getchunk{defun compForMode}
\getchunk{defun compFormPartiallyBottomUp}
\getchunk{defun compFormWithModemap}
\getchunk{defun compFromIf}
\getchunk{defun compFunctorBody}
\getchunk{defun compHas}
\getchunk{defun compHasFormat}
\getchunk{defun compIf}
\getchunk{defun compile}
\getchunk{defun compileCases}
\getchunk{defun compileConstructor}
\getchunk{defun compileConstructor1}
\getchunk{defun compileDocumentation}
\getchunk{defun compileFileQuietly}
\getchunk{defun compile-lib-file}
\getchunk{defun compiler}
\getchunk{defun compilerDoit}
\getchunk{defun compilerDoitWithScreenedLispLib}
\getchunk{defun compileSpad2Cmd}
\getchunk{defun compileSpadLispCmd}
\getchunk{defun compileTimeBindingOf}
\getchunk{defun compImport}
\getchunk{defun compInternalFunction}
\getchunk{defun compIs}
\getchunk{defun compJoin}
\getchunk{defun compLambda}
\getchunk{defun compLeave}
\getchunk{defun compList}
\getchunk{defun compMacro}
\getchunk{defun compMakeCategoryObject}
\getchunk{defun compMakeDeclaration}
\getchunk{defun compMapCond}
\getchunk{defun compMapCond'}
\getchunk{defun compMapCond''}
\getchunk{defun compMapCondFun}
\getchunk{defun compNoStacking}
\getchunk{defun compNoStacking1}
\getchunk{defun compOrCroak}
\getchunk{defun compOrCroak1}
\getchunk{defun compOrCroak1,compactify}
\getchunk{defun compPretend}
\getchunk{defun compQuote}
\getchunk{defun compRepeatOrCollect}
\getchunk{defun compReduce}
\getchunk{defun compReduce1}
\getchunk{defun compReturn}
\getchunk{defun compSeq}
\getchunk{defun compSeqItem}
\getchunk{defun compSeq1}
\getchunk{defun compSetq}

```

```

\getchunk{defun compSetq1}
\getchunk{defun compSingleCapsuleItem}
\getchunk{defun compString}
\getchunk{defun compSubDomain}
\getchunk{defun compSubDomain1}
\getchunk{defun compSymbol}
\getchunk{defun compSubsetCategory}
\getchunk{defun compSuchthat}
\getchunk{defun compToApply}
\getchunk{defun compTopLevel}
\getchunk{defun compTuple2Record}
\getchunk{defun compTypeOf}
\getchunk{defun compUniquely}
\getchunk{defun compVector}
\getchunk{defun compWhere}
\getchunk{defun compWithMappingMode}
\getchunk{defun compWithMappingMode1}
\getchunk{defun constructMacro}
\getchunk{defun containsBang}
\getchunk{defun convert}
\getchunk{defun convertOpAlist2compilerInfo}
\getchunk{defun convertOrCroak}
\getchunk{defun current-char}
\getchunk{defun current-symbol}
\getchunk{defun current-token}

\getchunk{defun dbReadLines}
\getchunk{defun dbWriteLines}
\getchunk{defun decodeScripts}
\getchunk{defun deepestExpression}
\getchunk{defun def-rename}
\getchunk{defun disallowNilAttribute}
\getchunk{defun displayMissingFunctions}
\getchunk{defun displayPreCompilationErrors}
\getchunk{defun doIt}
\getchunk{defun doItIf}
\getchunk{defun dollarTran}
\getchunk{defun domainMember}
\getchunk{defun drop}

\getchunk{defun eltModemapFilter}
\getchunk{defun encodeItem}
\getchunk{defun encodeFunctionName}
\getchunk{defun EqualBarGensym}
\getchunk{defun escape-keywords}
\getchunk{defun escaped}
\getchunk{defun evalAndRwriteLispForm}
\getchunk{defun evalAndSub}
\getchunk{defun expand-tabs}
\getchunk{defun extendLocalLibdb}

```



```

\getchunk{defun extractCodeAndConstructTriple}

\getchunk{defun flattenSignatureList}
\getchunk{defun finalizeDocumentation}
\getchunk{defun finalizeLisplib}
\getchunk{defun fincomblock}
\getchunk{defun firstNonBlankPosition}
\getchunk{defun fixUpPredicate}
\getchunk{defun floatexpid}
\getchunk{defun formal2Pattern}
\getchunk{defun freelist}

\getchunk{defun get-a-line}
\getchunk{defun getAbbreviation}
\getchunk{defun getArgumentMode}
\getchunk{defun getArgumentModeOrMoan}
\getchunk{defun getCaps}
\getchunk{defun getCategoryOpsAndAtts}
\getchunk{defun getConstructorExports}
\getchunk{defun getConstructorOpsAndAtts}
\getchunk{defun getDomainsInScope}
\getchunk{defun getFormModemaps}
\getchunk{defun getFunctorOpsAndAtts}
\getchunk{defun getInverseEnvironment}
\getchunk{defun getMatchingRightPren}
\getchunk{defun getModemap}
\getchunk{defun getModemapList}
\getchunk{defun getModemapListFromDomain}
\getchunk{defun getOperationAList}
\getchunk{defun getScriptName}
\getchunk{defun getSignature}
\getchunk{defun getSignatureFromMode}
\getchunk{defun getSlotFromCategoryForm}
\getchunk{defun getSlotFromFunctor}
\getchunk{defun getSpecialCaseAssoc}
\getchunk{defun getSuccessEnvironment}
\getchunk{defun getTargetFromRhs}
\getchunk{defun get-token}
\getchunk{defun getToken}
\getchunk{defun getUnionMode}
\getchunk{defun getUniqueModemap}
\getchunk{defun getUniqueSignature}
\getchunk{defun genDomainOps}
\getchunk{defun genDomainViewList0}
\getchunk{defun genDomainViewList}
\getchunk{defun genDomainView}
\getchunk{defun giveFormalParametersValues}

\getchunk{defun hackforis}
\getchunk{defun hackforis1}

```

```

\getchunk{defun hasAplExtension}
\getchunk{defun hasFormalMapVariable}
\getchunk{defun hasFullSignature}
\getchunk{defun hasNoVowels}
\getchunk{defun hasSigInTargetCategory}
\getchunk{defun hasType}
\getchunk{defun htcharPosition}

\getchunk{defun indent-pos}
\getchunk{defun infixtok}
\getchunk{defun initialize-prepare}
\getchunk{defun initial-substring}
\getchunk{defun initial-substring-p}
\getchunk{defun initializeLisplib}
\getchunk{defun insertModemap}
\getchunk{defun interactiveModemapForm}
\getchunk{defun isCategoryPackageName}
\getchunk{defun is-console}
\getchunk{defun isDomainConstructorForm}
\getchunk{defun isDomainForm}
\getchunk{defun isDomainSubst}
\getchunk{defun isFunctor}
\getchunk{defun isListConstructor}
\getchunk{defun isMacro}
\getchunk{defun isSuperDomain}
\getchunk{defun isTokenDelimiter}
\getchunk{defun isUnionMode}

\getchunk{defun killColons}

\getchunk{defun line-advance-char}
\getchunk{defun line-at-end-p}
\getchunk{defun line-current-segment}
\getchunk{defun line-next-char}
\getchunk{defun line-past-end-p}
\getchunk{defun line-print}
\getchunk{defun line-new-line}
\getchunk{defun lispize}
\getchunk{defun lisplibDoRename}
\getchunk{defun lisplibWrite}
\getchunk{defun loadIfNecessary}
\getchunk{defun loadLibIfNecessary}

\getchunk{defun macroExpand}
\getchunk{defun macroExpandInPlace}
\getchunk{defun macroExpandList}
\getchunk{defun makeCategoryForm}
\getchunk{defun makeCategoryPredicates}
\getchunk{defun makeFunctorArgumentParameters}
\getchunk{defun makeSimplePredicateOrNil}

```

```

\getchunk{defun make-symbol-of}
\getchunk{defun match-advance-string}
\getchunk{defun match-current-token}
\getchunk{defun match-next-token}
\getchunk{defun match-string}
\getchunk{defun match-token}
\getchunk{defun maxSuperType}
\getchunk{defun mergeModemap}
\getchunk{defun mergeSignatureAndLocalVarAlists}
\getchunk{defun meta-syntax-error}
\getchunk{defun mkAbbrev}
\getchunk{defun mkAListOfExplicitCategoryOps}
\getchunk{defun mkCategoryPackage}
\getchunk{defun mkConstructor}
\getchunk{defun mkDatabasePred}
\getchunk{defun mkEvalableCategoryForm}
\getchunk{defun mkExplicitCategoryFunction}
\getchunk{defun mkList}
\getchunk{defun mkNewModemapList}
\getchunk{defun mkOpVec}
\getchunk{defun mkRepetitionAssoc}
\getchunk{defun mkUnion}
\getchunk{defun modifyModeStack}
\getchunk{defun modeEqual}
\getchunk{defun modeEqualSubst}
\getchunk{defun modemapPattern}
\getchunk{defun moveORsOutside}
\getchunk{defun mustInstantiate}

\getchunk{defun ncINTERPFILE}
\getchunk{defun newWordFrom}
\getchunk{defun next-char}
\getchunk{defun next-line}
\getchunk{defun next-tab-loc}
\getchunk{defun next-token}
\getchunk{defun newConstruct}
\getchunk{defun newDef2Def}
\getchunk{defun newIf2Cond}
\getchunk{defun newString2Words}
\getchunk{defun new2OldDefForm}
\getchunk{defun new2OldTran}
\getchunk{defun new2OldLisp}
\getchunk{defun nonblankloc}
\getchunk{defun NRTassocIndex}
\getchunk{defun NRTgetLocalIndex}
\getchunk{defun NRTgetLookupFunction}
\getchunk{defun NRTputInHead}
\getchunk{defun NRTputInTail}

\getchunk{defun optCall}

```

```

\getchunk{defun optCallEval}
\getchunk{defun optCallSpecially}
\getchunk{defun optCatch}
\getchunk{defun optCond}
\getchunk{defun optCONDtail}
\getchunk{defun optEQ}
\getchunk{defun optIF2COND}
\getchunk{defun optimize}
\getchunk{defun optimizeFunctionDef}
\getchunk{defun optional}
\getchunk{defun optLESSP}
\getchunk{defun optMINUS}
\getchunk{defun optMkRecord}
\getchunk{defun optPackageCall}
\getchunk{defun optPredicateIfTrue}
\getchunk{defun optQSMINUS}
\getchunk{defun optRECORDCOPY}
\getchunk{defun optRECORDELT}
\getchunk{defun optSETRECORDELT}
\getchunk{defun optSEQ}
\getchunk{defun optSPADCALL}
\getchunk{defun optSpecialCall}
\getchunk{defun optSuchthat}
\getchunk{defun optXLAMCond}
\getchunk{defun opt-}
\getchunk{defun orderByDependency}
\getchunk{defun orderPredicateItems}
\getchunk{defun orderPredTran}
\getchunk{defun outputComp}

\getchunk{defun PARSE-AnyId}
\getchunk{defun PARSE-Application}
\getchunk{defun parse-argument-designator}
\getchunk{defun parse-identifier}
\getchunk{defun parse-keyword}
\getchunk{defun parse-number}
\getchunk{defun parse-spadstring}
\getchunk{defun parse-string}
\getchunk{defun PARSE-Category}
\getchunk{defun PARSE-Command}
\getchunk{defun PARSE-CommandTail}
\getchunk{defun PARSE-Conditional}
\getchunk{defun PARSE-Data}
\getchunk{defun PARSE-ElseClause}
\getchunk{defun PARSE-Enclosure}
\getchunk{defun PARSE-Exit}
\getchunk{defun PARSE-Expr}
\getchunk{defun PARSE-Expression}
\getchunk{defun PARSE-Float}
\getchunk{defun PARSE-FloatBase}

```

```

\getchunk{defun PARSE-FloatBasePart}
\getchunk{defun PARSE-FloatExponent}
\getchunk{defun PARSE-FloatTok}
\getchunk{defun PARSE-Form}
\getchunk{defun PARSE-FormalParameter}
\getchunk{defun PARSE-FormalParameterTok}
\getchunk{defun PARSE-getSemanticForm}
\getchunk{defun PARSE-GlyphTok}
\getchunk{defun PARSE-Import}
\getchunk{defun PARSE-Infix}
\getchunk{defun PARSE-InfixWith}
\getchunk{defun PARSE-IntegerTok}
\getchunk{defun PARSE-Iterator}
\getchunk{defun PARSE-IteratorTail}
\getchunk{defun PARSE-Label}
\getchunk{defun PARSE-LabelExpr}
\getchunk{defun PARSE-Leave}
\getchunk{defun PARSE-LedPart}
\getchunk{defun PARSE-leftBindingPowerOf}
\getchunk{defun PARSE-Loop}
\getchunk{defun PARSE-Name}
\getchunk{defun PARSE-NBGlyphTok}
\getchunk{defun PARSE-NewExpr}
\getchunk{defun PARSE-NudPart}
\getchunk{defun PARSE-OpenBrace}
\getchunk{defun PARSE-OpenBracket}
\getchunk{defun PARSE-Operation}
\getchunk{defun PARSE-Option}
\getchunk{defun PARSE-Prefix}
\getchunk{defun PARSE-Primary}
\getchunk{defun PARSE-Primary1}
\getchunk{defun PARSE-PrimaryNoFloat}
\getchunk{defun PARSE-PrimaryOrQM}
\getchunk{defun PARSE-Qualification}
\getchunk{defun PARSE-Quad}
\getchunk{defun PARSE-Reduction}
\getchunk{defun PARSE-ReductionOp}
\getchunk{defun PARSE-Return}
\getchunk{defun PARSE-rightBindingPowerOf}
\getchunk{defun PARSE-ScriptItem}
\getchunk{defun PARSE-Scripts}
\getchunk{defun PARSE-Seg}
\getchunk{defun PARSE-Selector}
\getchunk{defun PARSE-SemiColon}
\getchunk{defun PARSE-Sequence}
\getchunk{defun PARSE-Sequence1}
\getchunk{defun PARSE-Sexpr}
\getchunk{defun PARSE-Sexpr1}
\getchunk{defun PARSE-SpecialCommand}
\getchunk{defun PARSE-SpecialKeyWord}

```

```

\getchunk{defun PARSE-Statement}
\getchunk{defun PARSE-String}
\getchunk{defun PARSE-Suffix}
\getchunk{defun PARSE-TokenCommandTail}
\getchunk{defun PARSE-TokenList}
\getchunk{defun PARSE-TokenOption}
\getchunk{defun PARSE-TokTail}
\getchunk{defun PARSE-VarForm}
\getchunk{defun PARSE-With}
\getchunk{defun parsepiles}
\getchunk{defun parseAnd}
\getchunk{defun parseAtom}
\getchunk{defun parseAtSign}
\getchunk{defun parseCategory}
\getchunk{defun parseCoerce}
\getchunk{defun parseColon}
\getchunk{defun parseConstruct}
\getchunk{defun parseDEF}
\getchunk{defun parseDollarGreaterEqual}
\getchunk{defun parseDollarGreaterThan}
\getchunk{defun parseDollarLessEqual}
\getchunk{defun parseDollarNotEqual}
\getchunk{defun parseDropAssertions}
\getchunk{defun parseEquivalence}
\getchunk{defun parseExit}
\getchunk{defun postFlatten}
\getchunk{defun postFlattenLeft}
\getchunk{defun postForm}
\getchunk{defun parseGreaterEqual}
\getchunk{defun parseGreaterThan}
\getchunk{defun parseHas}
\getchunk{defun parseHasRhs}
\getchunk{defun parseIf}
\getchunk{defun parseIf,ifTran}
\getchunk{defun parseImplies}
\getchunk{defun parseIn}
\getchunk{defun parseInBy}
\getchunk{defun parseIs}
\getchunk{defun parseIsnt}
\getchunk{defun parseJoin}
\getchunk{defun parseLeave}
\getchunk{defun parseLessEqual}
\getchunk{defun parseLET}
\getchunk{defun parseLETD}
\getchunk{defun parseLhs}
\getchunk{defun parseMDEF}
\getchunk{defun parseNot}
\getchunk{defun parseNotEqual}
\getchunk{defun parseOr}
\getchunk{defun parsePretend}

```

```

\getchunk{defun parseprint}
\getchunk{defun parseReturn}
\getchunk{defun parseSegment}
\getchunk{defun parseSeq}
\getchunk{defun parseTran}
\getchunk{defun parseTranCheckForRecord}
\getchunk{defun parseTranList}
\getchunk{defun parseTransform}
\getchunk{defun parseType}
\getchunk{defun parseVCONS}
\getchunk{defun parseWhere}
\getchunk{defun Pop-Reduction}
\getchunk{defun postAdd}
\getchunk{defun postAtom}
\getchunk{defun postAtSign}
\getchunk{defun postBigFloat}
\getchunk{defun postBlock}
\getchunk{defun postBlockItem}
\getchunk{defun postBlockItemList}
\getchunk{defun postCapsule}
\getchunk{defun postCategory}
\getchunk{defun postcheck}
\getchunk{defun postCollect}
\getchunk{defun postCollect,finish}
\getchunk{defun postColon}
\getchunk{defun postColonColon}
\getchunk{defun postComma}
\getchunk{defun postConstruct}
\getchunk{defun postDef}
\getchunk{defun postDefArgs}
\getchunk{defun postError}
\getchunk{defun postExit}
\getchunk{defun postIf}
\getchunk{defun postin}
\getchunk{defun postIn}
\getchunk{defun postInSeq}
\getchunk{defun postIteratorList}
\getchunk{defun postJoin}
\getchunk{defun postMakeCons}
\getchunk{defun postMapping}
\getchunk{defun postMDef}
\getchunk{defun postOp}
\getchunk{defun postPretend}
\getchunk{defun postQUOTE}
\getchunk{defun postReduce}
\getchunk{defun postRepeat}
\getchunk{defun postScripts}
\getchunk{defun postScriptsForm}
\getchunk{defun postSemiColon}
\getchunk{defun postSignature}

```

```

\getchunk{defun postSlash}
\getchunk{defun postTran}
\getchunk{defun postTranList}
\getchunk{defun postTranScripts}
\getchunk{defun postTranSegment}
\getchunk{defun postTransform}
\getchunk{defun postTransformCheck}
\getchunk{defun postTuple}
\getchunk{defun postTupleCollect}
\getchunk{defun postType}
\getchunk{defun postWhere}
\getchunk{defun postWith}
\getchunk{defun print-package}
\getchunk{defun preparse}
\getchunk{defun preparse1}
\getchunk{defun preparse-echo}
\getchunk{defun preparseReadLine}
\getchunk{defun preparseReadLine1}
\getchunk{defun primitiveType}
\getchunk{defun print-defun}
\getchunk{defun processFunctor}
\getchunk{defun purgeNewConstructorLines}
\getchunk{defun push-reduction}
\getchunk{defun putDomainsInScope}
\getchunk{defun putInLocalDomainReferences}

\getchunk{defun quote-if-string}

\getchunk{defun read-a-line}
\getchunk{defun recompile-lib-file-if-necessary}
\getchunk{defun recordAttributeDocumentation}
\getchunk{defun recordDocumentation}
\getchunk{defun recordHeaderDocumentation}
\getchunk{defun recordSignatureDocumentation}
\getchunk{defun replaceExitEtc}
\getchunk{defun removeBackslashes}
\getchunk{defun removeSuperfluousMapping}
\getchunk{defun replaceVars}
\getchunk{defun resolve}
\getchunk{defun reportOnFunctorCompilation}
\getchunk{defun /rf}
\getchunk{defun /rq}
\getchunk{defun /rf-1}
\getchunk{defun /RQ,LIB}
\getchunk{defun rwriteLispForm}

\getchunk{defun screenLocalLine}
\getchunk{defun setDefOp}
\getchunk{defun seteltModemapFilter}
\getchunk{defun setqMultiple}

```



```

\getchunk{defun setqMultipleExplicit}
\getchunk{defun setqSetelt}
\getchunk{defun setqSingle}
\getchunk{defun signatureTran}
\getchunk{defun skip-blanks}
\getchunk{defun skip-ifblock}
\getchunk{defun skip-to-endif}
\getchunk{defun spad}
\getchunk{defun spadCompileOrSetq}
\getchunk{defun spad-fixed-arg}
\getchunk{defun spadSysBranch}
\getchunk{defun spadSysChoose}
\getchunk{defun splitEncodedFunctionName}
\getchunk{defun stack-clear}
\getchunk{defun stack-load}
\getchunk{defun stack-pop}
\getchunk{defun stack-push}
\getchunk{defun storeblanks}
\getchunk{defun string2BootTree}
\getchunk{defun stripOffArgumentConditions}
\getchunk{defun stripOffSubdomainConditions}
\getchunk{defun subrname}
\getchunk{defun substituteCategoryArguments}
\getchunk{defun substituteIntoFunctorModemap}
\getchunk{defun substNames}
\getchunk{defun substVars}
\getchunk{defun s-process}

\getchunk{defun token-install}
\getchunk{defun token-lookahead-type}
\getchunk{defun token-print}
\getchunk{defun transDoc}
\getchunk{defun transDocList}
\getchunk{defun transformAndRecheckComments}
\getchunk{defun transformOperationAlist}
\getchunk{defun transImplementation}
\getchunk{defun transIs}
\getchunk{defun transIs1}
\getchunk{defun translablel}
\getchunk{defun translablel1}
\getchunk{defun TruthP}
\getchunk{defun try-get-token}
\getchunk{defun tuple2List}

\getchunk{defun uncons}
\getchunk{defun underscore}
\getchunk{defun unget-tokens}
\getchunk{defun unknownTypeError}
\getchunk{defun unloadOneConstructor}
\getchunk{defun unTuple}

```

```
\getchunk{defun updateCategoryFrameForCategory}  
\getchunk{defun updateCategoryFrameForConstructor}  
  
\getchunk{defun whoOwns}  
\getchunk{defun wrapDomainSub}  
\getchunk{defun writeLib1}  
  
\getchunk{postvars}
```

—————▶

Bibliography

- [Floy63] R. W. Floyd. Semantic analysis and operator precedence. *JACM*, 10(3):316–333, 1963.
- [Knut92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, Stanford CA, 1992, 0-937073-81-4.
- [Prat73] Vaughan R. Pratt. Top down operator precedence. In *Proc. 1st annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL’73, pages 41–51, 1973.

Link: <http://hall.org.ua/halls/wizzard/pdf/Vaughan.Pratt.TDOP.pdf>

Index

- [+- >](#), [325](#)
 - [defplist](#), [325](#)
- [- >](#), [394](#)
 - [defplist](#), [394](#)
- [<=](#), [129](#)
 - [defplist](#), [129](#)
- [==>](#), [395](#)
 - [defplist](#), [395](#)
- [=>](#), [390](#)
 - [defplist](#), [390](#)
- [>](#), [115](#)
 - [defplist](#), [115](#)
- [>=](#), [114](#), [115](#)
 - [defplist](#), [114](#), [115](#)
- [*comp370-apply*](#)
 - [usedby](#) [spad](#), [572](#)
- [*eof*](#)
 - [usedby](#) [read-a-line](#), [629](#)
 - [usedby](#) [spad](#), [572](#)
- [*fileactq-apply*](#)
 - [usedby](#) [spad](#), [572](#)
- [*lisp-bin-filetype*](#)
 - [usedby](#) [recompile-lib-file-if-necessary](#), [625](#)
- [*standard-output*](#)
 - [usedby](#) [compileFileQuietly](#), [626](#)
- [*terminal-io*](#)
 - [usedby](#) [is-console](#), [553](#)
- [,,](#), [386](#)
 - [defplist](#), [386](#)
- [-](#), [241](#)
 - [defplist](#), [241](#)
- [/](#), [401](#)
 - [defplist](#), [401](#)
- [/RQ,LIB](#), [569](#)
 - [calledby](#) [compilerDoit](#), [568](#)
 - [calls](#) [/rf-1](#), [569](#)
 - [calls](#) [echo-meta\[5\]](#), [569](#)
- [uses](#) [\\$lisplib](#), [570](#)
 - [defun](#), [569](#)
- [/editfile](#)
 - [usedby](#) [/rf-1](#), [570](#)
 - [usedby](#) [compAdd](#), [276](#)
 - [usedby](#) [compFunctorBody](#), [172](#)
 - [usedby](#) [compileSpad2Cmd](#), [563](#)
 - [usedby](#) [compiler](#), [561](#)
 - [usedby](#) [initializeLisplib](#), [206](#)
 - [usedby](#) [spad](#), [572](#)
- [/major-version](#)
 - [usedby](#) [initializeLisplib](#), [206](#)
- [/rf](#), [569](#)
 - [calls](#) [/rf-1](#), [569](#)
 - [uses](#) [echo-meta](#), [569](#)
 - [defun](#), [569](#)
- [/rf-1](#), [570](#)
 - [calledby](#) [/RQ,LIB](#), [569](#)
 - [calledby](#) [/rf](#), [569](#)
 - [calledby](#) [/rq](#), [569](#)
 - [calls](#) [makeInputFilename\[5\]](#), [570](#)
 - [calls](#) [ncINTERPFILE](#), [570](#)
 - [uses](#) [/editfile](#), [570](#)
 - [uses](#) [echo-meta](#), [570](#)
 - [defun](#), [570](#)
- [/rf\[5\]](#)
 - [called by](#) [compilerDoit](#), [568](#)
- [/rq](#), [569](#)
 - [calls](#) [/rf-1](#), [569](#)
 - [uses](#) [echo-meta](#), [569](#)
 - [defun](#), [569](#)
- [/rq\[5\]](#)
 - [called by](#) [compilerDoit](#), [568](#)
- [:](#), [108](#), [294](#), [384](#)
 - [defplist](#), [108](#), [294](#), [384](#)
- ::, [107](#), [362](#), [385](#)
 - [defplist](#), [107](#), [362](#), [385](#)

- `:BF:`, 379
 - `defplist`, 379
- `;`, 399
 - `defplist`, 399
- `==`, 388
 - `defplist`, 388
- `$/editfile`
 - local ref `finalizeLisplib`, 207
- `$AttrLst`
 - local def `buildLibdb`, 476
- `$BasicPredicates`, 231
 - local ref `optPredicateIfTrue`, 231
 - `defvar`, 231
- `$Boolean`
 - local ref `compArgumentConditions`, 170
 - local ref `compHas`, 313
 - local ref `doItIf`, 285
 - usedby `compCase1`, 288
 - usedby `compIf`, 315
 - usedby `compIs`, 323
 - usedby `compReduce1`, 331
 - usedby `compRepeatOrCollect`, 333
 - usedby `compSubDomain1`, 351
 - usedby `compSuchthat`, 353
 - usedby `compSymbol`, 598
- `$CapsuleDomainsInScope`
 - local def `compDefineCapsuleFunction`, 156
 - local def `putDomainsInScope`, 255
 - local ref `getDomainsInScope`, 254
- `$CapsuleModemapFrame`
 - local def `addModemapKnown`, 273
 - local def `addModemap`, 273
 - local def `compDefineCapsuleFunction`, 156
 - local ref `addModemap`, 273
- `$CategoryFrame`
 - local def `updateCategoryFrameForCategory`, 121
 - local def `updateCategoryFrameForConstructor`, 120
 - local ref `isFunctor`, 253
 - local ref `loadLibIfNecessary`, 119
 - local ref `mkEvaluableCategoryForm`, 182
 - local ref `updateCategoryFrameForCategory`, 121
 - local ref `updateCategoryFrameForConstructor`, 120
- usedby `compDefineFunctor1`, 149
- usedby `compSubDomain1`, 351
- usedby `compWithMappingMode1`, 615
- usedby `parseHasRhs`, 118
- usedby `parseHas`, 116
- `$CategoryNames`
 - local ref `mkEvaluableCategoryForm`, 182
- `$Category`
 - local ref `augModemapsFromDomain`, 256
 - local ref `compApplication`, 603
 - local ref `compFocompFormWithModemap`, 610
 - local ref `compMakeCategoryObject`, 213
 - local ref `mkEvaluableCategoryForm`, 182
 - usedby `compConstructorCategory`, 301
 - usedby `compDefine1`, 145
 - usedby `compJoin`, 324
- `$CheckVectorList`
 - usedby `compDefineFunctor1`, 149
 - usedby `displayMissingFunctions`, 220
- `$ConditionalOperators`
 - usedby `genDomainOps`, 225
 - usedby `makeFunctorArgumentParameters`, 221
- `$ConstructorCache`
 - local ref `compileConstructor1`, 188
- `$ConstructorNames`
 - usedby `compDefine1`, 146
- `$DefLst`
 - local def `buildLibdb`, 476
- `$DomLst`
 - local def `buildLibdb`, 476
- `$DomainFrame`
 - usedby `s-process`, 582
- `$DomainsInScope`
 - local ref `compDefineCapsuleFunction`, 156
- `$DoubleFloat`
 - usedby `primitiveType`, 597
- `$DummyFunctorNames`
 - local ref `augModemapsFromDomain`, 256
 - local ref `mustInstantiate`, 293
- `$EchoLineStack`
 - local def `preparseReadLine1`, 93
 - local ref `preparse-echo`, 95
 - usedby `fincomblock`, 552
- `$EchoLines`

- usedby ncINTERPFILE, 624
- \$EmptyEnvironment
 - local ref augLisplibModemapsFromCategory, 192
 - local ref compHasFormat, 313
 - local ref getInverseEnvironment, 320
 - local ref getSuccessEnvironment, 319
 - usedby genDomainViewList, 223
 - usedby s-process, 582
- \$EmptyMode, 176
 - local def NRTgetLocalIndex, 215
 - local ref coerceEasy, 356
 - local ref compApply, 592
 - local ref compHasFormat, 313
 - local ref compToApply, 602
 - local ref compileDocumentation, 170
 - local ref doIt, 281
 - local ref getSuccessEnvironment, 319
 - local ref makeCategoryForm, 297
 - local ref mkEvalableCategoryForm, 182
 - local ref resolve, 365
 - local ref setqMultipleExplicit, 343
 - local ref setqMultiple, 341
 - usedby compAdd, 276
 - usedby compArgumentsAndTryAgain, 614
 - usedby compCase1, 288
 - usedby compColonInside, 594
 - usedby compCons1, 298
 - usedby compDefine1, 146
 - usedby compDefineAddSignature, 147
 - usedby compDefineCategory1, 162
 - usedby compForm1, 601
 - usedby compForm2, 608
 - usedby compIs, 323
 - usedby compMacro, 327
 - usedby compNoStacking, 588
 - usedby compPretend, 329
 - usedby compSetq1, 340
 - usedby compSubDomain1, 351
 - usedby compWhere, 355
 - usedby compWithMappingModel1, 615
 - usedby primitiveType, 597
 - usedby s-process, 581
 - usedby setqSingle, 345
 - defvar, 176
- \$EmptyVector
 - usedby compVector, 354
- \$Exit
 - local ref coerceEasy, 356
- \$Expression
 - local ref coerceExtraHard, 359
 - local ref compExpressionList, 607
 - local ref outputComp, 347
 - usedby compAtom, 595
 - usedby compForm1, 601
 - usedby compSymbol, 598
- \$FormalMapVariableList, 270
 - local ref applyMapping, 591
 - local ref buildLibAttr, 483
 - local ref compDefineCategory2, 164
 - local ref compFocompFormWithModemap, 610
 - local ref compHasFormat, 313
 - local ref finalizeDocumentation, 491
 - local ref finalizeLisplib, 208
 - local ref getSignatureFromMode, 304
 - local ref getSlotFromCategoryForm, 210
 - local ref interactiveModemapForm, 195
 - local ref isDomainConstructorForm, 349
 - local ref substVars, 203
 - usedby compColon, 295
 - usedby compDefineFunctor1, 149
 - usedby compSymbol, 598
 - usedby compTypeOf, 593
 - usedby compWithMappingModel1, 615
 - usedby makeCategoryPredicates, 179
 - usedby mkCategoryPackage, 180
 - usedby mkOpVec, 226
 - usedby substNames, 271
 - defvar, 270
- \$GensymAssoc
 - local def EqualBarGensym, 248
 - local ref EqualBarGensym, 248
- \$HTlinks
 - local ref checkRecordHash, 507
- \$HTlisplinks
 - local ref checkRecordHash, 507
- \$HTmacros
 - local ref checkAddMacros, 527
- \$HTspadmacros
 - local ref checkFixCommonProblem, 505
- \$Index

- usedby s-process, 581
- \$Information
 - local ref compMapCond", 261
- \$InitialDomainsInScope
 - usedby spad, 572
- \$InteractiveFrame
 - usedby s-process, 582
 - usedby spad, 572
- \$InteractiveMode
 - local def addConstructorModemaps, 258
 - local ref addModemap, 273
 - local ref coerce, 356
 - local ref displayPreCompilationErrors, 542
 - local ref isFunctor, 254
 - local ref loadLibIfNecessary, 119
 - local ref mkNewModemapList, 266
 - local ref optCatch, 245
 - local ref optSPADCALL, 243
 - usedby bumperrorcount, 543
 - usedby compileFileQuietly, 626
 - usedby compileSpad2Cmd, 563
 - usedby dollarTran, 462
 - usedby parseAnd, 105
 - usedby parseAtSign, 106
 - usedby parseCoerce, 108
 - usedby parseColon, 108
 - usedby parseHas, 116
 - usedby parseIf,ifTran, 122
 - usedby parseNot, 132
 - usedby postBigFloat, 380
 - usedby postDef, 389
 - usedby postError, 374
 - usedby postMDef, 395
 - usedby postReduce, 397
 - usedby spad, 572
 - usedby tuple2List, 547
- \$LocalDomainAlist
 - local def doIt, 281
 - local ref doIt, 281
 - usedby compDefineFunctor1, 149
- \$LocalFrame
 - usedby s-process, 582
- \$NRTaddForm
 - local ref NRTassocIndex, 346
 - local ref NRTgetLocalIndex, 215
 - usedby compAdd, 276
 - usedby compDefineFunctor1, 149
 - usedby compFunctorBody, 172
 - usedby compSubDomain, 350
- \$NRTaddList
 - usedby compDefineFunctor1, 149
- \$NRTattributeAlist
 - usedby compDefineFunctor1, 149
- \$NRTbase
 - local def NRTgetLocalIndex, 215
 - local ref NRTassocIndex, 346
 - usedby compDefineFunctor1, 149
- \$NRTdeltaLength
 - local ref NRTassocIndex, 346
 - local ref NRTgetLocalIndex, 215
 - usedby compDefineFunctor1, 149
- \$NRTdeltaListComp
 - local ref NRTgetLocalIndex, 215
 - usedby compDefineFunctor1, 149
- \$NRTdeltaList
 - local ref NRTassocIndex, 346
 - local ref NRTgetLocalIndex, 215
 - usedby compDefineFunctor1, 149
- \$NRTderivedTargetIfTrue
 - usedby compTopLevel, 583
- \$NRTdomainFormList
 - usedby compDefineFunctor1, 149
- \$NRTloadTimeAlist
 - usedby compDefineFunctor1, 149
- \$NRTopt
 - local ref doIt, 281
- \$NRTslot1Info
 - usedby compDefineFunctor1, 149
- \$NRTslot1PredicateList
 - local def finalizeLisplib, 208
 - usedby compDefineFunctor1, 149
- \$NegativeInteger
 - usedby primitiveType, 597
- \$NoValueMode, 176
 - local ref coerceEasy, 356
 - local ref compAtomWithModemap, 596
 - local ref resolve, 365
 - local ref setqMultipleExplicit, 343
 - local ref setqMultiple, 341
 - usedby compDefine1, 146
 - usedby compImport, 322
 - usedby compMacro, 327

- usedby compRepeatOrCollect, 333
 - usedby compSeq1, 337
 - usedby compSymbol, 598
 - usedby setqSingle, 345
 - defvar, 176
- \$NoValue
 - usedby compSymbol, 598
 - usedby parseAtom, 102
- \$NonMentionableDomainNames
 - local ref doIt, 281
- \$NonNegativeInteger
 - usedby primitiveType, 597
- \$NumberOfArgsIfInteger
 - usedby compForm1, 601
- \$One
 - usedby compElt, 310
- \$OpLst
 - local def buildLibdb, 477
- \$PakLst
 - local def buildLibdb, 476
- \$PatternVariableList
 - local ref augLisplibModemapsFromCategory, 192
 - local ref augmentLisplibModemapsFromFunctor, 216
 - local ref formal2Pattern, 218
 - local ref interactiveModemapForm, 195
 - local ref modemapPattern, 203
- \$PolyMode
 - usedby s-process, 581
- \$PositiveInteger
 - usedby primitiveType, 597
- \$PrettyPrint
 - usedby print-defun, 584
- \$PrintOnly
 - usedby s-process, 582
- \$QuickCode
 - local ref doIt, 281
 - local ref optCall, 234
 - local ref optSpecialCall, 236
 - local ref putInLocalDomainReferences, 189
 - usedby compDefineFunctor1, 150
 - usedby compWithMappingMode1, 615
 - usedby compileSpad2Cmd, 563
- \$QuickLet
 - usedby compileSpad2Cmd, 563
- usedby setqSingle, 344
- \$ReadingFile
 - usedby ncINTERPFILE, 624
- \$Representation
 - local def doIt, 281
 - local ref doIt, 281
 - usedby compDefineFunctor1, 149
 - usedby compNoStacking, 588
- \$Rep
 - local ref coerce, 356
 - local ref mkUnion, 366
- \$SpecialDomainNames
 - local ref isDomainForm, 348
 - usedby addEmptyCapsuleIfNecessary, 177
- \$StringCategory
 - usedby compString, 349
- \$String
 - local ref coerceHard, 358
 - local ref resolve, 365
 - usedby primitiveType, 597
- \$Symbol
 - usedby compSymbol, 598
- \$TranslateOnly
 - usedby s-process, 582
- \$Translation
 - usedby s-process, 582
- \$TriangleVariableList
 - local ref compDefineCategory2, 164
 - usedby compForm2, 608
 - usedby makeCategoryPredicates, 180
- \$Undef
 - local ref optSpecialCall, 236
- \$VariableCount
 - usedby s-process, 582
- \$Void
 - local ref coerceEasy, 356
- \$Zero
 - usedby compElt, 310
- \$abbreviationTable
 - local def getAbbreviation, 302
 - local ref getAbbreviation, 302
- \$addFormLhs
 - usedby compAdd, 276
 - usedby compSubDomain, 350
- \$addForm
 - local def compDefineCategory2, 164

- usedby compAdd, 276
 - usedby compCapsuleInner, 278
 - usedby compDefineFunctor1, 149
 - usedby compSubDomain, 350
- \$algebraOutputStream
 - local ref compDefineLisplib, 167
- \$alternateViewList
 - usedby makeFunctorArgumentParameters, 221
- \$argl
 - local def checkComments, 496
 - local def transDoc, 494
 - local ref checkDecorate, 502
 - local ref checkRewrite, 498
- \$argumentConditionList
 - local def addArgumentConditions, 304
 - local def compArgumentConditions, 170
 - local def compDefineCapsuleFunction, 156
 - local def stripOffArgumentConditions, 306
 - local def stripOffSubdomainConditions, 305
 - local ref addArgumentConditions, 304
 - local ref compArgumentConditions, 170
 - local ref stripOffArgumentConditions, 306
 - local ref stripOffSubdomainConditions, 305
- \$atList
 - local def compCategory, 290
 - local ref compCategoryItem, 291
 - local ref compCategory, 290
- \$attribute?
 - local def transDoc, 494
 - local ref checkComments, 496
 - local ref transDoc, 494
- \$attributesName
 - usedby compDefineFunctor1, 149
- \$base
 - local def augModemapsFromCategoryRep, 271
 - local def augModemapsFromCategory, 265
- \$beginEndList
 - local ref checkBeginEnd, 500
- \$bindings
 - local def compApplyModemap, 259
 - local ref compApplyModemap, 259
 - local ref compMapCond, 260
- \$body
 - local ref addArgumentConditions, 304
- \$bootStrapMode
 - local ref coerceHard, 358
 - local ref optCall, 234
 - usedby comp2, 589
 - usedby compAdd, 276
 - usedby compCapsule, 278
 - usedby compColon, 295
 - usedby compDefineCategory1, 162
 - usedby compDefineFunctor1, 149
 - usedby compFunctorBody, 172
- \$boot
 - local def string2BootTree, 78
 - local ref PARSE-FloatTok, 450
 - local ref PARSE-Primary1, 432
 - local ref aplTran1, 405
 - local ref postAtom, 371
 - usedby PARSE-Quad, 436
 - usedby PARSE-Selector, 430
 - usedby PARSE-TokTail, 427
 - usedby aplTran, 405
 - usedby postBigFloat, 380
 - usedby postColonColon, 385
 - usedby postDef, 389
 - usedby postForm, 374
 - usedby postIf, 391
 - usedby postMDef, 395
 - usedby quote-if-string, 454
 - usedby spad, 572
 - usedby tuple2List, 547
- \$byConstructors, 626
 - local ref preparsel, 87
 - usedby compilerDoit, 568
 - defvar, 626
- \$byteAddress
 - usedby compDefineFunctor1, 149
- \$byteVec
 - usedby compDefineFunctor1, 149
- \$catLst
 - local def buildLibdb, 476
- \$categoryPredicateList
 - usedby compDefineCategory1, 162
 - usedby mkCategoryPackage, 180
- \$charBlank
 - local ref checkAddSpaceSegments, 528

- local ref checkAddSpaces, 528
 - local ref checkLookForLeftBrace, 531
 - local ref checkSkipBlanks, 533
 - local ref checkTrim, 515
 - local ref newWordFrom, 538
- \$charDelimiters
 - local ref checkSkipOpToken, 524
- \$charEscapeList
 - local ref checkAddBackSlashes, 526
- \$charExclusions
 - local ref checkDecorate, 502
- \$charFauxNewline
 - local ref checkAddSpaces, 528
 - local ref checkIndentedLines, 522
 - local ref newWordFrom, 538
- \$charIdentifierEndings
 - local ref checkAlphabetic, 529
- \$charPlus
 - local ref checkTrim, 515
- \$charSplitList
 - local ref checkSplitOn, 534
- \$checkErrorFlag
 - local def checkComments, 496
 - local def checkDocError, 516
 - local ref checkComments, 496
 - local ref checkDocError, 516
 - local ref checkRewrite, 498
- \$checkPrenAList
 - local ref checkBalance, 499
 - local ref checkTransformFirsts, 512
- \$checkingXmptex?
 - local def transformAndRecheckComments, 495
 - local ref checkDecorateForHt, 504
 - local ref checkDecorate, 502
 - local ref checkRewrite, 498
- \$clamList
 - local def compileConstructor1, 188
 - local ref compileConstructor1, 188
- \$comblocklist, 551
 - local def collectComBlock, 469
 - local def recordHeaderDocumentation, 470
 - local ref collectAndDeleteAssoc, 470
 - local ref finalizeDocumentation, 490
 - local ref recordHeaderDocumentation, 470
 - usedby fincomblock, 552
 - usedby preparse, 86
 - defvar, 551
- \$compErrorMessageStack
 - usedby compOrCroak1, 587
- \$compForModeIfTrue
 - local def compForMode, 325
 - usedby compSymbol, 598
- \$compStack
 - local def compOrCroak1, 587
 - local ref compNoStacking1, 588
 - local ref compNoStacking, 588
 - local ref comp, 587
- \$compTimeSum
 - usedby compTopLevel, 583
- \$compUniquelyIfTrue
 - local def compUniquely, 613
 - local ref compForm3, 609
 - usedby s-process, 581
- \$compileDocumentation
 - local ref checkDocError1, 505
 - local ref compDefineLisplib, 167
- \$compileOnlyCertainItems
 - local ref compDefineCapsuleFunction, 156
 - local ref compile, 173
 - usedby compDefineFunctor1, 149
 - usedby compileSpad2Cmd, 563
- \$condAList
 - usedby compDefineFunctor1, 149
- \$conform
 - local def buildLibdb, 476
 - local ref buildLibAttr, 483
 - local ref buildLibOp, 482
 - local ref buildLibdbConEntry, 480
 - local ref buildLibdb, 476
- \$conname
 - local def buildLibdbConEntry, 480
 - local def buildLibdb, 476
 - local ref buildLibAttr, 483
- \$constructorLineNumber
 - usedby preparse, 86
- \$constructorName
 - local ref checkDocError, 516
 - local ref checkDocMessage, 518
 - local ref transDocList, 493
- \$constructorsSeen, 626
 - local ref preparse1, 87

- usedby compilerDoit, 568
 - defvar, 626
- \$createLocalLibDb
 - local ref extendLocalLibdb, 475
- \$currentFunction
 - usedby s-process, 581
- \$currentLine
 - usedby s-process, 582
- \$currentSysList
 - local ref checkRecordHash, 507
- \$defOp
 - usedby parseTransform, 101
 - usedby postError, 374
 - usedby postTransformCheck, 373
 - usedby setDefOp, 404
- \$definition
 - local def compDefineCategory2, 164
 - local ref compDefineCategory2, 164
- \$defstack, 411
 - defvar, 411
- \$devaluateList
 - local ref NRTputInTail, 189
- \$doNotCompileJustPrint
 - local ref compile, 173
- \$docList
 - local def recordDocumentation, 469
 - local ref finalizeDocumentation, 490
 - usedby postDef, 389
 - usedby preparse, 86
- \$doc
 - local def buildLibdbConEntry, 480
 - local def buildLibdb, 476
 - local ref buildLibAttr, 483
 - local ref buildLibOp, 482
- \$domainShell
 - local def compDefineCategory2, 164
 - local ref augLisplibModemapsFromCategory, 192
 - local ref hasSigInTargetCategory, 309
 - usedby compDefineCategory, 162
 - usedby compDefineFunctor1, 149
 - usedby compDefineFunctor, 148
 - usedby getOperationAlist, 270
- \$echolinestack, 81
 - local ref preparse1, 87
 - usedby initialize-preparse, 82
- defvar, 81
- \$elt
 - local def putInLocalDomainReferences, 189
 - local ref NRTputInHead, 190
 - local ref NRTputInTail, 189
- \$endTestList
 - usedby compReduce1, 331
- \$envHashTable
 - usedby compTopLevel, 583
- \$env
 - usedby displayMissingFunctions, 220
- \$erase
 - local ref initializeLisplib, 206
- \$exitModeStack
 - usedby compExit, 312
 - usedby compLeave, 327
 - usedby compOrCroak1, 587
 - usedby compRepeatOrCollect, 333
 - usedby compReturn, 335
 - usedby compSeq1, 336
 - usedby compSeq, 336
 - usedby comp, 587
 - usedby modifyModeStack, 622
 - usedby s-process, 581
- \$exitMode
 - local ref coerceExit, 361
 - usedby s-process, 581
- \$exposeFlagHeading
 - local def checkDocError, 516
 - local def transformAndRecheckComments, 495
 - local ref checkDocError, 516
 - local ref transformAndRecheckComments, 495
- \$exposeFlag
 - local ref checkDocError, 516
 - local ref whoOwns, 539
- \$exposed?
 - local def buildLibdbConEntry, 480
 - local def buildLibdb, 476
 - local ref buildLibAttr, 483
 - local ref buildLibOp, 482
 - local ref buildLibdbConEntry, 480
- \$extraParms
 - local def compDefineCategory2, 164

- local ref compDefineCategory2, 164
- \$e
 - local def NRTgetLocalIndex, 215
 - local def addEltModemap, 265
 - local def augmentLisplibModemapsFrom-Functor, 217
 - local def coerceHard, 358
 - local def compApplyModemap, 259
 - local def compCapsuleItems, 280
 - local def compHas, 313
 - local def doItIf, 285
 - local def doIt, 281
 - local def mkEvaluableCategoryForm, 182
 - local ref addModemapKnown, 272
 - local ref addModemap, 273
 - local ref augmentLisplibModemapsFrom-Functor, 216
 - local ref coerceHard, 358
 - local ref compApplyModemap, 259
 - local ref compCapsuleItems, 280
 - local ref compHasFormat, 313
 - local ref compHas, 313
 - local ref compMakeCategoryObject, 213
 - local ref compMapCond", 261
 - local ref compSingleCapsuleItem, 280
 - local ref compileDocumentation, 170
 - local ref compile, 174
 - local ref doItIf, 285
 - local ref doIt, 281
 - local ref finalizeDocumentation, 490
 - local ref getSignature, 307
 - local ref getSlotFromFunctor, 212
 - local ref mkAListOfExplicitCategoryOps, 193
 - local ref mkDatabasePred, 218
 - local ref mkEvaluableCategoryForm, 182
 - local ref optCallSpecially, 235
 - local ref signatureTran, 198
 - usedby comp3, 590
 - usedby compReduce1, 331
 - usedby genDomainOps, 225
 - usedby genDomainView, 224
 - usedby getOperationAlist, 270
 - usedby mkCategoryPackage, 180
 - usedby s-process, 582
- \$fcopy
 - local ref compileDocumentation, 169
- \$filep
 - local ref compDefineLisplib, 167
- \$finalEnv
 - local def compDefineCapsuleFunction, 156
 - local def replaceExitEtc, 337
 - local ref replaceExitEtc, 337
 - usedby compSeq1, 337
- \$forceAdd
 - local ref mergeModemap, 267
 - local ref mkNewModemapList, 266
 - usedby compTopLevel, 583
 - usedby makeFunctorArgumentParameters, 221
- \$formalArgList
 - local def compDefineCapsuleFunction, 156
 - local def compDefineCategory2, 164
 - local ref NRTgetLocalIndex, 215
 - local ref applyMapping, 591
 - local ref compDefineCapsuleFunction, 156
 - local ref compDefineCategory2, 164
 - usedby compDefine1, 145
 - usedby compReduce, 330
 - usedby compRepeatOrCollect, 333
 - usedby compSymbol, 598
 - usedby compWithMappingModel1, 615
 - usedby compWithMappingMode, 614
 - usedby displayMissingFunctions, 220
- \$formalMapVariables
 - local def hasFormalMapVariable, 621
- \$formatArgList
 - local ref compApplication, 603
 - usedby compWithMappingModel1, 615
- \$form
 - local def compDefineCapsuleFunction, 156
 - local def compDefineCategory2, 164
 - local ref applyMapping, 591
 - local ref compApplication, 603
 - local ref compDefineCategory2, 164
 - local ref compHasFormat, 313
 - usedby compCapsuleInner, 278
 - usedby compDefine1, 145
 - usedby compDefineFunctor1, 149
 - usedby s-process, 581
 - usedby setqSingle, 345
- \$found

- local ref NRTassocIndex, 346
- \$fromCoerceable
 - local ref autoCoerceByModemap, 364
 - local ref coerceable, 360
 - local ref coerce, 356
- \$frontier
 - local def compDefineCategory2, 164
- \$functionLocations
 - local def compDefineCapsuleFunction, 156
 - local ref compDefineCapsuleFunction, 156
 - local ref transformOperationAlist, 211
 - usedby compDefineFunctor1, 149
- \$functionName
 - local ref addArgumentConditions, 304
- \$functionStats
 - local def compDefineCapsuleFunction, 156
 - local def compDefineCategory2, 164
 - local def compile, 174
 - local ref compDefineCapsuleFunction, 156
 - local ref compile, 173
 - usedby compDefineFunctor1, 149
 - usedby reportOnFunctorCompilation, 220
- \$functorForm
 - local def compDefineCategory2, 164
 - local ref addModemap0, 274
 - local ref compile, 173
 - local ref getSpecialCaseAssoc, 304
 - usedby compAdd, 276
 - usedby compCapsule, 278
 - usedby compDefineFunctor1, 150
 - usedby compFunctorBody, 172
 - usedby getOperationAlist, 270
- \$functorLocalParameters
 - local def doItIf, 285
 - local def doIt, 281
 - local ref doItIf, 285
 - local ref doIt, 281
 - usedby compCapsuleInner, 279
 - usedby compDefineFunctor1, 150
 - usedby compSymbol, 598
- \$functorSpecialCases
 - local ref getSpecialCaseAssoc, 304
 - usedby compDefineFunctor1, 150
- \$functorStats
 - local def compDefineCategory2, 164
 - local ref compDefineCapsuleFunction, 156
 - usedby compDefineFunctor1, 150
 - usedby reportOnFunctorCompilation, 219
- \$functorTarget
 - usedby compDefineFunctor1, 150
- \$functorsUsed
 - local def doIt, 281
 - local ref doIt, 281
 - usedby compDefineFunctor1, 150
- \$funnameTail
 - usedby compWithMappingMode1, 615
- \$funname
 - usedby compWithMappingMode1, 615
- \$f
 - usedby compileSpad2Cmd, 563
- \$genFVar
 - usedby compDefineFunctor1, 150
 - usedby s-process, 582
- \$genSDVar
 - usedby compDefineFunctor1, 150
 - usedby s-process, 582
- \$genno
 - local def aplTran, 405
 - local def doIt, 281
- \$getDomainCode
 - local def compDefineCategory2, 164
 - local ref compileCases, 171
 - local ref doItIf, 285
 - local ref optCallSpecially, 235
 - usedby compCapsuleInner, 278
 - usedby compDefineFunctor1, 150
 - usedby genDomainOps, 225
 - usedby genDomainView, 224
- \$glossHash
 - local def checkRecordHash, 508
 - local ref checkRecordHash, 507
- \$goGetList
 - usedby compDefineFunctor1, 150
- \$headerDocumentation
 - local def recordHeaderDocumentation, 470
 - local ref recordHeaderDocumentation, 470
 - usedby postDef, 389
 - usedby preparse, 86
- \$htHash
 - local def checkRecordHash, 508
 - local ref checkRecordHash, 507
- \$htMacroTable

- local ref checkArguments, 499
 - local ref checkBeginEnd, 500
 - local ref checkSplitPunctuation, 535
- \$in-stream
 - local ref Advance-Char, 635
 - local ref next-line, 634
 - local ref preparse1, 87
- \$index, 81
 - local def preparse1, 87
 - local def preparseReadLine1, 93
 - local ref preparse1, 87
 - local ref preparseReadLine1, 93
 - usedby initialize-preparse, 82
 - usedby preparse, 86
 - defvar, 81
- \$initCapsuleErrorCount
 - local def compDefineCapsuleFunction, 156
- \$initList
 - usedby compReduce1, 331
- \$insideCapsuleFunctionIfTrue
 - local def compDefineCapsuleFunction, 156
 - local ref CapsuleModemapFrame, 273
 - local ref addEltModemap, 265
 - local ref addModemap, 273
 - local ref compile, 173
 - local ref getDomainsInScope, 254
 - local ref putDomainsInScope, 255
 - local ref spadCompileOrSetq, 186
 - usedby compDefine1, 145
 - usedby s-process, 581
- \$insideCategoryIfTrue
 - local def compDefineCategory2, 164
 - usedby compCapsuleInner, 278
 - usedby compColon, 295
 - usedby compDefine1, 145
 - usedby s-process, 581
- \$insideCategoryPackageIfTrue
 - local ref getFormModemaps, 605
 - usedby compCapsuleInner, 278
 - usedby compDefineCategory1, 162
 - usedby compDefineFunctor1, 150
- \$insideCoerceInteractiveHardIfTrue
 - usedby s-process, 581
- \$insideCompTypeOf
 - usedby comp3, 590
 - usedby compTypeOf, 593
- \$insideConstructIfTrue
 - local ref parseColon, 108
 - usedby parseConstruct, 103
- \$insideExpressionIfTrue
 - local def compDefineCapsuleFunction, 156
 - usedby compCapsule, 278
 - usedby compColon, 295
 - usedby compDefine1, 145, 146
 - usedby compExpression, 139
 - usedby compMakeDeclaration, 622
 - usedby compSeq1, 336
 - usedby compWhere, 355
 - usedby s-process, 581
- \$insideFunctorIfTrue
 - local ref compileCases, 171
 - usedby compColon, 295
 - usedby compDefine1, 145
 - usedby compDefineCategory, 162
 - usedby compDefineFunctor1, 150
 - usedby getOperationAlist, 270
 - usedby s-process, 581
- \$insidePostCategoryIfTrue
 - usedby postCategory, 381
 - usedby postWith, 404
- \$insideSetqSingleIfTrue
 - usedby setqSingle, 344
- \$insideWhereIfTrue
 - usedby compDefine1, 146
 - usedby compWhere, 355
 - usedby s-process, 581
- \$is-eqlist, 412
 - defvar, 412
- \$is-gensymlist, 412
 - defvar, 412
- \$is-spill-list, 411
 - defvar, 411
- \$is-spill, 411
 - defvar, 411
- \$isOpPackageName
 - usedby compDefineFunctor1, 150
- \$keywords
 - local ref escape-keywords, 454
- \$killOptimizeIfTrue
 - usedby compTopLevel, 583
 - usedby compWithMappingMode1, 615
- \$kind

- local def buildLibdbConEntry, 480
 - local def buildLibdb, 476
 - local ref buildLibAttr, 483
 - local ref buildLibOp, 482
 - local ref buildLibdbConEntry, 480
- \$leaveLevelStack
 - usedby compLeave, 327
 - usedby compRepeatOrCollect, 333
 - usedby s-process, 581
- \$leaveMode
 - usedby s-process, 581
- \$level
 - usedby compOrCroak1, 587
- \$lhsOfColon
 - local def evalAndSub, 269
 - usedby compColon, 295
 - usedby compSubsetCategory, 352
- \$lhs
 - usedby parseDEF, 109
 - usedby parseMDEF, 131
- \$libFile
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref compDefineCategory2, 164
 - local ref compilerDoitWithScreenedLisplib,\$lisplibCategory, 567
 - local ref finalizeLisplib, 207
 - local ref initializeLisplib, 206
 - local ref rwriteLispForm, 204
 - usedby compDefineFunctor1, 150
- \$line-handler
 - local ref next-line, 634
- \$linelist, 81
 - local def preparseReadLine1, 93
 - local ref preparse1, 87
 - local ref preparseReadLine1, 93
 - usedby initialize-preparse, 82
 - defvar, 81
- \$line
 - usedby Advance-Char, 635
 - usedby current-char, 460
 - usedby line-advance-char, 633
 - usedby line-at-end-p, 632
 - usedby line-clear, 632
 - usedby line-new-line, 634
 - usedby line-next-char, 633
 - usedby line-past-end-p, 633
 - usedby line-print, 632, 634
 - usedby match-advance-string, 453
 - usedby match-string, 451
- \$lispHash
 - local def checkRecordHash, 508
 - local ref checkRecordHash, 507
- \$lisplibAbbreviation
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref finalizeLisplib, 208
 - usedby compDefineFunctor1, 150
- \$lisplibAncestors
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref finalizeLisplib, 208
 - usedby compDefineFunctor1, 150
- \$lisplibAttributes
 - local ref finalizeLisplib, 208
- \$lisplibCategoriesExtended
 - local def compDefineLisplib, 167
 - usedby compDefineFunctor1, 150
- \$lisplibCategory
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 168
 - local def finalizeLisplib, 208
 - local ref compDefineCategory2, 164
 - local ref finalizeLisplib, 207
 - usedby compDefineCategory1, 162
 - usedby compDefineCategory, 162
 - usedby compDefineFunctor1, 150
- \$lisplibForm
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 167
 - local def initializeLisplib, 206
 - local ref finalizeDocumentation, 490
 - local ref finalizeLisplib, 207, 208
 - usedby compDefineFunctor1, 150
- \$lisplibFunctionLocations
 - usedby compDefineFunctor1, 150
- \$lisplibItemsAlreadyThere
 - local ref compile, 173
- \$lisplibKind
 - local def compDefineCategory2, 164

- local def compDefineLisplib, 168
- local def initializeLisplib, 206
- local ref compDefineLisplib, 167
- local ref finalizeLisplib, 207
- usedby compDefineFunctor1, 150
- \$lisplibMissingFunctions
 - usedby compDefineFunctor1, 150
- \$lisplibModemapAlist
 - local def augLisplibModemapsFromCategory, 192
 - local def augmentLisplibModemapsFromFunctor, 217
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref augLisplibModemapsFromCategory, 192
 - local ref augmentLisplibModemapsFromFunctor, 216
 - local ref finalizeLisplib, 207
- \$lisplibModemap
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref finalizeLisplib, 207, 208
 - usedby compDefineFunctor1, 150
- \$lisplibOpAlist
 - local def initializeLisplib, 206
- \$lisplibOperationAlist
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref getSlotFromFunctor, 212
 - usedby compDefineFunctor1, 150
- \$lisplibParents
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 167
 - local ref finalizeLisplib, 208
 - usedby compDefineFunctor1, 150
- \$lisplibPredicates
 - local def compDefineLisplib, 167
 - local ref finalizeLisplib, 208
- \$lisplibSignatureAlist
 - local def encodeFunctionName, 183
 - local def initializeLisplib, 206
 - local ref encodeFunctionName, 183
 - local ref finalizeLisplib, 208
- \$lisplibSlot1
 - local def compDefineLisplib, 168
 - local ref finalizeLisplib, 208
 - usedby compDefineFunctor1, 150
- \$lisplibSuperDomain
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref finalizeLisplib, 208
 - usedby compSubDomain1, 351
- \$lisplibVariableAlist
 - local def compDefineLisplib, 168
 - local def initializeLisplib, 206
 - local ref finalizeLisplib, 208
- \$lisplib
 - local def compDefineLisplib, 167
 - local ref compDefineCategory2, 164
 - local ref compile, 173
 - local ref encodeFunctionName, 183
 - local ref lisplibWrite, 214
 - local ref rwriteLispForm, 204
 - usedby /RQ,LIB, 570
 - usedby comp2, 589
 - usedby compDefineCategory, 162
 - usedby compDefineFunctor1, 149
 - usedby compDefineFunctor, 148
- \$lookupFunction
 - usedby compDefineFunctor1, 150
- \$macroIfTrue
 - local def compDefine, 144
 - local ref compile, 173
 - usedby compMacro, 327
- \$macroassoc
 - usedby s-process, 581
- \$maxSignatureLineNumber
 - local def recordDocumentation, 469
 - local ref recordHeaderDocumentation, 470
 - usedby postDef, 389
 - usedby preparse, 86
- \$mutableDomains
 - usedby compDefineFunctor1, 150
- \$mutableDomain
 - local ref compileConstructor1, 188
 - usedby compDefineFunctor1, 150
- \$mvl
 - usedby makeCategoryPredicates, 180
- \$myFunctorBody
 - local def compCapsuleItems, 280

- usedby compDefineFunctor1, 150
- \$m
 - usedby compileSpad2Cmd, 563
- \$name
 - local def transformAndRecheckComments, 495
 - local ref checkRecordHash, 507
- \$new2OldRenameAssoc
 - local ref new2OldTran, 78
- \$newCompilerUnionFlag
 - usedby compColonInside, 594
 - usedby compPretend, 329
- \$newComp
 - usedby compileSpad2Cmd, 563
- \$newConlist, 555
 - local def compDefineLisplib, 168
 - local ref compDefineLisplib, 167
 - usedby compileSpad2Cmd, 563
 - usedby compiler, 561
 - defvar, 555
- \$newConstructorList
 - local def extendLocalLibdb, 475
 - local ref extendLocalLibdb, 475
- \$newspad
 - usedby s-process, 581
- \$noEnv
 - local ref setqMultiple, 341
 - usedby compColon, 295
- \$noParseCommands
 - local ref PARSE-SpecialCommand, 416
- \$noSubsumption
 - usedby spad, 572
- \$optimizableConstructorNames
 - local ref optCallSpecially, 235
- \$options
 - usedby compileSpad2Cmd, 563
 - usedby compileSpadLispCmd, 566
 - usedby compiler, 561
 - usedby mkCategoryPackage, 180
- \$op
 - local def compDefineCapsuleFunction, 156
 - local def compDefineCategory2, 164
 - local def compDefineLisplib, 167
 - local ref applyMapping, 591
 - local ref compApplication, 603
 - local ref compDefineCapsuleFunction, 156
 - local ref compDefineCategory2, 164
 - local ref finalizeDocumentation, 490
 - usedby compDefine1, 145
 - usedby compDefineFunctor1, 150
 - usedby compSubDomain1, 351
 - usedby parseDollarGreaterEqual, 112
 - usedby parseDollarGreaterThan, 112
 - usedby parseDollarLessEqual, 113
 - usedby parseDollarNotEqual, 113
 - usedby parseGreaterEqual, 115
 - usedby parseGreaterThan, 116
 - usedby parseLessEqual, 129
 - usedby parseNotEqual, 133
 - usedby parseTran, 101
 - usedby reportOnFunctorCompilation, 219
- \$origin
 - local def transformAndRecheckComments, 495
 - local ref checkRecordHash, 508
- \$out-stream
 - local ref line-print, 632
 - local ref print-package, 547
- \$outStream
 - local def buildLibdb, 476
 - local def dbWriteLines, 479
 - local ref buildLibdb, 476
 - local ref checkDocError, 516
 - local ref dbWriteLines, 479
- \$packagesUsed
 - local def compDefine, 145
 - local def doIt, 281
 - local ref doIt, 281
 - usedby comp2, 589
 - usedby compAdd, 276
 - usedby compTopLevel, 583
- \$pairlis
 - local def finalizeLisplib, 208
 - local ref NRTgetLookupFunction, 214
 - usedby compDefineFunctor1, 150
- \$postStack
 - local ref displayPreCompilationErrors, 542
 - usedby postError, 374
 - usedby s-process, 581
- \$predAlist
 - usedby compDefWhereClause, 160
- \$predl

- local ref `doItIf`, 285
 - local ref `doIt`, 281
- `$pred`
 - local ref `compCapsuleItems`, 280
 - local ref `compSingleCapsuleItem`, 280
- `$prefix`
 - local ref `applyMapping`, 591
 - local ref `compApplication`, 603
 - local ref `compile`, 174
 - usedby `compDefine1`, 145
 - usedby `compDefineCategory2`, 164
- `$preparse-last-line`, 82
 - local def `preparse1`, 87
 - local def `preparseReadLine1`, 93
 - local ref `preparse1`, 87
 - usedby `initialize-preparse`, 82
 - usedby `preparse`, 86
 - defvar, 82
- `$preparseReportIfTrue`
 - usedby `preparse`, 86
- `$previousTime`
 - usedby `s-process`, 582
- `$profileAlist`
 - usedby `compDefineFunctor`, 148
- `$profileCompiler`
 - local ref `compDefineCapsuleFunction`, 156
 - local ref `finalizeLisplib`, 208
 - usedby `compDefineFunctor`, 148
 - usedby `setqSingle`, 345
- `$recheckingFlag`
 - local def `transformAndRecheckComments`, 495
 - local ref `checkDocError`, 516
- `$reportExitModeStack`
 - usedby `modifyModeStack`, 622
- `$reportOptimization`
 - local ref `optimizeFunctionDef`, 228
- `$resolveTimeSum`
 - usedby `compTopLevel`, 583
- `$returnMode`
 - local def `compDefineCapsuleFunction`, 156
 - local ref `compDefineCapsuleFunction`, 156
 - usedby `compReturn`, 335
 - usedby `s-process`, 581
- `$saveableItems`
 - local def `compile`, 174
- `$saveableItems`
 - local ref `compilerDoitWithScreenedLisplib`, 567
 - local ref `compile`, 173
- `$scanIfTrue`
 - usedby `compOrCroak1`, 587
 - usedby `compileSpad2Cmd`, 563
- `$semanticErrorStack`
 - local ref `compDefineCapsuleFunction`, 156
 - usedby `reportOnFunctorCompilation`, 220
 - usedby `s-process`, 581
- `$setOptions`
 - local ref `checkRecordHash`, 507
- `$setelt`
 - usedby `compDefineFunctor1`, 150
- `$sideEffectsList`
 - usedby `compReduce1`, 331
- `$sigAlist`
 - usedby `compDefWhereClause`, 160
- `$sigList`
 - local def `compCategory`, 290
 - local ref `compCategoryItem`, 291
 - local ref `compCategory`, 290
- `$signatureOfForm`
 - local def `compCapsuleItems`, 280
 - local def `compDefineCapsuleFunction`, 156
 - local ref `compDefineCapsuleFunction`, 156
 - local ref `compile`, 173, 174
 - local ref `doIt`, 281
- `$signature`
 - usedby `compCapsuleInner`, 278
 - usedby `compDefineFunctor1`, 150
- `$skipme`
 - local def `preparse1`, 87
 - usedby `preparse`, 86
- `$sourceFileTypes`
 - usedby `compileSpad2Cmd`, 563
- `$spad-errors`
 - usedby `bumperrorcount`, 543
- `$spadLibFT`
 - local ref `compDefineLisplib`, 167
 - local ref `compileDocumentation`, 169
 - local ref `finalizeLisplib`, 208
 - local ref `lisplibDoRename`, 205
- `$spad`
 - local def `string2BootTree`, 78

- usedby quote-if-string, [454](#)
 - usedby spad, [572](#)
- \$specialCaseKeyList
 - local def compileCases, [171](#)
 - local ref optCallSpecially, [235](#)
- \$splitUpItemsAlreadyThere
 - local ref compile, [173](#)
- \$stack
 - usedby reduce-stack, [465](#)
 - usedby stack-/empty, [96](#)
 - usedby stack-clear, [96](#)
 - usedby stack-load, [95](#)
 - usedby stack-pop, [97](#)
 - usedby stack-push, [96](#)
- \$stringFauxNewline
 - local ref newWordFrom, [538](#)
- \$suffix
 - local def compCapsuleItems, [280](#)
 - local def compile, [174](#)
 - local ref compile, [173](#)
- \$sysHash
 - local def checkRecordHash, [508](#)
 - local ref checkRecordHash, [507](#)
- \$s
 - usedby compOrCroak1, [587](#)
- \$template
 - usedby compDefineFunctor1, [150](#)
- \$tokenCommands
 - local ref PARSE-SpecialCommand, [416](#)
- \$token
 - usedby current-token, [98](#)
 - usedby make-symbol-of, [458](#)
 - usedby match-advance-string, [453](#)
 - usedby next-token, [98](#)
 - usedby prior-token, [97](#)
 - usedby token-install, [98](#)
 - usedby token-print, [99](#)
 - usedby valid-tokens, [98](#)
- \$top-level
 - local def compCapsuleItems, [280](#)
 - local def compCategory, [290](#)
 - local def compDefineCategory2, [164](#)
 - usedby compDefineFunctor1, [149](#)
 - usedby s-process, [581](#)
- \$stopOp
 - local ref displayPreCompilationErrors, [542](#)
- usedby s-process, [581](#)
 - usedby setDefOp, [404](#)
- \$tripleCache
 - local def compDefine, [144](#)
- \$tripleHits
 - local def compDefine, [144](#)
- \$true
 - local ref addArgumentConditions, [304](#)
 - local ref optCONDtail, [231](#)
 - local ref optIF2COND, [232](#)
- \$tv1
 - usedby makeCategoryPredicates, [180](#)
- \$uncondAlist
 - usedby compDefineFunctor1, [150](#)
- \$until
 - usedby compReduce1, [331](#)
 - usedby compRepeatOrCollect, [333](#)
- \$viewNames
 - usedby compDefineFunctor1, [150](#)
- \$v1, [412](#)
 - defvar, [412](#)
- \$warningStack
 - usedby reportOnFunctorCompilation, [220](#)
 - usedby s-process, [581](#)
- \$why
 - local def NRTgetLookupFunction, [214](#)
 - local ref NRTgetLookupFunction, [214](#)
- \$x
 - local def transDoc, [494](#)
 - local def transformAndRecheckComments, [495](#)
 - local ref checkDocMessage, [518](#)
 - local ref checkTrim, [515](#)
 - local ref transDoc, [494](#)
- , [39](#)
- abbreviation?
 - calledby checkIsValidType, [530](#)
 - calledby checkNumOfArgs, [532](#)
 - calledby parseHasRhs, [118](#)
- abbreviationsSpad2Cmd
 - calledby mkCategoryPackage, [180](#)
- action, [464](#)
 - calledby PARSE-AnyId, [441](#)
 - calledby PARSE-Category, [421](#)

- calledby PARSE-CommandTail, 418
- calledby PARSE-Data, 439
- calledby PARSE-FloatExponent, 434
- calledby PARSE-GlyphTok, 441
- calledby PARSE-Infix, 426
- calledby PARSE-NBGlyphTok, 441
- calledby PARSE-NewExpr, 415
- calledby PARSE-OpenBrace, 443
- calledby PARSE-OpenBracket, 443
- calledby PARSE-Operation, 424
- calledby PARSE-Prefix, 426
- calledby PARSE-ReductionOp, 428
- calledby PARSE-Sexpr1, 440
- calledby PARSE-SpecialCommand, 416
- calledby PARSE-SpecialKeyWord, 415
- calledby PARSE-Suffix, 445
- calledby PARSE-TokTail, 427
- calledby PARSE-TokenCommandTail, 417
- calledby PARSE-TokenList, 418
- defun, 464
- add, 376
 - defplist, 376
- addArgumentConditions, 304
 - calledby compDefineCapsuleFunction, 156
 - calls mkq, 304
 - calls systemErrorHere, 304
 - local def \$argumentConditionList, 304
 - local ref \$argumentConditionList, 304
 - local ref \$body, 304
 - local ref \$functionName, 304
 - local ref \$true, 304
 - defun, 304
- addBinding
 - calledby addModemap1, 274
 - calledby compDefineCategory2, 163
 - calledby getSuccessEnvironment, 319
 - calledby setqMultiple, 341
- addBinding[5]
 - called by setqSingle, 344
 - called by spad, 572
- addclose, 550
 - calls suffix, 550
 - defun, 550
- addConstructorModemaps, 258
 - calledby augModemapsFromDomain1, 257
 - calls addModemap, 258
- calls getl, 258
- calls putDomainsInScope, 258
- local def \$InteractiveMode, 258
- defun, 258
- AddContour
 - calledby compApply, 592
- addContour
 - calledby compWhere, 355
- addDomain, 252
 - calledby comp2, 589
 - calledby comp3, 589
 - calledby compAtSign, 361
 - calledby compCapsule, 278
 - calledby compCase, 287
 - calledby compCoerce, 362
 - calledby compColonInside, 594
 - calledby compColon, 294
 - calledby compDefineCapsuleFunction, 155
 - calledby compElt, 310
 - calledby compForm1, 600
 - calledby compImport, 322
 - calledby compPretend, 328
 - calledby compSubDomain1, 350
 - calls addNewDomain, 252
 - calls constructor?, 252
 - calls domainMember, 252
 - calls getDomainsInScope, 252
 - calls getmode, 252
 - calls identp, 252
 - calls isCategoryForm, 252
 - calls isFunctor, 252
 - calls isLiteral, 252
 - calls member, 252
 - calls qslessp, 252
 - calls unknownTypeError, 252
 - defun, 252
- addEltModemap, 265
 - calledby addModemap0, 274
 - calls addModemap1, 265
 - calls makeLiteral, 265
 - calls systemErrorHere, 265
 - local def \$e, 265
 - local ref \$insideCapsuleFunctionIfTrue, 265
 - defun, 265
- addEmptyCapsuleIfNecessary, 177

- calledby compDefine1, 145
 - uses \$SpecialDomainNames, 177
 - defun, 177
- addInformation
 - calledby compCapsuleInner, 278
- addModemap, 273
 - calledby addConstructorModemaps, 258
 - calledby augModemapsFromCategoryRep, addStats, 271
 - calledby genDomainOps, 225
 - calledby updateCategoryFrameForCategory, 121
 - calledby updateCategoryFrameForConstructor, 120
 - calls addModemap0, 273
 - calls knownInfo, 273
 - local def \$CapsuleModemapFrame, 273
 - local ref \$CapsuleModemapFrame, 273
 - local ref \$InteractiveMode, 273
 - local ref \$e, 273
 - local ref \$insideCapsuleFunctionIfTrue, 273
 - defun, 273
- addModemap0, 274
 - calledby addModemapKnown, 272
 - calledby addModemap, 273
 - calls addEltModemap, 274
 - calls addModemap1, 274
 - local ref \$functorForm, 274
 - defun, 274
- addModemap1, 274
 - calledby addEltModemap, 265
 - calledby addModemap0, 274
 - calls addBinding, 274
 - calls augProplist, 274
 - calls getProplist, 274
 - calls lassoc, 274
 - calls mkNewModemapList, 274
 - calls unErrorRef, 274
 - defun, 274
- addModemapKnown, 272
 - calledby augModemapsFromCategory, 265
 - calls addModemap0, 272
 - local def \$CapsuleModemapFrame, 273
 - local ref \$e, 272
 - defun, 272
- addNewDomain, 256
 - calledby addDomain, 252
 - calledby augModemapsFromDomain, 256
 - calls augModemapsFromDomain, 256
 - defun, 256
- adoptions
 - calledby initializeLisplib, 206
- addStats
 - calledby compDefineCapsuleFunction, 156
 - calledby compile, 173
 - calledby reportOnFunctorCompilation, 219
- addSuffix, 303
 - calledby mkAbbrev, 302
 - defun, 303
- Advance-Char, 635
 - calls Line-Advance-Char, 635
 - calls Line-At-End-P, 635
 - calls current-char, 635
 - calls next-line, 635
 - local ref \$in-stream, 635
 - uses \$line, 635
 - defun, 635
- advance-char
 - calledby skip-blanks, 451
- advance-token, 459
 - calledby PARSE-AnyId, 441
 - calledby PARSE-FloatExponent, 434
 - calledby PARSE-GlyphTok, 441
 - calledby PARSE-Infix, 426
 - calledby PARSE-NBGlyphTok, 441
 - calledby PARSE-OpenBrace, 443
 - calledby PARSE-OpenBracket, 443
 - calledby PARSE-Prefix, 426
 - calledby PARSE-ReductionOp, 428
 - calledby PARSE-Suffix, 445
 - calledby PARSE-TokenList, 418
 - calledby parse-argument-designator, 546
 - calledby parse-identifier, 545
 - calledby parse-keyword, 546
 - calledby parse-number, 545
 - calledby parse-spadstring, 544
 - calledby parse-string, 544
 - calls copy-token, 459
 - calls current-token, 459
 - calls try-get-token, 459
 - uses current-token, 459

- uses valid-tokens, [459](#)
 - defun, [459](#)
- alistSize, [303](#)
 - calledby mkAbbrev, [302](#)
 - defun, [303](#)
- allConstructors[5]
 - called by buildLibdb, [476](#)
- allLASSOCs, [218](#)
 - calledby augmentLisplibModemapsFrom-Functor, [216](#)
 - defun, [218](#)
- and, [105](#)
 - defplist, [105](#)
- aplTran, [405](#)
 - calledby postTransform, [369](#)
 - calls aplTran1, [405](#)
 - calls containsBang, [405](#)
 - local def \$genno, [405](#)
 - uses \$boot, [405](#)
 - defun, [405](#)
- aplTran1, [405](#)
 - calledby aplTran1, [405](#)
 - calledby aplTranList, [407](#)
 - calledby aplTran, [405](#)
 - calledby hasAplExtension, [407](#)
 - calls aplTran1, [405](#)
 - calls aplTranList, [405](#)
 - calls hasAplExtension, [405](#)
 - calls nreverse0, [405](#)
 - local ref \$boot, [405](#)
 - defun, [405](#)
- aplTranList, [407](#)
 - calledby aplTran1, [405](#)
 - calledby aplTranList, [407](#)
 - calls aplTran1, [407](#)
 - calls aplTranList, [407](#)
 - defun, [407](#)
- applyMapping, [591](#)
 - calledby comp3, [590](#)
 - calls comp, [591](#)
 - calls convert, [591](#)
 - calls encodeItem, [591](#)
 - calls getAbbreviation, [591](#)
 - calls get, [591](#)
 - calls isCategoryForm, [591](#)
 - calls member, [591](#)
 - calls sublis, [591](#)
 - local ref \$FormalMapVariableList, [591](#)
 - local ref \$formalArgList, [591](#)
 - local ref \$form, [591](#)
 - local ref \$op, [591](#)
 - local ref \$prefix, [591](#)
 - defun, [591](#)
- argsToSig, [621](#)
 - calledby compLambda, [325](#)
 - defun, [621](#)
- assignError, [346](#)
 - calledby setqSingle, [344](#)
 - calls stackMessage, [346](#)
 - defun, [346](#)
- assoc
 - calledby augModemapsFromCategoryRep, [271](#)
 - calledby checkBalance, [499](#)
 - calledby compColon, [294](#)
 - calledby compDefineAddSignature, [147](#)
 - calledby compForm2, [607](#)
 - calledby mkCategoryPackage, [180](#)
 - calledby mkNewModemapList, [266](#)
 - calledby mkOpVec, [225](#)
 - calledby stripOffSubdomainConditions, [305](#)
 - calledby transformOperationAlist, [211](#)
- AssocBarGensym, [226](#)
 - calledby mkOpVec, [226](#)
 - calls EqualBarGensym, [226](#)
 - defun, [226](#)
- assocleft
 - calledby compDefWhereClause, [160](#)
 - calledby compileCases, [171](#)
 - calledby finalizeDocumentation, [490](#)
 - calledby mkAlistOfExplicitCategoryOps, [193](#)
- assocright
 - calledby compDefWhereClause, [160](#)
 - calledby compileCases, [171](#)
 - calledby recordHeaderDocumentation, [470](#)
- assq
 - calledby getAbbreviation, [302](#)
 - calledby makeFunctorArgumentParameters, [221](#)
 - calledby mkOpVec, [225](#)
- assq[5]

- called by freelist, 623
- atEndOfLine
 - calledby PARSE-TokenCommandTail, 417
- augLisplibModemapsFromCategory, 191
 - calledby compDefineCategory2, 164
 - calls interactiveModemapForm, 192
 - calls isCategoryForm, 192
 - calls lassoc, 192
 - calls member, 192
 - calls mkAlistOfExplicitCategoryOps, 192
 - calls mkpf, 192
 - calls sublis, 191
 - local def \$lisplibModemapAlist, 192
 - local ref \$EmptyEnvironment, 192
 - local ref \$PatternVariableList, 192
 - local ref \$domainShell, 192
 - local ref \$lisplibModemapAlist, 192
 - defun, 191
- augmentLisplibModemapsFromFunctor, 216
 - calledby compDefineFunctor1, 149
 - calls allLASSOCs, 216
 - calls formal2Pattern, 216
 - calls interactiveModemapForm, 216
 - calls listOfPatternIds, 216
 - calls member, 216
 - calls mkAlistOfExplicitCategoryOps, 216
 - calls mkDatabasePred, 216
 - calls mkpf, 216
 - local def \$e, 217
 - local def \$lisplibModemapAlist, 217
 - local ref \$PatternVariableList, 216
 - local ref \$e, 216
 - local ref \$lisplibModemapAlist, 216
 - defun, 216
- augModemapsFromCategory, 265
 - calledby augModemapsFromDomain1, 257
 - calledby compDefineFunctor1, 148
 - calledby genDomainView, 224
 - calls addModemapKnown, 265
 - calls compilerMessage, 265
 - calls evalAndSub, 265
 - calls putDomainsInScope, 265
 - local def \$base, 265
 - defun, 265
- augModemapsFromCategoryRep, 271
 - calledby compDefineFunctor1, 148
- calls addModemap, 271
- calls assoc, 271
- calls compilerMessage, 271
- calls evalAndSub, 271
- calls isCategory, 271
- calls putDomainsInScope, 271
- local def \$base, 271
- defun, 271
- augModemapsFromDomain, 256
 - calledby addNewDomain, 256
 - calls addNewDomain, 256
 - calls augModemapsFromDomain1, 256
 - calls getDomainsInScope, 256
 - calls getdatabase, 256
 - calls listOrVectorElementNode, 256
 - calls member, 256
 - calls opOf, 256
 - calls stripUnionTags, 256
 - local ref \$Category, 256
 - local ref \$DummyFunctorNames, 256
 - defun, 256
- augModemapsFromDomain1, 257
 - calledby augModemapsFromDomain, 256
 - calledby compForm1, 600
 - calledby setqSingle, 344
 - calls addConstructorModemaps, 257
 - calls augModemapsFromCategory, 257
 - calls getl, 257
 - calls getmodeOrMapping, 257
 - calls getmode, 257
 - calls stackMessage, 257
 - calls substituteCategoryArguments, 257
 - defun, 257
- augProplist
 - calledby addModemap1, 274
- autoCoerceByModemap, 364
 - calledby coerceExtraHard, 359
 - calls getModemapList, 364
 - calls get, 364
 - calls member, 364
 - calls modeEqual, 364
 - calls stackMessage, 364
 - local ref \$fromCoerceable, 364
 - defun, 364
- awk
 - calledby whoOwns, 539

- Bang, [463](#)
 - defmacro, [463](#)
- bang
 - calledby PARSE-Category, [421](#)
 - calledby PARSE-CommandTail, [418](#)
 - calledby PARSE-Conditional, [448](#)
 - calledby PARSE-Form, [429](#)
 - calledby PARSE-Import, [422](#)
 - calledby PARSE-IteratorTail, [444](#)
 - calledby PARSE-Seg, [448](#)
 - calledby PARSE-Sexpr1, [440](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calledby PARSE-TokenCommandTail, [417](#)
- bfp-
 - calledby PARSE-FloatTok, [450](#)
- blankp, [550](#)
 - calledby nonblankloc, [553](#)
 - defun, [550](#)
- Block, [380](#)
 - defplist, [380](#)
- boot-line-stack
 - usedby spad, [572](#)
 - usedby string2BootTree, [77](#)
- bootStrapError, [219](#)
 - calledby compCapsule, [278](#)
 - calledby compFunctorBody, [172](#)
 - calls mkDomainConstructor, [219](#)
 - calls mkq, [219](#)
 - calls namestring, [219](#)
 - defun, [219](#)
- bpiname
 - calledby compileTimeBindingOf, [237](#)
 - calledby subrname, [232](#)
- bright
 - calledby NRTgetLookupFunction, [214](#)
 - calledby checkAndDeclare, [308](#)
 - calledby compDefineLisplib, [167](#)
 - calledby displayMissingFunctions, [220](#)
 - calledby doIt, [281](#)
 - calledby finalizeDocumentation, [490](#)
 - calledby hasSigInTargetCategory, [309](#)
 - calledby optimizeFunctionDef, [228](#)
 - calledby parseInBy, [126](#)
 - calledby postForm, [374](#)
 - calledby spadCompileOrSetq, [186](#)
- browserAutoloadOnceTrigger
 - calledby compileSpad2Cmd, [563](#)
- buildFunctor
 - calledby processFunctor, [279](#)
- buildLibAttr, [483](#)
 - calledby buildLibAttrs, [483](#)
 - calls buildLibdbString, [483](#)
 - calls checkCommentsForBraces, [483](#)
 - calls concatWithBlanks, [483](#)
 - calls form2LispString, [483](#)
 - calls lassoc, [483](#)
 - calls length, [483](#)
 - calls sublislis, [483](#)
 - calls writedb, [483](#)
 - local ref \$FormalMapVariableList, [483](#)
 - local ref \$conform, [483](#)
 - local ref \$conname, [483](#)
 - local ref \$doc, [483](#)
 - local ref \$exposed?, [483](#)
 - local ref \$kind, [483](#)
 - defun, [483](#)
- buildLibAttrs, [483](#)
 - calledby buildLibdb, [476](#)
 - calls buildLibAttr, [483](#)
 - defun, [483](#)
- buildLibdb, [476](#)
 - calledby extendLocalLibdb, [475](#)
 - calls allConstructors[5], [476](#)
 - calls buildLibAttrs, [476](#)
 - calls buildLibOps, [476](#)
 - calls buildLibdbConEntry, [476](#)
 - calls buildLibdbString, [476](#)
 - calls deleteFile[5], [476](#)
 - calls deleteFile, [476](#)
 - calls dsetq, [476](#)
 - calls getConstructorExports, [476](#)
 - calls ifcar, [476](#)
 - calls make-outstream[5], [476](#)
 - calls obey, [476](#)
 - calls shut, [476](#)
 - calls writedb, [476](#)
 - local def \$AttrLst, [476](#)
 - local def \$DefLst, [476](#)
 - local def \$DomLst, [476](#)
 - local def \$OpLst, [477](#)
 - local def \$PakLst, [476](#)
 - local def \$catLst, [476](#)

- local def \$conform, 476
 - local def \$conname, 476
 - local def \$doc, 476
 - local def \$exposed?, 476
 - local def \$kind, 476
 - local def \$outStream, 476
 - local ref \$conform, 476
 - local ref \$outStream, 476
 - defun, 476
- buildLibdbConEntry, 480
 - calledby buildLibdb, 476
 - calls buildLibdbString, 480
 - calls concatWithBlanks, 480
 - calls dbMkForm, 480
 - calls downcase, 480
 - calls form2HtString, 480
 - calls getdatabase, 480
 - calls isExposedConstructor, 480
 - calls lassoc, 480
 - calls length, 480
 - calls libConstructorSig, 480
 - calls libdbTrim, 480
 - calls maxindex, 480
 - calls msubst, 480
 - calls pname, 480
 - calls strconc, 480
 - local def \$conname, 480
 - local def \$doc, 480
 - local def \$exposed?, 480
 - local def \$kind, 480
 - local ref \$conform, 480
 - local ref \$exposed?, 480
 - local ref \$kind, 480
 - defun, 480
- buildLibdbString, 478
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
 - calledby buildLibdb, 476
 - calls strconc, 478
 - defun, 478
- buildLibOp, 482
 - calledby buildLibOps, 482
 - calls buildLibdbString, 482
 - calls checkCommentsForBraces, 482
 - calls concatWithBlanks, 482
 - calls form2LispString, 482
 - calls lassoc, 482
 - calls libdbTrim, 482
 - calls msubst, 482
 - calls strconc, 482
 - calls sublislis, 482
 - calls writedb, 482
 - local ref \$conform, 482
 - local ref \$doc, 482
 - local ref \$exposed?, 482
 - local ref \$kind, 482
 - defun, 482
- buildLibOps, 482
 - calledby buildLibdb, 476
 - calls buildLibOp, 482
 - defun, 482
- bumperrorcount, 543
 - calledby postError, 374
 - uses \$InteractiveMode, 543
 - uses \$spad-errors, 543
 - defun, 543
- call, 233
 - defplist, 233
- canFuncall?
 - calledby loadLibIfNecessary, 119
- cannotDo
 - calledby doIt, 281
- canReturn, 316
 - calledby canReturn, 316
 - calledby compIf, 315
 - calls canReturn, 316
 - calls say, 316
 - calls systemErrorHere, 316
 - defun, 316
- capsule, 277
 - defplist, 277
- CapsuleModemapFrame
 - local ref \$insideCapsuleFunctionIfTrue, 273
- case, 287
 - defplist, 287
- catch, 244
 - defplist, 244
- catches
 - compOrCroak1, 587

- compUniquely, 613
 - preparse1, 87
 - spad, 572
- category, 106, 290, 381
 - defplist, 106, 290, 381
- char
 - calledby isCategoryPackageName, 214
- char-eq, 461
 - calledby PARSE-FloatBase, 433
 - calledby PARSE-TokTail, 427
 - defun, 461
- char-ne, 461
 - calledby PARSE-FloatBase, 433
 - calledby PARSE-Selector, 430
 - defun, 461
- charp
 - calledby checkSplitBrace, 524
 - calledby checkSplitOn, 534
 - calledby checkSplitPunctuation, 535
- charPosition
 - calledby checkAddSpaceSegments, 528
 - calledby checkExtract, 519
 - calledby checkGetArgs, 520
 - calledby checkGetLispFunctionName, 506
 - calledby checkSplitBackslash, 533
 - calledby checkSplitOn, 534
 - calledby checkSplitPunctuation, 535
 - calledby checkTrim, 515
 - calledby htcharPosition, 537
 - calledby removeBackslashes, 539
 - calledby screenLocalLine, 484
- chaseInferences
 - calledby compHas, 313
- checkAddBackSlashes, 526
 - calledby checkAddBackSlashes, 526
 - calledby checkDecorate, 502
 - calls checkAddBackSlashes, 526
 - calls maxindex, 526
 - calls strconc, 526
 - local ref \$charEscapeList, 526
 - defun, 526
- checkAddIndented, 518
 - calledby checkRewrite, 497
 - calls checkAddSpaceSegments, 518
 - calls firstNonBlankPosition, 518
 - calls strconc, 518
- defun, 518
- checkAddMacros, 527
 - calledby checkRewrite, 497
 - calls lassoc, 527
 - calls nreverse, 527
 - local ref \$HTmacs, 527
 - defun, 527
- checkAddPeriod, 527
 - calledby checkComments, 496
 - calls maxindex, 527
 - calls setelt, 527
 - defun, 527
- checkAddSpaces, 528
 - calledby checkComments, 496
 - calledby checkRewrite, 497
 - local ref \$charBlank, 528
 - local ref \$charFauxNewline, 528
 - defun, 528
- checkAddSpaceSegments, 528
 - calledby checkAddIndented, 518
 - calledby checkAddSpaceSegments, 528
 - calledby checkIndentedLines, 522
 - calls charPosition, 528
 - calls checkAddSpaceSegments, 528
 - calls maxindex, 528
 - calls strconc, 528
 - local ref \$charBlank, 528
 - defun, 528
- checkAlphabetic, 529
 - calledby checkSkipIdentifierToken, 523
 - calledby checkSkipOpToken, 523
 - local ref \$charIdentifierEndings, 529
 - defun, 529
- checkAndDeclare, 308
 - calledby compDefineCapsuleFunction, 155
 - calls bright, 308
 - calls getArgumentMode, 308
 - calls modeEqual, 308
 - calls put, 308
 - calls sayBrightly, 308
 - defun, 308
- checkArguments, 499
 - calledby checkComments, 496
 - calledby checkRewrite, 497
 - calls checkHTargs, 499
 - calls hget, 499

- local ref \$htMacroTable, 499
 - defun, 499
- checkBalance, 499
 - calledby checkComments, 496
 - calls assoc, 499
 - calls checkBeginEnd, 499
 - calls checkDocError, 499
 - calls checkSayBracket, 499
 - calls nreverse, 499
 - calls rassoc, 499
 - local ref \$checkPrenAlist, 499
 - defun, 499
- checkBeginEnd, 500
 - calledby checkBalance, 499
 - calls checkDocError, 500
 - calls hget, 500
 - calls ifcar, 500
 - calls ifcdr, 500
 - calls length, 500
 - calls member, 500
 - calls substring?, 500
 - local ref \$beginEndList, 500
 - local ref \$htMacroTable, 500
 - defun, 500
- checkComments, 496
 - calledby transformAndRecheckComments, 495
 - calls checkAddPeriod, 496
 - calls checkAddSpaces, 496
 - calls checkArguments, 496
 - calls checkBalance, 496
 - calls checkDecorate, 496
 - calls checkFixCommonProblems, 496
 - calls checkGetArgs, 496
 - calls checkGetMargin, 496
 - calls checkIeEg, 496
 - calls checkIndentedLines, 496
 - calls checkSplit2Words, 496
 - calls checkTransformFirsts, 496
 - calls newString2Words, 496
 - calls pp, 496
 - calls strconc, 496
 - local def \$argl, 496
 - local def \$checkErrorFlag, 496
 - local ref \$attribute?, 496
 - local ref \$checkErrorFlag, 496
 - defun, 496
- checkCommentsForBraces
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
- checkDecorate, 502
 - calledby checkComments, 496
 - calls checkAddBackSlashes, 502
 - calls checkDocError, 502
 - calls hasNoVowels, 502
 - calls member, 502
 - local ref \$argl, 502
 - local ref \$charExclusions, 502
 - local ref \$checkingXmptex?, 502
 - defun, 502
- checkDecorateForHt, 504
 - calledby checkRewrite, 497
 - calls checkDocError, 504
 - calls member, 504
 - local ref \$checkingXmptex?, 504
 - defun, 504
- checkDocError, 515
 - calledby checkBalance, 499
 - calledby checkBeginEnd, 500
 - calledby checkDecorateForHt, 504
 - calledby checkDecorate, 502
 - calledby checkDocError1, 505
 - calledby checkFixCommonProblem, 505
 - calledby checkGetLispFunctionName, 506
 - calledby checkHTargs, 506
 - calledby checkRecordHash, 507
 - calledby checkTexht, 511
 - calledby checkTransformFirsts, 512
 - calledby checkTrim, 515
 - calledby transDocList, 493
 - calls checkDocMessage, 515
 - calls concat, 515
 - calls sayBrightly, 516
 - calls saybrightly1, 515
 - local def \$checkErrorFlag, 516
 - local def \$exposeFlagHeading, 516
 - local ref \$checkErrorFlag, 516
 - local ref \$constructorName, 516
 - local ref \$exposeFlagHeading, 516
 - local ref \$exposeFlag, 516
 - local ref \$outStream, 516
 - local ref \$recheckingFlag, 516

- defun, 515
- checkDocError1, 505
 - calledby transDocList, 493
 - calledby transDoc, 494
 - calls checkDocError, 505
 - local ref \$compileDocumentation, 505
 - defun, 505
- checkDocMessage, 518
 - calledby checkDocError, 515
 - calls concat, 518
 - calls getdatabase, 518
 - calls whoOwns, 518
 - local ref \$constructorName, 518
 - local ref \$x, 518
 - defun, 518
- checkExtract, 519
 - calledby transDoc, 494
 - calls charPosition, 519
 - calls firstNonBlankPosition, 519
 - calls length, 519
 - calls substring?, 519
 - defun, 519
- checkFixCommonProblem, 505
 - calledby checkRewrite, 497
 - calls checkDocError, 505
 - calls ifcar, 505
 - calls ifcdr, 505
 - calls member, 505
 - local ref \$HTspadmacros, 505
 - defun, 505
- checkFixCommonProblems
 - calledby checkComments, 496
- checkGetArgs, 520
 - calledby checkComments, 496
 - calledby checkGetArgs, 520
 - calledby checkRewrite, 497
 - calls charPosition, 520
 - calls checkGetArgs, 520
 - calls firstNonBlankPosition, 520
 - calls getMatchingRightPren, 520
 - calls maxindex, 520
 - calls stringPrefix?, 520
 - calls trimString, 520
 - defun, 520
- checkGetLispFunctionName, 506
 - calledby checkRecordHash, 507
- calls charPosition, 506
- calls checkDocError, 506
- defun, 506
- checkGetMargin, 520
 - calledby checkComments, 496
 - calls firstNonBlankPosition, 520
 - defun, 520
- checkGetParse, 521
 - calledby checkRecordHash, 507
 - calls ncParseFromString, 521
 - calls removeBackslashes, 521
 - defun, 521
- checkGetStringBeforeRightBrace, 521
 - calledby checkRecordHash, 507
 - defun, 521
- checkHTargs, 506
 - calledby checkArguments, 499
 - calledby checkHTargs, 507
 - calls checkDocError, 506
 - calls checkHTargs, 507
 - calls checkLookForLeftBrace, 506
 - calls checkLookForRightBrace, 506
 - calls ifcdr, 507
 - defun, 506
- checkIeEg, 522
 - calledby checkComments, 496
 - calls checkIeEgfun, 522
 - calls nreverse, 522
 - defun, 522
- checkIeEgFun
 - calledby checkIeEgfun, 529
- checkIeEgfun, 529
 - calledby checkIeEg, 522
 - calls checkIeEgFun, 529
 - calls maxindex, 529
 - defun, 529
- checkIndentedLines, 522
 - calledby checkComments, 496
 - calls checkAddSpaceSegments, 522
 - calls firstNonBlankPosition, 522
 - calls strconc, 522
 - local ref \$charFauxNewline, 522
 - defun, 522
- checkIsValidType, 530
 - calledby checkIsValidType, 530
 - calledby checkRecordHash, 507

- calls abbreviation?, 530
- calls checkIsValidType, 530
- calls constructor?, 530
- calls getdatabase, 530
- calls length, 530
- defun, 530
- checkLookForLeftBrace, 531
 - calledby checkHTargs, 506
 - calledby checkRecordHash, 507
 - local ref \$charBlank, 531
 - defun, 531
- checkLookForRightBrace, 531
 - calledby checkHTargs, 506
 - calledby checkRecordHash, 507
 - defun, 531
- checkNumOfArgs, 532
 - calledby checkRecordHash, 507
 - calls abbreviation?, 532
 - calls constructor?, 532
 - calls getdatabase, 532
 - calls opOf, 532
 - defun, 532
- checkRecordHash, 507
 - calledby checkRewrite, 497
 - calls checkDocError, 507
 - calls checkGetLispFunctionName, 507
 - calls checkGetParse, 507
 - calls checkGetStringBeforeRightBrace, 507
 - calls checkIsValidType, 507
 - calls checkLookForLeftBrace, 507
 - calls checkLookForRightBrace, 507
 - calls checkNumOfArgs, 507
 - calls form2HtString, 507
 - calls getl, 507
 - calls hget, 507
 - calls hput, 507
 - calls ifcdr, 507
 - calls intern, 507
 - calls member, 507
 - calls opOf, 507
 - calls spadSysChoose, 507
 - local def \$glossHash, 508
 - local def \$htHash, 508
 - local def \$lispHash, 508
 - local def \$sysHash, 508
 - local ref \$HTlinks, 507
 - local ref \$HTlisplinks, 507
 - local ref \$currentSysList, 507
 - local ref \$glossHash, 507
 - local ref \$htHash, 507
 - local ref \$lispHash, 507
 - local ref \$name, 507
 - local ref \$origin, 508
 - local ref \$setOptions, 507
 - local ref \$sysHash, 507
 - defun, 507
- checkRemoveComments, 516
 - calledby checkRewrite, 497
 - calls checkTrimCommented, 516
 - defun, 516
- checkRewrite, 497
 - calledby transformAndRecheckComments, 495
 - calls checkAddIndented, 497
 - calls checkAddMacros, 497
 - calls checkAddSpaces, 497
 - calls checkArguments, 497
 - calls checkDecorateForHt, 497
 - calls checkFixCommonProblem, 497
 - calls checkGetArgs, 497
 - calls checkRecordHash, 497
 - calls checkRemoveComments, 497
 - calls checkSplit2Words, 497
 - calls checkTexht, 497
 - calls newString2Words, 497
 - local ref \$argl, 498
 - local ref \$checkErrorFlag, 498
 - local ref \$checkingXmptex?, 498
 - defun, 497
- checkSayBracket, 532
 - calledby checkBalance, 499
 - defun, 532
- checkSkipBlanks, 533
 - calledby checkTransformFirsts, 512
 - local ref \$charBlank, 533
 - defun, 533
- checkSkipIdentifierToken, 523
 - calledby checkSkipToken, 517
 - calls checkAlphabetic, 523
 - defun, 523
- checkSkipOpToken, 523
 - calledby checkSkipToken, 517

- calls checkAlphabetic, 523
 - calls member, 523
 - local ref \$charDelimiters, 524
 - defun, 523
- checkSkipToken, 517
 - calledby checkTransformFirsts, 512
 - calls checkSkipIdentifierToken, 517
 - calls checkSkipOpToken, 517
 - defun, 517
- checkSplit2Words, 517
 - calledby checkComments, 496
 - calledby checkRewrite, 497
 - calls checkSplitBrace, 517
 - defun, 517
- checkSplitBackslash, 533
 - calledby checkSplitBackslash, 533
 - calledby checkSplitBrace, 524
 - calls charPosition, 533
 - calls checkSplitBackslash, 533
 - calls maxindex, 533
 - defun, 533
- checkSplitBrace, 524
 - calledby checkSplit2Words, 517
 - calledby checkSplitBrace, 524
 - calls charp, 524
 - calls checkSplitBackslash, 524
 - calls checkSplitBrace, 524
 - calls checkSplitOn, 524
 - calls checkSplitPunctuation, 524
 - calls length, 524
 - defun, 524
- checkSplitOn, 534
 - calledby checkSplitBrace, 524
 - calledby checkSplitOn, 534
 - calls charPosition, 534
 - calls charp, 534
 - calls checkSplitOn, 534
 - calls maxindex, 534
 - local ref \$charSplitList, 534
 - defun, 534
- checkSplitPunctuation, 535
 - calledby checkSplitBrace, 524
 - calledby checkSplitPunctuation, 535
 - calls charPosition, 535
 - calls charp, 535
 - calls checkSplitPunctuation, 535
- calls hget, 535
 - calls maxindex, 535
 - local ref \$htMacroTable, 535
 - defun, 535
- checkTexht, 511
 - calledby checkRewrite, 497
 - calls checkDocError, 511
 - calls ifcar, 511
 - calls ifcdr, 511
 - defun, 511
- checkTransformFirsts, 512
 - calledby checkComments, 496
 - calledby checkTransformFirsts, 512
 - calls checkDocError, 512
 - calls checkSkipBlanks, 512
 - calls checkSkipToken, 512
 - calls checkTransformFirsts, 512
 - calls fillerSpaces, 512
 - calls getMatchingRightPren, 512
 - calls getl, 512
 - calls lassoc, 512
 - calls leftTrim, 512
 - calls maxindex, 512
 - calls pname, 512
 - calls strconc, 512
 - local ref \$checkPrenAlist, 512
 - defun, 512
- checkTrim, 515
 - calledby transDoc, 494
 - calls charPosition, 515
 - calls checkDocError, 515
 - calls systemError, 515
 - local ref \$charBlank, 515
 - local ref \$charPlus, 515
 - local ref \$x, 515
 - defun, 515
- checkTrimCommented, 525
 - calledby checkRemoveComments, 516
 - calls htcharPosition, 525
 - calls length, 525
 - defun, 525
- checkWarning, 547
 - calledby postCapsule, 377
 - calls concat, 547
 - calls postError, 547
 - defun, 547

- clearClams
 - calledby compileConstructor, 187
- clearConstructorCache
 - calledby compileConstructor1, 188
- coerce, 355
 - calledby coerceExit, 361
 - calledby coerceExtraHard, 359
 - calledby coerceable, 360
 - calledby compApplication, 603
 - calledby compApplyModemap, 259
 - calledby compAtSign, 361
 - calledby compCase, 287
 - calledby compCoerce1, 363
 - calledby compCoerce, 362
 - calledby compColonInside, 594
 - calledby compForm1, 600
 - calledby compHas, 313
 - calledby compIf, 315
 - calledby compIs, 322
 - calledby convert, 597
 - calls coerceEasy, 355
 - calls coerceHard, 356
 - calls coerceSubset, 356
 - calls isSomeDomainVariable, 356
 - calls keyedSystemError, 355
 - calls rplac, 355
 - calls stackMessage, 356
 - local ref \$InteractiveMode, 356
 - local ref \$Rep, 356
 - local ref \$fromCoerceable, 356
 - defun, 355
- coerceable, 360
 - calledby compFocompFormWithModemap, 610
 - calledby compForm1, 600
 - calls coerce, 360
 - calls pmatch, 360
 - calls sublis, 360
 - local ref \$fromCoerceable, 360
 - defun, 360
- coerceByModemap, 363
 - calledby compCoerce1, 363
 - calls genDeltaEntry, 363
 - calls isSubset, 363
 - calls modeEqual, 363
 - defun, 363
- coerceEasy, 356
 - calledby coerce, 355
 - calls modeEqualSubst, 356
 - local ref \$EmptyMode, 356
 - local ref \$Exit, 356
 - local ref \$NoValueMode, 356
 - local ref \$Void, 356
 - defun, 356
- coerceExit, 361
 - calledby compRepeatOrCollect, 333
 - calls coerce, 361
 - calls replaceExitEsc, 361
 - calls resolve, 361
 - local ref \$exitMode, 361
 - defun, 361
- coerceExtraHard, 359
 - calledby coerceHard, 358
 - calls autoCoerceByModemap, 359
 - calls coerce, 359
 - calls hasType, 359
 - calls isUnionMode, 359
 - calls member, 359
 - local ref \$Expression, 359
 - defun, 359
- coerceHard, 358
 - calledby coerce, 356
 - calls coerceExtraHard, 358
 - calls extendsCategoryForm, 358
 - calls getmode, 358
 - calls get, 358
 - calls isCategoryForm, 358
 - calls modeEqual, 358
 - local def \$e, 358
 - local ref \$String, 358
 - local ref \$bootStrapMode, 358
 - local ref \$e, 358
 - defun, 358
- coerceSubset, 357
 - calledby coerce, 356
 - calls eval, 357
 - calls get, 357
 - calls isSubset, 357
 - calls lassoc, 357
 - calls maxSuperType, 357
 - calls opOf, 357
 - defun, 357

- collect, 332, 383
 - calledby floatexpid, 462
 - defplist, 332, 383
- collectAndDeleteAssoc, 470
 - calledby collectComBlock, 469
 - local ref \$comblocklist, 470
 - defun, 470
- collectComBlock, 469
 - calledby recordDocumentation, 469
 - calls collectAndDeleteAssoc, 469
 - local def \$comblocklist, 469
 - defun, 469
- comma2Tuple, 386
 - calledby postComma, 386
 - calledby postConstruct, 387
 - calls postFlatten, 386
 - defun, 386
- comp, 587
 - calledby applyMapping, 591
 - calledby compAdd, 275
 - calledby compApplication, 603
 - calledby compApplyModemap, 259
 - calledby compApply, 592
 - calledby compArgumentsAndTryAgain, 614
 - calledby compAtSign, 361
 - calledby compBoolean, 318
 - calledby compCase1, 288
 - calledby compCoerce1, 363
 - calledby compColonInside, 594
 - calledby compCons1, 298
 - calledby compDefWhereClause, 160
 - calledby compDefineAddSignature, 147
 - calledby compExit, 312
 - calledby compExpressionList, 607
 - calledby compForMode, 325
 - calledby compForm1, 600
 - calledby compFromIf, 316
 - calledby compHasFormat, 313
 - calledby compIs, 322
 - calledby compLeave, 327
 - calledby compList, 599
 - calledby compOrCroak1, 586
 - calledby compPretend, 329
 - calledby compReduce1, 330
 - calledby compRepeatOrCollect, 333
 - calledby compReturn, 335
 - calledby compSeqItem, 338
 - calledby compSubsetCategory, 352
 - calledby compSuchthat, 353
 - calledby compUniquely, 613
 - calledby compVector, 354
 - calledby compWhere, 354
 - calledby compWithMappingMode1, 615
 - calledby compileConstructor1, 188
 - calledby doItIf, 285
 - calledby getSuccessEnvironment, 319
 - calledby outputComp, 347
 - calledby setqSetelt, 344
 - calledby setqSingle, 344
 - calledby spadCompileOrSetq, 186
 - calls compNoStacking, 587
 - local ref \$compStack, 587
 - uses \$exitModeStack, 587
 - defun, 587
- comp-tran
 - calledby compWithMappingMode1, 615
- comp2, 589
 - calledby compNoStacking1, 588
 - calledby compNoStacking, 588
 - calls addDomain, 589
 - calls comp3, 589
 - calls insert, 589
 - calls isDomainForm, 589
 - calls isFunctor, 589
 - calls opOf, 589
 - uses \$bootStrapMode, 589
 - uses \$lisplib, 589
 - uses \$packagesUsed, 589
 - defun, 589
- comp3, 589
 - calledby comp2, 589
 - calledby compTypeOf, 593
 - calls addDomain, 589
 - calls applyMapping, 590
 - calls compApply, 590
 - calls compAtom, 589
 - calls compCoerce, 590
 - calls compColon, 590
 - calls compExpression, 590
 - calls compTypeOf, 590
 - calls compWithMappingMode, 589
 - calls getDomainsInScope, 590

- calls getmode, 589
- calls member[5], 590
- calls pname[5], 590
- calls stringPrefix?, 590
- uses \$e, 590
- uses \$insideCompTypeOf, 590
- defun, 589
- compAdd, 275
 - calls NRTgetLocalIndex, 275
 - calls compCapsule, 276
 - calls compOrCroak, 276
 - calls compSubDomain1, 275
 - calls compTuple2Record, 276
 - calls comp, 275
 - calls nreverse0, 275
 - uses /editfile, 276
 - uses \$EmptyMode, 276
 - uses \$NRTaddForm, 276
 - uses \$addFormLhs, 276
 - uses \$addForm, 276
 - uses \$bootStrapMode, 276
 - uses \$functorForm, 276
 - uses \$packagesUsed, 276
 - defun, 275
- compAndDefine, 189
 - calledby compileConstructor1, 188
 - defun, 189
- compApplication, 603
 - calledby compToApply, 602
 - calls coerce, 603
 - calls comp, 603
 - calls eltForm, 603
 - calls encodeItem, 603
 - calls getAbbreviation, 603
 - calls isCategoryForm, 603
 - calls length, 603
 - calls member, 603
 - calls resolve, 603
 - calls strconc, 603
 - local ref \$Category, 603
 - local ref \$formatArgList, 603
 - local ref \$form, 603
 - local ref \$op, 603
 - local ref \$prefix, 603
 - defun, 603
- compApply, 592
 - calledby comp3, 590
 - calls AddContour, 592
 - calls Pair, 592
 - calls comp, 592
 - calls removeEnv, 592
 - calls resolve, 592
 - local ref \$EmptyMode, 592
 - defun, 592
- compApplyModemap, 259
 - calledby compFocompFormWithModemap, 610
 - calledby getModemap, 259
 - calls coerce, 259
 - calls compMapCond, 259
 - calls comp, 259
 - calls genDeltaEntry, 259
 - calls length, 259
 - calls member, 259
 - calls pmatchWithSl, 259
 - calls sublis, 259
 - local def \$bindings, 259
 - local def \$e, 259
 - local ref \$bindings, 259
 - local ref \$e, 259
 - defun, 259
- compareMode2Arg
 - calledby hasSigInTargetCategory, 309
- compArgumentConditions, 170
 - calledby compDefineCapsuleFunction, 155
 - calls compOrCroak, 170
 - local def \$argumentConditionList, 170
 - local ref \$Boolean, 170
 - local ref \$argumentConditionList, 170
 - defun, 170
- compArgumentsAndTryAgain, 614
 - calledby compForm, 600
 - calls compForm1, 614
 - calls comp, 614
 - uses \$EmptyMode, 614
 - defun, 614
- compAtom, 595
 - calledby comp3, 589
 - calls compAtomWithModemap, 595
 - calls compList, 595
 - calls compSymbol, 595
 - calls compVector, 595

- calls convert, 595
- calls get, 595
- calls isSymbol, 595
- calls modeIsAggregateOf, 595
- calls primitiveType, 595
- uses \$Expression, 595
- defun, 595
- compAtomWithModemap, 596
 - calledby compAtom, 595
 - calls convert, 596
 - calls modeEqual, 596
 - calls transImplementation, 596
 - local ref \$NoValueMode, 596
 - defun, 596
- compAtSign, 361
 - calledby compLambda, 325
 - calls addDomain, 361
 - calls coerce, 361
 - calls comp, 361
 - defun, 361
- compBoolean, 318
 - calledby compIf, 315
 - calls comp, 318
 - calls getInverseEnvironment, 318
 - calls getSuccessEnvironment, 318
 - defun, 318
- compCapsule, 278
 - calledby compAdd, 276
 - calledby compSubDomain, 350
 - calls addDomain, 278
 - calls bootStrapError, 278
 - calls compCapsuleInner, 278
 - uses \$bootStrapMode, 278
 - uses \$functorForm, 278
 - uses \$insideExpressionIfTrue, 278
 - uses editfile, 278
 - defun, 278
- compCapsuleInner, 278
 - calledby compCapsule, 278
 - calls addInformation, 278
 - calls compCapsuleItems, 278
 - calls mkpf, 278
 - calls processFunctor, 278
 - uses \$addForm, 278
 - uses \$form, 278
 - uses \$functorLocalParameters, 279
 - uses \$getDomainCode, 278
 - uses \$insideCategoryIfTrue, 278
 - uses \$insideCategoryPackageIfTrue, 278
 - uses \$signature, 278
 - defun, 278
- compCapsuleItems, 279
 - calledby compCapsuleInner, 278
 - calls compSingleCapsuleItem, 280
 - local def \$e, 280
 - local def \$myFunctorBody, 280
 - local def \$signatureOfForm, 280
 - local def \$suffix, 280
 - local def \$top-level, 280
 - local ref \$e, 280
 - local ref \$pred, 280
 - defun, 279
- compCase, 287
 - calls addDomain, 287
 - calls coerce, 287
 - calls compCase1, 287
 - defun, 287
- compCase1, 288
 - calledby compCase, 287
 - calls comp, 288
 - calls getModemapList, 288
 - calls modeEqual, 288
 - calls nreverse0, 288
 - uses \$Boolean, 288
 - uses \$EmptyMode, 288
 - defun, 288
- compCat, 289
 - calls getl, 289
 - defun, 289
- compCategory, 290
 - calls compCategoryItem, 290
 - calls mkExplicitCategoryFunction, 290
 - calls resolve, 290
 - calls systemErrorHere, 290
 - local def \$atList, 290
 - local def \$sigList, 290
 - local def \$top-level, 290
 - local ref \$atList, 290
 - local ref \$sigList, 290
 - defun, 290
- compCategoryItem, 291
 - calledby compCategoryItem, 291

- calledby compCategory, 290
- calls compCategoryItem, 291
- calls mkpf, 291
- local ref \$atList, 291
- local ref \$sigList, 291
- defun, 291
- compCoerce, 362
 - calledby comp3, 590
 - calls addDomain, 362
 - calls coerce, 362
 - calls compCoerce1, 362
 - calls getmode, 362
 - defun, 362
- compCoerce1, 363
 - calledby compCoerce, 362
 - calls coerceByModemap, 363
 - calls coerce, 363
 - calls comp, 363
 - calls mkq, 363
 - calls resolve, 363
 - defun, 363
- compColon, 294
 - calledby comp3, 590
 - calledby compColon, 294
 - calledby compMakeDeclaration, 622
 - calls addDomain, 294
 - calls assoc, 294
 - calls compColonInside, 294
 - calls compColon, 294
 - calls eqsubstlist, 294
 - calls genSomeVariable, 295
 - calls getDomainsInScope, 294
 - calls getmode, 294
 - calls isCategoryForm, 294
 - calls isDomainForm, 294
 - calls length, 294
 - calls makeCategoryForm, 294
 - calls member[5], 294
 - calls nreverse0, 294
 - calls put, 294
 - calls systemErrorHere, 294
 - calls take, 294
 - calls unknownTypeError, 294
 - uses \$FormalMapVariableList, 295
 - uses \$bootStrapMode, 295
 - uses \$insideCategoryIfTrue, 295
 - uses \$insideExpressionIfTrue, 295
 - uses \$insideFunctorIfTrue, 295
 - uses \$lhsOfColon, 295
 - uses \$noEnv, 295
 - defun, 294
- compColonInside, 594
 - calledby compColon, 294
 - calls addDomain, 594
 - calls coerce, 594
 - calls comp, 594
 - calls opOf, 594
 - calls stackSemanticError, 594
 - calls stackWarning, 594
 - uses \$EmptyMode, 594
 - uses \$newCompilerUnionFlag, 594
 - defun, 594
- compCons, 298
 - calls compCons1, 298
 - calls compForm, 298
 - defun, 298
- compCons1, 298
 - calledby compCons, 298
 - calls comp, 298
 - calls convert, 298
 - uses \$EmptyMode, 298
 - defun, 298
- compConstruct, 299
 - calls compForm, 299
 - calls compList, 299
 - calls compVector, 299
 - calls convert, 299
 - calls getDomainsInScope, 299
 - calls modeIsAggregateOf, 299
 - defun, 299
- compConstructorCategory, 301
 - calls resolve, 301
 - uses \$Category, 301
 - defun, 301
- compDefine, 144
 - calls compDefine1, 144
 - local def \$macroIfTrue, 144
 - local def \$packagesUsed, 145
 - local def \$tripleCache, 144
 - local def \$tripleHits, 144
 - defun, 144
- compDefine1, 145

- calledby compDefine1, 145
- calledby compDefineCategory1, 162
- calledby compDefine, 144
- calls addEmptyCapsuleIfNecessary, 145
- calls compDefWhereClause, 145
- calls compDefine1, 145
- calls compDefineAddSignature, 145
- calls compDefineCapsuleFunction, 145
- calls compDefineCategory, 145
- calls compDefineFunctor, 145
- calls compInternalFunction, 145
- calls getAbbreviation, 145
- calls getSignatureFromMode, 145
- calls getTargetFromRhs, 145
- calls giveFormalParametersValues, 145
- calls isDomainForm, 145
- calls isMacro, 145
- calls length, 145
- calls macroExpand, 145
- calls stackAndThrow, 145
- calls strconc, 145
- uses \$Category, 145
- uses \$ConstructorNames, 146
- uses \$EmptyMode, 146
- uses \$NoValueMode, 146
- uses \$formalArgList, 145
- uses \$form, 145
- uses \$insideCapsuleFunctionIfTrue, 145
- uses \$insideCategoryIfTrue, 145
- uses \$insideExpressionIfTrue, 145, 146
- uses \$insideFunctorIfTrue, 145
- uses \$insideWhereIfTrue, 146
- uses \$op, 145
- uses \$prefix, 145
- defun, 145
- compDefineAddSignature, 147
 - calledby compDefine1, 145
 - calls assoc, 147
 - calls comp, 147
 - calls getProplist, 147
 - calls hasFullSignature, 147
 - calls lassoc, 147
 - uses \$EmptyMode, 147
 - defun, 147
- compDefineCapsuleFunction, 155
 - calledby compDefine1, 145
 - calls NRTassignCapsuleFunctionSlot, 156
 - calls addArgumentConditions, 156
 - calls addDomain, 155
 - calls addStats, 156
 - calls checkAndDeclare, 155
 - calls compArgumentConditions, 155
 - calls compOrCroak, 156
 - calls compileCases, 156
 - calls formatUnabbreviated, 156
 - calls getArgumentModeOrMoan, 155
 - calls getSignature, 155
 - calls getmode, 156
 - calls get, 155
 - calls giveFormalParametersValues, 155
 - calls hasSigInTargetCategory, 155
 - calls length, 155
 - calls member, 156
 - calls mkq, 156
 - calls profileRecord, 155
 - calls put, 155
 - calls replaceExitEtc, 156
 - calls resolve, 156
 - calls sayBrightly, 156
 - calls stripOffArgumentConditions, 155
 - calls stripOffSubdomainConditions, 155
 - local def \$CapsuleDomainsInScope, 156
 - local def \$CapsuleModemapFrame, 156
 - local def \$argumentConditionList, 156
 - local def \$finalEnv, 156
 - local def \$formalArgList, 156
 - local def \$form, 156
 - local def \$functionLocations, 156
 - local def \$functionStats, 156
 - local def \$initCapsuleErrorCount, 156
 - local def \$insideCapsuleFunctionIfTrue, 156
 - local def \$insideExpressionIfTrue, 156
 - local def \$op, 156
 - local def \$returnMode, 156
 - local def \$signatureOfForm, 156
 - local ref \$DomainsInScope, 156
 - local ref \$compileOnlyCertainItems, 156
 - local ref \$formalArgList, 156
 - local ref \$functionLocations, 156
 - local ref \$functionStats, 156
 - local ref \$functorStats, 156

- local ref \$op, 156
- local ref \$profileCompiler, 156
- local ref \$returnMode, 156
- local ref \$semanticErrorStack, 156
- local ref \$signatureOfForm, 156
- defun, 155
- compDefineCategory, 162
 - calledby compDefine1, 145
 - calls compDefineCategory1, 162
 - calls compDefineLisplib, 162
 - uses \$domainShell, 162
 - uses \$insideFunctorIfTrue, 162
 - uses \$lisplibCategory, 162
 - uses \$lisplib, 162
 - defun, 162
- compDefineCategory1, 162
 - calledby compDefineCategory, 162
 - calls compDefine1, 162
 - calls compDefineCategory2, 162
 - calls makeCategoryPredicates, 162
 - calls mkCategoryPackage, 162
 - uses \$EmptyMode, 162
 - uses \$bootstrapMode, 162
 - uses \$categoryPredicateList, 162
 - uses \$insideCategoryPackageIfTrue, 162
 - uses \$lisplibCategory, 162
 - defun, 162
- compDefineCategory2, 163
 - calledby compDefineCategory1, 162
 - calls addBinding, 163
 - calls augLisplibModemapsFromCategory, 164
 - calls compMakeDeclaration, 163
 - calls compOrCroak, 164
 - calls compile, 164
 - calls computeAncestorsOf, 164
 - calls constructor?, 164
 - calls evalAndRwriteLispForm, 164
 - calls eval, 164
 - calls getArgumentModeOrMoan, 163
 - calls getParentsFor, 164
 - calls giveFormalParametersValues, 163
 - calls lisplibWrite, 164
 - calls mkConstructor, 164
 - calls mkq, 164
 - calls opOf, 164
- calls optFunctorBody, 164
- calls removeZeroOne, 164
- calls sublis, 163
- calls take, 163
- local def \$addForm, 164
- local def \$definition, 164
- local def \$domainShell, 164
- local def \$extraParms, 164
- local def \$formalArgList, 164
- local def \$form, 164
- local def \$frontier, 164
- local def \$functionStats, 164
- local def \$functorForm, 164
- local def \$functorStats, 164
- local def \$getDomainCode, 164
- local def \$insideCategoryIfTrue, 164
- local def \$lisplibAbbreviation, 164
- local def \$lisplibAncestors, 164
- local def \$lisplibCategory, 164
- local def \$lisplibForm, 164
- local def \$lisplibKind, 164
- local def \$lisplibModemap, 164
- local def \$lisplibParents, 164
- local def \$op, 164
- local def \$top-level, 164
- local ref \$FormalMapVariableList, 164
- local ref \$TriangleVariableList, 164
- local ref \$definition, 164
- local ref \$extraParms, 164
- local ref \$formalArgList, 164
- local ref \$form, 164
- local ref \$libFile, 164
- local ref \$lisplibCategory, 164
- local ref \$lisplib, 164
- local ref \$op, 164
- uses \$prefix, 164
- defun, 163
- compDefineFunctor, 148
 - calledby compDefine1, 145
 - calls compDefineFunctor1, 148
 - calls compDefineLisplib, 148
 - uses \$domainShell, 148
 - uses \$lisplib, 148
 - uses \$profileAlist, 148
 - uses \$profileCompiler, 148
 - defun, 148

- compDefineFunctor1, 148
 - calledby compDefineFunctor, 148
 - calls NRTgenInitialAttributeAlist, 148
 - calls NRTgetLocalIndex, 148
 - calls NRTgetLookupFunction, 149
 - calls NRTmakeSlot1Info, 149
 - calls augModemapsFromCategoryRep, 148
 - calls augModemapsFromCategory, 148
 - calls augmentLisplibModemapsFromFunc-
tor, 149
 - calls compFunctorBody, 149
 - calls compMakeCategoryObject, 148
 - calls compMakeDeclaration, 148
 - calls compile, 149
 - calls computeAncestorsOf, 149
 - calls constructor?, 149
 - calls disallowNilAttribute, 148
 - calls evalAndRwriteLispForm, 149
 - calls getArgumentModeOrMoan, 148
 - calls getModemap, 148
 - calls getParentsFor, 149
 - calls getdatabase, 149
 - calls giveFormalParametersValues, 148
 - calls isCategoryPackageName, 148, 149
 - calls lisplibWrite, 149
 - calls makeFunctorArgumentParameters,
148
 - calls maxindex, 148
 - calls mkq, 149
 - calls pname, 148
 - calls pp, 148
 - calls remdup, 148
 - calls removeZeroOne, 149
 - calls reportOnFunctorCompilation, 149
 - calls sayBrightly, 148
 - calls simpBool, 149
 - calls strconc, 148
 - calls sublis, 148
 - uses \$CategoryFrame, 149
 - uses \$CheckVectorList, 149
 - uses \$FormalMapVariableList, 149
 - uses \$LocalDomainAlist, 149
 - uses \$NRTaddForm, 149
 - uses \$NRTaddList, 149
 - uses \$NRTattributeAlist, 149
 - uses \$NRTbase, 149
 - uses \$NRTdeltaLength, 149
 - uses \$NRTdeltaListComp, 149
 - uses \$NRTdeltaList, 149
 - uses \$NRTdomainFormList, 149
 - uses \$NRTloadTimeAlist, 149
 - uses \$NRTslot1Info, 149
 - uses \$NRTslot1PredicateList, 149
 - uses \$QuickCode, 150
 - uses \$Representation, 149
 - uses \$addForm, 149
 - uses \$attributesName, 149
 - uses \$bootStrapMode, 149
 - uses \$byteAddress, 149
 - uses \$byteVec, 149
 - uses \$compileOnlyCertainItems, 149
 - uses \$condAlist, 149
 - uses \$domainShell, 149
 - uses \$form, 149
 - uses \$functionLocations, 149
 - uses \$functionStats, 149
 - uses \$functorForm, 150
 - uses \$functorLocalParameters, 150
 - uses \$functorSpecialCases, 150
 - uses \$functorStats, 150
 - uses \$functorTarget, 150
 - uses \$functorsUsed, 150
 - uses \$genFVar, 150
 - uses \$genSDVar, 150
 - uses \$getDomainCode, 150
 - uses \$goGetList, 150
 - uses \$insideCategoryPackageIfTrue, 150
 - uses \$insideFunctorIfTrue, 150
 - uses \$isOpPackageName, 150
 - uses \$libFile, 150
 - uses \$lisplibAbbreviation, 150
 - uses \$lisplibAncestors, 150
 - uses \$lisplibCategoriesExtended, 150
 - uses \$lisplibCategory, 150
 - uses \$lisplibForm, 150
 - uses \$lisplibFunctionLocations, 150
 - uses \$lisplibKind, 150
 - uses \$lisplibMissingFunctions, 150
 - uses \$lisplibModemap, 150
 - uses \$lisplibOperationAlist, 150
 - uses \$lisplibParents, 150
 - uses \$lisplibSlot1, 150

- uses \$lisplib, 149
- uses \$lookupFunction, 150
- uses \$mutableDomains, 150
- uses \$mutableDomain, 150
- uses \$myFunctorBody, 150
- uses \$op, 150
- uses \$pairlis, 150
- uses \$setelt, 150
- uses \$signature, 150
- uses \$template, 150
- uses \$top-level, 149
- uses \$uncondAlist, 150
- uses \$viewNames, 150
- defun, 148
- compDefineLisplib, 167
 - calledby compDefineCategory, 162
 - calledby compDefineFunctor, 148
 - calls bright, 167
 - calls compileDocumentation, 167
 - calls filep, 167
 - calls fillerSpaces, 167
 - calls finalizeLisplib, 167
 - calls getConstructorAbbreviation, 167
 - calls getdatabase, 167
 - calls lisplibDoRename, 167
 - calls localdatabase, 167
 - calls rpackfile, 167
 - calls rshut, 167
 - calls sayMSG, 167
 - calls unloadOneConstructor, 167
 - calls updateCategoryFrameForCategory, 167
 - calls updateCategoryFrameForConstructor, 167
 - local def \$libFile, 168
 - local def \$lisplibAbbreviation, 168
 - local def \$lisplibAncestors, 168
 - local def \$lisplibCategoriesExtended, 167
 - local def \$lisplibCategory, 168
 - local def \$lisplibForm, 167
 - local def \$lisplibKind, 168
 - local def \$lisplibModemapAlist, 168
 - local def \$lisplibModemap, 168
 - local def \$lisplibOperationAlist, 168
 - local def \$lisplibParents, 167
 - local def \$lisplibPredicates, 167
 - local def \$lisplibSlot1, 168
 - local def \$lisplibSuperDomain, 168
 - local def \$lisplibVariableAlist, 168
 - local def \$lisplib, 167
 - local def \$newConlist, 168
 - local def \$op, 167
 - local ref \$algebraOutputStream, 167
 - local ref \$compileDocumentation, 167
 - local ref \$filep, 167
 - local ref \$lisplibKind, 167
 - local ref \$newConlist, 167
 - local ref \$spadLibFT, 167
 - defun, 167
- compDefWhereClause, 159
 - calledby compDefine1, 145
 - calls assocleft, 160
 - calls assocright, 160
 - calls comp, 160
 - calls concat, 159
 - calls delete, 160
 - calls getmode, 159
 - calls lassoc, 160
 - calls listOfIdentifiersIn, 160
 - calls orderByDependency, 160
 - calls pairList, 160
 - calls union, 160
 - calls userError, 159
 - uses \$predAlist, 160
 - uses \$sigAlist, 160
 - defun, 159
- compElt, 310
 - calls addDomain, 310
 - calls compForm, 310
 - calls convert, 310
 - calls getDeltaEntry, 310
 - calls getModemapListFromDomain, 310
 - calls isDomainForm, 310
 - calls length, 310
 - calls opOf, 310
 - calls stackMessage, 310
 - calls stackWarning, 310
 - uses \$One, 310
 - uses \$Zero, 310
 - defun, 310
- compExit, 312
 - calls comp, 312

- calls modifyModeStack, 312
 - calls stackMessageIfNone, 312
 - uses \$exitModeStack, 312
 - defun, 312
- compExpression, 139
 - calledby comp3, 590
 - calls compForm, 139
 - calls getl, 139
 - uses \$insideExpressionIfTrue, 139
 - defun, 139
- compExpressionList, 607
 - calledby compForm1, 600
 - calls comp, 607
 - calls convert, 607
 - calls nreverse0, 607
 - local ref \$Expression, 607
 - defun, 607
- compFocompFormWithModemap, 610
 - calls coerceable, 610
 - calls compApplyModemap, 610
 - calls convert, 610
 - calls get, 610
 - calls identp, 610
 - calls isCategoryForm, 610
 - calls isFunctor, 610
 - calls last, 610
 - calls listOfSharpVars, 610
 - calls substituteIntoFunctorModemap, 610
 - local ref \$Category, 610
 - local ref \$FormalMapVariableList, 610
 - defun, 610
- compForm, 600
 - calledby compConstruct, 299
 - calledby compCons, 298
 - calledby compElt, 310
 - calledby compExpression, 139
 - calls compArgumentsAndTryAgain, 600
 - calls compForm1, 600
 - calls stackMessageIfNone, 600
 - defun, 600
- compForm1, 600
 - calledby compArgumentsAndTryAgain, 614
 - calledby compForm, 600
 - calls addDomain, 600
 - calls augModemapsFromDomain1, 600
 - calls coerceable, 600
 - calls coerce, 600
 - calls compExpressionList, 600
 - calls compForm2, 600
 - calls compOrCroak, 600
 - calls compToApply, 600
 - calls comp, 600
 - calls getFormModemaps, 600
 - calls length, 600
 - calls nreverse0, 600
 - calls outputComp, 600
 - uses \$EmptyMode, 601
 - uses \$Expression, 601
 - uses \$NumberOfArgsIfInteger, 601
 - defun, 600
- compForm2, 607
 - calledby compForm1, 600
 - calls PredImplies, 608
 - calls assoc, 607
 - calls compForm3, 608
 - calls compFormPartiallyBottomUp, 608
 - calls compUniquely, 608
 - calls isSimple, 608
 - calls length, 607
 - calls nreverse0, 607
 - calls sublis, 607
 - calls take, 607
 - uses \$EmptyMode, 608
 - uses \$TriangleVariableList, 608
 - defun, 607
- compForm3, 609
 - calledby compForm2, 608
 - calledby compFormPartiallyBottomUp, 613
 - calls compFormWithModemap, 609
 - local ref \$compUniquelyIfTrue, 609
 - defun, 609
 - throws, 609
- compFormMatch, 613
 - calledby compFormPartiallyBottomUp, 613
 - defun, 613
- compForMode, 325
 - calledby compJoin, 323
 - calls comp, 325
 - local def \$compForModeIfTrue, 325
 - defun, 325
- compFormPartiallyBottomUp, 613
 - calledby compForm2, 608

- calls compForm3, 613
 - calls compFormMatch, 613
 - defun, 613
- compFormWithModemap
 - calledby compForm3, 609
- compFromIf, 316
 - calledby compIf, 315
 - calls comp, 316
 - defun, 316
- compFunctorBody, 172
 - calledby compDefineFunctor1, 149
 - calls bootStrapError, 172
 - calls compOrCroak, 172
 - uses /editfile, 172
 - uses \$NRTaddForm, 172
 - uses \$bootStrapMode, 172
 - uses \$functorForm, 172
 - defun, 172
- compHas, 313
 - calls chaseInferences, 313
 - calls coerce, 313
 - calls compHasFormat, 313
 - local def \$e, 313
 - local ref \$Boolean, 313
 - local ref \$e, 313
 - defun, 313
- compHasFormat, 313
 - calledby compHas, 313
 - calls comp, 313
 - calls isDomainForm, 313
 - calls length, 313
 - calls mkDomainConstructor, 313
 - calls mkList, 313
 - calls sublis, 313
 - calls take, 313
 - local ref \$EmptyEnvironment, 313
 - local ref \$EmptyMode, 313
 - local ref \$FormalMapVariableList, 313
 - local ref \$e, 313
 - local ref \$form, 313
 - defun, 313
- compIf, 315
 - calls canReturn, 315
 - calls coerce, 315
 - calls compBoolean, 315
 - calls compFromIf, 315
 - calls intersectionEnvironment, 315
 - calls quotify, 315
 - calls resolve, 315
 - uses \$Boolean, 315
 - defun, 315
- compile, 173
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby compileCases, 171
 - calls addStats, 173
 - calls constructMacro, 173
 - calls elapsedTime, 173
 - calls encodeFunctionName, 173
 - calls encodeItem, 173
 - calls getmode, 173
 - calls get, 173
 - calls member, 173
 - calls modeEqual, 173
 - calls optimizeFunctionDef, 173
 - calls printStats, 173
 - calls putInLocalDomainReferences, 173
 - calls sayBrightly, 173
 - calls spadCompileOrSetq, 173
 - calls splitEncodedFunctionName, 173
 - calls strconc, 173
 - calls userError, 173
 - local def \$functionStats, 174
 - local def \$savableItems, 174
 - local def \$suffix, 174
 - local ref \$compileOnlyCertainItems, 173
 - local ref \$doNotCompileJustPrint, 173
 - local ref \$e, 174
 - local ref \$functionStats, 173
 - local ref \$functorForm, 173
 - local ref \$insideCapsuleFunctionIfTrue, 173
 - local ref \$lisplibItemsAlreadyThere, 173
 - local ref \$lisplib, 173
 - local ref \$macroIfTrue, 173
 - local ref \$prefix, 174
 - local ref \$saveableItems, 173
 - local ref \$signatureOfForm, 173, 174
 - local ref \$splitUpItemsAlreadyThere, 173
 - local ref \$suffix, 173
 - defun, 173
- compile-lib-file, 625

- calledby recompile-lib-file-if-necessary, 625
 - defun, 625
- compileCases, 171
 - calledby compDefineCapsuleFunction, 156
 - calls assocleft, 171
 - calls assocright, 171
 - calls compile, 171
 - calls eval, 171
 - calls getSpecialCaseAssoc, 171
 - calls get, 171
 - calls mkpf, 171
 - calls outerProduct, 171
 - local def \$specialCaseKeyList, 171
 - local ref \$getDomainCode, 171
 - local ref \$insideFunctorIfTrue, 171
 - defun, 171
- compileConstructor, 187
 - calledby spadCompileOrSetq, 186
 - calls clearClams, 187
 - calls compileConstructor1, 187
 - defun, 187
- compileConstructor1, 188
 - calledby compileConstructor, 187
 - calls clearConstructorCache, 188
 - calls compAndDefine, 188
 - calls comp, 188
 - calls getdatabase, 188
 - local def \$clamList, 188
 - local ref \$ConstructorCache, 188
 - local ref \$clamList, 188
 - local ref \$mutableDomain, 188
 - defun, 188
- compiled-function-p
 - calledby subrname, 232
- compileDocumentation, 169
 - calledby compDefineLisplib, 167
 - calls finalizeDocumentation, 169
 - calls lisplibWrite, 169
 - calls makeInputFilename, 169
 - calls rdefiostream, 169
 - calls replaceFile, 169
 - calls rpackfile, 169
 - calls rshut, 169
 - local ref \$EmptyMode, 170
 - local ref \$e, 170
 - local ref \$fcopy, 169
 - local ref \$spadLibFT, 169
 - defun, 169
- compileFileQuietly, 626
 - uses *standard-output*, 626
 - uses \$InteractiveMode, 626
 - defun, 626
- compiler, 560
 - calls compileSpad2Cmd, 561
 - calls compileSpadLispCmd, 561
 - calls findfile, 561
 - calls helpSpad2Cmd[5], 561
 - calls mergePathnames[5], 561
 - calls namestring[5], 561
 - calls pathnameType[5], 561
 - calls pathname[5], 561
 - calls selectOptionLC[5], 561
 - calls throwKeyedMsg, 561
 - uses /editfile, 561
 - uses \$newConlist, 561
 - uses \$options, 561
 - defun, 560
- compilerDoit, 563, 568
 - calledby compileSpad2Cmd, 563
 - calledby compilerDoitWithScreenedLisplib, 567
 - calls /RQ,LIB, 568
 - calls /rf[5], 568
 - calls /rq[5], 568
 - calls member[5], 568
 - calls opOf, 568
 - calls sayBrightly, 568
 - uses \$byConstructors, 568
 - uses \$constructorsSeen, 568
 - defun, 568
- compilerDoitWithScreenedLisplib, 563
 - calledby compileSpad2Cmd, 563
 - calls compilerDoit, 567
 - calls embed, 567
 - calls rwrite, 567
 - calls unembed, 567
 - local ref \$libFile, 567
 - local ref \$saveableItems, 567
- compilerMessage
 - calledby augModemapsFromCategoryRep, 271
 - calledby augModemapsFromCategory, 265

- compileSpad2Cmd, 562
 - calledby compiler, 561
 - calls browserAutoloadOnceTrigger, 563
 - calls compilerDoitWithScreenedLisplib, 563
 - calls compilerDoit, 563
 - calls error, 563
 - calls extendLocalLibdb, 563
 - calls namestring[5], 563
 - calls object2String, 563
 - calls pathnameType[5], 563
 - calls pathname[5], 563
 - calls sayKeyedMsg[5], 563
 - calls selectOptionLC[5], 563
 - calls spad2AsTranslatorAutoloadOnceTrigger, 563
 - calls spadPrompt, 563
 - calls strconc, 563
 - calls terminateSystemCommand[5], 563
 - calls throwKeyedMsg, 563
 - calls updateSourceFiles[5], 563
 - uses /editfile, 563
 - uses \$InteractiveMode, 563
 - uses \$QuickCode, 563
 - uses \$QuickLet, 563
 - uses \$compileOnlyCertainItems, 563
 - uses \$f, 563
 - uses \$m, 563
 - uses \$newComp, 563
 - uses \$newConlist, 563
 - uses \$options, 563
 - uses \$scanIfTrue, 563
 - uses \$sourceFileTypes, 563
 - defun, 562
- compileSpadLispCmd, 565
 - calledby compiler, 561
 - calls fNameMake[5], 566
 - calls fNameReadable?[5], 566
 - calls localdatabase[5], 566
 - calls namestring[5], 566
 - calls object2String, 566
 - calls pathnameDirectory[5], 566
 - calls pathnameName[5], 566
 - calls pathnameType[5], 565
 - calls pathname[5], 565
 - calls recompile-lib-file-if-necessary, 566
 - calls sayKeyedMsg[5], 566
 - calls selectOptionLC[5], 565
 - calls spadPrompt, 566
 - calls terminateSystemCommand[5], 566
 - calls throwKeyedMsg, 566
 - uses \$options, 566
 - defun, 565
- compileTimeBindingOf, 237
 - calledby optSpecialCall, 236
 - calls bpiname, 237
 - calls keyedSystemError, 237
 - calls moan, 237
 - defun, 237
- compImport, 322
 - calls addDomain, 322
 - uses \$NoValueMode, 322
 - defun, 322
- compInternalFunction, 159
 - calledby compDefine1, 145
 - calls identp, 159
 - calls stackAndThrow, 159
 - defun, 159
- compls, 322
 - calls coerce, 322
 - calls comp, 322
 - uses \$Boolean, 323
 - uses \$EmptyMode, 323
 - defun, 322
- complterator
 - calledby compReduce1, 330
 - calledby compRepeatOrCollect, 333
- compJoin, 323
 - calls compForMode, 323
 - calls compJoin.getParms, 323
 - calls convert, 324
 - calls isCategoryForm, 323
 - calls nreverse0, 323
 - calls stackSemanticError, 323
 - calls union, 323
 - calls wrapDomainSub, 323
 - uses \$Category, 324
 - defun, 323
- compJoin.getParms
 - calledby compJoin, 323
- compLambda, 325
 - calledby compWithMappingMode1, 615
 - calls argsToSig, 325

- calls compAtSign, [325](#)
 - calls stackAndThrow, [325](#)
 - defun, [325](#)
- compLeave, [327](#)
 - calls comp, [327](#)
 - calls modifyModeStack, [327](#)
 - uses \$exitModeStack, [327](#)
 - uses \$leaveLevelStack, [327](#)
 - defun, [327](#)
- compList, [599](#)
 - calledby compAtom, [595](#)
 - calledby compConstruct, [299](#)
 - calls comp, [599](#)
 - defun, [599](#)
- compMacro, [327](#)
 - calls formatUnabbreviated, [327](#)
 - calls macroExpand, [327](#)
 - calls put, [327](#)
 - calls sayBrightly, [327](#)
 - uses \$EmptyMode, [327](#)
 - uses \$NoValueMode, [327](#)
 - uses \$macroIfTrue, [327](#)
 - defun, [327](#)
- compMakeCategoryObject, [213](#)
 - calledby compDefineFunctor1, [148](#)
 - calledby getOperationAlist, [269](#)
 - calledby getSlotFromFunctor, [212](#)
 - calls isCategoryForm, [213](#)
 - calls mkEvaluableCategoryForm, [213](#)
 - local ref \$Category, [213](#)
 - local ref \$e, [213](#)
 - defun, [213](#)
- compMakeDeclaration, [622](#)
 - calledby compDefineCategory2, [163](#)
 - calledby compDefineFunctor1, [148](#)
 - calledby compSetq1, [340](#)
 - calledby compSubDomain1, [350](#)
 - calledby compWithMappingMode1, [615](#)
 - calls compColon, [622](#)
 - uses \$insideExpressionIfTrue, [622](#)
 - defun, [622](#)
- compMapCond, [260](#)
 - calledby compApplyModemap, [259](#)
 - calls compMapCond', [260](#)
 - local ref \$bindings, [260](#)
 - defun, [260](#)
- compMapCond', [261](#)
 - calledby compMapCond, [260](#)
 - calls compMapCond", [261](#)
 - calls compMapConfFun, [261](#)
 - calls stackMessage, [261](#)
 - defun, [261](#)
- compMapCond", [261](#)
 - calledby compMapCond", [261](#)
 - calledby compMapCond', [261](#)
 - calls compMapCond", [261](#)
 - calls get, [261](#)
 - calls knownInfo, [261](#)
 - calls stackMessage, [261](#)
 - local ref \$Information, [261](#)
 - local ref \$e, [261](#)
 - defun, [261](#)
- compMapCondFun, [262](#)
 - defun, [262](#)
- compMapConfFun
 - calledby compMapCond', [261](#)
- compNoStacking, [588](#)
 - calledby compToApply, [602](#)
 - calledby comp, [587](#)
 - calls comp2, [588](#)
 - calls compNoStacking1, [588](#)
 - local ref \$compStack, [588](#)
 - uses \$EmptyMode, [588](#)
 - uses \$Representation, [588](#)
 - defun, [588](#)
- compNoStacking1, [588](#)
 - calledby compNoStacking, [588](#)
 - calls comp2, [588](#)
 - calls get, [588](#)
 - local ref \$compStack, [588](#)
 - defun, [588](#)
- compOrCroak, [586](#)
 - calledby NRTgetLocalIndex, [215](#)
 - calledby compAdd, [276](#)
 - calledby compArgumentConditions, [170](#)
 - calledby compDefineCapsuleFunction, [156](#)
 - calledby compDefineCategory2, [164](#)
 - calledby compForm1, [600](#)
 - calledby compFunctorBody, [172](#)
 - calledby compRepeatOrCollect, [333](#)
 - calledby compSubDomain1, [350](#)
 - calledby compTopLevel, [583](#)

- calledby doIt, 281
- calledby getTargetFromRhs, 177
- calledby makeCategoryForm, 297
- calledby mkEvalableCategoryForm, 182
- calledby substituteIntoFunctorModemap, 612
- calls compOrCroak1, 586
- defun, 586
- compOrCroak1, 586
 - calledby compOrCroak, 586
 - calls compOrCroak1, compactify, 586
 - calls comp, 586
 - calls displayComp, 587
 - calls displaySemanticErrors, 586
 - calls mkErrorExpr, 586
 - calls say, 586
 - calls stackSemanticError, 586
 - calls userError, 587
 - local def \$compStack, 587
 - uses \$compErrorMessageStack, 587
 - uses \$exitModeStack, 587
 - uses \$level, 587
 - uses \$scanIfTrue, 587
 - uses \$s, 587
 - catches, 587
 - defun, 586
- compOrCroak1, compactify, 624
 - calledby compOrCroak1, compactify, 624
 - calledby compOrCroak1, 586
 - calls compOrCroak1, compactify, 624
 - calls lassoc, 624
 - defun, 624
- compPretend, 328
 - calls addDomain, 328
 - calls comp, 329
 - calls opOf, 329
 - calls stackSemanticError, 329
 - calls stackWarning, 329
 - uses \$EmptyMode, 329
 - uses \$newCompilerUnionFlag, 329
 - defun, 328
- compQuote, 330
 - defun, 330
- compReduce, 330
 - calls compReduce1, 330
 - uses \$formalArgList, 330
- defun, 330
- compReduce1, 330
 - calledby compReduce, 330
 - calls compIterator, 330
 - calls comp, 330
 - calls getIdentity, 331
 - calls nreverse0, 330
 - calls parseTran, 330
 - calls systemError, 330
 - uses \$Boolean, 331
 - uses \$endTestList, 331
 - uses \$e, 331
 - uses \$initList, 331
 - uses \$sideEffectsList, 331
 - uses \$until, 331
 - defun, 330
- compRepeatOrCollect, 333
 - calls coerceExit, 333
 - calls compIterator, 333
 - calls compOrCroak, 333
 - calls comp, 333
 - calls length, 333
 - calls modelsAggregateOf, 333
 - calls stackMessage, 333
 - calls , 333
 - uses \$Boolean, 333
 - uses \$NoValueMode, 333
 - uses \$exitModeStack, 333
 - uses \$formalArgList, 333
 - uses \$leaveLevelStack, 333
 - uses \$until, 333
 - defun, 333
- compReturn, 335
 - calls comp, 335
 - calls modifyModeStack, 335
 - calls resolve, 335
 - calls stackSemanticError, 335
 - calls userError, 335
 - uses \$exitModeStack, 335
 - uses \$returnMode, 335
 - defun, 335
- compSeq, 336
 - calls compSeq1, 336
 - uses \$exitModeStack, 336
 - defun, 336
- compSeq1, 336

- calledby compSeq, 336
- calls compSeqItem, 336
- calls mkq, 336
- calls nreverse0, 336
- calls replaceExitEtc, 336
- uses \$NoValueMode, 337
- uses \$exitModeStack, 336
- uses \$finalEnv, 337
- uses \$insideExpressionIfTrue, 336
- defun, 336
- compSeqItem, 338
 - calledby compSeq1, 336
 - calls comp, 338
 - calls macroExpand, 338
 - defun, 338
- compSetq, 339
 - calledby compSetq1, 340
 - calls compSetq1, 339
 - defun, 339
- compSetq1, 339
 - calledby compSetq, 339
 - calledby setqMultipleExplicit, 343
 - calledby setqMultiple, 341
 - calls compMakeDeclaration, 340
 - calls compSetq, 340
 - calls identp[5], 340
 - calls setqMultiple, 340
 - calls setqSetelt, 340
 - calls setqSingle, 339
 - uses \$EmptyMode, 340
 - defun, 339
- compSingleCapsuleItem, 280
 - calledby compCapsuleItems, 280
 - calledby doItIf, 285
 - calledby doIt, 281
 - calls doit, 280
 - calls macroExpandInPlace, 280
 - local ref \$e, 280
 - local ref \$pred, 280
 - defun, 280
- compString, 349
 - calls resolve, 349
 - uses \$StringCategory, 349
 - defun, 349
- compSubDomain, 350
 - calls compCapsule, 350
 - calls compSubDomain1, 350
 - uses \$NRTaddForm, 350
 - uses \$addFormLhs, 350
 - uses \$addForm, 350
 - defun, 350
- compSubDomain1, 350
 - calledby compAdd, 275
 - calledby compSubDomain, 350
 - calls addDomain, 350
 - calls compMakeDeclaration, 350
 - calls compOrCroak, 350
 - calls evalAndRwriteLispForm, 351
 - calls lispize, 351
 - calls stackSemanticError, 350
 - uses \$Boolean, 351
 - uses \$CategoryFrame, 351
 - uses \$EmptyMode, 351
 - uses \$lisplibSuperDomain, 351
 - uses \$op, 351
 - defun, 350
- compSubsetCategory, 352
 - calls comp, 352
 - calls put, 352
 - uses \$lhsOfColon, 352
 - defun, 352
- compSuchthat, 353
 - calls comp, 353
 - calls put, 353
 - uses \$Boolean, 353
 - defun, 353
- compSymbol, 598
 - calledby compAtom, 595
 - calls NRTgetLocalIndex, 598
 - calls errorRef, 598
 - calls getmode, 598
 - calls get, 598
 - calls isFluid, 598
 - calls isFunction, 598
 - calls member[5], 598
 - calls stackMessage, 598
 - uses \$Boolean, 598
 - uses \$Expression, 598
 - uses \$FormalMapVariableList, 598
 - uses \$NoValueMode, 598
 - uses \$NoValue, 598
 - uses \$Symbol, 598

- uses \$compForModelIfTrue, 598
 - uses \$formalArgList, 598
 - uses \$functorLocalParameters, 598
- defun, 598
- compToApply, 602
 - calledby compForm1, 600
 - calls compApplication, 602
 - calls compNoStacking, 602
 - local ref \$EmptyMode, 602
- defun, 602
- compTopLevel, 583
 - calledby s-process, 581
 - calls compOrCroak, 583
 - uses \$NRTderivedTargetIfTrue, 583
 - uses \$compTimeSum, 583
 - uses \$envHashTable, 583
 - uses \$forceAdd, 583
 - uses \$skillOptimizeIfTrue, 583
 - uses \$packagesUsed, 583
 - uses \$resolveTimeSum, 583
- defun, 583
- compTuple2Record, 277
 - calledby compAdd, 276
- defun, 277
- compTypeOf, 593
 - calledby comp3, 590
 - calls comp3, 593
 - calls eqsubstlist, 593
 - calls get, 593
 - calls put, 593
 - uses \$FormalMapVariableList, 593
 - uses \$insideCompTypeOf, 593
- defun, 593
- compUniquely, 613
 - calledby compForm2, 608
 - calls comp, 613
 - local def \$compUniquelyIfTrue, 613
 - catches, 613
- defun, 613
- computeAncestorsOf
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
- compVector, 353
 - calledby compAtom, 595
 - calledby compConstruct, 299
 - calls comp, 354
 - uses \$EmptyVector, 354
- defun, 353
- compWhere, 354
 - calls addContour, 355
 - calls comp, 354
 - calls deltaContour, 355
 - calls macroExpand, 355
 - uses \$EmptyMode, 355
 - uses \$insideExpressionIfTrue, 355
 - uses \$insideWhereIfTrue, 355
- defun, 354
- compWithMappingMode, 614
 - calledby comp3, 589
 - calls compWithMappingModel1, 614
 - uses \$formalArgList, 614
- defun, 614
- compWithMappingModel1, 615
 - calledby compWithMappingMode, 614
 - calls comp-tran, 615
 - calls compLambda, 615
 - calls compMakeDeclaration, 615
 - calls comp, 615
 - calls extendsCategoryForm, 615
 - calls extractCodeAndConstructTriple, 615
 - calls freelist, 615
 - calls get, 615
 - calls hasFormalMapVariable, 615
 - calls isFunctor, 615
 - calls optimizeFunctionDef, 615
 - calls stackAndThrow, 615
 - calls take, 615
 - uses \$CategoryFrame, 615
 - uses \$EmptyMode, 615
 - uses \$FormalMapVariableList, 615
 - uses \$QuickCode, 615
 - uses \$formalArgList, 615
 - uses \$formatArgList, 615
 - uses \$funnameTail, 615
 - uses \$funname, 615
 - uses \$skillOptimizeIfTrue, 615
- defun, 615
- concat
 - calledby checkDocError, 515
 - calledby checkDocMessage, 518
 - calledby checkWarning, 547
 - calledby compDefWhereClause, 159

- concatWithBlanks
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
- cond, 246
 - defplist, 246
- cons, 298
 - defplist, 298
- consProplistOf
 - calledby getSuccessEnvironment, 319
 - calledby setqSingle, 344
- construct, 103, 299, 387
 - defplist, 103, 299, 387
- constructMacro, 186
 - calledby compile, 173
 - calls identp, 186
 - calls stackSemanticError, 186
 - defun, 186
- constructor?
 - calledby addDomain, 252
 - calledby checkIsValidType, 530
 - calledby checkNumOfArgs, 532
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby getAbbreviation, 302
 - calledby isFunctor, 253
- contained
 - calledby evalAndSub, 269
 - calledby parseCategory, 107
 - calledby spadCompileOrSetq, 186
 - calledby substVars, 202
- containsBang, 408
 - calledby aplTran, 405
 - calledby containsBang, 408
 - calls containsBang, 408
 - defun, 408
- convert, 597
 - calledby applyMapping, 591
 - calledby compAtomWithModemap, 596
 - calledby compAtom, 595
 - calledby compCons1, 298
 - calledby compConstruct, 299
 - calledby compElt, 310
 - calledby compExpressionList, 607
 - calledby compFocompFormWithModemap, 610
 - calledby compJoin, 324
 - calledby convertOrCroak, 338
 - calledby setqMultiple, 341
 - calledby setqSingle, 344
 - calls coerce, 597
 - calls resolve, 597
 - defun, 597
- convertOpAlist2compilerInfo, 120
 - calledby updateCategoryFrameForConstructor, 120
 - defun, 120
- convertOrCroak, 338
 - calledby replaceExitEtc, 337
 - calls convert, 338
 - calls userError, 338
 - defun, 338
- copy
 - calledby modifyModeStack, 622
- copy-token
 - calledby PARSE-TokTail, 427
 - calledby advance-token, 459
- croak
 - calledby drop, 551
- curoutstream
 - usedby s-process, 582
 - usedby spad, 572
- current-char, 460
 - calledby Advance-Char, 635
 - calledby PARSE-FloatBasePart, 434
 - calledby PARSE-FloatBase, 433
 - calledby PARSE-FloatExponent, 434
 - calledby PARSE-Selector, 430
 - calledby PARSE-TokTail, 427
 - calledby match-string, 451
 - calledby skip-blanks, 451
 - uses \$line, 460
 - uses current-line, 460
 - defun, 460
- current-fragment, 629
 - defvar, 629
- current-line, 632
 - usedby PARSE-Category, 421
 - usedby current-char, 460
 - usedby next-char, 461
 - defvar, 632
- current-symbol, 457

- calledby PARSE-AnyId, [441](#)
- calledby PARSE-ElseClause, [449](#)
- calledby PARSE-FloatBase, [433](#)
- calledby PARSE-FloatExponent, [434](#)
- calledby PARSE-Infix, [426](#)
- calledby PARSE-NewExpr, [415](#)
- calledby PARSE-OpenBrace, [443](#)
- calledby PARSE-OpenBracket, [443](#)
- calledby PARSE-Operation, [424](#)
- calledby PARSE-Prefix, [426](#)
- calledby PARSE-Primary1, [432](#)
- calledby PARSE-ReductionOp, [428](#)
- calledby PARSE-Selector, [430](#)
- calledby PARSE-SpecialCommand, [416](#)
- calledby PARSE-SpecialKeyWord, [415](#)
- calledby PARSE-Suffix, [445](#)
- calledby PARSE-TokTail, [427](#)
- calledby PARSE-TokenList, [418](#)
- calledby isTokenDelimiter, [455](#)
- calls current-token, [457](#)
- calls make-symbol-of, [457](#)
- defun, [457](#)
- current-token, [98](#), [458](#)
 - calledby PARSE-FloatBasePart, [434](#)
 - calledby PARSE-SpecialKeyWord, [415](#)
 - calledby advance-token, [459](#)
 - calledby current-symbol, [457](#)
 - calledby match-advance-string, [452](#)
 - calledby match-current-token, [456](#)
 - calledby next-token, [459](#)
 - calls try-get-token, [458](#)
 - usedby advance-token, [459](#)
 - usedby current-token, [458](#)
 - uses \$token, [98](#)
 - uses current-token, [458](#)
 - uses valid-tokens, [458](#)
 - defun, [458](#)
 - defvar, [98](#)
- curstrm
 - calledby s-process, [581](#)
- dbKind
 - calledby screenLocalLine, [484](#)
- dbMkForm
 - calledby buildLibdbConEntry, [480](#)
- dbName
 - calledby screenLocalLine, [484](#)
- dbPart
 - calledby screenLocalLine, [484](#)
- dbReadLines, [479](#)
 - calledby extendLocalLibdb, [475](#)
 - calls eofp, [479](#)
 - calls readline, [479](#)
 - defun, [479](#)
- dbWriteLines, [479](#)
 - calledby extendLocalLibdb, [475](#)
 - calls getTempPath, [479](#)
 - calls ifcar, [479](#)
 - calls make-outstream, [479](#)
 - calls shut, [479](#)
 - calls writedb, [479](#)
 - local def \$outStream, [479](#)
 - local ref \$outStream, [479](#)
 - defun, [479](#)
- dcq
 - calledby new2OldTran, [78](#)
- decodeScripts, [409](#)
 - calledby decodeScripts, [409](#)
 - calledby getScriptName, [409](#)
 - calls decodeScripts, [409](#)
 - calls strconc, [409](#)
 - defun, [409](#)
- deepestExpression, [408](#)
 - calledby deepestExpression, [408](#)
 - calledby hasAplExtension, [407](#)
 - calls deepestExpression, [408](#)
 - defun, [408](#)
- def, [109](#), [144](#)
 - defplist, [109](#), [144](#)
- def-process
 - calledby s-process, [581](#)
- def-rename, [585](#)
 - calledby def-rename, [585](#)
 - calledby s-process, [581](#)
 - calledby string2BootTree, [77](#)
 - calls def-rename, [585](#)
 - defun, [585](#)
- definition-name, [414](#)
 - usedby PARSE-NewExpr, [415](#)
 - defvar, [414](#)
- defmacro
 - Bang, [463](#)

- line-clear, [632](#)
- must, [463](#)
- nth-stack, [549](#)
- pop-stack-1, [548](#)
- pop-stack-2, [548](#)
- pop-stack-3, [549](#)
- pop-stack-4, [549](#)
- reduce-stack-clear, [465](#)
- stack-/empty, [96](#)
- star, [464](#)
- defplist, [105](#), [244](#), [275](#), [352](#), [361](#), [378](#)
- $+->$, [325](#)
- $->$, [394](#)
- $<=$, [129](#)
- $==>$, [395](#)
- $=>$, [390](#)
- $>$, [115](#)
- $>=$, [114](#), [115](#)
- $.,$ [386](#)
- $-$, [241](#)
- $/$, [401](#)
- $;$, [108](#), [294](#), [384](#)
- $::$, [107](#), [362](#), [385](#)
- $:BF;$, [379](#)
- $;$, [399](#)
- $==$, [388](#)
- add, [376](#)
- and, [105](#)
- Block, [380](#)
- call, [233](#)
- capsule, [277](#)
- case, [287](#)
- catch, [244](#)
- category, [106](#), [290](#), [381](#)
- collect, [332](#), [383](#)
- cond, [246](#)
- cons, [298](#)
- construct, [103](#), [299](#), [387](#)
- def, [109](#), [144](#)
- dollargreaterequal, [112](#)
- dollargreaterthan, [111](#)
- dollarnotequal, [113](#)
- elt, [310](#)
- eq, [240](#)
- eqv, [114](#)
- exit, [311](#)
- has, [116](#), [312](#)
- if, [121](#), [314](#), [391](#)
- implies, [124](#)
- import, [322](#)
- In, [393](#)
- in, [125](#), [392](#)
- inby, [126](#)
- is, [127](#), [322](#)
- isnt, [127](#)
- Join, [128](#), [323](#), [393](#)
- leave, [128](#), [326](#)
- lessp, [242](#)
- let, [130](#), [339](#)
- letd, [130](#)
- ListCategory, [300](#)
- Mapping, [289](#)
- mdef, [131](#), [327](#)
- minus, [240](#)
- mkRecord, [249](#)
- not, [132](#)
- notequal, [133](#)
- or, [133](#)
- pretend, [134](#), [328](#), [396](#)
- qsminus, [241](#)
- quote, [329](#), [396](#)
- Record, [288](#)
- RecordCategory, [300](#)
- recordcopy, [251](#)
- recordelt, [249](#)
- reduce, [330](#), [397](#)
- repeat, [332](#), [398](#)
- return, [135](#), [335](#)
- Scripts, [398](#)
- segment, [135](#), [136](#)
- seq, [238](#), [336](#)
- setq, [339](#)
- setrecordelt, [250](#)
- Signature, [400](#)
- spadcall, [243](#)
- String, [349](#)
- SubDomain, [350](#)
- SubsetCategory, [352](#)
- TupleCollect, [402](#)
- Union, [289](#)
- UnionCategory, [301](#)
- vcons, [136](#)

- vector, 353
- VectorCategory, 301
- where, 137, 354, 403
- with, 403
- defstruct
 - line, 631
 - reduction, 99
 - stack, 95
 - token, 97
- defun
 - /RQ,LIB, 569
 - /rf, 569
 - /rf-1, 570
 - /rq, 569
 - action, 464
 - addArgumentConditions, 304
 - addclose, 550
 - addConstructorModemaps, 258
 - addDomain, 252
 - addEltModemap, 265
 - addEmptyCapsuleIfNecessary, 177
 - addModemap, 273
 - addModemap0, 274
 - addModemap1, 274
 - addModemapKnown, 272
 - addNewDomain, 256
 - addSuffix, 303
 - Advance-Char, 635
 - advance-token, 459
 - alistSize, 303
 - allLASSOCs, 218
 - aplTran, 405
 - aplTran1, 405
 - aplTranList, 407
 - applyMapping, 591
 - argsToSig, 621
 - assignError, 346
 - AssocBarGensym, 226
 - augLisplibModemapsFromCategory, 191
 - augmentLisplibModemapsFromFunctor, 216
 - augModemapsFromCategory, 265
 - augModemapsFromCategoryRep, 271
 - augModemapsFromDomain, 256
 - augModemapsFromDomain1, 257
 - autoCoerceByModemap, 364
 - blankp, 550
 - bootStrapError, 219
 - buildLibAttr, 483
 - buildLibAttrs, 483
 - buildLibdb, 476
 - buildLibdbConEntry, 480
 - buildLibdbString, 478
 - buildLibOp, 482
 - buildLibOps, 482
 - bumperrorcount, 543
 - canReturn, 316
 - char-eq, 461
 - char-ne, 461
 - checkAddBackSlashes, 526
 - checkAddIndented, 518
 - checkAddMacros, 527
 - checkAddPeriod, 527
 - checkAddSpaces, 528
 - checkAddSpaceSegments, 528
 - checkAlphabetic, 529
 - checkAndDeclare, 308
 - checkArguments, 499
 - checkBalance, 499
 - checkBeginEnd, 500
 - checkComments, 496
 - checkDecorate, 502
 - checkDecorateForHt, 504
 - checkDocError, 515
 - checkDocError1, 505
 - checkDocMessage, 518
 - checkExtract, 519
 - checkFixCommonProblem, 505
 - checkGetArgs, 520
 - checkGetLispFunctionName, 506
 - checkGetMargin, 520
 - checkGetParse, 521
 - checkGetStringBeforeRightBrace, 521
 - checkHTargs, 506
 - checkIeEg, 522
 - checkIeEgfun, 529
 - checkIndentedLines, 522
 - checkIsValidType, 530
 - checkLookForLeftBrace, 531
 - checkLookForRightBrace, 531
 - checkNumOfArgs, 532
 - checkRecordHash, 507
 - checkRemoveComments, 516

- checkRewrite, [497](#)
- checkSayBracket, [532](#)
- checkSkipBlanks, [533](#)
- checkSkipIdentifierToken, [523](#)
- checkSkipOpToken, [523](#)
- checkSkipToken, [517](#)
- checkSplit2Words, [517](#)
- checkSplitBackslash, [533](#)
- checkSplitBrace, [524](#)
- checkSplitOn, [534](#)
- checkSplitPunctuation, [535](#)
- checkTexht, [511](#)
- checkTransformFirsts, [512](#)
- checkTrim, [515](#)
- checkTrimCommented, [525](#)
- checkWarning, [547](#)
- coerce, [355](#)
- coerceable, [360](#)
- coerceByModemap, [363](#)
- coerceEasy, [356](#)
- coerceExit, [361](#)
- coerceExtraHard, [359](#)
- coerceHard, [358](#)
- coerceSubset, [357](#)
- collectAndDeleteAssoc, [470](#)
- collectComBlock, [469](#)
- comma2Tuple, [386](#)
- comp, [587](#)
- comp2, [589](#)
- comp3, [589](#)
- compAdd, [275](#)
- compAndDefine, [189](#)
- compApplication, [603](#)
- compApply, [592](#)
- compApplyModemap, [259](#)
- compArgumentConditions, [170](#)
- compArgumentsAndTryAgain, [614](#)
- compAtom, [595](#)
- compAtomWithModemap, [596](#)
- compAtSign, [361](#)
- compBoolean, [318](#)
- compCapsule, [278](#)
- compCapsuleInner, [278](#)
- compCapsuleItems, [279](#)
- compCase, [287](#)
- compCase1, [288](#)
- compCat, [289](#)
- compCategory, [290](#)
- compCategoryItem, [291](#)
- compCoerce, [362](#)
- compCoerce1, [363](#)
- compColon, [294](#)
- compColonInside, [594](#)
- compCons, [298](#)
- compCons1, [298](#)
- compConstruct, [299](#)
- compConstructorCategory, [301](#)
- compDefine, [144](#)
- compDefine1, [145](#)
- compDefineAddSignature, [147](#)
- compDefineCapsuleFunction, [155](#)
- compDefineCategory, [162](#)
- compDefineCategory1, [162](#)
- compDefineCategory2, [163](#)
- compDefineFunctor, [148](#)
- compDefineFunctor1, [148](#)
- compDefineLisplib, [167](#)
- compDefWhereClause, [159](#)
- compElt, [310](#)
- compExit, [312](#)
- compExpression, [139](#)
- compExpressionList, [607](#)
- compFocompFormWithModemap, [610](#)
- compForm, [600](#)
- compForm1, [600](#)
- compForm2, [607](#)
- compForm3, [609](#)
- compFormMatch, [613](#)
- compForMode, [325](#)
- compFormPartiallyBottomUp, [613](#)
- compFromIf, [316](#)
- compFunctorBody, [172](#)
- compHas, [313](#)
- compHasFormat, [313](#)
- compIf, [315](#)
- compile, [173](#)
- compile-lib-file, [625](#)
- compileCases, [171](#)
- compileConstructor, [187](#)
- compileConstructor1, [188](#)
- compileDocumentation, [169](#)
- compileFileQuietly, [626](#)

- compiler, 560
- compilerDoit, 568
- compileSpad2Cmd, 562
- compileSpadLispCmd, 565
- compileTimeBindingOf, 237
- compImport, 322
- compInternalFunction, 159
- compIs, 322
- compJoin, 323
- compLambda, 325
- compLeave, 327
- compList, 599
- compMacro, 327
- compMakeCategoryObject, 213
- compMakeDeclaration, 622
- compMapCond, 260
- compMapCond', 261
- compMapCond", 261
- compMapCondFun, 262
- compNoStacking, 588
- compNoStacking1, 588
- compOrCroak, 586
- compOrCroak1, 586
- compOrCroak1,compactify, 624
- compPretend, 328
- compQuote, 330
- compReduce, 330
- compReduce1, 330
- compRepeatOrCollect, 333
- compReturn, 335
- compSeq, 336
- compSeq1, 336
- compSeqItem, 338
- compSetq, 339
- compSetq1, 339
- compSingleCapsuleItem, 280
- compString, 349
- compSubDomain, 350
- compSubDomain1, 350
- compSubsetCategory, 352
- compSuchthat, 353
- compSymbol, 598
- compToApply, 602
- compTopLevel, 583
- compTuple2Record, 277
- compTypeOf, 593
- compUniquely, 613
- compVector, 353
- compWhere, 354
- compWithMappingMode, 614
- compWithMappingMode1, 615
- constructMacro, 186
- containsBang, 408
- convert, 597
- convertOpAlist2compilerInfo, 120
- convertOrCroak, 338
- current-char, 460
- current-symbol, 457
- current-token, 458
- dbReadLines, 479
- dbWriteLines, 479
- decodeScripts, 409
- deepestExpression, 408
- def-rename, 585
- disallowNilAttribute, 219
- displayMissingFunctions, 220
- displayPreCompilationErrors, 542
- doIt, 281
- doItIf, 285
- dollarTran, 462
- domainMember, 264
- drop, 551
- eltModemapFilter, 605
- encodeFunctionName, 183
- encodeItem, 185
- EqualBarGensym, 248
- escape-keywords, 454
- escaped, 551
- evalAndRwriteLispForm, 204
- evalAndSub, 269
- expand-tabs, 94
- extendLocalLibdb, 475
- extractCodeAndConstructTriple, 620
- finalizeDocumentation, 490
- finalizeLisplib, 207
- fincomblock, 551
- firstNonBlankPosition, 536
- fixUpPredicate, 196
- flattenSignatureList, 194
- floatexpid, 462
- formal2Pattern, 218
- freelist, 623

- genDomainOps, 224
- genDomainView, 224
- genDomainViewList, 223
- genDomainViewList0, 223
- get-a-line, 636
- get-token, 460
- getAbbreviation, 302
- getArgumentMode, 309
- getArgumentModeOrMoan, 191
- getCaps, 185
- getCategoryOpsAndAtts, 210
- getConstructorExports, 473
- getConstructorOpsAndAtts, 209
- getDomainsInScope, 254
- getFormModemaps, 604
- getFunctorOpsAndAtts, 212
- getInverseEnvironment, 320
- getMatchingRightPren, 536
- getModemap, 258
- getModemapList, 263
- getModemapListFromDomain, 264
- getOperationAlist, 269
- getScriptName, 408
- getSignature, 306
- getSignatureFromMode, 303
- getSlotFromCategoryForm, 210
- getSlotFromFunctor, 212
- getSpecialCaseAssoc, 304
- getSuccessEnvironment, 318
- getTargetFromRhs, 177
- getToken, 455
- getUnionMode, 321
- getUniqueModemap, 263
- getUniqueSignature, 263
- giveFormalParametersValues, 178
- hackforis, 412
- hackforisl, 413
- hasAplExtension, 407
- hasFormalMapVariable, 621
- hasFullSignature, 176
- hasNoVowels, 537
- hasSigInTargetCategory, 308
- hasType, 360
- htcharPosition, 537
- indent-pos, 552
- infixtok, 553
- initial-substring, 635
- initial-substring-p, 453
- initialize-prepare, 82
- initializeLisplib, 206
- insertModemap, 267
- interactiveModemapForm, 195
- is-console, 553
- isCategoryPackageName, 214
- isDomainConstructorForm, 348
- isDomainForm, 348
- isDomainSubst, 200
- isFunctor, 253
- isListConstructor, 111
- isMacro, 286
- isSuperDomain, 255
- isTokenDelimiter, 455
- isUnionMode, 321
- killColons, 401
- line-advance-char, 633
- line-at-end-p, 632
- line-current-segment, 634
- line-new-line, 634
- line-next-char, 633
- line-past-end-p, 633
- line-print, 632
- lispize, 351
- lisplibDoRename, 205
- lisplibWrite, 214
- loadIfNecessary, 118
- loadLibIfNecessary, 119
- macroExpand, 178
- macroExpandInPlace, 178
- macroExpandList, 179
- make-symbol-of, 458
- makeCategoryForm, 297
- makeCategoryPredicates, 179
- makeFunctorArgumentParameters, 221
- makeSimplePredicateOrNil, 544
- match-advance-string, 452
- match-current-token, 456
- match-next-token, 457
- match-string, 451
- match-token, 457
- maxSuperType, 348
- mergeModemap, 267
- mergeSignatureAndLocalVarAlists, 213

- meta-syntax-error, 462
- mkAbbrev, 302
- mkAListOfExplicitCategoryOps, 193
- mkCategoryPackage, 180
- mkConstructor, 205
- mkDatabasePred, 218
- mkEvalableCategoryForm, 182
- mkExplicitCategoryFunction, 292
- mkList, 314
- mkNewModemapList, 266
- mkOpVec, 225
- mkRepetitionAssoc, 184
- mkUnion, 366
- modeEqual, 366
- modeEqualSubst, 367
- modemapPattern, 203
- modifyModeStack, 622
- moveORsOutside, 201
- mustInstantiate, 293
- ncINTERPFILE, 624
- new2OldDefTran, 80
- new2OldLisp, 78
- new2OldTran, 78
- newConstruct, 80
- newDef2Def, 80
- newIf2Cond, 79
- newString2Words, 525
- newWordFrom, 538
- next-char, 460
- next-line, 634
- next-tab-loc, 553
- next-token, 459
- nonblankloc, 553
- NRTassocIndex, 346
- NRTgetLocalIndex, 215
- NRTgetLookupFunction, 214
- NRTputInHead, 190
- NRTputInTail, 189
- opt-, 242
- optCall, 233
- optCallEval, 237
- optCallSpecially, 235
- optCatch, 245
- optCond, 246
- optCONDtail, 231
- optEQ, 240
- optIF2COND, 232
- optimize, 229
- optimizeFunctionDef, 228
- optional, 464
- optLESSP, 242
- optMINUS, 240
- optMkRecord, 249
- optPackageCall, 235
- optPredicateIfTrue, 231
- optQSMINUS, 241
- optRECORDCOPY, 251
- optRECORDELT, 250
- optSEQ, 238
- optSETRECORDELT, 251
- optSPADCALL, 243
- optSpecialCall, 236
- optSuchthat, 244
- optXLAMCond, 230
- orderByDependency, 227
- orderPredicateItems, 197
- orderPredTran, 198
- outputComp, 347
- PARSE-AnyId, 441
- PARSE-Application, 429
- parse-argument-designator, 546
- PARSE-Category, 421
- PARSE-Command, 415
- PARSE-CommandTail, 418
- PARSE-Conditional, 448
- PARSE-Data, 439
- PARSE-ElseClause, 449
- PARSE-Enclosure, 435
- PARSE-Exit, 447
- PARSE-Expr, 423
- PARSE-Expression, 422
- PARSE-Float, 432
- PARSE-FloatBase, 433
- PARSE-FloatBasePart, 434
- PARSE-FloatExponent, 434
- PARSE-FloatTok, 450
- PARSE-Form, 429
- PARSE-FormalParameter, 436
- PARSE-FormalParameterTok, 436
- PARSE-getSemanticForm, 425
- PARSE-GlyphTok, 441
- parse-identifier, 545

- PARSE-Import, [422](#)
- PARSE-Infix, [426](#)
- PARSE-InfixWith, [420](#)
- PARSE-IntegerTok, [436](#)
- PARSE-Iterator, [444](#)
- PARSE-IteratorTail, [444](#)
- parse-keyword, [546](#)
- PARSE-Label, [430](#)
- PARSE-LabelExpr, [450](#)
- PARSE-Leave, [447](#)
- PARSE-LedPart, [423](#)
- PARSE-leftBindingPowerOf, [425](#)
- PARSE-Loop, [449](#)
- PARSE-Name, [438](#)
- PARSE-NBGlyphTok, [441](#)
- PARSE-NewExpr, [414](#)
- PARSE-NudPart, [424](#)
- parse-number, [545](#)
- PARSE-OpenBrace, [443](#)
- PARSE-OpenBracket, [443](#)
- PARSE-Operation, [424](#)
- PARSE-Option, [419](#)
- PARSE-Prefix, [426](#)
- PARSE-Primary, [431](#)
- PARSE-Primary1, [431](#)
- PARSE-PrimaryNoFloat, [431](#)
- PARSE-PrimaryOrQM, [419](#)
- PARSE-Quad, [436](#)
- PARSE-Qualification, [427](#)
- PARSE-Reduction, [428](#)
- PARSE-ReductionOp, [428](#)
- PARSE-Return, [446](#)
- PARSE-rightBindingPowerOf, [425](#)
- PARSE-ScriptItem, [438](#)
- PARSE-Scripts, [437](#)
- PARSE-Seg, [448](#)
- PARSE-Selector, [430](#)
- PARSE-SemiColon, [446](#)
- PARSE-Sequence, [442](#)
- PARSE-Sequence1, [442](#)
- PARSE-Sexpr, [439](#)
- PARSE-Sexpr1, [439](#)
- parse-spadstring, [544](#)
- PARSE-SpecialCommand, [416](#)
- PARSE-SpecialKeyWord, [415](#)
- PARSE-Statement, [419](#)
- PARSE-String, [437](#)
- parse-string, [544](#)
- PARSE-Suffix, [445](#)
- PARSE-TokenCommandTail, [417](#)
- PARSE-TokenList, [418](#)
- PARSE-TokenOption, [417](#)
- PARSE-TokTail, [427](#)
- PARSE-VarForm, [437](#)
- PARSE-With, [420](#)
- parseAnd, [105](#)
- parseAtom, [102](#)
- parseAtSign, [106](#)
- parseCategory, [107](#)
- parseCoerce, [108](#)
- parseColon, [108](#)
- parseConstruct, [103](#)
- parseDEF, [109](#)
- parseDollarGreaterEqual, [112](#)
- parseDollarGreaterThen, [112](#)
- parseDollarLessEqual, [113](#)
- parseDollarNotEqual, [113](#)
- parseDropAssertions, [107](#)
- parseEquivalence, [114](#)
- parseExit, [114](#)
- parseGreaterEqual, [115](#)
- parseGreaterThen, [116](#)
- parseHas, [116](#)
- parseHasRhs, [118](#)
- parseIf, [122](#)
- parseIf,ifTran, [122](#)
- parseImplies, [124](#)
- parseIn, [125](#)
- parseInBy, [126](#)
- parseIs, [127](#)
- parseIsnt, [127](#)
- parseJoin, [128](#)
- parseLeave, [129](#)
- parseLessEqual, [129](#)
- parseLET, [130](#)
- parseLETD, [131](#)
- parseLhs, [110](#)
- parseMDEF, [131](#)
- parseNot, [132](#)
- parseNotEqual, [133](#)
- parseOr, [133](#)
- parsePretend, [134](#)

- parseprint, 554
- parseReturn, 135
- parseSegment, 135
- parseSeq, 136
- parseTran, 101
- parseTranCheckForRecord, 543
- parseTranList, 103
- parseTransform, 101
- parseType, 106
- parseVCONS, 137
- parseWhere, 137
- Pop-Reduction, 550
- postAdd, 376
- postAtom, 371
- postAtSign, 378
- postBigFloat, 380
- postBlock, 380
- postBlockItem, 377
- postBlockItemList, 377
- postCapsule, 377
- postCategory, 381
- postcheck, 373
- postCollect, 383
- postCollect,finish, 381
- postColon, 385
- postColonColon, 385
- postComma, 386
- postConstruct, 387
- postDef, 388
- postDefArgs, 390
- postError, 374
- postExit, 391
- postFlatten, 386
- postFlattenLeft, 399
- postForm, 374
- postIf, 391
- postIn, 393
- postin, 392
- postInSeq, 392
- postIteratorList, 384
- postJoin, 394
- postMakeCons, 382
- postMapping, 394
- postMDef, 395
- postOp, 371
- postPretend, 396
- postQUOTE, 397
- postReduce, 397
- postRepeat, 398
- postScripts, 398
- postScriptsForm, 372
- postSemiColon, 399
- postSignature, 400
- postSlash, 401
- postTran, 370
- postTranList, 372
- postTranScripts, 372
- postTranSegment, 388
- postTransform, 369
- postTransformCheck, 373
- postTuple, 402
- postTupleCollect, 402
- postType, 379
- postWhere, 403
- postWith, 404
- preparse, 86
- preparse-echo, 95
- preparse1, 87
- preparseReadLine1, 93
- primitiveType, 597
- print-defun, 584
- print-package, 547
- processFunctor, 279
- purgeNewConstructorLines, 479
- push-reduction, 465
- putDomainsInScope, 255
- putInLocalDomainReferences, 189
- quote-if-string, 453
- read-a-line, 629
- recompile-lib-file-if-necessary, 625
- recordAttributeDocumentation, 468
- recordDocumentation, 469
- recordHeaderDocumentation, 470
- recordSignatureDocumentation, 468
- removeBackslashes, 539
- removeSuperfluousMapping, 400
- replaceExitEtc, 337
- replaceVars, 196
- reportOnFunctorCompilation, 219
- resolve, 365
- rwriteLispForm, 204
- s-process, 574

- screenLocalLine, 484
- setDefOp, 404
- seteltModemapFilter, 606
- setqMultiple, 341
- setqMultipleExplicit, 343
- setqSetelt, 344
- setqSingle, 344
- signatureTran, 197
- skip-blanks, 451
- skip-ifblock, 92
- skip-to-endif, 554
- spad, 571
- spad-fixed-arg, 625
- spadCompileOrSetq, 186
- spadSysBranch, 510
- spadSysChoose, 510
- splitEncodedFunctionName, 184
- stack-clear, 96
- stack-load, 95
- stack-pop, 97
- stack-push, 96
- storeblanks, 635
- string2BootTree, 77
- stripOffArgumentConditions, 306
- stripOffSubdomainConditions, 305
- subname, 232
- substituteCategoryArguments, 257
- substituteIntoFunctorModemap, 612
- substNames, 270
- substVars, 202
- token-install, 98
- token-lookahead-type, 452
- token-print, 99
- transDoc, 494
- transDocList, 493
- transformAndRecheckComments, 495
- transformOperationAlist, 210
- transImplementation, 597
- transIs, 110
- transIs1, 110
- translabel, 541
- translabel1, 541
- TruthP, 268
- try-get-token, 458
- tuple2List, 547
- uncons, 340
- underscore, 455
- unget-tokens, 456
- unknownTypeError, 253
- unloadOneConstructor, 205
- unTuple, 413
- updateCategoryFrameForCategory, 121
- updateCategoryFrameForConstructor, 120
- whoOwns, 539
- wrapDomainSub, 294
- writeLib1, 207
- defvar
 - \$BasicPredicates, 231
 - \$EmptyMode, 176
 - \$FormalMapVariableList, 270
 - \$NoValueMode, 176
 - \$byConstructors, 626
 - \$comblocklist, 551
 - \$constructorsSeen, 626
 - \$defstack, 411
 - \$echolinestack, 81
 - \$index, 81
 - \$is-eqlist, 412
 - \$is-gensymlist, 412
 - \$is-spill-list, 411
 - \$is-spill, 411
 - \$linelist, 81
 - \$newConlist, 555
 - \$preparse-last-line, 82
 - \$vl, 412
 - current-fragment, 629
 - current-line, 632
 - current-token, 98
 - definition-name, 414
 - initial-gensym, 412
 - lablasoc, 414
 - meta-error-handler, 461
 - next-token, 98
 - nonblank, 97
 - ParseMode, 414
 - prior-token, 97
 - reduce-stack, 465
 - tmptok, 413
 - tok, 414
 - valid-tokens, 98
 - XTokenReader, 460
- delete

- calledby compDefWhereClause, 160
- calledby getInverseEnvironment, 320
- calledby orderPredTran, 198
- calledby putDomainsInScope, 255
- deleteFile
 - calledby buildLibdb, 476
- deleteFile[5]
 - called by buildLibdb, 476
 - called by extendLocalLibdb, 475
- deltaContour
 - calledby compWhere, 355
- digitp[5]
 - called by PARSE-FloatBasePart, 434
 - called by PARSE-FloatBase, 433
 - called by floatexpid, 462
- disallowNilAttribute, 219
 - calledby compDefineFunctor1, 148
 - defun, 219
- displayComp
 - calledby compOrCroak1, 587
- displayMissingFunctions, 220
 - calledby reportOnFunctorCompilation, 219
 - calls bright, 220
 - calls formatUnabbreviatedSig, 220
 - calls getmode, 220
 - calls member, 220
 - calls sayBrightly, 220
 - uses \$CheckVectorList, 220
 - uses \$env, 220
 - uses \$formalArgList, 220
 - defun, 220
- displayPreCompilationErrors, 542
 - calledby s-process, 581
 - calls length, 542
 - calls remdup, 542
 - calls sayBrightly, 542
 - calls sayMath, 542
 - local ref \$InteractiveMode, 542
 - local ref \$postStack, 542
 - local ref \$stopOp, 542
 - defun, 542
- displaySemanticErrors
 - calledby compOrCroak1, 586
 - calledby reportOnFunctorCompilation, 219
 - calledby s-process, 581
- displayWarnings
 - calledby reportOnFunctorCompilation, 219
- doIt, 281
 - calledby doIt, 281
 - calls NRTgetLocalIndex, 281
 - calls bright, 281
 - calls cannotDo, 281
 - calls compOrCroak, 281
 - calls compSingleCapsuleItem, 281
 - calls doItIf, 281
 - calls doIt, 281
 - calls formatUnabbreviated, 281
 - calls get, 281
 - calls insert, 281
 - calls isDomainForm, 281
 - calls isMacro, 281
 - calls lastnode, 281
 - calls member, 281
 - calls opOf, 281
 - calls put, 281
 - calls sayBrightly, 281
 - calls stackSemanticError, 281
 - calls stackWarning, 281
 - calls sublis, 281
 - local def \$LocalDomainAlist, 281
 - local def \$Representation, 281
 - local def \$e, 281
 - local def \$functorLocalParameters, 281
 - local def \$functorsUsed, 281
 - local def \$genno, 281
 - local def \$packagesUsed, 281
 - local ref \$EmptyMode, 281
 - local ref \$LocalDomainAlist, 281
 - local ref \$NRTopt, 281
 - local ref \$NonMentionableDomainNames, 281
 - local ref \$QuickCode, 281
 - local ref \$Representation, 281
 - local ref \$e, 281
 - local ref \$functorLocalParameters, 281
 - local ref \$functorsUsed, 281
 - local ref \$packagesUsed, 281
 - local ref \$predl, 281
 - local ref \$signatureOfForm, 281
 - defun, 281
- doIt
 - calledby compSingleCapsuleItem, 280

- doItIf, 285
 - calledby doIt, 281
 - calls compSingleCapsuleItem, 285
 - calls comp, 285
 - calls getSuccessEnvironment, 285
 - calls localExtras, 285
 - calls rplaca, 285
 - calls rplacd, 285
 - calls userError, 285
 - local def \$e, 285
 - local def \$functorLocalParameters, 285
 - local ref \$Boolean, 285
 - local ref \$e, 285
 - local ref \$functorLocalParameters, 285
 - local ref \$getDomainCode, 285
 - local ref \$predl, 285
 - defun, 285
- dollargreaterequal, 112
 - defplist, 112
- dollargreaterthan, 111
 - defplist, 111
- dollarnotequal, 113
 - defplist, 113
- dollarTran, 462
 - calledby PARSE-Qualification, 427
 - uses \$InteractiveMode, 462
 - defun, 462
- domainMember, 264
 - calledby addDomain, 252
 - calls modeEqual, 264
 - defun, 264
- doSystemCommand[5]
 - called by prepare1, 87
- downcase
 - calledby buildLibdbConEntry, 480
 - calledby getCaps, 185
- drop, 551
 - calledby drop, 551
 - calls croak, 551
 - calls drop, 551
 - calls take, 551
 - defun, 551
- dsetq
 - calledby buildLibdb, 476
- Echo-Meta
 - usedby prepare-echo, 95
- echo-meta
 - usedby /rf-1, 570
 - usedby /rf, 569
 - usedby /rq, 569
 - usedby spad, 572
- echo-meta[5]
 - called by /RQ,LIB, 569
- editfile
 - usedby compCapsule, 278
- elapsedTime
 - calledby compile, 173
- elemn
 - calledby PARSE-Operation, 424
 - calledby PARSE-leftBindingPowerOf, 425
 - calledby PARSE-rightBindingPowerOf, 425
- elt, 310
 - defplist, 310
- eltForm
 - calledby compApplication, 603
- eltModemapFilter, 605
 - calledby getFormModemaps, 604
 - calls isConstantId, 605
 - calls stackMessage, 605
 - defun, 605
- embed
 - calledby compilerDoitWithScreenedLisplib, 567
- encodeFunctionName, 183
 - calledby compile, 173
 - calls encodeItem, 183
 - calls getAbbreviation, 183
 - calls internl, 183
 - calls length, 183
 - calls mkRepititionAssoc, 183
 - local def \$lisplibSignatureAlist, 183
 - local ref \$lisplibSignatureAlist, 183
 - local ref \$lisplib, 183
 - defun, 183
- encodeItem, 185
 - calledby applyMapping, 591
 - calledby compApplication, 603
 - calledby compile, 173
 - calledby encodeFunctionName, 183
 - calls getCaps, 185
 - calls identp, 185

- calls pname, 185
 - defun, 185
- eofp
 - calledby dbReadLines, 479
- eq, 240
 - defplist, 240
- eqcar
 - calledby PARSE-OpenBrace, 443
 - calledby PARSE-OpenBracket, 443
 - calledby getToken, 455
 - calledby hackforis1, 413
- eqsubstlist
 - calledby compColon, 294
 - calledby compTypeOf, 593
 - calledby getSignatureFromMode, 303
 - calledby isDomainConstructorForm, 348
 - calledby substNames, 270
 - calledby substituteIntoFunctorModemap, 612
- EqualBarGensym, 248
 - calledby AssocBarGensym, 226
 - calledby optCond, 247
 - calls gensymp, 248
 - local def \$GensymAssoc, 248
 - local ref \$GensymAssoc, 248
 - defun, 248
- eqv, 114
 - defplist, 114
- erase
 - calledby initializeLisplib, 206
- error
 - calledby compileSpad2Cmd, 563
 - calledby processFunctor, 279
- errorRef
 - calledby compSymbol, 598
- errors
 - usedby initializeLisplib, 206
- Escape-Character
 - usedby token-lookahead-type, 452
- escape-keywords, 454
 - calledby quote-if-string, 454
 - local ref \$keywords, 454
 - defun, 454
- escaped, 551
 - calledby preparse1, 87
 - defun, 551
- eval
 - calledby coerceSubset, 357
 - calledby compDefineCategory2, 164
 - calledby compileCases, 171
 - calledby evalAndRwriteLispForm, 204
 - calledby getSlotFromCategoryForm, 210
 - calledby optCallEval, 238
- evalAndRwriteLispForm, 204
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby compSubDomain1, 351
 - calls eval, 204
 - calls rwriteLispForm, 204
 - defun, 204
- evalAndSub, 269
 - calledby augModemapsFromCategoryRep, 271
 - calledby augModemapsFromCategory, 265
 - calls contained, 269
 - calls getOperationAlist, 269
 - calls get, 269
 - calls isCategory, 269
 - calls put, 269
 - calls substNames, 269
 - local def \$lhsOfColon, 269
 - defun, 269
- exit, 311
 - defplist, 311
- expand-tabs, 94
 - calledby preparseReadLine1, 93
 - calls indent-pos, 94
 - calls nonblankloc, 94
 - defun, 94
- extendLocalLibdb, 475
 - calledby compileSpad2Cmd, 563
 - calls buildLibdb, 475
 - calls dbReadLines, 475
 - calls dbWriteLines, 475
 - calls deleteFile[5], 475
 - calls msort, 475
 - calls purgeNewConstructorLines, 475
 - calls union, 475
 - local def \$newConstructorList, 475
 - local ref \$createLocalLibDb, 475
 - local ref \$newConstructorList, 475
 - defun, 475

- extendsCategoryForm
 - calledby coerceHard, 358
 - calledby compWithMappingModel, 615
- extractCodeAndConstructTriple, 620
 - calledby compWithMappingModel, 615
 - defun, 620
- FactoredForm
 - calledby optCallEval, 238
- File-Closed
 - usedby read-a-line, 629
- file-closed
 - usedby spad, 572
- filep
 - calledby compDefineLisplib, 167
- fillerSpaces
 - calledby checkTransformFirsts, 512
 - calledby compDefineLisplib, 167
- finalizeDocumentation, 490
 - calledby compileDocumentation, 169
 - calledby finalizeLisplib, 207
 - calls assocleft, 490
 - calls bright, 490
 - calls form2String, 490
 - calls formatOpSignature, 490
 - calls macroExpand, 490
 - calls remdup, 490
 - calls sayKeyedMsg, 490
 - calls sayMSG, 490
 - calls strconc, 490
 - calls sublislis, 490
 - calls transDocList, 490
 - local ref \$FormalMapVariableList, 491
 - local ref \$comblocklist, 490
 - local ref \$docList, 490
 - local ref \$e, 490
 - local ref \$lisplibForm, 490
 - local ref \$op, 490
 - defun, 490
- finalizeLisplib, 207
 - calledby compDefineLisplib, 167
 - calls NRTgenInitialAttributeAlist, 207
 - calls finalizeDocumentation, 207
 - calls getConstructorOpsAndAtts, 207
 - calls lisplibWrite, 207
 - calls mergeSignatureAndLocalVarAlists, 207
 - calls namestring, 207
 - calls profileWrite, 207
 - calls removeZeroOne, 207
 - calls sayMSG, 207
 - local def \$NRTslot1PredicateList, 208
 - local def \$lisplibCategory, 208
 - local def \$pairlis, 208
 - local ref \$/editfile, 207
 - local ref \$FormalMapVariableList, 208
 - local ref \$libFile, 207
 - local ref \$lisplibAbbreviation, 208
 - local ref \$lisplibAncestors, 208
 - local ref \$lisplibAttributes, 208
 - local ref \$lisplibCategory, 207
 - local ref \$lisplibForm, 207, 208
 - local ref \$lisplibKind, 207
 - local ref \$lisplibModemapAlist, 207
 - local ref \$lisplibModemap, 207, 208
 - local ref \$lisplibParents, 208
 - local ref \$lisplibPredicates, 208
 - local ref \$lisplibSignatureAlist, 208
 - local ref \$lisplibSlot1, 208
 - local ref \$lisplibSuperDomain, 208
 - local ref \$lisplibVariableAlist, 208
 - local ref \$profileCompiler, 208
 - local ref \$spadLibFT, 208
 - defun, 207
- fincomblock, 551
 - calledby preparse1, 87
 - calls preparse-echo, 552
 - uses \$EchoLineStack, 552
 - uses \$comblocklist, 552
 - defun, 551
- findfile
 - calledby compiler, 561
- firstNonBlankPosition, 536
 - calledby checkAddIndented, 518
 - calledby checkExtract, 519
 - calledby checkGetArgs, 520
 - calledby checkGetMargin, 520
 - calledby checkIndentedLines, 522
 - calls maxindex, 536
 - defun, 536
- fixUpPredicate, 196

- calledby interactiveModemapForm, 195
- calls length, 196
- calls moveORsOutside, 196
- calls orderPredicateItems, 196
- defun, 196
- flattenSignatureList, 194
 - calledby flattenSignatureList, 194
 - calledby mkAListOfExplicitCategoryOps, 193
 - calls flattenSignatureList, 194
 - defun, 194
- floatexpid, 462
 - calledby PARSE-FloatExponent, 434
 - calls collect, 462
 - calls digitp[5], 462
 - calls identp[5], 462
 - calls maxindex, 462
 - calls pname[5], 462
 - calls spadreduce, 462
 - calls step, 462
 - defun, 462
- fnameMake[5]
 - calledby compileSpadLispCmd, 566
- fnameReadable?[5]
 - calledby compileSpadLispCmd, 566
- form2HtString
 - calledby buildLibdbConEntry, 480
 - calledby checkRecordHash, 507
- form2LispString
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
- form2String
 - calledby NRTgetLookupFunction, 214
 - calledby finalizeDocumentation, 490
- formal2Pattern, 218
 - calledby augmentLisplibModemapsFrom-Functor, 216
 - calls pairList, 218
 - calls sublis, 218
 - local ref \$PatternVariableList, 218
 - defun, 218
- formatOpSignature
 - calledby finalizeDocumentation, 490
- formatUnabbreviated
 - calledby compDefineCapsuleFunction, 156
 - calledby compMacro, 327
 - calledby doIt, 281
- formatUnabbreviatedSig
 - calledby displayMissingFunctions, 220
- fp-output-stream
 - calledby is-console, 553
- freelist, 623
 - calledby compWithMappingModel, 615
 - calledby freelist, 623
 - calls assq[5], 623
 - calls freelist, 623
 - calls getmode, 623
 - calls identp[5], 623
 - calls unionq, 623
 - defun, 623
- function
 - calledby optSpecialCall, 236
- genDeltaEntry
 - calledby coerceByModemap, 363
 - calledby compApplyModemap, 259
 - calledby transImplementation, 597
- genDomainOps, 224
 - calledby genDomainView, 224
 - calls addModemap, 225
 - calls getOperationAlist, 224
 - calls mkDomainConstructor, 225
 - calls mkq, 225
 - calls substNames, 225
 - uses \$ConditionalOperators, 225
 - uses \$e, 225
 - uses \$getDomainCode, 225
 - defun, 224
- genDomainView, 224
 - calledby genDomainViewList, 223
 - calls augModemapsFromCategory, 224
 - calls genDomainOps, 224
 - calls member, 224
 - calls mkDomainConstructor, 224
 - uses \$e, 224
 - uses \$getDomainCode, 224
 - defun, 224
- genDomainViewList, 223
 - calledby genDomainViewList, 223
 - calls genDomainViewList, 223
 - calls genDomainView, 223
 - calls isCategoryForm, 223

- uses `$EmptyEnvironment`, 223
 - defun, 223
- `genDomainViewList0`, 223
 - calledby `makeFunctorArgumentParameters`, 221
 - calls `getDomainViewList`, 223
 - defun, 223
- `genSomeVariable`
 - calledby `compColon`, 295
 - calledby `setqMultiple`, 341
- `gensymp`
 - calledby `EqualBarGensym`, 248
- `genvar`
 - calledby `hasAplExtension`, 407
- `genVariable`
 - calledby `setqMultipleExplicit`, 343
 - calledby `setqMultiple`, 341
- `get`
 - calledby `applyMapping`, 591
 - calledby `autoCoerceByModemap`, 364
 - calledby `coerceHard`, 358
 - calledby `coerceSubset`, 357
 - calledby `compAtom`, 595
 - calledby `compDefineCapsuleFunction`, 155
 - calledby `compFocompFormWithModemap`, 610
 - calledby `compMapCond`, 261
 - calledby `compNoStacking1`, 588
 - calledby `compSymbol`, 598
 - calledby `compTypeOf`, 593
 - calledby `compWithMappingMode1`, 615
 - calledby `compileCases`, 171
 - calledby `compile`, 173
 - calledby `doIt`, 281
 - calledby `evalAndSub`, 269
 - calledby `getArgumentMode`, 309
 - calledby `getDomainsInScope`, 254
 - calledby `getFormModemaps`, 604
 - calledby `getInverseEnvironment`, 320
 - calledby `getModemapListFromDomain`, 264
 - calledby `getModemapList`, 264
 - calledby `getModemap`, 258
 - calledby `getSignature`, 306
 - calledby `getSuccessEnvironment`, 319
 - calledby `giveFormalParametersValues`, 178
 - calledby `hasFullSignature`, 176
 - calledby `hasType`, 360
 - calledby `isFunctor`, 253
 - calledby `isMacro`, 286
 - calledby `isSuperDomain`, 255
 - calledby `isUnionMode`, 321
 - calledby `maxSuperType`, 348
 - calledby `mkEvalableCategoryForm`, 182
 - calledby `optCallSpecially`, 235
 - calledby `outputComp`, 347
 - calledby `parseHasRhs`, 118
 - calledby `setqSingle`, 344
- `get-a-line`, 636
 - calledby `initialize-prepare`, 82
 - calledby `prepareReadLine1`, 93
 - calls `is-console`, 636
 - calls `mkprompt[5]`, 636
 - calls `read-a-line`, 636
 - defun, 636
- `get-internal-run-time`
 - calledby `s-process`, 581
- `get-token`, 460
 - calledby `try-get-token`, 458
 - calls `XTokenReader`, 460
 - uses `XTokenReader`, 460
 - defun, 460
- `getAbbreviation`, 302
 - calledby `applyMapping`, 591
 - calledby `compApplication`, 603
 - calledby `compDefine1`, 145
 - calledby `encodeFunctionName`, 183
 - calls `assq`, 302
 - calls `constructor?`, 302
 - calls `mkAbbrev`, 302
 - calls `rplac`, 302
 - local def `$abbreviationTable`, 302
 - local ref `$abbreviationTable`, 302
 - defun, 302
- `getArgumentMode`, 309
 - calledby `checkAndDeclare`, 308
 - calledby `getArgumentModeOrMoan`, 191
 - calledby `hasSigInTargetCategory`, 308
 - calls `get`, 309
 - defun, 309
- `getArgumentModeOrMoan`, 191
 - calledby `compDefineCapsuleFunction`, 155
 - calledby `compDefineCategory2`, 163

- calledby compDefineFunctor1, 148
- calls getArgumentMode, 191
- calls stackSemanticError, 191
- defun, 191
- getCaps, 185
 - calledby encodeItem, 185
 - calls downcase, 185
 - calls maxindex, 185
 - calls strconc, 185
 - defun, 185
- getCategoryOpsAndAtts, 210
 - calledby getConstructorOpsAndAtts, 209
 - calls getSlotFromCategoryForm, 210
 - calls transformOperationAlist, 210
 - defun, 210
- getConstructorAbbreviation
 - calledby compDefineLisplib, 167
- getConstructorExports, 473
 - calledby buildLibdb, 476
 - defun, 473
- getConstructorOpsAndAtts, 209
 - calledby finalizeLisplib, 207
 - calls getCategoryOpsAndAtts, 209
 - calls getFunctorOpsAndAtts, 209
 - defun, 209
- getdatabase
 - calledby augModemapsFromDomain, 256
 - calledby buildLibdbConEntry, 480
 - calledby checkDocMessage, 518
 - calledby checkIsValidType, 530
 - calledby checkNumOfArgs, 532
 - calledby compDefineFunctor1, 149
 - calledby compDefineLisplib, 167
 - calledby compileConstructor1, 188
 - calledby getOperationAlist, 269
 - calledby isFunctor, 253
 - calledby loadLibIfNecessary, 119
 - calledby macroExpandList, 179
 - calledby mkCategoryPackage, 180
 - calledby mkEvaluableCategoryForm, 182
 - calledby parseHas, 116
 - calledby updateCategoryFrameForCategory, 121
 - calledby updateCategoryFrameForConstructor, 120
 - calledby whoOwns, 539
- getDeltaEntry
 - calledby compElt, 310
- getDomainsInScope, 254
 - calledby addDomain, 252
 - calledby augModemapsFromDomain, 256
 - calledby comp3, 590
 - calledby compColon, 294
 - calledby compConstruct, 299
 - calledby putDomainsInScope, 255
 - calls get, 254
 - local ref \$CapsuleDomainsInScope, 254
 - local ref \$insideCapsuleFunctionIfTrue, 254
 - defun, 254
- getDomainViewList
 - calledby genDomainViewList0, 223
- getExportCategory
 - calledby NRTgetLookupFunction, 214
- getFormModemaps, 604
 - calledby compForm1, 600
 - calledby getFormModemaps, 604
 - calls eltModemapFilter, 604
 - calls getFormModemaps, 604
 - calls get, 604
 - calls last, 604
 - calls length, 604
 - calls nreverse0, 604
 - calls stackMessage, 604
 - local ref \$insideCategoryPackageIfTrue, 605
 - defun, 604
- getFunctorOpsAndAtts, 212
 - calledby getConstructorOpsAndAtts, 209
 - calls getSlotFromFunctor, 212
 - calls transformOperationAlist, 212
 - defun, 212
- getIdentity
 - calledby compReduce1, 331
- getInverseEnvironment, 320
 - calledby compBoolean, 318
 - calls delete, 320
 - calls getUnionMode, 320
 - calls get, 320
 - calls identp, 320
 - calls isDomainForm, 320
 - calls member, 320

- calls mkpf, 320
- calls put, 320
- local ref \$EmptyEnvironment, 320
- defun, 320
- getl
 - calledby PARSE-Operation, 424
 - calledby PARSE-ReductionOp, 428
 - calledby PARSE-leftBindingPowerOf, 425
 - calledby PARSE-rightBindingPowerOf, 425
 - calledby addConstructorModemaps, 258
 - calledby augModemapsFromDomain1, 257
 - calledby checkRecordHash, 507
 - calledby checkTransformFirst, 512
 - calledby compCat, 289
 - calledby compExpression, 139
 - calledby loadLibIfNecessary, 119
 - calledby mustInstantiate, 293
 - calledby optSpecialCall, 236
 - calledby optimize, 229
 - calledby parseTran, 101
- getMatchingRightPren, 536
 - calledby checkGetArgs, 520
 - calledby checkTransformFirst, 512
 - calls maxindex, 536
 - defun, 536
- getmode
 - calledby addDomain, 252
 - calledby augModemapsFromDomain1, 257
 - calledby coerceHard, 358
 - calledby comp3, 589
 - calledby compCoerce, 362
 - calledby compColon, 294
 - calledby compDefWhereClause, 159
 - calledby compDefineCapsuleFunction, 156
 - calledby compSymbol, 598
 - calledby compile, 173
 - calledby displayMissingFunctions, 220
 - calledby freelist, 623
 - calledby getSignatureFromMode, 303
 - calledby getSignature, 306
 - calledby getUnionMode, 321
 - calledby isUnionMode, 321
 - calledby setqSingle, 344
- getModemap, 258
 - calledby compDefineFunctor1, 148
 - calls compApplyModemap, 259
 - calls get, 258
 - calls sublis, 259
 - defun, 258
- getModemapList, 263
 - calledby autoCoerceByModemap, 364
 - calledby compCase1, 288
 - calledby getUniqueModemap, 263
 - calls getModemapListFromDomain, 263
 - calls get, 264
 - calls nreverse0, 264
 - defun, 263
- getModemapListFromDomain, 264
 - calledby compElt, 310
 - calledby getModemapList, 263
 - calls get, 264
 - defun, 264
- getmodeOrMapping
 - calledby augModemapsFromDomain1, 257
- getOperationAlist, 269
 - calledby evalAndSub, 269
 - calledby genDomainOps, 224
 - calls compMakeCategoryObject, 269
 - calls getdatabase, 269
 - calls isFunctor, 269
 - calls stackMessage, 269
 - calls systemError, 269
 - uses \$domainShell, 270
 - uses \$e, 270
 - uses \$functorForm, 270
 - uses \$insideFunctorIfTrue, 270
 - defun, 269
- getOperationAlistFromLisplib
 - calledby mkOpVec, 225
- getParentsFor
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
- getPrincipalView
 - calledby mkOpVec, 225
- getProplist
 - calledby addModemap1, 274
 - calledby compDefineAddSignature, 147
 - calledby getSuccessEnvironment, 318
 - calledby loadLibIfNecessary, 119
- getProplist[5]
 - called by setqSingle, 344
- getScriptName, 408

- calledby postScriptsForm, 372
- calledby postScripts, 398
- calls decodeScripts, 409
- calls identp[5], 408
- calls internl, 408
- calls pname[5], 409
- calls postError, 408
- defun, 408
- getSignature, 306
 - calledby compDefineCapsuleFunction, 155
 - calls SourceLevelSubsume, 307
 - calls getmode, 306
 - calls get, 306
 - calls knownInfo, 306
 - calls length, 306
 - calls printSignature, 307
 - calls remdup, 306
 - calls say, 306
 - calls stackSemanticError, 307
 - local ref \$e, 307
 - defun, 306
- getSignatureFromMode, 303
 - calledby compDefine1, 145
 - calledby hasSigInTargetCategory, 308
 - calls eqsubstlist, 303
 - calls getmode, 303
 - calls length, 303
 - calls opOf, 303
 - calls stackAndThrow, 303
 - calls take, 303
 - local ref \$FormalMapVariableList, 304
 - defun, 303
- getSlotFromCategoryForm, 210
 - calledby getCategoryOpsAndAtts, 210
 - calls eval, 210
 - calls systemErrorHere, 210
 - calls take, 210
 - local ref \$FormalMapVariableList, 210
 - defun, 210
- getSlotFromFunctor, 212
 - calledby getFunctorOpsAndAtts, 212
 - calls compMakeCategoryObject, 212
 - calls systemErrorHere, 212
 - local ref \$e, 212
 - local ref \$lisplibOperationAlist, 212
 - defun, 212
- getSpecialCaseAssoc, 304
 - calledby compileCases, 171
 - local ref \$functorForm, 304
 - local ref \$functorSpecialCases, 304
 - defun, 304
- getSuccessEnvironment, 318
 - calledby compBoolean, 318
 - calledby doItIf, 285
 - calls addBinding, 319
 - calls comp, 319
 - calls consProplistOf, 319
 - calls getProplist, 318
 - calls get, 319
 - calls identp, 318
 - calls isDomainForm, 318
 - calls put, 318
 - calls removeEnv, 319
 - local ref \$EmptyEnvironment, 319
 - local ref \$EmptyMode, 319
 - defun, 318
- getTargetFromRhs, 177
 - calledby compDefine1, 145
 - calledby getTargetFromRhs, 177
 - calls compOrCroak, 177
 - calls getTargetFromRhs, 177
 - calls stackSemanticError, 177
 - defun, 177
- getTempPath
 - calledby dbWriteLines, 479
- getToken, 455
 - calledby PARSE-OpenBrace, 443
 - calledby PARSE-OpenBracket, 443
 - calls eqcar, 455
 - defun, 455
- getUnionMode, 321
 - calledby getInverseEnvironment, 320
 - calls getmode, 321
 - calls isUnionMode, 321
 - defun, 321
- getUniqueModemap, 263
 - calledby getUniqueSignature, 263
 - calls getModemapList, 263
 - calls qslessp, 263
 - calls stackWarning, 263
 - defun, 263
- getUniqueSignature, 263

- calls getUniqueModemap, 263
 - defun, 263
- giveFormalParametersValues, 178
 - calledby compDefine1, 145
 - calledby compDefineCapsuleFunction, 155
 - calledby compDefineCategory2, 163
 - calledby compDefineFunctor1, 148
 - calls get, 178
 - calls put, 178
 - defun, 178
- hackforis, 412
 - calls hackforis1, 412
 - defun, 412
- hackforis1, 413
 - calledby hackforis, 412
 - calls eqcar, 413
 - defun, 413
- has, 116, 312
 - defplist, 116, 312
- hasAplExtension, 407
 - calledby aplTran1, 405
 - calls aplTran1, 407
 - calls deepestExpression, 407
 - calls genvar, 407
 - calls nreverse0, 407
 - defun, 407
- hasFormalMapVariable, 621
 - calledby compWithMappingModel, 615
 - calls ScanOrPairVec[5], 621
 - local def \$formalMapVariables, 621
 - defun, 621
- hasFullSignature, 176
 - calledby compDefineAddSignature, 147
 - calls get, 176
 - defun, 176
- hasNoVowels, 537
 - calledby checkDecorate, 502
 - calls maxindex, 537
 - defun, 537
- hasSigInTargetCategory, 308
 - calledby compDefineCapsuleFunction, 155
 - calls bright, 309
 - calls compareMode2Arg, 309
 - calls getArgumentMode, 308
 - calls getSignatureFromMode, 308
 - calls length, 308
 - calls remdup, 308
 - calls stackWarning, 309
 - local ref \$domainShell, 309
 - defun, 308
- hasType, 360
 - calledby coerceExtraHard, 359
 - calls get, 360
 - defun, 360
- helpSpad2Cmd[5]
 - called by compiler, 561
- hget
 - calledby checkArguments, 499
 - calledby checkBeginEnd, 500
 - calledby checkRecordHash, 507
 - calledby checkSplitPunctuation, 535
- hput
 - calledby checkRecordHash, 507
- htcharPosition, 537
 - calledby checkTrimCommented, 525
 - calledby htcharPosition, 537
 - calls charPosition, 537
 - calls htcharPosition, 537
 - calls length, 537
 - defun, 537
- identp
 - calledby addDomain, 252
 - calledby compFocompFormWithModemap, 610
 - calledby compInternalFunction, 159
 - calledby constructMacro, 186
 - calledby encodeItem, 185
 - calledby getInverseEnvironment, 320
 - calledby getSuccessEnvironment, 318
 - calledby isFunctor, 253
 - calledby mkExplicitCategoryFunction, 292
 - calledby subrname, 232
- identp[5]
 - called by PARSE-FloatExponent, 434
 - called by compSetq1, 340
 - called by floatexpid, 462
 - called by freelist, 623
 - called by getScriptName, 408
 - called by postTransform, 369
 - called by setqSingle, 344

- if, [121](#), [314](#), [391](#)
 - defplist, [121](#), [314](#), [391](#)
- ifcar
 - calledby buildLibdb, [476](#)
 - calledby checkBeginEnd, [500](#)
 - calledby checkFixCommonProblem, [505](#)
 - calledby checkTexht, [511](#)
 - calledby dbWriteLines, [479](#)
 - calledby preparse, [86](#)
- ifcdr
 - calledby checkBeginEnd, [500](#)
 - calledby checkFixCommonProblem, [505](#)
 - calledby checkHTargs, [507](#)
 - calledby checkRecordHash, [507](#)
 - calledby checkTexht, [511](#)
 - calledby recordAttributeDocumentation, [468](#)
- implies, [124](#)
 - defplist, [124](#)
- import, [322](#)
 - defplist, [322](#)
- In, [393](#)
 - defplist, [393](#)
- in, [125](#), [392](#)
 - defplist, [125](#), [392](#)
- inby, [126](#)
 - defplist, [126](#)
- incExitLevel
 - calledby parseIf,ifTran, [122](#)
- indent-pos, [552](#)
 - calledby expand-tabs, [94](#)
 - calledby preparse1, [87](#)
 - calls next-tab-loc, [552](#)
 - defun, [552](#)
- infixtok, [553](#)
 - calls string2id-n, [553](#)
 - defun, [553](#)
- init-boot/spad-reader[5]
 - called by spad, [572](#)
- initial-gensym, [412](#)
 - defvar, [412](#)
- initial-substring, [635](#)
 - calledby skip-ifblock, [92](#)
 - calledby skip-to-endif, [554](#)
 - defun, [635](#)
- initial-substring-p, [453](#)
 - calledby match-string, [451](#)
 - calls string-not-greaterp, [453](#)
 - defun, [453](#)
- initialize-preparse, [82](#)
 - calledby spad, [572](#)
 - calls get-a-line, [82](#)
 - uses \$echolinestack, [82](#)
 - uses \$index, [82](#)
 - uses \$linelist, [82](#)
 - uses \$preparse-last-line, [82](#)
 - defun, [82](#)
- initializeLisplib, [206](#)
 - calls LAM,FILEACTQ, [206](#)
 - calls adoptions, [206](#)
 - calls erase, [206](#)
 - calls pathnameTypeId, [206](#)
 - calls writeLib1, [206](#)
 - local def \$libFile, [206](#)
 - local def \$lisplibAbbreviation, [206](#)
 - local def \$lisplibAncestors, [206](#)
 - local def \$lisplibForm, [206](#)
 - local def \$lisplibKind, [206](#)
 - local def \$lisplibModemapAlist, [206](#)
 - local def \$lisplibModemap, [206](#)
 - local def \$lisplibOpAlist, [206](#)
 - local def \$lisplibOperationAlist, [206](#)
 - local def \$lisplibSignatureAlist, [206](#)
 - local def \$lisplibSuperDomain, [206](#)
 - local def \$lisplibVariableAlist, [206](#)
 - local ref \$erase, [206](#)
 - local ref \$libFile, [206](#)
 - uses /editfile, [206](#)
 - uses /major-version, [206](#)
 - uses errors, [206](#)
 - defun, [206](#)
- insert
 - calledby comp2, [589](#)
 - calledby doIt, [281](#)
- insertAlist
 - calledby transformOperationAlist, [211](#)
- insertModemap, [267](#)
 - calledby mkNewModemapList, [266](#)
 - defun, [267](#)
- insertWOC
 - calledby orderPredTran, [198](#)
- Integer

- calledby optCallEval, 237
- interactiveModemapForm, 195
 - calledby augLisplibModemapsFromCategory, 192
 - calledby augmentLisplibModemapsFromFunctor, 216
 - calls fixUpPredicate, 195
 - calls modemapPattern, 195
 - calls replaceVars, 195
 - calls substVars, 195
 - local ref \$FormalMapVariableList, 195
 - local ref \$PatternVariableList, 195
 - defun, 195
- intern
 - calledby checkRecordHash, 507
- internl
 - calledby encodeFunctionName, 183
 - calledby getScriptName, 408
 - calledby postForm, 374
 - calledby substituteCategoryArguments, 257
- intersection
 - calledby orderByDependency, 227
- intersectionEnvironment
 - calledby compIf, 315
 - calledby replaceExitEtc, 337
- intersectionq
 - calledby orderPredTran, 198
- ioclear
 - calledby spad, 572
- is, 127, 322
 - defplist, 127, 322
- is-console, 553
 - calledby get-a-line, 636
 - calledby prepare1, 87
 - calledby print-defun, 584
 - calls fp-output-stream, 553
 - uses *terminal-io*, 553
 - defun, 553
- isAlmostSimple
 - calledby makeSimplePredicateOrNil, 544
- isCategory
 - calledby augModemapsFromCategoryRep, 271
 - calledby evalAndSub, 269
- isCategoryForm
 - calledby addDomain, 252
 - calledby applyMapping, 591
 - calledby augLisplibModemapsFromCategory, 192
 - calledby coerceHard, 358
 - calledby compApplication, 603
 - calledby compColon, 294
 - calledby compFocompFormWithModemap, 610
 - calledby compJoin, 323
 - calledby compMakeCategoryObject, 213
 - calledby genDomainViewList, 223
 - calledby isDomainConstructorForm, 348
 - calledby isDomainForm, 348
 - calledby makeCategoryForm, 297
 - calledby makeFunctorArgumentParameters, 221
 - calledby mkAlistOfExplicitCategoryOps, 193
 - calledby mkDatabasePred, 218
 - calledby signatureTran, 197
- isCategoryPackageName, 214
 - calledby compDefineFunctor1, 148, 149
 - calledby substNames, 270
 - calls char, 214
 - calls maxindex, 214
 - calls pname, 214
 - defun, 214
- isConstantId
 - calledby eltModemapFilter, 605
 - calledby seteltModemapFilter, 606
- isDomainConstructorForm, 348
 - calledby isDomainForm, 348
 - calls eqsubstlist, 348
 - calls isCategoryForm, 348
 - local ref \$FormalMapVariableList, 349
 - defun, 348
- isDomainForm, 348
 - calledby comp2, 589
 - calledby compColon, 294
 - calledby compDefine1, 145
 - calledby compElt, 310
 - calledby compHasFormat, 313
 - calledby doIt, 281
 - calledby getInverseEnvironment, 320
 - calledby getSuccessEnvironment, 318
 - calledby setqSingle, 344

- calls isCategoryForm, 348
- calls isDomainConstructorForm, 348
- calls isFunctor, 348
- local ref \$SpecialDomainNames, 348
- defun, 348
- isDomainInScope
 - calledby setqSingle, 344
- isDomainSubst, 200
 - calledby orderPredTran, 198
 - defun, 200
- isExposedConstructor
 - calledby buildLibdbConEntry, 480
- isFluid
 - calledby compSymbol, 598
- isFunction
 - calledby compSymbol, 598
- isFunctor, 253
 - calledby addDomain, 252
 - calledby comp2, 589
 - calledby compFocompFormWithModemap, 610
 - calledby compWithMappingModel, 615
 - calledby getOperationAlist, 269
 - calledby isDomainForm, 348
 - calls constructor?, 253
 - calls getdatabase, 253
 - calls get, 253
 - calls identp, 253
 - calls opOf, 253
 - calls updateCategoryFrameForCategory, 253
 - calls updateCategoryFrameForConstructor, 253
 - local ref \$CategoryFrame, 253
 - local ref \$InteractiveMode, 254
 - defun, 253
- isListConstructor, 111
 - calledby transIs, 110
 - calls member, 111
 - defun, 111
- isLiteral
 - calledby addDomain, 252
- isMacro, 286
 - calledby compDefine1, 145
 - calledby doIt, 281
 - calls get, 286
 - defun, 286
- isnt, 127
 - defplist, 127
- isSimple
 - calledby compForm2, 608
 - calledby makeSimplePredicateOrNil, 544
- isSomeDomainVariable
 - calledby coerce, 356
- isSubset
 - calledby coerceByModemap, 363
 - calledby coerceSubset, 357
 - calledby isSuperDomain, 255
- isSuperDomain, 255
 - calledby mergeModemap, 267
 - calls get, 255
 - calls isSubset, 255
 - calls lassoc, 255
 - calls opOf, 255
 - defun, 255
- isSymbol
 - calledby compAtom, 595
- isTokenDelimiter, 455
 - calledby PARSE-TokenList, 418
 - calls current-symbol, 455
 - defun, 455
- isUnionMode, 321
 - calledby coerceExtraHard, 359
 - calledby getUnionMode, 321
 - calls getmode, 321
 - calls get, 321
 - defun, 321
- Join, 128, 323, 393
 - defplist, 128, 323, 393
- JoinInner
 - calledby mkCategoryPackage, 180
- keyedSystemError
 - calledby NRTputInHead, 190
 - calledby coerce, 355
 - calledby compileTimeBindingOf, 237
 - calledby mkAlistOfExplicitCategoryOps, 193
 - calledby optRECORDELT, 250
 - calledby optSETRECORDELT, 251
 - calledby optSpecialCall, 236

- calledby substituteIntoFunctorModemap, 612
 - calledby transformOperationAlist, 211
- killColons, 401
 - calledby killColons, 401
 - calledby postSignature, 400
 - calls killColons, 401
 - defun, 401
- knownInfo
 - calledby addModemap, 273
 - calledby compMapCond", 261
 - calledby getSignature, 306
- labasoc
 - usedby PARSE-Data, 439
- lablasoc, 414
 - defvar, 414
- LAM,EVALANDFILEACTQ
 - calledby spadCompileOrSetq, 186
- LAM,FILEACTQ
 - calledby initializeLisplib, 206
- lassoc
 - calledby NRTputInTail, 189
 - calledby addModemap1, 274
 - calledby augLisplibModemapsFromCategory, 192
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
 - calledby checkAddMacros, 527
 - calledby checkTransformFirsts, 512
 - calledby coerceSubset, 357
 - calledby compDefWhereClause, 160
 - calledby compDefineAddSignature, 147
 - calledby compOrCroak1,compactify, 624
 - calledby isSuperDomain, 255
 - calledby loadLibIfNecessary, 119
 - calledby mergeSignatureAndLocalVarAlists, 213
 - calledby optCallSpecially, 235
 - calledby spadSysChoose, 510
 - calledby translabl1, 541
- lassq
 - calledby transformOperationAlist, 211
- last
 - calledby compFocompFormWithModemap, 610
 - calledby getFormModemaps, 604
 - calledby parseSeq, 136
 - calledby setqMultipleExplicit, 343
- lastnode
 - calledby NRTputInHead, 190
 - calledby doIt, 281
- leave, 128, 326
 - defplist, 128, 326
- leftTrim
 - calledby checkTransformFirsts, 512
- length
 - calledby buildLibAttr, 483
 - calledby buildLibdbConEntry, 480
 - calledby checkBeginEnd, 500
 - calledby checkExtract, 519
 - calledby checkIsValidType, 530
 - calledby checkSplitBrace, 524
 - calledby checkTrimCommented, 525
 - calledby compApplication, 603
 - calledby compApplyModemap, 259
 - calledby compColon, 294
 - calledby compDefine1, 145
 - calledby compDefineCapsuleFunction, 155
 - calledby compElt, 310
 - calledby compForm1, 600
 - calledby compForm2, 607
 - calledby compHasFormat, 313
 - calledby compRepeatOrCollect, 333
 - calledby displayPreCompilationErrors, 542
 - calledby encodeFunctionName, 183
 - calledby fixUpPredicate, 196
 - calledby getFormModemaps, 604
 - calledby getSignatureFromMode, 303
 - calledby getSignature, 306
 - calledby hasSigInTargetCategory, 308
 - calledby htcharPosition, 537
 - calledby mkOpVec, 225
 - calledby modeEqualSubst, 367
 - calledby optMkRecord, 249
 - calledby postScriptsForm, 372
 - calledby removeBackslashes, 539
 - calledby setqMultiple, 341
- lessp, 242
 - defplist, 242

- let, 130, 339
 - defplist, 130, 339
- letd, 130
 - defplist, 130
- letError
 - calledby newDef2Def, 80
 - calledby newIf2Cond, 79
- libConstructorSig
 - calledby buildLibdbConEntry, 480
- libdbTrim
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
- line, 631
 - usedby match-string, 451
 - usedby spad, 572
 - defstruct, 631
- Line-Advance-Char
 - calledby Advance-Char, 635
- line-advance-char, 633
 - uses \$line, 633
 - defun, 633
- Line-At-End-P
 - calledby Advance-Char, 635
- line-at-end-p, 632
 - calledby next-char, 461
 - uses \$line, 632
 - defun, 632
- line-clear, 632
 - uses \$line, 632
 - defmacro, 632
- line-current-char
 - calledby match-advance-string, 453
- line-current-index
 - calledby match-advance-string, 452
- line-current-segment, 634
 - calledby unget-tokens, 456
 - defun, 634
- line-handler
 - usedby string2BootTree, 77
- Line-New-Line
 - calledby read-a-line, 629
- line-new-line, 634
 - calledby unget-tokens, 456
 - uses \$line, 634
 - defun, 634
- line-next-char, 633
 - calledby next-char, 461
 - uses \$line, 633
 - defun, 633
- line-number
 - calledby PARSE-Category, 421
 - calledby unget-tokens, 456
- line-past-end-p, 633
 - calledby match-advance-string, 453
 - calledby match-string, 451
 - uses \$line, 633
 - defun, 633
- line-print, 632
 - local ref \$out-stream, 632
 - uses \$line, 632, 634
 - defun, 632
- lispize, 351
 - calledby compSubDomain1, 351
 - calls optimize, 351
 - defun, 351
- lisplibDoRename, 205
 - calledby compDefineLisplib, 167
 - calls replaceFile, 205
 - local ref \$spadLibFT, 205
 - defun, 205
- lisplibWrite, 214
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby compileDocumentation, 169
 - calledby finalizeLisplib, 207
 - calls rwrite128, 214
 - local ref \$lisplib, 214
 - defun, 214
- List
 - calledby optCallEval, 237
- ListCategory, 300
 - defplist, 300
- listOfIdentifiersIn
 - calledby compDefWhereClause, 160
- listOfPatternIds
 - calledby augmentLisplibModemapsFrom-Functor, 216
 - calledby orderPredTran, 198
- listOfSharpVars
 - calledby compFocompFormWithModemap, 610
- listOrVectorElementNode

- calledby augModemapsFromDomain, 256
- loadIfNecessary, 118
 - calledby parseHasRhs, 118
 - calls loadLibIfNecessary, 118
 - defun, 118
- loadLib
 - calledby loadLibIfNecessary, 119
- loadLibIfNecessary, 119
 - calledby loadIfNecessary, 118
 - calledby loadLibIfNecessary, 119
 - calls canFuncall?, 119
 - calls getProplist, 119
 - calls getdatabase, 119
 - calls getl, 119
 - calls lassoc, 119
 - calls loadLibIfNecessary, 119
 - calls loadLib, 119
 - calls macrop, 119
 - calls throwKeyedMsg, 119
 - calls updateCategoryFrameForCategory, 119
 - calls updateCategoryFrameForConstructor, 119
 - local ref \$CategoryFrame, 119
 - local ref \$InteractiveMode, 119
 - defun, 119
- localdatabase
 - calledby compDefineLisplib, 167
- localdatabase[5]
 - calledby compileSpadLispCmd, 566
- localExtras
 - calledby doItIf, 285
- lt
 - calledby PARSE-Operation, 424
- macroExpand, 178
 - calledby compDefine1, 145
 - calledby compMacro, 327
 - calledby compSeqItem, 338
 - calledby compWhere, 355
 - calledby finalizeDocumentation, 490
 - calledby macroExpandInPlace, 178
 - calledby macroExpandList, 179
 - calledby macroExpand, 178
 - calls macroExpandList, 179
 - calls macroExpand, 178
 - defun, 178
- macroExpandInPlace, 178
 - calledby compSingleCapsuleItem, 280
 - calls macroExpand, 178
 - defun, 178
- macroExpandList, 179
 - calledby macroExpand, 179
 - calls getdatabase, 179
 - calls macroExpand, 179
 - defun, 179
- macrop
 - calledby loadLibIfNecessary, 119
- make-float
 - calledby PARSE-Float, 433
- make-full-cvec
 - calledby prepare1, 87
- make-outstream
 - calledby dbWriteLines, 479
- make-outstream[5]
 - called by buildLibdb, 476
- make-reduction
 - calledby push-reduction, 465
- make-symbol-of, 458
 - calledby PARSE-Expression, 422
 - calledby current-symbol, 457
 - uses \$token, 458
 - defun, 458
- makeCategoryForm, 297
 - calledby compColon, 294
 - calls compOrCroak, 297
 - calls isCategoryForm, 297
 - local ref \$EmptyMode, 297
 - defun, 297
- makeCategoryPredicates, 179
 - calledby compDefineCategory1, 162
 - uses \$FormalMapVariableList, 179
 - uses \$TriangleVariableList, 180
 - uses \$mvl, 180
 - uses \$tvl, 180
 - defun, 179
- makeFunctorArgumentParameters, 221
 - calledby compDefineFunctor1, 148
 - calls assq, 221
 - calls genDomainViewList0, 221
 - calls isCategoryForm, 221
 - calls union, 221

- uses \$ConditionalOperators, [221](#)
 - uses \$alternateViewList, [221](#)
 - uses \$forceAdd, [221](#)
 - defun, [221](#)
- makeInitialModemapFrame[5]
 - called by spad, [572](#)
- makeInputFilename
 - called by compileDocumentation, [169](#)
- makeInputFilename[5]
 - called by /rf-1, [570](#)
- makeLiteral
 - called by addEltModemap, [265](#)
- makeNonAtomic
 - called by parseHas, [116](#)
- makeSimplePredicateOrNil, [544](#)
 - called by parseIf,ifTran, [122](#)
 - calls isAlmostSimple, [544](#)
 - calls isSimple, [544](#)
 - calls wrapSEQExit, [544](#)
 - defun, [544](#)
- mapInto
 - called by parseSeq, [136](#)
 - called by parseWhere, [137](#)
- Mapping, [289](#)
 - defplist, [289](#)
- match-advance-string, [452](#)
 - called by PARSE-Category, [421](#)
 - called by PARSE-Command, [415](#)
 - called by PARSE-Conditional, [448](#)
 - called by PARSE-Enclosure, [435](#)
 - called by PARSE-Exit, [447](#)
 - called by PARSE-FloatBasePart, [434](#)
 - called by PARSE-FloatExponent, [434](#)
 - called by PARSE-Form, [429](#)
 - called by PARSE-Import, [422](#)
 - called by PARSE-IteratorTail, [444](#)
 - called by PARSE-Iterator, [444](#)
 - called by PARSE-Label, [430](#)
 - called by PARSE-Leave, [447](#)
 - called by PARSE-Loop, [449](#)
 - called by PARSE-Option, [419](#)
 - called by PARSE-Primary1, [432](#)
 - called by PARSE-PrimaryOrQM, [419](#)
 - called by PARSE-Quad, [436](#)
 - called by PARSE-Qualification, [427](#)
 - called by PARSE-Return, [446](#)
 - called by PARSE-ScriptItem, [438](#)
 - called by PARSE-Scripts, [437](#)
 - called by PARSE-Selector, [430](#)
 - called by PARSE-SemiColon, [446](#)
 - called by PARSE-Sequence, [442](#)
 - called by PARSE-Sexpr1, [440](#)
 - called by PARSE-SpecialCommand, [416](#)
 - called by PARSE-Statement, [419](#)
 - called by PARSE-TokenOption, [417](#)
 - called by PARSE-With, [420](#)
 - calls current-token, [452](#)
 - calls line-current-char, [453](#)
 - calls line-current-index, [452](#)
 - calls line-past-end-p, [453](#)
 - calls match-string, [452](#)
 - calls quote-if-string, [452](#)
 - uses \$line, [453](#)
 - uses \$token, [453](#)
 - defun, [452](#)
- match-current-token, [456](#)
 - called by PARSE-GlyphTok, [441](#)
 - called by PARSE-NBGlyphTok, [441](#)
 - called by PARSE-Operation, [424](#)
 - called by PARSE-SpecialKeyWord, [415](#)
 - called by parse-argument-designator, [546](#)
 - called by parse-identifier, [545](#)
 - called by parse-keyword, [546](#)
 - called by parse-number, [545](#)
 - called by parse-spadstring, [544](#)
 - called by parse-string, [544](#)
 - calls current-token, [456](#)
 - calls match-token, [456](#)
 - defun, [456](#)
- match-next-token, [457](#)
 - called by PARSE-ReductionOp, [428](#)
 - calls match-token, [457](#)
 - calls next-token, [457](#)
 - defun, [457](#)
- match-string, [451](#)
 - called by PARSE-AnyId, [441](#)
 - called by PARSE-NewExpr, [414](#)
 - called by PARSE-Primary1, [432](#)
 - called by match-advance-string, [452](#)
 - calls current-char, [451](#)
 - calls initial-substring-p, [451](#)
 - calls line-past-end-p, [451](#)

- calls skip-blanks, [451](#)
 - calls subseq, [451](#)
 - calls unget-tokens, [451](#)
 - uses \$line, [451](#)
 - uses line, [451](#)
 - defun, [451](#)
- match-token, [457](#)
 - calledby match-current-token, [456](#)
 - calledby match-next-token, [457](#)
 - calls token-symbol, [457](#)
 - calls token-type, [457](#)
 - defun, [457](#)
- Matrix
 - calledby optCallEval, [238](#)
- maxindex
 - calledby buildLibdbConEntry, [480](#)
 - calledby checkAddBackSlashes, [526](#)
 - calledby checkAddPeriod, [527](#)
 - calledby checkAddSpaceSegments, [528](#)
 - calledby checkGetArgs, [520](#)
 - calledby checkIeEgfun, [529](#)
 - calledby checkSplitBackslash, [533](#)
 - calledby checkSplitOn, [534](#)
 - calledby checkSplitPunctuation, [535](#)
 - calledby checkTransformFirsts, [512](#)
 - calledby compDefineFunctor1, [148](#)
 - calledby firstNonBlankPosition, [536](#)
 - calledby floatexpid, [462](#)
 - calledby getCaps, [185](#)
 - calledby getMatchingRightPren, [536](#)
 - calledby hasNoVowels, [537](#)
 - calledby isCategoryPackageName, [214](#)
 - calledby preparse1, [87](#)
 - calledby preparseReadLine1, [93](#)
 - calledby translabe11, [541](#)
- maxSuperType, [348](#)
 - calledby coerceSubset, [357](#)
 - calledby maxSuperType, [348](#)
 - calledby setqSingle, [344](#)
 - calls get, [348](#)
 - calls maxSuperType, [348](#)
 - defun, [348](#)
- mbpip
 - calledby subrname, [232](#)
- mdef, [131](#), [327](#)
 - defplist, [131](#), [327](#)
- member
 - calledby addDomain, [252](#)
 - calledby applyMapping, [591](#)
 - calledby augLisplibModemapsFromCategory, [192](#)
 - calledby augModemapsFromDomain, [256](#)
 - calledby augmentLisplibModemapsFromFunctor, [216](#)
 - calledby autoCoerceByModemap, [364](#)
 - calledby checkBeginEnd, [500](#)
 - calledby checkDecorateForHt, [504](#)
 - calledby checkDecorate, [502](#)
 - calledby checkFixCommonProblem, [505](#)
 - calledby checkRecordHash, [507](#)
 - calledby checkSkipOpToken, [523](#)
 - calledby coerceExtraHard, [359](#)
 - calledby compApplication, [603](#)
 - calledby compApplyModemap, [259](#)
 - calledby compDefineCapsuleFunction, [156](#)
 - calledby compile, [173](#)
 - calledby displayMissingFunctions, [220](#)
 - calledby doIt, [281](#)
 - calledby genDomainView, [224](#)
 - calledby getInverseEnvironment, [320](#)
 - calledby isListConstructor, [111](#)
 - calledby mkNewModemapList, [266](#)
 - calledby orderByDependency, [227](#)
 - calledby orderPredTran, [198](#)
 - calledby parseHasRhs, [118](#)
 - calledby parseHas, [116](#)
 - calledby putDomainsInScope, [255](#)
 - calledby spadSysBranch, [510](#)
 - calledby transformOperationAlist, [211](#)
- member[5]
 - called by comp3, [590](#)
 - called by compColon, [294](#)
 - called by compSymbol, [598](#)
 - called by compilerDoit, [568](#)
- mergeModemap, [267](#)
 - calledby mkNewModemapList, [266](#)
 - calls TruthP, [267](#)
 - calls isSuperDomain, [267](#)
 - local ref \$forceAdd, [267](#)
 - defun, [267](#)
- mergePathnames[5]
 - called by compiler, [561](#)

- mergeSignatureAndLocalVarAlists, 213
 - calledby finalizeLisplib, 207
 - calls lassoc, 213
 - defun, 213
- meta-error-handler, 461
 - calledby meta-syntax-error, 462
 - usedby meta-syntax-error, 462
 - defvar, 461
- meta-syntax-error, 462
 - calledby must, 463
 - calls meta-error-handler, 462
 - uses meta-error-handler, 462
 - defun, 462
- minus, 240
 - defplist, 240
- mkAbbrev, 302
 - calledby getAbbreviation, 302
 - calls addSuffix, 302
 - calls alistSize, 302
 - defun, 302
- mkAlistOfExplicitCategoryOps, 193
 - calledby augLisplibModemapsFromCategory, 192
 - calledby augmentLisplibModemapsFrom-Functor, 216
 - calledby mkAlistOfExplicitCategoryOps, 193
 - calls assocleft, 193
 - calls flattenSignatureList, 193
 - calls isCategoryForm, 193
 - calls keyedSystemError, 193
 - calls mkAlistOfExplicitCategoryOps, 193
 - calls nreverse0, 193
 - calls remdup, 193
 - calls union, 193
 - local ref \$e, 193
 - defun, 193
- mkAutoLoad
 - calledby unloadOneConstructor, 205
- mkCategoryPackage, 180
 - calledby compDefineCategory1, 162
 - calls JoinInner, 180
 - calls abbreviationsSpad2Cmd, 180
 - calls assoc, 180
 - calls getdatabase, 180
 - calls pname, 180
 - calls strconc, 180
 - calls sublis, 180
 - uses \$FormalMapVariableList, 180
 - uses \$categoryPredicateList, 180
 - uses \$e, 180
 - uses \$options, 180
 - defun, 180
- mkConstructor, 205
 - calledby compDefineCategory2, 164
 - calledby mkConstructor, 205
 - calls mkConstructor, 205
 - defun, 205
- mkDatabasePred, 218
 - calledby augmentLisplibModemapsFrom-Functor, 216
 - calls isCategoryForm, 218
 - local ref \$e, 218
 - defun, 218
- mkDomainConstructor
 - calledby bootStrapError, 219
 - calledby compHasFormat, 313
 - calledby genDomainOps, 225
 - calledby genDomainView, 224
- mkErrorExpr
 - calledby compOrCroak1, 586
- mkEvalableCategoryForm, 182
 - calledby compMakeCategoryObject, 213
 - calledby mkEvalableCategoryForm, 182
 - calls compOrCroak, 182
 - calls getdatabase, 182
 - calls get, 182
 - calls mkEvalableCategoryForm, 182
 - calls mkq, 182
 - local def \$e, 182
 - local ref \$CategoryFrame, 182
 - local ref \$CategoryNames, 182
 - local ref \$Category, 182
 - local ref \$EmptyMode, 182
 - local ref \$e, 182
 - defun, 182
- mkExplicitCategoryFunction, 292
 - calledby compCategory, 290
 - calls identp, 292
 - calls mkq, 292
 - calls mustInstantiate, 292
 - calls remdup, 292

- calls union, 292
- calls wrapDomainSub, 292
- defun, 292
- mkList, 314
 - calledby compHasFormat, 313
 - defun, 314
- mkNewModemapList, 266
 - calledby addModemap1, 274
 - calls assoc, 266
 - calls insertModemap, 266
 - calls member, 266
 - calls mergeModemap, 266
 - calls nreverse0, 266
 - local ref \$InteractiveMode, 266
 - local ref \$forceAdd, 266
 - defun, 266
- mkOpVec, 225
 - calls AssocBarGensym, 226
 - calls assoc, 225
 - calls assq, 225
 - calls getOperationAlistFromLisplib, 225
 - calls getPrincipalView, 225
 - calls length, 225
 - calls opOf, 225
 - calls sublis, 225
 - uses Undef, 226
 - uses \$FormalMapVariableList, 226
 - defun, 225
- mkpfn
 - calledby augLisplibModemapsFromCategory, 192
 - calledby augmentLisplibModemapsFromFunctor, 216
 - calledby compCapsuleInner, 278
 - calledby compCategoryItem, 291
 - calledby compileCases, 171
 - calledby getInverseEnvironment, 320
 - calledby stripOffSubdomainConditions, 305
- mkprogn
 - calledby setqMultiple, 341
- mkprompt[5]
 - called by get-a-line, 636
- mkq
 - calledby addArgumentConditions, 304
 - calledby bootStrapError, 219
 - calledby compCoerce1, 363
 - calledby compDefineCapsuleFunction, 156
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby compSeq1, 336
 - calledby genDomainOps, 225
 - calledby mkEvalableCategoryForm, 182
 - calledby mkExplicitCategoryFunction, 292
 - calledby optSpecialCall, 236
 - calledby spadCompileOrSetq, 186
- mkRecord, 249
 - defplist, 249
- mkRepfun
 - calledby mkRepititionAssoc, 184
- mkRepititionAssoc, 184
 - calledby encodeFunctionName, 183
 - calls mkRepfun, 184
 - defun, 184
- mkUnion, 366
 - calledby resolve, 365
 - calls union, 366
 - local ref \$Rep, 366
 - defun, 366
- moan
 - calledby compileTimeBindingOf, 237
 - calledby parseExit, 114
 - calledby parseReturn, 135
- modeEqual, 366
 - calledby autoCoerceByModemap, 364
 - calledby checkAndDeclare, 308
 - calledby coerceByModemap, 363
 - calledby coerceHard, 358
 - calledby compAtomWithModemap, 596
 - calledby compCase1, 288
 - calledby compile, 173
 - calledby domainMember, 264
 - calledby modeEqualSubst, 367
 - calledby resolve, 365
 - defun, 366
- modeEqualSubst, 367
 - calledby coerceEasy, 356
 - calledby modeEqualSubst, 367
 - calls length, 367
 - calls modeEqualSubst, 367
 - calls modeEqual, 367
 - defun, 367
- modeIsAggregateOf

- calledby compAtom, 595
 - calledby compConstruct, 299
 - calledby compRepeatOrCollect, 333
- modemapPattern, 203
 - calledby interactiveModemapForm, 195
 - calls rassoc, 203
 - local ref \$PatternVariableList, 203
 - defun, 203
- modifyModeStack, 622
 - calledby compExit, 312
 - calledby compLeave, 327
 - calledby compReturn, 335
 - calls copy, 622
 - calls resolve, 622
 - calls say, 622
 - calls setelt, 622
 - uses \$exitModeStack, 622
 - uses \$reportExitModeStack, 622
 - defun, 622
- moveORsOutside, 201
 - calledby fixUpPredicate, 196
 - calledby moveORsOutside, 201
 - calls moveORsOutside, 201
 - defun, 201
- msort
 - calledby extendLocalLibdb, 475
- msubst
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
- must, 463
 - calledby PARSE-Category, 421
 - calledby PARSE-Command, 415
 - calledby PARSE-Conditional, 448
 - calledby PARSE-Enclosure, 435
 - calledby PARSE-Exit, 447
 - calledby PARSE-FloatBasePart, 434
 - calledby PARSE-FloatBase, 433
 - calledby PARSE-FloatExponent, 434
 - calledby PARSE-Float, 432
 - calledby PARSE-Form, 429
 - calledby PARSE-Import, 422
 - calledby PARSE-Infix, 426
 - calledby PARSE-Iterator, 444
 - calledby PARSE-LabelExpr, 450
 - calledby PARSE-Label, 430
 - calledby PARSE-Leave, 447
 - calledby PARSE-Loop, 449
 - calledby PARSE-NewExpr, 415
 - calledby PARSE-Option, 419
 - calledby PARSE-Prefix, 426
 - calledby PARSE-Primary1, 432
 - calledby PARSE-Qualification, 427
 - calledby PARSE-Reduction, 428
 - calledby PARSE-Return, 446
 - calledby PARSE-ScriptItem, 438
 - calledby PARSE-Scripts, 437
 - calledby PARSE-Selector, 430
 - calledby PARSE-SemiColon, 446
 - calledby PARSE-Sequence, 442
 - calledby PARSE-Sexpr1, 439
 - calledby PARSE-SpecialCommand, 416
 - calledby PARSE-Statement, 419
 - calledby PARSE-TokenOption, 417
 - calledby PARSE-With, 420
 - calls meta-syntax-error, 463
 - defmacro, 463
- mustInstantiate, 293
 - calledby mkExplicitCategoryFunction, 292
 - calls getl, 293
 - local ref \$DummyFunctorNames, 293
 - defun, 293
- namestring
 - calledby bootStrapError, 219
 - calledby finalizeLisplib, 207
- namestring[5]
 - called by compileSpad2Cmd, 563
 - called by compileSpadLispCmd, 566
 - called by compiler, 561
- ncINTERPFILE, 624
 - calledby /rf-1, 570
 - calls SpadInterpretStream[5], 624
 - uses \$EchoLines, 624
 - uses \$ReadingFile, 624
 - defun, 624
- ncParseFromString
 - calledby checkGetParse, 521
- new2OldDefForm, 80
 - calledby new2OldDefForm, 80
 - calledby newDef2Def, 80
 - calls new2OldDefForm, 80
 - calls new2OldTran, 80

- defun, 80
- new2OldLisp, 78
 - calledby s-process, 581
 - calledby string2BootTree, 77
 - calls new2OldTran, 78
 - calls postTransform, 78
 - defun, 78
- new2OldTran, 78
 - calledby new2OldDefForm, 80
 - calledby new2OldLisp, 78
 - calledby new2OldTran, 78
 - calledby newDef2Def, 80
 - calledby newIf2Cond, 79
 - calls dcq, 78
 - calls new2OldTran, 78
 - calls newConstruct, 78
 - calls newDef2Def, 78
 - calls newIf2Cond, 78
 - local ref \$new2OldRenameAssoc, 78
 - defun, 78
- newConstruct, 80
 - calledby new2OldTran, 78
 - defun, 80
- newDef2Def, 80
 - calledby new2OldTran, 78
 - calls letError, 80
 - calls new2OldDefForm, 80
 - calls new2OldTran, 80
 - defun, 80
- newIf2Cond, 79
 - calledby new2OldTran, 78
 - calls letError, 79
 - calls new2OldTran, 79
 - defun, 79
- newString2Words, 525
 - calledby checkComments, 496
 - calledby checkRewrite, 497
 - calls newWordFrom, 525
 - calls nreverse0, 525
 - defun, 525
- newWordFrom, 538
 - calledby newString2Words, 525
 - local ref \$charBlank, 538
 - local ref \$charFauxNewline, 538
 - local ref \$stringFauxNewline, 538
 - defun, 538
- next-char, 460
 - calledby PARSE-FloatBase, 433
 - calls line-at-end-p, 461
 - calls line-next-char, 461
 - uses current-line, 461
 - defun, 460
- next-line, 634
 - calledby Advance-Char, 635
 - local ref \$in-stream, 634
 - local ref \$line-handler, 634
 - defun, 634
- next-tab-loc, 553
 - calledby indent-pos, 552
 - defun, 553
- next-token, 98, 459
 - calledby match-next-token, 457
 - calls current-token, 459
 - calls try-get-token, 459
 - usedby next-token, 459
 - uses \$token, 98
 - uses next-token, 459
 - uses valid-tokens, 459
 - defun, 459
 - defvar, 98
- nonblank, 97
 - defvar, 97
- nonblankloc, 553
 - calledby expand-tabs, 94
 - calls blankp, 553
 - defun, 553
- normalizeStatAndStringify
 - calledby reportOnFunctorCompilation, 219
- not, 132
 - defplist, 132
- notequal, 133
 - defplist, 133
- nreverse
 - calledby checkAddMacros, 527
 - calledby checkBalance, 499
 - calledby checkIeEg, 522
 - calledby transDoc, 494
- nreverse0
 - calledby aplTran1, 405
 - calledby compAdd, 275
 - calledby compCase1, 288
 - calledby compColon, 294

- calledby compExpressionList, 607
- calledby compForm1, 600
- calledby compForm2, 607
- calledby compJoin, 323
- calledby compReduce1, 330
- calledby compSeq1, 336
- calledby getFormModemaps, 604
- calledby getModemapList, 264
- calledby hasAplExtension, 407
- calledby mkAListOfExplicitCategoryOps, 193
- calledby mkNewModemapList, 266
- calledby newString2Words, 525
- calledby outputComp, 347
- calledby parseHas, 116
- calledby postCategory, 381
- calledby postDef, 389
- calledby postIf, 391
- calledby postMDef, 395
- calledby setqMultiple, 341
- calledby substNames, 270
- calledby transIs1, 110
- NRTaddInner
 - calledby NRTgetLocalIndex, 215
- NRTassignCapsuleFunctionSlot
 - calledby compDefineCapsuleFunction, 156
- NRTassocIndex, 346
 - calledby NRTgetLocalIndex, 215
 - calledby NRTputInHead, 190
 - calledby NRTputInTail, 189
 - calledby setqSingle, 344
 - local ref \$NRTaddForm, 346
 - local ref \$NRTbase, 346
 - local ref \$NRTdeltaLength, 346
 - local ref \$NRTdeltaList, 346
 - local ref \$found, 346
 - defun, 346
- NRTextendsCategory1
 - calledby NRTgetLookupFunction, 214
- NRTgenInitialAttributeAlist
 - calledby compDefineFunctor1, 148
 - calledby finalizeLisplib, 207
- NRTgetLocalIndex, 215
 - calledby compAdd, 275
 - calledby compDefineFunctor1, 148
 - calledby compSymbol, 598
 - calledby doIt, 281
 - calls NRTaddInner, 215
 - calls NRTassocIndex, 215
 - calls compOrCroak, 215
 - calls rplaca, 215
 - local def \$EmptyMode, 215
 - local def \$NRTbase, 215
 - local def \$e, 215
 - local ref \$NRTaddForm, 215
 - local ref \$NRTdeltaLength, 215
 - local ref \$NRTdeltaListComp, 215
 - local ref \$NRTdeltaList, 215
 - local ref \$formalArgList, 215
 - defun, 215
- NRTgetLookupFunction, 214
 - calledby compDefineFunctor1, 149
 - calls NRTextendsCategory1, 214
 - calls bright, 214
 - calls form2String, 214
 - calls getExportCategory, 214
 - calls sayBrightlyNT, 214
 - calls sayBrightly, 214
 - calls sublis, 214
 - local def \$why, 214
 - local ref \$pairlis, 214
 - local ref \$why, 214
 - defun, 214
- NRTmakeSlot1Info
 - calledby compDefineFunctor1, 149
- NRTputInHead, 190
 - calledby NRTputInHead, 190
 - calledby NRTputInTail, 189
 - calls NRTassocIndex, 190
 - calls NRTputInHead, 190
 - calls NRTputInTail, 190
 - calls keyedSystemError, 190
 - calls lastnode, 190
 - local ref \$elt, 190
 - defun, 190
- NRTputInTail, 189
 - calledby NRTputInHead, 190
 - calledby putInLocalDomainReferences, 189
 - calls NRTassocIndex, 189
 - calls NRTputInHead, 189
 - calls lassoc, 189
 - calls rplaca, 189

- local ref \$devaluateList, 189
 - local ref \$elt, 189
 - defun, 189
- nsbst
 - calledby substVars, 202
- nth-stack, 549
 - calledby PARSE-Category, 421
 - calledby PARSE-Sexpr1, 440
 - calls reduction-value, 549
 - calls stack-store, 549
 - defmacro, 549
- obey
 - calledby buildLibdb, 476
- object2String
 - calledby compileSpad2Cmd, 563
 - calledby compileSpadLispCmd, 566
- opFf
 - calledby parseDEF, 109
- opOf
 - calledby augModemapsFromDomain, 256
 - calledby checkNumOfArgs, 532
 - calledby checkRecordHash, 507
 - calledby coerceSubset, 357
 - calledby comp2, 589
 - calledby compColonInside, 594
 - calledby compDefineCategory2, 164
 - calledby compElt, 310
 - calledby compPretend, 329
 - calledby compilerDoit, 568
 - calledby doIt, 281
 - calledby getSignatureFromMode, 303
 - calledby isFunctor, 253
 - calledby isSuperDomain, 255
 - calledby mkOpVec, 225
 - calledby optCallSpecially, 235
 - calledby parseHas, 116
 - calledby parseLET, 130
 - calledby parseMDEF, 131
 - calledby recordAttributeDocumentation, 468
- opt-, 242
 - defun, 242
- optCall, 233
 - calledby optSPADCALL, 243
 - calls optCallSpecially, 234
 - calls optPackageCall, 234
 - calls optimize, 233
 - calls rplac, 233
 - calls systemErrorHere, 234
 - local ref \$QuickCode, 234
 - local ref \$bootStrapMode, 234
 - defun, 233
- optCallEval, 237
 - calledby optSpecialCall, 236
 - calls FactoredForm, 238
 - calls Integer, 237
 - calls List, 237
 - calls Matrix, 238
 - calls PrimititveArray, 238
 - calls Vector, 238
 - calls eval, 238
 - defun, 237
- optCallSpecially, 235
 - calledby optCall, 234
 - calls get, 235
 - calls lassoc, 235
 - calls opOf, 235
 - calls optSpecialCall, 235
 - local ref \$e, 235
 - local ref \$getDomainCode, 235
 - local ref \$optimizableConstructorNames, 235
 - local ref \$specialCaseKeyList, 235
 - defun, 235
- optCatch, 245
 - calls optimize, 245
 - calls rplac, 245
 - local ref \$InteractiveMode, 245
 - defun, 245
- optCond, 246
 - calls EqualBarGensym, 247
 - calls TruthP, 246
 - calls rplacd, 246
 - calls rplac, 247
 - defun, 246
- optCONDtail, 231
 - calledby optCONDtail, 231
 - calledby optXLAMCond, 230
 - calls optCONDtail, 231
 - local ref \$true, 231
 - defun, 231

- optEQ, 240
 - defun, 240
- optFunctorBody
 - calledby compDefineCategory2, 164
- optIF2COND, 232
 - calledby optIF2COND, 232
 - calledby optimize, 229
 - calls optIF2COND, 232
 - local ref \$true, 232
 - defun, 232
- optimize, 229
 - calledby lispize, 351
 - calledby optCall, 233
 - calledby optCatch, 245
 - calledby optSpecialCall, 236
 - calledby optimizeFunctionDef, 228
 - calledby optimize, 229
 - calls getl, 229
 - calls optIF2COND, 229
 - calls optimize, 229
 - calls prettyprint, 229
 - calls rplac, 229
 - calls say, 229
 - calls subname, 229
 - defun, 229
- optimizeFunctionDef, 228
 - calledby compWithMappingModel, 615
 - calledby compile, 173
 - calls bright, 228
 - calls optimize, 228
 - calls pp, 228
 - calls rplac, 228
 - calls sayBrightlyI, 228
 - local ref \$reportOptimization, 228
 - defun, 228
- optional, 464
 - calledby PARSE-Application, 429
 - calledby PARSE-Category, 421
 - calledby PARSE-CommandTail, 418
 - calledby PARSE-Conditional, 448
 - calledby PARSE-Expr, 423
 - calledby PARSE-Form, 429
 - calledby PARSE-Import, 422
 - calledby PARSE-Infix, 426
 - calledby PARSE-IteratorTail, 444
 - calledby PARSE-Iterator, 444
 - calledby PARSE-Prefix, 426
 - calledby PARSE-Primary1, 432
 - calledby PARSE-PrimaryNoFloat, 431
 - calledby PARSE-ScriptItem, 438
 - calledby PARSE-Seg, 448
 - calledby PARSE-Sequence1, 442
 - calledby PARSE-Sexpr1, 439
 - calledby PARSE-SpecialCommand, 416
 - calledby PARSE-Statement, 419
 - calledby PARSE-Suffix, 445
 - calledby PARSE-TokenCommandTail, 417
 - calledby PARSE-VarForm, 437
 - defun, 464
- optionlist
 - usedby spad, 572
- optLESSP, 242
 - defun, 242
- optMINUS, 240
 - defun, 240
- optMkRecord, 249
 - calls length, 249
 - defun, 249
- optPackageCall, 235
 - calledby optCall, 234
 - calls rplaca, 235
 - calls rplacd, 235
 - defun, 235
- optPredicateIfTrue, 231
 - calledby optXLAMCond, 230
 - local ref \$BasicPredicates, 231
 - defun, 231
- optQSMINUS, 241
 - defun, 241
- optRECORDCOPY, 251
 - defun, 251
- optRECORDELT, 250
 - calls keyedSystemError, 250
 - defun, 250
- optSEQ, 238
 - defun, 238
- optSETRECORDELT, 251
 - calls keyedSystemError, 251
 - defun, 251
- optSPADCALL, 243
 - calls optCall, 243
 - local ref \$InteractiveMode, 243

- defun, 243
- optSpecialCall, 236
 - calledby optCallSpecially, 235
 - calls compileTimeBindingOf, 236
 - calls function, 236
 - calls getl, 236
 - calls keyedSystemError, 236
 - calls mkq, 236
 - calls optCallEval, 236
 - calls optimize, 236
 - calls rplaca, 236
 - calls rplacw, 236
 - calls rplac, 236
 - local ref \$QuickCode, 236
 - local ref \$Undef, 236
 - defun, 236
- optSuchthat, 244
 - defun, 244
- optXLAMCond, 230
 - calledby optXLAMCond, 230
 - calls optCONDtail, 230
 - calls optPredicateIfTrue, 230
 - calls optXLAMCond, 230
 - calls rplac, 230
 - defun, 230
- or, 133
 - defplist, 133
- orderByDependency, 227
 - calledby compDefWhereClause, 160
 - calls intersection, 227
 - calls member, 227
 - calls remdup, 227
 - calls say, 227
 - calls userError, 227
 - defun, 227
- orderPredicateItems, 197
 - calledby fixUpPredicate, 196
 - calls orderPredTran, 197
 - calls signatureTran, 197
 - defun, 197
- orderPredTran, 198
 - calledby orderPredicateItems, 197
 - calls delete, 198
 - calls insertWOC, 198
 - calls intersectionq, 198
 - calls isDomainSubst, 198
 - calls listOfPatternIds, 198
 - calls member, 198
 - calls setdifference, 198
 - calls unionq, 198
 - defun, 198
- outerProduct
 - calledby compileCases, 171
- outputComp, 347
 - calledby compForm1, 600
 - calledby outputComp, 347
 - calledby setqSingle, 344
 - calls comp, 347
 - calls get, 347
 - calls nreverse0, 347
 - calls outputComp, 347
 - local ref \$Expression, 347
 - defun, 347
- pack
 - calledby quote-if-string, 454
- Pair
 - calledby compApply, 592
- pairList
 - calledby compDefWhereClause, 160
 - calledby formal2Pattern, 218
- PARSE-AnyId, 441
 - calledby PARSE-Sexpr1, 439
 - calls action, 441
 - calls advance-token, 441
 - calls current-symbol, 441
 - calls match-string, 441
 - calls parse-identifier, 441
 - calls parse-keyword, 441
 - calls push-reduction, 441
 - defun, 441
- PARSE-Application, 429
 - calledby PARSE-Application, 429
 - calledby PARSE-Category, 421
 - calledby PARSE-Form, 429
 - calls PARSE-Application, 429
 - calls PARSE-Primary, 429
 - calls PARSE-Selector, 429
 - calls optional, 429
 - calls pop-stack-1, 429
 - calls pop-stack-2, 429
 - calls push-reduction, 429

- calls star, [429](#)
- defun, [429](#)
- parse-argument-designator, [546](#)
 - calledby PARSE-FormalParameterTok, [436](#)
 - calls advance-token, [546](#)
 - calls match-current-token, [546](#)
 - calls push-reduction, [546](#)
 - calls token-symbol, [546](#)
 - defun, [546](#)
- PARSE-Category, [421](#)
 - calledby PARSE-Category, [421](#)
 - calls PARSE-Application, [421](#)
 - calls PARSE-Category, [421](#)
 - calls PARSE-Expression, [421](#)
 - calls action, [421](#)
 - calls bang, [421](#)
 - calls line-number, [421](#)
 - calls match-advance-string, [421](#)
 - calls must, [421](#)
 - calls nth-stack, [421](#)
 - calls optional, [421](#)
 - calls pop-stack-1, [421](#)
 - calls pop-stack-2, [421](#)
 - calls pop-stack-3, [421](#)
 - calls push-reduction, [421](#)
 - calls recordAttributeDocumentation, [421](#)
 - calls recordSignatureDocumentation, [421](#)
 - calls star, [421](#)
 - uses current-line, [421](#)
 - defun, [421](#)
- PARSE-Command, [415](#)
 - calls PARSE-SpecialCommand, [415](#)
 - calls PARSE-SpecialKeyWord, [415](#)
 - calls match-advance-string, [415](#)
 - calls must, [415](#)
 - calls push-reduction, [415](#)
 - defun, [415](#)
- PARSE-CommandTail, [418](#)
 - calledby PARSE-CommandTail, [418](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calls PARSE-CommandTail, [418](#)
 - calls PARSE-Option, [418](#)
 - calls action, [418](#)
 - calls bang, [418](#)
 - calls optional, [418](#)
 - calls pop-stack-1, [418](#)
 - calls pop-stack-2, [418](#)
 - calls push-reduction, [418](#)
 - calls star, [418](#)
 - defun, [418](#)
- PARSE-Conditional, [448](#)
 - calledby PARSE-ElseClause, [449](#)
 - calls PARSE-ElseClause, [448](#)
 - calls PARSE-Expression, [448](#)
 - calls bang, [448](#)
 - calls match-advance-string, [448](#)
 - calls must, [448](#)
 - calls optional, [448](#)
 - calls pop-stack-1, [448](#)
 - calls pop-stack-2, [448](#)
 - calls pop-stack-3, [448](#)
 - calls push-reduction, [448](#)
 - defun, [448](#)
- PARSE-Data, [439](#)
 - calledby PARSE-Primary1, [432](#)
 - calls PARSE-Sexpr, [439](#)
 - calls action, [439](#)
 - calls pop-stack-1, [439](#)
 - calls push-reduction, [439](#)
 - calls translabel, [439](#)
 - uses labasoc, [439](#)
 - defun, [439](#)
- PARSE-ElseClause, [449](#)
 - calledby PARSE-Conditional, [448](#)
 - calls PARSE-Conditional, [449](#)
 - calls PARSE-Expression, [449](#)
 - calls current-symbol, [449](#)
 - defun, [449](#)
- PARSE-Enclosure, [435](#)
 - calledby PARSE-Primary1, [432](#)
 - calls PARSE-Expr, [435](#)
 - calls match-advance-string, [435](#)
 - calls must, [435](#)
 - calls pop-stack-1, [435](#)
 - calls push-reduction, [435](#)
 - defun, [435](#)
- PARSE-Exit, [447](#)
 - calls PARSE-Expression, [447](#)
 - calls match-advance-string, [447](#)
 - calls must, [447](#)
 - calls pop-stack-1, [447](#)

- calls push-reduction, [447](#)
- defun, [447](#)
- PARSE-Expr, [423](#)
 - calledby PARSE-Enclosure, [435](#)
 - calledby PARSE-Expression, [422](#)
 - calledby PARSE-Import, [422](#)
 - calledby PARSE-Iterator, [444](#)
 - calledby PARSE-LabelExpr, [450](#)
 - calledby PARSE-Loop, [449](#)
 - calledby PARSE-Primary1, [432](#)
 - calledby PARSE-Reduction, [428](#)
 - calledby PARSE-ScriptItem, [438](#)
 - calledby PARSE-SemiColon, [446](#)
 - calledby PARSE-Statement, [419](#)
 - calls PARSE-LedPart, [423](#)
 - calls PARSE-NudPart, [423](#)
 - calls optional, [423](#)
 - calls pop-stack-1, [423](#)
 - calls push-reduction, [423](#)
 - calls star, [423](#)
 - defun, [423](#)
- PARSE-Expression, [422](#)
 - calledby PARSE-Category, [421](#)
 - calledby PARSE-Conditional, [448](#)
 - calledby PARSE-ElseClause, [449](#)
 - calledby PARSE-Exit, [447](#)
 - calledby PARSE-Infix, [426](#)
 - calledby PARSE-Iterator, [444](#)
 - calledby PARSE-Leave, [447](#)
 - calledby PARSE-Prefix, [426](#)
 - calledby PARSE-Return, [446](#)
 - calledby PARSE-Seg, [448](#)
 - calledby PARSE-Sequence1, [442](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calls PARSE-Expr, [422](#)
 - calls PARSE-rightBindingPowerOf, [422](#)
 - calls make-symbol-of, [422](#)
 - calls pop-stack-1, [422](#)
 - calls push-reduction, [422](#)
 - uses ParseMode, [422](#)
 - uses prior-token, [422](#)
 - defun, [422](#)
- PARSE-Float, [432](#)
 - calledby PARSE-Primary, [431](#)
 - calledby PARSE-Selector, [430](#)
 - calls PARSE-FloatBase, [432](#)
 - calls PARSE-FloatExponent, [432](#)
 - calls make-float, [433](#)
 - calls must, [432](#)
 - calls pop-stack-1, [433](#)
 - calls pop-stack-2, [433](#)
 - calls pop-stack-3, [433](#)
 - calls pop-stack-4, [433](#)
 - calls push-reduction, [433](#)
 - defun, [432](#)
- PARSE-FloatBase, [433](#)
 - calledby PARSE-Float, [432](#)
 - calls PARSE-FloatBasePart, [433](#)
 - calls PARSE-IntegerTok, [433](#)
 - calls char-eq, [433](#)
 - calls char-ne, [433](#)
 - calls current-char, [433](#)
 - calls current-symbol, [433](#)
 - calls digitp[5], [433](#)
 - calls must, [433](#)
 - calls next-char, [433](#)
 - calls push-reduction, [433](#)
 - defun, [433](#)
- PARSE-FloatBasePart, [434](#)
 - calledby PARSE-FloatBase, [433](#)
 - calls PARSE-IntegerTok, [434](#)
 - calls current-char, [434](#)
 - calls current-token, [434](#)
 - calls digitp[5], [434](#)
 - calls match-advance-string, [434](#)
 - calls must, [434](#)
 - calls push-reduction, [434](#)
 - calls token-nonblank, [434](#)
 - defun, [434](#)
- PARSE-FloatExponent, [434](#)
 - calledby PARSE-Float, [432](#)
 - calls PARSE-IntegerTok, [434](#)
 - calls action, [434](#)
 - calls advance-token, [434](#)
 - calls current-char, [434](#)
 - calls current-symbol, [434](#)
 - calls floatexpid, [434](#)
 - calls identp[5], [434](#)
 - calls match-advance-string, [434](#)
 - calls must, [434](#)
 - calls push-reduction, [434](#)
 - defun, [434](#)

- PARSE-FloatTok, 450
 - calls bfp-, 450
 - calls parse-number, 450
 - calls pop-stack-1, 450
 - calls push-reduction, 450
 - local ref \$boot, 450
 - defun, 450
- PARSE-Form, 429
 - calledby PARSE-NudPart, 424
 - calls PARSE-Application, 429
 - calls bang, 429
 - calls match-advance-string, 429
 - calls must, 429
 - calls optional, 429
 - calls pop-stack-1, 429
 - calls push-reduction, 429
 - defun, 429
- PARSE-FormalParameter, 436
 - calledby PARSE-Primary1, 432
 - calls PARSE-FormalParameterTok, 436
 - defun, 436
- PARSE-FormalParameterTok, 436
 - calledby PARSE-FormalParameter, 436
 - calls parse-argument-designator, 436
 - defun, 436
- PARSE-getSemanticForm, 425
 - calledby PARSE-Operation, 424
 - calls PARSE-Infix, 425
 - calls PARSE-Prefix, 425
 - defun, 425
- PARSE-GlyphTok, 441
 - calledby PARSE-Quad, 436
 - calledby PARSE-Seg, 448
 - calledby PARSE-Sexpr1, 440
 - calls action, 441
 - calls advance-token, 441
 - calls match-current-token, 441
 - uses tok, 441
 - defun, 441
- parse-identifier, 545
 - calledby PARSE-AnyId, 441
 - calledby PARSE-Name, 438
 - calls advance-token, 545
 - calls match-current-token, 545
 - calls push-reduction, 545
 - calls token-symbol, 545
 - defun, 545
- PARSE-Import, 422
 - calls PARSE-Expr, 422
 - calls bang, 422
 - calls match-advance-string, 422
 - calls must, 422
 - calls optional, 422
 - calls pop-stack-1, 423
 - calls pop-stack-2, 422
 - calls push-reduction, 422
 - calls star, 422
 - defun, 422
- PARSE-Infix, 426
 - calledby PARSE-getSemanticForm, 425
 - calls PARSE-Expression, 426
 - calls PARSE-TokTail, 426
 - calls action, 426
 - calls advance-token, 426
 - calls current-symbol, 426
 - calls must, 426
 - calls optional, 426
 - calls pop-stack-1, 427
 - calls pop-stack-2, 426
 - calls push-reduction, 426
 - defun, 426
- PARSE-InfixWith, 420
 - calls PARSE-With, 420
 - calls pop-stack-1, 420
 - calls pop-stack-2, 420
 - calls push-reduction, 420
 - defun, 420
- PARSE-IntegerTok, 436
 - calledby PARSE-FloatBasePart, 434
 - calledby PARSE-FloatBase, 433
 - calledby PARSE-FloatExponent, 434
 - calledby PARSE-Primary1, 432
 - calledby PARSE-Sexpr1, 440
 - calls parse-number, 436
 - defun, 436
- PARSE-Iterator, 444
 - calledby PARSE-IteratorTail, 444
 - calledby PARSE-Loop, 449
 - calls PARSE-Expression, 444
 - calls PARSE-Expr, 444
 - calls PARSE-Primary, 444
 - calls match-advance-string, 444

- calls must, [444](#)
- calls optional, [444](#)
- calls pop-stack-1, [444](#)
- calls pop-stack-2, [444](#)
- calls pop-stack-3, [444](#)
- defun, [444](#)
- PARSE-IteratorTail, [444](#)
 - calledby PARSE-Sequence1, [442](#)
 - calls PARSE-Iterator, [444](#)
 - calls bang, [444](#)
 - calls match-advance-string, [444](#)
 - calls optional, [444](#)
 - calls star, [444](#)
 - defun, [444](#)
- parse-keyword, [546](#)
 - calledby PARSE-AnyId, [441](#)
 - calls advance-token, [546](#)
 - calls match-current-token, [546](#)
 - calls push-reduction, [546](#)
 - calls token-symbol, [546](#)
 - defun, [546](#)
- PARSE-Label, [430](#)
 - calledby PARSE-LabelExpr, [450](#)
 - calledby PARSE-Leave, [447](#)
 - calls PARSE-Name, [430](#)
 - calls match-advance-string, [430](#)
 - calls must, [430](#)
 - defun, [430](#)
- PARSE-LabelExpr, [450](#)
 - calls PARSE-Expr, [450](#)
 - calls PARSE-Label, [450](#)
 - calls must, [450](#)
 - calls pop-stack-1, [450](#)
 - calls pop-stack-2, [450](#)
 - calls push-reduction, [450](#)
 - defun, [450](#)
- PARSE-Leave, [447](#)
 - calls PARSE-Expression, [447](#)
 - calls PARSE-Label, [447](#)
 - calls match-advance-string, [447](#)
 - calls must, [447](#)
 - calls pop-stack-1, [447](#)
 - calls push-reduction, [447](#)
 - defun, [447](#)
- PARSE-LedPart, [423](#)
 - calledby PARSE-Expr, [423](#)
 - calls PARSE-Operation, [423](#)
 - calls pop-stack-1, [423](#)
 - calls push-reduction, [423](#)
 - defun, [423](#)
- PARSE-leftBindingPowerOf, [425](#)
 - calledby PARSE-Operation, [424](#)
 - calls elemn, [425](#)
 - calls getl, [425](#)
 - defun, [425](#)
- PARSE-Loop, [449](#)
 - calls PARSE-Expr, [449](#)
 - calls PARSE-Iterator, [449](#)
 - calls match-advance-string, [449](#)
 - calls must, [449](#)
 - calls pop-stack-1, [449](#)
 - calls pop-stack-2, [449](#)
 - calls push-reduction, [449](#)
 - calls star, [449](#)
 - defun, [449](#)
- PARSE-Name, [438](#)
 - calledby PARSE-Label, [430](#)
 - calledby PARSE-VarForm, [437](#)
 - calls parse-identifier, [438](#)
 - calls pop-stack-1, [438](#)
 - calls push-reduction, [438](#)
 - defun, [438](#)
- PARSE-NBGliphTok, [441](#)
 - calledby PARSE-Sexpr1, [439](#)
 - calls action, [441](#)
 - calls advance-token, [441](#)
 - calls match-current-token, [441](#)
 - uses tok, [441](#)
 - defun, [441](#)
- PARSE-NewExpr, [414](#)
 - calledby spad, [572](#)
 - calls PARSE-Statement, [415](#)
 - calls action, [415](#)
 - calls current-symbol, [415](#)
 - calls match-string, [414](#)
 - calls must, [415](#)
 - calls processSynonyms[5], [415](#)
 - uses definition-name, [415](#)
 - defun, [414](#)
- PARSE-NudPart, [424](#)
 - calledby PARSE-Expr, [423](#)
 - calls PARSE-Form, [424](#)

- calls PARSE-Operation, 424
- calls PARSE-Reduction, 424
- calls pop-stack-1, 424
- calls push-reduction, 424
- uses rbp, 424
- defun, 424
- parse-number, 545
 - calledby PARSE-FloatTok, 450
 - calledby PARSE-IntegerTok, 436
 - calls advance-token, 545
 - calls match-current-token, 545
 - calls push-reduction, 545
 - calls token-symbol, 545
 - defun, 545
- PARSE-OpenBrace, 443
 - calledby PARSE-Sequence, 442
 - calls action, 443
 - calls advance-token, 443
 - calls current-symbol, 443
 - calls eqcar, 443
 - calls getToken, 443
 - calls push-reduction, 443
 - defun, 443
- PARSE-OpenBracket, 443
 - calledby PARSE-Sequence, 442
 - calls action, 443
 - calls advance-token, 443
 - calls current-symbol, 443
 - calls eqcar, 443
 - calls getToken, 443
 - calls push-reduction, 443
 - defun, 443
- PARSE-Operation, 424
 - calledby PARSE-LedPart, 423
 - calledby PARSE-NudPart, 424
 - calls PARSE-getSemanticForm, 424
 - calls PARSE-leftBindingPowerOf, 424
 - calls PARSE-rightBindingPowerOf, 424
 - calls action, 424
 - calls current-symbol, 424
 - calls elemn, 424
 - calls getl, 424
 - calls lt, 424
 - calls match-current-token, 424
 - uses ParseMode, 424
 - uses rbp, 424
 - uses tmptok, 424
 - defun, 424
- PARSE-Option, 419
 - calledby PARSE-CommandTail, 418
 - calls PARSE-PrimaryOrQM, 419
 - calls match-advance-string, 419
 - calls must, 419
 - calls star, 419
 - defun, 419
- PARSE-Prefix, 426
 - calledby PARSE-getSemanticForm, 425
 - calls PARSE-Expression, 426
 - calls PARSE-TokTail, 426
 - calls action, 426
 - calls advance-token, 426
 - calls current-symbol, 426
 - calls must, 426
 - calls optional, 426
 - calls pop-stack-1, 426
 - calls pop-stack-2, 426
 - calls push-reduction, 426
 - defun, 426
- PARSE-Primary, 431
 - calledby PARSE-Application, 429
 - calledby PARSE-Iterator, 444
 - calledby PARSE-PrimaryOrQM, 419
 - calledby PARSE-Selector, 430
 - calls PARSE-Float, 431
 - calls PARSE-PrimaryNoFloat, 431
 - defun, 431
- PARSE-Primary1, 431
 - calledby PARSE-Primary1, 432
 - calledby PARSE-PrimaryNoFloat, 431
 - calledby PARSE-Qualification, 427
 - calls PARSE-Data, 432
 - calls PARSE-Enclosure, 432
 - calls PARSE-Expr, 432
 - calls PARSE-FormalParameter, 432
 - calls PARSE-IntegerTok, 432
 - calls PARSE-Primary1, 432
 - calls PARSE-Quad, 432
 - calls PARSE-Sequence, 432
 - calls PARSE-String, 432
 - calls PARSE-VarForm, 431
 - calls current-symbol, 432
 - calls match-advance-string, 432

- calls match-string, [432](#)
- calls must, [432](#)
- calls optional, [432](#)
- calls pop-stack-1, [432](#)
- calls pop-stack-2, [432](#)
- calls push-reduction, [432](#)
- local ref \$boot, [432](#)
- defun, [431](#)
- PARSE-PrimaryNoFloat, [431](#)
 - calledby PARSE-Primary, [431](#)
 - calledby PARSE-Selector, [430](#)
 - calls PARSE-Primary1, [431](#)
 - calls PARSE-TokTail, [431](#)
 - calls optional, [431](#)
 - defun, [431](#)
- PARSE-PrimaryOrQM, [419](#)
 - calledby PARSE-Option, [419](#)
 - calledby PARSE-PrimaryOrQM, [419](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calls PARSE-PrimaryOrQM, [419](#)
 - calls PARSE-Primary, [419](#)
 - calls match-advance-string, [419](#)
 - calls push-reduction, [419](#)
 - defun, [419](#)
- PARSE-Quad, [436](#)
 - calledby PARSE-Primary1, [432](#)
 - calls PARSE-GlyphTok, [436](#)
 - calls match-advance-string, [436](#)
 - calls push-reduction, [436](#)
 - uses \$boot, [436](#)
 - defun, [436](#)
- PARSE-Qualification, [427](#)
 - calledby PARSE-TokTail, [427](#)
 - calls PARSE-Primary1, [427](#)
 - calls dollarTran, [427](#)
 - calls match-advance-string, [427](#)
 - calls must, [427](#)
 - calls pop-stack-1, [428](#)
 - calls push-reduction, [427](#)
 - defun, [427](#)
- PARSE-Reduction, [428](#)
 - calledby PARSE-NudPart, [424](#)
 - calls PARSE-Expr, [428](#)
 - calls PARSE-ReductionOp, [428](#)
 - calls must, [428](#)
 - calls pop-stack-1, [428](#)
 - calls pop-stack-2, [428](#)
 - calls push-reduction, [428](#)
 - defun, [428](#)
- PARSE-ReductionOp, [428](#)
 - calledby PARSE-Reduction, [428](#)
 - calls action, [428](#)
 - calls advance-token, [428](#)
 - calls current-symbol, [428](#)
 - calls getl, [428](#)
 - calls match-next-token, [428](#)
 - defun, [428](#)
- PARSE-Return, [446](#)
 - calls PARSE-Expression, [446](#)
 - calls match-advance-string, [446](#)
 - calls must, [446](#)
 - calls pop-stack-1, [446](#)
 - calls push-reduction, [446](#)
 - defun, [446](#)
- PARSE-rightBindingPowerOf, [425](#)
 - calledby PARSE-Expression, [422](#)
 - calledby PARSE-Operation, [424](#)
 - calls elemn, [425](#)
 - calls getl, [425](#)
 - defun, [425](#)
- PARSE-ScriptItem, [438](#)
 - calledby PARSE-ScriptItem, [438](#)
 - calledby PARSE-Scripts, [437](#)
 - calls PARSE-Expr, [438](#)
 - calls PARSE-ScriptItem, [438](#)
 - calls match-advance-string, [438](#)
 - calls must, [438](#)
 - calls optional, [438](#)
 - calls pop-stack-1, [438](#)
 - calls pop-stack-2, [438](#)
 - calls push-reduction, [438](#)
 - calls star, [438](#)
 - defun, [438](#)
- PARSE-Scripts, [437](#)
 - calledby PARSE-VarForm, [437](#)
 - calls PARSE-ScriptItem, [437](#)
 - calls match-advance-string, [437](#)
 - calls must, [437](#)
 - defun, [437](#)
- PARSE-Seg, [448](#)
 - calls PARSE-Expression, [448](#)
 - calls PARSE-GlyphTok, [448](#)

- calls bang, 448
- calls optional, 448
- calls pop-stack-1, 448
- calls pop-stack-2, 448
- calls push-reduction, 448
- defun, 448
- PARSE-Selector, 430
 - calledby PARSE-Application, 429
 - calls PARSE-Float, 430
 - calls PARSE-PrimaryNoFloat, 430
 - calls PARSE-Primary, 430
 - calls char-ne, 430
 - calls current-char, 430
 - calls current-symbol, 430
 - calls match-advance-string, 430
 - calls must, 430
 - calls pop-stack-1, 430
 - calls pop-stack-2, 430
 - calls push-reduction, 430
 - uses \$boot, 430
 - defun, 430
- PARSE-SemiColon, 446
 - calls PARSE-Expr, 446
 - calls match-advance-string, 446
 - calls must, 446
 - calls pop-stack-1, 446
 - calls pop-stack-2, 446
 - calls push-reduction, 446
 - defun, 446
- PARSE-Sequence, 442
 - calledby PARSE-Primary1, 432
 - calls PARSE-OpenBrace, 442
 - calls PARSE-OpenBracket, 442
 - calls PARSE-Sequence1, 442
 - calls match-advance-string, 442
 - calls must, 442
 - calls pop-stack-1, 442
 - calls push-reduction, 442
 - defun, 442
- PARSE-Sequence1, 442
 - calledby PARSE-Sequence, 442
 - calls PARSE-Expression, 442
 - calls PARSE-IteratorTail, 442
 - calls optional, 442
 - calls pop-stack-1, 442
 - calls pop-stack-2, 442
 - calls push-reduction, 442
 - defun, 442
- PARSE-Sexpr, 439
 - calledby PARSE-Data, 439
 - calls PARSE-Sexpr1, 439
 - defun, 439
- PARSE-Sexpr1, 439
 - calledby PARSE-Sexpr1, 440
 - calledby PARSE-Sexpr, 439
 - calls PARSE-AnyId, 439
 - calls PARSE-GlyphTok, 440
 - calls PARSE-IntegerTok, 440
 - calls PARSE-NBGlyphTok, 439
 - calls PARSE-Sexpr1, 440
 - calls PARSE-String, 440
 - calls action, 440
 - calls bang, 440
 - calls match-advance-string, 440
 - calls must, 439
 - calls nth-stack, 440
 - calls optional, 439
 - calls pop-stack-1, 440
 - calls pop-stack-2, 440
 - calls push-reduction, 440
 - calls star, 440
 - defun, 439
- parse-spadstring, 544
 - calledby PARSE-String, 437
 - calls advance-token, 544
 - calls match-current-token, 544
 - calls push-reduction, 544
 - calls token-symbol, 544
 - defun, 544
- PARSE-SpecialCommand, 416
 - calledby PARSE-Command, 415
 - calledby PARSE-SpecialCommand, 416
 - calls PARSE-CommandTail, 416
 - calls PARSE-Expression, 416
 - calls PARSE-PrimaryOrQM, 416
 - calls PARSE-SpecialCommand, 416
 - calls PARSE-TokenCommandTail, 416
 - calls PARSE-TokenList, 416
 - calls action, 416
 - calls bang, 416
 - calls current-symbol, 416
 - calls match-advance-string, 416

- calls must, [416](#)
- calls optional, [416](#)
- calls pop-stack-1, [416](#)
- calls push-reduction, [416](#)
- calls star, [416](#)
- local ref \$noParseCommands, [416](#)
- local ref \$tokenCommands, [416](#)
- defun, [416](#)
- PARSE-SpecialKeyword, [415](#)
 - calledby PARSE-Command, [415](#)
 - calls action, [415](#)
 - calls current-symbol, [415](#)
 - calls current-token, [415](#)
 - calls match-current-token, [415](#)
 - calls token-symbol, [415](#)
 - calls unAbbreviateKeyword[5], [415](#)
 - defun, [415](#)
- PARSE-Statement, [419](#)
 - calledby PARSE-NewExpr, [415](#)
 - calls PARSE-Expr, [419](#)
 - calls match-advance-string, [419](#)
 - calls must, [419](#)
 - calls optional, [419](#)
 - calls pop-stack-1, [419](#)
 - calls pop-stack-2, [419](#)
 - calls push-reduction, [419](#)
 - calls star, [419](#)
 - defun, [419](#)
- PARSE-String, [437](#)
 - calledby PARSE-Primary1, [432](#)
 - calledby PARSE-Sexpr1, [440](#)
 - calls parse-spadstring, [437](#)
 - defun, [437](#)
- parse-string, [544](#)
 - calls advance-token, [544](#)
 - calls match-current-token, [544](#)
 - calls push-reduction, [544](#)
 - calls token-symbol, [544](#)
 - defun, [544](#)
- PARSE-Suffix, [445](#)
 - calls PARSE-TokTail, [445](#)
 - calls action, [445](#)
 - calls advance-token, [445](#)
 - calls current-symbol, [445](#)
 - calls optional, [445](#)
 - calls pop-stack-1, [446](#)
 - calls push-reduction, [445](#)
 - defun, [445](#)
- PARSE-TokenCommandTail, [417](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calledby PARSE-TokenCommandTail, [417](#)
 - calls PARSE-TokenCommandTail, [417](#)
 - calls PARSE-TokenOption, [417](#)
 - calls action, [417](#)
 - calls atEndOfLine, [417](#)
 - calls bang, [417](#)
 - calls optional, [417](#)
 - calls pop-stack-1, [417](#)
 - calls pop-stack-2, [417](#)
 - calls push-reduction, [417](#)
 - calls star, [417](#)
 - calls systemCommand[5], [417](#)
 - defun, [417](#)
- PARSE-TokenList, [418](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calledby PARSE-TokenOption, [417](#)
 - calls action, [418](#)
 - calls advance-token, [418](#)
 - calls current-symbol, [418](#)
 - calls isTokenDelimiter, [418](#)
 - calls push-reduction, [418](#)
 - calls star, [418](#)
 - defun, [418](#)
- PARSE-TokenOption, [417](#)
 - calledby PARSE-TokenCommandTail, [417](#)
 - calls PARSE-TokenList, [417](#)
 - calls match-advance-string, [417](#)
 - calls must, [417](#)
 - defun, [417](#)
- PARSE-TokTail, [427](#)
 - calledby PARSE-Infix, [426](#)
 - calledby PARSE-Prefix, [426](#)
 - calledby PARSE-PrimaryNoFloat, [431](#)
 - calledby PARSE-Suffix, [445](#)
 - calls PARSE-Qualification, [427](#)
 - calls action, [427](#)
 - calls char-eq, [427](#)
 - calls copy-token, [427](#)
 - calls current-char, [427](#)
 - calls current-symbol, [427](#)
 - uses \$boot, [427](#)
 - defun, [427](#)

- PARSE-VarForm, [437](#)
 - calledby PARSE-Primary1, [431](#)
 - calls PARSE-Name, [437](#)
 - calls PARSE-Scripts, [437](#)
 - calls optional, [437](#)
 - calls pop-stack-1, [437](#)
 - calls pop-stack-2, [437](#)
 - calls push-reduction, [437](#)
 - defun, [437](#)
- PARSE-With, [420](#)
 - calledby PARSE-InfixWith, [420](#)
 - calls match-advance-string, [420](#)
 - calls must, [420](#)
 - calls pop-stack-1, [420](#)
 - calls push-reduction, [420](#)
 - defun, [420](#)
- parseAnd, [105](#)
 - calledby parseAnd, [105](#)
 - calls parseAnd, [105](#)
 - calls parseIf, [105](#)
 - calls parseTranList, [105](#)
 - calls parseTran, [105](#)
 - uses \$InteractiveMode, [105](#)
 - defun, [105](#)
- parseAtom, [102](#)
 - calledby parseTran, [101](#)
 - calls parseLeave, [102](#)
 - uses \$NoValue, [102](#)
 - defun, [102](#)
- parseAtSign, [106](#)
 - calls parseTran, [106](#)
 - calls parseType, [106](#)
 - uses \$InteractiveMode, [106](#)
 - defun, [106](#)
- parseCategory, [107](#)
 - calls contained, [107](#)
 - calls parseDropAssertions, [107](#)
 - calls parseTranList, [107](#)
 - defun, [107](#)
- parseCoerce, [108](#)
 - calls parseTran, [108](#)
 - calls parseType, [108](#)
 - uses \$InteractiveMode, [108](#)
 - defun, [108](#)
- parseColon, [108](#)
 - calls parseTran, [108](#)
- calls parseType, [108](#)
 - local ref \$insideConstructIfTrue, [108](#)
 - uses \$InteractiveMode, [108](#)
 - defun, [108](#)
- parseConstruct, [103](#)
 - calledby parseTran, [101](#)
 - calls parseTranList, [103](#)
 - uses \$insideConstructIfTrue, [103](#)
 - defun, [103](#)
- parseDEF, [109](#)
 - calls opFf, [109](#)
 - calls parseLhs, [109](#)
 - calls parseTranCheckForRecord, [109](#)
 - calls parseTranList, [109](#)
 - calls setDefOp, [109](#)
 - uses \$lhs, [109](#)
 - defun, [109](#)
- parseDollarGreaterEqual, [112](#)
 - calls parseTran, [112](#)
 - uses \$op, [112](#)
 - defun, [112](#)
- parseDollarGreaterThan, [112](#)
 - calls parseTran, [112](#)
 - uses \$op, [112](#)
 - defun, [112](#)
- parseDollarLessEqual, [113](#)
 - calls parseTran, [113](#)
 - uses \$op, [113](#)
 - defun, [113](#)
- parseDollarNotEqual, [113](#)
 - calls parseTran, [113](#)
 - uses \$op, [113](#)
 - defun, [113](#)
- parseDropAssertions, [107](#)
 - calledby parseCategory, [107](#)
 - calledby parseDropAssertions, [107](#)
 - calls parseDropAssertions, [107](#)
 - defun, [107](#)
- parseEquivalence, [114](#)
 - calls parseIf, [114](#)
 - defun, [114](#)
- parseExit, [114](#)
 - calls moan, [114](#)
 - calls parseTran, [114](#)
 - defun, [114](#)
- parseGreaterEqual, [115](#)

- calls parseTran, 115
 - uses \$op, 115
 - defun, 115
- parseGreaterThan, 116
 - calls parseTran, 116
 - uses \$op, 116
 - defun, 116
- parseHas, 116
 - calls getdatabase, 116
 - calls makeNonAtomic, 116
 - calls member, 116
 - calls nreverse0, 116
 - calls opOf, 116
 - calls parseHasRhs, 116
 - calls parseType, 116
 - calls unabbrevAndLoad, 116
 - uses \$CategoryFrame, 116
 - uses \$InteractiveMode, 116
 - defun, 116
- parseHasRhs, 118
 - calledby parseHas, 116
 - calls abbreviation?, 118
 - calls get, 118
 - calls loadIfNecessary, 118
 - calls member, 118
 - calls unabbrevAndLoad, 118
 - uses \$CategoryFrame, 118
 - defun, 118
- parseIf, 122
 - calledby parseAnd, 105
 - calledby parseEquivalence, 114
 - calledby parseImplies, 124
 - calledby parseOr, 133
 - calls parseIf,ifTran, 122
 - calls parseTran, 122
 - defun, 122
- parseIf,ifTran, 122
 - calledby parseIf,ifTran, 122
 - calledby parseIf, 122
 - calls incExitLevel, 122
 - calls makeSimplePredicateOrNil, 122
 - calls parseIf,ifTran, 122
 - calls parseTran, 122
 - uses \$InteractiveMode, 122
 - defun, 122
- parseImplies, 124
 - calls parseIf, 124
 - defun, 124
- parseIn, 125
 - calledby parseInBy, 126
 - calls parseTran, 125
 - calls postError, 125
 - defun, 125
- parseInBy, 126
 - calls bright, 126
 - calls parseIn, 126
 - calls parseTran, 126
 - calls postError, 126
 - defun, 126
- parseIs, 127
 - calls parseTran, 127
 - calls transIs, 127
 - defun, 127
- parseIsnt, 127
 - calls parseTran, 127
 - calls transIs, 128
 - defun, 127
- parseJoin, 128
 - calls parseTranList, 128
 - defun, 128
- parseLeave, 129
 - calledby parseAtom, 102
 - calls parseTran, 129
 - defun, 129
- parseLessEqual, 129
 - calls parseTran, 129
 - uses \$op, 129
 - defun, 129
- parseLET, 130
 - calls opOf, 130
 - calls parseTranCheckForRecord, 130
 - calls parseTran, 130
 - calls transIs, 130
 - defun, 130
- parseLETD, 131
 - calls parseTran, 131
 - calls parseType, 131
 - defun, 131
- parseLhs, 110
 - calledby parseDEF, 109
 - calls parseTran, 110
 - calls transIs, 110

- defun, 110
- parseMDEF, 131
 - calls opOf, 131
 - calls parseTranCheckForRecord, 131
 - calls parseTranList, 131
 - calls parseTran, 131
 - uses \$lhs, 131
 - defun, 131
- ParseMode, 414
 - usedby PARSE-Expression, 422
 - usedby PARSE-Operation, 424
 - defvar, 414
- parseNot, 132
 - calls parseTran, 132
 - uses \$InteractiveMode, 132
 - defun, 132
- parseNotEqual, 133
 - calls parseTran, 133
 - uses \$op, 133
 - defun, 133
- parseOr, 133
 - calledby parseOr, 133
 - calls parseIf, 133
 - calls parseOr, 133
 - calls parseTranList, 133
 - calls parseTran, 133
 - defun, 133
- parsepiles
 - calledby preparse1, 87
- parsePretend, 134
 - calls parseTran, 134
 - calls parseType, 134
 - defun, 134
- parseprint, 554
 - calledby preparse, 86
 - defun, 554
- parseReturn, 135
 - calls moan, 135
 - calls parseTran, 135
 - defun, 135
- parseSegment, 135
 - calls parseTran, 135
 - defun, 135
- parseSeq, 136
 - calls last, 136
 - calls mapInto, 136
 - calls postError, 136
 - calls transSeq, 136
 - defun, 136
- parseTran, 101
 - calledby compReduce1, 330
 - calledby parseAnd, 105
 - calledby parseAtSign, 106
 - calledby parseCoerce, 108
 - calledby parseColon, 108
 - calledby parseDollarGreaterEqual, 112
 - calledby parseDollarGreaterThan, 112
 - calledby parseDollarLessEqual, 113
 - calledby parseDollarNotEqual, 113
 - calledby parseExit, 114
 - calledby parseGreaterEqual, 115
 - calledby parseGreaterThan, 116
 - calledby parseIf,ifTran, 122
 - calledby parseIf, 122
 - calledby parseInBy, 126
 - calledby parseIn, 125
 - calledby parseIsnt, 127
 - calledby parseIs, 127
 - calledby parseLETD, 131
 - calledby parseLET, 130
 - calledby parseLeave, 129
 - calledby parseLessEqual, 129
 - calledby parseLhs, 110
 - calledby parseMDEF, 131
 - calledby parseNotEqual, 133
 - calledby parseNot, 132
 - calledby parseOr, 133
 - calledby parsePretend, 134
 - calledby parseReturn, 135
 - calledby parseSegment, 135
 - calledby parseTranCheckForRecord, 543
 - calledby parseTranList, 103
 - calledby parseTransform, 101
 - calledby parseTran, 101
 - calledby parseType, 106
 - calls getl, 101
 - calls parseAtom, 101
 - calls parseConstruct, 101
 - calls parseTranList, 101
 - calls parseTran, 101
 - uses \$op, 101
 - defun, 101

- parseTranCheckForRecord, 543
 - calledby parseDEF, 109
 - calledby parseLET, 130
 - calledby parseMDEF, 131
 - calls parseTran, 543
 - calls postError, 543
 - defun, 543
- parseTranList, 103
 - calledby parseAnd, 105
 - calledby parseCategory, 107
 - calledby parseConstruct, 103
 - calledby parseDEF, 109
 - calledby parseJoin, 128
 - calledby parseMDEF, 131
 - calledby parseOr, 133
 - calledby parseTranList, 103
 - calledby parseTran, 101
 - calledby parseVCONS, 137
 - calls parseTranList, 103
 - calls parseTran, 103
 - defun, 103
- parseTransform, 101
 - calledby s-process, 581
 - calls parseTran, 101
 - uses \$defOp, 101
 - defun, 101
- parseType, 106
 - calledby parseAtSign, 106
 - calledby parseCoerce, 108
 - calledby parseColon, 108
 - calledby parseHas, 116
 - calledby parseLETD, 131
 - calledby parsePretend, 134
 - calls parseTran, 106
 - defun, 106
- parseVCONS, 137
 - calls parseTranList, 137
 - defun, 137
- parseWhere, 137
 - calls mapInto, 137
 - defun, 137
- pathname[5]
 - called by compileSpad2Cmd, 563
 - called by compileSpadLispCmd, 565
 - called by compiler, 561
- pathnameDirectory[5]
 - called by compileSpadLispCmd, 566
- pathnameName[5]
 - called by compileSpadLispCmd, 566
- pathnameType[5]
 - called by compileSpad2Cmd, 563
 - called by compileSpadLispCmd, 565
 - called by compiler, 561
- pathnameTypeId
 - calledby initializeLisplib, 206
- pmatch
 - calledby coerceable, 360
- pmatchWithSl
 - calledby compApplyModemap, 259
- pname
 - calledby buildLibdbConEntry, 480
 - calledby checkTransformFirsts, 512
 - calledby compDefineFunctor1, 148
 - calledby encodeItem, 185
 - calledby isCategoryPackageName, 214
 - calledby mkCategoryPackage, 180
 - calledby recordAttributeDocumentation, 468
- pname[5]
 - called by comp3, 590
 - called by floatexpid, 462
 - called by getScriptName, 409
- Pop-Reduction, 550
 - calledby pop-stack-1, 548
 - calledby pop-stack-2, 548
 - calledby pop-stack-3, 549
 - calledby pop-stack-4, 549
 - calls stack-pop, 550
 - defun, 550
- pop-stack-1, 548
 - calledby PARSE-Application, 429
 - calledby PARSE-Category, 421
 - calledby PARSE-CommandTail, 418
 - calledby PARSE-Conditional, 448
 - calledby PARSE-Data, 439
 - calledby PARSE-Enclosure, 435
 - calledby PARSE-Exit, 447
 - calledby PARSE-Expression, 422
 - calledby PARSE-Expr, 423
 - calledby PARSE-FloatTok, 450
 - calledby PARSE-Float, 433
 - calledby PARSE-Form, 429

- calledby PARSE-Import, [423](#)
- calledby PARSE-InfixWith, [420](#)
- calledby PARSE-Infix, [427](#)
- calledby PARSE-Iterator, [444](#)
- calledby PARSE-LabelExpr, [450](#)
- calledby PARSE-Leave, [447](#)
- calledby PARSE-LedPart, [423](#)
- calledby PARSE-Loop, [449](#)
- calledby PARSE-Name, [438](#)
- calledby PARSE-NudPart, [424](#)
- calledby PARSE-Prefix, [426](#)
- calledby PARSE-Primary1, [432](#)
- calledby PARSE-Qualification, [428](#)
- calledby PARSE-Reduction, [428](#)
- calledby PARSE-Return, [446](#)
- calledby PARSE-ScriptItem, [438](#)
- calledby PARSE-Seg, [448](#)
- calledby PARSE-Selector, [430](#)
- calledby PARSE-SemiColon, [446](#)
- calledby PARSE-Sequence1, [442](#)
- calledby PARSE-Sequence, [442](#)
- calledby PARSE-Sexpr1, [440](#)
- calledby PARSE-SpecialCommand, [416](#)
- calledby PARSE-Statement, [419](#)
- calledby PARSE-Suffix, [446](#)
- calledby PARSE-TokenCommandTail, [417](#)
- calledby PARSE-VarForm, [437](#)
- calledby PARSE-With, [420](#)
- calledby spad, [572](#)
- calledby star, [464](#)
- calls Pop-Reduction, [548](#)
- calls reduction-value, [548](#)
- defmacro, [548](#)
- pop-stack-2, [548](#)
 - calledby PARSE-Application, [429](#)
 - calledby PARSE-Category, [421](#)
 - calledby PARSE-CommandTail, [418](#)
 - calledby PARSE-Conditional, [448](#)
 - calledby PARSE-Float, [433](#)
 - calledby PARSE-Import, [422](#)
 - calledby PARSE-InfixWith, [420](#)
 - calledby PARSE-Infix, [426](#)
 - calledby PARSE-Iterator, [444](#)
 - calledby PARSE-LabelExpr, [450](#)
 - calledby PARSE-Loop, [449](#)
 - calledby PARSE-Prefix, [426](#)
 - calledby PARSE-Primary1, [432](#)
 - calledby PARSE-Reduction, [428](#)
 - calledby PARSE-ScriptItem, [438](#)
 - calledby PARSE-Seg, [448](#)
 - calledby PARSE-Selector, [430](#)
 - calledby PARSE-SemiColon, [446](#)
 - calledby PARSE-Sequence1, [442](#)
 - calledby PARSE-Sequence, [442](#)
 - calledby PARSE-Sexpr1, [440](#)
 - calledby PARSE-SpecialCommand, [416](#)
 - calledby PARSE-Statement, [419](#)
 - calledby PARSE-Suffix, [446](#)
 - calledby PARSE-TokenCommandTail, [417](#)
 - calledby PARSE-VarForm, [437](#)
 - calledby PARSE-With, [420](#)
 - calledby spad, [572](#)
 - calledby star, [464](#)
 - calls Pop-Reduction, [548](#)
 - calls reduction-value, [548](#)
 - defmacro, [548](#)
- pop-stack-3, [549](#)
 - calledby PARSE-Category, [421](#)
 - calledby PARSE-Conditional, [448](#)
 - calledby PARSE-Float, [433](#)
 - calledby PARSE-Iterator, [444](#)
 - calls Pop-Reduction, [549](#)
 - calls reduction-value, [549](#)
 - calls stack-push, [549](#)
 - defmacro, [549](#)
- pop-stack-4, [549](#)
 - calledby PARSE-Float, [433](#)
 - calls Pop-Reduction, [549](#)
 - calls reduction-value, [549](#)
 - calls stack-push, [549](#)
 - defmacro, [549](#)
- postAdd, [376](#)
 - calls postCapsule, [376](#)
 - calls postTran, [376](#)
 - defun, [376](#)
- postAtom, [371](#)
 - calledby postTran, [370](#)
 - local ref \$boot, [371](#)
 - defun, [371](#)
- postAtSign, [378](#)
 - calls postTran, [378](#)
 - calls postType, [378](#)
 - defun, [378](#)
- postBigFloat, [380](#)
 - calls postTran, [380](#)
 - uses \$InteractiveMode, [380](#)
 - uses \$boot, [380](#)

- defun, 380
- postBlock, 380
 - calledby postSemiColon, 399
 - calls postBlockItemList, 380
 - calls postTran, 380
 - defun, 380
- postBlockItem, 377
 - calledby postBlockItemList, 377
 - calledby postCapsule, 377
 - calls postTran, 377
 - defun, 377
- postBlockItemList, 377
 - calledby postBlock, 380
 - calledby postCapsule, 377
 - calls postBlockItem, 377
 - defun, 377
- postCapsule, 377
 - calledby postAdd, 376
 - calls checkWarning, 377
 - calls postBlockItemList, 377
 - calls postBlockItem, 377
 - calls postFlatten, 377
 - defun, 377
- postCategory, 381
 - calls nreverse0, 381
 - calls postTran, 381
 - uses \$insidePostCategoryIfTrue, 381
 - defun, 381
- postcheck, 373
 - calledby postTransformCheck, 373
 - calledby postcheck, 373
 - calls postcheck, 373
 - calls setDefOp, 373
 - defun, 373
- postCollect, 383
 - calledby postCollect, 383
 - calledby postTupleCollect, 402
 - calls postCollect,finish, 383
 - calls postCollect, 383
 - calls postIteratorList, 383
 - calls postTran, 383
 - defun, 383
- postCollect,finish, 381
 - calledby postCollect, 383
 - calls postMakeCons, 381
 - calls postTranList, 382
 - calls tuple2List, 382
 - defun, 381
- postColon, 385
 - calls postTran, 385
 - calls postType, 385
 - defun, 385
- postColonColon, 385
 - calls postForm, 385
 - uses \$boot, 385
 - defun, 385
- postComma, 386
 - calls comma2Tuple, 386
 - calls postTuple, 386
 - defun, 386
- postConstruct, 387
 - calls comma2Tuple, 387
 - calls postMakeCons, 387
 - calls postTranList, 387
 - calls postTranSegment, 387
 - calls postTran, 387
 - calls tuple2List, 387
 - defun, 387
- postDef, 388
 - calls nreverse0, 389
 - calls postDefArgs, 389
 - calls postMDef, 388
 - calls postTran, 388
 - calls recordHeaderDocumentation, 388
 - uses \$InteractiveMode, 389
 - uses \$boot, 389
 - uses \$docList, 389
 - uses \$headerDocumentation, 389
 - uses \$maxSignatureLineNumber, 389
 - defun, 388
- postDefArgs, 390
 - calledby postDefArgs, 390
 - calledby postDef, 389
 - calls postDefArgs, 390
 - calls postError, 390
 - defun, 390
- postError, 374
 - calledby checkWarning, 547
 - calledby getScriptName, 408
 - calledby parseInBy, 126
 - calledby parseIn, 125
 - calledby parseSeq, 136

- calledby parseTranCheckForRecord, 543
- calledby postDefArgs, 390
- calledby postForm, 374
- calls bumperrorcount, 374
- uses \$InteractiveMode, 374
- uses \$defOp, 374
- uses \$postStack, 374
- defun, 374
- postExit, 391
 - calls postTran, 391
 - defun, 391
- postFlatten, 386
 - calledby comma2Tuple, 386
 - calledby postCapsule, 377
 - calledby postFlatten, 386
 - calls postFlatten, 386
 - defun, 386
- postFlattenLeft, 399
 - calledby postFlattenLeft, 399
 - calledby postSemiColon, 399
 - calls postFlattenLeft, 399
 - defun, 399
- postForm, 374
 - calledby postColonColon, 385
 - calledby postTran, 370
 - calls bright, 374
 - calls internl, 374
 - calls postError, 374
 - calls postTranList, 374
 - calls postTran, 374
 - uses \$boot, 374
 - defun, 374
- postIf, 391
 - calls nreverse0, 391
 - calls postTran, 391
 - uses \$boot, 391
 - defun, 391
- postIn, 393
 - calls postInSeq, 393
 - calls postTran, 393
 - calls systemErrorHere, 393
 - defun, 393
- postin, 392
 - calls postInSeq, 392
 - calls postTran, 392
 - calls systemErrorHere, 392
- defun, 392
- postInSeq, 392
 - calledby postIn, 393
 - calledby postIteratorList, 384
 - calledby postin, 392
 - calls postTranSegment, 392
 - calls postTran, 392
 - calls tuple2List, 392
 - defun, 392
- postIteratorList, 384
 - calledby postCollect, 383
 - calledby postIteratorList, 384
 - calledby postRepeat, 398
 - calls postInSeq, 384
 - calls postIteratorList, 384
 - calls postTran, 384
 - defun, 384
- postJoin, 394
 - calls postTranList, 394
 - calls postTran, 394
 - defun, 394
- postMakeCons, 382
 - calledby postCollect,finish, 381
 - calledby postConstruct, 387
 - calledby postMakeCons, 382
 - calls postMakeCons, 382
 - calls postTran, 382
 - defun, 382
- postMapping, 394
 - calls postTran, 394
 - calls unTuple, 394
 - defun, 394
- postMDef, 395
 - calledby postDef, 388
 - calls nreverse0, 395
 - calls postTran, 395
 - calls throwkeyedmsg, 395
 - uses \$InteractiveMode, 395
 - uses \$boot, 395
 - defun, 395
- postOp, 371
 - calledby postTran, 370
 - defun, 371
- postPretend, 396
 - calls postTran, 396
 - calls postType, 396

- defun, 396
- postQUOTE, 397
 - defun, 397
- postReduce, 397
 - calledby postReduce, 397
 - calls postReduce, 397
 - calls postTran, 397
 - uses \$InteractiveMode, 397
 - defun, 397
- postRepeat, 398
 - calls postIteratorList, 398
 - calls postTran, 398
 - defun, 398
- postScripts, 398
 - calls getScriptName, 398
 - calls postTranScripts, 398
 - defun, 398
- postScriptsForm, 372
 - calledby postTran, 370
 - calls getScriptName, 372
 - calls length, 372
 - calls postTranScripts, 372
 - defun, 372
- postSemiColon, 399
 - calls postBlock, 399
 - calls postFlattenLeft, 399
 - defun, 399
- postSignature, 400
 - calls killColons, 400
 - calls postType, 400
 - calls removeSuperfluousMapping, 400
 - defun, 400
- postSlash, 401
 - calls postTran, 401
 - defun, 401
- postTran, 370
 - calledby postAdd, 376
 - calledby postAtSign, 378
 - calledby postBigFloat, 380
 - calledby postBlockItem, 377
 - calledby postBlock, 380
 - calledby postCategory, 381
 - calledby postCollect, 383
 - calledby postColon, 385
 - calledby postConstruct, 387
 - calledby postDef, 388
 - calledby postExit, 391
 - calledby postForm, 374
 - calledby postIf, 391
 - calledby postInSeq, 392
 - calledby postIn, 393
 - calledby postIteratorList, 384
 - calledby postJoin, 394
 - calledby postMDef, 395
 - calledby postMakeCons, 382
 - calledby postMapping, 394
 - calledby postPretend, 396
 - calledby postReduce, 397
 - calledby postRepeat, 398
 - calledby postSlash, 401
 - calledby postTranList, 372
 - calledby postTranScripts, 372
 - calledby postTranSegment, 388
 - calledby postTransform, 369
 - calledby postTran, 370
 - calledby postType, 379
 - calledby postWhere, 403
 - calledby postWith, 404
 - calledby postin, 392
 - calledby tuple2List, 547
 - calls postAtom, 370
 - calls postForm, 370
 - calls postOp, 370
 - calls postScriptsForm, 370
 - calls postTranList, 370
 - calls postTran, 370
 - calls unTuple, 370
 - defun, 370
- postTranList, 372
 - calledby postCollect,finish, 382
 - calledby postConstruct, 387
 - calledby postForm, 374
 - calledby postJoin, 394
 - calledby postTran, 370
 - calledby postTuple, 402
 - calledby postWhere, 403
 - calls postTran, 372
 - defun, 372
- postTranScripts, 372
 - calledby postScriptsForm, 372
 - calledby postScripts, 398
 - calledby postTranScripts, 372

- calls postTranScripts, 372
- calls postTran, 372
- defun, 372
- postTranSegment, 388
 - calledby postConstruct, 387
 - calledby postInSeq, 392
 - calledby tuple2List, 547
 - calls postTran, 388
 - defun, 388
- postTransform, 369
 - calledby new2OldLisp, 78
 - calledby recordAttributeDocumentation, 468
 - calledby recordSignatureDocumentation, 468
 - calledby s-process, 581
 - calls aplTran, 369
 - calls identp[5], 369
 - calls postTransformCheck, 369
 - calls postTran, 369
 - defun, 369
- postTransformCheck, 373
 - calledby postTransform, 369
 - calls postcheck, 373
 - uses \$defOp, 373
 - defun, 373
- postTuple, 402
 - calledby postComma, 386
 - calls postTranList, 402
 - defun, 402
- postTupleCollect, 402
 - calls postCollect, 402
 - defun, 402
- postType, 379
 - calledby postAtSign, 378
 - calledby postColon, 385
 - calledby postPretend, 396
 - calledby postSignature, 400
 - calls postTran, 379
 - calls unTuple, 379
 - defun, 379
- postWhere, 403
 - calls postTranList, 403
 - calls postTran, 403
 - defun, 403
- postWith, 404
 - calls postTranScripts, 372
 - calls postTran, 372
 - defun, 372
- pp
 - calledby checkComments, 496
 - calledby compDefineFunctor1, 148
 - calledby optimizeFunctionDef, 228
- PredImplies
 - calledby compForm2, 608
- preparse, 81, 86
 - calledby preparse, 86
 - calledby spad, 572
 - calls ifcar, 86
 - calls parseprint, 86
 - calls preparse1, 86
 - calls preparse, 86
 - uses \$comblocklist, 86
 - uses \$constructorLineNumber, 86
 - uses \$docList, 86
 - uses \$headerDocumentation, 86
 - uses \$index, 86
 - uses \$maxSignatureLineNumber, 86
 - uses \$preparse-last-line, 86
 - uses \$preparseReportIfTrue, 86
 - uses \$skipme, 86
 - defun, 86
- preparse-echo, 95
 - calledby fincomblock, 552
 - calledby preparse1, 87
 - local ref \$EchoLineStack, 95
 - uses Echo-Meta, 95
 - defun, 95
- preparse1, 87
 - calledby preparse, 86
 - calls doSystemCommand[5], 87
 - calls escaped, 87
 - calls fincomblock, 87
 - calls indent-pos, 87
 - calls is-console, 87
 - calls make-full-cvec, 87
 - calls maxindex, 87
 - calls parsepiles, 87
 - calls preparse-echo, 87
 - calls preparseReadLine, 87
 - calls strposl[5], 87
 - local def \$index, 87

- local def \$preparse-last-line, 87
 - local def \$skipme, 87
 - local ref \$byConstructors, 87
 - local ref \$constructorsSeen, 87
 - local ref \$echolinestack, 87
 - local ref \$in-stream, 87
 - local ref \$index, 87
 - local ref \$linelist, 87
 - local ref \$preparse-last-line, 87
 - catches, 87
 - defun, 87
- preparseReadLine
 - calledby preparse1, 87
 - calledby skip-to-endif, 554
- preparseReadLine1, 93
 - calledby preparseReadLine1, 93
 - calledby skip-ifblock, 92
 - calledby skip-to-endif, 554
 - calls expand-tabs, 93
 - calls get-a-line, 93
 - calls maxindex, 93
 - calls preparseReadLine1, 93
 - calls strconc, 93
 - local def \$EchoLineStack, 93
 - local def \$index, 93
 - local def \$linelist, 93
 - local def \$preparse-last-line, 93
 - local ref \$index, 93
 - local ref \$linelist, 93
 - defun, 93
- pretend, 134, 328, 396
 - defplist, 134, 328, 396
- prettyprint
 - calledby optimize, 229
 - calledby s-process, 581
- PrimitivteArray
 - calledby optCallEval, 238
- primitiveType, 597
 - calledby compAtom, 595
 - uses \$DoubleFloat, 597
 - uses \$EmptyMode, 597
 - uses \$NegativeInteger, 597
 - uses \$NonNegativeInteger, 597
 - uses \$PositiveInteger, 597
 - uses \$String, 597
 - defun, 597
- print-defun, 584
 - calls is-console, 584
 - calls print-full, 584
 - uses \$PrettyPrint, 584
 - uses vmlisp::optionlist, 584
 - defun, 584
- print-full
 - calledby print-defun, 584
- print-package, 547
 - local ref \$out-stream, 547
 - defun, 547
- printSignature
 - calledby getSignature, 307
- printStats
 - calledby compile, 173
- prior-token, 97
 - usedby PARSE-Expression, 422
 - uses \$token, 97
 - defvar, 97
- processFunctor, 279
 - calledby compCapsuleInner, 278
 - calls buildFunctor, 279
 - calls error, 279
 - defun, 279
- processInteractive[5]
 - called by s-process, 581
- processSynonyms[5]
 - called by PARSE-NewExpr, 415
- profileRecord
 - calledby compDefineCapsuleFunction, 155
 - calledby setqSingle, 344
- profileWrite
 - calledby finalizeLisplib, 207
- purgeNewConstructorLines, 479
 - calledby extendLocalLibdb, 475
 - calls screenLocalLine, 479
 - defun, 479
- push-reduction, 465
 - calledby PARSE-AnyId, 441
 - calledby PARSE-Application, 429
 - calledby PARSE-Category, 421
 - calledby PARSE-CommandTail, 418
 - calledby PARSE-Command, 415
 - calledby PARSE-Conditional, 448
 - calledby PARSE-Data, 439
 - calledby PARSE-Enclosure, 435

- calledby PARSE-Exit, [447](#)
- calledby PARSE-Expression, [422](#)
- calledby PARSE-Expr, [423](#)
- calledby PARSE-FloatBasePart, [434](#)
- calledby PARSE-FloatBase, [433](#)
- calledby PARSE-FloatExponent, [434](#)
- calledby PARSE-FloatTok, [450](#)
- calledby PARSE-Float, [433](#)
- calledby PARSE-Form, [429](#)
- calledby PARSE-Import, [422](#)
- calledby PARSE-InfixWith, [420](#)
- calledby PARSE-Infix, [426](#)
- calledby PARSE-LabelExpr, [450](#)
- calledby PARSE-Leave, [447](#)
- calledby PARSE-LedPart, [423](#)
- calledby PARSE-Loop, [449](#)
- calledby PARSE-Name, [438](#)
- calledby PARSE-NudPart, [424](#)
- calledby PARSE-OpenBrace, [443](#)
- calledby PARSE-OpenBracket, [443](#)
- calledby PARSE-Prefix, [426](#)
- calledby PARSE-Primary1, [432](#)
- calledby PARSE-PrimaryOrQM, [419](#)
- calledby PARSE-Quad, [436](#)
- calledby PARSE-Qualification, [427](#)
- calledby PARSE-Reduction, [428](#)
- calledby PARSE-Return, [446](#)
- calledby PARSE-ScriptItem, [438](#)
- calledby PARSE-Seg, [448](#)
- calledby PARSE-Selector, [430](#)
- calledby PARSE-SemiColon, [446](#)
- calledby PARSE-Sequence1, [442](#)
- calledby PARSE-Sequence, [442](#)
- calledby PARSE-Sexpr1, [440](#)
- calledby PARSE-SpecialCommand, [416](#)
- calledby PARSE-Statement, [419](#)
- calledby PARSE-Suffix, [445](#)
- calledby PARSE-TokenCommandTail, [417](#)
- calledby PARSE-TokenList, [418](#)
- calledby PARSE-VarForm, [437](#)
- calledby PARSE-With, [420](#)
- calledby parse-argument-designator, [546](#)
- calledby parse-identifier, [545](#)
- calledby parse-keyword, [546](#)
- calledby parse-number, [545](#)
- calledby parse-spadstring, [544](#)
- calledby parse-string, [544](#)
- calledby star, [464](#)
- calls make-reduction, [465](#)
- calls stack-push, [465](#)
- uses reduce-stack, [465](#)
- defun, [465](#)
- put
 - calledby checkAndDeclare, [308](#)
 - calledby compColon, [294](#)
 - calledby compDefineCapsuleFunction, [155](#)
 - calledby compMacro, [327](#)
 - calledby compSubsetCategory, [352](#)
 - calledby compSuchthat, [353](#)
 - calledby compTypeOf, [593](#)
 - calledby doIt, [281](#)
 - calledby evalAndSub, [269](#)
 - calledby getInverseEnvironment, [320](#)
 - calledby getSuccessEnvironment, [318](#)
 - calledby giveFormalParametersValues, [178](#)
 - calledby putDomainsInScope, [255](#)
 - calledby setqMultiple, [341](#)
 - calledby updateCategoryFrameForCategory, [121](#)
 - calledby updateCategoryFrameForConstructor, [120](#)
- putDomainsInScope, [255](#)
 - calledby addConstructorModemaps, [258](#)
 - calledby augModemapsFromCategoryRep, [271](#)
 - calledby augModemapsFromCategory, [265](#)
 - calls delete, [255](#)
 - calls getDomainsInScope, [255](#)
 - calls member, [255](#)
 - calls put, [255](#)
 - calls say, [255](#)
 - local def \$CapsuleDomainsInScope, [255](#)
 - local ref \$insideCapsuleFunctionIfTrue, [255](#)
 - defun, [255](#)
- putInLocalDomainReferences, [189](#)
 - calledby compile, [173](#)
 - calls NRTputInTail, [189](#)
 - local def \$elt, [189](#)
 - local ref \$QuickCode, [189](#)
 - defun, [189](#)

- qslessp
 - calledby addDomain, 252
 - calledby getUniqueModemap, 263
- qsminus, 241
 - defplist, 241
- quote, 329, 396
 - defplist, 329, 396
- quote-if-string, 453
 - calledby match-advance-string, 452
 - calledby unget-tokens, 456
 - calls escape-keywords, 454
 - calls pack, 454
 - calls strconc, 453
 - calls token-nonblank, 454
 - calls token-symbol, 453
 - calls token-type, 453
 - calls underscore, 454
 - uses \$boot, 454
 - uses \$spad, 454
 - defun, 453
- quotify
 - calledby compIf, 315
- rassoc
 - calledby checkBalance, 499
 - calledby modemapPattern, 203
- rbp
 - usedby PARSE-NudPart, 424
 - usedby PARSE-Operation, 424
- rdefiostream
 - calledby compileDocumentation, 169
 - calledby writeLib1, 207
- read-a-line, 629
 - calledby get-a-line, 636
 - calls Line-New-Line, 629
 - uses *eof*, 629
 - uses File-Closed, 629
 - defun, 629
- readline
 - calledby dbReadLines, 479
- recompile-lib-file-if-necessary, 625
 - calledby compileSpadLispCmd, 566
 - calls compile-lib-file, 625
 - uses *lisp-bin-filetype*, 625
 - defun, 625
- Record, 288
 - defplist, 288
- recordAttributeDocumentation, 468
 - calledby PARSE-Category, 421
 - calls ifcdr, 468
 - calls opOf, 468
 - calls pname, 468
 - calls postTransform, 468
 - calls recordDocumentation, 468
 - calls upper-case-p, 468
 - defun, 468
- RecordCategory, 300
 - defplist, 300
- recordcopy, 251
 - defplist, 251
- recordDocumentation, 469
 - calledby recordAttributeDocumentation, 468
 - calledby recordSignatureDocumentation, 468
 - calls collectComBlock, 469
 - calls recordHeaderDocumentation, 469
 - local def \$docList, 469
 - local def \$maxSignatureLineNumber, 469
 - defun, 469
- recordelt, 249
 - defplist, 249
- recordHeaderDocumentation, 470
 - calledby postDef, 388
 - calledby recordDocumentation, 469
 - calls assocright, 470
 - local def \$comblocklist, 470
 - local def \$headerDocumentation, 470
 - local ref \$comblocklist, 470
 - local ref \$headerDocumentation, 470
 - local ref \$maxSignatureLineNumber, 470
 - defun, 470
- recordSignatureDocumentation, 468
 - calledby PARSE-Category, 421
 - calls postTransform, 468
 - calls recordDocumentation, 468
 - defun, 468
- reduce, 330, 397
 - defplist, 330, 397
- reduce-stack, 465
 - usedby push-reduction, 465
 - uses \$stack, 465

- defvar, 465
- reduce-stack-clear, 465
 - defmacro, 465
- reduction, 99
 - defstruct, 99
- reduction-value
 - calledby nth-stack, 549
 - calledby pop-stack-1, 548
 - calledby pop-stack-2, 548
 - calledby pop-stack-3, 549
 - calledby pop-stack-4, 549
- refvecp
 - calledby translabel1, 541
- remdup
 - calledby compDefineFunctor1, 148
 - calledby displayPreCompilationErrors, 542
 - calledby finalizeDocumentation, 490
 - calledby getSignature, 306
 - calledby hasSigInTargetCategory, 308
 - calledby mkAListOfExplicitCategoryOps, 193
 - calledby mkExplicitCategoryFunction, 292
 - calledby orderByDependency, 227
- removeBackslashes, 539
 - calledby checkGetParse, 521
 - calledby removeBackslashes, 539
 - calls charPosition, 539
 - calls length, 539
 - calls removeBackslashes, 539
 - calls strconc, 539
 - defun, 539
- removeEnv
 - calledby compApply, 592
 - calledby getSuccessEnvironment, 319
 - calledby setqSingle, 344
- removeSuperfluousMapping, 400
 - calledby postSignature, 400
 - defun, 400
- removeZeroOne
 - calledby compDefineCategory2, 164
 - calledby compDefineFunctor1, 149
 - calledby finalizeLisplib, 207
- remprop
 - calledby unloadOneConstructor, 205
- repeat, 332, 398
 - defplist, 332, 398
- replaceExitEsc
 - calledby coerceExit, 361
- replaceExitEtc, 337
 - calledby compDefineCapsuleFunction, 156
 - calledby compSeq1, 336
 - calledby replaceExitEtc, 337
 - calls convertOrCroak, 337
 - calls intersectionEnvironment, 337
 - calls replaceExitEtc, 337
 - calls rplac, 337
 - local def \$finalEnv, 337
 - local ref \$finalEnv, 337
 - defun, 337
- replaceFile
 - calledby compileDocumentation, 169
 - calledby lisplibDoRename, 205
- replaceVars, 196
 - calledby interactiveModemapForm, 195
 - defun, 196
- reportOnFunctorCompilation, 219
 - calledby compDefineFunctor1, 149
 - calls addStats, 219
 - calls displayMissingFunctions, 219
 - calls displaySemanticErrors, 219
 - calls displayWarnings, 219
 - calls normalizeStatAndStringify, 219
 - calls sayBrightly, 219
 - uses \$functionStats, 220
 - uses \$functorStats, 219
 - uses \$op, 219
 - uses \$semanticErrorStack, 220
 - uses \$warningStack, 220
 - defun, 219
- resolve, 365
 - calledby coerceExit, 361
 - calledby compApplication, 603
 - calledby compApply, 592
 - calledby compCategory, 290
 - calledby compCoerce1, 363
 - calledby compConstructorCategory, 301
 - calledby compDefineCapsuleFunction, 156
 - calledby compIf, 315
 - calledby compReturn, 335
 - calledby compString, 349
 - calledby convert, 597
 - calledby modifyModeStack, 622

- calls mkUnion, 365
- calls modeEqual, 365
- local ref \$EmptyMode, 365
- local ref \$NoValueMode, 365
- local ref \$String, 365
- defun, 365
- return, 135, 335
 - defplist, 135, 335
- rpackfile
 - calledby compDefineLisplib, 167
 - calledby compileDocumentation, 169
- rplac
 - calledby coerce, 355
 - calledby getAbbreviation, 302
 - calledby optCall, 233
 - calledby optCatch, 245
 - calledby optCond, 247
 - calledby optSpecialCall, 236
 - calledby optXLAMCond, 230
 - calledby optimizeFunctionDef, 228
 - calledby optimize, 229
 - calledby replaceExitEtc, 337
- rplaca
 - calledby NRTgetLocalIndex, 215
 - calledby NRTputInTail, 189
 - calledby doItIf, 285
 - calledby optPackageCall, 235
 - calledby optSpecialCall, 236
- rplacd
 - calledby doItIf, 285
 - calledby optCond, 246
 - calledby optPackageCall, 235
- rplacw
 - calledby optSpecialCall, 236
- rshut
 - calledby compDefineLisplib, 167
 - calledby compileDocumentation, 169
- rwrite
 - calledby compilerDoitWithScreenedLisplib, 567
- rwrite128
 - calledby lisplibWrite, 214
- rwriteLispForm, 204
 - calledby evalAndRwriteLispForm, 204
 - local ref \$libFile, 204
 - local ref \$lisplib, 204
- defun, 204
- s-process, 574
 - calledby spad, 572
 - calls compTopLevel, 581
 - calls curstrm, 581
 - calls def-process, 581
 - calls def-rename, 581
 - calls displayPreCompilationErrors, 581
 - calls displaySemanticErrors, 581
 - calls get-internal-run-time, 581
 - calls new2OldLisp, 581
 - calls parseTransform, 581
 - calls postTransform, 581
 - calls prettyprint, 581
 - calls processInteractive[5], 581
 - calls terpri, 581
 - uses \$DomainFrame, 582
 - uses \$EmptyEnvironment, 582
 - uses \$EmptyMode, 581
 - uses \$Index, 581
 - uses \$InteractiveFrame, 582
 - uses \$LocalFrame, 582
 - uses \$PolyMode, 581
 - uses \$PrintOnly, 582
 - uses \$TranslateOnly, 582
 - uses \$Translation, 582
 - uses \$VariableCount, 582
 - uses \$compUniquelyIfTrue, 581
 - uses \$currentFunction, 581
 - uses \$currentLine, 582
 - uses \$exitModeStack, 581
 - uses \$exitMode, 581
 - uses \$e, 582
 - uses \$form, 581
 - uses \$genFVar, 582
 - uses \$genSDVar, 582
 - uses \$insideCapsuleFunctionIfTrue, 581
 - uses \$insideCategoryIfTrue, 581
 - uses \$insideCoerceInteractiveHardIfTrue, 581
 - uses \$insideExpressionIfTrue, 581
 - uses \$insideFunctorIfTrue, 581
 - uses \$insideWhereIfTrue, 581
 - uses \$leaveLevelStack, 581
 - uses \$leaveMode, 581

- uses \$macroassoc, [581](#)
 - uses \$newspad, [581](#)
 - uses \$postStack, [581](#)
 - uses \$previousTime, [582](#)
 - uses \$returnMode, [581](#)
 - uses \$semanticErrorStack, [581](#)
 - uses \$stop-level, [581](#)
 - uses \$stopOp, [581](#)
 - uses \$warningStack, [581](#)
 - uses curoutstream, [582](#)
 - defun, [574](#)
- say
 - calledby canReturn, [316](#)
 - calledby compOrCroak1, [586](#)
 - calledby getSignature, [306](#)
 - calledby modifyModeStack, [622](#)
 - calledby optimize, [229](#)
 - calledby orderByDependency, [227](#)
 - calledby putDomainsInScope, [255](#)
- sayBrightly
 - calledby NRTgetLookupFunction, [214](#)
 - calledby checkAndDeclare, [308](#)
 - calledby checkDocError, [516](#)
 - calledby compDefineCapsuleFunction, [156](#)
 - calledby compDefineFunctor1, [148](#)
 - calledby compMacro, [327](#)
 - calledby compilerDoit, [568](#)
 - calledby compile, [173](#)
 - calledby displayMissingFunctions, [220](#)
 - calledby displayPreCompilationErrors, [542](#)
 - calledby doIt, [281](#)
 - calledby reportOnFunctorCompilation, [219](#)
 - calledby spadCompileOrSetq, [186](#)
 - calledby transDocList, [493](#)
 - calledby transformAndRecheckComments, [495](#)
- saybrightly1
 - calledby checkDocError, [515](#)
- sayBrightlyI
 - calledby optimizeFunctionDef, [228](#)
- sayBrightlyNT
 - calledby NRTgetLookupFunction, [214](#)
- sayKeyedMsg
 - calledby finalizeDocumentation, [490](#)
- sayKeyedMsg[5]
 - called by compileSpad2Cmd, [563](#)
- called by compileSpadLispCmd, [566](#)
- sayMath
 - calledby displayPreCompilationErrors, [542](#)
- sayMSG
 - calledby compDefineLisplib, [167](#)
 - calledby finalizeDocumentation, [490](#)
 - calledby finalizeLisplib, [207](#)
- ScanOrPairVec[5]
 - called by hasFormalMapVariable, [621](#)
- screenLocalLine, [484](#)
 - calledby purgeNewConstructorLines, [479](#)
- calls charPosition, [484](#)
- calls dbKind, [484](#)
- calls dbName, [484](#)
- calls dbPart, [484](#)
- defun, [484](#)
- Scripts, [398](#)
 - defplist, [398](#)
- segment, [135](#), [136](#)
 - defplist, [135](#), [136](#)
- selectOptionLC[5]
 - called by compileSpad2Cmd, [563](#)
 - called by compileSpadLispCmd, [565](#)
 - called by compiler, [561](#)
- seq, [238](#), [336](#)
 - defplist, [238](#), [336](#)
- setDefOp, [404](#)
 - calledby parseDEF, [109](#)
 - calledby postcheck, [373](#)
 - uses \$defOp, [404](#)
 - uses \$stopOp, [404](#)
 - defun, [404](#)
- setdifference
 - calledby orderPredTran, [198](#)
- setelt
 - calledby checkAddPeriod, [527](#)
 - calledby modifyModeStack, [622](#)
- seteltModemapFilter, [606](#)
 - calls isConstantId, [606](#)
 - calls stackMessage, [606](#)
 - defun, [606](#)
- setq, [339](#)
 - defplist, [339](#)
- setqMultiple, [341](#)
 - calledby compSetq1, [340](#)
 - calls addBinding, [341](#)

- calls compSetq1, 341
- calls convert, 341
- calls genSomeVariable, 341
- calls genVariable, 341
- calls length, 341
- calls mkprogn, 341
- calls nreverse0, 341
- calls put, 341
- calls setqMultipleExplicit, 341
- calls stackMessage, 341
- local ref \$EmptyMode, 341
- local ref \$NoValueMode, 341
- local ref \$noEnv, 341
- defun, 341
- setqMultipleExplicit, 343
 - calledby setqMultiple, 341
 - calls compSetq1, 343
 - calls genVariable, 343
 - calls last, 343
 - calls stackMessage, 343
 - local ref \$EmptyMode, 343
 - local ref \$NoValueMode, 343
 - defun, 343
- setqSetelt, 344
 - calledby compSetq1, 340
 - calls comp, 344
 - defun, 344
- setqSingle, 344
 - calledby compSetq1, 339
 - calls NRTassocIndex, 344
 - calls addBinding[5], 344
 - calls assignError, 344
 - calls augModemapsFromDomain1, 344
 - calls comp, 344
 - calls consPropListOf, 344
 - calls convert, 344
 - calls getPropList[5], 344
 - calls getmode, 344
 - calls get, 344
 - calls identp[5], 344
 - calls isDomainForm, 344
 - calls isDomainInScope, 344
 - calls maxSuperType, 344
 - calls outputComp, 344
 - calls profileRecord, 344
 - calls removeEnv, 344
 - calls stackWarning, 344
 - uses \$EmptyMode, 345
 - uses \$NoValueMode, 345
 - uses \$QuickLet, 344
 - uses \$form, 345
 - uses \$insideSetqSingleIfTrue, 344
 - uses \$profileCompiler, 345
 - defun, 344
- setrecordelt, 250
 - defplist, 250
- shut
 - calledby buildLibdb, 476
 - calledby dbWriteLines, 479
 - calledby whoOwns, 539
- shut[5]
 - called by spad, 572
- Signature, 400
 - defplist, 400
- signatures
 - a, 1
 - compiler, 561
 - compileSpad2Cmd, 563
 - get-a-line, 636
 - is-console, 553
 - preparse1, 87
 - preparseReadLine1, 93
 - read-a-line, 629
- signatureTran, 197
 - calledby orderPredicateItems, 197
 - calledby signatureTran, 197
 - calls isCategoryForm, 197
 - calls signatureTran, 197
 - local ref \$e, 198
 - defun, 197
- simpBool
 - calledby compDefineFunctor1, 149
- skip-blanks, 451
 - calledby match-string, 451
 - calls advance-char, 451
 - calls current-char, 451
 - calls token-lookahead-type, 451
 - defun, 451
- skip-ifblock, 92
 - calledby skip-ifblock, 92
 - calls initial-substring, 92
 - calls preparseReadLine1, 92

- calls skip-ifblock, 92
- calls storeblanks, 92
- calls string2BootTree, 92
- defun, 92
- skip-to-endif, 554
 - calledby skip-to-endif, 554
 - calls initial-substring, 554
 - calls preparsedReadLine1, 554
 - calls preparsedReadLine, 554
 - calls skip-to-endif, 554
 - defun, 554
- SourceLevelSubsume
 - calledby getSignature, 307
- spad, 571
 - calls PARSE-NewExpr, 572
 - calls addBinding[5], 572
 - calls init-boot/spad-reader[5], 572
 - calls initialize-preparse, 572
 - calls ioclear, 572
 - calls makeInitialModemapFrame[5], 572
 - calls pop-stack-1, 572
 - calls preparse, 572
 - calls s-process, 572
 - calls shut[5], 572
 - uses *comp370-apply*, 572
 - uses *eof*, 572
 - uses *fileactq-apply*, 572
 - uses /editfile, 572
 - uses \$InitialDomainsInScope, 572
 - uses \$InteractiveFrame, 572
 - uses \$InteractiveMode, 572
 - uses \$boot, 572
 - uses \$noSubsumption, 572
 - uses \$spad, 572
 - uses boot-line-stack, 572
 - uses curoutstream, 572
 - uses echo-meta, 572
 - uses file-closed, 572
 - uses line, 572
 - uses optionlist, 572
 - catches, 572
 - defun, 571
- spad-fixed-arg, 625
 - defun, 625
- spad2AsTranslatorAutoloadOnceTrigger
 - calledby compileSpad2Cmd, 563
- spadcall, 243
 - defplist, 243
- spadCompileOrSetq, 186
 - calledby compile, 173
 - calls LAM,EVALANDFILEACTQ, 186
 - calls bright, 186
 - calls compileConstructor, 186
 - calls comp, 186
 - calls contained, 186
 - calls mkq, 186
 - calls sayBrightly, 186
 - local ref \$insideCapsuleFunctionIfTrue, 186
 - defun, 186
- SpadInterpretStream[5]
 - called by ncINTERPFILE, 624
- spadPrompt
 - calledby compileSpad2Cmd, 563
 - calledby compileSpadLispCmd, 566
- spadreduce
 - calledby floatexpid, 462
- spadSysBranch, 510
 - calledby spadSysChoose, 510
 - calls member, 510
 - calls spadSysChoose, 510
 - calls systemError, 510
 - defun, 510
- spadSysChoose, 510
 - calledby checkRecordHash, 507
 - calledby spadSysBranch, 510
 - calls lassoc, 510
 - calls spadSysBranch, 510
 - defun, 510
- splitEncodedFunctionName, 184
 - calledby compile, 173
 - calls strpos, 184
 - defun, 184
- stack, 95
 - defstruct, 95
- stack-/empty, 96
 - uses \$stack, 96
 - defmacro, 96
- stack-clear, 96
 - uses \$stack, 96
 - defun, 96
- stack-load, 95

- uses \$stack, 95
 - defun, 95
- stack-pop, 97
 - calledby Pop-Reduction, 550
 - uses \$stack, 97
 - defun, 97
- stack-push, 96
 - calledby pop-stack-2, 548
 - calledby pop-stack-3, 549
 - calledby pop-stack-4, 549
 - calledby push-reduction, 465
 - uses \$stack, 96
 - defun, 96
- stack-size
 - calledby star, 464
- stack-store
 - calledby nth-stack, 549
- stackAndThrow
 - calledby compDefine1, 145
 - calledby compInternalFunction, 159
 - calledby compLambda, 325
 - calledby compWithMappingMode1, 615
 - calledby getSignatureFromMode, 303
- stackMessage
 - calledby assignError, 346
 - calledby augModemapsFromDomain1, 257
 - calledby autoCoerceByModemap, 364
 - calledby coerce, 356
 - calledby compElt, 310
 - calledby compMapCond", 261
 - calledby compMapCond', 261
 - calledby compRepeatOrCollect, 333
 - calledby compSymbol, 598
 - calledby eltModemapFilter, 605
 - calledby getFormModemaps, 604
 - calledby getOperationAlist, 269
 - calledby seteltModemapFilter, 606
 - calledby setqMultipleExplicit, 343
 - calledby setqMultiple, 341
- stackMessageIfNone
 - calledby compExit, 312
 - calledby compForm, 600
- stackSemanticError
 - calledby compColonInside, 594
 - calledby compJoin, 323
 - calledby compOrCroak1, 586
 - calledby compPretend, 329
 - calledby compReturn, 335
 - calledby compSubDomain1, 350
 - calledby constructMacro, 186
 - calledby doIt, 281
 - calledby getArgumentModeOrMoan, 191
 - calledby getSignature, 307
 - calledby getTargetFromRhs, 177
 - calledby unknownTypeError, 253
- stackWarning
 - calledby compColonInside, 594
 - calledby compElt, 310
 - calledby compPretend, 329
 - calledby doIt, 281
 - calledby getUniqueModemap, 263
 - calledby hasSigInTargetCategory, 309
 - calledby setqSingle, 344
- star, 464
 - calledby PARSE-Application, 429
 - calledby PARSE-Category, 421
 - calledby PARSE-CommandTail, 418
 - calledby PARSE-Expr, 423
 - calledby PARSE-Import, 422
 - calledby PARSE-IteratorTail, 444
 - calledby PARSE-Loop, 449
 - calledby PARSE-Option, 419
 - calledby PARSE-ScriptItem, 438
 - calledby PARSE-Sexpr1, 440
 - calledby PARSE-SpecialCommand, 416
 - calledby PARSE-Statement, 419
 - calledby PARSE-TokenCommandTail, 417
 - calledby PARSE-TokenList, 418
 - calls pop-stack-1, 464
 - calls push-reduction, 464
 - calls stack-size, 464
 - defmacro, 464
- step
 - calledby floatexpid, 462
- storeblanks, 635
 - calledby skip-ifblock, 92
 - defun, 635
- strconc
 - calledby buildLibOp, 482
 - calledby buildLibdbConEntry, 480
 - calledby buildLibdbString, 478
 - calledby checkAddBackSlashes, 526

- calledby checkAddIndented, 518
- calledby checkAddSpaceSegments, 528
- calledby checkComments, 496
- calledby checkIndentedLines, 522
- calledby checkTransformFirst, 512
- calledby compApplication, 603
- calledby compDefine1, 145
- calledby compDefineFunctor1, 148
- calledby compileSpad2Cmd, 563
- calledby compile, 173
- calledby decodeScripts, 409
- calledby finalizeDocumentation, 490
- calledby getCaps, 185
- calledby mkCategoryPackage, 180
- calledby preparseReadLine1, 93
- calledby quote-if-string, 453
- calledby removeBackslashes, 539
- calledby unget-tokens, 456
- calledby whoOwns, 539
- String, 349
 - defplist, 349
- string-not-greaterp
 - calledby initial-substring-p, 453
- string2BootTree, 77
 - calledby skip-ifblock, 92
 - calls def-rename, 77
 - calls new2OldLisp, 77
 - local def \$boot, 78
 - local def \$spad, 78
 - uses boot-line-stack, 77
 - uses line-handler, 77
 - uses xtokenreader, 77
 - defun, 77
- string2id-n
 - calledby infixtok, 553
- stringPrefix?
 - calledby checkGetArgs, 520
 - calledby comp3, 590
- stripOffArgumentConditions, 306
 - calledby compDefineCapsuleFunction, 155
 - local def \$argumentConditionList, 306
 - local ref \$argumentConditionList, 306
 - defun, 306
- stripOffSubdomainConditions, 305
 - calledby compDefineCapsuleFunction, 155
 - calls assoc, 305
- calls mkpf, 305
- local def \$argumentConditionList, 305
- local ref \$argumentConditionList, 305
- defun, 305
- stripUnionTags
 - calledby augModemapsFromDomain, 256
- strpos
 - calledby splitEncodedFunctionName, 184
- strposl[5]
 - called by preparse1, 87
- SubDomain, 350
 - defplist, 350
- sublis
 - calledby NRTgetLookupFunction, 214
 - calledby applyMapping, 591
 - calledby augLisplibModemapsFromCategory, 191
 - calledby coerceable, 360
 - calledby compApplyModemap, 259
 - calledby compDefineCategory2, 163
 - calledby compDefineFunctor1, 148
 - calledby compForm2, 607
 - calledby doIt, 281
 - calledby formal2Pattern, 218
 - calledby getModemap, 259
 - calledby mkOpVec, 225
 - calledby substituteCategoryArguments, 257
 - calledby substituteIntoFunctorModemap, 612
- sublislis
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
 - calledby compHasFormat, 313
 - calledby finalizeDocumentation, 490
 - calledby mkCategoryPackage, 180
- subname, 232
 - calledby optimize, 229
 - calls bpiname, 232
 - calls compiled-function-p, 232
 - calls identp, 232
 - calls mbpip, 232
 - defun, 232
- subseq
 - calledby match-string, 451
- SubsetCategory, 352
 - defplist, 352

- substituteCategoryArguments, 257
 - calledby augModemapsFromDomain1, 257
 - calls internl, 257
 - calls sublis, 257
 - defun, 257
- substituteIntoFunctorModemap, 612
 - calledby compFocompFormWithModemap, take 610
 - calls compOrCroak, 612
 - calls eqsubstlist, 612
 - calls keyedSystemError, 612
 - calls sublis, 612
 - defun, 612
- substNames, 270
 - calledby evalAndSub, 269
 - calledby genDomainOps, 225
 - calls eqsubstlist, 270
 - calls isCategoryPackageName, 270
 - calls nreverse0, 270
 - uses \$FormalMapVariableList, 271
 - defun, 270
- substring?
 - calledby checkBeginEnd, 500
 - calledby checkExtract, 519
- substVars, 202
 - calledby interactiveModemapForm, 195
 - calls contained, 202
 - calls nsubst, 202
 - local ref \$FormalMapVariableList, 203
 - defun, 202
- suffix
 - calledby addclose, 550
- systemCommand[5]
 - called by PARSE-CommandTail, 418
 - called by PARSE-TokenCommandTail, 417
- systemError
 - calledby checkTrim, 515
 - calledby compReduce1, 330
 - calledby getOperationAlist, 269
 - calledby spadSysBranch, 510
- systemErrorHere
 - calledby addArgumentConditions, 304
 - calledby addEltModemap, 265
 - calledby canReturn, 316
 - calledby compCategory, 290
 - calledby compColon, 294
 - calledby getSlotFromCategoryForm, 210
 - calledby getSlotFromFunctor, 212
 - calledby optCall, 234
 - calledby postIn, 393
 - calledby postin, 392
- calledby compColon, 294
- calledby compDefineCategory2, 163
- calledby compForm2, 607
- calledby compHasFormat, 313
- calledby compWithMappingMode1, 615
- calledby drop, 551
- calledby getSignatureFromMode, 303
- calledby getSlotFromCategoryForm, 210
- terminateSystemCommand[5]
 - called by compileSpad2Cmd, 563
 - called by compileSpadLispCmd, 566
- terpri
 - calledby s-process, 581
- throwKeyedMsg
 - calledby compileSpad2Cmd, 563
 - calledby compileSpadLispCmd, 566
 - calledby compiler, 561
 - calledby loadLibIfNecessary, 119
- throwkeyedmsg
 - calledby postMDef, 395
- throws
 - compForm3, 609
- tmptok, 413
 - usedby PARSE-Operation, 424
 - defvar, 413
- tok, 414
 - usedby PARSE-GlyphTok, 441
 - usedby PARSE-NBGlyphTok, 441
 - defvar, 414
- token, 97
 - defstruct, 97
- token-install, 98
 - uses \$token, 98
 - defun, 98
- token-lookahead-type, 452
 - calledby skip-blanks, 451
 - uses Escape-Character, 452
 - defun, 452
- token-nonblank

- calledby PARSE-FloatBasePart, 434
- calledby quote-if-string, 454
- calledby unget-tokens, 456
- token-print, 99
 - uses \$token, 99
 - defun, 99
- token-symbol
 - calledby PARSE-SpecialKeyword, 415
 - calledby match-token, 457
 - calledby parse-argument-designator, 546
 - calledby parse-identifier, 545
 - calledby parse-keyword, 546
 - calledby parse-number, 545
 - calledby parse-spadstring, 544
 - calledby parse-string, 544
 - calledby quote-if-string, 453
- token-type
 - calledby match-token, 457
 - calledby quote-if-string, 453
- TPDHERE
 - Note that this function was missing without error, so may be junk, 518
 - See LocalAlgebra for an example call, 352
 - The use of and in spadreduce is undefined. rewrite this to loop, 462
 - This function is used but never defined.
 - Remove it., 189
 - test with BASTYPE, 176
- transDoc, 494
 - calledby transDocList, 493
 - calls checkDocError1, 494
 - calls checkExtract, 494
 - calls checkTrim, 494
 - calls nreverse, 494
 - calls transformAndRecheckComments, 494
 - local def \$arg1, 494
 - local def \$attribute?, 494
 - local def \$x, 494
 - local ref \$attribute?, 494
 - local ref \$x, 494
 - defun, 494
- transDocList, 493
 - calledby finalizeDocumentation, 490
 - calls checkDocError1, 493
 - calls checkDocError, 493
 - calls sayBrightly, 493
 - calls transDoc, 493
 - local ref \$constructorName, 493
 - defun, 493
- transformAndRecheckComments, 495
 - calledby transDoc, 494
 - calls checkComments, 495
 - calls checkRewrite, 495
 - calls sayBrightly, 495
 - local def \$checkingXmptex?, 495
 - local def \$exposeFlagHeading, 495
 - local def \$name, 495
 - local def \$origin, 495
 - local def \$recheckingFlag, 495
 - local def \$x, 495
 - local ref \$exposeFlagHeading, 495
 - defun, 495
- transformOperationAlist, 210
 - calledby getCategoryOpsAndAtts, 210
 - calledby getFunctorOpsAndAtts, 212
 - calls assoc, 211
 - calls insertAlist, 211
 - calls keyedSystemError, 211
 - calls lassq, 211
 - calls member, 211
 - local ref \$functionLocations, 211
 - defun, 210
- transImplementation, 597
 - calledby compAtomWithModemap, 596
 - calls genDeltaEntry, 597
 - defun, 597
- transIs, 110
 - calledby parseIsnt, 128
 - calledby parseIs, 127
 - calledby parseLET, 130
 - calledby parseLhs, 110
 - calledby transIs1, 110
 - calls isListConstructor, 110
 - calls transIs1, 110
 - defun, 110
- transIs1, 110
 - calledby transIs1, 110
 - calledby transIs, 110
 - calls nreverse, 110
 - calls transIs1, 110
 - calls transIs, 110

- defun, [110](#)
- translabel, [541](#)
 - calledby PARSE-Data, [439](#)
 - calls translabel1, [541](#)
 - defun, [541](#)
- translabel1, [541](#)
 - calledby translabel1, [541](#)
 - calledby translabel, [541](#)
 - calls lassoc, [541](#)
 - calls maxindex, [541](#)
 - calls refvecp, [541](#)
 - calls translabel1, [541](#)
 - defun, [541](#)
- transSeq
 - calledby parseSeq, [136](#)
- trimString
 - calledby checkGetArgs, [520](#)
- TruthP, [268](#)
 - calledby mergeModemap, [267](#)
 - calledby optCond, [246](#)
 - defun, [268](#)
- try-get-token, [458](#)
 - calledby advance-token, [459](#)
 - calledby current-token, [458](#)
 - calledby next-token, [459](#)
 - calls get-token, [458](#)
 - uses valid-tokens, [458](#)
 - defun, [458](#)
- tuple2List, [547](#)
 - calledby postCollect,finish, [382](#)
 - calledby postConstruct, [387](#)
 - calledby postInSeq, [392](#)
 - calledby tuple2List, [547](#)
 - calls postTranSegment, [547](#)
 - calls postTran, [547](#)
 - calls tuple2List, [547](#)
 - uses \$InteractiveMode, [547](#)
 - uses \$boot, [547](#)
 - defun, [547](#)
- TupleCollect, [402](#)
 - defplist, [402](#)
- unabbrevAndLoad
 - calledby parseHasRhs, [118](#)
 - calledby parseHas, [116](#)
- unAbbreviateKeyword[5]
 - called by PARSE-SpecialKeyWord, [415](#)
- uncons, [340](#)
 - calledby uncons, [340](#)
 - calls uncons, [340](#)
 - defun, [340](#)
- Undef
 - usedby mkOpVec, [226](#)
- underscore, [455](#)
 - calledby quote-if-string, [454](#)
 - calls vector-push, [455](#)
 - defun, [455](#)
- unembed
 - calledby compilerDoitWithScreenedLisplib, [567](#)
- unErrorRef
 - calledby addModemap1, [274](#)
- unget-tokens, [456](#)
 - calledby match-string, [451](#)
 - calls line-current-segment, [456](#)
 - calls line-new-line, [456](#)
 - calls line-number, [456](#)
 - calls quote-if-string, [456](#)
 - calls strconc, [456](#)
 - calls token-nonblank, [456](#)
 - uses valid-tokens, [456](#)
 - defun, [456](#)
- Union, [289](#)
 - defplist, [289](#)
- union
 - calledby compDefWhereClause, [160](#)
 - calledby compJoin, [323](#)
 - calledby extendLocalLibdb, [475](#)
 - calledby makeFunctorArgumentParameters, [221](#)
 - calledby mkAlistOfExplicitCategoryOps, [193](#)
 - calledby mkExplicitCategoryFunction, [292](#)
 - calledby mkUnion, [366](#)
- UnionCategory, [301](#)
 - defplist, [301](#)
- unionq
 - calledby freelist, [623](#)
 - calledby orderPredTran, [198](#)
- unknownTypeError, [253](#)
 - calledby addDomain, [252](#)
 - calledby compColon, [294](#)

- calls stackSemanticError, 253
- defun, 253
- unloadOneConstructor, 205
 - calledby compDefineLisplib, 167
 - calls mkAutoLoad, 205
 - calls remprop, 205
 - defun, 205
- unTuple, 413
 - calledby postMapping, 394
 - calledby postTran, 370
 - calledby postType, 379
 - defun, 413
- updateCategoryFrameForCategory, 121
 - calledby compDefineLisplib, 167
 - calledby isFunctor, 253
 - calledby loadLibIfNecessary, 119
 - calls addModemap, 121
 - calls getdatabase, 121
 - calls put, 121
 - local def \$CategoryFrame, 121
 - local ref \$CategoryFrame, 121
 - defun, 121
- updateCategoryFrameForConstructor, 120
 - calledby compDefineLisplib, 167
 - calledby isFunctor, 253
 - calledby loadLibIfNecessary, 119
 - calls addModemap, 120
 - calls convertOpAlist2compilerInfo, 120
 - calls getdatabase, 120
 - calls put, 120
 - local def \$CategoryFrame, 120
 - local ref \$CategoryFrame, 120
 - defun, 120
- updateSourceFiles[5]
 - called by compileSpad2Cmd, 563
- upper-case-p
 - calledby recordAttributeDocumentation, 468
- userError
 - calledby compDefWhereClause, 159
 - calledby compOrCroak1, 587
 - calledby compReturn, 335
 - calledby compile, 173
 - calledby convertOrCroak, 338
 - calledby doItIf, 285
 - calledby orderByDependency, 227
- valid-tokens, 98
 - usedby advance-token, 459
 - usedby current-token, 458
 - usedby next-token, 459
 - usedby try-get-token, 458
 - usedby unget-tokens, 456
 - uses \$token, 98
 - defvar, 98
- vcons, 136
 - defplist, 136
- Vector
 - calledby optCallEval, 238
- vector, 353
 - defplist, 353
- vector-push
 - calledby underscore, 455
- VectorCategory, 301
 - defplist, 301
- vmisp::optionlist
 - usedby print-defun, 584
- where, 137, 354, 403
 - defplist, 137, 354, 403
- whoOwns, 539
 - calledby checkDocMessage, 518
 - calls awk, 539
 - calls getdatabase, 539
 - calls shut, 539
 - calls strconc, 539
 - local ref \$exposeFlag, 539
 - defun, 539
- with, 403
 - defplist, 403
- wrapDomainSub, 294
 - calledby compJoin, 323
 - calledby mkExplicitCategoryFunction, 292
 - defun, 294
- wrapSEQExit
 - calledby makeSimplePredicateOrNil, 544
- writedb
 - calledby buildLibAttr, 483
 - calledby buildLibOp, 482
 - calledby buildLibdb, 476
 - calledby dbWriteLines, 479
- writeLib1, 207
 - calledby initializeLisplib, 206

- calls `rdefiostream`, [207](#)
 - `defun`, [207](#)
- `XTokenReader`, [460](#)
 - calledby `get-token`, [460](#)
 - usedby `get-token`, [460](#)
 - `defvar`, [460](#)
- `xtokenreader`
 - usedby `string2BootTree`, [77](#)